

```

1| import { formatMonto, formatImporteDisplay, formatImporteParaInput, formatearCeldaMoneda, formatearCeldaMonedaConSigno, htmlTipoOperacionIconos, htmlIconoMonedaTipoOp, isHttps
2|   UrlSegura } from './utils.js';
3| // XLSX se carga por script en index.html (CDN) para que funcione en producción sin bundler
4| const XLSX = window.XLSX;
5|
6| const SUPABASE_URL = (typeof window.SUPABASE_URL !== 'undefined' && window.SUPABASE_URL) ? window.SUPABASE_URL : '';
7| const SUPABASE_ANON_KEY = (typeof window.SUPABASE_ANON_KEY !== 'undefined' && window.SUPABASE_ANON_KEY) ? window.SUPABASE_ANON_KEY : '';
8|
9| if (!SUPABASE_ANON_KEY || !SUPABASE_URL) {
10|   document.body.innerHTML = '<div class="card" style="margin:2rem; color:#b91c1c;">Falta config. Copiá <code>config.example.js</code> a <code>config.js</code> y configurá SUPA
11|   BASE URL y SUPABASE_ANON_KEY.</div>';
12|   throw new Error('Missing Supabase config');
13| }
14|
15| const client = window.supabase.createClient(SUPABASE_URL, SUPABASE_ANON_KEY);
16|
17| /** Código interno en BD para el operador ('pagador'/'cobrador'); no se renombra en datos. */
18| const MARCA_OPERADOR_CODIGO_DB = 'pandy';
19| const MARCA_OPERADOR_DEFAULT_NOMBRE_VISIBLE = 'Pandi';
20|
21| let appEmpresaState = {
22|   nombre_legal: '',
23|   nombre_sistema: MARCA_OPERADOR_DEFAULT_NOMBRE_VISIBLE,
24|   logo_url: '',
25| };
26|
27| function mergeAppEmpresaRow(row) {
28|   if (!row) return;
29|   if (row.nombre_legal != null) appEmpresaState.nombre_legal = String(row.nombre_legal);
30|   if (row.nombre_sistema != null) appEmpresaState.nombre_sistema = String(row.nombre_sistema);
31|   if (row.logo_url != null) appEmpresaState.logo_url = String(row.logo_url).trim();
32| }
33|
34| function nombreMarcaSistema() {
35|   const s = (appEmpresaState.nombre_sistema || '').trim();
36|   return s || MARCA_OPERADOR_DEFAULT_NOMBRE_VISIBLE;
37| }
38|
39| /** Etiqueta visible para pagador/cobrador en tablas y modales (valor BD 'pandy' → nombre de empresa). */
40| function etiquetaRolParticipanteUi(rol) {
41|   const r = String(rol || '').toLowerCase();
42|   if (r === MARCA_OPERADOR_CODIGO_DB) return nombreMarcaSistema();
43|   if (r === 'cliente') return 'Cliente';
44|   if (r === 'intermediario') return 'Intermediario';
45|   return rol || '-';
46| }
47|
48| function logoUrlSeguroParaImgSrc() {
49|   const u = (appEmpresaState.logo_url || '').trim();
50|   if (!u) return '/assets/favicon-192x192.png';
51|   if (u.startsWith('/') && !u.startsWith('//')) return u;
52|   if (isHttpsUrlSegura(u)) return u;
53|   return '/assets/favicon-192x192.png';
54| }
55|
56| function validarLogoUrlInput(u) {
57|   const s = String(u || '').trim();
58|   if (!s) return true;
59|   if (s.startsWith('/') && !s.startsWith('//')) return true;
60|   return isHttpsUrlSegura(s);
61| }
62|
63| function fetchAppEmpresaIntoState() {
64|   return client
65|     .from('app_empresa')
66|     .select('nombre_legal,nombre_sistema,logo_url')
67|     .eq('id', 1)
68|     .maybeSingle()
69|     .then((r) => {
70|       if (!r.error && r.data) mergeAppEmpresaRow(r.data);
71|       return r;
72|     });
73| }
74|
75| /**
76|  * Estado del control Supabase: `none` = OK; el resto muestra banner con texto acorde.
77|  * - unreachable: red, CORS, timeout, 5xx, PGRST301 (JWT/capa auth), excepciones fetch.
78|  * - schema: tabla/caché (p. ej. falta app_empresa o migración).
79|  * - rls: permisos / políticas que bloquean la lectura de control.
80|  * - config: el API respondió con otro error (referencia app_empresa).
81|  */
82| let pandiSupabaseConnectivityIssue = 'none';
83| let pandiSupabaseHealthIntervalId = null;
84|
85| /** Modo reducido: tras ~10 min seguidos sin llegar a Supabase (solo `unreachable`). */
86| let pandiUnreachableSinceMs = null;
87| let pandiModoReducidoOffline = false;
88| const PANDI_OFFLINE_QUEUE_KEY = 'pandi_offline_ordenes_queue_v1';
89| const PANDI_OFFLINE_CACHE_KEY = 'pandi_offline_catalogos_cache_v1';
90| const PANDI_CACHED_PERMISSIONS_KEY = 'pandi_cached_permissions_v1';
91| const PANDI_OFFLINE_MS_PARA_MODO_REDUCIDO = 10 * 60 * 500;
92|
93| /** Evita registrar dos veces listeners de sesión (sidebar, modales, etc.). Se resetea en showLoginScreenDom. */
94| let pandiSessionUiBootstrapped = false;
95|
96| function pandiCachePermissionsLocal(perms) {
97|   try {

```

```

96 |     if (Array.isArray(perms)) localStorage.setItem(PANDI_CACHED_PERMISSIONS_KEY, JSON.stringify(perms));
97 |   } catch (e) {
98 |     /* ignore */
99 |   }
100 | }
101 |
102 | function pandiLoadCachedPermissionsLocal() {
103 |   try {
104 |     const s = localStorage.getItem(PANDI_CACHED_PERMISSIONS_KEY);
105 |     if (!s) return [];
106 |     const a = JSON.parse(s);
107 |     return Array.isArray(a) ? a : [];
108 |   } catch (e) {
109 |     return [];
110 |   }
111 | }
112 |
113 | /** Sesión válida pero sin RPC (red caída): mostrar shell de la app con permisos cacheados. */
114 | function pandiEsFalloConectividadBootstrap(err) {
115 |   if (pandiSupabaseConnectivityIssue === 'unreachable') return true;
116 |   const code = err && (err.code || (err.error && err.error.code) || '');
117 |   if (code === '42501' || code === 'PGRST204') return false;
118 |   const msg = String((err && err.message) || (err && err.error && err.error.message) || err || '').toLowerCase();
119 |   if (msg.includes('failed to fetch') || msg.includes('network error') || msg.includes('fetch')) return true;
120 |   if (err && err.code === 'PGRST301') return true;
121 |   return false;
122 | }
123 |
124 | /** Clasifica error del ping a app_empresa (PostgREST / red). */
125 | function classifySupabasePingError(error) {
126 |   const msg = (error.message || '').toLowerCase();
127 |   const code = String(error.code || '');
128 |   const details = (error.details || '').toLowerCase();
129 |   const hint = (error.hint || '').toLowerCase();
130 |   const blob = `${msg} ${details} ${hint}`;
131 |
132 |   if (
133 |     msg.includes('fetch') ||
134 |     msg.includes('network') ||
135 |     msg.includes('failed to fetch') ||
136 |     msg.includes('timeout') ||
137 |     msg.includes('cors') ||
138 |     (code.length && code.startsWith('5')) ||
139 |     code === 'PGRST301'
140 |   ) {
141 |     return 'unreachable';
142 |   }
143 |
144 |   if (
145 |     code === 'PGRST205' ||
146 |     code === '42P01' ||
147 |     blob.includes('does not exist') ||
148 |     blob.includes('no existe la relación') ||
149 |     blob.includes('could not find the table') ||
150 |     (blob.includes('relation') && blob.includes('not exist')) ||
151 |     (msg.includes('schema cache') && (msg.includes('could not find') || msg.includes('not find')))
152 |   ) {
153 |     return 'schema';
154 |   }
155 |
156 |   if (
157 |     code === '42501' ||
158 |     blob.includes('permission denied') ||
159 |     blob.includes('row-level security') ||
160 |     (blob.includes('rls') && blob.includes('polic'))
161 |   ) {
162 |     return 'rls';
163 |   }
164 |
165 |   return 'config';
166 | }
167 |
168 | /**
169 |  * Ping mínimo a Supabase. Distingue servicio caído vs proyecto vivo con esquema/RLS mal configurados.
170 |  * @returns {Promise<{ issue: 'none' | 'unreachable' | 'schema' | 'rls' | 'config' }>}
171 |  */
172 | async function checkSupabaseConnectivity() {
173 |   try {
174 |     const { error } = await client
175 |       .from('app_empresa')
176 |       .select('id')
177 |       .limit(1)
178 |       .maybeSingle();
179 |     if (!error) return { issue: 'none' };
180 |     return { issue: classifySupabasePingError(error) };
181 |   } catch (e) {
182 |     console.warn('[Pandi] Conectividad Supabase:', e && e.message ? e.message : e);
183 |     return { issue: 'unreachable' };
184 |   }
185 | }
186 |
187 | const PANDI_SUPABASE_BANNER_COPY = {
188 |   unreachable: {
189 |     title: 'Problemas de conectividad',
190 |     text: 'No pudimos conectar con el servicio. Puede ser temporal (red, tiempo de espera o mantenimiento). Probá de nuevo en unos minutos.',
191 |     warn: false,
192 |   },

```

```

193|   schema: {
194|     title: 'Falta tabla o migración en Supabase',
195|     text: 'El proyecto respondió, pero no se pudo leer la tabla de referencia app_empresa. Ejecutá en el SQL Editor: sql/migracion_app_empresa.sql (y revisá que exista la fila
id = 1).',
    warn: true,
  },
  },
  rls: {
    title: 'Bloqueo de permisos (RLS)',
    text: 'El proyecto respondió, pero la lectura de app_empresa fue rechazada. Revisá políticas RLS para roles anon y authenticated en Supabase.',
    warn: true,
  },
  },
  config: {
    title: 'Error al comprobar el proyecto',
    text: 'El servicio respondió con un error inesperado al verificar datos de referencia. Revisá el SQL Editor, logs del proyecto y la tabla app_empresa.',
    warn: true,
  },
  },
};

function updateSupabaseConnectivityBanner() {
  const wrap = document.getElementById('pandi-supabase-connectivity-banner');
  if (!wrap) return;
  const titleEl = document.getElementById('pandi-supabase-connectivity-title');
  const textEl = document.getElementById('pandi-supabase-connectivity-text');
  const issue = pandiSupabaseConnectivityIssue;
  if (issue === 'none') {
    wrap.classList.remove('is-visible', 'pandi-connectivity-banner--warn');
    return;
  }
  const copy = PANDI_SUPABASE_BANNER_COPY[issue] || PANDI_SUPABASE_BANNER_COPY.config;
  if (titleEl) titleEl.textContent = copy.title;
  if (textEl) textEl.textContent = copy.text;
  wrap.classList.toggle('pandi-connectivity-banner--warn', !!copy.warn);
  wrap.classList.add('is-visible');
}

async function runSupabaseHealthCheck() {
  const prevIssue = pandiSupabaseConnectivityIssue;
  const { issue } = await checkSupabaseConnectivity();
  pandiSupabaseConnectivityIssue = issue;
  updateSupabaseConnectivityBanner();

  if (issue === 'unreachable') {
    if (pandiUnreachableSinceMs === null) pandiUnreachableSinceMs = Date.now();
    if (!pandiModoReducidoOffline && Date.now() - pandiUnreachableSinceMs >= PANDI_OFFLINE_MS_PARA_MOD0_REDUCIDO) {
      pandiModoReducidoOffline = true;
      pandiApplyOfflineReducedModeUi();
      showToast('Modo reducido: Supabase no responde. Solo podés cargar órdenes en cola local (este navegador).', 'info');
    }
  } else {
    pandiUnreachableSinceMs = null;
    if (pandiModoReducidoOffline) {
      pandiModoReducidoOffline = false;
      pandiApplyOfflineReducedModeUi();
    }
    if (issue === 'none' && prevIssue !== 'none') {
      pandiMaybePromptImportOfflineQueue();
      pandiRefreshOfflineCatalogosCache();
    }
  }
  pandiUpdateOfflineToolbarButtons();
}

function setupSupabaseConnectivityMonitoring() {
  const retry = document.getElementById('pandi-supabase-connectivity-retry');
  if (retry && retry.dataset.bound !== '1') {
    retry.dataset.bound = '1';
    retry.addEventListener('click', () => {
      runSupabaseHealthCheck().then(() => {
        if (pandiSupabaseConnectivityIssue === 'none') {
          showToast('Comprobación correcta: el servicio responde.', 'success');
        }
      });
    });
  }
}

if (!window.__pandiSupabaseUnhandledRejectionBound) {
  window.__pandiSupabaseUnhandledRejectionBound = true;
  window.addEventListener('unhandledrejection', (event) => {
    const msg = event && event.reason && event.reason.message !== null
      ? String(event.reason.message)
      : String(event && event.reason !== null ? event.reason : '');
    if (
      typeof msg === 'string' &&
      (msg.includes('Failed to fetch') ||
        msg.includes('AuthRetryableFetchError') ||
        msg.includes('FetchError') ||
        msg.includes('NetworkError'))
    ) {
      pandiSupabaseConnectivityIssue = 'unreachable';
      if (pandiUnreachableSinceMs === null) pandiUnreachableSinceMs = Date.now();
      updateSupabaseConnectivityBanner();
      pandiUpdateOfflineToolbarButtons();
    }
  });
}

if (pandiSupabaseHealthIntervalId !== null) {
  clearInterval(pandiSupabaseHealthIntervalId);
  pandiSupabaseHealthIntervalId = null;
}

```

```

289 | }
290 | pandiSupabaseHealthIntervalId = setInterval(runSupabaseHealthCheck, 60000);
291 | runSupabaseHealthCheck();
292 | const stripGo = document.getElementById('pandi-offline-strip-goto-ordenes');
293 | if (stripGo && stripGo.dataset.bound !== '1') {
294 |   stripGo.dataset.bound = '1';
295 |   stripGo.addEventListener('click', () => {
296 |     try {
297 |       showView('vista-ordenes', 'Órdenes');
298 |     } catch (e) {
299 |       /* ignore */
300 |     }
301 |   });
302 | }
303 | }
304 |
305 | function pandiRandomLocalId() {
306 |   try {
307 |     if (typeof crypto !== 'undefined' && crypto.randomUUID) return crypto.randomUUID();
308 |   } catch (e) { /* ignore */ }
309 |   return 'l' + Date.now() + '-' + Math.random().toString(36).slice(2, 11);
310 | }
311 |
312 | function pandiOfflineQueueRead() {
313 |   try {
314 |     const s = localStorage.getItem(PANDI_OFFLINE_QUEUE_KEY);
315 |     const arr = s ? JSON.parse(s) : [];
316 |     return Array.isArray(arr) ? arr : [];
317 |   } catch (e) {
318 |     return [];
319 |   }
320 | }
321 |
322 | function pandiOfflineQueueWrite(arr) {
323 |   try {
324 |     localStorage.setItem(PANDI_OFFLINE_QUEUE_KEY, JSON.stringify(arr || []));
325 |   } catch (e) {
326 |     showToast('No se pudo guardar en el navegador (espacio o permisos).', 'error');
327 |   }
328 | }
329 |
330 | function pandiOfflineCatalogosRead() {
331 |   try {
332 |     const s = localStorage.getItem(PANDI_OFFLINE_CACHE_KEY);
333 |     return s ? JSON.parse(s) : null;
334 |   } catch (e) {
335 |     return null;
336 |   }
337 | }
338 |
339 | function pandiOfflineCacheWritePayload(clientesRows, intRows, tiposRows) {
340 |   try {
341 |     localStorage.setItem(
342 |       PANDI_OFFLINE_CACHE_KEY,
343 |       JSON.stringify({
344 |         savedAt: new Date().toISOString(),
345 |         clientes: clientesRows || [],
346 |         intermediarios: intRows || [],
347 |         tipos_operacion: tiposRows || [],
348 |       })
349 |     );
350 |   } catch (e) { /* ignore */ }
351 | }
352 |
353 | /** Catálogo completo en localStorage para contingencia (clientes, intermediarios, tipos con iconos). Sin sesión no hace nada. */
354 | function pandiRefreshOfflineCatalogosCache() {
355 |   if (!currentUserId) return Promise.resolve();
356 |   const tiposCols = 'id, nombre, codigo, moneda_in, moneda_out, usa_intermediario, icono_modo, icono_url_publica';
357 |   return Promise.all([
358 |     client.from('clientes').select('id, nombre').eq('activo', true).order('nombre', { ascending: true }),
359 |     client.from('intermediarios').select('id, nombre').eq('activo', true).order('nombre', { ascending: true }),
360 |     tiposOperacionFetchConFallbackOrdenVisual(
361 |       () =>
362 |       client
363 |         .from('tipos_operacion')
364 |         .select(`${tiposCols}, orden_visual`)
365 |         .eq('activo', true)
366 |         .order('orden_visual', { ascending: true })
367 |         .order('codigo')
368 |         .order('usa_intermediario')
369 |         .order('id'),
370 |       () =>
371 |       client
372 |         .from('tipos_operacion')
373 |         .select(tiposCols)
374 |         .eq('activo', true)
375 |         .order('codigo')
376 |         .order('usa_intermediario')
377 |         .order('id'),
378 |     )
379 |   ])
380 |   .then(([cr, ir, rTipos]) => {
381 |     if (cr.error || ir.error || rTipos.error) return;
382 |     pandiOfflineCacheWritePayload(cr.data || [], ir.data || [], rTipos.data || []);
383 |   })
384 |   .catch(() => {});
385 | }

```

```

386 |
387 | /** @deprecated Usar pandiRefreshOfflineCatalogosCache; se mantiene la firma por compatibilidad con loadOrdenes. */
388 | function pandiTrySaveOfflineCatalogosCache(_clientesRows, _intRows) {
389 |   return pandiRefreshOfflineCatalogosCache();
390 | }
391 |
392 | function pandiUpdateOfflineReducedStripText() {
393 |   const stripText = document.getElementById('pandi-offline-reduced-strip-text');
394 |   if (!stripText) return;
395 |   const n = pandiOfflineQueueRead().length;
396 |   stripText.textContent =
397 |     'Sin respuesta de Supabase hace más de 10 minutos. Solo la vista Órdenes está habilitada. ' +
398 |     (n
399 |       ? `Cola local: ${n} orden(es) para enviar cuando vuelva el servicio.`
400 |       : 'Usá «Orden en cola local» para registrar acuerdos en este navegador.');
```

```

483|         okCount += 1;
484|     }
485| }
486| pandiOfflineQueueWrite(failed);
487| pandiUpdateOfflineReducedStripText();
488| pandiUpdateOfflineToolBarButtons();
489| if (okCount) showToast('Se importaron ' + okCount + ' orden(es). Revisá la lista e instrumentación.', 'success');
490| loadOrdenes();
491| }
492|
493| function pandiExportOfflineQueueJsonFile() {
494|     const q = pandiOfflineQueueRead();
495|     const blob = new Blob([JSON.stringify(q, null, 2)], { type: 'application/json;charset=utf-8' });
496|     const a = document.createElement('a');
497|     a.href = URL.createObjectURL(blob);
498|     a.download = 'pandi-cola-ordenes-offline-' + fechaHoyYYYYMMDDArgentina() + '.json';
499|     a.click();
500|     URL.revokeObjectURL(a.href);
501|     showToast('Archivo JSON descargado (respaldo de la cola).', 'info');
502| }
503|
504| function pandiOrdenOfflineCloseModal() {
505|     closeOrdenOfflineTipoOperacionListBox();
506|     const back = document.getElementById('modal-orden-offline-backdrop');
507|     if (back) {
508|         back.classList.remove('activo');
509|         back.setAttribute('aria-hidden', 'true');
510|     }
511| }
512|
513| function pandiOrdenOfflineSyncTipoAMonedas() {
514|     const sel = document.getElementById('orden-offline-tipo');
515|     const opt = sel && sel.selectedOptions && sel.selectedOptions[0];
516|     const mr = document.getElementById('orden-offline-monedas-r');
517|     const me = document.getElementById('orden-offline-monedas-e');
518|     if (opt && mr && me) {
519|         const mi = (opt.getAttribute('data-monedas-in') || 'USD').toUpperCase();
520|         const mo = (opt.getAttribute('data-monedas-out') || 'USD').toUpperCase();
521|         mr.value = mi;
522|         me.value = mo;
523|         const codigo = (opt.getAttribute('data-codigo') || '').toUpperCase();
524|         const wrapTasa = document.getElementById('orden-offline-wrap-tasa-cheque');
525|         if (wrapTasa) {
526|             const esCh = esTipoOperacionChequeArs(codigo, mi, mo);
527|             wrapTasa.style.display = esCh ? '' : 'none';
528|         }
529|     }
530|     syncOrdenOfflineTipoOperacionIconosPreview();
531| }
532|
533| function pandiOrdenOfflineOpenModal() {
534|     if (!currentUser) {
535|         showToast('Iniciá sesión para usar la cola local.', 'error');
536|         return;
537|     }
538|     if (!userPermissions.includes('ingresar_orden')) {
539|         showToast('No tenés permiso para crear órdenes.', 'error');
540|         return;
541|     }
542|     const cache = pandiOfflineCatalogosRead();
543|     const backdrop = document.getElementById('modal-orden-offline-backdrop');
544|     if (!backdrop) return;
545|     const selC = document.getElementById('orden-offline-cliente');
546|     const selI = document.getElementById('orden-offline-intermediario');
547|     const selT = document.getElementById('orden-offline-tipo');
548|     const aviso = document.getElementById('orden-offline-sin-cache');
549|     if (!cache || !(cache.clientes && cache.clientes.length) || (cache.intermediarios && cache.intermediarios.length)) || !(cache.tipos_operacion && cache.tipos_operacion.length) {
550|         h) {
551|             if (aviso) aviso.style.display = 'block';
552|             if (selC) selC.innerHTML = '';
553|             if (selI) selI.innerHTML = '';
554|             if (selT) selT.innerHTML = '<option value="">Elegir...</option>';
555|         } else {
556|             if (aviso) aviso.style.display = 'none';
557|             if (selC) {
558|                 selC.innerHTML = '<option value=""></option>' + (cache.clientes || []).map((c) => '<option value="${escapeHtml(String(c.id))}">${escapeHtml(c.nombre || '')}</option>');
559|             }
560|             if (selI) {
561|                 selI.innerHTML = '<option value=""></option>' + (cache.intermediarios || []).map((i) => '<option value="${escapeHtml(String(i.id))}">${escapeHtml(i.nombre || '')}</option>');
562|             }
563|             if (selT) {
564|                 const escUrl = (s) => String(s || '').replace(/&/g, '&amp;').replace(/"/g, '&quot;');
565|                 const escAttr = (s) => String(s || '').replace(/&/g, '&amp;').replace(/"/g, '&quot;');
566|                 const im = (m) => (m === 'cheque' || m === 'custom' ? m : 'auto');
567|                 const tipos = ordenarTiposOperacionListaParaOrden(cache.tipos_operacion || []);
568|                 selT.innerHTML =
569|                     '<option value="">Elegir...</option>' +
570|                     tipos
571|                     .map((t) => {
572|                         const modo = in((t.icono_modo || 'auto').toString().trim().toLowerCase());
573|                         const baseNombre = t.nombre != null ? String(t.nombre).trim() : '';
574|                         const etiqueta = nombreTipoOperacionOrdenUi(t);
575|                         return '<option value="${escapeHtml(String(t.id))}" data-nombre-base="${escAttr(baseNombre)}" data-codigo="${escapeHtml(t.codigo || '')}" data-icono-modo="${escapeHtml(modos)}" data-icono-url="${escUrl(t.icono_url_publica || '')}" data-monedas-in="${escapeHtml((t.moneda_in || 'USD').toString().trim().toUpperCase())}" data-monedas-out="${escapeHtml((t.moneda_out || 'USD').toString().trim().toUpperCase())}" data-usa-intermediario="${t.usa_intermediario === true ? 'true' : 'false'}">${escapeHtml(etiqueta)}</option>';

```

```

>;
575     })
576     .join('');
577 }
578 }
579 closeOrdenOfflineTipoOperacionListbox();
580 syncOrdenOfflineTipoOperacionIconosPreview();
581 rebuildOrdenOfflineTipoOperacionListbox();
582 const fhoy = fechaHoyYYYYMMDDArgentina();
583 const fe = document.getElementById('orden-offline-fecha');
584 if (fe) fe.value = fhoy;
585 ['orden-offline-monto-r', 'orden-offline-monto-e', 'orden-offline-cotizacion', 'orden-offline-tasa-cheque-pct', 'orden-offline-observaciones'].forEach((id) => {
586     const el = document.getElementById(id);
587     if (el) el.value = '';
588 });
589 pandiOrdenOfflineSyncTipoAMonedas();
590 const listEl = document.getElementById('orden-offline-queue-resumen');
591 if (listEl) {
592     const qq = pandiOfflineQueueRead();
593     listEl.innerHTML = qq.length
594     ? '<strong>En cola (' + qq.length + ')</strong> ' + qq.map((x) => escapeHtml((x.createdAt || '').slice(0, 19)) + ' · ' + escapeHtml((x.payload && x.payload.monto_recibi
do) != null ? String(x.payload.monto_recibido) : '?')).join(' · ')
595     : '<em>Sin borradores en cola.</em>';
596 }
597 backdrop.classList.add('activo');
598 backdrop.setAttribute('aria-hidden', 'false');
599 }
600
601 function pandiOrdenOfflineGuardarEnCola() {
602     const cache = pandiOfflineCatalogosRead();
603     if (!cache || !(cache.tipos_operacion && cache.tipos_operacion.length)) {
604         showToast('No hay catálogo en caché. Conectá la app: el respaldo local se actualiza al iniciar sesión y cuando el servicio responde.', 'error');
605         return;
606     }
607     const fecha = (document.getElementById('orden-offline-fecha') || {}).value;
608     const clienteId = (document.getElementById('orden-offline-cliente') || {}).value.trim() || null;
609     let intermediarioId = (document.getElementById('orden-offline-intermediario') || {}).value.trim() || null;
610     const tipoOperacionId = (document.getElementById('orden-offline-tipo') || {}).value.trim() || null;
611     const selTipo = document.getElementById('orden-offline-tipo')?.selectedOptions?.[0];
612     const usaInt = selTipo ? selTipo.getAttribute('data-usa-intermediario') === 'true' : false;
613     if (usaInt && !intermediarioId) {
614         showToast('Este tipo exige intermediario.', 'error');
615         return;
616     }
617     if (!clienteId && !intermediarioId) {
618         showToast('Elegí al menos cliente o intermediario.', 'error');
619         return;
620     }
621     if (!tipoOperacionId || !fecha) {
622         showToast('Completá fecha y tipo de operación.', 'error');
623         return;
624     }
625     const monedaRecibida = (document.getElementById('orden-offline-moneda-r') || {}).value || 'USD';
626     const monedaEntregada = (document.getElementById('orden-offline-moneda-e') || {}).value || 'USD';
627     const montoRecibido = parseImporteInput((document.getElementById('orden-offline-monto-r') || {}).value || '');
628     const montoEntregado = parseImporteInput((document.getElementById('orden-offline-monto-e') || {}).value || '');
629     const cotizacionRaw = (document.getElementById('orden-offline-cotizacion') || {}).value.trim();
630     const cotizacion = cotizacionRaw ? parseImporteInput(cotizacionRaw) : null;
631     const observaciones = (document.getElementById('orden-offline-observaciones') || {}).value.trim() || null;
632     if (isNaN(montoRecibido) || montoRecibido <= 0 || isNaN(montoEntregado) || montoEntregado <= 0) {
633         showToast('Montos recibido y entregado deben ser números positivos.', 'error');
634         return;
635     }
636     const codigo = selTipo ? (selTipo.getAttribute('data-codigo') || '') : '';
637     const patronTc = patronTipoCambioOrden(monedaRecibida, monedaEntregada);
638     if (patronTc && (!cotizacion || !(cotizacion > 0))) {
639         showToast('En cruces con tipo de cambio, el tipo de cambio del acuerdo es obligatorio (> 0).', 'error');
640         return;
641     }
642     if (codigo === 'USD-USD' && montoRecibido <= montoEntregado) {
643         showToast('En USD-USD el monto recibido debe ser mayor al entregado.', 'error');
644         return;
645     }
646     let tasaDescuentoIntermediarioSave = null;
647     const esCh = esTipoOperacionChequeArs(codigo, monedaRecibida, monedaEntregada);
648     if (esCh) {
649         if (montoRecibido <= montoEntregado) {
650             showToast('En CHEQUE-ARS el monto recibido debe ser mayor al entregado.', 'error');
651             return;
652         }
653         const tasaPctRaw = (document.getElementById('orden-offline-tasa-cheque-pct') || {}).value.trim();
654         const tasaPct = tasaPctRaw ? parseImporteInput(tasaPctRaw) : null;
655         if (typeof tasaPct !== 'number' || isNaN(tasaPct) || tasaPct <= 0 || tasaPct >= 100) {
656             showToast('Indicá la tasa de descuento del intermediario (% , ej. 1 para 1%).', 'error');
657             return;
658         }
659         tasaDescuentoIntermediarioSave = tasaPct / 100;
660     }
661     if (!usaInt) intermediarioId = null;
662     const operacionDirecta = !intermediarioId;
663     const payload = {
664         cliente_id: clienteId,
665         fecha,
666         estado: 'pendiente_instrumentar',
667         tipo_operacion_id: tipoOperacionId,
668         operacion_directa: operacionDirecta,
669         intermediario_id: intermediarioId,

```

```

670 |     moneda_recibida: monedaRecibida,
671 |     moneda_entregada: monedaEntregada,
672 |     monto_recibido: montoRecibido,
673 |     monto_entregado: montoEntregado,
674 |     cotizacion: cotizacion,
675 |     tasa_descuento_intermediario: tasaDescuentoIntermediarioSave,
676 |     observaciones: observaciones
677 |     ? observaciones + '': [Cola offline ' + new Date().toISOString().slice(0, 16) + ']'
678 |     : '[Cola offline ' + new Date().toISOString().slice(0, 16) + ']',
679 | };
680 | const item = {
681 |   v: 1,
682 |   localId: pandiRandomLocalId(),
683 |   createdAt: new Date().toISOString(),
684 |   createdByUserId: currentUserId,
685 |   payload,
686 | };
687 | const q = pandiOfflineQueueRead();
688 | q.push(item);
689 | pandiOfflineQueueWrite(q);
690 | showToast('Orden guardada en cola local (' + q.length + ' en total).', 'success');
691 | pandiUpdateOfflineReducedStripText();
692 | pandiUpdateOfflineToolbarButtons();
693 | pandiOrdenOfflineOpenModal();
694 | }
695 |
696 | function setupModalOrdenOffline() {
697 |   const backdrop = document.getElementById('modal-orden-offline-backdrop');
698 |   if (!backdrop || backdrop.dataset.offlineBound === '1') return;
699 |   backdrop.dataset.offlineBound = '1';
700 |   const c = document.getElementById('modal-orden-offline-close');
701 |   const x = document.getElementById('modal-orden-offline-cancelar');
702 |   const form = document.getElementById('form-orden-offline');
703 |   const selTipo = document.getElementById('orden-offline-tipo');
704 |   if (c) c.addEventListener('click', pandiOrdenOfflineCloseModal);
705 |   if (x) x.addEventListener('click', pandiOrdenOfflineCloseModal);
706 |   setupBackdropCloseOnlyOnRealClick(backdrop, pandiOrdenOfflineCloseModal);
707 |   if (form) form.addEventListener('submit', (e) => { e.preventDefault(); pandiOrdenOfflineGuardarEnCola(); });
708 |   if (selTipo) selTipo.addEventListener('change', pandiOrdenOfflineSyncTipoAMonedas);
709 |   const comboUi = backdrop.querySelector('.orden-tipo-operacion-combo-ui');
710 |   const comboBtn = document.getElementById('orden-offline-tipo-combo-btn');
711 |   const comboList = document.getElementById('orden-offline-tipo-listbox');
712 |   const comboWrap = backdrop.querySelector('.orden-tipo-operacion-combo-wrap');
713 |   if (comboUi && comboBtn && comboList && comboWrap && comboUi.dataset.offlineComboBound !== '1') {
714 |     comboUi.dataset.offlineComboBound = '1';
715 |     function reposicionarOfflineListBoxSiAbierto() {
716 |       if (!comboList || comboList.hidden) return;
717 |       positionOrdenOfflineTipoOperacionListBox();
718 |     }
719 |     window.addEventListener('scroll', reposicionarOfflineListBoxSiAbierto, true);
720 |     window.addEventListener('resize', reposicionarOfflineListBoxSiAbierto);
721 |     comboBtn.addEventListener('click', (ev) => {
722 |       ev.preventDefault();
723 |       ev.stopPropagation();
724 |       if (!comboList.hidden) {
725 |         closeOrdenOfflineTipoOperacionListBox();
726 |         return;
727 |       }
728 |       rebuildOrdenOfflineTipoOperacionListBox();
729 |       comboList.hidden = false;
730 |       comboBtn.setAttribute('aria-expanded', 'true');
731 |       requestAnimationFrame(() => {
732 |         positionOrdenOfflineTipoOperacionListBox();
733 |       });
734 |     });
735 |     comboList.addEventListener('click', (ev) => {
736 |       const optBtn = ev.target && ev.target.closest ? ev.target.closest('.orden-tipo-operacion-option') : null;
737 |       if (!optBtn) return;
738 |       ev.preventDefault();
739 |       ev.stopPropagation();
740 |       const val = optBtn.getAttribute('data-value');
741 |       const sel = document.getElementById('orden-offline-tipo');
742 |       closeOrdenOfflineTipoOperacionListBox();
743 |       if (sel && val !== null) {
744 |         sel.value = val;
745 |         sel.dispatchEvent(new Event('change', { bubbles: true }));
746 |       }
747 |       comboBtn.focus();
748 |     });
749 |     document.addEventListener(
750 |       'mousedown',
751 |       (ev) => {
752 |         if (!comboList || comboList.hidden) return;
753 |         if (comboWrap.contains(ev.target)) return;
754 |         closeOrdenOfflineTipoOperacionListBox();
755 |       },
756 |       true
757 |     );
758 |   }
759 |   if (!backdrop.dataset.offlineKeydownBound) {
760 |     backdrop.dataset.offlineKeydownBound = '1';
761 |     document.addEventListener(
762 |       'keydown',
763 |       (e) => {
764 |         if (e.key !== 'Escape') return;
765 |         const back = document.getElementById('modal-orden-offline-backdrop');
766 |         if (!back || !back.classList.contains('activo')) return;

```



```

767 |     const lb = document.getElementById('orden-offline-tipo-listbox');
768 |     if (lb && !lb.hidden) {
769 |         e.preventDefault();
770 |         closeOrdenOfflineTipoOperacionListbox();
771 |     }
772 | },
773 | true
774 | );
775 | }
776 | const btnLoc = document.getElementById('btn-orden-offline-local');
777 | if (btnLoc) btnLoc.addEventListener('click', () => pandiOrdenOfflineOpenModal());
778 | const btnSync = document.getElementById('btn-orden-offline-sync');
779 | if (btnSync) btnSync.addEventListener('click', () => {
780 |     showConfirm(
781 |         'Se enviarán las órdenes de la cola local a Supabase en orden. ¿Continuar?',
782 |         'Enviar',
783 |         () => { pandiImportOfflineQueueSequential(); }
784 |     );
785 | });
786 | const btnJson = document.getElementById('orden-offline-btn-json');
787 | if (btnJson) btnJson.addEventListener('click', () => pandiExportOfflineQueueJsonFile());
788 | }
789 |
790 | function aplicarMarcaEnTodaLaUI() {
791 |     const nombre = nombreMarcaSistema();
792 |     document.title = nombre;
793 |     document.querySelectorAll('.js-marca-sistema-nombre').forEach((el) => {
794 |         el.textContent = nombre;
795 |     });
796 |     document.querySelectorAll('option[value="pandy"]').forEach((opt) => {
797 |         opt.textContent = nombre;
798 |     });
799 |     const src = logoUrlSeguroParaImgSrc();
800 |     document.querySelectorAll('.login-logo, #page-logo').forEach((img) => {
801 |         if (img && img.tagName === 'IMG') {
802 |             img.src = src;
803 |             img.alt = nombre;
804 |         }
805 |     });
806 |     const inpPart = document.getElementById('orden-participante-nombre-marca');
807 |     if (inpPart) inpPart.value = nombre;
808 | }
809 |
810 | /** Bucket público para iconos de tipos de operación. Ver sql/storage_bucket_tipo_operacion_iconos.sql */
811 | const STORAGE_BUCKET_TIPO_OP_ICONOS = 'tipo-operacion-iconos';
812 |
813 | let userPermissions = [];
814 | let currentUserEmail = '';
815 | let currentUserId = null;
816 |
817 | // --- Wizard Orden + Instrumentación ---
818 | let ordenWizardOrdenIdActual = null;
819 | /** Si la columna ordenes.numero no existe (migración no ejecutada), se pone en false tras el primer error. */
820 | let ordenesTieneNumeroColumn = true;
821 | /** Ya no se crea borrador al abrir "Nueva orden"; la orden se inserta solo al guardar o ir a Instrumentación (se preserva la correlación del número). */
822 | let ordenIdBorradorParaEliminar = null;
823 | let ordenWizardInstrumentacionIdActual = null;
824 | /** Se incrementa al abrir/cerrar el modal de orden: evita que un fetch lento pise datos de otra apertura. */
825 | let ordenModalLoadSeq = 0;
826 |
827 | const SIDEBAR_KEY = 'pandi-sidebar-expanded';
828 |
829 | /** Layout estrecho: menú lateral fijo + drawer con backdrop (debe coincidir con CSS max-width: 768px). */
830 | function pandiIsMobileNavLayout() {
831 |     return typeof window.matchMedia === 'function' && window.matchMedia('(max-width: 768px)').matches;
832 | }
833 |
834 | function pandiSyncSidebarBackdrop() {
835 |     const bd = document.getElementById('sidebar-backdrop');
836 |     const sidebar = document.getElementById('sidebar');
837 |     if (!bd || !sidebar) return;
838 |     const loggedIn = document.body.classList.contains('pandi-shell-logged-in');
839 |     if (!loggedIn || sidebar.style.display === 'none' || !pandiIsMobileNavLayout()) {
840 |         bd.classList.remove('is-visible');
841 |         bd.setAttribute('aria-hidden', 'true');
842 |         return;
843 |     }
844 |     if (sidebar.classList.contains('expanded')) {
845 |         bd.classList.add('is-visible');
846 |         bd.setAttribute('aria-hidden', 'false');
847 |     } else {
848 |         bd.classList.remove('is-visible');
849 |         bd.setAttribute('aria-hidden', 'true');
850 |     }
851 | }
852 |
853 | /** Tras elegir una vista en el menú, cerrar el drawer en móvil para ver el contenido. */
854 | function pandiCollapseMobileSidebarAfterNav() {
855 |     if (!pandiIsMobileNavLayout()) return;
856 |     const sidebar = document.getElementById('sidebar');
857 |     if (!sidebar || !sidebar.classList.contains('expanded')) return;
858 |     sidebar.classList.remove('expanded');
859 |     try {
860 |         localStorage.setItem(SIDEBAR_KEY, '0');
861 |     } catch (e) {
862 |         /* ignore */
863 |     }

```

```

864 | const toggle = document.getElementById('sidebar-toggle');
865 | if (toggle) {
866 |     toggle.setAttribute('aria-label', 'Expandir menú');
867 |     toggle.setAttribute('title', 'Expandir menú');
868 | }
869 | pandiSyncSidebarBackdrop();
870 | }
871 |
872 | /** Tiempo mínimo (ms) que se muestra el spinner al cambiar de solapa, para que se vea el "trabajando". */
873 | const VISTA_LOADING_MIN_MS = 450;
874 | function delayMinLoading(shownAt, minMs) {
875 |     const elapsed = Date.now() - (shownAt || 0);
876 |     const wait = Math.max(0, (minMs || VISTA_LOADING_MIN_MS) - elapsed);
877 |     return wait > 0 ? new Promise((r) => setTimeout(r, wait)) : Promise.resolve();
878 | }
879 |
880 | /** True mientras corre el refresco automático por intervalo: no vaciar tablas ni mostrar spinners; los datos previos quedan hasta reemplazarlos. */
881 | let pandiBackgroundRefreshActive = false;
882 | function isPandiBackgroundRefresh() {
883 |     return pandiBackgroundRefreshActive;
884 | }
885 | /** Espera mínima de "cargando" solo al abrir la vista por menú; en refresco en background no se fuerza demora ni parpadeo. */
886 | function delayMinLoadingSiNoEsBackground(shownAt) {
887 |     if (isPandiBackgroundRefresh()) return Promise.resolve();
888 |     return delayMinLoading(shownAt);
889 | }
890 |
891 | // Tiempo de inactividad: tras X minutos sin usar la app se cierra la sesión (configurable por Admin en Seguridad)
892 | let lastActivityTime = 0;
893 | let sessionTimeoutMinutes = 60;
894 | /** USD-USD + cp.ic: intermediario con comisión fija en USD ('app_config' key `usd_usd_comision_fija_config`, editable en Seguridad). */
895 | let usdUsdComisionFijaConfig = { intermediario_id: '', opcion_a: 50, opcion_b: 75 };
896 | const PANDI_LS_USD_COMISION_FIJA_PREF = 'pandi_usd_comision_fija_pref';
897 |
898 | function parseUsdUsdComisionFijaConfigJson(raw) {
899 |     const d = typeof raw === 'string' ? raw.trim() : '';
900 |     if (!d) return { intermediario_id: '', opcion_a: 50, opcion_b: 75 };
901 |     try {
902 |         const o = JSON.parse(d);
903 |         const id = o.intermediario_id != null ? String(o.intermediario_id).trim() : '';
904 |         const a = Number(o.opcion_a);
905 |         const b = Number(o.opcion_b);
906 |         return {
907 |             intermediario_id: id,
908 |             opcion_a: (typeof a === 'number' && !isNaN(a) && a > 0 && a < 1e7) ? a : 50,
909 |             opcion_b: (typeof b === 'number' && !isNaN(b) && b > 0 && b < 1e7) ? b : 75,
910 |         };
911 |     } catch (_) {
912 |         return { intermediario_id: '', opcion_a: 50, opcion_b: 75 };
913 |     }
914 | }
915 |
916 | function aplicarValoresRadiosComisionFijaUsdWizard() {
917 |     if (typeof document === 'undefined') return;
918 |     const a = Number(usdUsdComisionFijaConfig.opcion_a) || 50;
919 |     const b = Number(usdUsdComisionFijaConfig.opcion_b) || 75;
920 |     const rads = document.querySelectorAll('input[name="orden-usd-nacho-comision-usd"]');
921 |     if (rads[0]) {
922 |         rads[0].value = String(a);
923 |         const sp0 = rads[0].parentElement && rads[0].parentElement.querySelector('.orden-usd-comision-fija-text-a');
924 |         if (sp0) sp0.textContent = 'USD ' + formatImporteDisplay(a);
925 |     }
926 |     if (rads[1]) {
927 |         rads[1].value = String(b);
928 |         const spl = rads[1].parentElement && rads[1].parentElement.querySelector('.orden-usd-comision-fija-text-b');
929 |         if (spl) spl.textContent = 'USD ' + formatImporteDisplay(b);
930 |     }
931 | }
932 |
933 | function preferenciaComisionFijaInterUsdDesdeStorage() {
934 |     const oa = Number(usdUsdComisionFijaConfig.opcion_a) || 50;
935 |     const ob = Number(usdUsdComisionFijaConfig.opcion_b) || 75;
936 |     try {
937 |         if (typeof localStorage === 'undefined') return oa;
938 |         if (!localStorage.getItem(PANDI_LS_USD_COMISION_FIJA_PREF) && localStorage.getItem('pandi_nacho_comision_usd_pref')) {
939 |             localStorage.setItem(PANDI_LS_USD_COMISION_FIJA_PREF, localStorage.getItem('pandi_nacho_comision_usd_pref'));
940 |         }
941 |         const p = localStorage.getItem(PANDI_LS_USD_COMISION_FIJA_PREF);
942 |         if (p == null || p === '') return oa;
943 |         const n = Number(p);
944 |         if (Math.abs(n - ob) < 0.01) return ob;
945 |         if (Math.abs(n - oa) < 0.01) return oa;
946 |     } catch (_) {}
947 |     return oa;
948 | }
949 |
950 | function persistirPreferenciaComisionFijaInterUsd(val) {
951 |     const n = Number(val);
952 |     if (!(n > 0 && n < 1e7)) return;
953 |     try {
954 |         if (typeof localStorage !== 'undefined') localStorage.setItem(PANDI_LS_USD_COMISION_FIJA_PREF, String(n));
955 |     } catch (_) {}
956 | }
957 |
958 | /** Compatibilidad si no hay `intermediario_id` en config: nombre con la palabra «nacho». */
959 | function esIntermediarioNachoComisionFijaUsd(nombre) {
960 |     const s = String(nombre || '').trim();

```

```

961|   if (!s) return false;
962|   return /\bnacho\b/i.test(s);
963| }
964|
965| function intermediarioTieneComisionFijaUsdAcordada(intermediarioId, nombreIntermediario) {
966|   if (!intermediarioId) return false;
967|   const cfg = usdUsdComisionFijaConfig;
968|   if (cfg.intermediario_id && String(cfg.intermediario_id) === String(intermediarioId)) return true;
969|   if (!cfg.intermediario_id && esIntermediarioNachoComisionFijaUsd(nombreIntermediario)) return true;
970|   return false;
971| }
972|
973| function marcarRadioComisionFijaUsdSegunMontoGuardado(interMonto) {
974|   aplicarValoresRadiosComisionFijaUsdWizard();
975|   const ir = Math.round(Number(interMonto) * 100) / 100;
976|   const rads = document.querySelectorAll('input[name="orden-usd-nacho-comision-usd"]');
977|   let best = null;
978|   let bestD = Infinity;
979|   rads.forEach((r) => {
980|     const v = Number(r.value);
981|     if (isNaN(v)) return;
982|     const d = Math.abs(ir - v);
983|     if (d < bestD) {
984|       bestD = d;
985|       best = r;
986|     }
987|   });
988|   if (best && bestD < 0.05) best.checked = true;
989| }
990|
991| let sessionCheckIntervalId = null;
992| /** Vista actual para el refresco automático de datos cada 30 s. */
993| let currentVistaId = 'vista-inicio';
994| let refreshDataIntervalId = null;
995| const REFRESH_DATA_INTERVAL_MS = 30000;
996| const SESSION_ACTIVITY_THROTTLE_MS = 30000; // actualizar lastActivityTime como máximo cada 30 s
997| let lastActivityUpdate = 0;
998| /** Evita carrera: si el usuario envía el login antes de que termine getSession() al cargar, el .then tardío no debe volver a mostrar login ni registrar de nuevo los listeners
999|  */
1000| let authBootstrapFromGetSessionDone = false;
1001|
1002| function showLoginScreenDom() {
1003|   pandiSessionUiBootstrapped = false;
1004|   ccDetalleMovimientosRangoInicializado = false;
1005|   document.body.classList.remove('pandi-shell-logged-in');
1006|   document.getElementById('sidebar').style.display = 'none';
1007|   document.getElementById('login-screen').style.display = 'block';
1008|   document.getElementById('register-screen').style.display = 'none';
1009|   document.getElementById('app-content').style.display = 'none';
1010|   pandiSyncSidebarBackdrop();
1011| }
1012|
1013| /** Cierra sesión o carga inicial: actualiza marca desde Supabase y muestra login/registro. */
1014| function showLogin() {
1015|   fetchAppEmpresaIntoState()
1016|     .catch(() => {})
1017|     .finally(() => {
1018|       aplicarMarcaEnTodaLaUI();
1019|       showLoginScreenDom();
1020|     });
1021| }
1022|
1023| function showAppContent() {
1024|   document.body.classList.add('pandi-shell-logged-in');
1025|   document.getElementById('sidebar').style.display = 'flex';
1026|   document.getElementById('login-screen').style.display = 'none';
1027|   document.getElementById('register-screen').style.display = 'none';
1028|   document.getElementById('app-content').style.display = 'block';
1029|   pandiSyncSidebarBackdrop();
1030| }
1031|
1032| function ensureProfile(session) {
1033|   return client
1034|     .from('user_profiles')
1035|     .upsert({ id: session.user.id, email: session.user.email || '' }, { onConflict: 'id' })
1036|     .then(() => client.from('app_user_profile').select('role').eq('user_id', session.user.id).maybeSingle())
1037|     .then((res) => {
1038|       if (res.data === null) {
1039|         return client.from('app_user_profile').insert({ user_id: session.user.id, role: 'visor' });
1040|       }
1041|       return Promise.resolve();
1042|     });
1043| }
1044|
1045| function setupLoginAndRegister() {
1046|   const loginForm = document.getElementById('login-form');
1047|   const registerForm = document.getElementById('register-form');
1048|   const loginError = document.getElementById('login-error');
1049|   const registerError = document.getElementById('register-error');
1050|
1051|   document.getElementById('link-registro').addEventListener('click', (e) => {
1052|     e.preventDefault();
1053|     document.getElementById('login-screen').style.display = 'none';
1054|     document.getElementById('register-screen').style.display = 'block';
1055|     loginError.textContent = '';
1056|     registerError.textContent = '';
1057|   });

```

```

1057 |
1058 | document.getElementById('link-login').addEventListener('click', (e) => {
1059 |     e.preventDefault();
1060 |     document.getElementById('register-screen').style.display = 'none';
1061 |     document.getElementById('login-screen').style.display = 'block';
1062 |     loginError.textContent = '';
1063 |     registerError.textContent = '';
1064 | });
1065 |
1066 | loginForm.addEventListener('submit', (e) => {
1067 |     e.preventDefault();
1068 |     loginError.textContent = '';
1069 |     client.auth
1070 |         .signInWithPassword({
1071 |             email: document.getElementById('login-email').value.trim(),
1072 |             password: document.getElementById('login-password').value,
1073 |         })
1074 |         .then((res) => {
1075 |             if (res.error) {
1076 |                 loginError.textContent = res.error.message || 'Error al iniciar sesión';
1077 |                 return;
1078 |             }
1079 |             authBootstrapFromGetSessionDone = true;
1080 |             onSessionReady(res.data.session);
1081 |         });
1082 | });
1083 |
1084 | registerForm.addEventListener('submit', (e) => {
1085 |     e.preventDefault();
1086 |     registerError.textContent = '';
1087 |     client.auth
1088 |         .signUp({
1089 |             email: document.getElementById('register-email').value.trim(),
1090 |             password: document.getElementById('register-password').value,
1091 |         })
1092 |         .then((res) => {
1093 |             if (res.error) {
1094 |                 registerError.textContent = res.error.message || 'Error al registrarse';
1095 |                 return;
1096 |             }
1097 |             if (res.data.session) {
1098 |                 authBootstrapFromGetSessionDone = true;
1099 |                 onSessionReady(res.data.session);
1100 |             } else {
1101 |                 registerError.textContent = 'Revisá tu email para confirmar la cuenta y luego iniciá sesión.';
1102 |             }
1103 |         });
1104 | });
1105 | }
1106 |
1107 | function refreshPermisosYVista() {
1108 |     client
1109 |         .rpc('get_my_permissions')
1110 |         .then((res) => {
1111 |             if (res.error) return;
1112 |             userPermissions = res.data || [];
1113 |             pandiCachePermissionsLocal(userPermissions);
1114 |             applyVistasMenuVisibility();
1115 |             updateCcBotonesMovimientoManual();
1116 |             const currentVistaId = VIEWS_CONFIG.find((r) => {
1117 |                 const el = document.getElementById(r[1]);
1118 |                 return el && el.style.display === 'block';
1119 |             })?.[1];
1120 |             if (currentVistaId && !canViewVista(currentVistaId)) {
1121 |                 const [firstId, firstTitle] = getFirstAllowedView();
1122 |                 showView(firstId, firstTitle);
1123 |             } else if (currentVistaId === 'vista-inicio') {
1124 |                 loadInicio();
1125 |             } else if (currentVistaId === 'vista-cajas') {
1126 |                 loadCajas({ soloFiltros: true });
1127 |             }
1128 |             pandiRefreshOfflineCatalogosCache();
1129 |             return fetchAppEmpresaIntoState();
1130 |         })
1131 |         .catch(() => {})
1132 |         .then(() => aplicarMarcaEnTodaLaUI());
1133 | }
1134 |
1135 | function showView(vistaId, pageTitle) {
1136 |     if (!canViewVista(vistaId)) {
1137 |         const [firstId, firstTitle] = getFirstAllowedView();
1138 |         showView(firstId, firstTitle);
1139 |         return;
1140 |     }
1141 |     if (typeof pandiModoReducidoOffline !== 'undefined' && pandiModoReducidoOffline && vistaId !== 'vista-ordenes') {
1142 |         showToast('Modo reducido (sin Supabase): solo está disponible la vista Órdenes y la cola local.', 'info');
1143 |         vistaId = 'vista-ordenes';
1144 |         pageTitle = 'Órdenes';
1145 |     }
1146 |     currentVistaId = vistaId;
1147 |     ['vista-inicio', 'vista-ordenes', 'vista-cajas', 'vista-clientes', 'vista-intermediarios', 'vista-tipos-operacion', 'vista-cuenta-corriente', 'vista-reglas-negocio', 'vista-
configuracion-empresa', 'vista-seguridad'].forEach((id) => {
1148 |         const el = document.getElementById(id);
1149 |         if (el) el.style.display = id === vistaId ? 'block' : 'none';
1150 |     });
1151 |     const titleEl = document.getElementById('page-title');
1152 |     if (titleEl) titleEl.textContent = pageTitle;

```

```

1153 document.querySelectorAll('.sidebar-nav .menu-item').forEach((m) => m.classList.remove('menu-item-active'));
1154 const activeItem = document.querySelector('.sidebar-nav .menu-item[id="menu-${vistaId.replace('vista-', '')}"]');
1155 if (activeItem) activeItem.classList.add('menu-item-active');
1156
1157 if (vistaId === 'vista-seguridad') loadSeguridad();
1158 if (vistaId === 'vista-clientes') loadClientes();
1159 if (vistaId === 'vista-cajas') {
1160     invalidateCajasMovimientosFullCache();
1161     loadCajas();
1162 }
1163 if (vistaId === 'vista-ordenes') loadOrdenes();
1164 if (vistaId === 'vista-inicio') loadInicio();
1165 if (vistaId === 'vista-cuenta-corriente') {
1166     // Evitar flash de datos viejos (ej. -200.000): mostrar loading y ocultar paneles antes de cargar.
1167     const ccLoading = document.getElementById('cc-loading');
1168     const ccContenido = document.getElementById('cc-contenido');
1169     const ccPanelSalDOS = document.getElementById('cc-panel-saldos');
1170     const ccPanelMov = document.getElementById('cc-panel-movimientos');
1171     if (ccLoading) ccLoading.style.display = 'block';
1172     if (ccContenido) ccContenido.style.display = 'none';
1173     if (ccPanelSalDOS) ccPanelSalDOS.style.display = 'none';
1174     if (ccPanelMov) ccPanelMov.style.display = 'none';
1175     loadCuentaCorriente();
1176 }
1177 if (vistaId === 'vista-intermediarios') loadIntermediarios();
1178 if (vistaId === 'vista-tipos-operacion') loadTiposOperacion();
1179 if (vistaId === 'vista-reglas-negocio') loadReglasNegocioVista();
1180 if (vistaId === 'vista-configuracion-empresa') loadConfiguracionEmpresa();
1181 }
1182 pandiCollapseMobileSidebarAfterNav();
1183
1184 /** Mensaje al desactivar un permiso (para mostrar contexto al administrador). Solo permisos con mensaje específico. */
1185 const MENSAJE_AL_DESACTIVAR_PERMISO = {
1186     ver_cajas: 'Los usuarios con este rol no podrán acceder a la vista Cajas.',
1187     ver_cajas_efectivo: 'Los usuarios no verán la tarjeta Efectivo en Panel de Control ni en Cajas. Si tienen Operar, igual podrán registrar movimientos en efectivo desde órdenes o transacciones.',
1188     ver_cajas_banco: 'Los usuarios no verán la tarjeta Banco en Panel de Control ni en Cajas. Si tienen Operar, igual podrán registrar movimientos en banco desde órdenes o transacciones.',
1189     ver_cajas_cheque: 'Los usuarios no verán la tarjeta Cheque (solo ARS) en Panel de Control ni en Cajas. Los movimientos con modo de pago cheque se imputan a Caja Cheque (no a efectivo).',
1190     abm_movimientos_caja: 'Los usuarios podrán ver los saldos (según Ver Efectivo/Banco/Cheque) pero no crear ni editar movimientos en Cajas.',
1191     abm_tipos_movimiento_caja: 'Los usuarios no podrán crear ni editar tipos de movimiento de caja.',
1192     abm_reglas_negocio: 'Sin este permiso no podrán abrir el menú crítico de reglas de cuenta corriente (riesgo de romper el modelo CC).',
1193     abm_configuracion_empresa: 'Sin este permiso no podrán abrir Empresa / marca ni editar nombre legal, nombre visible ni URL del logo.',
1194     registrar_movimiento_cc_manual: 'Sin este permiso no podrán registrar movimientos de cuenta corriente sin orden desde la vista CC (sigue aplicando editar transacciones si el rol lo tiene).',
1195     editar_movimiento_cc_manual: 'Sin este permiso no podrán editar movimientos de CC marcados como manual (sin orden) desde la lista de Movimientos.',
1196     eliminar_movimiento_cc_manual: 'Sin este permiso no podrán anular movimientos de CC manuales desde la lista de Movimientos.',
1197     ver_auditoria: 'Sin este permiso no podrán consultar la tabla de auditoría (registro de acciones sensibles) en Supabase o futuras pantallas de log.',
1198 };
1199
1200 /** Si faltan políticas RLS + GRANT en app_role_permission, DELETE/INSERT no afectan filas y PostgreSQL puede no devolver error. */
1201 const MSG_APP_ROLE_PERM_MIGRACION =
1202     'En Supabase ejecutá sql/migracion_permisos_rol_editable.sql (políticas INSERT/DELETE y GRANT en app_role_permission).';
1203
1204 /** Devuelve el permiso "padre" (acceso a la vista) y los "hijos" del menú que contiene este permiso. Si permission es el padre, hijos = resto Ver + Operar. */
1205 function getMenuParentAndChildren(permission) {
1206     for (let i = 0; i < PERMISOS_POR_MENU.length; i++) {
1207         const menu = PERMISOS_POR_MENU[i];
1208         const inVer = (menu.ver || []).includes(permission);
1209         const inOperar = (menu.operar || []).includes(permission);
1210         if (!inVer && !inOperar) continue;
1211         const parentVer = menu.ver && menu.ver[0] ? menu.ver[0] : null;
1212         const children = [...(menu.ver || []).slice(1), ...(menu.operar || [])].filter(p => p !== permission);
1213         return { parentVer, children };
1214     }
1215     return null;
1216 }
1217
1218 /** Permisos agrupados por opción de menú. Ver = acceso a la vista e información; Operar = crear/editar/anular. */
1219 const PERMISOS_POR_MENU = [
1220     {
1221         id: 'inicio',
1222         titulo: 'Panel de Control',
1223         ver: ['ver_inicio', 'ver_inicio_pendientes', 'ver_inicio_gp_operativo'],
1224         verSubPerms: ['ver_inicio_pendientes', 'ver_inicio_gp_operativo'],
1225         operar: [],
1226     },
1227     { id: 'ordenes', titulo: 'Órdenes', ver: ['ver_ordenes'], operar: ['ingresar_orden', 'editar_orden', 'anular_orden', 'editar_estado_orden', 'ingresar_transacciones', 'editar_transacciones', 'eliminar_transacciones'] },
1228     { id: 'cajas', titulo: 'Cajas', ver: ['ver_cajas', 'ver_cajas_efectivo', 'ver_cajas_banco', 'ver_cajas_cheque'], verSubPerms: ['ver_cajas_efectivo', 'ver_cajas_banco', 'ver_cajas_cheque'], operar: ['abm_movimientos_caja', 'abm_tipos_movimiento_caja'] },
1229     { id: 'clientes', titulo: 'Clientes', ver: ['ver_clientes'], operar: ['abm_clientes'] },
1230     { id: 'intermediarios', titulo: 'Intermediarios', ver: ['ver_intermediarios'], operar: ['abm_intermediarios'] },
1231     { id: 'tipos-operacion', titulo: 'Tipos de operación', ver: [], operar: ['abm_tipos_operacion'] },
1232     { id: 'cuenta-corriente', titulo: 'Cuenta corriente', ver: ['ver_cuenta_corriente'], operar: ['registrar_movimiento_cc_manual', 'editar_movimiento_cc_manual', 'eliminar_movimiento_cc_manual'] },
1233     { id: 'configuracion-empresa', titulo: 'Empresa / marca', ver: [], operar: ['abm_configuracion_empresa'] },
1234     { id: 'seguridad', titulo: 'Seguridad', ver: ['ver_seguridad', 'ver_auditoria'], operar: ['assign_roles'] },
1235     { id: 'reglas-negocio', titulo: 'Reglas de negocio (CC)', ver: [], operar: ['abm_reglas_negocio'] },
1236 ];
1237
1238 function loadConfiguracionEmpresa() {
1239     const loading = document.getElementById('config-empresa-loading');
1240     const sinPerm = document.getElementById('config-empresa-sin-permiso');
1241     const formWrap = document.getElementById('config-empresa-form-wrap');
1242     const inpLegal = document.getElementById('config-empresa-nombre-legal');

```

```

1243| const inpNombre = document.getElementById('config-empresa-nombre-sistema');
1244| const inpLogo = document.getElementById('config-empresa-logo-url');
1245| let btnGuardar = document.getElementById('config-empresa-guardar');
1246| if (!loading || !sinPerm || !formWrap) return;
1247|
1248| const puede = userPermissions.includes('abm_configuracion_empresa');
1249| const silent = isPandiBackgroundRefresh();
1250| if (!silent) {
1251|   loading.style.display = 'block';
1252|   sinPerm.style.display = 'none';
1253|   formWrap.style.display = 'none';
1254| }
1255|
1256| return fetchAppEmpresaIntoState()
1257|   .then(() => {
1258|     loading.style.display = 'none';
1259|     if (!puede) {
1260|       sinPerm.style.display = 'block';
1261|       formWrap.style.display = 'none';
1262|       return;
1263|     }
1264|     sinPerm.style.display = 'none';
1265|     formWrap.style.display = 'block';
1266|     setupConfigEmpresaMarcaControles();
1267|     if (inpLegal) inpLegal.value = appEmpresaState.nombre_legal || '';
1268|     if (inpNombre) inpNombre.value = (appEmpresaState.nombre_sistema || '').trim() || MARCA_OPERADOR_DEFAULT_NOMBRE_VISIBLE;
1269|     if (inpLogo) inpLogo.value = appEmpresaState.logo_url || '';
1270|     syncConfigEmpresaLogoPreview();
1271|     btnGuardar = document.getElementById('config-empresa-guardar');
1272|     if (btnGuardar) {
1273|       btnGuardar.replaceWith(btnGuardar.cloneNode(true));
1274|       document.getElementById('config-empresa-guardar').addEventListener('click', () => {
1275|         const legal = inpLegal ? inpLegal.value.trim() : '';
1276|         const nom = inpNombre ? inpNombre.value.trim() : '';
1277|         const logo = inpLogo ? inpLogo.value.trim() : '';
1278|         if (!nom) {
1279|           showToast('El nombre en el sistema no puede estar vacío.', 'error');
1280|           return;
1281|         }
1282|         if (!validarLogoUrlInput(logo)) {
1283|           showToast('URL del logo: usá https o una ruta que empiece con ./, 'error');
1284|           return;
1285|         }
1286|         const row = {
1287|           id: 1,
1288|           nombre_legal: legal,
1289|           nombre_sistema: nom,
1290|           logo_url: logo,
1291|           updated_at: new Date().toISOString(),
1292|           updated_by: currentUserId,
1293|         };
1294|         client
1295|           .from('app_empresa')
1296|           .upsert(row, { onConflict: 'id' })
1297|           .then((res) => {
1298|             if (res.error) {
1299|               showToast('Error: ' + (res.error.message || 'No se pudo guardar. ¿Ejecutaste sql/migracion_app_empresa.sql?'), 'error');
1300|               return;
1301|             }
1302|             mergeAppEmpresaRow(row);
1303|             aplicarMarcaEnTodaLaUI();
1304|             showToast('Configuración de empresa actualizada.', 'success');
1305|           });
1306|       });
1307|     }
1308|   })
1309|   .catch(() => {
1310|     loading.style.display = 'none';
1311|     if (!puede) {
1312|       sinPerm.style.display = 'block';
1313|       formWrap.style.display = 'none';
1314|     } else {
1315|       sinPerm.style.display = 'none';
1316|       formWrap.style.display = 'block';
1317|       showToast('No se pudo cargar app_empresa. Revisá sql/migracion_app_empresa.sql en Supabase.', 'error');
1318|     }
1319|   });
1320| }
1321|
1322| function loadSeguridad() {
1323|   const loadingEl = document.getElementById('seguridad-loading');
1324|   const wrapEl = document.getElementById('seguridad-tabla-wrap');
1325|   const tbody = document.getElementById('seguridad-tbody');
1326|   const permisosWrap = document.getElementById('seguridad-permisos-wrap');
1327|   const permisosGrid = document.getElementById('seguridad-permisos-grid');
1328|   if (!loadingEl || !wrapEl || !tbody) return;
1329|
1330|   if (!userPermissions.includes('assign_roles')) {
1331|     loadingEl.style.display = 'none';
1332|     wrapEl.style.display = 'block';
1333|     tbody.innerHTML = '<tr><td colspan="3">No tenés permiso para gestionar usuarios y roles. Solo un Admin puede asignar roles.</td></tr>';
1334|     if (permisosWrap) permisosWrap.style.display = 'none';
1335|     return;
1336|   }
1337|
1338|   const silentSeg = isPandiBackgroundRefresh();
1339|   if (!silentSeg) {

```

```

1340|     loadingEl.style.display = 'block';
1341|     wrapEl.style.display = 'none';
1342|     tbody.innerHTML = '';
1343|     if (permisosWrap) permisosWrap.style.display = 'none';
1344|     if (permisosGrid) permisosGrid.innerHTML = '';
1345| }
1346|
1347| return Promise.all([
1348|     client.rpc('get_users_for_admin'),
1349|     client.rpc('get_my_role'),
1350|     client.from('app_role').select('role, label').order('role'),
1351|     client.from('app_permission').select('permission, description').order('permission'),
1352|     client.from('app_role_permission').select('role, permission'),
1353| ]).then(([rUsers, rMyRole, rRoles, rPerms, rRolePerms]) => {
1354|     loadingEl.style.display = 'none';
1355|
1356|     const myRole = (rMyRole && rMyRole.data != null) ? String(rMyRole.data) : '';
1357|     const tiempoSesionWrap = document.getElementById('seguridad-tiempo-sesion-wrap');
1358|     const inputTimeout = document.getElementById('seguridad-session-timeout-min');
1359|     const btnGuardarTimeout = document.getElementById('seguridad-session-timeout-guardar');
1360|     if (tiempoSesionWrap && myRole === 'admin') {
1361|         tiempoSesionWrap.style.display = 'block';
1362|         client.from('app_config').select('value').eq('key', 'session_timeout_minutes').maybeSingle().then((r) => {
1363|             const val = (r && r.data && r.data.value) ? parseInt(r.data.value, 10) : 60;
1364|             if (inputTimeout) inputTimeout.value = (val >= 1 && val <= 1440) ? val : 60;
1365|         });
1366|         if (btnGuardarTimeout && inputTimeout) {
1367|             btnGuardarTimeout.replaceWith(btnGuardarTimeout.cloneNode(true));
1368|             document.getElementById('seguridad-session-timeout-guardar').addEventListener('click', () => {
1369|                 const v = parseInt(inputTimeout.value, 10);
1370|                 if (isNaN(v) || v < 1 || v > 1440) {
1371|                     showToast('Ingresá un número entre 1 y 1440.', 'error');
1372|                     return;
1373|                 }
1374|                 client.from('app_config').upsert({ key: 'session_timeout_minutes', value: String(v), updated_at: new Date().toISOString(), updated_by: currentUserId }, { onConflict:
1375| 'key' }).then((res) => {
1376|                     if (res.error) {
1377|                         showToast('Error: ' + (res.error.message || 'No se pudo guardar.'), 'error');
1378|                         return;
1379|                     }
1380|                     sessionTimeoutMinutes = v;
1381|                     showToast('Tiempo de inactividad actualizado. Se aplicará a todas las sesiones.', 'success');
1382|                 });
1383|             }
1384|         } else if (tiempoSesionWrap) {
1385|             tiempoSesionWrap.style.display = 'none';
1386|         }
1387|
1388|     const wrapComFijaUsd = document.getElementById('seguridad-usd-comision-fija-wrap');
1389|     if (wrapComFijaUsd && myRole === 'admin') {
1390|         wrapComFijaUsd.style.display = 'block';
1391|         Promise.all([
1392|             client.from('app_config').select('value').eq('key', 'usd_usd_comision_fija_config').maybeSingle(),
1393|             client.from('intermediarios').select('id, nombre').eq('activo', true).order('nombre', { ascending: true }),
1394| ]).then(([rCfgUsd, rIntUsd]) => {
1395|             const cfgUsd = parseUsdUsdComisionFijaConfigJson(rCfgUsd.data && rCfgUsd.data.value);
1396|             const selUsd = document.getElementById('seguridad-usd-comision-fija-intermediario');
1397|             const inA = document.getElementById('seguridad-usd-comision-fija-opcion-a');
1398|             const inB = document.getElementById('seguridad-usd-comision-fija-opcion-b');
1399|             if (inA) inA.value = String(cfgUsd.opcion_a);
1400|             if (inB) inB.value = String(cfgUsd.opcion_b);
1401|             if (selUsd) {
1402|                 selUsd.innerHTML = '<option value="">Ninguno (compatibilidad: nombre con «nacho»)</option>';
1403|                 (rIntUsd.data || []).forEach((row) => {
1404|                     const o = document.createElement('option');
1405|                     o.value = row.id;
1406|                     o.textContent = row.nombre || row.id;
1407|                     selUsd.appendChild(o);
1408|                 });
1409|                 selUsd.value = cfgUsd.intermediario_id || '';
1410|             }
1411|             const btnUsd = document.getElementById('seguridad-usd-comision-fija-guardar');
1412|             if (btnUsd) {
1413|                 const btnClone = btnUsd.cloneNode(true);
1414|                 btnUsd.replaceWith(btnClone);
1415|                 btnClone.addEventListener('click', () => {
1416|                     const aRaw = document.getElementById('seguridad-usd-comision-fija-opcion-a')?.value;
1417|                     const bRaw = document.getElementById('seguridad-usd-comision-fija-opcion-b')?.value;
1418|                     const ia = parseFloat(String(aRaw || '').replace(',', ''));
1419|                     const ib = parseFloat(String(bRaw || '').replace(',', ''));
1420|                     if (!(ia > 0 && ia < 1e7) || !(ib > 0 && ib < 1e7)) {
1421|                         showToast('Los dos importes deben ser mayores a cero.', 'error');
1422|                         return;
1423|                     }
1424|                     const idInt = String(document.getElementById('seguridad-usd-comision-fija-intermediario')?.value || '').trim();
1425|                     const payload = JSON.stringify({ intermediario_id: idInt, opcion_a: ia, opcion_b: ib });
1426|                     client.from('app_config').upsert({ key: 'usd_usd_comision_fija_config', value: payload, updated_at: new Date().toISOString(), updated_by: currentUserId }, { onConF
1427| lict: 'key' }).then((res) => {
1428|                         if (res.error) {
1429|                             showToast('Error: ' + (res.error.message || 'No se pudo guardar. ¿Migración SQL ejecutada?'), 'error');
1430|                             return;
1431|                         }
1432|                         usdUsdComisionFijaConfig = parseUsdUsdComisionFijaConfigJson(payload);
1433|                         aplicarValoresRadiosComisionFijaUsdWizard();
1434|                         showToast('Comisión fija USD-USD guardada.', 'success');
1435|                     });
1436|                 });
1437|             }
1438|         });
1439|     }
1440| }

```

```

1435 |     });
1436 |   });
1437 | } else if (wrapComFijaUsd) {
1438 |   wrapComFijaUsd.style.display = 'none';
1439 | }
1440 |
1441 | const users = rUsers.data || [];
1442 | const roles = (rRoles.data || []).slice();
1443 |
1444 | if (rUsers.error || users.length === 0) {
1445 |   if (rUsers.error) tbody.innerHTML = '<tr><td colspan="3">Error: ' + (rUsers.error.message || '') + '</td></tr>';
1446 |   else tbody.innerHTML = '<tr><td colspan="3">No hay usuarios.</td></tr>';
1447 |   wrapEl.style.display = 'block';
1448 | } else {
1449 |   const esc = (s) => (s === null ? '' : String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;'));
1450 |   tbody.innerHTML = users
1451 |     .map((u) => {
1452 |       const uid = u.user_id;
1453 |       const email = esc(u.email || '');
1454 |       const role = u.role || 'visor';
1455 |       const optionsWithSelected = roles.map((r) => `<option value="${escapeHtml(r.role)}" ${role === r.role ? 'selected' : ''}>${escapeHtml(r.label || r.role)}</option>`);
1456 |       .join('');
1457 |       return `<tr data-user-id="${uid}">
1458 |         <td>${email}</td>
1459 |         <td><select class="seguridad-rol" data-user-id="${uid}">${optionsWithSelected}</select></td>
1460 |         <td><button type="button" class="btn-guardar-rol btn-primary" data-user-id="${uid}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor"
1461 |           stroke-width="2"><polyline points="20 6 9 17 4 12"/></svg></span>Guardar</button></td>
1462 |       </tr>`;
1463 |     })
1464 |     .join('');
1465 |
1466 | tbody.querySelectorAll('.btn-guardar-rol').forEach((btn) => {
1467 |   btn.addEventListener('click', () => {
1468 |     const uid = btn.getAttribute('data-user-id');
1469 |     const row = btn.closest('tr');
1470 |     const sel = row.querySelector('.seguridad-rol');
1471 |     const newRole = sel.value;
1472 |     client
1473 |       .rpc('set_user_role', { p_user_id: uid, p_role: newRole })
1474 |       .then((r) => {
1475 |         if (r.error) showToast('Error: ' + (r.error.message || 'No se pudo guardar.'), 'error');
1476 |       });
1477 |   });
1478 |   wrapEl.style.display = 'block';
1479 | }
1480 |
1481 | const allPerms = rPerms.data || [];
1482 | const permMap = {};
1483 | allPerms.forEach((p) => { permMap[p.permission] = p.description || p.permission; });
1484 | const rolePermList = rRolePerms.data || [];
1485 | const rolePermSet = new Set(rolePermList.map((r) => r.role + '|' + r.permission));
1486 |
1487 | const iconChevron = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><polyline points="6 9 12 15 18 9"/></svg>`;
1488 | const iconVer = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M1 12s4-8 11-8 11 8 8-11 8-11 8-11 8z"/><circle cx="12" cy="12" r="3"/></svg>`;
1489 | const iconOperar = `<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M11 4H4a2 2 0 0-2 2v14a2 2 0 002 2h14a2 2 0 002 2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013.121 15.4 1 1-4 9.5-9.5z"/></svg>`;
1490 |
1491 | const renderPermToggle = (permKey, roleKey, permMap, isSub) => {
1492 |   const desc = escapeHtml(permMap[permKey] || permKey);
1493 |   const checked = rolePermSet.has(roleKey + '|' + permKey);
1494 |   const id = 'perm-' + roleKey + '-' + permKey.replace(/_/g, '-');
1495 |   const rowClass = 'seguridad-perm-row' + (isSub ? ' seguridad-perm-row-sub' : '');
1496 |   const labelContent = isSub ? `<span class="seguridad-perm-bullet" aria-hidden="true"></span><span>${desc}</span>` : desc;
1497 |   return `<div class="${rowClass}">
1498 |     <label for="${id}">${labelContent}</label>
1499 |     <span class="toggle-switch">
1500 |       <input type="checkbox" id="${id}" class="seguridad-perm-toggle" data-role="${roleKey}" data-permission="${permKey}" ${checked ? 'checked' : ''} />
1501 |       <span class="slider"></span>
1502 |     </span>
1503 |   </div>`;
1504 | };
1505 |
1506 | const renderMenuBlock = (menu, roleKey, permMap) => {
1507 |   const verPerms = (menu.ver || []).filter((p) => permMap[p]);
1508 |   const operarPerms = (menu.operar || []).filter((p) => permMap[p]);
1509 |   if (verPerms.length === 0 && operarPerms.length === 0) return '';
1510 |   const titulo = escapeHtml(menu.titulo);
1511 |   const menuBodyId = `seguridad-menu-body-${roleKey}-${menu.id}`;
1512 |   const menuHeaderId = `seguridad-menu-header-${roleKey}-${menu.id}`;
1513 |   let bodyHtml = '';
1514 |   if (verPerms.length > 0) {
1515 |     const verSubPerms = menu.verSubPerms || [];
1516 |     bodyHtml += `<div class="seguridad-permisos-subgrupo"><span class="seguridad-permisos-subgrupo-label"><span class="seguridad-permisos-grupo-icone" aria-hidden="true">${iconVer}</span></span><span><p class="seguridad-permisos-subgrupo-leyenda">Acceso a la vista y a la información.</p>`;
1517 |     verPerms.forEach((p) => { bodyHtml += renderPermToggle(p, roleKey, permMap, verSubPerms.includes(p)); });
1518 |     bodyHtml += `</div>`;
1519 |   }
1520 |   if (operarPerms.length > 0) {
1521 |     bodyHtml += `<div class="seguridad-permisos-subgrupo"><span class="seguridad-permisos-subgrupo-label"><span class="seguridad-permisos-grupo-icone" aria-hidden="true">${iconOperar}</span></span><span><p class="seguridad-permisos-subgrupo-leyenda">Crear, editar o anular según corresponda.</p>`;
1522 |     operarPerms.forEach((p) => { bodyHtml += renderPermToggle(p, roleKey, permMap, false); });
1523 |     bodyHtml += `</div>`;
1524 |   }

```



```

1525 |     return `<div class="seguridad-permisos-por-menu seguridad-permisos-menu-colapsable collapsed" data-menu="${menu.id}">
1526 |       <button type="button" class="seguridad-permisos-menu-header" id="${menuHeaderId}" aria-expanded="false" aria-controls="${menuBodyId}" aria-label="Expandir ${titulo}">
1527 |         <span class="seguridad-permisos-menu-titulo">${titulo}</span>
1528 |         <span class="seguridad-permisos-menu-chevron" aria-hidden="true">${iconChevron}</span>
1529 |       </button>
1530 |       <div class="seguridad-permisos-menu-body" id="${menuBodyId}" role="region" aria-labelledby="${menuHeaderId}">${bodyHtml}</div>
1531 |     </div>`;
1532 |   };
1533 |
1534 |   if (permisosGrid && roles.length > 0 && allPerms.length > 0) {
1535 |     permisosGrid.innerHTML = roles
1536 |       .map((r) => {
1537 |         const roleKey = r.role;
1538 |         const label = escapeHtml(r.label || roleKey);
1539 |         const blocksByMenu = PERMISOS_POR_MENU.map((menu) => renderMenuBlock(menu, roleKey, permMap)).filter(Boolean).join('');
1540 |         return `<div class="seguridad-permisos-rol" data-role="${roleKey}">
1541 |           <button type="button" class="seguridad-permisos-rol-header" aria-expanded="false" aria-controls="seguridad-rol-body-${roleKey}" id="seguridad-rol-header-${roleKey}"
1542 |             <span>${label}</span>
1543 |             <span class="seguridad-permisos-rol-chevron" aria-hidden="true">${iconChevron}</span>
1544 |           </button>
1545 |           <div class="seguridad-permisos-rol-body" id="seguridad-rol-body-${roleKey}" role="region" aria-labelledby="seguridad-rol-header-${roleKey}">${blocksByMenu}</div>
1546 |         </div>`;
1547 |       })
1548 |       .join('');
1549 |
1550 |     permisosGrid.querySelectorAll('.seguridad-permisos-rol-header').forEach((btn) => {
1551 |       btn.addEventListener('click', () => {
1552 |         const card = btn.closest('.seguridad-permisos-rol');
1553 |         if (!card) return;
1554 |         const isExpanded = card.classList.toggle('expanded');
1555 |         btn.setAttribute('aria-expanded', isExpanded);
1556 |         btn.setAttribute('aria-label', isExpanded ? 'Contraer rol' : 'Expandir rol');
1557 |       });
1558 |     });
1559 |
1560 |     permisosGrid.querySelectorAll('.seguridad-permisos-menu-header').forEach((btn) => {
1561 |       btn.addEventListener('click', () => {
1562 |         const block = btn.closest('.seguridad-permisos-menu-colapsable');
1563 |         if (!block) return;
1564 |         block.classList.toggle('collapsed');
1565 |         const isExpanded = !block.classList.contains('collapsed');
1566 |         btn.setAttribute('aria-expanded', isExpanded);
1567 |         const titulo = block.querySelector('.seguridad-permisos-menu-titulo');
1568 |         const name = titulo ? titulo.textContent.trim() : 'menú';
1569 |         btn.setAttribute('aria-label', isExpanded ? 'Contraer ${name}' : 'Expandir ${name}');
1570 |       });
1571 |     });
1572 |
1573 |     permisosGrid.querySelectorAll('.seguridad-perm-toggle').forEach((chk) => {
1574 |       chk.addEventListener('change', function () {
1575 |         const role = this.getAttribute('data-role');
1576 |         const permission = this.getAttribute('data-permission');
1577 |         const enable = this.checked;
1578 |         const mismoRolQueSesion = String(role || '').trim() === String(myRole || '').trim();
1579 |         if (enable) {
1580 |           client
1581 |             .from('app_role_permission')
1582 |             .insert({ role, permission })
1583 |             .select('role, permission')
1584 |             .then((res) => {
1585 |               if (res.error) {
1586 |                 showToast('Error: ' + (res.error.message || 'No se pudo guardar.'), 'error');
1587 |                 this.checked = false;
1588 |                 return;
1589 |               }
1590 |               if (!res.data || res.data.length === 0) {
1591 |                 showToast('No se guardó el permiso (0 filas). ' + MSG_APP_ROLE_PERM_MIGRACION, 'error');
1592 |                 this.checked = false;
1593 |                 return;
1594 |               }
1595 |               showToast('Permiso activado.', 'success');
1596 |               if (mismoRolQueSesion) refreshPermisosYVista();
1597 |             });
1598 |         } else {
1599 |           return;
1600 |         }
1601 |         const menuInfo = getMenuParentAndChildren(permission);
1602 |         const isParent = menuInfo && menuInfo.parentVer === permission;
1603 |         const toRemove = (isParent && menuInfo.children
1604 |           ? menuInfo.children.filter((p) => permMap[p])
1605 |           : []);
1606 |         const removeParent = () => {
1607 |           client
1608 |             .from('app_role_permission')
1609 |             .delete()
1610 |             .eq('role', role)
1611 |             .eq('permission', permission)
1612 |             .select('role, permission')
1613 |             .then((res) => {
1614 |               if (res.error) {
1615 |                 showToast('Error: ' + (res.error.message || 'No se pudo guardar.'), 'error');
1616 |                 this.checked = true;
1617 |                 return;
1618 |               }
1619 |               if (!res.data || res.data.length === 0) {
1620 |                 showToast('No se eliminó el permiso (0 filas). ' + MSG_APP_ROLE_PERM_MIGRACION, 'error');

```

```

1621 |         return;
1622 |     }
1623 |     if (toRemove.length > 0) {
1624 |         showToast('Se desactivó el acceso al menú y también los demás permisos de este ítem (sin acceso no aplican).', 'info');
1625 |     } else {
1626 |         const msg = MENSAJE_AL_DESACTIVAR_PERMISO[permission];
1627 |         if (msg) showToast(msg, 'info');
1628 |         else showToast('Permiso desactivado.', 'success');
1629 |     }
1630 |     if (mismoRolQueSesion) refreshPermisosYVista();
1631 | });
1632 | };
1633 | if (toRemove.length === 0) {
1634 |     removeParent();
1635 |     return;
1636 | }
1637 | Promise.all(
1638 |     toRemove.map((p) =>
1639 |         client.from('app_role_permission').delete().eq('role', role).eq('permission', p).select('role, permission')
1640 |     )
1641 | ).then((results) => {
1642 |     const fail = results.find((r) => r.error);
1643 |     if (fail) {
1644 |         showToast('Error: ' + (fail.error.message || 'No se pudo guardar.'), 'error');
1645 |         this.checked = true;
1646 |         return;
1647 |     }
1648 |     toRemove.forEach((p) => {
1649 |         const other = permisosGrid.querySelector(`.seguridad-perm-toggle[data-role="${role}"][data-permission="${p}"]`);
1650 |         if (other) other.checked = false;
1651 |     });
1652 |     removeParent();
1653 | });
1654 | });
1655 | });
1656 | if (permisosWrap) permisosWrap.style.display = 'block';
1657 | }
1658 | }).catch(() => {
1659 |     loadingEl.style.display = 'none';
1660 |     if (!silentSeg) wrapEl.style.display = 'block';
1661 | });
1662 | }
1663 | }
1664 | /** Fecha calendario YYYY-MM-DD en zona Argentina (operación del negocio). */
1665 | function fechaHoyYYYYMMDDArgentina() {
1666 |     const parts = new Intl.DateTimeFormat('en-CA', {
1667 |         timeZone: 'America/Argentina/Buenos_Aires',
1668 |         year: 'numeric',
1669 |         month: '2-digit',
1670 |         day: '2-digit',
1671 |     }).formatToParts(new Date());
1672 |     const y = parts.find((p) => p.type === 'year').value;
1673 |     const mo = parts.find((p) => p.type === 'month').value;
1674 |     const d = parts.find((p) => p.type === 'day').value;
1675 |     return y + '-' + mo + '-' + d;
1676 | }
1677 | }
1678 | /** Primeras filas de auditoría en exportaciones Excel (quién exporta y cuándo, Argentina). */
1679 | function metaFilasExportacionExcel() {
1680 |     const emailExp = String(currentUserEmail || '').trim();
1681 |     const fh = new Date().toLocaleString('es-AR', {
1682 |         timeZone: 'America/Argentina/Buenos_Aires',
1683 |         year: 'numeric',
1684 |         month: '2-digit',
1685 |         day: '2-digit',
1686 |         hour: '2-digit',
1687 |         minute: '2-digit',
1688 |         second: '2-digit',
1689 |         hour12: false,
1690 |     });
1691 |     return [
1692 |         ['Exportado por', emailExp || '-'],
1693 |         ['Fecha y hora exportación (Argentina)', fh],
1694 |     ];
1695 | }
1696 | }
1697 | /**
1698 |  * Emails de `user_profiles` para los UUID indicados (según RLS: propio usuario y, si aplica, permiso assign_roles).
1699 |  * @returns {Promise<Record<string, string>>}}
1700 |  */
1701 | function fetchMapaEmailUsuarioPorIds(idsUnicos) {
1702 |     const ids = [...new Set((idsUnicos || []).filter(Boolean))];
1703 |     if (ids.length === 0) return Promise.resolve({});
1704 |     return client
1705 |         .from('user_profiles')
1706 |         .select('id, email')
1707 |         .in('id', ids)
1708 |         .then((r) => {
1709 |             const map = {};
1710 |             (r.data || []).forEach((row) => {
1711 |                 if (row.id) map[String(row.id)] = String(row.email || '').trim();
1712 |             });
1713 |             return map;
1714 |         })
1715 |         .catch(() => ({}));
1716 | }
1717 | }

```

```

1718 /** Une metadatos + fila en blanco + cabecera + filas de datos para SheetJS. */
1719 function aoaExcelConMetaExportacion(header, rows) {
1720     return [...metaFilasExportacionExcel(), [], header, ...rows];
1721 }
1722
1723 /** Suma días a YYYY-MM-DD interpretando el instante como mediodía ART (UTC+3 → 15:00Z) para no cruzar fecha al sumar. */
1724 function fechaAddDaysYYYYMMDDArgentina(ymd, deltaDays) {
1725     const d = new Date(ymd + 'T15:00:00.000Z');
1726     d.setUTCDate(d.getUTCDate() + deltaDays);
1727     const y = d.getUTCFullYear();
1728     const mo = String(d.getUTCMonth() + 1).padStart(2, '0');
1729     const day = String(d.getUTCDate()).padStart(2, '0');
1730     return `${y}-${mo}-${day}`;
1731 }
1732
1733 /** Rango inclusive para G/P Operativa (calendario Argentina; semana lun-dom). */
1734 function inicioGpOperativoRangoFechas(periodo) {
1735     const hoy = fechaHoyYYYYMMDDArgentina();
1736     if (periodo === 'total') return { desde: null, hasta: null };
1737     if (periodo === 'dia') return { desde: hoy, hasta: hoy };
1738     if (periodo === 'mes') {
1739         const y = parseInt(hoy.slice(0, 4), 10);
1740         const mo = parseInt(hoy.slice(5, 7), 10);
1741         const ultimo = new Date(y, mo, 0).getDate();
1742         const d1 = `${hoy.slice(0, 7)}-01`;
1743         const dlast = `${hoy.slice(0, 7)}-${String(ultimo).padStart(2, '0')}`;
1744         return { desde: d1, hasta: dlast };
1745     }
1746     if (periodo === 'semana') {
1747         const d = new Date(hoy + 'T15:00:00.000Z');
1748         const wd = d.getUTCDay();
1749         const diasHastaLun = wd === 0 ? 6 : wd - 1;
1750         const lun = fechaAddDaysYYYYMMDDArgentina(hoy, -diasHastaLun);
1751         const dom = fechaAddDaysYYYYMMDDArgentina(lun, 6);
1752         return { desde: lun, hasta: dom };
1753     }
1754     return { desde: hoy, hasta: hoy };
1755 }
1756
1757 // --- Cajas ---
1758 let cajasMonedaActual = 'TODO';
1759 /** Tabla movimientos: por defecto solo el día (fechas en AR); «Todo el historial» sin filtro de fecha. */
1760 let cajasMovMostrarTodoHistorial = false;
1761 let cajasMovFechaDesde = '';
1762 let cajasMovFechaHasta = '';
1763 /** 'todo' | 'efectivo' | 'banco' | 'cheque' – filtro de la tabla (no afecta saldos de las cards). */
1764 let cajasMovCajaTipoTab = 'todo';
1765 let cajasMovFiltrosListenersAttached = false;
1766 /** Solapa principal vista Cajas: 'movimientos' | 'tipos' (mismo patrón que CC). */
1767 let cajasVistaSolapa = 'movimientos';
1768 /** Panel Inicio – G/P Operativa: período activo en la card. */
1769 let inicioGpOperativoPeriodo = 'dia';
1770 let inicioGpOperativoListenersAttached = false;
1771 let tiposMovimientoCaja = [];
1772 /** Última carga completa (sync + fetch). Cambiar solo moneda/fecha/tipo caja en UI repinta desde acá sin tocar Supabase ni sync masivo – mismo criterio que filtros en memoria en PortfolioDetail (Sistema-Contable). */
1773 let cajasMovimientosFullCache = null;
1774 function invalidateCajasMovimientosFullCache() {
1775     cajasMovimientosFullCache = null;
1776 }
1777
1778 /** Concepto para movimientos de cuenta corriente: "Cobro por USD 5.000,00" (Pandy cobró), "Deuda por ARS 4.170.000,00" (Pandy debe), "Pago por comisión USD 60,00". */
1779 function conceptoCcMovimiento(moneda, monto, tipo) {
1780     const m = Number(monto) || 0;
1781     const mon = moneda || 'USD';
1782     const txt = mon + ' ' + formatMonto(m, mon);
1783     if (tipo === 'comision') return 'Pago por comisión ' + txt;
1784     if (tipo === 'cobro') return 'Cobro por ' + txt;
1785     if (tipo === 'deuda') return 'Deuda por ' + txt;
1786     return 'Pago por ' + txt;
1787 }
1788
1789 /** Leyenda de concepto + " - Orden Nro x" (ordenLabel suele ser "nro orden 1"). Para que todo movimiento CC lleve referencia a la orden. */
1790 function conceptoConOrden(leyenda, ordenLabel) {
1791     if (!leyenda) return (ordenLabel || '').replace('nro orden ', 'Orden Nro ');
1792     const sufijo = ordenLabel ? ' - ' + (ordenLabel + '').replace('nro orden ', 'Orden Nro ') : '';
1793     return leyenda + sufijo;
1794 }
1795
1796 /** Leyendas unificadas para movimientos CC: "Pago Realizado - Orden x y Trans x", "Cobro Realizado - Orden x y Trans x", "Compromiso de Pago - Orden x y Trans x", "Compromiso a Cobrar - Orden x y Trans x", "Compromiso a Cobrar (ingreso pendiente hacia Pandy) va con monto positivo en CC cliente = pendiente de cobro. En listados de detalle se muestra n columnas Pagador y Cobrador de la transacción. */
1797 function conceptoCcLeyenda(tipo, ordenNumero, transNumero) {
1798     const ord = ordenNumero !== null && ordenNumero !== '' ? String(ordenNumero) : '?';
1799     const tr = transNumero !== null && transNumero !== '' ? String(transNumero) : '?';
1800     const suf = ' - Orden ' + ord + ' y Trans ' + tr;
1801     if (tipo === 'cobro_realizado') return 'Cobro Realizado' + suf;
1802     if (tipo === 'pago_realizado') return 'Pago Realizado' + suf;
1803     if (tipo === 'compromiso_pago') return 'Compromiso de Pago' + suf;
1804     if (tipo === 'compromiso_cobrar') return 'Compromiso a Cobrar' + suf;
1805     if (tipo === 'comision_acuerdo') return 'Comisión del acuerdo' + suf;
1806     if (tipo === 'contra_cobro_entrega_pendiente') return 'Contra cobro (entrega pendiente)' + suf;
1807     return 'Movimiento' + suf;
1808 }
1809
1810 /**
1811  * Nro. de transacción del ingreso principal Cliente-Pandy ejecutado (excluye fila cuyo monto coincide con la comisión Pandy del acuerdo).

```

```

1812 | * Leyendas de comisión sintética CHEQUE-ARS / coherencia con USD-USD+int.
1813 | */
1814 | function nroTransIngresoClientePandyPrincipalParaComisionConcepto(transacciones, comisionPandyMonto) {
1815 |   const com = Number(comisionPandyMonto) || 0;
1816 |   const tr = (transacciones || []).
1817 |     .filter((t) =>
1818 |       (t.tipo || '').toLowerCase() === 'ingreso' &&
1819 |       String(t.pagador || '').toLowerCase() === 'cliente' &&
1820 |       String(t.cobrador || '').toLowerCase() === 'pandy' &&
1821 |       (t.estado || '').toLowerCase() === 'ejecutada' &&
1822 |       Math.abs(Number(t.monto) - com) >= 1e-6
1823 |     )
1824 |     .sort((a, b) => (Number(a.numero) || 0) - (Number(b.numero) || 0))[0];
1825 |   return tr && tr.numero !== null ? tr.numero : null;
1826 | }
1827 |
1828 | /**
1829 |  * Convención para concepto de movimiento de caja originado por una transacción.
1830 |  * Formato: "Ingreso de [moneda], por [monto], nro orden [nro], nro transacción [nro]" o "Egreso de ...".
1831 |  * nroOrden y nroTransaccion pueden ser null/undefined; en ese caso se omite esa parte o se pone "?".
1832 |  */
1833 | function conceptoCajaTransaccion(esIngreso, moneda, monto, nroOrden, nroTransaccion) {
1834 |   const tipo = esIngreso ? 'Ingreso' : 'Egreso';
1835 |   const mon = (moneda || 'USD').toUpperCase();
1836 |   const montoStr = formatMonto(Math.abs(Number(monto)) || 0, mon);
1837 |   const ord = nroOrden !== null && nroOrden !== '' ? String(nroOrden) : '?';
1838 |   const trx = nroTransaccion !== null && nroTransaccion !== '' ? String(nroTransaccion) : '?';
1839 |   return tipo + ' de ' + mon + ', por ' + montoStr + ', nro orden ' + ord + ', nro transacción ' + trx;
1840 | }
1841 |
1842 | /**
1843 |  * Igual que conceptoCajaTransaccion pero con un concepto especial (ej. "Ganancia del acuerdo", "Comisión del acuerdo").
1844 |  * Formato: "[Concepto]. Ingreso de [moneda], por [monto], nro orden [nro], nro transacción [nro]".
1845 |  */
1846 | function conceptoCajaTransaccionEspecial(nombreConcepto, moneda, monto, nroOrden, nroTransaccion) {
1847 |   const base = conceptoCajaTransaccion(true, moneda, monto, nroOrden, nroTransaccion);
1848 |   return (nombreConcepto || 'Transacción') + '. ' + base;
1849 | }
1850 |
1851 | /** Objeto monto_usd, monto_ars, monto_eur para inserts en movimientos_cuenta_corriente. Moneda que participa = valor; las demás = 0 (nunca null). */
1852 | function montosCcPorMoneda(moneda, valor) {
1853 |   const v = Number(valor) || 0;
1854 |   const mon = (moneda || 'USD').toUpperCase();
1855 |   return {
1856 |     monto_usd: mon === 'USD' ? v : 0,
1857 |     monto_ars: mon === 'ARS' ? v : 0,
1858 |     monto_eur: mon === 'EUR' ? v : 0,
1859 |   };
1860 | }
1861 |
1862 | function ccNormalizarNombreEmpresaCc(s) {
1863 |   return String(s || '').trim().toLowerCase().replace(/s+/g, ' ');
1864 | }
1865 |
1866 | function ccManualNombreVisibleLeg(leg, nomCliPorId, nomIntPorId) {
1867 |   if (!leg) return '';
1868 |   if (leg.kind === 'cliente' && leg.cliente_id) {
1869 |     const n = nomCliPorId[String(leg.cliente_id)] || nomCliPorId[leg.cliente_id];
1870 |     return n !== null ? String(n) : '';
1871 |   }
1872 |   if (leg.kind === 'intermediario' && leg.intermediario_id) {
1873 |     const n = nomIntPorId[String(leg.intermediario_id)] || nomIntPorId[leg.intermediario_id];
1874 |     return n !== null ? String(n) : '';
1875 |   }
1876 |   return '';
1877 | }
1878 |
1879 | /** Coincide el nombre de la entidad del movimiento con Empresa / marca (app_empresa: nombre legal o nombre en sistema). */
1880 | function ccManualLegEsEmpresaPorNombre(nombreEntidad) {
1881 |   const n = ccNormalizarNombreEmpresaCc(nombreEntidad);
1882 |   if (!n) return false;
1883 |   const legal = ccNormalizarNombreEmpresaCc(appEmpresaState.nombre_legal);
1884 |   const sis = ccNormalizarNombreEmpresaCc(appEmpresaState.nombre_sistema);
1885 |   if (legal && n === legal) return true;
1886 |   if (sis && n === sis) return true;
1887 |   return false;
1888 | }
1889 |
1890 | /**
1891 |  * CC manual – signo en la fila de cada entidad:
1892 |  * - Terceros: quien **paga** (cobro_entidad_pandy) → monto **negativo**; quien **recibe** (pago_pandy_entidad) → **positivo**.
1893 |  * - Fila cuya entidad coincide con **Empresa / marca** (nombre legal o nombre en sistema en Menú Empresa): se **invierte** (empresa **recibe** → negativo; empresa **paga** →
1894 |  positivo), alineado a la óptica de caja.
1895 |  */
1896 | function montoCuentaCorrienteManualSigno(leg, montoAbs, nomCliPorId, nomIntPorId) {
1897 |   const v = Math.abs(Number(montoAbs) || 0);
1898 |   if (v < 1e-9) return 0;
1899 |   const cobro = leg && leg.tip === 'cobro_entidad_pandy';
1900 |   let sign = cobro ? -1 : 1;
1901 |   const nombreLeg = ccManualNombreVisibleLeg(leg, nomCliPorId || {}, nomIntPorId || {});
1902 |   if (ccManualLegEsEmpresaPorNombre(nombreLeg)) sign *= -1;
1903 |   return sign * v;
1904 | }
1905 |
1906 | function puedeRegistrarMovCcManual() {
1907 |   return userPermissions.includes('registrar_movimiento_cc_manual') || userPermissions.includes('editar_transacciones');
1908 | }

```

```

1908 |
1909 | function puedeEditarMovimientoCcManual() {
1910 |   return userPermissions.includes('editar_movimiento_cc_manual');
1911 | }
1912 |
1913 | function puedeEliminarMovimientoCcManual() {
1914 |   return userPermissions.includes('eliminar_movimiento_cc_manual');
1915 | }
1916 |
1917 | /** Registro de auditoria (tabla auditoria_app tras migración). Falla silenciosa si la tabla no existe aún. */
1918 | function registrarAuditoriaApp(categoria, acción, detalle, metadata) {
1919 |   if (!currentUserId) return Promise.resolve();
1920 |   return client.from('auditoria_app').insert({
1921 |     usuario_id: currentUserId,
1922 |     categoria: categoria || 'app',
1923 |     accion: accion || '',
1924 |     detalle: (detalle || '').slice(0, 8000),
1925 |     metadata: metadata || null,
1926 |   }).then((r) => {
1927 |     if (r.error) console.warn('auditoria_app:', r.error.message || r.error);
1928 |   });
1929 | }
1930 |
1931 | /** Instrumentación 1:1 con orden: transacciones e id de instrumentación para anulación. */
1932 | function fetchTransaccionesParaAnulacionOrden(ordenId) {
1933 |   return client
1934 |     .from('instrumentacion')
1935 |     .select('id')
1936 |     .eq('orden_id', ordenId)
1937 |     .maybeSingle()
1938 |     .then((rInst) => {
1939 |       const instId = rInst.data && rInst.data.id;
1940 |       if (!instId) return { transacciones: [], instrumentacionId: null };
1941 |       return client
1942 |         .from('transacciones')
1943 |         .select('id, estado')
1944 |         .eq('instrumentacion_id', instId)
1945 |         .then((rTr) => ({ transacciones: rTr.data || [], instrumentacionId: instId }));
1946 |     });
1947 | }
1948 |
1949 | /** Marca todas las transacciones de la instrumentación como anuladas (coherente con orden anulada). */
1950 | function anularTodasTransaccionesInstrumentacion(instrumentacionId, ahora) {
1951 |   if (!instrumentacionId) return Promise.resolve();
1952 |   return client
1953 |     .from('transacciones')
1954 |     .update({ estado: 'anulada', fecha_ejecucion: null, updated_at: ahora })
1955 |     .eq('instrumentacion_id', instrumentacionId)
1956 |     .neq('estado', 'anulada')
1957 |     .then((r) => {
1958 |       if (r.error) showToast('Error al anular transacciones: ' + (r.error.message || ''), 'error');
1959 |       return r;
1960 |     });
1961 | }
1962 |
1963 | function transaccionesTodasPendientesParaAnulacion(list) {
1964 |   const txs = list || [];
1965 |   if (txs.length === 0) return true;
1966 |   return txs.every((t) => String(t.estado || '').toLowerCase() === 'pendiente');
1967 | }
1968 |
1969 | /** Anula movimientos CC (cliente e intermediario, no manual) y caja por orden_id. Único lugar que aplica estos UPDATE. */
1970 | function anularMovimientosCcYCajaNoManualPorOrden(ordenId, ahora) {
1971 |   return client
1972 |     .from('movimientos_cuenta_corriente')
1973 |     .update({ estado: 'anulado', estado_fecha: ahora })
1974 |     .eq('orden_id', ordenId)
1975 |     .neq('estado', 'anulado')
1976 |     .or('es_movimiento_manual.is.null, es_movimiento_manual.eq.false')
1977 |     .then((rCc) => {
1978 |       if (rCc.error) showToast('Error al anular CC cliente: ' + (rCc.error.message || ''), 'error');
1979 |       return client
1980 |         .from('movimientos_cuenta_corriente_intermediario')
1981 |         .update({ estado: 'anulado', estado_fecha: ahora })
1982 |         .eq('orden_id', ordenId)
1983 |         .neq('estado', 'anulado')
1984 |         .or('es_movimiento_manual.is.null, es_movimiento_manual.eq.false');
1985 |     })
1986 |     .then((rCi) => {
1987 |       if (rCi.error) showToast('Error al anular CC intermediario: ' + (rCi.error.message || ''), 'error');
1988 |       return client
1989 |         .from('movimientos_caja')
1990 |         .update({ estado: 'anulado', estado_fecha: ahora })
1991 |         .eq('orden_id', ordenId)
1992 |         .neq('estado', 'anulado');
1993 |     })
1994 |     .then((rCaja) => {
1995 |       if (rCaja.error) showToast('Error al anular caja: ' + (rCaja.error.message || ''), 'error');
1996 |       return { ok: true };
1997 |     });
1998 | }
1999 |
2000 | /**
2001 |  * Persiste anulación: orden → anulada. Si todas las transacciones están pendientes (o no hay), no toca CC/caja; si no, anula movimientos vía anularMovimientosCcYCajaNoManualP
2002 |  * orOrden.
2003 |  * Incluye órdenes en estado orden_ejecutada (acción grave; confirmación en UI).
2004 |  */

```

```

2004 function ejecutarAnulacionOrdenCompleta(ordenId) {
2005     const ahora = new Date().toISOString();
2006     return client
2007         .from('ordenes')
2008         .select('id, estado, numero')
2009         .eq('id', ordenId)
2010         .single()
2011         .then((rOrd) => {
2012             const ord = rOrd.data;
2013             if (rOrd.error || !ord) return Promise.reject(new Error((rOrd.error && rOrd.error.message) || 'Orden no encontrada'));
2014             if (ord.estado === 'anulada') {
2015                 showToast('La orden ya está anulada.', 'info');
2016                 return { yaAnulada: true };
2017             }
2018             const eraEjecutada = ord.estado === 'orden_ejecutada';
2019             return fetchTransaccionesParaAnulacionOrden(ordenId).then(({ transacciones, instrumentacionId }) => {
2020                 const todasPendientes = transaccionesTodasPendientesParaAnulacion(transacciones);
2021                 return client
2022                     .from('ordenes')
2023                     .update({ estado: 'anulada', updated_at: ahora })
2024                     .eq('id', ordenId)
2025                     .then((rUp) => {
2026                         if (rUp.error) return Promise.reject(new Error(rUp.error.message || 'No se pudo anular la orden'));
2027                         const detalle =
2028                             `Orden anulada` +
2029                             (ord.numero !== null ? ` #` + ord.numero + ` `) +
2030                             (eraEjecutada ? ` (estaba Orden ejecutada).` : `.`) +
2031                             `Transacciones de instrumentación marcadas anulada.` +
2032                             (todasPendientes
2033                                 ? ` Sin cambios en CC/caja (todas las transacciones estaban pendientes o no había transacciones).`
2034                                 : ` Movimientos CC cliente e intermediario (no manual) y caja vinculados a la orden marcados como anulados.`);
2035                         function finAuditoria() {
2036                             registrarAuditoriaApp('orden', 'anular', detalle, {
2037                                 orden_id: ordenId,
2038                                 instrumentacion_id: instrumentacionId,
2039                                 todas_transacciones_pendientes: todasPendientes,
2040                                 afecto_cc_caja: !todasPendientes,
2041                                 orden_estaba_ejecutada: eraEjecutada,
2042                             });
2043                         }
2044                         return anularTodasTransaccionesInstrumentacion(instrumentacionId, ahora).then(() => {
2045                             if (todasPendientes) {
2046                                 finAuditoria();
2047                                 return { ok: true, todasPendientes: true };
2048                             }
2049                             return anularMovimientosCcYCajaNoManualPorOrden(ordenId, ahora).then(() => {
2050                                 finAuditoria();
2051                                 return { ok: true, todasPendientes: false };
2052                             });
2053                         });
2054                     });
2055             });
2056         });
2057 }
2058
2059 function refrescarVistasTrasAnularOrden(cerrarModalOrden) {
2060     if (cerrarModalOrden) ejecutarCierreModalOrden();
2061     loadOrdenes();
2062     loadCajas();
2063     const vistaCc = document.getElementById('vista-cuenta-corriente');
2064     if (vistaCc && vistaCc.style.display !== 'none') loadCuentaCorriente();
2065 }
2066
2067 function solicitarConfirmacionYAnularOrden(ordenId, callbacks) {
2068     client
2069         .from('ordenes')
2070         .select('estado')
2071         .eq('id', ordenId)
2072         .single()
2073         .then((rSt) => {
2074             const st = (rSt.data && rSt.data.estado) || '';
2075             const esEjecutada = st === 'orden_ejecutada';
2076             const base =
2077                 `La orden pasará a estado Anulada.\n\n` +
2078                 `Si todas las transacciones de la instrumentación están pendientes (o no hay ninguna), no se modifican movimientos de cuenta corriente ni de caja.\n\n` +
2079                 `Si hay alguna transacción en otro estado (por ejemplo ejecutada), se marcarán como anulados los movimientos de cuenta corriente del cliente y del intermediario vinculados a la orden (excepto movimientos cargados como manuales), y los movimientos de caja de esa orden. Es una operación sensible.\n\n` +
2080                 `¿Confirmas la anulación?`;
2081             const msg = esEjecutada
2082                 ? `La orden está en estado Orden ejecutada: suele implicar transacciones ya cerradas y movimientos contables reales. Al anular se marcará la orden como Anulada y, salvo el caso excepcional de transacciones todas pendientes, se anularán los movimientos de CC y caja vinculados a esta orden.\n\n` + base
2083                 : base;
2084             showConfirm(
2085                 msg,
2086                 'Anular orden',
2087                 () => {
2088                     ejecutarAnulacionOrdenCompleta(ordenId)
2089                     .then((res) => {
2090                         if (res && res.yaAnulada) return;
2091                         if (!res || res.ok) return;
2092                         showToast('Orden anulada.', 'success');
2093                         if (callbacks && callbacks.onExit) callbacks.onExit();
2094                         else refrescarVistasTrasAnularOrden(false);
2095                     })
2096                     .catch((err) => {
2097                         showToast('Error al anular: ' + (err && err.message ? err.message : String(err)), 'error');
2098                     });
2099                 });
2100         });

```

```

2099 |     },
2100 |     null,
2101 |     'Cancelar',
2102 |     esEjecutada ? 'Anular orden ejecutada' : 'Anular orden'
2103 |   });
2104 | });
2105 | .catch(() => {
2106 |   showToast('No se pudo cargar el estado de la orden. Intentá de nuevo.', 'error');
2107 | });
2108 | }
2109 |
2110 | function ccTablaMovCcPorTipoEntidad(tipo) {
2111 |   return tipo === 'cliente' ? 'movimientos_cuenta_corriente' : 'movimientos_cuenta_corriente_intermediario';
2112 | }
2113 |
2114 | /**
2115 |  * Carga filas CC a editar/anular: una fila o todo el manual_grupo_id (dos patas cliente=cliente, etc.).
2116 |  */
2117 | function ccManualFetchContextOperacion(m) {
2118 |   const gid = m.manual_grupo_id;
2119 |   if (gid) {
2120 |     return Promise.all([
2121 |       client.from('movimientos_cuenta_corriente').select('id, concepto, fecha, movimiento_caja_id, manual_grupo_id').eq('manual_grupo_id', gid),
2122 |       client.from('movimientos_cuenta_corriente_intermediario').select('id, concepto, fecha, movimiento_caja_id, manual_grupo_id').eq('manual_grupo_id', gid),
2123 |     ]).then(([r1, r2]) => {
2124 |       if (r1.error) return Promise.reject(r1.error);
2125 |       if (r2.error) return Promise.reject(r2.error);
2126 |       const filas = [];
2127 |       (r1.data || []).forEach((row) => {
2128 |         filas.push({ tabla: 'movimientos_cuenta_corriente', id: row.id, movimiento_caja_id: row.movimiento_caja_id || null, concepto: row.concepto, fecha: row.fecha });
2129 |       });
2130 |       (r2.data || []).forEach((row) => {
2131 |         filas.push({ tabla: 'movimientos_cuenta_corriente_intermediario', id: row.id, movimiento_caja_id: row.movimiento_caja_id || null, concepto: row.concepto, fecha: row.fe
2132 | cha });
2133 |       });
2134 |       if (filas.length === 0) return Promise.reject(new Error('No se encontraron líneas del grupo manual.'));
2135 |       const cajaId = filas.map(f) => f.movimiento_caja_id).find(Boolean) || null;
2136 |       const concepto0 = filas[0].concepto || '';
2137 |       const fecha0 = (filas[0].fecha || '').toString().slice(0, 10);
2138 |       return {
2139 |         filas: filas.map(f) => ({ tabla: f.tabla, id: f.id, movimiento_caja_id: f.movimiento_caja_id }),
2140 |         concepto: concepto0,
2141 |         fecha: fecha0,
2142 |         tocaCaja: !!cajaId,
2143 |         cajaId,
2144 |         grupoId: gid,
2145 |       };
2146 |     });
2147 |   }
2148 |   const tabla = ccTablaMovCcPorTipoEntidad(m.tipo);
2149 |   return client.from(tabla).select('id, concepto, fecha, movimiento_caja_id').eq('id', m.id).single().then((r) => {
2150 |     if (r.error || !r.data) return Promise.reject(r.error || new Error('Movimiento no encontrado'));
2151 |     const row = r.data;
2152 |     return {
2153 |       filas: [{ tabla, id: row.id, movimiento_caja_id: row.movimiento_caja_id || null }],
2154 |       concepto: row.concepto || '',
2155 |       fecha: (row.fecha || '').toString().slice(0, 10),
2156 |       tocaCaja: !!row.movimiento_caja_id,
2157 |       cajaId: row.movimiento_caja_id || null,
2158 |       grupoId: null,
2159 |     };
2160 |   });
2161 | }
2162 |
2163 | let ccManualEditContext = null;
2164 |
2165 | function closeModalCcManualEditar() {
2166 |   const backdrop = document.getElementById('modal-cc-manual-editar-backdrop');
2167 |   if (backdrop) backdrop.classList.remove('activo');
2168 |   ccManualEditContext = null;
2169 | }
2170 |
2171 | function openModalCcManualEditarDesdeFila(m) {
2172 |   if (!m || !m.es_movimiento_manual || !puedeEditarMovimientoCcManual()) return;
2173 |   ccManualFetchContextOperacion(m).then((ctx) => {
2174 |     ccManualEditContext = ctx;
2175 |     const backdrop = document.getElementById('modal-cc-manual-editar-backdrop');
2176 |     const conceptoEl = document.getElementById('cc-manual-edit-concepto');
2177 |     const fechaEl = document.getElementById('cc-manual-edit-fecha');
2178 |     const avisoCaja = document.getElementById('cc-manual-editar-aviso-caja');
2179 |     const resGrupo = document.getElementById('cc-manual-editar-resumen-grupo');
2180 |     if (!backdrop || !conceptoEl || !fechaEl) return;
2181 |     conceptoEl.value = ctx.concepto || '';
2182 |     fechaEl.value = ctx.fecha || '';
2183 |     if (avisoCaja) avisoCaja.style.display = ctx.tocaCaja ? 'block' : 'none';
2184 |     if (resGrupo) {
2185 |       if (ctx.filas.length > 1) {
2186 |         resGrupo.style.display = 'block';
2187 |         resGrupo.textContent = `Operación con ` + String(ctx.filas.length) + ` líneas en cuenta corriente (mismo grupo). Los cambios se aplican a todas.`;
2188 |       } else {
2189 |         resGrupo.style.display = 'none';
2190 |         resGrupo.textContent = '';
2191 |       }
2192 |     }
2193 |     backdrop.classList.add('activo');
2194 |   }).catch((err) => {
2195 |     showToast('No se pudo cargar el movimiento: ' + (err && err.message ? err.message : 'error'), 'error');

```

```

2195 | });
2196 | }
2197 |
2198 | function htmlCcAccionesMovimientoManualRow(m) {
2199 |     const puedeEd = puedeEditarMovimientoCcManual();
2200 |     const puedeEl = puedeEliminarMovimientoCcManual();
2201 |     if (!m.es movimiento manual || (!puedeEd && !puedeEl)) return '-';
2202 |     const safeTipo = m.tipo === 'intermediario' ? 'intermediario' : 'cliente';
2203 |     const safeId = escapeHtml(String(m.id || ''));
2204 |     let html = '<div class="cc-acciones-manual-wrap" style="display:flex;flex-wrap:wrap;gap:0.25rem;align-items:center;">';
2205 |     if (puedeEd) {
2206 |         html += '<button type="button" class="btn-secondary btn-icon-only btn-cc-manual-editar" data-cc-manual-tipo="' + safeTipo + '" data-cc-manual-id="' + safeId + '" title="Ed
itar movimiento manual" aria-label="Editar movimiento manual"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M11 4
H4a2 2 0 0-2 2v14a2 2 0 002 2h14a2 2 0 002 2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013 3L12 15L4 1 1-4 9.5-9.5z"/></svg></span></button>';
2207 |     }
2208 |     if (puedeEl) {
2209 |         html += '<button type="button" class="btn-secondary btn-icon-only btn-cc-manual-eliminar" data-cc-manual-tipo="' + safeTipo + '" data-cc-manual-id="' + safeId + '" title="
Anular movimiento manual" aria-label="Anular movimiento manual"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><polyline po
ints="3 6 5 6 21 6"/><path d="M19 6v14a2 2 0 01-2 2H7a2 2 0 01-2-2V6m3 0V4a2 2 0 012-2h4a2 2 0 012 2v2"/><line x1="10" y1="11" x2="10" y2="17"/><line x1="14" y1="11" x2="14" y
2="17"/></svg></span></button>';
2210 |     }
2211 |     html += '</div>';
2212 |     return html;
2213 | }
2214 |
2215 | function ccBuscarMovimientoDetallePorIdTipo(id, tipo) {
2216 |     const list = ccMovimientosDetalleList || [];
2217 |     const m = list.find((x) => String(x.id) === String(id) && x.tipo === tipo);
2218 |     if (m) return m;
2219 |     const alt = (ccMovimientosList || []).find((x) => String(x.id) === String(id));
2220 |     if (!alt || !alt.es movimiento manual) return null;
2221 |     const inferred = alt.intermediario id ? 'intermediario' : 'cliente';
2222 |     if (inferred !== tipo) return null;
2223 |     return { ...alt, tipo: inferred };
2224 | }
2225 |
2226 | function setupDelegacionAccionesCcManual() {
2227 |     if (document.body.dataset.ccManualAccionesBound === '1') return;
2228 |     document.body.dataset.ccManualAccionesBound = '1';
2229 |     document.body.addEventListener('click', (e) => {
2230 |         const ed = e.target.closest('.btn-cc-manual-editar');
2231 |         const el = e.target.closest('.btn-cc-manual-eliminar');
2232 |         if (!ed && !el) return;
2233 |         const btn = ed || el;
2234 |         const tipo = btn.getAttribute('data-cc-manual-tipo');
2235 |         const id = btn.getAttribute('data-cc-manual-id');
2236 |         if (!tipo || !id) return;
2237 |         const m = ccBuscarMovimientoDetallePorIdTipo(id, tipo);
2238 |         if (!m || !m.es movimiento manual) {
2239 |             showToast('Movimiento no encontrado o no es manual. Refrescá la vista.', 'info');
2240 |             return;
2241 |         }
2242 |         if (ed) {
2243 |             e.preventDefault();
2244 |             openModalCcManualEditarDesdeFila(m);
2245 |         } else {
2246 |             e.preventDefault();
2247 |             confirmarYAnularCcManualDesdeFila(m);
2248 |         }
2249 |     });
2250 | }
2251 |
2252 | function confirmarYAnularCcManualDesdeFila(m) {
2253 |     if (!m || !m.es movimiento manual || !puedeEliminarMovimientoCcManual()) return;
2254 |     ccManualFetchContextOperacion(m).then((ctx) => {
2255 |         const baseMsg = '¿Anular este movimiento manual? Las líneas quedarán como anuladas y no sumarán al saldo.';
2256 |         const msgCaja = 'Este registro vinculó caja (efectivo): también se anulará el movimiento de caja asociado. La acción quedará registrada en el log de auditoría.';
2257 |         const mensaje = ctx.tocaCaja ? baseMsg + msgCaja : baseMsg;
2258 |         showConfirm(mensaje, 'Anular', () => {
2259 |             ejecutarAnularCcManualContexto(ctx);
2260 |         }, null, 'Cancelar', 'Confirmar anulación');
2261 |     }).catch((err) => {
2262 |         showToast('No se pudo cargar el movimiento: ' + (err && err.message ? err.message : 'error'), 'error');
2263 |     });
2264 | }
2265 |
2266 | function ejecutarAnularCcManualContexto(ctx) {
2267 |     const ahora = new Date().toISOString();
2268 |     const promesas = (ctx.filas || []).map((f) => client.from(f.tabla).update({ estado: 'anulado', estado_fecha: ahora }).eq('id', f.id));
2269 |     const fin = (opts) => {
2270 |         const detalle = 'CC manual anulado. Líneas: ' + (ctx.filas || []).map((f) => f.tabla + ':' + f.id).join(', ');
2271 |         + (ctx.cajaId ? '. Caja: ' + ctx.cajaId : '');
2272 |         registrarAuditoriaApp('cc manual', 'anular', detalle, { grupo_id: ctx.grupoId, caja_id: ctx.cajaId, filas: ctx.filas });
2273 |         if (!opts || !opts.skipOkToast) showToast('Movimiento manual anulado.', 'success');
2274 |         loadCuentaCorriente();
2275 |         if (typeof loadCajas === 'function') loadCajas();
2276 |         if (ccDetalleTipo && ccDetalleId) {
2277 |             fetchMovimientosCcPorEntidad(ccDetalleTipo, ccDetalleId).then(({ movimientos, saldos, ordenes, pendienteEnMoneda, pendienteClasePorMoneda }) => {
2278 |                 ccDetalleMovimientosList = ccDetalleRowsConTipoOpDesdeOrdenes(movimientos, ordenes);
2279 |                 ccDetalleOrdenesList = ordenes || [];
2280 |                 renderCcDetalleTable();
2281 |                 const saldosWrap = document.getElementById('modal-cc-detalle-saldos');
2282 |                 if (saldosWrap && saldos) {
2283 |                     const pendMonModal = pendienteEnMoneda || ccPendientePorMonedaDesdeMovs(movimientos);
2284 |                     saldosWrap.innerHTML = htmlCcModalSaldosCards(saldos, pendMonModal, pendienteClasePorMoneda);
2285 |                     reapiicarVisibilidadMonedasCuentaCorrienteDom();
2286 |                 }

```



```

2287 |         renderCcDetalleOperaciones();
2288 |     }).catch(() => {});
2289 | }
2290 | };
2291 | Promise.all(promesas).then((results) => {
2292 |     const err = results.find((r) => r && r.error);
2293 |     if (err && err.error) {
2294 |         showToast('Error al anular CC: ' + (err.error.message || ''), 'error');
2295 |         return;
2296 |     }
2297 |     if (ctx.cajaId) {
2298 |         client.from('movimientos_caja').update({ estado: 'anulado', estado_fecha: ahora }).eq('id', ctx.cajaId).then((rC) => {
2299 |             if (rC.error) showToast('Líneas CC anuladas; no se pudo anular caja: ' + (rC.error.message || 'revisá permisos.'), 'error');
2300 |             fin({ skipOkToast: !!rC.error });
2301 |         });
2302 |     } else {
2303 |         fin();
2304 |     }
2305 | }).catch((err) => {
2306 |     showToast('Error: ' + (err && err.message ? err.message : ''), 'error');
2307 | });
2308 | }
2309 |
2310 | function ejecutarGuardarCcManualEditar(concepto, fecha) {
2311 |     const ctx = ccManualEditContext;
2312 |     if (!ctx || !ctx.filas || ctx.filas.length === 0) return;
2313 |     const ahora = new Date().toISOString();
2314 |     const promesas = ctx.filas.map((f) => client.from(f.tabla).update({ concepto, fecha }).eq('id', f.id));
2315 |     const fin = (opts) => {
2316 |         const detalle = 'CC manual editado (concepto/fecha). Líneas: ' + ctx.filas.map((f) => f.tabla + ':' + f.id).join(', ');
2317 |         + (ctx.cajaId ? ', Caja: ' + ctx.cajaId : '');
2318 |         registrarAuditoriaApp('cc_manual', 'editar', detalle, { grupo_id: ctx.grupoId, caja_id: ctx.cajaId, concepto, fecha });
2319 |         closeModalCcManualEditar();
2320 |         if (!opts || !opts.skipOkToast) showToast('Cambios guardados.', 'success');
2321 |         loadCuentaCorriente();
2322 |         if (typeof loadCajas === 'function') loadCajas();
2323 |         if (ccDetalleTipo && ccDetalleId) {
2324 |             fetchMovimientosCcPorEntidad(ccDetalleTipo, ccDetalleId).then(({ movimientos, saldos, ordenes, pendienteEnMoneda, pendienteClasePorMoneda }) => {
2325 |                 ccDetalleMovimientosList = ccDetalleRowsConTipoOpDesdeOrdenes(movimientos, ordenes);
2326 |                 ccDetalleOrdenesList = ordenes || [];
2327 |                 renderCcDetalleTable();
2328 |                 const saldosWrap = document.getElementById('modal-cc-detalle-saldos');
2329 |                 if (saldosWrap && saldos) {
2330 |                     const pendMonModal = pendienteEnMoneda || ccPendientePorMonedaDesdeMovs(movimientos);
2331 |                     saldosWrap.innerHTML = htmlCcModalSaldosCards(saldos, pendMonModal, pendienteClasePorMoneda);
2332 |                     replicarVisibilidadMonedasCuentaCorrienteDom();
2333 |                 }
2334 |                 renderCcDetalleOperaciones();
2335 |             }).catch(() => {});
2336 |         }
2337 |     };
2338 |     Promise.all(promesas).then((results) => {
2339 |         const err = results.find((r) => r && r.error);
2340 |         if (err && err.error) {
2341 |             showToast('Error al guardar CC: ' + (err.error.message || ''), 'error');
2342 |             return;
2343 |         }
2344 |         if (ctx.cajaId) {
2345 |             const conceptoCaja = concepto && concepto.indexOf('CC manual') >= 0 ? concepto : ('CC manual efectivo · ' + (concepto || ''));
2346 |             client.from('movimientos_caja').update({ concepto: conceptoCaja, fecha }).eq('id', ctx.cajaId).then((rC) => {
2347 |                 if (rC.error) showToast('CC actualizada; no se pudo actualizar caja: ' + (rC.error.message || 'revisá permisos.'), 'error');
2348 |                 fin({ skipOkToast: !!rC.error });
2349 |             });
2350 |         } else {
2351 |             fin();
2352 |         }
2353 |     }).catch((e) => {
2354 |         showToast('Error: ' + (e && e.message ? e.message : ''), 'error');
2355 |     });
2356 | }
2357 |
2358 | function submitGuardarCcManualEditar() {
2359 |     const ctx = ccManualEditContext;
2360 |     if (!ctx) return;
2361 |     const concepto = (document.getElementById('cc-manual-edit-concepto') || {}).value.trim() || null;
2362 |     const fecha = (document.getElementById('cc-manual-edit-fecha') || {}).value;
2363 |     if (!fecha) {
2364 |         showToast('Indicá la fecha.', 'error');
2365 |         return;
2366 |     }
2367 |     if (ctx.tocaCaja) {
2368 |         const msg = 'Este movimiento vinculó caja (efectivo). Al guardar también se actualizarán la fecha y el concepto del movimiento de caja. La acción quedará registrada en el log de auditoría. ¿Continuar?';
2369 |         showConfirm(msg, 'Guardar', () => {
2370 |             ejecutarGuardarCcManualEditar(concepto, fecha);
2371 |         }, null, 'Cancelar', 'Confirmar edición');
2372 |         return;
2373 |     }
2374 |     ejecutarGuardarCcManualEditar(concepto, fecha);
2375 | }
2376 |
2377 | function setupModalCcManualEditar() {
2378 |     const backdrop = document.getElementById('modal-cc-manual-editar-backdrop');
2379 |     if (!backdrop || backdrop.dataset.ccManualEditBound === '1') return;
2380 |     backdrop.dataset.ccManualEditBound = '1';
2381 |     const btnClose = document.getElementById('modal-cc-manual-editar-close');
2382 |     const btnCancel = document.getElementById('modal-cc-manual-editar-cancelar');

```

```

2383|     const form = document.getElementById('form-cc-manual-editor');
2384|     if (btnClose) btnClose.addEventListener('click', closeModalCcManualEditor);
2385|     if (btnCancel) btnCancel.addEventListener('click', closeModalCcManualEditor);
2386|     setupBackdropCloseOnlyOnRealClick(backdrop, closeModalCcManualEditor);
2387|     if (form) form.addEventListener('submit', (e) => { e.preventDefault(); submitGuardarCcManualEditor(); });
2388| }
2389|
2390| function updateCcBotonesMovimientoManual() {
2391|     const can = puedeRegistrarMovCcManual();
2392|     ['cc-btn-movimiento-manual', 'cc-btn-movimiento-manual-mov'].forEach((id) => {
2393|         const el = document.getElementById(id);
2394|         if (el) el.style.display = can ? '' : 'none';
2395|     });
2396| }
2397|
2398| /**
2399|  * Montos CC cuando la orden tiene dos monedas (monR, monE): la fila debe llevar el mismo importe económico en AMBAS monedas (con signo que corresponda).
2400|  * monedaTransaccion + valorTransaccion = monto de la transacción; se convierte a la otra moneda con mr/me para que nunca quede 0 en una moneda participante.
2401|  */
2402| function montosCcPorOrden(monR, monE, mr, me, monedaTransaccion, valorTransaccion) {
2403|     const v = numCc(valorTransaccion);
2404|     const mR = (monR || 'USD').toUpperCase();
2405|     const mE = (monE || 'USD').toUpperCase();
2406|     const mon = (monedaTransaccion || 'USD').toUpperCase();
2407|     if (mR === mE) return montosCcPorMoneda(mon, v);
2408|     const enMonR = mon === mR ? v : ratioCc(v * mr, me, v);
2409|     const enMonE = mon === mE ? v : ratioCc(v * me, mr, v);
2410|     return {
2411|         monto_usd: mR === 'USD' ? numCc(enMonR) : (mE === 'USD' ? numCc(enMonE) : 0),
2412|         monto_ars: mR === 'ARS' ? numCc(enMonR) : (mE === 'ARS' ? numCc(enMonE) : 0),
2413|         monto_eur: mR === 'EUR' ? numCc(enMonR) : (mE === 'EUR' ? numCc(enMonE) : 0),
2414|     };
2415| }
2416|
2417| /** División segura para montos CC: evita NaN e infinitos. Si denominador es 0 o muy chico, devuelve fallback (nunca null). */
2418| function ratioCc(num, denom, fallback) {
2419|     const d = Number(denom);
2420|     if (d == null || isNaN(d) || Math.abs(d) < 1e-6) return fallback != null ? Number(fallback) || 0 : 0;
2421|     const q = Number(num) / d;
2422|     if (q == null || isNaN(q) || !isFinite(q)) return fallback != null ? Number(fallback) || 0 : 0;
2423|     return q;
2424| }
2425|
2426| /** Asegura que un monto por moneda para CC nunca sea null; si es NaN o no numérico, devuelve 0. Para monedas participantes no usar 0 cuando hay monto real (evitar en el cálculo). */
2427| function numCc(val) {
2428|     const n = Number(val);
2429|     return (n != null && !isNaN(n) && isFinite(n)) ? n : 0;
2430| }
2431|
2432| /** Montos para movimiento "Cancelación de deuda" a partir de la transacción y la orden. Usa siempre el monto de la transacción (item.monto), no mr/me. En órdenes misma moneda
    (monR === monE) se usa solo ese monto en la moneda participante. */
2433| function montosCancelacionDesdeOrden(item, orden) {
2434|     const monR = (orden.moneda_recibida || 'USD').toUpperCase();
2435|     const monE = (orden.moneda_entregada || 'USD').toUpperCase();
2436|     const mr = Number(orden.monto_recibido) || 0;
2437|     const me = Number(orden.monto_entregado) || 0;
2438|     const montoTrx = Number(item.monto) || 0;
2439|     const esIngreso = (item.pagador || '').toLowerCase() === 'cliente';
2440|     if (monR === monE) {
2441|         const signo = esIngreso ? 1 : -1;
2442|         return {
2443|             monto_usd: numCc(monR === 'USD' ? signo * montoTrx : 0),
2444|             monto_ars: numCc(monR === 'ARS' ? signo * montoTrx : 0),
2445|             monto_eur: numCc(monR === 'EUR' ? signo * montoTrx : 0),
2446|         };
2447|     }
2448|     if (esIngreso) {
2449|         const enMonE = ratioCc(montoTrx * me, mr, montoTrx);
2450|         return {
2451|             monto_usd: numCc(monR === 'USD' ? montoTrx : (monE === 'USD' ? enMonE : 0)),
2452|             monto_ars: numCc(monR === 'ARS' ? montoTrx : (monE === 'ARS' ? enMonE : 0)),
2453|             monto_eur: numCc(monR === 'EUR' ? montoTrx : (monE === 'EUR' ? enMonE : 0)),
2454|         };
2455|     }
2456|     const enMonR = ratioCc(montoTrx * mr, me, montoTrx);
2457|     return {
2458|         monto_usd: numCc(monR === 'USD' ? -enMonR : (monE === 'USD' ? -montoTrx : 0)),
2459|         monto_ars: numCc(monR === 'ARS' ? -enMonR : (monE === 'ARS' ? -montoTrx : 0)),
2460|         monto_eur: numCc(monR === 'EUR' ? -enMonR : (monE === 'EUR' ? -montoTrx : 0)),
2461|     };
2462| }
2463|
2464| /**
2465|  * SELECT movimientos_caja cerrados + dedupe por id (sin sync). Para pintar rápido; el sync global corre en paralelo y luego se refresca si sigue en vista.
2466|  * @returns {Promise<Array<{id, moneda, monto, concepto, fecha, caja_tipo, ...}>>}}
2467|  */
2468| function fetchMovimientosCajaCerradosSinSync() {
2469|     return client
2470|         .from('movimientos_caja')
2471|         .select('id, moneda, monto, concepto, fecha, tipo_movimiento_id, orden_id, transaccion_id, orden_numero, transaccion_numero, estado, estado_fecha, caja_tipo, usuario_id')
2472|         .eq('estado', 'cerrado')
2473|         .order('fecha', { ascending: false })
2474|         .order('created_at', { ascending: false })
2475|         .then((res) => {
2476|             if (res.error) return [];
2477|             const raw = res.data || [];

```

```

2478 |     const seenIds = new Set();
2479 |     return raw.filter((m) => {
2480 |         if (m.id !== null && seenIds.has(m.id)) return false;
2481 |         if (m.id !== null) seenIds.add(m.id);
2482 |         return true;
2483 |     });
2484 | });
2485 | }
2486 |
2487 | /**
2488 |  * Sync global y luego lista (export Excel y rutas que requieren datos ya alineados antes de leer).
2489 |  * @returns {Promise<Array>}
2490 |  */
2491 | function getListaMovimientosCajaParaSalDOS() {
2492 |     return sincronizarCcYCajaParaTodasLasOrdenesConInstrumentacion().then(() => fetchMovimientosCajaCerradosSinSync());
2493 | }
2494 |
2495 | /**
2496 |  * A partir de la lista de movimientos (de getListaMovimientosCajaParaSalDOS), calcula saldos por caja_tipo y moneda.
2497 |  * Misma lógica que las cards de Cajas: efectivo, banco y cheque (por moneda).
2498 |  */
2499 | function saldosCajaDesdeLista(list) {
2500 |     const saldos = { efectivo: { USD: 0, EUR: 0, ARS: 0 }, banco: { USD: 0, EUR: 0, ARS: 0 }, cheque: { USD: 0, EUR: 0, ARS: 0 } };
2501 |     (list || []).forEach((m) => {
2502 |         const tipo = (m.caja_tipo || 'efectivo').toLowerCase();
2503 |         const moneda = (m.moneda || '').toUpperCase();
2504 |         if (saldos[tipo] && saldos[tipo][moneda] !== null) saldos[tipo][moneda] += Number(m.monto);
2505 |     });
2506 |     return saldos;
2507 | }
2508 |
2509 | const EPS_SALDO_CAJA_VALIDACION = 1e-6;
2510 |
2511 | function saldoCajaTipoMonedaDesdeLista(list, cajaTipo, moneda) {
2512 |     const saldos = saldosCajaDesdeLista(list);
2513 |     const tipo = String(cajaTipo || 'efectivo').toLowerCase();
2514 |     const mon = String(moneda || '').toUpperCase();
2515 |     if (!saldos[tipo] || saldos[tipo][mon] == null) return 0;
2516 |     return Number(saldos[tipo][mon]) || 0;
2517 | }
2518 |
2519 | function etiquetaCajaTipoUi(tipo) {
2520 |     const t = String(tipo || '').toLowerCase();
2521 |     if (t === 'banco') return 'banco';
2522 |     if (t === 'cheque') return 'cheques';
2523 |     return 'efectivo';
2524 | }
2525 |
2526 | /** Mensaje claro cuando un egreso de caja empresa supera el saldo del bucket (efectivo / banco / cheque) y moneda. */
2527 | function mensajeSaldoCajaInsuficienteEgreso({ cajaTipo, moneda, disponible, requerido }) {
2528 |     const mon = String(moneda || '').toUpperCase();
2529 |     return (
2530 |         'Saldo insuficiente en caja de la empresa (' + etiquetaCajaTipoUi(cajaTipo) + ', ' + mon + '). Disponible: ' +
2531 |         formatMonto(disponible, mon) + '. Esta operación requiere al menos ' + formatMonto(requerido, mon) + '.'
2532 |     );
2533 | }
2534 |
2535 | /**
2536 |  * @param {Array} list - movimientos caja cerrados (misma convención que saldosCajaDesdeLista)
2537 |  * @returns {{ ok: true } | { ok: false, mensaje: string }}
2538 |  */
2539 | function validarSaldoCajaParaEgresoLista(list, { cajaTipo, moneda, montoAbs }) {
2540 |     const req = Number(montoAbs) || 0;
2541 |     if (req <= EPS_SALDO_CAJA_VALIDACION) return { ok: true };
2542 |     const disp = saldoCajaTipoMonedaDesdeLista(list, cajaTipo, moneda);
2543 |     if (disp + EPS_SALDO_CAJA_VALIDACION >= req) return { ok: true };
2544 |     return {
2545 |         ok: false,
2546 |         mensaje: mensajeSaldoCajaInsuficienteEgreso({
2547 |             cajaTipo,
2548 |             moneda: String(moneda || '').toUpperCase(),
2549 |             disponible: disp,
2550 |             requerido: req,
2551 |         }),
2552 |     };
2553 | }
2554 |
2555 | /**
2556 |  * Devuelve true si existe al menos un tipo de operación activo que use la moneda en IN u OUT.
2557 |  * Si hay error de consulta, fallback conservador: true (no ocultar columna).
2558 |  */
2559 | function hayTipoOperacionActivoConMoneda(moneda) {
2560 |     const mon = String(moneda || '').toUpperCase().trim();
2561 |     if (!mon) return Promise.resolve(true);
2562 |     return client
2563 |         .from('tipos_operacion')
2564 |         .select('id', { head: true, count: 'exact' })
2565 |         .eq('activo', true)
2566 |         .or('moneda_in.eq.${mon}', 'moneda_out.eq.${mon}')
2567 |         .then((res) => {
2568 |             if (res.error) return true;
2569 |             return Number(res.count || 0) > 0;
2570 |         })
2571 |         .catch(() => true);
2572 | }
2573 |
2574 | const MONEDAS_PANEL_CAJA_EFECTIVO = ['USD', 'ARS', 'EUR'];

```

```

2575 const MONEDAS_PANEL_CAJA_BANCO = ['USD', 'ARS'];
2576
2577 /** Monedas en UI CC (resumen y movimientos): mismo criterio que Panel/Cajas (`hayTipoOperacionActivoConMoneda`). Orden resumen = USD, EUR, ARS. */
2578 const MONEDAS_CC_UI = ['USD', 'EUR', 'ARS'];
2579 /** Orden de columnas monetarias en tablas de movimientos CC (vista + modal). */
2580 const MONEDAS_CC_MOVIMIENTOS_COLS = ['USD', 'ARS', 'EUR'];
2581 let ccUiMonedasVisibles = { USD: true, EUR: true, ARS: true };
2582
2583 /** Iconos moneda fiat en UI (Panel, CC, listado órdenes). */
2584 const URL_ICONO_MONEDA_ASSETS = { USD: '/assets/Icono_Dolar.avif', EUR: '/assets/Icono_Euro.avif', ARS: '/assets/Icono_ARS.webp' };
2585
2586 /**
2587  * Cruce con tipo de cambio (misma convención que ARS-USD / USD-ARS, anclado en USD o en EUR).
2588  * - compra_usd / compra_eur: el cliente compra la moneda entregada (USD o EUR); la otra fiat se calcula con tc = unidades de la moneda recibida por 1 unidad entregada.
2589  * - vende_usd / vende_eur: el cliente vende la moneda recibida (USD o EUR); la otra se calcula con tc.
2590  * No aplica a USD-USD ni CHEQUE-ARS.
2591  */
2592 function patronTipoCambioOrden(recNorm, entNorm) {
2593   const r = (recNorm || '').toUpperCase();
2594   const e = (entNorm || '').toUpperCase();
2595   if (!r || !e || r === e) return '';
2596   const fiat = ['USD', 'EUR', 'ARS'];
2597   if (!fiat.includes(r) || !fiat.includes(e)) return '';
2598   if (e === 'USD' && r !== 'USD') return 'compra_usd';
2599   if (r === 'USD' && e !== 'USD') return 'vende_usd';
2600   if (e === 'EUR' && r !== 'EUR') return 'compra_eur';
2601   if (r === 'EUR' && e !== 'EUR') return 'vende_eur';
2602   return '';
2603 }
2604
2605 /** compra_usd o compra_eur: TC × monto entregado → monto recibido. */
2606 function esPatronCompraFiatConTc(p) {
2607   return p === 'compra_usd' || p === 'compra_eur';
2608 }
2609 /** vende_usd o vende_eur: monto recibido × TC → monto entregado. */
2610 function esPatronVendeFiatConTc(p) {
2611   return p === 'vende_usd' || p === 'vende_eur';
2612 }
2613
2614 function reaplicarVisibilidadMonedasCuentaCorrienteDom() {
2615   ['#vista-cuenta-corriente', '#modal-cc-detalle-backdrop'].forEach((sel) => {
2616     const root = document.querySelector(sel);
2617     if (!root) return;
2618     MONEDAS_CC_UI.forEach((m) => {
2619       const show = !!ccUiMonedasVisibles[m];
2620       root.querySelectorAll('[data-cc-moneda-col="'+m+'"]').forEach((el) => {
2621         el.style.display = show ? '' : 'none';
2622       });
2623     });
2624   });
2625 }
2626
2627 /** Aplica visibilidad USD/EUR/ARS en CC según tipos de operación activos (IN/OUT). */
2628 function aplicarVisibilidadMonedasCuentaCorriente(flagsRaw) {
2629   ccUiMonedasVisibles = normalizarFlagsCajaMonedas(flagsRaw, MONEDAS_CC_UI);
2630   reaplicarVisibilidadMonedasCuentaCorrienteDom();
2631 }
2632
2633 function ccColspanResumenSaldoVacio() {
2634   const n = MONEDAS_CC_UI.filter((m) => ccUiMonedasVisibles[m]).length;
2635   return 1 + n + 1;
2636 }
2637
2638 /** Montos por moneda de un movimiento CC (cliente o intermediario), misma lógica que buildCcResumenRows. */
2639 function getMontosMovimientoCcResumen(m) {
2640   const hasPorMoneda = m.monto_usd !== null || m.monto_ars !== null || m.monto_eur !== null;
2641   if (hasPorMoneda) {
2642     return {
2643       USD: Number(m.monto_usd) || 0,
2644       ARS: Number(m.monto_ars) || 0,
2645       EUR: Number(m.monto_eur) || 0,
2646     };
2647   }
2648   const mon = (m.moneda || '').toString().toUpperCase();
2649   const val = Number(m.monto) || 0;
2650   return {
2651     USD: mon === 'USD' ? val : 0,
2652     ARS: mon === 'ARS' ? val : 0,
2653     EUR: mon === 'EUR' ? val : 0,
2654   };
2655 }
2656
2657 /** True por moneda si hay al menos un movimiento en estado pendiente con monto en esa moneda. */
2658 function ccPendientePorMonedaDesdeMovs(movs) {
2659   const f = { USD: false, EUR: false, ARS: false };
2660   (movs || []).forEach((m) => {
2661     if (String(m.estado || '').toLowerCase() !== 'pendiente') return;
2662     const z = getMontosMovimientoCcResumen(m);
2663     if (Math.abs(z.USD) >= 1e-9) f.USD = true;
2664     if (Math.abs(z.EUR) >= 1e-9) f.EUR = true;
2665     if (Math.abs(z.ARS) >= 1e-9) f.ARS = true;
2666   });
2667   return f;
2668 }
2669
2670 /**
2671  * Resumen CC (tabla Saldos y export): convierte la suma algebraica de movimientos en base al sentido Pandi.

```

```

2672 | * @deprecated Usar ccSaldoDisplayOpticaResumen con clase; se mantiene como fallback (mixto / sin pendiente clasificable).
2673 | */
2674 | function ccSaldoResumenOpticaPandy(algebraico) {
2675 |     return -Number(algebraico) || 0;
2676 | }
2677 |
2678 | /** +1 = Pandy cobra (ingreso hacia Pandy); -1 = Pandy paga (egreso desde Pandy); 0 = no aplica regla simple. */
2679 | function ccSentidoPandyDesdeTipoPagCob(tipo, pagador, cobrador) {
2680 |     const tipol = (tipo || '').toLowerCase();
2681 |     const pag = String(pagador != null ? pagador : (tipol === 'egreso' ? 'pandy' : 'cliente')).toLowerCase();
2682 |     const cob = String(cobrador != null ? cobrador : (tipol === 'ingreso' ? 'pandy' : 'cliente')).toLowerCase();
2683 |     if (tipol === 'egreso' && pag === 'pandy') return -1;
2684 |     if (tipol === 'ingreso' && cob === 'pandy') return +1;
2685 |     return 0;
2686 | }
2687 |
2688 | /**
2689 |  * Por moneda: si solo hay pendientes de cobro → 'cobro'; solo de pago (Pandy debe) → 'pago'; ambos → 'mixto'; ninguno → 'ninguno'.
2690 |  * Misma regla para CC cliente e intermediario (siempre respecto de Pandy).
2691 |  */
2692 | function ccClasificarPendienteMoneda(movs, mon, trTipoById, trPagadorById, trCobradorById, clienteIdPropio, ordenById, trParticipanteIdsByTrx) {
2693 |     const tipoM = trTipoById || {};
2694 |     const pagM = trPagadorById || {};
2695 |     const cobM = trCobradorById || {};
2696 |     let tieneCobro = false;
2697 |     let tienePago = false;
2698 |     (movs || []).forEach((m) => {
2699 |         if (String(m.estado || '').toLowerCase() !== 'pendiente') return;
2700 |         const mm = getMontosMovimientoCcResumen(m);
2701 |         if (Math.abs(mm[mon] || 0) < 1e-9) return;
2702 |         const tid = m.transaccion_id;
2703 |         if (!tid) return;
2704 |         const s = ccSentidoPandyDesdeTipoPagCob(tipoM[tid], pagM[tid], cobM[tid]);
2705 |         if (s === 1) tieneCobro = true;
2706 |         if (s === -1) tienePago = true;
2707 |     });
2708 |     if (tienePago && !tieneCobro) return 'pago';
2709 |     if (tieneCobro && !tienePago) return 'cobro';
2710 |     if (tienePago && tieneCobro) return 'mixto';
2711 |     /** Sin pendientes clasificables: multicontraparte u otras patas con solo mov. cerrados (p. ej. tercero cobró ingreso del acuerdo). */
2712 |     if (clienteIdPropio && trParticipanteIdsByTrx && ordenById) {
2713 |         const pagCli = trParticipanteIdsByTrx.pagadorClienteIdByTrx || {};
2714 |         const cobCli = trParticipanteIdsByTrx.cobradorClienteIdByTrx || {};
2715 |         let cCobro = false;
2716 |         let cPago = false;
2717 |         (movs || []).forEach((m) => {
2718 |             if (String(m.estado || '').toLowerCase() !== 'cerrado') return;
2719 |             const mm = getMontosMovimientoCcResumen(m);
2720 |             if (Math.abs(mm[mon] || 0) < 1e-9) return;
2721 |             const tid = m.transaccion_id;
2722 |             if (!tid) return;
2723 |             const tipol = String(tipoM[tid] || '').toLowerCase();
2724 |             let pag = String(pagM[tid] || '').toLowerCase();
2725 |             let cob = String(cobM[tid] || '').toLowerCase();
2726 |             if (!pag) {
2727 |                 pag = tipol === 'egreso' ? 'pandy' : 'cliente';
2728 |             }
2729 |             if (!cob) {
2730 |                 cob = tipol === 'ingreso' ? 'pandy' : 'cliente';
2731 |             }
2732 |             const orden = ordenById[m.orden_id];
2733 |             const pagIdAcuerdo = pag === 'cliente' && orden && orden.cliente_id ? orden.cliente_id : null;
2734 |             const pagEf = pagCli[tid] || pagIdAcuerdo;
2735 |             const cobEf = cobCli[tid] || (cob === 'cliente' && String(m.cliente_id) === String(clienteIdPropio) ? clienteIdPropio : null);
2736 |             if (tipol === 'ingreso' && cob === 'cliente' && cobEf && String(cobEf) === String(clienteIdPropio) && pagEf && String(pagEf) !== String(clienteIdPropio)) {
2737 |                 cCobro = true;
2738 |             }
2739 |             if (tipol === 'egreso' && pag === 'cliente') {
2740 |                 const pagEf = pagCli[tid] || (String(m.cliente_id) === String(clienteIdPropio) ? clienteIdPropio : null);
2741 |                 const cobEf = cobCli[tid];
2742 |                 if (pagEf && String(pagEf) === String(clienteIdPropio) && cobEf && String(cobEf) !== String(clienteIdPropio)) {
2743 |                     cPago = true;
2744 |                 }
2745 |             }
2746 |         });
2747 |         if (cPago && !cCobro) return 'pago';
2748 |         if (cCobro && !cPago) return 'cobro';
2749 |         if (cPago && cCobro) return 'mixto';
2750 |     }
2751 | }
2752 | /**
2753 |  * Manuales cerrados sin transaccion_id: la suma algebraica coincide con Movimientos (mismo signo en grilla).
2754 |  * Saldos/resumen muestran ese algebraico tal cual; esta clasificación sigue sirviendo para pendienteEnMoneda y merges CHEQUE / USD-USD+int.
2755 |  * cobro_entidad_pandy / pago_pandy_entidad alimentan manPago/manCobro (varios manuales → mixto sin invertir el neto en pantalla).
2756 |  */
2757 | let manCobro = false;
2758 | let manPago = false;
2759 | (movs || []).forEach((m) => {
2760 |     if (String(m.estado || '').toLowerCase() !== 'cerrado') return;
2761 |     if (!m.es_movimiento_manual) return;
2762 |     const mm = getMontosMovimientoCcResumen(m);
2763 |     if (Math.abs(mm[mon] || 0) < 1e-9) return;
2764 |     const tip = String(m.manual_tip_movimiento || '');
2765 |     if (tip === 'cobro_entidad_pandy') manPago = true;
2766 |     else if (tip === 'pago_pandy_entidad') manCobro = true;
2767 | });
2768 | if (manPago && !manCobro) return 'pago';
2769 | if (manCobro && !manPago) return 'cobro';

```

```

2769 | if (manPago && manCobro) return 'mixto';
2770 | return 'ninguno';
2771 | }
2772 |
2773 | function ccPendienteClasePorMonedaDesdeMovs(movs, trTipoById, trPagadorById, trCobradorById, clienteIdPropio, ordenById, trParticipanteIdsByTrx) {
2774 |     const out = { USD: 'ninguno', EUR: 'ninguno', ARS: 'ninguno' };
2775 |     ['USD', 'EUR', 'ARS'].forEach((mon) => {
2776 |         out[mon] = ccClasificarPendienteMoneda(
2777 |             movs,
2778 |             mon,
2779 |             trTipoById,
2780 |             trPagadorById,
2781 |             trCobradorById,
2782 |             clienteIdPropio || null,
2783 |             ordenById || null,
2784 |             trParticipanteIdsByTrx || null
2785 |         );
2786 |     });
2787 |     return out;
2788 | }
2789 |
2790 | /**
2791 |  * CHEQUE-ARS + intermediario: ingreso Cliente-Pandy en instrumentación pendiente (cheques aún no entregados).
2792 |  * El CC puede no tener movimiento pendiente con transaccion_id para esa trx; igual el ARS del resumen debe clasificarse como cobro a favor de Pandy.
2793 |  */
2794 | function ccMapClienteChequeIngresoPrincipalPendiente(ordenById, transaccionesByOrdenId) {
2795 |     const out = {};
2796 |     Object.keys(ordenById || {}).forEach((ordenId) => {
2797 |         const orden = ordenById[ordenId];
2798 |         if (!orden || !orden.cliente_id || !orden.intermediario_id) return;
2799 |         const meta = tiposOperacionNestedMeta(orden.tipos_operacion);
2800 |         const codigo = meta.codigo !== '-' ? meta.codigo : null;
2801 |         const toJoin = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
2802 |         if (!esTipoOperacionChequeArs(codigo, toJoin?.moneda_in, toJoin?.moneda_out)) return;
2803 |         const trans = (transaccionesByOrdenId && transaccionesByOrdenId[ordenId]) || [];
2804 |         const tiene = trans.some((t) => {
2805 |             if (String(t.estado || '').toLowerCase() !== 'pendiente') return false;
2806 |             if (String(t.tipo || '').toLowerCase() !== 'ingreso') return false;
2807 |             if (String(t.pagador || '').toLowerCase() !== 'cliente') return false;
2808 |             if (String(t.cobrador || '').toLowerCase() !== 'pandy') return false;
2809 |             return true;
2810 |         });
2811 |         if (tiene) out[orden.cliente_id] = true;
2812 |     });
2813 |     return out;
2814 | }
2815 |
2816 | /**
2817 |  * CHEQUE-ARS + intermediario: egreso Pandy-Cliente en instrumentación pendiente (p. ej. efectivo a entregar al cliente).
2818 |  * El CC puede no tener movimiento pendiente con transaccion_id; el resumen ARS debe clasificarse como pago (Pandy debe).
2819 |  */
2820 | function ccMapClienteChequeEgresoPandyClientePendiente(ordenById, transaccionesByOrdenId) {
2821 |     const out = {};
2822 |     Object.keys(ordenById || {}).forEach((ordenId) => {
2823 |         const orden = ordenById[ordenId];
2824 |         if (!orden || !orden.cliente_id || !orden.intermediario_id) return;
2825 |         const meta = tiposOperacionNestedMeta(orden.tipos_operacion);
2826 |         const codigo = meta.codigo !== '-' ? meta.codigo : null;
2827 |         const toJoin = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
2828 |         if (!esTipoOperacionChequeArs(codigo, toJoin?.moneda_in, toJoin?.moneda_out)) return;
2829 |         const trans = (transaccionesByOrdenId && transaccionesByOrdenId[ordenId]) || [];
2830 |         const tiene = trans.some((t) => {
2831 |             if (String(t.estado || '').toLowerCase() !== 'pendiente') return false;
2832 |             if (String(t.tipo || '').toLowerCase() !== 'egreso') return false;
2833 |             if (String(t.pagador || '').toLowerCase() !== 'pandy') return false;
2834 |             if (String(t.cobrador || '').toLowerCase() !== 'cliente') return false;
2835 |             return true;
2836 |         });
2837 |         if (tiene) out[orden.cliente_id] = true;
2838 |     });
2839 |     return out;
2840 | }
2841 |
2842 | /**
2843 |  * CHEQUE-ARS + intermediario: ingreso Intermediario-Pandy en instrumentación pendiente (p. ej. efectivo que el int. aún debe entregar).
2844 |  * CC intermediario puede no clasificar por transaccion_id → resumen ARS como cobro a favor de Pandy.
2845 |  */
2846 | function ccMapIntermediarioChequeIngresoIntPandyPendiente(ordenById, transaccionesByOrdenId) {
2847 |     const out = {};
2848 |     Object.keys(ordenById || {}).forEach((ordenId) => {
2849 |         const orden = ordenById[ordenId];
2850 |         if (!orden || !orden.intermediario_id) return;
2851 |         const meta = tiposOperacionNestedMeta(orden.tipos_operacion);
2852 |         const codigo = meta.codigo !== '-' ? meta.codigo : null;
2853 |         const toJoin = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
2854 |         if (!esTipoOperacionChequeArs(codigo, toJoin?.moneda_in, toJoin?.moneda_out)) return;
2855 |         const trans = (transaccionesByOrdenId && transaccionesByOrdenId[ordenId]) || [];
2856 |         const tiene = trans.some((t) => {
2857 |             if (String(t.estado || '').toLowerCase() !== 'pendiente') return false;
2858 |             if (String(t.tipo || '').toLowerCase() !== 'ingreso') return false;
2859 |             if (String(t.pagador || '').toLowerCase() !== 'intermediario') return false;
2860 |             if (String(t.cobrador || '').toLowerCase() !== 'pandy') return false;
2861 |             return true;
2862 |         });
2863 |         if (tiene) out[orden.intermediario_id] = true;
2864 |     });
2865 |     return out;

```

```

2866 | }
2867 |
2868 | /**
2869 |  * CHEQUE-ARS + intermediario: egreso Pandy-Intermediario en instrumentación pendiente (p. ej. cheque a entregar al int.).
2870 |  */
2871 | function ccMapIntermediarioChequeEgresoPandyIntPendiente(ordemById, transaccionesByOrdenId) {
2872 |   const out = {};
2873 |   Object.keys(ordemById || {}).forEach((ordenId) => {
2874 |     const orden = ordenById[ordenId];
2875 |     if (!orden || !orden.intermediario_id) return;
2876 |     const meta = tiposOperacionNestedMeta(orden.tipos_operacion);
2877 |     const codigo = meta.codigo !== '-' ? meta.codigo : null;
2878 |     const toJoin = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
2879 |     if (!esTipoOperacionChequeArs(codigo, toJoin?.moneda_in, toJoin?.moneda_out)) return;
2880 |     const trans = (transaccionesByOrdenId && transaccionesByOrdenId[ordenId]) || [];
2881 |     const tiene = trans.some((t) => {
2882 |       if (String(t.estado || '').toLowerCase() !== 'pendiente') return false;
2883 |       if (String(t.tipo || '').toLowerCase() !== 'egreso') return false;
2884 |       if (String(t.pagador || '').toLowerCase() !== 'pandy') return false;
2885 |       if (String(t.cobrador || '').toLowerCase() !== 'intermediario') return false;
2886 |       return true;
2887 |     });
2888 |     if (tiene) out[orden.intermediario_id] = true;
2889 |   });
2890 |   return out;
2891 | }
2892 |
2893 | /** USD-USD orden: moneda de la transacción o, si falta, acuerdo solo-USD (evita mapas vacíos si `transacciones.moneda` es null). */
2894 | function esTrxMonedaUsdEnOrdenUsd(t, orden) {
2895 |   const m = String(t.moneda || '').toUpperCase();
2896 |   if (m === 'USD') return true;
2897 |   if (m) return false;
2898 |   const mr = String(orden.moneda_recibida || '').toUpperCase();
2899 |   const me = String(orden.moneda_entregada || '').toUpperCase();
2900 |   return mr === 'USD' && me === 'USD';
2901 | }
2902 |
2903 | /**
2904 |  * USD-USD + intermediario: ingreso desde cliente pendiente en USD (instrumentación).
2905 |  * - cp_ic: Cliente-Pandy pendiente.
2906 |  * - ci_pc: Cliente-Intermediario pendiente.
2907 |  * El CC puede no reflejar pendiente con transaccion_id → resumen USD como cobro (misma idea que CHEQUE-ARS).
2908 |  */
2909 | function ccMapClienteUsdUsdIntIngresoClientePandyPendiente(ordemById, transaccionesByOrdenId) {
2910 |   const out = {};
2911 |   Object.keys(ordemById || {}).forEach((ordenId) => {
2912 |     const orden = ordenById[ordenId];
2913 |     if (!orden || !orden.cliente_id || !orden.intermediario_id) return;
2914 |     const meta = tiposOperacionNestedMeta(orden.tipos_operacion);
2915 |     const codigo = (meta.codigo !== '-' ? String(meta.codigo) : '').toUpperCase();
2916 |     if (codigo !== 'USD-USD') return;
2917 |     const trans = (transaccionesByOrdenId && transaccionesByOrdenId[ordenId]) || [];
2918 |     const tiene = trans.some((t) => {
2919 |       if (String(t.estado || '').toLowerCase() !== 'pendiente') return false;
2920 |       if (String(t.tipo || '').toLowerCase() !== 'ingreso') return false;
2921 |       if (String(t.pagador || '').toLowerCase() !== 'cliente') return false;
2922 |       const cob = String(t.cobrador || '').toLowerCase();
2923 |       if (cob !== 'pandy' && cob !== 'intermediario') return false;
2924 |       return esTrxMonedaUsdEnOrdenUsd(t, orden);
2925 |     });
2926 |     if (tiene) out[orden.cliente_id] = true;
2927 |   });
2928 |   return out;
2929 | }
2930 |
2931 | /**
2932 |  * USD-USD + intermediario: egreso “entrega al cliente” pendiente en USD (instrumentación).
2933 |  * - ci_pc: Pandy-Cliente pendiente.
2934 |  * - cp_ic: Intermediario-Cliente pendiente (el mapa anterior solo miraba Pandy-Cliente y no aplicaba al patrón más habitual con ingreso C-P ejecutado).
2935 |  */
2936 | function ccMapClienteUsdUsdIntEgresoPandyClientePendiente(ordemById, transaccionesByOrdenId) {
2937 |   const out = {};
2938 |   Object.keys(ordemById || {}).forEach((ordenId) => {
2939 |     const orden = ordenById[ordenId];
2940 |     if (!orden || !orden.cliente_id || !orden.intermediario_id) return;
2941 |     const meta = tiposOperacionNestedMeta(orden.tipos_operacion);
2942 |     const codigo = (meta.codigo !== '-' ? String(meta.codigo) : '').toUpperCase();
2943 |     if (codigo !== 'USD-USD') return;
2944 |     const trans = (transaccionesByOrdenId && transaccionesByOrdenId[ordenId]) || [];
2945 |     const tiene = trans.some((t) => {
2946 |       if (String(t.estado || '').toLowerCase() !== 'pendiente') return false;
2947 |       if (String(t.tipo || '').toLowerCase() !== 'egreso') return false;
2948 |       if (String(t.cobrador || '').toLowerCase() !== 'cliente') return false;
2949 |       const pag = String(t.pagador || '').toLowerCase();
2950 |       if (pag !== 'pandy' && pag !== 'intermediario') return false;
2951 |       return esTrxMonedaUsdEnOrdenUsd(t, orden);
2952 |     });
2953 |     if (tiene) out[orden.cliente_id] = true;
2954 |   });
2955 |   return out;
2956 | }
2957 |
2958 | /**
2959 |  * Ajusta pendienteClasePorMoneda.USD según instrumentación USD-USD+int.
2960 |  * Si la instrumentación muestra una sola pata pendiente, esa clase manda sobre un cobro/pago mal inferido desde CC (leyenda explícita y color).
2961 |  */
2962 | function mergePendienteClaseUsdUsdIntCliente(clienteId, saldos, pendienteEnMoneda, pendienteClasePorMoneda, mapIng, mapEgreso) {

```

```

2963| const ing = mapIng && mapIng[clienteId];
2964| const pagoInst = mapEgreso && mapEgreso[clienteId];
2965| if (!ing && !pagoInst) return pendienteClasePorMoneda;
2966| const pendU = pendienteEnMoneda.USD;
2967| const absS = Math.abs(Number(saldos.USD) || 0);
2968| if (!pendU && absS < 1e-6) return pendienteClasePorMoneda;
2969| const cur = pendienteClasePorMoneda.USD;
2970| if (ing && pagoInst) return { ...pendienteClasePorMoneda, USD: 'mixto' };
2971| if (ing && !pagoInst) {
2972|   return { ...pendienteClasePorMoneda, USD: cur === 'mixto' ? 'mixto' : 'cobro' };
2973| }
2974| if (pagoInst && !ing) {
2975|   return { ...pendienteClasePorMoneda, USD: cur === 'mixto' ? 'mixto' : 'pago' };
2976| }
2977| return pendienteClasePorMoneda;
2978| }
2979|
2980| /**
2981|  * USD-USD + intermediario: saldo CC intermediario en USD negativo con todo cerrado = deuda de Pandi hacia el intermediario.
2982|  * Sin merge, la clase queda "ninguno" y la óptica de resumen invierte el signo (verde en vez de rojo).
2983|  */
2984| function mergePendienteClaseUsdUsdIntIntermediario(intId, saldos, pendienteEnMoneda, pendienteClasePorMoneda, movsI, ordenById) {
2985|   ordenById = ordenById || {};
2986|   const pendU = pendienteEnMoneda && pendienteEnMoneda.USD;
2987|   const alg = Number(saldos.USD) || 0;
2988|   // Con deuda neta a favor del int. (saldo negativo), no cortar por pendientes puntuales (p. ej. comisión int. pendiente en cp_ic P,E).
2989|   if (Math.abs(alg) < 1e-6 || (pendU && alg >= -1e-6)) return pendienteClasePorMoneda;
2990|   if (pendienteClasePorMoneda.USD !== 'ninguno') return pendienteClasePorMoneda;
2991|   if (alg >= -1e-6) return pendienteClasePorMoneda;
2992|   const hayOrdenUsdUsdInt = (movsI || []).some((m) => {
2993|     const oid = m.orden_id;
2994|     if (!oid) return false;
2995|     const o = ordenById[oid];
2996|     if (!o || o.intermediario_id == null || String(o.intermediario_id) !== String(intId)) return false;
2997|     const meta = tiposOperacionNestadoMeta(o.tipos_operacion);
2998|     return meta.codigo === 'USD-USD' && meta.usa_intermediario === true;
2999|   });
3000|   if (!hayOrdenUsdUsdInt) return pendienteClasePorMoneda;
3001|   return { ...pendienteClasePorMoneda, USD: 'pago' };
3002| }
3003|
3004| /**
3005|  * CHEQUE-ARS + intermediario: saldo CC intermediario en ARS negativo sin pendiente en movimientos = deuda Pandi (óptica roja en resumen).
3006|  * Paralelo a mergePendienteClaseUsdUsdIntIntermediario (USD).
3007|  */
3008| function mergePendienteClaseChequeArsIntermediarioCerradoPago(intId, saldos, pendienteEnMoneda, pendienteClasePorMoneda, movsI, ordenById) {
3009|   ordenById = ordenById || {};
3010|   const pendA = pendienteEnMoneda && pendienteEnMoneda.ARS;
3011|   const alg = Number(saldos.ARS) || 0;
3012|   if (pendA || Math.abs(alg) < 1e-6) return pendienteClasePorMoneda;
3013|   if (pendienteClasePorMoneda.ARS !== 'ninguno') return pendienteClasePorMoneda;
3014|   if (alg >= -1e-6) return pendienteClasePorMoneda;
3015|   const hayOrdenChequeInt = (movsI || []).some((m) => {
3016|     const oid = m.orden_id;
3017|     if (!oid) return false;
3018|     const o = ordenById[oid];
3019|     if (!o || o.intermediario_id == null || String(o.intermediario_id) !== String(intId)) return false;
3020|     return esTipoOperacionChequeArsDesdeJoin(o.tipos_operacion);
3021|   });
3022|   if (!hayOrdenChequeInt) return pendienteClasePorMoneda;
3023|   return { ...pendienteClasePorMoneda, ARS: 'pago' };
3024| }
3025|
3026| /**
3027|  * Importe en resumen CC / Excel resumen / modal cards.
3028|  * Misma convención que la grilla Movimientos: saldo = suma algebraica de movimientos no anulados.
3029|  * Positivo → verde y «Pendiente de cobro»; negativo → rojo y «Pendiente de pago» (ver ccLeyendaSaldoResumenHtml).
3030|  * El parámetro clase se mantiene por compatibilidad con callers y merges CHEQUE / USD-USD+int (ya no invierte el signo).
3031|  */
3032| function ccSaldoDisplayOpticaResumen(sAlg, clase) {
3033|   void clase;
3034|   const a = Number(sAlg) || 0;
3035|   if (Math.abs(a) < 1e-9) return 0;
3036|   return a;
3037| }
3038|
3039| /** Leyenda bajo saldo resumen/modal: misma convención que el color (verde/positivo → cobro, rojo/negativo → pago). Los parámetros mon/pendMon/clase se ignoran (compat. llamad
3040| as). */
3041| function ccLeyendaSaldoResumenHtml(_mon, _pendMon, _clase, sDisp) {
3042|   const EPS = 1e-6;
3043|   const n = Number(sDisp) || 0;
3044|   if (Math.abs(n) < EPS) return '';
3045|   if (n > 0) return '<span class="cc-saldo-leyenda">Pendiente de cobro</span>';
3046|   return '<span class="cc-saldo-leyenda">Pendiente de pago</span>';
3047| }
3048|
3049| function ccColspanVistaDetalleMovimientos() {
3050|   const n = MONEDAS_CC_MOVIMIENTOS_COLS.filter((m) => ccUiMonedasVisibles[m]).length;
3051|   return 5 + n + 3 + 1;
3052| }
3053|
3054| function ccColspanModalDetalleMovimientos() {
3055|   const n = MONEDAS_CC_MOVIMIENTOS_COLS.filter((m) => ccUiMonedasVisibles[m]).length;
3056|   return 5 + n + 3 + 1;
3057| }
3058|
3059| function htmlCcModalSaldosCards(saldos, pendMon, pendClase) {

```



```

3059|     const sal = saldos || { USD: 0, EUR: 0, ARS: 0 };
3060|     const pend = pendMon || { USD: false, EUR: false, ARS: false };
3061|     const pcl = pendClase || { USD: 'ninguno', EUR: 'ninguno', ARS: 'ninguno' };
3062|     return MONEDAS_CC_UI.map(mon) => {
3063|         const sAlg = Number(sal[mon]) || 0;
3064|         const sD = cSaldoDisplayOpticasumen(sAlg, pcl[mon]);
3065|         const label = sD >= 0 ? 'Positivo' : 'Negativo';
3066|         const val = formatMonto(sD >= 0 ? sD : -sD, mon);
3067|         const cls = 'valor ' + (sD >= 0 ? 'positivo' : 'negativo');
3068|         const leyenda = ccLeyendaSaldoResumenHtml(mon, pend, pcl[mon], sD);
3069|         const src = URL_ICONO_MONEDA_ASSETS[mon] || '';
3070|         return `<div class="card" data-cc-moneda-col="${mon}" style="min-width:120px;"><span class="card-titulo"> ${mon}</span><span class="cc-saldo-label" aria-hidden="true">${label}</span><span class="${cls}">${val}</span>${leyenda}</div>`;
3071|     }).join('');
3072| }
3073|
3074| function normalizarFlagsCajaMonedas(flags, ordenKeys) {
3075|     const out = {};
3076|     ordenKeys.forEach((k) => { out[k] = !(flags && flags[k]); });
3077|     if (ordenKeys.every((k) => !out[k])) ordenKeys.forEach((k) => { out[k] = true; });
3078|     return out;
3079| }
3080|
3081| function labelWidthRemPanelCaja() {
3082|     try {
3083|         return window.matchMedia && window.matchMedia('(max-width: 768px)').matches ? '5.5rem' : '6.5rem';
3084|     } catch (e) {
3085|         return '6.5rem';
3086|     }
3087| }
3088|
3089| /** Oculta columnas USD/ARS/EUR (Efectivo) y USD/ARS (Banco) en Panel y Cajas si ningún tipo de operación activo usa esa moneda en IN/OUT. */
3090| function setVisibilidadColumnasMonedasPanelCajas(efectivoFlagsRaw, bancoFlagsRaw) {
3091|     const ef = normalizarFlagsCajaMonedas(efectivoFlagsRaw, MONEDAS_PANEL_CAJA_EFECTIVO);
3092|     const ba = normalizarFlagsCajaMonedas(bancoFlagsRaw, MONEDAS_PANEL_CAJA_BANCO);
3093|     const labelW = labelWidthRemPanelCaja();
3094|
3095|     function aplicarCard(cardId, flags, ordenMonedas) {
3096|         const card = document.getElementById(cardId);
3097|         if (!card) return;
3098|         const n = ordenMonedas.filter((m) => flags[m]).length;
3099|         /* Mínimo ancho por moneda: importes largos + Var. con %; si no entra, scroll en .inicio-caja-tabla. */
3100|         const colMon = window.matchMedia && window.matchMedia('(max-width: 480px)').matches ? 'minmax(5.5rem, 1fr)' : 'minmax(7rem, 1fr)';
3101|         const gridTpl = n === 0 ? labelW : `${labelW} repeat(${n}, ${colMon})`;
3102|         ordenMonedas.forEach((m) => {
3103|             const show = !flags[m];
3104|             card.querySelectorAll('[data-caja-moneda-col="${m}"]`).forEach((el) => {
3105|                 el.style.display = show ? '' : 'none';
3106|             });
3107|         });
3108|         card.querySelectorAll('.inicio-caja-fila').forEach((fila) => {
3109|             fila.style.gridTemplateColumns = gridTpl;
3110|         });
3111|     }
3112|
3113|     aplicarCard('inicio-card-efectivo', ef, MONEDAS_PANEL_CAJA_EFECTIVO);
3114|     aplicarCard('cajas-card-efectivo', ef, MONEDAS_PANEL_CAJA_EFECTIVO);
3115|     aplicarCard('inicio-card-banco', ba, MONEDAS_PANEL_CAJA_BANCO);
3116|     aplicarCard('cajas-card-banco', ba, MONEDAS_PANEL_CAJA_BANCO);
3117| }
3118|
3119| function syncCajasMovFechaInputs() {
3120|     const dEl = document.getElementById('cajas-mov-desde');
3121|     const hEl = document.getElementById('cajas-mov-hasta');
3122|     if (!dEl || !hEl) return;
3123|     if (cajasMovMostrarTodoHistorial) {
3124|         dEl.value = '';
3125|         hEl.value = '';
3126|         return;
3127|     }
3128|     dEl.value = cajasMovFechaDesde || '';
3129|     hEl.value = cajasMovFechaHasta || '';
3130| }
3131|
3132| /** Permisos de ver saldos/filtros por tipo de caja (alineado a cards de Cajas e Inicio). */
3133| function getPermisosVerCajaTipoMovimientos() {
3134|     const verEfectivo = userPermissions.includes('ver_cajas_efectivo');
3135|     const verBanco = userPermissions.includes('ver_cajas_banco');
3136|     const verCheque = userPermissions.includes('ver_cajas_cheque');
3137|     const tieneAlgunoSubPerm = verEfectivo || verBanco || verCheque;
3138|     return { verEfectivo, verBanco, verCheque, tieneAlgunoSubPerm };
3139| }
3140|
3141| /** Ajusta pestaña activa si el usuario ya no puede ver ese tipo o si solo puede una caja (no tiene sentido "Todas"). */
3142| function normalizarCajasMovCajaTipoTabSegunPermisos() {
3143|     const { verEfectivo, verBanco, verCheque, tieneAlgunoSubPerm } = getPermisosVerCajaTipoMovimientos();
3144|     if (!tieneAlgunoSubPerm) return;
3145|     const permitidos = [];
3146|     if (verEfectivo) permitidos.push('efectivo');
3147|     if (verBanco) permitidos.push('banco');
3148|     if (verCheque) permitidos.push('cheque');
3149|     const n = permitidos.length;
3150|     if (n === 0) return;
3151|     if (cajasMovCajaTipoTab === 'todo') {
3152|         if (n === 1) cajasMovCajaTipoTab = permitidos[0];
3153|         return;
3154|     }

```

```

3155 |   if (!permitidos.includes(cajasMovCajaTipoTab)) {
3156 |     cajasMovCajaTipoTab = n === 1 ? permitidos[0] : 'todo';
3157 |   }
3158 | }
3159 |
3160 | /** Muestra/oculta botones [data-caja-tab] según ver_cajas efectivo/banco/cheque (misma lógica que las cards). */
3161 | function aplicarVisibilidadBotonesFiltroCajaMovimientos() {
3162 |   const { verEfectivo, verBanco, verCheque, tieneAlgunoSubPerm } = getPermisosVerCajaTipoMovimientos();
3163 |   document.querySelectorAll('[data-caja-tab]').forEach((b) => {
3164 |     const v = (b.getAttribute('data-caja-tab') || 'todo').toLowerCase();
3165 |     if (v === 'todo') {
3166 |       if (!tieneAlgunoSubPerm) b.style.display = '';
3167 |       else {
3168 |         const cuantos = [verEfectivo, verBanco, verCheque].filter(Boolean).length;
3169 |         b.style.display = cuantos >= 2 ? '' : 'none';
3170 |       }
3171 |       return;
3172 |     }
3173 |     if (!tieneAlgunoSubPerm) {
3174 |       b.style.display = '';
3175 |       return;
3176 |     }
3177 |     const show =
3178 |       (v === 'efectivo' && verEfectivo) ||
3179 |       (v === 'banco' && verBanco) ||
3180 |       (v === 'cheque' && verCheque);
3181 |     b.style.display = show ? '' : 'none';
3182 |   });
3183 | }
3184 |
3185 | /** Filtra la lista completa para la tabla (moneda, tipo caja, rango fechas). Los saldos usan `list` sin este filtro. */
3186 | function filtrarMovimientosCajaVista(list) {
3187 |   const { verEfectivo, verBanco, verCheque, tieneAlgunoSubPerm } = getPermisosVerCajaTipoMovimientos();
3188 |   let filtrados = (cajasMonedaActual === 'TODO' ? list : list.filter((m) => m.moneda === cajasMonedaActual))
3189 |     .filter((m) => ['efectivo', 'banco', 'cheque'].includes((m.caja_tipo || 'efectivo').toLowerCase()));
3190 |   if (tieneAlgunoSubPerm) {
3191 |     filtrados = filtrados.filter((m) => {
3192 |       const t = (m.caja_tipo || 'efectivo').toLowerCase();
3193 |       const tipo = t === 'cheque' ? 'cheque' : t === 'banco' ? 'banco' : 'efectivo';
3194 |       if (tipo === 'efectivo') return verEfectivo;
3195 |       if (tipo === 'banco') return verBanco;
3196 |       return verCheque;
3197 |     });
3198 |   }
3199 |   if (cajasMovCajaTipoTab !== 'todo') {
3200 |     const tab = cajasMovCajaTipoTab;
3201 |     filtrados = filtrados.filter((m) => (m.caja_tipo || 'efectivo').toLowerCase() === tab);
3202 |   }
3203 |   if (!cajasMovMostrarTodoHistorial) {
3204 |     const hoyM = fechaHoyYYYYMMDDArgentina();
3205 |     const desde = cajasMovFechaDesde || hoyM;
3206 |     const hasta = cajasMovFechaHasta || hoyM;
3207 |     filtrados = filtrados.filter((m) => {
3208 |       const f = (m.fecha || '').toString().slice(0, 10);
3209 |       if (desde && f < desde) return false;
3210 |       if (hasta && f > hasta) return false;
3211 |       return true;
3212 |     });
3213 |   }
3214 |   return filtrados;
3215 | }
3216 |
3217 | function exportarMovimientosCajaExcel() {
3218 |   getListaMovimientosCajaParaSaldos().then((list) => {
3219 |     const filtrados = filtrarMovimientosCajaVista(list);
3220 |     if (filtrados.length === 0) {
3221 |       showToast('No hay movimientos para exportar con el filtro actual.', 'info');
3222 |       return;
3223 |     }
3224 |     const idsUsuario = filtrados.map((m) => m.usuario_id).filter(Boolean);
3225 |     return fetchMapaEmailUsuarioPorIds(idsUsuario).then((emailPorUsuario) => {
3226 |       const origenLabel = (m) => {
3227 |         if (m.tipo_movimiento_id) return 'Manual';
3228 |         if (m.transaccion_id) return 'Acuerdo';
3229 |         if (m.orden_id) return 'Orden concertada';
3230 |         return '-';
3231 |       };
3232 |       const tipoIngresoEgreso = (m) => (Number(m.monto) >= 0 ? 'Ingreso' : 'Egreso');
3233 |       const cajaTipoLabel = (m) => {
3234 |         const t = (m.caja_tipo || 'efectivo').toLowerCase();
3235 |         if (t === 'banco') return 'Banco';
3236 |         if (t === 'cheque') return 'Cheque';
3237 |         return 'Efectivo';
3238 |       };
3239 |       const header = ['Fecha', 'Origen', 'Nro orden', 'Nro Trx', 'Tipo', 'Moneda', 'Monto', 'Caja', 'Concepto', 'Usuario (registro)'];
3240 |       const rows = filtrados.map((m) => {
3241 |         const nroOrden = m.orden_numero != null ? Number(m.orden_numero) : null;
3242 |         const nroTrans = m.transaccion_numero != null ? Number(m.transaccion_numero) : null;
3243 |         const fecha = (m.fecha || '').toString().slice(0, 10);
3244 |         const uid = m.usuario_id != null ? String(m.usuario_id) : '';
3245 |         const mailReg = uid ? (emailPorUsuario[uid] || '') : '';
3246 |         return [
3247 |           fecha,
3248 |           origenLabel(m),
3249 |           nroOrden,
3250 |           nroTrans,
3251 |           tipoIngresoEgreso(m),

```

```

3252 |         (m.moneda || '').toString(),
3253 |         m.monto !== null ? Number(m.monto) : null,
3254 |         cajaTipoLabel(m),
3255 |         (m.concepto || '').toString(),
3256 |         mailReg || '- ',
3257 |     ];
3258 | });
3259 | const aoa = aoaExcelConMetaExportacion(header, rows);
3260 | const ws = XLSX.utils.aoa_to_sheet(aoa);
3261 | const wb = XLSX.utils.book_new();
3262 | XLSX.utils.book_append_sheet(wb, ws, 'Movimientos caja');
3263 | const nombreArchivo = `caja movimientos_${fechaHoyYYYYMMDDArgentina()} + '.xlsx';
3264 | XLSX.writeFile(wb, nombreArchivo);
3265 | showToast('Exportado: ' + nombreArchivo, 'success');
3266 | });
3267 | });
3268 | }
3269 |
3270 | /**
3271 |  * Pinta cards + tabla de movimientos desde lista en memoria (sin sync ni fetch).
3272 |  * @param {object} ctx - omitirDelayYLoading: true al solo cambiar filtros (sin spinner ni espera).
3273 |  */
3274 | function pintarCajasMovimientosUi(list, resTipos, monUsd, monArs, monEur, ctx) {
3275 |     const { loadingEl, wrapEl, tbody, loadingShownAtCajas, omitirDelayYLoading } = ctx;
3276 |     const waitIntro = omitirDelayYLoading ? Promise.resolve() : delayMinLoadingSiNoEsBackground(loadingShownAtCajas);
3277 |     return waitIntro.then(() => {
3278 |         loadingEl.style.display = 'none';
3279 |         const efRaw = { USD: !!monUsd, ARS: !!monArs, EUR: !!monEur };
3280 |         const baRaw = { USD: !!monUsd, ARS: !!monArs };
3281 |         setVisibilidadColumnasMonedasPanelCajas(efRaw, baRaw);
3282 |         const efVis = normalizarFlagsCajaMonedas(efRaw, MONEDAS_PANEL_CAJA_EFECTIVO);
3283 |         const baVis = normalizarFlagsCajaMonedas(baRaw, MONEDAS_PANEL_CAJA_BANCO);
3284 |         const saldos = saldosCajaDesdeLista(list);
3285 |         const tiposMap = {};
3286 |         (resTipos.data || []).forEach((t) => { tiposMap[t.id] = t.nombre || '-'; });
3287 |         const setVal = (el, valor, moneda) => {
3288 |             if (!el) return;
3289 |             el.textContent = formatMonto(valor, moneda);
3290 |             const base = el.id && el.id.startsWith('cajas-saldo-') ? 'inicio-caja-valor valor ' : 'valor ';
3291 |             el.className = base + (valor >= 0 ? 'positivo' : 'negativo');
3292 |         };
3293 |         if (efVis.USD) setVal(document.getElementById('cajas-saldo-efectivo-usd'), saldos.efectivo.USD, 'USD');
3294 |         if (efVis.ARS) setVal(document.getElementById('cajas-saldo-efectivo-ars'), saldos.efectivo.ARS, 'ARS');
3295 |         if (efVis.EUR) setVal(document.getElementById('cajas-saldo-efectivo-eur'), saldos.efectivo.EUR, 'EUR');
3296 |         if (baVis.USD) setVal(document.getElementById('cajas-saldo-banco-usd'), saldos.banco.USD, 'USD');
3297 |         if (baVis.ARS) setVal(document.getElementById('cajas-saldo-banco-ars'), saldos.banco.ARS, 'ARS');
3298 |         setVal(document.getElementById('cajas-saldo-cheque-ars'), saldos.cheque.ARS, 'ARS');
3299 |
3300 |         if (!cajasMovMostrarTodoHistorial && !cajasMovFechaDesde && !cajasMovFechaHasta) {
3301 |             const hm = fechaHoyYYYYMMDDArgentina();
3302 |             cajasMovFechaDesde = hm;
3303 |             cajasMovFechaHasta = hm;
3304 |         }
3305 |         syncCajasMovFechaInputs();
3306 |         document.querySelectorAll('[data-caja-tab]').forEach((b) => {
3307 |             const v = (b.getAttribute('data-caja-tab') || 'todo').toLowerCase();
3308 |             const on = cajasMovCajaTipoTab === 'todo' ? v === 'todo' : v === cajasMovCajaTipoTab;
3309 |             b.classList.toggle('activo', on);
3310 |             b.setAttribute('aria-pressed', on ? 'true' : 'false');
3311 |         });
3312 |
3313 |         const filtrados = filtrarMovimientosCajaVista(list);
3314 |         const origenLabel = (m) => {
3315 |             if (m.tipo_movimiento_id) return 'Manual';
3316 |             if (m.transaccion_id) return 'Acuerdo';
3317 |             if (m.orden_id) return 'Orden concertada';
3318 |             return '-';
3319 |         };
3320 |         const tipoIngresoEgreso = (m) => (Number(m.monto) >= 0 ? 'Ingreso' : 'Egreso');
3321 |         const cajaTipoLabel = (m) => {
3322 |             const t = (m.caja_tipo || 'efectivo').toLowerCase();
3323 |             if (t === 'banco') return 'Banco';
3324 |             if (t === 'cheque') return 'Cheque';
3325 |             return 'Efectivo';
3326 |         };
3327 |         const canAbmCaja = userPermissions.includes('abm_movimientos_caja');
3328 |         tbody.innerHTML = filtrados
3329 |             .map(
3330 |                 (m) =>
3331 |                 <tr>
3332 |                     <td class="td-caja-fecha">${(m.fecha || '').toString().slice(0, 10)}</td>
3333 |                     <td class="td-caja-origen">${escapeHtml(origenLabel(m))}</td>
3334 |                     <td class="td-caja-nro">${m.orden_numero !== null ? escapeHtml(String(m.orden_numero)) : '-'}</td>
3335 |                     <td class="td-caja-nro">${m.transaccion_numero !== null ? escapeHtml(String(m.transaccion_numero)) : '-'}</td>
3336 |                     <td class="td-caja-tipo-mov">${tipoIngresoEgreso(m)}</td>
3337 |                     <td class="td-orden-monedas">${htmlIconoMonedaCeldaOrden(m.moneda)}</td>
3338 |                     <td class="td-caja-monto">${Number(m.monto) >= 0 ? 'monto-positivo' : 'monto-negativo'}>${formatMonto(m.monto)}</td>
3339 |                     <td class="td-caja-caja-tipo">${cajaTipoLabel(m)}</td>
3340 |                     <td class="concepto-mov-caja">${escapeHtml((m.concepto || '-').slice(0, 80))}${(m.concepto && m.concepto.length > 80) ? '...' : ''}</td>
3341 |                     <td>${canAbmCaja ? <button type="button" class="btn-editar btn-editar-mov-caja" data-id="${m.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M11 4H4a2 2 0 0-2 2v14a2 2 0 02 2h14a2 2 0 02-2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013 3l12 15-4 1 1-4 9.5-9.5z"/></svg></span>Editar</button> : '}</td>
3342 |                 </tr>
3343 |             )
3344 |             .join('');
3345 |         if (filtrados.length === 0) tbody.innerHTML = `<tr><td colspan="10"> + (cajasMonedaActual === 'TODO' ? 'No hay movimientos.' : 'No hay movimientos en esta moneda.') + '</td></tr>`;

```

```

3346|     else {
3347|         tbody.querySelectorAll('.btn-editar-mov-caja').forEach((btn) => {
3348|             btn.addEventListener('click', () => {
3349|                 const id = btn.getAttribute('data-id');
3350|                 const mov = list.find((x) => String(x.id) === String(id));
3351|                 if (mov) openModalMovimientoCaja(mov);
3352|             });
3353|         });
3354|     }
3355|     wrapEl.style.display = 'block';
3356| });
3357| }
3358|
3359| /** Tras terminar sync global CC/caja: si el usuario sigue en Cajas, una sola refetch + pintada (sin volver a encadenar sync). */
3360| function refrescarVistaCajasTrasSyncGlobal() {
3361|     if (currentVistaId !== 'vista-cajas') return;
3362|     if (document.getElementById('modal-orden-backdrop')?.classList?.contains('activo')) return;
3363|     const loadingEl = document.getElementById('cajas-loading');
3364|     const wrapEl = document.getElementById('cajas-tabla-wrap');
3365|     const tbody = document.getElementById('movimientos-caja-tbody');
3366|     if (!loadingEl || !wrapEl || !tbody) return;
3367|     const prevBg = pandiBackgroundRefreshActive;
3368|     pandiBackgroundRefreshActive = true;
3369|     Promise.all([
3370|         fetchMovimientosCajaCerradosSinSync(),
3371|         client.from('tipos_movimiento_caja').select('id, nombre'),
3372|         hayTipoOperacionActivoConMoneda('USD'),
3373|         hayTipoOperacionActivoConMoneda('ARS'),
3374|         hayTipoOperacionActivoConMoneda('EUR'),
3375|     ])
3376|     .then(([list, resTipos, monUsd, monArs, monEur]) => {
3377|         cajasMovimientosFullCache = { list, resTipos, monUsd, monArs, monEur };
3378|         return pintarCajasMovimientosUi(list, resTipos, monUsd, monArs, monEur, {
3379|             loadingEl,
3380|             wrapEl,
3381|             tbody,
3382|             loadingShownAtCajas: 0,
3383|             omitirDelayYLoading: true,
3384|         });
3385|     })
3386|     .catch(() => {})
3387|     .finally(() => { pandiBackgroundRefreshActive = prevBg; });
3388| }
3389|
3390| /**
3391|  * @param {{ soloFiltros?: boolean }} opts - soloFiltros: moneda/fecha/tabs; repinta desde caché sin sync masivo ni SELECT (como filtros en memoria en PortfolioDetail).
3392|  */
3393| function loadCajas(opts) {
3394|     opts = opts || {};
3395|     const soloFiltros = opts.soloFiltros === true;
3396|     syncCajasPaneles();
3397|     const loadingEl = document.getElementById('cajas-loading');
3398|     const wrapEl = document.getElementById('cajas-tabla-wrap');
3399|     const tbody = document.getElementById('movimientos-caja-tbody');
3400|     const btnNuevo = document.getElementById('btn-nuevo-movimiento-caja');
3401|     const toggleMoneda = document.getElementById('cajas-toggle-moneda');
3402|     if (!loadingEl || !wrapEl || !tbody) return Promise.resolve();
3403|
3404|     const canAbm = userPermissions.includes('abm_movimientos_caja');
3405|     if (btnNuevo) btnNuevo.style.display = canAbm ? '' : 'none';
3406|
3407|     const verEfectivo = userPermissions.includes('ver_cajas_efectivo');
3408|     const verBanco = userPermissions.includes('ver_cajas_banco');
3409|     const verCheque = userPermissions.includes('ver_cajas_cheque');
3410|     const cardEfectivo = document.getElementById('cajas-card-efectivo');
3411|     const cardBanco = document.getElementById('cajas-card-banco');
3412|     const cardCheque = document.getElementById('cajas-card-cheque');
3413|     if (cardEfectivo) cardEfectivo.style.display = verEfectivo ? '' : 'none';
3414|     if (cardBanco) cardBanco.style.display = verBanco ? '' : 'none';
3415|     if (cardCheque) cardCheque.style.display = verCheque ? '' : 'none';
3416|
3417|     normalizarCajasMovCajaTipoTabSegunPermisos();
3418|     aplicarVisibilidadBotonesFiltroCajaMovimientos();
3419|
3420|     if (soloFiltros && cajasMovimientosFullCache) {
3421|         const c = cajasMovimientosFullCache;
3422|         if (cajasVistaSolapa === 'tipos') {
3423|             loadTiposMovimientoCajaTable();
3424|         }
3425|         return pintarCajasMovimientosUi(c.list, c.resTipos, c.monUsd, c.monArs, c.monEur, {
3426|             loadingEl,
3427|             wrapEl,
3428|             tbody,
3429|             loadingShownAtCajas: 0,
3430|             omitirDelayYLoading: true,
3431|         }).catch(() => {});
3432|     }
3433|
3434|     const silentCajas = isPandiBackgroundRefresh();
3435|     const loadingShownAtCajas = silentCajas ? 0 : Date.now();
3436|     if (!silentCajas) {
3437|         loadingEl.style.display = 'block';
3438|         wrapEl.style.display = 'none';
3439|         const cajasSaldoIds = ['cajas-saldo-efectivo-usd', 'cajas-saldo-efectivo-eur', 'cajas-saldo-efectivo-ars', 'cajas-saldo-banco-usd', 'cajas-saldo-banco-ars', 'cajas-saldo-c
3440|         heque-ars'];
3441|         cajasSaldoIds.forEach((id) => { const el = document.getElementById(id); if (el) el.textContent = '-'; });

```

```

3442 |
3443 |     sincronizarCcYCajaParaTodasLasOrdenesConInstrumentacion()
3444 |     .catch(() => {})
3445 |     .then(() => refrescarVistaCajasTrasSyncGlobal());
3446 |
3447 |     const promCajas = Promise.all([
3448 |         fetchMovimientosCajaCerradosSinSync(),
3449 |         client.from('tipos_movimiento_caja').select('id, nombre'),
3450 |         hayTipoOperacionActivoConMoneda('USD'),
3451 |         hayTipoOperacionActivoConMoneda('ARS'),
3452 |         hayTipoOperacionActivoConMoneda('EUR'),
3453 |     ])
3454 |     .then(([list, resTipos, monUsd, monArs, monEur]) => {
3455 |         cajasMovimientosFullCache = { list, resTipos, monUsd, monArs, monEur };
3456 |         return pintarCajasMovimientosUi(list, resTipos, monUsd, monArs, monEur, {
3457 |             loadingEl,
3458 |             wrapEl,
3459 |             tbody,
3460 |             loadingShownAtCajas,
3461 |             omitirDelayYLoading: false,
3462 |         });
3463 |     })
3464 |     .catch(() => {
3465 |         cajasMovimientosFullCache = null;
3466 |         loadingEl.style.display = 'none';
3467 |         if (!silentCajas) wrapEl.style.display = 'block';
3468 |     });
3469 |
3470 |     if (cajasVistaSolapa === 'tipos') {
3471 |         loadTiposMovimientoCajaTable();
3472 |     }
3473 |     return promCajas;
3474 | }
3475 |
3476 | /** Solo tarjetas de caja del panel Inicio (sin pendientes). Misma lista/cálculo que vista Cajas. */
3477 | function aplicarInicioTarjetasCajaDesdeLista(list, monUsd, monArs, monEur) {
3478 |     const hoy = new Date();
3479 |     const ayer = new Date(hoy);
3480 |     ayer.setDate(ayer.getDate() - 1);
3481 |     const ayerStr = ayer.getFullYear() + '-' + String(ayer.getMonth() + 1).padStart(2, '0') + '-' + String(ayer.getDate()).padStart(2, '0');
3482 |     const efRaw = { USD: !!monUsd, ARS: !!monArs, EUR: !!monEur };
3483 |     const baRaw = { USD: !!monUsd, ARS: !!monArs };
3484 |     setVisibilidadColumnasMonedasPanelCajas(efRaw, baRaw);
3485 |     const efVis = normalizarFlagsCajaMonedas(efRaw, MONEDAS_PANEL_CAJA_EFECTIVO);
3486 |     const baVis = normalizarFlagsCajaMonedas(baRaw, MONEDAS_PANEL_CAJA_BANCO);
3487 |     const saldoActual = saldosCajaDesdeLista(list);
3488 |     const saldoTl = { efectivo: { USD: 0, EUR: 0, ARS: 0 }, banco: { USD: 0, EUR: 0, ARS: 0 }, cheque: { USD: 0, EUR: 0, ARS: 0 } };
3489 |     (list || []).forEach((m) => {
3490 |         const tipoRaw = (m.caja.tipo || 'efectivo').toLowerCase();
3491 |         const t = tipoRaw === 'cheque' ? 'cheque' : tipoRaw === 'efectivo' || tipoRaw === 'banco' ? tipoRaw : 'efectivo';
3492 |         const moneda = (m.moneda || '').toUpperCase();
3493 |         if (!saldoTl[t] || saldoTl[t][moneda] == null) return;
3494 |         const fecha = (m.fecha || '').toString().slice(0, 10);
3495 |         if (fecha && fecha <= ayerStr) saldoTl[t][moneda] += Number(m.monto);
3496 |     });
3497 |     const svgSube = '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><polyline points="18 15 12 9 6 15'
3498 |     const svgBaja = '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><polyline points="6 9 12 15 18 9'
3499 |     const svgIgual = '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"><line x1="5" y1="12" x2="19" y2="12"/></svg>';
3500 |     const formatInicio = (n) => (n == null || isNaN(n)) ? '-' : Math.round(Number(n)).toLocaleString('es-AR', { maximumFractionDigits: 0, minimumFractionDigits: 0 });
3501 |     const setFila = (caja, moneda) => {
3502 |         const s1 = saldoTl[caja][moneda] ?? 0;
3503 |         const sT = saldoActual[caja][moneda] ?? 0;
3504 |         const variacion = sT - s1;
3505 |         const pct = s1 !== 0 ? (variacion / Math.abs(s1)) * 100 : (variacion !== 0 ? 100 : 0);
3506 |         const elInicial = document.getElementById('inicio-${caja}-${moneda.toLowerCase()}-inicial');
3507 |         const elActual = document.getElementById('inicio-${caja}-${moneda.toLowerCase()}-actual');
3508 |         const elVar = document.getElementById('inicio-${caja}-${moneda.toLowerCase()}-var');
3509 |         const elTend = document.getElementById('inicio-${caja}-${moneda.toLowerCase()}-tendencia');
3510 |         if (elInicial) {
3511 |             elInicial.textContent = formatInicio(s1);
3512 |             elInicial.className = 'inicio-caja-valor' + (s1 >= 0 ? 'positivo' : 'negativo');
3513 |         }
3514 |         if (elActual) {
3515 |             elActual.textContent = formatInicio(sT);
3516 |             elActual.className = 'inicio-caja-valor' + (sT >= 0 ? 'positivo' : 'negativo');
3517 |         }
3518 |         if (elVar) {
3519 |             const signo = variacion > 0 ? '+' : '';
3520 |             elVar.textContent = `${signo}${formatInicio(variacion)} (${variacion >= 0 ? '+' : ''}${Math.round(pct)}%)`;
3521 |             elVar.className = 'inicio-caja-var-valor' + (variacion > 0 ? 'sube' : variacion < 0 ? 'baja' : 'igual');
3522 |         }
3523 |         if (elTend) {
3524 |             elTend.className = 'inicio-caja-tendencia' + (variacion > 0 ? 'tendencia-sube' : variacion < 0 ? 'tendencia-baja' : 'tendencia-igual');
3525 |             elTend.innerHTML = variacion > 0 ? svgSube : variacion < 0 ? svgBaja : svgIgual;
3526 |         }
3527 |     };
3528 |     MONEDAS_PANEL_CAJA_EFECTIVO.filter((mon) => efVis[mon]).forEach((mon) => setFila('efectivo', mon));
3529 |     MONEDAS_PANEL_CAJA_BANCO.filter((mon) => baVis[mon]).forEach((mon) => setFila('banco', mon));
3530 |     setFila('cheque', 'ARS');
3531 | }
3532 |
3533 | function refrescarPanelInicioCajasTrasSyncGlobal() {
3534 |     if (currentVistaId !== 'vista-inicio') return;
3535 |     Promise.all([
3536 |         fetchMovimientosCajaCerradosSinSync(),

```

```

3537|     hayTipoOperacionActivoConMoneda('USD'),
3538|     hayTipoOperacionActivoConMoneda('ARS'),
3539|     hayTipoOperacionActivoConMoneda('EUR'),
3540|   ])
3541|   .then(([list, monUsd, monArs, monEur]) => { aplicarInicioTarjetasCajaDesdeLista(list, monUsd, monArs, monEur); })
3542|   .catch(() => {});
3543|   // G/P: se carga en loadInicio (Promise.all) para no duplicar RPC ni vaciar la matriz dos veces tras el sync global.
3544| }
3545|
3546| function setupInicioGpOperativo() {
3547|   if (inicioGpOperativoListenersAttached) return;
3548|   const wrap = document.getElementById('inicio-card-gp-operativo');
3549|   if (!wrap) return;
3550|   wrap.querySelectorAll('[data-inicio-gp-periodo]').forEach(btn => {
3551|     btn.addEventListener('click', () => {
3552|       const p = btn.getAttribute('data-inicio-gp-periodo');
3553|       if (!p || p === inicioGpOperativoPeriodo) return;
3554|       inicioGpOperativoPeriodo = p;
3555|       wrap.querySelectorAll('[data-inicio-gp-periodo]').forEach((b) => {
3556|         const on = b.getAttribute('data-inicio-gp-periodo') === p;
3557|         b.classList.toggle('activo', on);
3558|         b.setAttribute('aria-selected', on ? 'true' : 'false');
3559|       });
3560|       loadInicioGpOperativo();
3561|     });
3562|   });
3563|   inicioGpOperativoListenersAttached = true;
3564| }
3565|
3566| function inicioGpSumarCuatroBolsas(cajaMan, caja0rd, ccC, ccI) {
3567|   const monedas = ['USD', 'ARS', 'EUR'];
3568|   const tot = {};
3569|   monedas.forEach((m) => {
3570|     const a = Number(cajaMan[m] != null ? cajaMan[m] : 0);
3571|     const o = Number(caja0rd[m] != null ? caja0rd[m] : 0);
3572|     const b = Number(ccC[m] != null ? ccC[m] : 0);
3573|     const c = Number(ccI[m] != null ? ccI[m] : 0);
3574|     tot[m] = a + o + b + c;
3575|   });
3576|   return tot;
3577| }
3578|
3579| const MONEDAS_GP_PANEL = ['USD', 'ARS', 'EUR'];
3580|
3581| function inicioGpClaseSigno(num) {
3582|   if (num > 0) return 'positivo';
3583|   if (num < 0) return 'negativo';
3584|   return '';
3585| }
3586|
3587| function pintarInicioGpMatriz(elMatriz, cajaMan, caja0rd, ccC, ccI) {
3588|   if (!elMatriz) return;
3589|   const monedas = MONEDAS_GP_PANEL;
3590|   const tot = inicioGpSumarCuatroBolsas(cajaMan, caja0rd, ccC, ccI);
3591|   function numBolsa(bolsa, mon) {
3592|     const v = bolsa[mon];
3593|     return v != null && !Number.isNaN(Number(v)) ? Number(v) : 0;
3594|   }
3595|   function celNum(bolsa, mon, esTotal) {
3596|     const num = numBolsa(bolsa, mon);
3597|     const cls = inicioGpClaseSigno(num);
3598|     const sizeCls = esTotal ? 'inicio-gp-matriz-num-total' : 'inicio-gp-matriz-num-sub';
3599|     return `<div class="inicio-gp-matriz-num ${sizeCls} ${cls}">${escapeHtml(formatImportedDisplay(num))}</div>`;
3600|   }
3601|   const helpIconSvg =
3602|     `<svg viewBox="0 0 24 24" fill="currentColor" aria-hidden="true"><path d="M12 2C6.48 2 2 6.48 2 12s4.48 10 10 10 4.48 10 10s17.52 2 12 2zm1 15h-2v-6h2v6zm0-8h-2V7h2v2z"
3603|   /></svg>`;
3604|   const headers = monedas
3605|     .map(mon) => {
3606|       const src = URL_ICONO_MONEDA_ASSETS[mon] || '';
3607|       const img = src
3608|         ? ``
3609|         : '';
3610|       return `<div class="inicio-gp-matriz-h">${img}<span>${escapeHtml(mon)}</span></div>`;
3611|     }
3612|     .join('');
3613|   const rowTotal =
3614|     `<div class="inicio-gp-matriz-label-total">Total</div>` + monedas.map((m) => celNum(tot, m, true)).join('');
3615|   const rowCaja =
3616|     `<div class="inicio-gp-matriz-label-sub inicio-gp-matriz-label-caja-manual">Movimientos de caja manuales<span class="help-inline"><button type="button" class="help-icon-bt
3617| n" aria-label="Ayuda: movimientos de caja manuales en G/P Operativa">` +
3618|     helpIconSvg +
3619|     `</button><span class="help-popover"><strong>Movimientos de caja manuales</strong> en esta fila: suma solo movimientos de caja <strong>sin orden asociada</strong> cuyo tip
3620| o tiene activo <strong>incluye en G/P</strong> en <strong>Cajas - Tipos</strong>. Configurá ahí qué tipos entran en G/P Operativa.</span></div>` +
3621|     monedas.map(m) => celNum(cajaMan, m, false)).join('');
3622|   const rowCaja0rd =
3623|     `<div class="inicio-gp-matriz-label-sub inicio-gp-matriz-label-caja-ordenes">Movimientos de caja por órdenes<span class="help-inline"><button type="button" class="help-ico
3624| n-btn" aria-label="Ayuda: movimientos de caja por órdenes en G/P Operativa">` +
3625|     helpIconSvg +
3626|     `</button><span class="help-popover"><strong>Movimientos de caja por órdenes</strong>: suma movimientos de caja con <strong>orden asociada</strong> y estado cerrado (al <s
3627| trong>ejecutar</strong> transacciones: efectivo, banco o cheque según el modo de pago). Ahí aparece el resultado neto en valores de <span class="js-marca-sistema-nombre">` +
3628|     escapeHtml(nombreMarcaSistema()) +
3629|     `</span> cuando la orden cierra (p. ej. diferencia tras cliente e intermediario). Va aparte de los movimientos de caja manuales filtrados por tipo.</span></div>` +
3630|     monedas.map(m) => celNum(caja0rd, m, false)).join('');
3631|   const rowCli =
3632|     `<div class="inicio-gp-matriz-label-sub">Cuenta Corriente Clientes</div>` + monedas.map(m) => celNum(ccC, m, false)).join('');
3633|   const rowInt =

```

```

3629 |     '<div class="inicio-gp-matriz-label-sub">Cuenta Corriente Intermediarios</div>' + monedas.map((m) => celNum(ccI, m, false)).join('');
3630 |     elMatriz.innerHTML =
3631 |     '<div class="inicio-gp-matriz-spacer" aria-hidden="true"></div>' +
3632 |     headers +
3633 |     rowTotal +
3634 |     rowCaja +
3635 |     rowCajaOrd +
3636 |     rowCli +
3637 |     rowInt;
3638 |     elMatriz.style.display = 'grid';
3639 | }
3640 |
3641 | function loadInicioGpOperativo() {
3642 |     const wrap = document.getElementById('inicio-card-gp-operativo');
3643 |     if (!wrap) return;
3644 |     if (!userPermissions.includes('ver_inicio_gp_operativo')) {
3645 |         wrap.style.display = 'none';
3646 |         return;
3647 |     }
3648 |     wrap.style.display = '';
3649 |     setupInicioGpOperativo();
3650 |     const leyenda = document.getElementById('inicio-gp-leyenda-periodo');
3651 |     const matrizEl = document.getElementById('inicio-gp-matriz');
3652 |     const loading = document.getElementById('inicio-gp-loading');
3653 |     const rango = inicioGpOperativoRangoFechas(inicioGpOperativoPeriodo);
3654 |     if (leyenda) {
3655 |         if (inicioGpOperativoPeriodo === 'total') {
3656 |             leyenda.textContent = 'Periodo: todo el historial (desde siempre). Zona horaria: Argentina.';
3657 |         } else if (rango.desde && rango.hasta) {
3658 |             leyenda.textContent = `Periodo: ${rango.desde} al ${rango.hasta} (inclusive). Argentina.`;
3659 |         } else {
3660 |             leyenda.textContent = '';
3661 |         }
3662 |     }
3663 |     const silentGp = isPandiBackgroundRefresh();
3664 |     if (!silentGp) {
3665 |         if (loading) loading.style.display = 'block';
3666 |         if (matrizEl) {
3667 |             matrizEl.innerHTML = '';
3668 |             matrizEl.style.display = 'none';
3669 |         }
3670 |     } else if (loading) {
3671 |         loading.style.display = 'none';
3672 |     }
3673 |     wrap.querySelectorAll('[data-inicio-gp-periodo]').forEach((b) => {
3674 |         const on = b.getAttribute('data-inicio-gp-periodo') === inicioGpOperativoPeriodo;
3675 |         b.classList.toggle('activo', on);
3676 |         b.setAttribute('aria-selected', on ? 'true' : 'false');
3677 |     });
3678 |
3679 |     client
3680 |         .rpc('gp_operativa_resumen', { p_desde: rango.desde, p_hasta: rango.hasta })
3681 |         .then((res) => {
3682 |             if (loading) loading.style.display = 'none';
3683 |             if (res.error) {
3684 |                 if (!silentGp) {
3685 |                     showToast(
3686 |                         'G/P Operativa: ' +
3687 |                         (res.error.message || 'No se pudo calcular. Ejecutó sql/migracion_gp_operativa_panel.sql en Supabase.'),
3688 |                         'error'
3689 |                     );
3690 |                 }
3691 |                 if (matrizEl && !silentGp) {
3692 |                     matrizEl.innerHTML =
3693 |                     '<div class="inicio-gp-matriz-error">Sin datos (revisá migración SQL en Supabase).</div>';
3694 |                     matrizEl.style.display = 'block';
3695 |                 }
3696 |                 return;
3697 |             }
3698 |             let raw = res.data;
3699 |             if (typeof raw === 'string') {
3700 |                 try {
3701 |                     raw = JSON.parse(raw);
3702 |                 } catch (e) {
3703 |                     raw = {};
3704 |                 }
3705 |             }
3706 |             const data = raw && typeof raw === 'object' ? raw : {};
3707 |             function bolsaNum(obj) {
3708 |                 const o = typeof obj === 'object' && obj ? obj : {};
3709 |                 const out = {};
3710 |                 Object.keys(o).forEach((k) => {
3711 |                     const n = Number(o[k]);
3712 |                     out[k] = Number.isFinite(n) ? n : 0;
3713 |                 });
3714 |                 return out;
3715 |             }
3716 |             const cajaMan = bolsaNum(data.caja_manual);
3717 |             const cajaOrd = bolsaNum(data.caja_ordenes);
3718 |             const ccC = bolsaNum(data.cc_cliente);
3719 |             const ccI = bolsaNum(data.cc_intermediario);
3720 |             pintarInicioGpMatriz(matrizEl, cajaMan, cajaOrd, ccC, ccI);
3721 |         })
3722 |         .catch(() => {
3723 |             if (loading) loading.style.display = 'none';
3724 |             if (!silentGp) showToast('G/P Operativa: error de red o servidor.', 'error');
3725 |             if (matrizEl && !silentGp) {

```

```

3726 |         matrizEl.innerHTML = '<div class="inicio-gp-matriz-error">No se pudo cargar. Reintentá más tarde.</div>';
3727 |         matrizEl.style.display = 'block';
3728 |     });
3729 | });
3730 | }
3731 |
3732 | function loadInicio() {
3733 |     const elEfectivo = document.getElementById('inicio-card-efectivo');
3734 |     const elBanco = document.getElementById('inicio-card-banco');
3735 |     const elSalDOS = document.getElementById('inicio-saldos');
3736 |     const elPendientes = document.getElementById('inicio-cards-pendientes');
3737 |     const canIngresarOrden = userPermissions.includes('ingresar_orden');
3738 |     const btnChatInicio = document.getElementById('btn-orden-por-chat-inicio');
3739 |     if (btnChatInicio) btnChatInicio.style.display = canIngresarOrden ? '' : 'none';
3740 |
3741 |     const elCheque = document.getElementById('inicio-card-cheque');
3742 |     const hasEfectivoPerm = userPermissions.includes('ver_cajas_efectivo');
3743 |     const hasBancoPerm = userPermissions.includes('ver_cajas_banco');
3744 |     const hasChequePerm = userPermissions.includes('ver_cajas_cheque');
3745 |     const hasPendientesPerm = userPermissions.includes('ver_inicio_pendientes');
3746 |     const verCajasMenu = userPermissions.includes('ver_cajas');
3747 |     const algunaGranularCaja = hasEfectivoPerm || hasBancoPerm || hasChequePerm;
3748 |     const legacyTodasLasTarjetasCaja = verCajasMenu && !algunaGranularCaja;
3749 |     if (elEfectivo) elEfectivo.style.display = hasEfectivoPerm || legacyTodasLasTarjetasCaja ? '' : 'none';
3750 |     if (elBanco) elBanco.style.display = hasBancoPerm || legacyTodasLasTarjetasCaja ? '' : 'none';
3751 |     if (elCheque) elCheque.style.display = hasChequePerm || legacyTodasLasTarjetasCaja ? '' : 'none';
3752 |     if (elSalDOS) elSalDOS.style.display = algunaGranularCaja || legacyTodasLasTarjetasCaja ? '' : 'none';
3753 |     if (elPendientes) elPendientes.style.display = hasPendientesPerm ? '' : 'none';
3754 |
3755 |     // Refresco automático (~30 s): no encadenar sync global de todas las órdenes (pesado y vacía/repinta G/P).
3756 |     // Igual criterio que loadCuentaCorriente con isPandiBackgroundRefresh.
3757 |     if (!isPandiBackgroundRefresh()) {
3758 |         sincronizarCcYCajaParaTodasLasOrdenesConInstrumentacion()
3759 |             .catch(() => {})
3760 |             .then(() => refrescarPanelInicioCajasTrasSyncGlobal());
3761 |     } else {
3762 |         refrescarPanelInicioCajasTrasSyncGlobal();
3763 |     }
3764 |
3765 |     return Promise.all([
3766 |         fetchMovimientosCajaCerradosSinSync(),
3767 |         hayTipoOperacionActivoConMoneda('USD'),
3768 |         hayTipoOperacionActivoConMoneda('ARS'),
3769 |         hayTipoOperacionActivoConMoneda('EUR'),
3770 |     ])
3771 |         .then(([list, monUsd, monArs, monEur]) => {
3772 |             aplicarInicioTarjetasCajaDesdeLista(list, monUsd, monArs, monEur);
3773 |             loadInicioPendientes();
3774 |             loadInicioGpOperativo();
3775 |         })
3776 |         .catch(() => {});
3777 | }
3778 |
3779 | function loadInicioPendientes() {
3780 |     const bodyOrd = document.getElementById('inicio-ordenes-pendientes-body');
3781 |     const elCountTr = document.getElementById('inicio-count-transacciones-pendientes');
3782 |
3783 |     const estadoLabelOrd = (e) => ({ pendiente_instrumentar: 'Pend. Instrumentar', instrumentacion_parcial: 'Instrumentación Parcial', instrumentacion_cerrada_ejecucion: 'Cerrad
3784 | a en Ejecución', orden_ejecutada: 'Orden Ejecutada', anulada: 'Anulada' }[e] || (e ? String(e) : '-'));
3785 |     const estadoBadgeOrd = (e) => (e && ['pendiente_instrumentar', 'instrumentacion_parcial', 'instrumentacion_cerrada_ejecucion', 'orden_ejecutada', 'anulada'].includes(e) ? `b
3786 | adge-estado-${e.replace(/_/g, '-')}` : '');
3787 |
3788 |     const ordenEstados = ['pendiente_instrumentar', 'instrumentacion_parcial', 'instrumentacion_cerrada_ejecucion'];
3789 |     if (bodyOrd) {
3790 |         client.from('ordenes').select('id, estado').neq('estado', 'orden_ejecutada').neq('estado', 'anulada').then((r) => {
3791 |             const list = r.data || [];
3792 |             const byEstado = {};
3793 |             list.forEach((o) => { byEstado[o.estado] = (byEstado[o.estado] || 0) + 1; });
3794 |             const svgOjo = `<svg viewBox="0 0 24 24" fill="currentColor"><path d="M12 4.5C7 4.5 2.73 7.61 1 12c1.73 4.39 6 7.5 11 7.5s9.27-3.11 11-7.5c-1.73-4.39-6-7.5-11-7.5zM12 17
3795 | c-2.76 0-5.22-5-5.22-5.52 2.45 5 5 2.24 5 5.22 2.4 5.5 5.22 0 8c1.66 0 3 1.34 3 3s1.34 3 3 3-1.34 3-3 3z"/></svg>`;
3796 |             const filas = ordenEstados.filter((e) => (byEstado[e] || 0) > 0).map((estado) => {
3797 |                 const badgeClass = estadoBadgeOrd(estado);
3798 |                 const label = estadoLabelOrd(estado);
3799 |                 const num = byEstado[estado];
3800 |                 return `<div class="inicio-card-ordenes-fila" data-estado="${estado}">
3801 |                     <span class="inicio-card-ordenes-badge"><span class="${badgeClass}">${label}</span></span>
3802 |                     <span class="inicio-card-ordenes-num">${num}</span>
3803 |                     <button type="button" class="btn-ver-estado" data-estado="${estado}" title="Ver estas órdenes" aria-label="Ver estas órdenes">${svgOjo}</button>
3804 |                 </div>`;
3805 |             });
3806 |             bodyOrd.innerHTML = filas.length ? filas.join('') : `<div class="inicio-card-ordenes-fila"><span class="inicio-card-pendientes-valor" style="grid-column:1/-1;"></span><
3807 | /div>`;
3808 |             bodyOrd.querySelectorAll('.btn-ver-estado').forEach((btn) => {
3809 |                 btn.addEventListener('click', () => { openModalOrdenesPendientes(btn.getAttribute('data-estado')); });
3810 |             });
3811 |         });
3812 |         if (elCountTr) client.from('transacciones').select('id', { count: 'exact', head: true }).eq('estado', 'pendiente').then((r) => { elCountTr.textContent = r.count != null ? St
3813 | ring(r.count) : '-'; });
3814 |     }
3815 |
3816 |     /** Datos del modal de órdenes pendientes para filtrado y re-render. */
3817 |     let ordenesPendientesList = [];
3818 |     let ordenesPendientesClientesMap = {};
3819 |     let ordenesPendientesTiposOpMap = {};
3820 |     let ordenesPendientesIntermediariosMap = {};

```



```

3818 function renderOrdenesPendientesFiltros(list, clientesMap, intermediariosMap) {
3819   const selCliente = document.getElementById('ordenes-pendientes-filtro-cliente');
3820   const selIntermediario = document.getElementById('ordenes-pendientes-filtro-intermediario');
3821   if (!selCliente || !selIntermediario) return;
3822   const clienteIds = [...new Set(list.map(o => o.cliente_id).filter(Boolean))].sort((a, b) => (clientesMap[a] || '').localeCompare(clientesMap[b] || ''));
3823   const intIds = [...new Set(list.map(o => o.intermediario_id).filter(Boolean))].sort((a, b) => (intermediariosMap[a] || '').localeCompare(intermediariosMap[b] || ''));
3824   selCliente.innerHTML = '<option value="">Todos</option>' + clienteIds.map(id => '<option value="' + id + '>' + clienteId + '</option>').join('');
3825   selIntermediario.innerHTML = '<option value="">Todos</option>' + intIds.map(id => '<option value="' + id + '>' + intermediarioId + '</option>').join('');
3826 }
3827
3828 function renderOrdenesPendientesTabla() {
3829   const backdrop = document.getElementById('modal-ordenes-pendientes-backdrop');
3830   const tbody = document.getElementById('ordenes-pendientes-tbody');
3831   const selCliente = document.getElementById('ordenes-pendientes-filtro-cliente');
3832   const selIntermediario = document.getElementById('ordenes-pendientes-filtro-intermediario');
3833   const selEstado = document.getElementById('ordenes-pendientes-filtro-estado');
3834   if (!tbody) return;
3835   const clienteId = selCliente.value ? selCliente.value : '';
3836   const intermediarioId = selIntermediario.value ? selIntermediario.value : '';
3837   const estadoVal = selEstado.value ? selEstado.value : '';
3838   let list = ordenesPendientesList;
3839   if (clienteId) list = list.filter(o => o.cliente_id === clienteId);
3840   if (intermediarioId) list = list.filter(o => o.intermediario_id === intermediarioId);
3841   if (estadoVal) list = list.filter(o => o.estado === estadoVal);
3842   const canEditarOrden = userPermissions.includes('editar_orden');
3843   const canIngresarTransacciones = userPermissions.includes('ingresar_transacciones');
3844   const canEditarTransacciones = userPermissions.includes('editar_transacciones');
3845   const canEliminarTransacciones = userPermissions.includes('eliminar_transacciones');
3846   const canVerAccionesOrden = userPermissions.includes('editar_orden') || userPermissions.includes('anular_orden') || userPermissions.includes('editar_estado_orden') || canIngresarTransacciones || canEditarTransacciones || canEliminarTransacciones;
3847   const estadoLabel = (e) => ({ pendiente_instrumentar: 'Pendiente Instrumentar', instrumentacion_parcial: 'Instrumentación Parcial', instrumentacion_cerrada_ejecucion: 'Cerrada en Ejecución', orden_ejecutada: 'Orden Ejecutada', anulada: 'Anulada' })[e] || (e ? String(e) : '-');
3848   const estadoBadgeClass = (e) => (e && ['pendiente_instrumentar', 'instrumentacion_parcial', 'instrumentacion_cerrada_ejecucion', 'orden_ejecutada', 'anulada'].includes(e) ? 'badge-estado-' + e.replace(/./g, '-') : '');
3849   const clientesMap = ordenesPendientesClientesMap;
3850   const tiposOpMap = ordenesPendientesTiposOpMap;
3851   const intermediariosMap = ordenesPendientesIntermediariosMap;
3852   tbody.innerHTML = list.length ? list.map(o => {
3853     const estado = o.estado || '';
3854     const badgeClass = estadoBadgeClass(estado);
3855     const estadoHtml = badgeClass ? `<span class="${badgeClass}">${estadoLabel(estado)}</span>` : estadoLabel(estado);
3856     return `<tr data-id="${o.id}">
3857       <td>${(o.fecha || '').toString().slice(0, 10)}</td>
3858       <td class="td-tipo-op-iconos">${o.tipo_operacion_id ? htmlCeldaTipoOperacionDesdeMap(o.tipo_operacion_id, tiposOpMap) : htmlTipoOperacionIconos()}</td>
3859       <td>${escapeHtml(o.cliente_id ? clientesMap[o.cliente_id] || '-' : '-')}</td>
3860       <td>${escapeHtml(o.intermediario_id ? intermediariosMap[o.intermediario_id] || '-' : '-')}</td>
3861       <td>${estadoHtml}</td>
3862       <td class="td-orden-moneda">${formatMonto(o.monto_recibido)}</td>
3863       <td class="td-orden-moneda">${htmlIconoMonedaCeldaOrden(o.moneda_recibida)}</td>
3864       <td class="td-orden-moneda">${formatMonto(o.monto_entregado)}</td>
3865       <td class="td-orden-moneda">${htmlIconoMonedaCeldaOrden(o.moneda_entregada)}</td>
3866       <td>${canVerAccionesOrden ? `<button type="button" class="btn-editar-orden-pendiente" data-id="${o.id}" title="Editar"><span class="btn-icon">svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M11 4H4a2 2 0 0 2 2v14a2 2 0 0 2 2h14a2 2 0 0 2 2v-7"/></path>` : `<button type="button" class="btn-secondary btn-transacciones-pendiente" data-id="${o.id}" title="Transacciones"><span class="btn-icon">svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M14 2H6a2 2 0 0 2 2v16a2 2 0 0 2 2h12a2 2 0 0 2 2v8"/></path>`}</td>
3867     </tr>`;
3868   }).join('') : `<tr><td colspan="10">No hay órdenes que coincidan con los filtros.</td></tr>`;
3869   tbody.querySelectorAll('.btn-editar-orden-pendiente').forEach(btn => {
3870     btn.addEventListener('click', () => {
3871       const id = btn.getAttribute('data-id');
3872       const row = ordenesPendientesList.find(r => r.id === id);
3873       if (row && backdrop) { backdrop.classList.remove('activo'); openModalOrden(row); }
3874     });
3875   });
3876   tbody.querySelectorAll('.btn-transacciones-pendiente').forEach(btn => {
3877     btn.addEventListener('click', () => {
3878       const id = btn.getAttribute('data-id');
3879       const orden = ordenesPendientesList.find(r => r.id === id);
3880       if (!orden) return;
3881       if (backdrop) backdrop.classList.remove('activo');
3882       showView('vista-ordenes', 'ordenes');
3883       loadOrdenes().then(() => { expandOrdenTransacciones(id, orden); });
3884     });
3885   });
3886 }
3887
3888 /** Abre el modal de órdenes pendientes (estado ≠ orden_ejecutada). Si se pasa estadoFilter, solo se muestran órdenes de ese estado inicialmente y se preselecciona el filtro E
3889 stado. */
3889 function openModalOrdenesPendientes(estadoFilter) {
3890   const backdrop = document.getElementById('modal-ordenes-pendientes-backdrop');
3891   const loadingEl = document.getElementById('ordenes-pendientes-loading');
3892   const wrapEl = document.getElementById('ordenes-pendientes-tabla-wrap');
3893   const filtrosWrap = document.getElementById('ordenes-pendientes-filtros-wrap');
3894   const tbody = document.getElementById('ordenes-pendientes-tbody');
3895   if (!backdrop || !loadingEl || !wrapEl || !tbody) return;
3896   backdrop.classList.add('activo');
3897   loadingEl.style.display = 'block';
3898   if (filtrosWrap) filtrosWrap.style.display = 'none';
3899   wrapEl.style.display = 'none';
3900   tbody.innerHTML = '';
3901   const selEstado = document.getElementById('ordenes-pendientes-filtro-estado');
3902   if (selEstado) selEstado.value = estadoFilter || '';
3903   const selectOrdPend = ordenesTieneNumeroColumn ? 'id, numero, cliente_id, fecha, estado, tipo_operacion_id, operacion_directa, intermediario_id, moneda_recibida, moneda_entregada, monto_recibido, monto_entregado, cotizacion, observaciones' : 'id, cliente_id, fecha, estado, tipo_operacion_id, operacion_directa, intermediario_id, moneda_recibida, moneda_entregada, monto_recibido, monto_entregado, cotizacion, observaciones';
3904   client.from('ordenes').select(selectOrdPend).neq('estado', 'orden_ejecutada').neq('estado', 'anulada').order('fecha', { ascending: false }).order('created_at', { ascending:

```

```

false }).then((res) => {
3905   if (res.error) {
3906     loadingEl.style.display = 'none';
3907     tbody.innerHTML = '<tr><td colspan="10">Error: ' + (res.error.message || '') + '</td></tr>';
3908     wrapEl.style.display = 'block';
3909     return;
3910   }
3911   const list = res.data || [];
3912   if (list.length === 0) {
3913     loadingEl.style.display = 'none';
3914     tbody.innerHTML = '<tr><td colspan="10">No hay órdenes pendientes.</td></tr>';
3915     wrapEl.style.display = 'block';
3916     return;
3917   }
3918   const clienteIds = [...new Set(list.map((o) => o.cliente_id).filter(Boolean))];
3919   const tipoOpIds = [...new Set(list.map((o) => o.tipo_operacion_id).filter(Boolean))];
3920   const intIds = [...new Set(list.map((o) => o.intermediario_id).filter(Boolean))];
3921   return Promise.all([
3922     clienteIds.length ? client.from('clientes').select('id, nombre').in('id', clienteIds) : Promise.resolve({ data: [] }),
3923     tipoOpIds.length ? client.from('tipos_operacion').select('id, nombre, codigo, moneda_in, moneda_out, usa_intermediario, icono_moda, icono_url_publica').in('id', tipoOp
Ids) : Promise.resolve({ data: [] }),
    intIds.length ? client.from('intermediarios').select('id, nombre').in('id', intIds) : Promise.resolve({ data: [] }),
  ]).then(([cr, tr, ir]) => {
3926     const clientesMap = {};
3927     (cr.data || []).forEach((c) => { clientesMap[c.id] = c.nombre || ''; });
3928     const tiposOpMap = {};
3929     (tr.data || []).forEach((t) => {
3930       tiposOpMap[t.id] = {
3931         codigo: t.codigo || '',
3932         nombre: t.nombre || '',
3933         moneda_in: (t.moneda_in || '').toString().trim(),
3934         moneda_out: (t.moneda_out || '').toString().trim(),
3935         usa_intermediario: t.usa_intermediario === true,
3936         icono_moda: (t.icono_moda || 'auto').toString().trim().toLowerCase(),
3937         icono_url_publica: (t.icono_url_publica || '').toString().trim(),
3938       };
3939     });
3940     const intermediariosMap = {};
3941     (ir.data || []).forEach((i) => { intermediariosMap[i.id] = i.nombre || ''; });
3942     ordenesPendientesList = list;
3943     ordenesPendientesClientesMap = clientesMap;
3944     ordenesPendientesTiposOpMap = tiposOpMap;
3945     ordenesPendientesIntermediariosMap = intermediariosMap;
3946     renderOrdenesPendientesFiltros(list, clientesMap, intermediariosMap);
3947     if (filtrosWrap) filtrosWrap.style.display = 'flex';
3948     renderOrdenesPendientesTabla();
3949     loadingEl.style.display = 'none';
3950     wrapEl.style.display = 'block';
3951   });
3952   });
3953 }
3954
3955 function setupOrdenesPendientesFiltrosListeners() {
3956   const selCliente = document.getElementById('ordenes-pendientes-filtro-cliente');
3957   const selIntermediario = document.getElementById('ordenes-pendientes-filtro-intermediario');
3958   const selEstado = document.getElementById('ordenes-pendientes-filtro-estado');
3959   if (selCliente) selCliente.addEventListener('change', () => renderOrdenesPendientesTabla());
3960   if (selIntermediario) selIntermediario.addEventListener('change', () => renderOrdenesPendientesTabla());
3961   if (selEstado) selEstado.addEventListener('change', () => renderOrdenesPendientesTabla());
3962 }
3963
3964 /** Lista de transacciones pendientes con filtros (cliente, intermediario, solo Pandý). Guarda en ventana para filtrado. */
3965 let transaccionesPendientesList = [];
3966 let transaccionesPendientesOrdenesMap = {};
3967 let transaccionesPendientesClientesMap = {};
3968 let transaccionesPendientesIntermediariosMap = {};
3969
3970 function openModalTransaccionesPendientes() {
3971   const backdrop = document.getElementById('modal-transacciones-pendientes-backdrop');
3972   const loadingEl = document.getElementById('transacciones-pendientes-loading');
3973   const wrapEl = document.getElementById('transacciones-pendientes-tabla-wrap');
3974   const tbody = document.getElementById('transacciones-pendientes-tbody');
3975   const selCliente = document.getElementById('transacciones-pendientes-filtro-cliente');
3976   const selIntermediario = document.getElementById('transacciones-pendientes-filtro-intermediario');
3977   const chkPandy = document.getElementById('transacciones-pendientes-filtro-pandy');
3978   if (!backdrop || !loadingEl || !wrapEl || !tbody) return;
3979   backdrop.classList.add('activo');
3980   loadingEl.style.display = 'block';
3981   wrapEl.style.display = 'none';
3982   tbody.innerHTML = '';
3983   client.from('transacciones').select('id, tipo, moneda, monto, cobrador, pagador, estado, concepto, tipo_cambio, modo_pago_id, instrumentacion_id, pagador_cliente_id, cobrado
r_cliente_id, pagador_intermediario_id, cobrador_intermediario_id').eq('estado', 'pendiente').order('created_at', { ascending: false }).then((rTr) => {
3984     if (rTr.error) {
3985       loadingEl.style.display = 'none';
3986       tbody.innerHTML = '<tr><td colspan="10">Error: ' + (rTr.error.message || '') + '</td></tr>';
3987       wrapEl.style.display = 'block';
3988       return;
3989     }
3990     const transacciones = rTr.data || [];
3991     if (transacciones.length === 0) {
3992       loadingEl.style.display = 'none';
3993       tbody.innerHTML = '<tr><td colspan="10">No hay transacciones pendientes.</td></tr>';
3994       wrapEl.style.display = 'block';
3995       renderTransaccionesPendientesFiltros([], {}, {});
3996       return;
3997     }
3998     const instIds = [...new Set(transacciones.map((t) => t.instrumentacion_id).filter(Boolean))];

```

```

3999 client.from('instrumentacion').select('id, orden_id').in('id', instIds).then((rInst) => {
4000   const instToOrden = {};
4001   (rInst.data || []).forEach((i) => { instToOrden[i.id] = i.orden_id; });
4002   const ordenIds = [...new Set(Object.values(instToOrden).filter(Boolean))];
4003   const selectOrdTr = ordenesTieneNumeroColumn ? 'id, numero, cliente_id, intermediario_id, fecha' : 'id, cliente_id, intermediario_id, fecha';
4004   client.from('ordenes').select(selectOrdTr).in('id', ordenIds).then((rOrd) => {
4005     const ordenesMap = {};
4006     (rOrd.data || []).forEach((o) => { ordenesMap[o.id] = o; });
4007     transaccionesPendientesOrdenesMap = ordenesMap;
4008     const clienteIds = new Set((rOrd.data || []).map((o) => o.cliente_id).filter(Boolean));
4009     const intIds = new Set((rOrd.data || []).map((o) => o.intermediario_id).filter(Boolean));
4010     transacciones.forEach((t) => {
4011       if (t.pagador_cliente_id) clienteIds.add(t.pagador_cliente_id);
4012       if (t.cobrador_cliente_id) clienteIds.add(t.cobrador_cliente_id);
4013       if (t.pagador_intermediario_id) intIds.add(t.pagador_intermediario_id);
4014       if (t.cobrador_intermediario_id) intIds.add(t.cobrador_intermediario_id);
4015     });
4016     const clienteIdsArr = [...clienteIds];
4017     const intIdsArr = [...intIds];
4018     Promise.all([
4019       clienteIdsArr.length ? client.from('clientes').select('id, nombre').in('id', clienteIdsArr) : Promise.resolve({ data: [] }),
4020       intIdsArr.length ? client.from('intermediarios').select('id, nombre').in('id', intIdsArr) : Promise.resolve({ data: [] })
4021     ]).then(([rC, rI]) => {
4022       const clientesMap = {};
4023       (rC.data || []).forEach((c) => { clientesMap[c.id] = c.nombre || ''; });
4024       const intermediariosMap = {};
4025       (rI.data || []).forEach((i) => { intermediariosMap[i.id] = i.nombre || ''; });
4026       transaccionesPendientesClientesMap = clientesMap;
4027       transaccionesPendientesIntermediariosMap = intermediariosMap;
4028       transaccionesPendientesList = transacciones.map((t) => {
4029         const ordenId = instToOrden[t.instrumentacion_id];
4030         const orden = ordenesMap[ordenId];
4031         return { ...t, orden_id: ordenId, cliente_id: orden && orden.cliente_id, intermediario_id: orden && orden.intermediario_id, orden_fecha: orden && orden.fecha };
4032       });
4033       renderTransaccionesPendientesFiltros(transaccionesPendientesList, clientesMap, intermediariosMap);
4034       renderTransaccionesPendientesTabla();
4035       loadingEl.style.display = 'none';
4036       wrapEl.style.display = 'block';
4037     });
4038   });
4039 });
4040 });
4041 }
4042
4043 function renderTransaccionesPendientesFiltros(list, clientesMap, intermediariosMap) {
4044   const selCliente = document.getElementById('transacciones-pendientes-filtro-cliente');
4045   const selIntermediario = document.getElementById('transacciones-pendientes-filtro-intermediario');
4046   if (!selCliente || !selIntermediario) return;
4047   const clientesUniq = [...new Set(list.map((t) => t.cliente_id).filter(Boolean))];
4048   const intUniq = [...new Set(list.map((t) => t.intermediario_id).filter(Boolean))];
4049   selCliente.innerHTML = '<option value="">Todos</option>' + clientesUniq.map((id) => '<option value="' + id + '>' + escapeHtml(clientesMap[id] || id) + '</option>').join('');
4050   selIntermediario.innerHTML = '<option value="">Todos</option>' + intUniq.map((id) => '<option value="' + id + '>' + escapeHtml(intermediariosMap[id] || id) + '</option>').join('');
4051 }
4052
4053 function renderTransaccionesPendientesTabla() {
4054   const tbody = document.getElementById('transacciones-pendientes-tbody');
4055   const selCliente = document.getElementById('transacciones-pendientes-filtro-cliente');
4056   const selIntermediario = document.getElementById('transacciones-pendientes-filtro-intermediario');
4057   const chkPandy = document.getElementById('transacciones-pendientes-filtro-pandy');
4058   if (!tbody) return;
4059   const canEditarTransacciones = userPermissions.includes('editar_transacciones');
4060   const clienteId = selCliente && selCliente.value ? selCliente.value : '';
4061   const intermediarioId = selIntermediario && selIntermediario.value ? selIntermediario.value : '';
4062   const soloPandy = chkPandy && chkPandy.checked;
4063   let list = transaccionesPendientesList;
4064   if (clienteId) list = list.filter((t) => t.cliente_id === clienteId);
4065   if (intermediarioId) list = list.filter((t) => t.intermediario_id === intermediarioId);
4066   if (soloPandy) list = list.filter((t) => t.cobrador === 'pandy' || t.pagador === 'pandy');
4067   const ordenesMap = transaccionesPendientesOrdenesMap;
4068   const clientesMap = transaccionesPendientesClientesMap;
4069   const intermediariosMap = transaccionesPendientesIntermediariosMap;
4070   const estadoTrxCombo = (t) => {
4071     if (String(t.estado || '').toLowerCase() === 'anulada') {
4072       return '<span class="badge badge-estado-anulada">Anulada</span>';
4073     }
4074     const est = t.estado === 'ejecutada' ? 'ejecutada' : 'pendiente';
4075     return '<select class="combo-estado-transaccion combo-estado-${est}" data-id="' + t.id + '" data-instrumentacion-id="' + t.instrumentacion_id + '" aria-label="Estado"><option value="pendiente"${t.estado === 'pendiente' ? 'selected' : ''}>Pendiente</option><option value="ejecutada"${t.estado === 'ejecutada' ? 'selected' : ''}>Ejecutada</option></select>';
4076   };
4077   const estadoTexto = (t) => {
4078     if (String(t.estado || '').toLowerCase() === 'anulada') return 'Anulada';
4079     return t.estado === 'ejecutada' ? 'Ejecutada' : 'Pendiente';
4080   };
4081   const listSorted = sortTransaccionesIngresosPrimero(list);
4082   tbody.innerHTML = listSorted.map((t) => {
4083     const orden = ordenesMap[t.orden_id];
4084     const ordenMini = orden ? { cliente_id: orden.cliente_id, intermediario_id: orden.intermediario_id, clientes: null, intermediarios: null } : null;
4085     const mapsPend = { clientesById: clientesMap, intermediariosById: intermediariosMap };
4086     const ordenLabel = orden ? (orden.numero != null ? '#' + orden.numero + ' : ' + (orden.fecha || '').toString().slice(0, 10) + (orden.cliente_id ? ' : ' + (clientesMa
4087       p[orden.cliente_id] || '-') : '-') : '-');
4088     return '<tr data-id="' + t.id + '" data-instrumentacion-id="' + t.instrumentacion_id + ">
4089       <td>${escapeHtml(ordenLabel)}</td>
4090       <td>${escapeHtml(t.cliente_id ? clientesMap[t.cliente_id] || '-' : '-')}</td>
4091       <td>${escapeHtml(t.intermediario_id ? intermediariosMap[t.intermediario_id] || '-' : '-')}</td>
4092       <td>${tipoTransaccionHtml(t.tipo)}</td>
4093       <td>${escapeHtml(t.moneda)}</td>

```

```

4093|         <td>${formatMonto(t.monto)}</td>
4094|         <td>${transaccionParticipanteCeldaHtml(t, ordenMini, 'pagador', mapsPend)}</td>
4095|         <td>${transaccionParticipanteCeldaHtml(t, ordenMini, 'cobrador', mapsPend)}</td>
4096|         <td>${canEditarTransacciones ? estadoTrxCombo(t) : estadoTexto(t)}</td>
4097|         <td>${canEditarTransacciones ? <button type="button" class="btn-editar btn-editar-transaccion-pendiente" data-id="${t.id}" data-instrumentacion-id="${t.instrumentacion-
id}" title="Editar"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M11 4H4a2 2 0 0-2 2v14a2 2 0 00 2h14a2 2 0 00
2-2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013 3L12 15l-4 1 1-4 9.5-9.5z"/></svg></span></button> : '}</td>
4098|     </tr>;
4099|   }).join('');
4100|   if (canEditarTransacciones) {
4101|     tbody.querySelectorAll('.combo-estado-transaccion').forEach((sel) => {
4102|       sel.addEventListener('change', function() {
4103|         const transaccionId = this.getAttribute('data-id');
4104|         const instrumentacionId = this.getAttribute('data-instrumentacion-id');
4105|         const nuevoEstado = this.value;
4106|         cambiarEstadoTransaccion(transaccionId, nuevoEstado, instrumentacionId, this).then(() => {
4107|           if (nuevoEstado === 'ejecutada') {
4108|             const idx = transaccionesPendientesList.findIndex((r) => r.id === transaccionId);
4109|             if (idx >= 0) transaccionesPendientesList.splice(idx, 1);
4110|           } else {
4111|             const item = list.find((r) => r.id === transaccionId);
4112|             if (item) item.estado = nuevoEstado;
4113|           }
4114|           renderTransaccionesPendientesTabla();
4115|           loadInicio();
4116|         });
4117|       });
4118|     });
4119|     tbody.querySelectorAll('.btn-editar-transaccion-pendiente').forEach((btn) => {
4120|       btn.addEventListener('click', () => {
4121|         const id = btn.getAttribute('data-id');
4122|         const instId = btn.getAttribute('data-instrumentacion-id');
4123|         const row = list.find((r) => r.id === id);
4124|         if (row) openModalTransaccion(row, instId);
4125|       });
4126|     });
4127|   }
4128| }
4129|
4130| function setupPanelControl() {
4131|   const btnVerOrdenes = document.getElementById('btn-ver-ordenes-pendientes');
4132|   const btnVerTrans = document.getElementById('btn-ver-transacciones-pendientes');
4133|   if (btnVerOrdenes) btnVerOrdenes.addEventListener('click', () => openModalOrdenesPendientes());
4134|   if (btnVerTrans) btnVerTrans.addEventListener('click', () => openModalTransaccionesPendientes());
4135|   const closeOrdenes = document.getElementById('modal-ordenes-pendientes-close');
4136|   const closeTrans = document.getElementById('modal-transacciones-pendientes-close');
4137|   if (closeOrdenes) closeOrdenes.addEventListener('click', () => document.getElementById('modal-ordenes-pendientes-backdrop')?.classList.remove('activo'));
4138|   if (closeTrans) closeTrans.addEventListener('click', () => document.getElementById('modal-transacciones-pendientes-backdrop')?.classList.remove('activo'));
4139|   const backdropOrdenes = document.getElementById('modal-ordenes-pendientes-backdrop');
4140|   const backdropTrans = document.getElementById('modal-transacciones-pendientes-backdrop');
4141|   if (backdropOrdenes) setupBackdropCloseOnlyOnRealClick(backdropOrdenes, () => backdropOrdenes.classList.remove('activo'));
4142|   if (backdropTrans) setupBackdropCloseOnlyOnRealClick(backdropTrans, () => backdropTrans.classList.remove('activo'));
4143|   const selCliente = document.getElementById('transacciones-pendientes-filtro-cliente');
4144|   const selIntermediario = document.getElementById('transacciones-pendientes-filtro-intermediario');
4145|   const chkPandy = document.getElementById('transacciones-pendientes-filtro-pandy');
4146|   if (selCliente) selCliente.addEventListener('change', () => renderTransaccionesPendientesTabla());
4147|   if (selIntermediario) selIntermediario.addEventListener('change', () => renderTransaccionesPendientesTabla());
4148|   if (chkPandy) chkPandy.addEventListener('change', () => renderTransaccionesPendientesTabla());
4149|   setupOrdenesPendientesFiltrosListeners();
4150| }
4151|
4152| // --- Cuenta corriente ---
4153| let ccMonedaActual = 'TODAS';
4154| let ccMovimientosList = [];
4155| let ccEsIntermediario = false;
4156| let ccDetalleTipo = null;
4157| let ccDetalleId = null;
4158| let ccDetalleMovimientosList = [];
4159| let ccDetalleOrdenesList = [];
4160| /** Todas las filas de resumen CC (todas las entidades con sus saldos). */
4161| let ccResumenRowsTodos = [];
4162| /** Lista plana de todos los movimientos CC con tipo y nombre (cliente/intermediario) para la vista "Detalle de movimientos". */
4163| let ccMovimientosDetalleList = [];
4164| let ccFiltroTipo = 'cliente';
4165| /** 'resumen' | 'detalle': vista actual en Cuenta corriente. */
4166| let ccVistaToggle = 'resumen';
4167| /** Filtro de fechas para vista Detalle de movimientos (desde/hasta). Con «solo día» ambos suelen ser hoy. */
4168| let ccDetalleDesde = '';
4169| let ccDetalleHasta = '';
4170| /** Si true, no se filtra por fecha en la tabla detalle (todo el historial). Los saldos del resumen siguen siendo históricos completos. */
4171| let ccMovimientosMostrarTodoHistorial = false;
4172| /** Tras el primer armado de CC, no resetear desde/hasta a «hoy» en cada loadCuentaCorriente (respeta «Todo el historial» y fechas elegidas). */
4173| let ccDetalleMovimientosRangoInicializado = false;
4174| /** Evita que una carga de CC antigua pise una más nueva (fetch paralelo + recarga post-sync). */
4175| let ccCargaSerial = 0;
4176| /** Filtro opcional por cliente_id o intermediario_id en solapa Movimientos (vacío = todos). */
4177| let ccDetalleFiltroEntidadId = '';
4178| /** Columnas extra movimientos CC manuales (pagador/cobrador) en SELECT Supabase. */
4179| const cc_MOV_MANUAL_PAG_COB_COLS = "manual_grupo_id, manual_pagador_rol, manual_cobrador_rol, manual_pagador_cliente_id, manual_pagador_intermediario_id, manual_cobrador_cliente_id, manual_cobrador_intermediario_id, movimiento_caja_id";
4180| /** Nombres exactos en tipos movimiento_caja (ver sql/migracion tipos_caja_cc_manual.sql). */
4181| const CC_MANUAL_TIPO_CAJA_NOMBRE_INGRESO = 'Ingreso de Dinero (Mov Manual en CC)';
4182| const CC_MANUAL_TIPO_CAJA_NOMBRE_EGRESO = 'Egreso de Dinero (Mov Manual en CC)';
4183| /** Filas actuales de la Vista Detalle (para ordenar sin volver a filtrar). */
4184| let ccDetalleVistaRowsActual = [];
4185| /** Ordenación vista Detalle: clave (fecha, nroOrden, ...) y dirección 1 | -1. */
4186| let ccDetalleSortCol = null;

```

```

4187 let ccDetalleSortDir = 1;
4188 /** Ordenación modal Detalle. */
4189 let ccDetalleModalSortCol = null;
4190 let ccDetalleModalSortDir = 1;
4191
4192 /** Join tipos operacion de Supabase (objeto o array de un elemento). */
4193 function tiposOperacionNestedMeta(tiposNested) {
4194   const t = tiposNested && (Array.isArray(tiposNested) ? tiposNested[0] : tiposNested);
4195   if (!t || typeof t !== 'object') {
4196     return { codigo: '-', nombre: '', icono_modo: 'auto', icono_url_publica: '', usa_intermediario: false };
4197   }
4198   const modRaw = (t.icono_modo !== null ? String(t.icono_modo) : 'auto').trim().toLowerCase();
4199   const modo = modRaw === 'cheque' || modRaw === 'custom' ? modRaw : 'auto';
4200   return {
4201     codigo: t.codigo !== null && String(t.codigo).trim() !== '' ? String(t.codigo) : '-',
4202     nombre: t.nombre !== null ? String(t.nombre) : '',
4203     icono_modo: modo,
4204     icono_url_publica: t.icono_url_publica !== null ? String(t.icono_url_publica).trim() : '',
4205     usa_intermediario: t.usa_intermediario === true,
4206   };
4207 }
4208
4209 function ordenTipoOpMetaMapFromOrdenes(ordenes) {
4210   const out = {};
4211   (ordenes || []).forEach((o) => {
4212     out[o.id] = tiposOperacionNestedMeta(o.tipos_operacion);
4213   });
4214   return out;
4215 }
4216
4217 /** Nombre UI para rol en movimiento CC manual (persistido en fila). */
4218 function ccManualNombreDisplayRol(rol, clienteId, intermediarioId, clientesById, intermediariosById) {
4219   const r = String(rol || '').toLowerCase();
4220   if (r === 'pandy') return etiquetaRolParticipanteUi('pandy');
4221   if (r === 'cliente') return (clientesById[clienteId] && clientesById[clienteId].nombre) || '-';
4222   if (r === 'intermediario') return (intermediariosById[intermediarioId] && intermediariosById[intermediarioId].nombre) || '-';
4223   return '-';
4224 }
4225
4226 /** Clave estable para mapas indexados por transaccion_id (evita fallas UUID string vs otro tipo). */
4227 function ccTrxMapKey(transaccionId) {
4228   if (transaccionId == null || transaccionId === '') return '';
4229   return String(transaccionId);
4230 }
4231
4232 /**
4233  * Completa pagador/cobrador/tipo en mapas desde filas de `transacciones` (clave = String(id)).
4234  * Útil tras el SELECT por movimientos y el SELECT por instrumentación: unifica metadatos.
4235  */
4236 function ccAbsorberMetaTransaccionesEnMaps(trPagadorById, trCobradorById, trTipoById, lista) {
4237   (lista || []).forEach((t) => {
4238     if (!t || t.id == null || t.id === '') return;
4239     const id = String(t.id);
4240     if (t.tipo !== null && String(t.tipo).trim() !== '') trTipoById[id] = String(t.tipo).toLowerCase();
4241     if (t.pagador !== null && String(t.pagador).trim() !== '') trPagadorById[id] = String(t.pagador).toLowerCase();
4242     else if (trPagadorById[id] === undefined) trPagadorById[id] = (t.pagador || '').toString().toLowerCase();
4243     if (t.cobrador !== null && String(t.cobrador).trim() !== '') trCobradorById[id] = String(t.cobrador).toLowerCase();
4244   });
4245 }
4246
4247 /** Enriquece movimientos CC (modal detalle / refresh) con código y metadatos de icono del tipo de la orden. */
4248 /** Nombres resueltos de Pagador/Cobrador de la transacción para filas de detalle CC (vista detalle y modal). */
4249 function ccNombresPagadorCobradorMovimiento(mov, tipoEntidad, ordenById, clientesById, intermediariosById, trTipoById, trPagadorById, trCobradorById, trParticipanteIdsByTrx) {
4250   ordenById = ordenById || {};
4251   clientesById = clientesById || {};
4252   intermediariosById = intermediariosById || {};
4253   trTipoById = trTipoById || {};
4254   trPagadorById = trPagadorById || {};
4255   trCobradorById = trCobradorById || {};
4256   trParticipanteIdsByTrx = trParticipanteIdsByTrx || {};
4257   const pagCliTrx = trParticipanteIdsByTrx.pagadorClienteIdByTrx || {};
4258   const cobCliTrx = trParticipanteIdsByTrx.cobradorClienteIdByTrx || {};
4259   const pagIntTrx = trParticipanteIdsByTrx.pagadorIntermediarioIdByTrx || {};
4260   const cobIntTrx = trParticipanteIdsByTrx.cobradorIntermediarioIdByTrx || {};
4261   function nombreClienteId(clienteId) {
4262     if (!clienteId) return '-';
4263     const kid = String(clienteId);
4264     const row = clientesById[kid] || clientesById[clienteId];
4265     const nom = row && (row.nombre !== null ? String(row.nombre) : '');
4266     return nom.trim() ? nom : '-';
4267   }
4268   function nombreEntidadPagCobValor(valor, m, tipo, lado) {
4269     const v = String(valor || '').toLowerCase();
4270     const orden = ordenById[m.orden_id];
4271     const tid = ccTrxMapKey(m.transaccion_id);
4272     if (!v) return '-';
4273     if (v === 'pandy') return etiquetaRolParticipanteUi('pandy');
4274     if (v === 'cliente') {
4275       let clienteId = tid ? (lado === 'pagador' ? pagCliTrx[tid] : cobCliTrx[tid]) : null;
4276       if (!clienteId) clienteId = tipo === 'cliente' ? m.cliente_id : (orden && orden.cliente_id);
4277       const nom = nombreClienteId(clienteId);
4278       if (nom !== '-') return nom;
4279       return etiquetaRolParticipanteUi('cliente');
4280     }
4281     if (v === 'intermediario') {
4282       let intId = tid ? (lado === 'pagador' ? pagIntTrx[tid] : cobIntTrx[tid]) : null;
4283       if (!intId) intId = orden && orden.intermediario_id;

```

```

4284 |     const row = intId && (intermediariosById[String(intId)] || intermediariosById[intId]);
4285 |     const nomStr = row && row.nombre != null ? String(row.nombre) : '';
4286 |     if (nomStr.trim()) return nomStr.trim();
4287 |     return etiquetaRolParticipanteUi('intermediario');
4288 | }
4289 | return '-';
4290 | }
4291 | function rolPagadorTransaccion(m) {
4292 |     const k = ccTrxMapKey(m.transaccion_id);
4293 |     let pag = k ? trPagadorById[k] : undefined;
4294 |     if (pag == null || String(pag).trim() === '') {
4295 |         const tipoTr = String((k && trTipoById[k]) || '').toLowerCase();
4296 |         pag = tipoTr === 'egreso' ? 'pandy' : 'cliente';
4297 |     } else pag = String(pag).toLowerCase();
4298 |     return pag;
4299 | }
4300 | function rolCobradorTransaccion(m) {
4301 |     const k = ccTrxMapKey(m.transaccion_id);
4302 |     let cob = k ? trCobradorById[k] : undefined;
4303 |     if (cob == null || String(cob).trim() === '') {
4304 |         const tipoTr = String((k && trTipoById[k]) || '').toLowerCase();
4305 |         cob = tipoTr === 'ingreso' ? 'pandy' : 'cliente';
4306 |     } else cob = String(cob).toLowerCase();
4307 |     return cob;
4308 | }
4309 | const conceptoM = (mov.concepto || '').toString();
4310 | /** Compromiso con la empresa; en UI el cobrador es quien recibe la derivación (UUID en trx aunque el rol venga genérico). */
4311 | if (conceptoM.startsWith('Instrumentación pendiente')) {
4312 |     const orden = ordenById[mov.orden_id];
4313 |     const cidAcuerdo = orden && orden.cliente_id;
4314 |     const tid = ccTrxMapKey(mov.transaccion_id);
4315 |     if (cobCliTrx[tid]) {
4316 |         return { ccPagador: nombreClienteId(cidAcuerdo), ccCobrador: nombreClienteId(cobCliTrx[tid]) };
4317 |     }
4318 |     if (tid && cobIntTrx[tid]) {
4319 |         const iid = String(cobIntTrx[tid]);
4320 |         const row = intermediariosById[iid] || intermediariosById[cobIntTrx[tid]];
4321 |         const nom = row && row.nombre != null ? String(row.nombre) : '';
4322 |         return { ccPagador: nombreClienteId(cidAcuerdo), ccCobrador: nom.trim() ? nom.trim() : etiquetaRolParticipanteUi('intermediario') };
4323 |     }
4324 |     const cob = rolCobradorTransaccion(mov);
4325 |     return {
4326 |         ccPagador: nombreClienteId(cidAcuerdo),
4327 |         ccCobrador: nombreEntidadPagCobValor(cob, mov, tipoEntidad, 'cobrador'),
4328 |     };
4329 | }
4330 | if (conceptoM.startsWith('Entrega ') && conceptoM.includes(' pendiente')) {
4331 |     const orden = ordenById[mov.orden_id];
4332 |     const cid = orden && orden.cliente_id;
4333 |     return { ccPagador: etiquetaRolParticipanteUi('pandy'), ccCobrador: nombreClienteId(cid) };
4334 | }
4335 | if (mov.es_movimiento_manual && mov.manual_pagador_rol && mov.manual_cobrador_rol) {
4336 |     return {
4337 |         ccPagador: ccManualNombreDisplayRol(mov.manual_pagador_rol, mov.manual_pagador_cliente_id, mov.manual_pagador_intermediario_id, clientesById, intermediariosById),
4338 |         ccCobrador: ccManualNombreDisplayRol(mov.manual_cobrador_rol, mov.manual_cobrador_cliente_id, mov.manual_cobrador_intermediario_id, clientesById, intermediariosById),
4339 |     };
4340 | }
4341 | if (mov.es_movimiento_manual && mov.manual_tip_movimiento === 'cobro_entidad_pandy') {
4342 |     if (tipoEntidad === 'cliente') {
4343 |         const nombre = (clientesById[mov.cliente_id] && clientesById[mov.cliente_id].nombre) || '-';
4344 |         return { ccPagador: nombre, ccCobrador: etiquetaRolParticipanteUi('pandy') };
4345 |     }
4346 |     const intId = mov.intermediario_id;
4347 |     const nombreInt = (intermediariosById[intId] && intermediariosById[intId].nombre) || '-';
4348 |     return { ccPagador: nombreInt, ccCobrador: etiquetaRolParticipanteUi('pandy') };
4349 | }
4350 | if (mov.es_movimiento_manual && mov.manual_tip_movimiento === 'pago_pandy_entidad') {
4351 |     if (tipoEntidad === 'cliente') {
4352 |         const nombre = (clientesById[mov.cliente_id] && clientesById[mov.cliente_id].nombre) || '-';
4353 |         return { ccPagador: etiquetaRolParticipanteUi('pandy'), ccCobrador: nombre };
4354 |     }
4355 |     const intId = mov.intermediario_id;
4356 |     const nombreInt = (intermediariosById[intId] && intermediariosById[intId].nombre) || '-';
4357 |     return { ccPagador: etiquetaRolParticipanteUi('pandy'), ccCobrador: nombreInt };
4358 | }
4359 | if (ccMovEsComisionAcuerdoSinTransaccion(mov)) {
4360 |     const orden = ordenById[mov.orden_id];
4361 |     const emp = etiquetaRolParticipanteUi('pandy');
4362 |     const cid = (tipoEntidad === 'cliente' ? mov.cliente_id : null) || (orden && orden.cliente_id);
4363 |     const nomCli = (clientesById[cid] && clientesById[cid].nombre) || '-';
4364 |     const iid = (tipoEntidad === 'intermediario' ? mov.intermediario_id : null) || (orden && orden.intermediario_id);
4365 |     const nomInt = (intermediariosById[iid] && intermediariosById[iid].nombre) || '-';
4366 |     if (tipoEntidad === 'cliente') {
4367 |         return { ccPagador: nomCli, ccCobrador: emp };
4368 |     }
4369 |     return { ccPagador: emp, ccCobrador: nomInt };
4370 | }
4371 | return {
4372 |     ccPagador: nombreEntidadPagCobValor(rolPagadorTransaccion(mov), mov, tipoEntidad, 'pagador'),
4373 |     ccCobrador: nombreEntidadPagCobValor(rolCobradorTransaccion(mov), mov, tipoEntidad, 'cobrador'),
4374 | };
4375 | }
4376 | }
4377 | function ccDetalleRowsConTipoOpDesdeOrdenes(movimientos, ordenes) {
4378 |     const ordenNumeroById = Object.fromEntries((ordenes || []).map(o => [o.id, o.numero])).filter(([n]) => n != null);
4379 |     const metaByOrdenId = ordenTipoOpMetaMapFromOrdenes(ordenes);
4380 |     return (movimientos || []).map(m => {

```

```

4381|     const meta = m.orden_id && metaByOrdenId[m.orden_id];
4382|     return {
4383|         ...m,
4384|         orden_numero: ordenNumeroById[m.orden_id] != null ? ordenNumeroById[m.orden_id] : (m.orden_numero != null ? m.orden_numero : null),
4385|         tipo_operacion: meta ? meta.codigo : '-',
4386|         tipo_op_nombre: meta ? meta.nombre : '-',
4387|         tipo_op_icono_modo: meta ? meta.icono_modo : 'auto',
4388|         tipo_op_icono_url: meta ? meta.icono_url_publica : '-',
4389|         tipo_op_usa_intermediario: meta ? meta.usa_intermediario === true : false,
4390|     };
4391| });
4392| }
4393|
4394| // Conceptos de movimientos CC (legibles para el usuario). Incluimos textos legacy en listas para borrar/consultar datos ya guardados.
4395| const CONCEPTO_CC_CONVERSION = 'Conversión de moneda';
4396| const CONCEPTO_CC_COMISION = 'Comisión del acuerdo';
4397| const CONCEPTOS_CC_CONVERSION TODOS = ['Conversión por tipo de cambio', 'Conversión de moneda'];
4398| const CONCEPTOS_CC_COMISION TODOS = ['Comisión', 'Comisión del acuerdo'];
4399| const CONCEPTOS_CC_AUTOGENERADOS = [...CONCEPTOS_CC_CONVERSION TODOS, ...CONCEPTOS_CC_COMISION TODOS];
4400|
4401| /** Comisión sintética (motor CC): sin transacción enlazada; pagador/cofrador sale de reglas, no de `transacciones`. */
4402| function ccMovEsComisionAcuerdoSinTransaccion(mov) {
4403|     if (mov == null) return false;
4404|     if (mov.transaccion_id != null && mov.transaccion_id !== '') return false;
4405|     const c = (mov.concepto || '').toString();
4406|     return CONCEPTOS_CC_COMISION TODOS.some((pref) => c.startsWith(pref));
4407| }
4408|
4409| /** Pagador/cofrador efectivos en sync CC/caja (misma convención que el `forEach` de `sincronizarCcYCajaDesdeOrden`: null + tipo -> defaults). */
4410| function pagCobEfectivosTransaccionSync(t) {
4411|     const tipoL = (t.tipo || '').toLowerCase();
4412|     const cob = String(t.cobrador != null ? t.cobrador : (tipoL === 'ingreso' ? 'pandy' : 'cliente')).toLowerCase();
4413|     const pag = String(t.pagador != null ? t.pagador : (tipoL === 'egreso' ? 'pandy' : 'cliente')).toLowerCase();
4414|     return { pag, cob };
4415| }
4416|
4417| /**
4418|  * Completa `pagador * id` / `cobrador * id` en un payload de transacción según roles y la orden (cliente/intermediario del acuerdo).
4419|  * No pisa valores ya definidos (MC, combos). Si pagador y cobrador son ambos `cliente`, no rellena (dos UUID distintos salen del formulario MC).
4420|  * Así CC y grillas resuelven nombres con `pagCliTrx` / `cobCliTrx` sin caer en fallback a `mov.cliente_id`.
4421|  */
4422| function completarPayloadIdsContraparteDesdeOrden(payload, orden) {
4423|     if (!payload || !orden || typeof payload !== 'object') return;
4424|     const pag = String(payload.pagador || '').toLowerCase();
4425|     const cob = String(payload.cobrador || '').toLowerCase();
4426|     if (pag === 'cliente' && cob === 'cliente') return;
4427|     const cid = orden.cliente_id != null && String(orden.cliente_id).trim() !== '' ? String(orden.cliente_id).trim() : null;
4428|     const iid = orden.intermediario_id != null && String(orden.intermediario_id).trim() !== '' ? String(orden.intermediario_id).trim() : null;
4429|     const vacio = (x) => x == null || x === '';
4430|     if (pag === 'cliente' && vacio(payload.pagador.cliente_id) && cid) payload.pagador.cliente_id = cid;
4431|     if (cob === 'cliente' && vacio(payload.cobrador.cliente_id) && cid) payload.cobrador.cliente_id = cid;
4432|     if (pag === 'intermediario' && vacio(payload.pagador.intermediario_id) && iid) payload.pagador.intermediario_id = iid;
4433|     if (cob === 'intermediario' && vacio(payload.cobrador.intermediario_id) && iid) payload.cobrador.intermediario_id = iid;
4434| }
4435|
4436| /**
4437|  * Fecha contable (`fecha`) y `estado_fecha` para movimientos CC/caja derivados de una transacción.
4438|  * Ancla al hecho: `fecha_ejecucion` y, si existe, `updated_at` de la transacción; si no hay datos, fallback al sync (comportamiento previo).
4439|  * Prepara etapa posterior de sync por diff (opción 2).
4440|  */
4441| function fechaYEstadoFechaMovimientoCcCajaDesdeTransaccion(t, fechaFallbackDia, ahoraFallbackIso) {
4442|     if (!t || typeof t !== 'object') {
4443|         return { fecha: fechaFallbackDia, estado_fecha: ahoraFallbackIso };
4444|     }
4445|     const fe = t.fecha_ejecucion;
4446|     const feStr = fe != null && String(fe).trim() !== '' ? String(fe).trim().slice(0, 10) : '';
4447|     const ua = t.updated_at;
4448|     const diaDesdeUa = ua ? String(ua).slice(0, 10) : '';
4449|     const dia = feStr || diaDesdeUa || fechaFallbackDia;
4450|     let estadoFecha = ahoraFallbackIso;
4451|     if (ua) {
4452|         const d = new Date(ua);
4453|         if (!isNaN(d.getTime())) estadoFecha = d.toISOString();
4454|     } else if (feStr) {
4455|         estadoFecha = `${feStr}T12:00:00.000Z`;
4456|     }
4457|     return { fecha: dia, estado_fecha: estadoFecha };
4458| }
4459|
4460| function fechaYEstadoFechaMovimientoCcCajaDesdeNumeroTransaccion(transacciones, nro, fechaFallbackDia, ahoraFallbackIso) {
4461|     if (nro == null || nro === '') return { fecha: fechaFallbackDia, estado_fecha: ahoraFallbackIso };
4462|     const t = (transacciones || []).find((x) => Number(x.numero) === Number(nro));
4463|     return fechaYEstadoFechaMovimientoCcCajaDesdeTransaccion(t, fechaFallbackDia, ahoraFallbackIso);
4464| }
4465|
4466| /** Filas sintéticas (comisión / cierre sin trx propia): referencia temporal por última transacción ejecutada de la orden. */
4467| function fechaYEstadoFechaMovimientoCcCajaDesdeUltimaEjecutada(transacciones, fechaFallbackDia, ahoraFallbackIso) {
4468|     const ejecutadas = (transacciones || []).filter((x) => (x.estado || '').toLowerCase() === 'ejecutada');
4469|     if (!ejecutadas.length) return { fecha: fechaFallbackDia, estado_fecha: ahoraFallbackIso };
4470|     const last = ejecutadas.slice().sort((a, b) => (Number(b.numero) || 0) - (Number(a.numero) || 0))[0];
4471|     return fechaYEstadoFechaMovimientoCcCajaDesdeTransaccion(last, fechaFallbackDia, ahoraFallbackIso);
4472| }
4473|
4474| /** ARS-USD / USD-ARS sin intermediario en orden ni en catálogo (instrumentación manual multicontraparte). */
4475| function esTipoOpMulticontraparteElegibleDesdeOrden(orden, toJoin) {
4476|     if (!orden || !toJoin) return false;
4477|     const codRaw = toJoin.codigo || '';

```

```

4478 |     const cod = (normalizarCodigoTipoOperacion(codRaw) || codRaw || '').toString().toUpperCase();
4479 |     if (cod !== 'ARS-USD' && cod !== 'USD-ARS') return false;
4480 |     if (orden.intermediario_id) return false;
4481 |     if (toJoin.usa_intermediario === true) return false;
4482 |     return true;
4483 | }
4484 |
4485 | function idClientePagadorEfectivoMulticontraparte(t, orden) {
4486 |     const { pag } = pagCobEfectivosTransaccionSync(t);
4487 |     if (pag !== 'cliente') return null;
4488 |     return t.pagador_cliente_id || orden.cliente_id || null;
4489 | }
4490 |
4491 | function idClienteCobradorEfectivoMulticontraparte(t, orden) {
4492 |     const { pag, cob } = pagCobEfectivosTransaccionSync(t);
4493 |     if (cob !== 'cliente') return null;
4494 |     if (t.cobrador_cliente_id !== null && t.cobrador_cliente_id !== '') return t.cobrador_cliente_id;
4495 |     // Cliente-Cliente: sin cobrador_cliente_id no usar orden.cliente_id (sería el mismo que el pagador del acuerdo y rompe el neteo -m/+m + línea al tercero en aplicarCcMultico
4496 |     ntraparteManualConciliacionCompleta).
4497 |     if (pag === 'cliente') return null;
4498 |     return orden.cliente_id || null;
4499 | }
4500 |
4501 | function idInterPagadorEfectivoMulticontraparte(t, orden) {
4502 |     const { pag } = pagCobEfectivosTransaccionSync(t);
4503 |     if (pag !== 'intermediario') return null;
4504 |     return t.pagador_intermediario_id || orden.intermediario_id || null;
4505 | }
4506 |
4507 | function idInterCobradorEfectivoMulticontraparte(t, orden) {
4508 |     const { cob } = pagCobEfectivosTransaccionSync(t);
4509 |     if (cob !== 'intermediario') return null;
4510 |     return t.cobrador_intermediario_id || orden.intermediario_id || null;
4511 | }
4512 |
4513 | /** True si pagador y cobrador son el mismo rol y la misma entidad (UUID resuelto). Permite Cliente+Cliente cuando son clientes distintos (multicontraparte). */
4514 | function esMismoParticipantePagadorCobrador(pagador, cobrador, orden, idsMc) {
4515 |     const p = (pagador || '').toLowerCase();
4516 |     const c = (cobrador || '').toLowerCase();
4517 |     if (p !== c) return false;
4518 |     if (p === 'pandy') return true;
4519 |     const o = orden || {};
4520 |     if (p === 'cliente') {
4521 |         const cidAc = o.cliente_id || null;
4522 |         if (!cidAc) return false;
4523 |         const pa = (idsMc && idsMc.pagador_cliente_id !== null && idsMc.pagador_cliente_id !== '') ? idsMc.pagador_cliente_id : cidAc;
4524 |         const cb = (idsMc && idsMc.cobrador_cliente_id !== null && idsMc.cobrador_cliente_id !== '') ? idsMc.cobrador_cliente_id : cidAc;
4525 |         return String(pa) === String(cb);
4526 |     }
4527 |     if (p === 'intermediario') {
4528 |         const iOrd = o.intermediario_id || null;
4529 |         if (!iOrd && !(idsMc && (idsMc.pagador_intermediario_id || idsMc.cobrador_intermediario_id))) return false;
4530 |         const pa = (idsMc && idsMc.pagador_intermediario_id !== null && idsMc.pagador_intermediario_id !== '') ? idsMc.pagador_intermediario_id : iOrd;
4531 |         const cb = (idsMc && idsMc.cobrador_intermediario_id !== null && idsMc.cobrador_intermediario_id !== '') ? idsMc.cobrador_intermediario_id : iOrd;
4532 |         if (!pa || !cb) return false;
4533 |         return String(pa) === String(cb);
4534 |     }
4535 |     return false;
4536 | }
4537 |
4538 | function pushMcClienteRow(rowsCcCliente, cid, ordenId, fecha, ahora, partial) {
4539 |     if (!cid) return;
4540 |     rowsCcCliente.push({
4541 |         cliente_id: cid,
4542 |         orden_id: ordenId,
4543 |         fecha,
4544 |         usuario_id: currentUser.id,
4545 |         estado: 'cerrado',
4546 |         estado_fecha: ahora,
4547 |         ...partial,
4548 |     });
4549 | }
4550 |
4551 | /**
4552 |  * Multicontraparte manual: CC **solo** si hay al menos una transacción ejecutada; si todas pendientes, ningún movimiento CC.
4553 |  * Cliente del acuerdo frente a Pandý: ingresos ARS pendientes (pagador = acuerdo) → -m; ingreso ejecutado acuerdo-otro cliente → par -m/+m (no cambia obligación neta con Pand
4554 |  y) + línea +m al cobrador tercero;
4555 |  * ingreso ejecutado acuerdo-Pandý: -m (cobro realizado) +m (ajuste libro acuerdo), mismo criterio de neteo en el libro del acuerdo que el pago a tercero;
4556 |  * egreso en moneda entregada pendiente a favor del acuerdo → +m (entrega pendiente, p. ej. USD);
4557 |  * egreso ejecutado con cobrador = cliente del acuerdo en monE (pagador Pandý u otro cliente): -m (Pago realizado) +m (ajuste libro acuerdo) en CC del acuerdo; si pagador es o
4558 |  tro cliente, también -m (Cobro realizado) en su CC.
4559 |  * Resto de clientes y egresos/ingresos ejecutados: movimientos por entidad. Intermediario: delega en aplicarCcMulticontraparteManualTrx.
4560 |  */
4561 | function aplicarCcMulticontraparteManualConciliacionCompleta(transacciones, orden, ordenId, ordenNumero, fecha, ahora, rowsCcCliente, rowsCcInt) {
4562 |     const lista = transacciones || [];
4563 |     const hayEjecutada = lista.some((t) => (t.estado || '').toLowerCase() === 'ejecutada'));
4564 |     if (!hayEjecutada) return;
4565 |
4566 |     const cidAcuerdo = orden.cliente_id || null;
4567 |     const monR = (orden.moneda_recibida || 'USD').toUpperCase();
4568 |     const monE = (orden.moneda_entregada || 'USD').toUpperCase();
4569 |
4570 |     lista.forEach((t) => {
4571 |         const estado = (t.estado || '').toLowerCase();
4572 |         const tipo = (t.tipo || '').toLowerCase();
4573 |         const mon = (t.moneda || 'USD').toUpperCase();
4574 |         const monto = Number(t.monto) || 0;

```



```

4572 | if (monto < 1e-9) return;
4573 | const nro = t.numero != null ? t.numero : null;
4574 | const { pag, cob } = pagCobEfectivosTransaccionSync(t);
4575 | const cidPag = idClientePagadorEfectivoMulticontraparte(t, orden);
4576 | const cidCob = idClienteCobradorEfectivoMulticontraparte(t, orden);
4577 | const feMc = fechaYEstadoFechaMovimientoCcCajaDesdeTransaccion(t, fecha, ahora);
4578 |
4579 | if (pag === 'intermediario' || cob === 'intermediario') {
4580 |   aplicarCcMulticontraparteManualTrx(t, orden, ordenId, ordenNumero, feMc.fecha, feMc.estado_fecha, rowsCcCliente, rowsCcInt);
4581 |   return;
4582 | }
4583 |
4584 | if (estado === 'pendiente') {
4585 |   if (!cidAcuerdo) return;
4586 |   if (tipo === 'ingreso' && mon === monR && pag === 'cliente' && cidPag && String(cidPag) === String(cidAcuerdo)) {
4587 |     pushMcClienteRow(rowsCcCliente, cidAcuerdo, ordenId, feMc.fecha, feMc.estado_fecha, {
4588 |       transaccion_id: t.id,
4589 |       transaccion_numero: nro,
4590 |       concepto: 'Instrumentación pendiente - Orden ${ordenNumero} · Trans ${nro != null ? nro : '-'}`,
4591 |       moneda: mon,
4592 |       monto: -monto,
4593 |       ...montosCcPorMoneda(mon, -monto),
4594 |     });
4595 |   }
4596 |   if (tipo === 'egreso' && mon === monE && cob === 'cliente' && cidCob && String(cidCob) === String(cidAcuerdo)) {
4597 |     pushMcClienteRow(rowsCcCliente, cidAcuerdo, ordenId, feMc.fecha, feMc.estado_fecha, {
4598 |       transaccion_id: t.id,
4599 |       transaccion_numero: nro,
4600 |       concepto: 'Entrega ${mon} pendiente - Orden ${ordenNumero} · Trans ${nro != null ? nro : '-'}`,
4601 |       moneda: mon,
4602 |       monto,
4603 |       ...montosCcPorMoneda(mon, monto),
4604 |     });
4605 |   }
4606 |   return;
4607 | }
4608 |
4609 | if (tipo === 'ingreso') {
4610 |   const esAcuerdoPag = cidAcuerdo && cidPag && String(cidPag) === String(cidAcuerdo);
4611 |   // Pago a tercero: cobrador rol cliente y distinto del acuerdo, o UUID aún no persistido (evita solo -m sin +m de ajuste en CC del acuerdo).
4612 |   const cobOtroCli = cob === 'cliente' && (!cidAcuerdo || !cidCob || String(cidCob) !== String(cidAcuerdo));
4613 |   const cobPandy = cob === 'pandy';
4614 |   if (esAcuerdoPag && cobOtroCli) {
4615 |     pushMcClienteRow(rowsCcCliente, cidPag, ordenId, feMc.fecha, feMc.estado_fecha, {
4616 |       transaccion_id: t.id,
4617 |       transaccion_numero: nro,
4618 |       concepto: conceptoCcLeyenda('cobro_realizado', ordenNumero, nro) + ' (pago a tercero)',
4619 |       moneda: mon,
4620 |       monto: -monto,
4621 |       ...montosCcPorMoneda(mon, -monto),
4622 |     });
4623 |     pushMcClienteRow(rowsCcCliente, cidPag, ordenId, feMc.fecha, feMc.estado_fecha, {
4624 |       transaccion_id: t.id,
4625 |       transaccion_numero: nro,
4626 |       concepto: 'Ajuste libro acuerdo - Orden ${ordenNumero} · Trans ${nro != null ? nro : '-'}`,
4627 |       moneda: mon,
4628 |       monto,
4629 |       ...montosCcPorMoneda(mon, monto),
4630 |     });
4631 |     pushMcClienteRow(rowsCcCliente, cidCob, ordenId, feMc.fecha, feMc.estado_fecha, {
4632 |       transaccion_id: t.id,
4633 |       transaccion_numero: nro,
4634 |       concepto: conceptoCcLeyenda('compromiso_pago', ordenNumero, nro),
4635 |       moneda: mon,
4636 |       monto,
4637 |       ...montosCcPorMoneda(mon, monto),
4638 |     });
4639 |     return;
4640 |   }
4641 |   if (esAcuerdoPag && cobPandy) {
4642 |     pushMcClienteRow(rowsCcCliente, cidPag, ordenId, feMc.fecha, feMc.estado_fecha, {
4643 |       transaccion_id: t.id,
4644 |       transaccion_numero: nro,
4645 |       concepto: conceptoCcLeyenda('cobro_realizado', ordenNumero, nro),
4646 |       moneda: mon,
4647 |       monto: -monto,
4648 |       ...montosCcPorMoneda(mon, -monto),
4649 |     });
4650 |     pushMcClienteRow(rowsCcCliente, cidPag, ordenId, feMc.fecha, feMc.estado_fecha, {
4651 |       transaccion_id: t.id,
4652 |       transaccion_numero: nro,
4653 |       concepto: 'Ajuste libro acuerdo - Orden ${ordenNumero} · Trans ${nro != null ? nro : '-'}`,
4654 |       moneda: mon,
4655 |       monto,
4656 |       ...montosCcPorMoneda(mon, monto),
4657 |     });
4658 |     return;
4659 |   }
4660 |   if (cidPag) {
4661 |     pushMcClienteRow(rowsCcCliente, cidPag, ordenId, feMc.fecha, feMc.estado_fecha, {
4662 |       transaccion_id: t.id,
4663 |       transaccion_numero: nro,
4664 |       concepto: conceptoCcLeyenda('cobro_realizado', ordenNumero, nro),
4665 |       moneda: mon,
4666 |       monto: -monto,
4667 |       ...montosCcPorMoneda(mon, -monto),
4668 |     });

```

```

4669 |     }
4670 |     if (cidCob && (!cidPag || String(cidCob) !== String(cidPag))) {
4671 |         pushMcClienteRow(rowsCcCliente, cidCob, ordenId, feMc.fecha, feMc.estado_fecha, {
4672 |             transaccion_id: t.id,
4673 |             transaccion_numero: nro,
4674 |             concepto: conceptoCcLeyenda('compromiso_pago', ordenNumero, nro),
4675 |             moneda: mon,
4676 |             monto,
4677 |             ...montosCcPorMoneda(mon, monto),
4678 |         });
4679 |     }
4680 |     return;
4681 | }
4682 |
4683 | if (tipo === 'egreso') {
4684 |     const esAcuerdoCob = cidAcuerdo && cidCob && String(cidCob) === String(cidAcuerdo);
4685 |     if (esAcuerdoCob && mon === monE) {
4686 |         pushMcClienteRow(rowsCcCliente, cidAcuerdo, ordenId, feMc.fecha, feMc.estado_fecha, {
4687 |             transaccion_id: t.id,
4688 |             transaccion_numero: nro,
4689 |             concepto: conceptoCcLeyenda('pago_realizado', ordenNumero, nro),
4690 |             moneda: mon,
4691 |             monto: -monto,
4692 |             ...montosCcPorMoneda(mon, -monto),
4693 |         });
4694 |         pushMcClienteRow(rowsCcCliente, cidAcuerdo, ordenId, feMc.fecha, feMc.estado_fecha, {
4695 |             transaccion_id: t.id,
4696 |             transaccion_numero: nro,
4697 |             concepto: 'Ajuste libro acuerdo - Orden ${ordenNumero} · Trans ${nro != null ? nro : '-'}',
4698 |             moneda: mon,
4699 |             monto,
4700 |             ...montosCcPorMoneda(mon, monto),
4701 |         });
4702 |         if (cidPag && String(cidPag) !== String(cidCob)) {
4703 |             pushMcClienteRow(rowsCcCliente, cidPag, ordenId, feMc.fecha, feMc.estado_fecha, {
4704 |                 transaccion_id: t.id,
4705 |                 transaccion_numero: nro,
4706 |                 concepto: conceptoCcLeyenda('cobro_realizado', ordenNumero, nro),
4707 |                 moneda: mon,
4708 |                 monto: -monto,
4709 |                 ...montosCcPorMoneda(mon, -monto),
4710 |             });
4711 |         }
4712 |         return;
4713 |     }
4714 |     if (cidPag) {
4715 |         pushMcClienteRow(rowsCcCliente, cidPag, ordenId, feMc.fecha, feMc.estado_fecha, {
4716 |             transaccion_id: t.id,
4717 |             transaccion_numero: nro,
4718 |             concepto: conceptoCcLeyenda('cobro_realizado', ordenNumero, nro),
4719 |             moneda: mon,
4720 |             monto: -monto,
4721 |             ...montosCcPorMoneda(mon, -monto),
4722 |         });
4723 |     }
4724 |     if (cidCob && (!cidPag || String(cidCob) !== String(cidPag))) {
4725 |         pushMcClienteRow(rowsCcCliente, cidCob, ordenId, feMc.fecha, feMc.estado_fecha, {
4726 |             transaccion_id: t.id,
4727 |             transaccion_numero: nro,
4728 |             concepto: conceptoCcLeyenda('compromiso_pago', ordenNumero, nro),
4729 |             moneda: mon,
4730 |             monto,
4731 |             ...montosCcPorMoneda(mon, monto),
4732 |         });
4733 |     }
4734 | }
4735 | }
4736 | }
4737 |
4738 | /**
4739 |  * CC por transacción (instrumentación multicontraparte manual), una sola trx: usado para patas con intermediario y delegación interna.
4740 |  */
4741 | function aplicarCcMulticontraparteManualTrx(t, orden, ordenId, ordenNumero, fecha, ahora, rowsCcCliente, rowsCcInt) {
4742 |     const monto = Number(t.monto) || 0;
4743 |     const mon = (t.moneda || 'USD').toUpperCase();
4744 |     const transaccionId = t.id;
4745 |     const nro = t.numero != null ? t.numero : null;
4746 |     const { pag: pagMc, cob: cobMc } = pagCobEfectivosTransaccionSync(t);
4747 |     const tipoMc = (t.tipo || '').toLowerCase();
4748 |     const estadoMc = (t.estado || '').toLowerCase();
4749 |     const cidAc = orden.cliente_id || null;
4750 |     const cidPag = idClientePagadorEfectivoMulticontraparte(t, orden);
4751 |     const cidCob = idClienteCobradorEfectivoMulticontraparte(t, orden);
4752 |     /** Ingreso ejecutado: cliente del acuerdo paga a intermediario - mismo neteo en libro del acuerdo que pago a tercero cliente (-m cobro +m ajuste). */
4753 |     const esIngresoAcuerdoAIntermediario =
4754 |         estadoMc === 'ejecutada' && tipoMc === 'ingreso' && pagMc === 'cliente' && cobMc === 'intermediario' && cidAc && cidPag && String(cidPag) === String(cidAc);
4755 |     /** Egreso ejecutado: intermediario paga al cliente del acuerdo - mismo neteo en libro del acuerdo que Pandy-Cliente (-m pago +m ajuste). Sin esto queda solo «compromiso» (+
4756 | m) y no cierra el saldo. */
4757 |     const esEgresoIntermediarioAClienteAcuerdo =
4758 |         estadoMc === 'ejecutada' &&
4759 |         tipoMc === 'egreso' &&
4760 |         pagMc === 'intermediario' &&
4761 |         cobMc === 'cliente' &&
4762 |         cidAc &&
4763 |         cidCob &&
4764 |         String(cidCob) === String(cidAc);
4765 |     if (cidPag) {

```

```

4765 |     rowsCcCliente.push({
4766 |         cliente_id: cidPag,
4767 |         orden_id: ordenId,
4768 |         transaccion_id: transaccionId,
4769 |         transaccion_numero: nro,
4770 |         concepto: conceptoCcLeyenda('cobro_realizado', ordenNumero, nro) + (esIngresoAcuerdoAIntermediario ? ' (pago a intermediario)' : ''),
4771 |         fecha,
4772 |         usuario_id: currentUserId,
4773 |         moneda: mon,
4774 |         monto: -monto,
4775 |         estado: 'cerrado',
4776 |         estado_fecha: ahora,
4777 |         ...montosCcPorMoneda(mon, -monto),
4778 |     });
4779 |     if (esIngresoAcuerdoAIntermediario) {
4780 |         rowsCcCliente.push({
4781 |             cliente_id: cidPag,
4782 |             orden_id: ordenId,
4783 |             transaccion_id: transaccionId,
4784 |             transaccion_numero: nro,
4785 |             concepto: 'Ajuste libro acuerdo - Orden ${ordenNumero} · Trans ${nro != null ? nro : '-'}`,
4786 |             fecha,
4787 |             usuario_id: currentUserId,
4788 |             moneda: mon,
4789 |             monto,
4790 |             estado: 'cerrado',
4791 |             estado_fecha: ahora,
4792 |             ...montosCcPorMoneda(mon, monto),
4793 |         });
4794 |     }
4795 | }
4796 | if (cidCob) {
4797 |     if (esEgresoIntermediarioAClienteAcuerdo) {
4798 |         rowsCcCliente.push({
4799 |             cliente_id: cidCob,
4800 |             orden_id: ordenId,
4801 |             transaccion_id: transaccionId,
4802 |             transaccion_numero: nro,
4803 |             concepto: conceptoCcLeyenda('pago_realizado', ordenNumero, nro),
4804 |             fecha,
4805 |             usuario_id: currentUserId,
4806 |             moneda: mon,
4807 |             monto: -monto,
4808 |             estado: 'cerrado',
4809 |             estado_fecha: ahora,
4810 |             ...montosCcPorMoneda(mon, -monto),
4811 |         });
4812 |         rowsCcCliente.push({
4813 |             cliente_id: cidCob,
4814 |             orden_id: ordenId,
4815 |             transaccion_id: transaccionId,
4816 |             transaccion_numero: nro,
4817 |             concepto: 'Ajuste libro acuerdo - Orden ${ordenNumero} · Trans ${nro != null ? nro : '-'}`,
4818 |             fecha,
4819 |             usuario_id: currentUserId,
4820 |             moneda: mon,
4821 |             monto,
4822 |             estado: 'cerrado',
4823 |             estado_fecha: ahora,
4824 |             ...montosCcPorMoneda(mon, monto),
4825 |         });
4826 |     } else {
4827 |         rowsCcCliente.push({
4828 |             cliente_id: cidCob,
4829 |             orden_id: ordenId,
4830 |             transaccion_id: transaccionId,
4831 |             transaccion_numero: nro,
4832 |             concepto: conceptoCcLeyenda('compromiso_pago', ordenNumero, nro),
4833 |             fecha,
4834 |             usuario_id: currentUserId,
4835 |             moneda: mon,
4836 |             monto,
4837 |             estado: 'cerrado',
4838 |             estado_fecha: ahora,
4839 |             ...montosCcPorMoneda(mon, monto),
4840 |         });
4841 |     }
4842 | }
4843 | const iidPag = idInterPagadorEfectivoMulticontraparte(t, orden);
4844 | if (iidPag) {
4845 |     rowsCcInt.push({
4846 |         intermediario_id: iidPag,
4847 |         orden_id: ordenId,
4848 |         transaccion_id: transaccionId,
4849 |         transaccion_numero: nro,
4850 |         moneda: mon,
4851 |         monto: -monto,
4852 |         concepto: conceptoCcLeyenda('pago_realizado', ordenNumero, nro),
4853 |         fecha,
4854 |         usuario_id: currentUserId,
4855 |         estado: 'cerrado',
4856 |         estado_fecha: ahora,
4857 |         ...montosCcPorMoneda(mon, -monto),
4858 |     });
4859 | }
4860 | const iidCob = idInterCobradorEfectivoMulticontraparte(t, orden);
4861 | if (iidCob) {

```

```

4862 |     rowsCcInt.push({
4863 |         intermediario_id: iidCob,
4864 |         orden_id: ordenId,
4865 |         transaccion_id: transaccionId,
4866 |         transaccion_numero: nro,
4867 |         moneda: mon,
4868 |         monto,
4869 |         concepto: conceptoCcLeyenda('cobro_realizado', ordenNumero, nro),
4870 |         fecha,
4871 |         usuario_id: currentUserId,
4872 |         estado: 'cerrado',
4873 |         estado_fecha: ahora,
4874 |         ...montosCcPorMoneda(mon, monto),
4875 |     });
4876 | }
4877 | }
4878 |
4879 | /**
4880 |  * Regla unificada CC: a partir de una transacción y su orden devuelve la contribución al saldo (pendientes) para cliente e intermediario.
4881 |  * Sirve para resumen y para cualquier flujo que deba reflejar la realidad según estado, pagador, cobrador, moneda.
4882 |  * @param {Object} t - transacción { estado, pagador, cobrador, moneda, monto }
4883 |  * @param {Object} orden - orden { cliente_id, intermediario_id, monto_recibido, monto_entregado, moneda_recibida, moneda_entregada }
4884 |  * @param {Array} transaccionesOrden - todas las transacciones de la misma instrumentación (para cliente: ingreso/egreso)
4885 |  * @returns {{ cliente: { id, delta: {USD,EUR,ARS} } | null, intermediario: { id, delta: {USD,EUR,ARS} } | null }}
4886 |  */
4887 | function contribucionPendienteCcUnificada(t, orden, transaccionesOrden, ordenEsMcManualElegible) {
4888 |     const estado = (t.estado || '').toLowerCase();
4889 |     const pagador = String(t.pagador || '').toLowerCase();
4890 |     const cobrador = String(t.cobrador || '').toLowerCase();
4891 |     const mon = ((t.moneda || 'ARS') + '').toUpperCase();
4892 |     if (mon !== 'USD' && mon !== 'EUR' && mon !== 'ARS') return { cliente: null, intermediario: null };
4893 |     const val = Number(t.monto) || 0;
4894 |     const out = { cliente: null, intermediario: null };
4895 |     if (estado !== 'pendiente') return out;
4896 |     // Multicontraparte manual elegible: pendientes del acuerdo van en movimientos CC al haber alguna ejecutada; no duplicar con ajuste de resumen.
4897 |     if (ordenEsMcManualElegible === true) return out;
4898 |     // Intermediario: Pandy debe → restar del saldo (se muestra en rojo).
4899 |     if (orden.intermediario_id && pagador === 'pandy' && cobrador === 'intermediario') {
4900 |         out.intermediario = { id: orden.intermediario_id, delta: { USD: 0, EUR: 0, ARS: 0 } };
4901 |         out.intermediario.delta[mon] = -val;
4902 |     }
4903 |     // Cliente: mismo criterio por moneda (ingreso pendiente + egreso ejecutada → +mr; ingreso ejecutada + egreso pendiente → -me).
4904 |     if (orden.cliente_id && transaccionesOrden && transaccionesOrden.length) {
4905 |         const mr = Number(orden.monto_recibido) || 0;
4906 |         const me = Number(orden.monto_entregado) || 0;
4907 |         const monR = (orden.moneda_recibida || 'USD').toUpperCase();
4908 |         const monE = (orden.moneda_entregada || 'USD').toUpperCase();
4909 |         if (monR !== monE) return out;
4910 |         const ingreso = transaccionesOrden.find((x) => String(x.pagador || '').toLowerCase() === 'cliente' && String(x.cobrador || '').toLowerCase() === 'pandy');
4911 |         const egreso = transaccionesOrden.find((x) => String(x.cobrador || '').toLowerCase() === 'cliente' && String(x.pagador || '').toLowerCase() === 'pandy');
4912 |         if (ingreso && egreso) {
4913 |             const ingresoPend = (ingreso.estado || '').toLowerCase() !== 'ejecutada';
4914 |             const egresoPend = (egreso.estado || '').toLowerCase() !== 'ejecutada';
4915 |             const delta = { USD: 0, EUR: 0, ARS: 0 };
4916 |             if (ingresoPend && !egresoPend) delta[monR] = mr;
4917 |             if (!ingresoPend && egresoPend) delta[monE] = -me;
4918 |             if (delta.USD !== 0 || delta.EUR !== 0 || delta.ARS !== 0) out.cliente = { id: orden.cliente_id, delta };
4919 |         }
4920 |     }
4921 |     return out;
4922 | }
4923 |
4924 | /**
4925 |  * Contribución al saldo CC cliente según modelo CC_MODELO (docs/CC_MODELO_REFERENCIA.md).
4926 |  * Solo aplica a órdenes ARS-ARS con intermediario. Regla: si hay ingreso Cliente-Pandy ejecutado y egreso Pandy-Cliente pendiente, suma -(me - sum_egreso) (Pandy debe al cliente). Cualquier orden de cambios de estado.
4927 |  * @param {Object} orden - { monto_recibido, monto_entregado, moneda_recibida, moneda_entregada }
4928 |  * @param {Array} transacciones - de la instrumentación de la orden: { tipo, pagador, cobrador, monto, estado }
4929 |  * @returns {{ USD: number, EUR: number, ARS: number }}
4930 |  */
4931 | function contribucionSaldoClienteModeloCc(orden, transacciones) {
4932 |     const mr = Number(orden.monto_recibido) || 0;
4933 |     const me = Number(orden.monto_entregado) || 0;
4934 |     const monR = ((orden.moneda_recibida || 'ARS') + '').toUpperCase();
4935 |     const monE = ((orden.moneda_entregada || 'ARS') + '').toUpperCase();
4936 |     const out = { USD: 0, EUR: 0, ARS: 0 };
4937 |     if (!transacciones || transacciones.length === 0) return out;
4938 |     const sumIngreso = transacciones
4939 |         .filter((t) => (t.tipo || '').toLowerCase() === 'ingreso' && String(t.pagador || '').toLowerCase() === 'cliente' && String(t.cobrador || '').toLowerCase() === 'pandy' && (t.estado || '').toLowerCase() === 'ejecutada'))
4940 |         .reduce((s, t) => s + (Number(t.monto) || 0), 0);
4941 |     const sumEgreso = transacciones
4942 |         .filter((t) => (t.tipo || '').toLowerCase() === 'egreso' && String(t.pagador || '').toLowerCase() === 'pandy' && String(t.cobrador || '').toLowerCase() === 'cliente' && (t.estado || '').toLowerCase() === 'ejecutada'))
4943 |         .reduce((s, t) => s + (Number(t.monto) || 0), 0);
4944 |     const mon = monE === 'USD' ? 'USD' : (monE === 'EUR' ? 'EUR' : 'ARS');
4945 |     const monCli = monR === 'USD' ? 'USD' : (monR === 'EUR' ? 'EUR' : 'ARS');
4946 |     // Tx1 ejecutada + Tx2 pendiente: Pandy debe al cliente → -pendiente (egreso pendiente).
4947 |     if (sumIngreso >= 1e-6 && sumEgreso < me - 1e-6) {
4948 |         out[mon] = -(me - sumEgreso);
4949 |         return out;
4950 |     }
4951 |     // Tx1 pendiente + Tx2 ejecutada: regla SUMA SALDO Y para Tx1 → -mr en saldo (modelo).
4952 |     if (sumIngreso < 1e-6 && sumEgreso >= 1e-6) {
4953 |         out[monCli] = -mr;
4954 |         return out;
4955 |     }

```

```

4956|     return out;
4957| }
4958|
4959| /**
4960|  * Contribución al saldo CC intermediario según modelo CC_MODELO (docs/CC_MODELO_REFERENCIA.md).
4961|  * Solo aplica a órdenes ARS-ARS con intermediario. Regla: solo suma la contrapartida pendiente (Int-Pandy) cuando el egreso Pandy-Int está ejecutado; el valor es -(monto pendiente), Se usa el monto de la transacción Int-Pandy pendiente (197k en el modelo), no el bruto 200k.
4962|  * @param {Object} orden { monto recibido, tasa descuento intermediario, moneda recibida }
4963|  * @param {Array} transacciones - de la instrumentación: { tipo, pagador, cobrador, monto, estado }
4964|  * @returns {{ USD: number, EUR: number, ARS: number }}
4965|  */
4966| function contribucionSaldoIntermediarioModeloCc(orden, transacciones) {
4967|     const out = { USD: 0, EUR: 0, ARS: 0 };
4968|     if (!transacciones || transacciones.length === 0) return out;
4969|     const hayEgresoIntEjecutado = transacciones.some(t => (t.tipo || '').toLowerCase() === 'egreso' && String(t.pagador || '').toLowerCase() === 'pandy' && String(t.cobrador || '').toLowerCase() === 'intermediario' && (t.estado || '').toLowerCase() === 'ejecutada'));
4970|     const hayIngresoIntEjecutado = transacciones.some(t => (t.tipo || '').toLowerCase() === 'ingreso' && String(t.pagador || '').toLowerCase() === 'intermediario' && String(t.cobrador || '').toLowerCase() === 'pandy' && (t.estado || '').toLowerCase() === 'ejecutada'));
4971|     const sumEgresoPandyIntEjecutado = transacciones
4972|         .filter((t) => (t.tipo || '').toLowerCase() === 'egreso' && String(t.pagador || '').toLowerCase() === 'pandy' && String(t.cobrador || '').toLowerCase() === 'intermediario' && (t.estado || '').toLowerCase() === 'ejecutada'))
4973|         .reduce((s, t) => s + (Number(t.monto) || 0), 0);
4974|     const sumIngresoIntEjecutado = transacciones
4975|         .filter((t) => (t.tipo || '').toLowerCase() === 'ingreso' && String(t.pagador || '').toLowerCase() === 'intermediario' && String(t.cobrador || '').toLowerCase() === 'pandy' && (t.estado || '').toLowerCase() === 'ejecutada'))
4976|         .reduce((s, t) => s + (Number(t.monto) || 0), 0);
4977|     const pendientesIntPandy = transacciones.filter((t) => (t.tipo || '').toLowerCase() === 'ingreso' && String(t.pagador || '').toLowerCase() === 'intermediario' && String(t.cobrador || '').toLowerCase() === 'pandy' && (t.estado || '').toLowerCase() === 'pendiente'));
4978|     const pendientesPandyInt = transacciones.filter((t) => (t.tipo || '').toLowerCase() === 'egreso' && String(t.pagador || '').toLowerCase() === 'pandy' && String(t.cobrador || '').toLowerCase() === 'intermediario' && (t.estado || '').toLowerCase() === 'pendiente'));
4979|     // Ambas ejecutadas (montos de transacción): fórmula legacy coherente con neto en ARS (-cheque bruto + efectivo cobrado).
4980|     // Los movimientos persistidos en CC int usan +cheque, -comisión, -efectivo (reglas CHEQUE-ARS); la suma de movimientos = 0 al cerrar.
4981|     if (hayEgresoIntEjecutado && hayIngresoIntEjecutado) {
4982|         out.ARS = -sumEgresoPandyIntEjecutado + sumIngresoIntEjecutado;
4983|         return out;
4984|     }
4985|     // Tx3 pendiente + Tx4 ejecutada: saldo = -(monto Tx3 pendiente) + 197k = -200k + 197k = -3k.
4986|     if (!hayEgresoIntEjecutado && hayIngresoIntEjecutado && pendientesPandyInt.length > 0) {
4987|         const montoPandyIntPendiente = pendientesPandyInt.reduce((s, t) => s + (Number(t.monto) || 0), 0);
4988|         if (montoPandyIntPendiente >= 1e-6) {
4989|             out.ARS = -montoPandyIntPendiente + sumIngresoIntEjecutado;
4990|             return out;
4991|         }
4992|     }
4993|     // Ambas pendientes (escenario Excel: Tx3 P, Tx4 P): saldo = -200k + 197k = -3.000.
4994|     if (!hayEgresoIntEjecutado && !hayIngresoIntEjecutado && (pendientesPandyInt.length > 0 || pendientesIntPandy.length > 0)) {
4995|         const sumPandyIntP = pendientesPandyInt.reduce((s, t) => s + (Number(t.monto) || 0), 0);
4996|         const sumIntPandyP = pendientesIntPandy.reduce((s, t) => s + (Number(t.monto) || 0), 0);
4997|         out.ARS = -sumPandyIntP + sumIntPandyP;
4998|         return out;
4999|     }
5000|     if (!hayEgresoIntEjecutado) return out;
5001|     // Tx3 ejecutada + Tx4 pendiente: saldo = -(monto Int-Pandy pendiente) = -197k.
5002|     const montoPendienteIntPandy = pendientesIntPandy.reduce((s, t) => s + (Number(t.monto) || 0), 0);
5003|     if (montoPendienteIntPandy >= 1e-6) {
5004|         out.ARS = -montoPendienteIntPandy;
5005|         return out;
5006|     }
5007|     const mr = Number(orden.monto_recibido) || 0;
5008|     const tasa = Number(orden.tasa_descuento_intermediario) || 0;
5009|     const montoEfectivoInt = (typeof tasa === 'number' && !isNaN(tasa) && tasa >= 0 && tasa < 1) ? mr * (1 - tasa) : (tasa >= 1 && tasa <= 100 ? mr * (1 - tasa / 100) : mr);
5010|     if (sumIngresoIntEjecutado >= montoEfectivoInt - 1e-6) return out;
5011|     out.ARS = -(montoEfectivoInt - sumIngresoIntEjecutado);
5012|     return out;
5013| }
5014|
5015| /**
5016|  * Según modelo CC_MODELO.xlsx (columnas INCLUIR EN MOV CC CLIENTE / INCLUIR EN MOV DE CC INTERMEDIARIO).
5017|  * Solo aplica a órdenes ARS-ARS con intermediario. Devuelve true si la transacción t debe generar un movimiento en la CC del cliente.
5018|  * En el Excel: Y para Tx1 (Ingreso Cliente-Pandy) y Tx2 (Egreso Pandy-Cliente) cuando están ejecutadas.
5019|  */
5020| function incluirEnMovimientosCcClienteModelo(orden, t) {
5021|     if (!esTipoOperacionChequeArsDesdeJoin(orden.tipos_operacion) || !orden.intermediario_id) return true; // No modelo: incluir todo
5022|     if ((t.estado || '').toLowerCase() !== 'ejecutada') return false;
5023|     const tipo = (t.tipo || '').toLowerCase();
5024|     const pag = String(t.pagador || '').toLowerCase();
5025|     const cob = String(t.cobrador || '').toLowerCase();
5026|     return (tipo === 'ingreso' && pag === 'cliente' && cob === 'pandy') || (tipo === 'egreso' && pag === 'pandy' && cob === 'cliente');
5027| }
5028|
5029| /**
5030|  * Según modelo CC_MODELO.xlsx. True si la transacción t debe generar un movimiento en la CC del intermediario.
5031|  * En el Excel: Y para Tx3 (Egreso Pandy-Intermediario) y Tx4 (Ingreso Intermediario-Pandy) cuando están ejecutadas.
5032|  */
5033| function incluirEnMovimientosCcIntermediarioModelo(orden, t) {
5034|     if (!esTipoOperacionChequeArsDesdeJoin(orden.tipos_operacion) || !orden.intermediario_id) return true;
5035|     if ((t.estado || '').toLowerCase() !== 'ejecutada') return false;
5036|     const tipo = (t.tipo || '').toLowerCase();
5037|     const pag = String(t.pagador || '').toLowerCase();
5038|     const cob = String(t.cobrador || '').toLowerCase();
5039|     return (tipo === 'egreso' && pag === 'pandy' && cob === 'intermediario') || (tipo === 'ingreso' && pag === 'intermediario' && cob === 'pandy');
5040| }
5041|
5042| /**
5043|  * Normaliza código de tipo de operación para coincidir con `reglas_de_negocio` / catálogo (ej. "CHEQUE - ARS" → "CHEQUE-ARS").
5044|  * @param {string} codigo
5045|  * @returns {string|null}

```

```

5046 | */
5047 | function normalizarCodigoTipoOperacion(codigo) {
5048 |   if (codigo == null || typeof codigo !== 'string') return null;
5049 |   const s = codigo.trim();
5050 |   if (!s) return null;
5051 |   return s.replace(/\s*\s*/g, '-').replace(/\/s*\s*\s*/g, '-').replace(/\/s*\s*\s*/g, '-').replace(/\/s*/g, '-').replace(/\/s*/g, '-');
5052 | }
5053 |
5054 | /** Catálogo tipos operacion: CHEQUE en UI; en tabla ordenes solo USD/EUR/ARS (cheque = ARS). */
5055 | function monedaCatálogoParaOrden(m) {
5056 |   const u = (m || '').toString().trim().toUpperCase();
5057 |   return u === 'CHEQUE' ? 'ARS' : u;
5058 | }
5059 |
5060 | /**
5061 |  * Operación cheque en pesos: **CHEQUE-ARS** (catálogo) o moneda_in/out CHEQUE+ARS.
5062 |  * El código **ARS-ARS** ya no se usa como tipo de operación (eliminado del catálogo).
5063 |  */
5064 | function esTipoOperacionChequeArs(codigo, monedaIn, monedaOut) {
5065 |   const c = (codigo || '').toString().trim().toUpperCase();
5066 |   if (c.includes('CHEQUE')) return true;
5067 |   const mi = (monedaIn || '').toString().trim().toUpperCase();
5068 |   const mo = (monedaOut || '').toString().trim().toUpperCase();
5069 |   return (mi === 'CHEQUE' && mo === 'ARS') || (mi === 'ARS' && mo === 'CHEQUE');
5070 | }
5071 |
5072 | function esTipoOperacionChequeArsDesdeJoin(tiposNested, codigoFallback) {
5073 |   const t = tiposNested && (Array.isArray(tiposNested) ? tiposNested[0] : tiposNested);
5074 |   const codigo = (t && t.codigo) || codigoFallback || '';
5075 |   return esTipoOperacionChequeArs(codigo, t && t.moneda_in, t && t.moneda_out);
5076 | }
5077 |
5078 | /** Desde <option data-codigo data-moneda-in data-moneda-out> del tipo en el modal de orden. */
5079 | function esChequeArsDesdeSelectOption(opt) {
5080 |   if (!opt) return false;
5081 |   return esTipoOperacionChequeArs(
5082 |     opt.getAttribute('data-codigo') || '',
5083 |     opt.getAttribute('data-moneda-in') || '',
5084 |     opt.getAttribute('data-moneda-out') || ''
5085 |   );
5086 | }
5087 |
5088 | /**
5089 |  * Orden de lista puede no traer join tipos_operacion; se completa con ordenesVistaTiposOpMap (loadOrdenes).
5090 |  */
5091 | function tiposOperacionEfectivoParaOrden(orden) {
5092 |   if (!orden) return null;
5093 |   const nested = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
5094 |   if (nested && (nested.codigo !== null || nested.moneda_in !== null || nested.moneda_out !== null)) return nested;
5095 |   const tid = orden.tipo_operacion_id;
5096 |   let m = tid ? ordenesVistaTiposOpMap[tid] : null;
5097 |   if (!m && tid) m = ordenesPendientesTiposOpMap[tid];
5098 |   if (!m) return null;
5099 |   return { codigo: m.codigo, moneda_in: m.moneda_in, moneda_out: m.moneda_out };
5100 | }
5101 |
5102 | function esOrdenChequeArsDesdeOrden(orden) {
5103 |   const t = tiposOperacionEfectivoParaOrden(orden);
5104 |   return esTipoOperacionChequeArs(t && t.codigo, t && t.moneda_in, t && t.moneda_out);
5105 | }
5106 |
5107 | /**
5108 |  * Booleano desde Postgres/JSON (evita que el string "false" sea truthy con !!v en filtros de reglas).
5109 |  */
5110 | function coercePgBooleanStrict(v) {
5111 |   if (v === true) return true;
5112 |   if (v === false) return false;
5113 |   if (v === null) return false;
5114 |   const s = String(v).trim().toLowerCase();
5115 |   return s === 'true' || s === 't' || s === '1';
5116 | }
5117 |
5118 | /** True si la contrapartida del par (cliente-pandy o pandy-intermediario) está ejecutada. */
5119 | function contrapartidaEjecutada(transacciones, pagador, cobrador, tipoTransaccion) {
5120 |   const pag = String(pagador || '').toLowerCase();
5121 |   const cob = String(cobrador || '').toLowerCase();
5122 |   const tipo = String(tipoTransaccion || '').toLowerCase();
5123 |   // Flujo inverso operativo: Cliente->Intermediario se considera contrapartida de Pandy->Cliente.
5124 |   if (tipo === 'ingreso' && pag === 'cliente' && cob === 'intermediario') {
5125 |     return (transacciones || []).some((t) =>
5126 |       (t.tipo || '').toLowerCase() === 'egreso' &&
5127 |       String(t.pagador || '').toLowerCase() === 'pandy' &&
5128 |       String(t.cobrador || '').toLowerCase() === 'cliente' &&
5129 |       (t.estado || '').toLowerCase() === 'ejecutada'
5130 |     ));
5131 |   }
5132 |   if (tipo === 'egreso' && pag === 'pandy' && cob === 'cliente') {
5133 |     return (transacciones || []).some((t) =>
5134 |       (t.tipo || '').toLowerCase() === 'ingreso' &&
5135 |       String(t.pagador || '').toLowerCase() === 'cliente' &&
5136 |       (
5137 |         String(t.cobrador || '').toLowerCase() === 'pandy' ||
5138 |         String(t.cobrador || '').toLowerCase() === 'intermediario'
5139 |       ) &&
5140 |       (t.estado || '').toLowerCase() === 'ejecutada'
5141 |     ));
5142 |   }

```

```

5143 // Flujo intermediado (Pandy central): cuando la transacción es Int->Cliente (egreso),
5144 // su contrapartida operativa es Cliente->Pandy (ingreso), no Cliente->Intermediario.
5145 if (tipo === 'egreso' && pag === 'intermediario' && cob === 'cliente') {
5146   return (transacciones || []).some((t) =>
5147     (t.tipo || '').toLowerCase() === 'ingreso' &&
5148     String(t.pagador || '').toLowerCase() === 'cliente' &&
5149     String(t.cobrador || '').toLowerCase() === 'pandy' &&
5150     (t.estado || '').toLowerCase() === 'ejecutada'
5151   ));
5152 }
5153 // Simétrico: para Cliente->Pandy (ingreso), considerar como contrapartida pagador hacia Cliente
5154 // tanto Pandy->Cliente como Intermediario->Cliente.
5155 if (tipo === 'ingreso' && pag === 'cliente' && cob === 'pandy') {
5156   return (transacciones || []).some((t) =>
5157     (t.tipo || '').toLowerCase() === 'egreso' &&
5158     ((String(t.cobrador || '').toLowerCase() === 'cliente') || (String(t.cobrador || '').toLowerCase() === 'intermediario')) &&
5159     ((String(t.pagador || '').toLowerCase() === 'pandy' || (String(t.pagador || '').toLowerCase() === 'intermediario')) &&
5160     (t.estado || '').toLowerCase() === 'ejecutada'
5161   ));
5162 }
5163 // Pandy->Intermediario (egreso): sin caso especial. La contrapartida simétrica es
5164 // Ingreso Intermediario->Pandy ejecutada (Tx4). No usar Cliente->Pandy: en E,P,P,P
5165 // Tx1 ejecutada no debe marcar contrapartida de Tx3 si Tx4 sigue pendiente (CHEQUE-ARS).
5166 const otroTipo = tipo === 'ingreso' ? 'egreso' : 'ingreso';
5167 const otroPag = cob;
5168 const otroCob = pag;
5169 return (transacciones || []).some((t) =>
5170   (t.tipo || '').toLowerCase() === otroTipo &&
5171   String(t.pagador || '').toLowerCase() === otroPag &&
5172   String(t.cobrador || '').toLowerCase() === otroCob &&
5173   (t.estado || '').toLowerCase() === 'ejecutada'
5174 ));
5175 }
5176
5177 /** True si hay egreso ejecutado que entrega al cliente (Pandy-Cliente o Intermediario-Cliente). Usado para "par cliente cerrado" en USD-USD+int. */
5178 function egresoEntregaAClienteEjecutado(transacciones) {
5179   return (transacciones || []).some((t) =>
5180     (t.tipo || '').toLowerCase() === 'egreso' &&
5181     String(t.cobrador || '').toLowerCase() === 'cliente' &&
5182     (String(t.pagador || '').toLowerCase() === 'pandy' || String(t.pagador || '').toLowerCase() === 'intermediario') &&
5183     (t.estado || '').toLowerCase() === 'ejecutada'
5184   ));
5185 }
5186
5187 /** Patrón cp_ic: egreso Intermediario-Cliente ya ejecutado (el int. entregó al cliente). */
5188 function egresoIntermediarioAClienteEjecutado(transacciones) {
5189   return (transacciones || []).some((t) =>
5190     (t.tipo || '').toLowerCase() === 'egreso' &&
5191     String(t.pagador || '').toLowerCase() === 'intermediario' &&
5192     String(t.cobrador || '').toLowerCase() === 'cliente' &&
5193     (t.estado || '').toLowerCase() === 'ejecutada'
5194   ));
5195 }
5196
5197 /**
5198  * Patrón de instrumentación con intermediario (2 tx típicas):
5199  * - **ci_pc**: ingreso Cliente-Intermediario + egreso Pandy-Cliente (cobro al int., entrega desde caja Pandy).
5200  * - **cp_ic**: ingreso Cliente-Pandy + egreso Intermediario-Cliente.
5201  * Se infiere por las filas de instrumentación (no solo por estado ejecutada), para P/E y comisiones.
5202  */
5203 function patronInstrumentacionIntDesdeTransacciones(transacciones) {
5204   const txs = transacciones || [];
5205   const hayIngClienteInter = txs.some((t) =>
5206     (t.tipo || '').toLowerCase() === 'ingreso' &&
5207     String(t.pagador || '').toLowerCase() === 'cliente' &&
5208     String(t.cobrador || '').toLowerCase() === 'intermediario');
5209   const hayIngClientePandy = txs.some((t) =>
5210     (t.tipo || '').toLowerCase() === 'ingreso' &&
5211     String(t.pagador || '').toLowerCase() === 'cliente' &&
5212     String(t.cobrador || '').toLowerCase() === 'pandy');
5213   if (hayIngClienteInter && !hayIngClientePandy) return 'ci_pc';
5214   return 'cp_ic';
5215 }
5216
5217 /** Ingreso ejecutado del cliente hacia Pandy o hacia intermediario (patrones USD-USD+int). */
5218 function ingresoDesdeClienteHaciaPandyOIntermediarioEjecutado(transacciones) {
5219   return (transacciones || []).some((t) =>
5220     (t.tipo || '').toLowerCase() === 'ingreso' &&
5221     String(t.pagador || '').toLowerCase() === 'cliente' &&
5222     (String(t.cobrador || '').toLowerCase() === 'pandy' || String(t.cobrador || '').toLowerCase() === 'intermediario') &&
5223     (t.estado || '').toLowerCase() === 'ejecutada'
5224   ));
5225 }
5226
5227 /**
5228  * Estado efectivo para comisión sin transacción propia. La tabla define condicion_estado_comision; este motor solo interpreta el nombre.
5229  * par_pandy_int: ejecutada si al menos Tx3 o Tx4 ejecutada. Si existen transacciones Pandy-Intermediario
5230  * en la instrumentación, no se usa el cierre Cliente-Pandy como sustituto (comisión int. solo con actividad en ese par).
5231  * par_cliente (comisión Pandy): ejecutada si Tx1 o Tx2 ejecutada.
5232  */
5233 function estadoEfectivoComision(transacciones, condicion) {
5234   if (condicion === 'par_pandy_int') {
5235     const hayTx3 = (transacciones || []).some((t) => (t.tipo || '').toLowerCase() === 'egreso' && String(t.pagador || '').toLowerCase() === 'pandy' && String(t.cobrador || '').toLowerCase() === 'intermediario' && (t.estado || '').toLowerCase() === 'ejecutada'));
5236     const hayTx4 = (transacciones || []).some((t) => (t.tipo || '').toLowerCase() === 'ingreso' && String(t.pagador || '').toLowerCase() === 'intermediario' && String(t.cobrador || '').toLowerCase() === 'pandy' && (t.estado || '').toLowerCase() === 'ejecutada'));
5237     if (hayTx3 || hayTx4) return 'ejecutada';

```

```

5238 // Si la orden tiene patas Pandy=Intermediario instrumentadas (CHEQUE-ARS canónico), la comisión int. no debe
5239 // devengar solo por cerrar Cliente=Pandy (evita E,E,P,P con comisión int. indebida; ver E2E 12 combinaciones).
5240 const hayInstrumentacionPandyInt = (transacciones || []).some((t) => {
5241   const tipo = (t.tipo || '').toLowerCase();
5242   const pag = String(t.pagador || '').toLowerCase();
5243   const cob = String(t.cobrador || '').toLowerCase();
5244   return (tipo === 'egreso' && pag === 'pandy' && cob === 'intermediario')
5245     || (tipo === 'ingreso' && pag === 'intermediario' && cob === 'pandy');
5246 });
5247 if (hayInstrumentacionPandyInt) return 'pendiente';
5248 // Legacy: órdenes que solo instrumentan Cliente=Pandy o Cliente=Intermediario+Pandy-Cliente (sin filas Tx3/Tx4).
5249 const tx1 = ingresoDesdeClienteHaciaPandyOIntermediarioEjecutado(transacciones);
5250 const tx2 = egresoEntregaAClienteEjecutado(transacciones);
5251 return tx1 && tx2 ? 'ejecutada' : 'pendiente';
5252 }
5253 if (condicion === 'par_cliente') {
5254   const tx1 = ingresoDesdeClienteHaciaPandyOIntermediarioEjecutado(transacciones);
5255   const tx2 = egresoEntregaAClienteEjecutado(transacciones);
5256   return (tx1 || tx2) ? 'ejecutada' : 'pendiente';
5257 }
5258 return 'ejecutada';
5259 }
5260
5261 /** Condición de comisión desde `reglas_de_negocio` (filas es_comision con condicion_estado_comision). */
5262 function getCondicionComisionReglasNegocio(reglas, tipoCodigo, pagador, cobrador, tipoTransaccion) {
5263   const tc = String(tipoCodigo || '').toUpperCase();
5264   const pag = String(pagador || '').toLowerCase();
5265   const cob = String(cobrador || '').toLowerCase();
5266   const tip = String(tipoTransaccion || '').toLowerCase();
5267   const candidates = (reglas || []).filter((r) =>
5268     String(r.tipo_operacion_codigo || '').toUpperCase() === tc &&
5269     coercePgBooleanStrict(r.es_comision) === true &&
5270     (r.pagador || '').toLowerCase() === pag &&
5271     (r.cobrador || '').toLowerCase() === cob &&
5272     (r.tipo_transaccion || '').toLowerCase() === tip
5273   );
5274   const withC = candidates.find((r) => (r.condicion_estado_comision || '').trim() !== '');
5275   return (withC && withC.condicion_estado_comision) ? withC.condicion_estado_comision : null;
5276 }
5277
5278 /**
5279  * Reglas de negocio (`reglas_de_negocio`): fuente de verdad del motor CC (ver docs/MIGRACION_UNA_TABLA_REGLAS_DE_NEGOCIO.md).
5280  * Si no hay filas para el par `{codigo, usa_intermediario}`, el sync usa fallbacks legacy (sin motor de tabla).
5281  */
5282 function getReglasDeNegocio(codigoTipoOperacion, usaIntermediario) {
5283   const c = String(codigoTipoOperacion || '').trim().toUpperCase();
5284   if (!c) return Promise.resolve([]);
5285   return client.from('reglas_de_negocio').select('*')
5286     .eq('tipo_operacion_codigo', c)
5287     .eq('usa_intermediario', !!usaIntermediario)
5288     .then((r) => (r.data && Array.isArray(r.data) ? r.data : []))
5289     .catch(() => []);
5290 }
5291
5292 function lookupReglasDeNegocio(reglas, tipoOperacionCodigo, pagador, cobrador, tipoTransaccion, esComision, estadoTransaccion, contrapartidaEjecutada) {
5293   if (!reglas || !reglas.length) return [];
5294   const cod = String(tipoOperacionCodigo || '').toUpperCase();
5295   const pag = String(pagador || '').toLowerCase();
5296   const cob = String(cobrador || '').toLowerCase();
5297   const tipo = String(tipoTransaccion || '').toLowerCase();
5298   const est = String(estadoTransaccion || '').toLowerCase();
5299   const cont = !!contrapartidaEjecutada;
5300   const out = reglas.filter((r) =>
5301     String(r.tipo_operacion_codigo || '').toUpperCase() === cod &&
5302     (r.pagador || '').toLowerCase() === pag &&
5303     (r.cobrador || '').toLowerCase() === cob &&
5304     (r.tipo_transaccion || '').toLowerCase() === tipo &&
5305     coercePgBooleanStrict(r.es_comision) === !!esComision &&
5306     (r.estado_transaccion || '').toLowerCase() === est &&
5307     coercePgBooleanStrict(r.contrapartida_ejecutada) === cont
5308   );
5309   out.sort((a, b) => (Number(a.linea) || 0) - (Number(b.linea) || 0));
5310   return out;
5311 }
5312
5313 /**
5314  * Base numérica para un movimiento CC según `monto_origen` de la regla.
5315  * me_prorratedo / mr_prorratedo: permiten varias transacciones del mismo rol cuya suma concilia con el acuerdo (orden mr/me).
5316  */
5317 function montoBaseReglaNegocio(montoOrigen, ctx) {
5318   const mo = String(montoOrigen || '').toLowerCase();
5319   const mrN = Number(ctx.mr) || 0;
5320   const meN = Number(ctx.me) || 0;
5321   const montoT = Number(ctx.montoT) || 0;
5322   if (mo === 'mr') return mrN;
5323   if (mo === 'me') return meN;
5324   if (mo === 'monto_transaccion') return montoT;
5325   if (mo === 'me_prorratedo') {
5326     if (mrN < 1e-12) return 0;
5327     return montoT * (meN / mrN);
5328   }
5329   if (mo === 'mr_prorratedo') {
5330     if (meN < 1e-12) return 0;
5331     return montoT * (mrN / meN);
5332   }
5333   if (mo === 'mr_menos_me') return mrN - meN;
5334   if (mo === 'monto_efectivo_intermediario') return Number(ctx.montoEfectivoInt) || 0;

```



```

5335 | if (mo === 'comision_intermediario') return Number(ctx.comisionIntMonto) || 0;
5336 | return montoT;
5337 | }
5338 |
5339 | /**
5340 |  * Motor CC desde `reglas_de_negocio` (una fila = un movimiento; `entidad_cc` → cliente o intermediario).
5341 |  */
5342 | function aplicarMotorCcDesdeReglasDeNegocio(opts) {
5343 |   const {
5344 |     tipoOperacionCodigo,
5345 |     transacciones,
5346 |     reglasDeNegocio,
5347 |     orden,
5348 |     clienteId,
5349 |     intermediarioId,
5350 |     ordenId,
5351 |     rowsCcCliente,
5352 |     rowsCcInt,
5353 |     fecha,
5354 |     ahora,
5355 |     comisionPandyMonto,
5356 |     comisionPandyMon,
5357 |     comisionIntMonto,
5358 |     comisionIntMon,
5359 |     montoEfectivoInt,
5360 |   } = opts;
5361 |   /** CHEQUE-ARS + int.: importe de la fila sintética «comisión» en CC cliente = mr-me (docs/CHEQUE_ARS_INTERMEDIARIO.md). `comisionPandyMonto` sigue siendo el reparto Pand
5362 |   (caja / trx ganancia / skip ingreso chico). */
5363 |   const spreadAcuerdoClienteCheque = 0,
5364 |   } = opts;
5365 |   if (!reglasDeNegocio || !reglasDeNegocio.length) return;
5366 |   const mr = Number(orden.monto_recibido) || 0;
5367 |   const me = Number(orden.monto_entregado) || 0;
5368 |   const comM = Number(comisionPandyMonto) || 0;
5369 |   const spreadUsdUsd = mr - me;
5370 |   const esUsdUsd = String(tipoOperacionCodigo || '').toUpperCase() === 'USD-USD';
5371 |   /** Si el catálogo trae comisión E,P (pendiente/false) y hay spread, el motor agrega +mr-me; la 2.ª línea del egreso pendiente debe ser -mr (no -me) para no duplicar el spre
5372 |   ad en el saldo (-9700 neto, no -9400). */
5373 |   const tablaTieneComisionUsdUsdEp =
5374 |     esUsdUsd &&
5375 |     !intermediarioId &&
5376 |     spreadUsdUsd >= 1e-6 &&
5377 |     lookupReglasDeNegocio(
5378 |       reglasDeNegocio,
5379 |       tipoOperacionCodigo,
5380 |       'cliente',
5381 |       'pandy',
5382 |       'ingreso',
5383 |       true,
5384 |       'pendiente',
5385 |       false
5386 |     ).length > 0;
5387 |   const epUsdUsdSinIntComisionExplicita =
5388 |     tablaTieneComisionUsdUsdEp &&
5389 |     ingresoDesdeClienteHaciaPandy0IntermediarioEjecutado(transacciones) &&
5390 |     !egresoEntregaAClienteEjecutado(transacciones);
5391 |   /** ci_pc: el egreso P=C es bilateral Pandy=cliente; no debe figurar en CC intermediario (el +mr del C=I ya ancla el flujo frente al int.; -me duplicaba y rompía el saldo vs
5392 |   deudas previas). */
5393 |   const patronUsdIntMotor = esUsdUsd && intermediarioId
5394 |     ? patronInstrumentacionIntDesdeTransacciones(transacciones)
5395 |     : '';
5396 |   (transacciones || []).forEach((t) => {
5397 |     if ((t.concepto || '').includes('Ganancia del acuerdo')) return;
5398 |     const montoT = Number(t.monto) || 0;
5399 |     // Instrumentación USD-USD: el ingreso de cobro lleva monto = mr (acuerdo). `comisiones_orden` solo reparte el spread (Pandy/int.); si existiera otra fila ingreso con mont
5400 |     o = parte Pandy de ese reparto, no duplicar aquí movimientos por transacción (la comisión implícita cliente sale de la fila `es_comision` + `mr_menos_me` en reglas, bloque más
5401 |     abajo). El cobro principal tiene siempre monto = mr; solo omitir cuando el monto coincide con la parte Pandy y es claramente menor que mr (no es el cobro nominal).
5402 |     if (
5403 |       comM >= 1e-6 &&
5404 |       (t.tipo || '').toLowerCase() === 'ingreso' &&
5405 |       String(t.pagador || '').toLowerCase() === 'cliente' &&
5406 |       String(t.cobrador || '').toLowerCase() === 'pandy' &&
5407 |       Math.abs(montoT - comM) < 1e-6 &&
5408 |       montoT < mr - 1e-6
5409 |     ) {
5410 |       return;
5411 |     }
5412 |     const pag = String(t.pagador || '').toLowerCase();
5413 |     const cob = String(t.cobrador || '').toLowerCase();
5414 |     const tipo = (t.tipo || '').toLowerCase();
5415 |     const estado = (t.estado || '').toLowerCase();
5416 |     const contrapartida = contrapartidaEjecutada(transacciones, t.pagador, t.cobrador, t.tipo);
5417 |     const reglasTx = lookupReglasDeNegocio(reglasDeNegocio, tipoOperacionCodigo, pag, cob, tipo, false, estado, contrapartida);
5418 |     if (!reglasTx.length) return;
5419 |     const feI = fechaYEstadoFechaMovimientoCcCajaDesdeTransaccion(t, fecha, ahora);
5420 |     const estadoMov = (estado === 'ejecutada' ? 'cerrado' : 'pendiente');
5421 |     const codOp = String(tipoOperacionCodigo || '').toUpperCase();
5422 |     for (const regla of reglasTx) {
5423 |       let entidad = (regla.entidad_cc == null || String(regla.entidad_cc).trim() === '')
5424 |         ? 'cliente'
5425 |         : String(regla.entidad_cc).toLowerCase();
5426 |       const moR = String(regla.monto_origen || '').toLowerCase();
5427 |       if (codOp === 'CHEQUE-ARS' && moR === 'monto efectivo_intermediario') entidad = 'intermediario';
5428 |       if (entidad === 'cliente' && !clienteId) continue;
5429 |       if (entidad === 'intermediario' && !intermediarioId) continue;
5430 |       // P.E: egreso ejecutado con ingreso aún pendiente (contrapartida=false) no debe duplicar la pata “equivalente” en moneda recibida (netea el compromiso pendiente del ing
5431 |       reso). Solo CC cliente (sin int).
5432 |       if (entidad === 'cliente' && tipo === 'egreso' && estado === 'ejecutada' && !contrapartida) {

```

```

5426|     const monR = String(regla.moneda || '').toUpperCase();
5427|     const mo = String(regla.monto_origen || '').toLowerCase();
5428|     if (codOp === 'USD-ARS' && monR === 'USD' && mo === 'mr_prorrrateado') continue;
5429|     if (codOp === 'ARS-USD' && monR === 'ARS' && mo === 'mr_prorrrateado') continue;
5430|   }
5431|   let montoOrigenParaBase = regla.monto_origen;
5432|   if (
5433|     epUsdUsdSinIntComisionExplicita &&
5434|     entidad === 'cliente' &&
5435|     tipo === 'egreso' &&
5436|     estado === 'pendiente' &&
5437|     contrapartida &&
5438|     String(regla.monto_origen || '').toLowerCase() === 'me' &&
5439|     Number(regla.linea) === 1
5440|   ) {
5441|     montoOrigenParaBase = 'mr';
5442|   }
5443|   const base = montoBaseReglaNegocio(montoOrigenParaBase, {
5444|     mr, me, montoT, montoEfectivoInt,
5445|     comisionIntMonto: Number(comisionIntMonto) || 0,
5446|   });
5447|   const montoCc = Number(regla.signo) * base;
5448|   if (Math.abs(montoCc) < 1e-12) continue;
5449|   const moneda = String(regla.moneda || 'USD').toUpperCase();
5450|   const ley = regla.concepto_leyenda || 'cobro_realizado';
5451|   const rowBase = {
5452|     orden_id: ordenId,
5453|     transaccion_id: t.id,
5454|     transaccion_numero: t.numero != null ? t.numero : null,
5455|     concepto: conceptoCcLeyenda(ley, orden.numero, t.numero != null ? t.numero : null),
5456|     fecha: feT.fecha,
5457|     usuario_id: currentUserId,
5458|     moneda,
5459|     monto: montoCc,
5460|     estado: estadoMov,
5461|     estado_fecha: feT.estado_fecha,
5462|     incluir_en_detalle: coercePgBooleanStrict(regla.incluir_en_detalle),
5463|     ...montosCcPorMoneda(moneda, montoCc),
5464|   };
5465|   if (entidad === 'intermediario') {
5466|     if (
5467|       patronUsdIntMotor === 'ci_pc' &&
5468|       tipo === 'egreso' &&
5469|       pag === 'pandy' &&
5470|       cob === 'cliente'
5471|     ) {
5472|       continue;
5473|     }
5474|     rowsCcInt.push({
5475|       intermediario_id: intermediarioId,
5476|       ...rowBase,
5477|     });
5478|   } else {
5479|     rowsCcCliente.push({
5480|       cliente_id: clienteId,
5481|       ...rowBase,
5482|     });
5483|   }
5484| }
5485| });
5486| // Comisión implícita USD-USD: `reglas_de_negocio` fila es_comision + monto_origen mr_menos_me (ver reglas_de_negocio_tabla / migraciones). Condición de negocio: acuerdo con
spread (mr > me) en la orden; par cliente Cerrado. No usar comisiones_orden como "llave" para esta fila - el monto sale de mr-me (tabla + orden), no del reparto Pandy/interme
diario.
5487| let nroTransComisionConceptoUsd = null;
5488| if (esUsdUsd) {
5489|   const trIngresoClienteAcuerdoUsd = (transacciones || []).
5490|     .filter((t) =>
5491|       (t.tipo || '').toLowerCase() === 'ingreso' &&
5492|       String(t.pagador || '').toLowerCase() === 'cliente' &&
5493|       (String(t.cobrador || '').toLowerCase() === 'pandy' || String(t.cobrador || '').toLowerCase() === 'intermediario') &&
5494|       (t.estado || '').toLowerCase() === 'ejecutada' &&
5495|       Math.abs(Number(t.monto) - comM) >= 1e-6
5496|     )
5497|     .sort((a, b) => (Number(a.numero) || 0) - (Number(b.numero) || 0))[0];
5498|   nroTransComisionConceptoUsd =
5499|     trIngresoClienteAcuerdoUsd && trIngresoClienteAcuerdoUsd.numero != null ? trIngresoClienteAcuerdoUsd.numero : null;
5500| }
5501| const feComUsd = esUsdUsd
5502|   ? (nroTransComisionConceptoUsd != null
5503|     ? fechaYEstadoFechaMovimientoCcCajaDesdeNumeroTransaccion(transacciones, nroTransComisionConceptoUsd, fecha, ahora)
5504|     : fechaYEstadoFechaMovimientoCcCajaDesdeUltimaEjecutada(transacciones, fecha, ahora))
5505|   : { fecha, estado_fecha: ahora };
5506| if (esUsdUsd && clienteId && spreadUsdUsd >= 1e-6) {
5507|   const ingresoCobroClienteEjecutado = ingresoDesdeClienteHaciaPandyOIntermediarioEjecutado(transacciones);
5508|   const egresoEntregaClienteEjecutado = egresoEntregaClienteEjecutado(transacciones);
5509|   const parClienteCerrado = ingresoCobroClienteEjecutado && egresoEntregaClienteEjecutado;
5510|   let reglaCom = null;
5511|   if (parClienteCerrado) {
5512|     // cp ic: ingreso Cliente-Pandy. ci_pc: ingreso Cliente-Intermediario (misma comisión mr-me en tabla).
5513|     reglaCom = lookupReglasDeNegocio(
5514|       reglasDeNegocio,
5515|       tipoOperacionCodigo,
5516|       'cliente',
5517|       'intermediario',
5518|       'ingreso',
5519|       true,
5520|       'ejecutada',

```

```

5521|         true
5522|     )[0]
5523|     || lookupReglasDeNegocio(
5524|         reglasDeNegocio,
5525|         tipoOperacionCodigo,
5526|         'cliente',
5527|         'pandy',
5528|         'ingreso',
5529|         true,
5530|         'ejecutada',
5531|         true
5532|     )[0];
5533| } else if (
5534|     !intermediarioId &&
5535|     ingresoCobroClienteEjecutado &&
5536|     !egresoEntregaClienteEjecutado
5537| ) {
5538|     // Sin int. E,P: comisión mr-me en CC como pendiente (visible; no entra en G/P hasta cerrar par).
5539|     reglaCom = lookupReglasDeNegocio(
5540|         reglasDeNegocio,
5541|         tipoOperacionCodigo,
5542|         'cliente',
5543|         'pandy',
5544|         'ingreso',
5545|         true,
5546|         'pendiente',
5547|         false
5548|     )[0];
5549| }
5550| if (reglaCom) {
5551|     const base = montoBaseReglaNegocio(reglaCom.monto_origen, {
5552|         mr, me, montoI: 0,
5553|         comisionIntMonto: Number(comisionIntMonto) || 0,
5554|     });
5555|     const montoCc = Number(reglaCom.signo) * base;
5556|     if (Math.abs(montoCc) >= 1e-12) {
5557|         const moneda = String(reglaCom.moneda || 'USD').toUpperCase();
5558|         const ley = reglaCom.concepto_leyenda || 'comision_acuerdo';
5559|         const cerrado = parClienteCerrado;
5560|         rowsCcCliente.push({
5561|             cliente_id: clienteId,
5562|             orden_id: ordenId,
5563|             transaccion_id: null,
5564|             transaccion_numero: null,
5565|             concepto: conceptoCcLeyenda(ley, orden.numero, nroTransComisionConceptoUsd),
5566|             fecha: feComUsd.fecha,
5567|             usuario_id: currentUserId,
5568|             moneda,
5569|             monto: montoCc,
5570|             estado: cerrado ? 'cerrado' : 'pendiente',
5571|             estado_fecha: feComUsd.estado_fecha,
5572|             incluir_en_detalle: coercePgBooleanStrict(reglaCom.incluir_en_detalle),
5573|             ...montosCCPorMoneda(moneda, montoCc),
5574|         });
5575|     }
5576| }
5577| }
5578| // Comisión intermediario USD-USD (comisiones_orden). Par cerrado: ingreso C-P/I + entrega al cliente ejecutados.
5579| // cp_ic P,E: ya entregó Int-Cliente pero el cobro C-P sigue pendiente → devengar comisión int. (regla contrapartida_ejecutada false en tabla).
5580| if (esUsdUsd && intermediarioId && Number(comisionIntMonto) >= 1e-6) {
5581|     const parClienteCerradoUsdInt =
5582|         ingresoDesdeClienteHaciaPandy0IntermediarioEjecutado(transacciones) &&
5583|         egresoEntregaAClienteEjecutado(transacciones);
5584|     const patronIntUsdCom = patronInstrumentacionIntDesdeTransacciones(transacciones);
5585|     const comisionIntCpIcParcial =
5586|         patronIntUsdCom === 'cp_ic' &&
5587|         egresoIntermediarioAClienteEjecutado(transacciones) &&
5588|         !parClienteCerradoUsdInt;
5589|     const estadoComIntLookup =
5590|         parClienteCerradoUsdInt || comisionIntCpIcParcial ? 'ejecutada' : 'pendiente';
5591|     const contrapartidaComIntLookup = comisionIntCpIcParcial ? false : parClienteCerradoUsdInt;
5592|     const reglaComIntUsd = lookupReglasDeNegocio(
5593|         reglasDeNegocio,
5594|         tipoOperacionCodigo,
5595|         'pandy',
5596|         'intermediario',
5597|         'egreso',
5598|         true,
5599|         estadoComIntLookup,
5600|         contrapartidaComIntLookup
5601|     )[0];
5602|     if (reglaComIntUsd && coercePgBooleanStrict(reglaComIntUsd.incluir_en_detalle)) {
5603|         const baseInt = montoBaseReglaNegocio(reglaComIntUsd.monto_origen, {
5604|             mr, me, montoI: 0,
5605|             comisionIntMonto: Number(comisionIntMonto) || 0,
5606|         });
5607|         const rawComInt = Number(reglaComIntUsd.signo) * baseInt;
5608|         // Misma convención que cp_ic: signo de la tabla (Pandy → intermediario) = deuda de Pandy hacia el int. en la suma algebraica.
5609|         const montoCCInt = rawComInt;
5610|         if (Math.abs(montoCCInt) >= 1e-12) {
5611|             const monedaInt = String(comisionIntMon || reglaComIntUsd.moneda || 'USD').toUpperCase();
5612|             const cerradoInt = parClienteCerradoUsdInt;
5613|             rowsCcInt.push({
5614|                 intermediario_id: intermediarioId,
5615|                 orden_id: ordenId,
5616|                 transaccion_id: null,
5617|                 transaccion_numero: null,

```

```

5618     concepto: conceptoCcLeyenda('comision_acuerdo', orden.numero, nroTransComisionConceptoUsd),
5619     fecha: feComUsd.fecha,
5620     usuario_id: currentUserId,
5621     moneda: monedaInt,
5622     monto: montoCcInt,
5623     estado: cerradoInt ? 'cerrado' : 'pendiente',
5624     estado.fecha: feComUsd.estado.fecha,
5625     incluir_en_detalle: coercePgBooleanStrict(reglaComIntUsd.incluir_en_detalle),
5626     ...montosCCPorMoneda(monedaInt, montoCcInt),
5627   });
5628 }
5629 }
5630 }
5631 // Comisión CHEQUE-ARS (comisiones_orden Pandy e intermediario; filas es_comision + condicion_estado_comision).
5632 const codOpCh = String(tipoOperacionCodigo || '').toUpperCase();
5633 const spreadClienteCheque =
5634   codOpCh === 'CHEQUE-ARS' && intermediarioId && Number(comisionSpreadAcuerdoClienteCheque) >= 1e-6
5635   ? Number(comisionSpreadAcuerdoClienteCheque)
5636   : 0;
5637 const montoComisionLineaCcClienteCheque = spreadClienteCheque >= 1e-6 ? spreadClienteCheque : comM;
5638 const nroTransComisionConceptoChequeArs =
5639   codOpCh === 'CHEQUE-ARS' ? nroTransIngresoClientePandyPrincipalParaComisionConcepto(transacciones, comM) : null;
5640 const feComChequeArs = codOpCh === 'CHEQUE-ARS'
5641   ? (nroTransComisionConceptoChequeArs != null
5642     ? fechaEstadoFechaMovimientoCcCajaDesdeNumeroTransaccion(transacciones, nroTransComisionConceptoChequeArs, fecha, ahora)
5643     : fechaEstadoFechaMovimientoCcCajaDesdeUltimaEjecutada(transacciones, fecha, ahora))
5644   : { fecha, estado.fecha: ahora };
5645 if (codOpCh === 'CHEQUE-ARS' && clienteId && montoComisionLineaCcClienteCheque >= 1e-6) {
5646   const parClienteCerrado =
5647     (transacciones || []).some((t) =>
5648       (t.tipo || '').toLowerCase() === 'ingreso' &&
5649       String(t.pagador || '').toLowerCase() === 'cliente' &&
5650       String(t.cobrador || '').toLowerCase() === 'pandy' &&
5651       (t.estado || '').toLowerCase() === 'ejecutada'
5652     ) &&
5653     (transacciones || []).some((t) =>
5654       (t.tipo || '').toLowerCase() === 'egreso' &&
5655       String(t.pagador || '').toLowerCase() === 'pandy' &&
5656       String(t.cobrador || '').toLowerCase() === 'cliente' &&
5657       (t.estado || '').toLowerCase() === 'ejecutada'
5658     )
5659   );
5660   const condicionPandy = getCondicionComisionReglasNegocio(reglasDeNegocio, tipoOperacionCodigo, 'cliente', 'pandy', 'ingreso');
5661   const estadoComPandy = condicionPandy
5662     ? estadoEfectivoComision(transacciones, condicionPandy)
5663     : (parClienteCerrado ? 'ejecutada' : 'pendiente');
5664   const reglaComPandy = lookupReglasDeNegocio(
5665     reglasDeNegocio,
5666     tipoOperacionCodigo,
5667     'cliente',
5668     'pandy',
5669     'ingreso',
5670     true,
5671     estadoComPandy,
5672     parClienteCerrado
5673   )[0];
5674   // Comisión Pandy: estado efectivo ejecutada si Tx1 o Tx2 ejecutada (ver estadoEfectivoComision par_cliente). P,P,P,P sin ninguna → pendiente → no fila.
5675   if (estadoComPandy === 'ejecutada' && reglaComPandy && coercePgBooleanStrict(reglaComPandy.incluir_en_detalle)) {
5676     const signo = Number(reglaComPandy.signo) != null ? Number(reglaComPandy.signo) : 1;
5677     const cerrado = estadoComPandy === 'ejecutada';
5678     const moneda = String(comisionPandyMon || reglaComPandy.moneda || 'ARS').toUpperCase();
5679     rowsCcCliente.push({
5680       cliente_id: clienteId,
5681       orden_id: ordenId,
5682       transaccion_id: null,
5683       transaccion_numero: null,
5684       concepto: conceptoCcLeyenda('comision_acuerdo', orden.numero, nroTransComisionConceptoChequeArs),
5685       fecha: feComChequeArs.fecha,
5686       usuario_id: currentUserId,
5687       moneda: moneda,
5688       monto: signo * montoComisionLineaCcClienteCheque,
5689       estado: cerrado ? 'cerrado' : 'pendiente',
5690       estado.fecha: feComChequeArs.estado.fecha,
5691       incluir_en_detalle: coercePgBooleanStrict(reglaComPandy.incluir_en_detalle),
5692       ...montosCCPorMoneda(moneda, signo * montoComisionLineaCcClienteCheque)
5693     });
5694   }
5695 }
5696 if (codOpCh === 'CHEQUE-ARS' && intermediarioId && Number(comisionIntMonto) >= 1e-6) {
5697   const condicion = getCondicionComisionReglasNegocio(reglasDeNegocio, tipoOperacionCodigo, 'pandy', 'intermediario', 'egreso');
5698   const estadoEf = condicion
5699     ? estadoEfectivoComision(transacciones, condicion)
5700     : estadoEfectivoComision(transacciones, 'par_pandy_int');
5701   const parIntCerrado = estadoEf === 'ejecutada';
5702   const reglaComInt = lookupReglasDeNegocio(
5703     reglasDeNegocio,
5704     tipoOperacionCodigo,
5705     'pandy',
5706     'intermediario',
5707     'egreso',
5708     true,
5709     estadoEf,
5710     parIntCerrado
5711   )[0];
5712   // Comisión Intermediario en CC solo cuando Tx3 o Tx4 ejecutada (par_pandy_int efectivo); no fila con todo pendiente (E2E P,P,P,P).
5713   if (estadoEf === 'ejecutada' && reglaComInt && coercePgBooleanStrict(reglaComInt.incluir_en_detalle)) {
5714     const signo = Number(reglaComInt.signo) != null ? Number(reglaComInt.signo) : 1;
5715     const comInt = Number(comisionIntMonto) || 0;

```

```

5715 |     const monCom = String(comisionIntMon || 'ARS').toUpperCase();
5716 |     rowsCcInt.push({
5717 |       intermediario_id: intermediarioId,
5718 |       orden_id: ordenId,
5719 |       transaccion_id: null,
5720 |       transaccion_numero: null,
5721 |       concepto: conceptoCcLeyenda('comision_acuerdo', orden.numero, nroTransComisionConceptoChequeArs),
5722 |       fecha: feComChequeArs.fecha,
5723 |       usuario_id: currentUserId,
5724 |       moneda: monCom,
5725 |       monto: signo * comInt,
5726 |       estado: parIntCerrado ? 'cerrado' : 'pendiente',
5727 |       estado_fecha: feComChequeArs.estado_fecha,
5728 |       incluir_en_detalle: coercePgBooleanStrict(reglaComInt.incluir_en_detalle),
5729 |       ...montosCcPorMoneda(monCom, signo * comInt)
5730 |     });
5731 |   }
5732 | }
5733 | }
5734 |
5735 | /** Actualiza las tarjetas de saldo de cuenta corriente: etiqueta "Saldo a favor" / "Saldo negativo" y monto. saldos = { USD, EUR, ARS } o null para reset. */
5736 | function setCcSaldoCards(saldos) {
5737 |   const monedas = ['USD', 'EUR', 'ARS'];
5738 |   monedas.forEach(moneda) => {
5739 |     const labelEl = document.getElementById('cc-saldo-label-' + moneda.toLowerCase());
5740 |     const valorEl = document.getElementById('cc-saldo-' + moneda.toLowerCase());
5741 |     if (!valorEl) return;
5742 |     if (saldos == null) {
5743 |       if (labelEl) labelEl.textContent = '';
5744 |       valorEl.textContent = '-';
5745 |       valorEl.className = 'valor';
5746 |       return;
5747 |     }
5748 |     const saldo = Number(saldos[moneda]) || 0;
5749 |     if (labelEl) {
5750 |       labelEl.textContent = saldo >= 0 ? 'Saldo a favor' : 'Saldo negativo';
5751 |     }
5752 |     valorEl.textContent = formatMonto(saldo >= 0 ? saldo : -saldo, moneda);
5753 |     valorEl.className = 'valor ' + (saldo >= 0 ? 'positivo' : 'negativo');
5754 |   });
5755 | }
5756 |
5757 | /**
5758 |  * Recalcula CC y caja desde orden + transacciones para todas las órdenes que tienen instrumentación.
5759 |  * Así, al refrescar la página o abrir Cuenta corriente, los movimientos quedan derivados de la fuente de verdad (no hace falta truncar).
5760 |  */
5761 | function sincronizarCcYCajaParaTodasLasOrdenesConInstrumentacion() {
5762 |   return client.from('instrumentacion').select('orden_id').then((r) => {
5763 |     const ordenIds = (r.data || []).map(x) => x.orden_id).filter(Boolean);
5764 |     if (ordenIds.length === 0) return Promise.resolve();
5765 |     // Encadenar (no Promise.all): evita picos de carga y condiciones de carrera en DB al leer transacciones
5766 |     // mientras otra orden termina su RPC sync_cc_caja_orden; el detalle/modal y el resumen quedan más alineados.
5767 |     return ordenIds.reduce(
5768 |       (prev, ordenId) => prev.then(() => sincronizarCcYCajaDesdeOrden(ordenId)),
5769 |       Promise.resolve()
5770 |     );
5771 |   });
5772 | }
5773 |
5774 | function loadCuentaCorriente(opts) {
5775 |   opts = opts || {};
5776 |   const esRecargaPostSync = opts.esRecargaPostSync === true;
5777 |   /** Tras sync explícito (p. ej. botón Refrescar): solo leer movimientos, no volver a borrar/reinsertar toda la CC. */
5778 |   const skipSyncGlobal = opts.skipSyncGlobal === true;
5779 |   const loadingEl = document.getElementById('cc-loading');
5780 |   const contenido = document.getElementById('cc-contenido');
5781 |   const tbody = document.getElementById('cc-resumen-tbody');
5782 |   const panelSaldos = document.getElementById('cc-panel-saldos');
5783 |   const panelMov = document.getElementById('cc-panel-movimientos');
5784 |   if (!contenido || !tbody) return;
5785 |
5786 |   const miTicket = ++ccCargaSerial;
5787 |   const silentCc = isPandiBackgroundRefresh();
5788 |   const loadingShownAtCc = silentCc ? 0 : Date.now();
5789 |   if (!silentCc) {
5790 |     if (loadingEl) loadingEl.style.display = 'block';
5791 |     contenido.style.display = 'none';
5792 |     if (panelSaldos) panelSaldos.style.display = 'none';
5793 |     if (panelMov) panelMov.style.display = 'none';
5794 |   }
5795 |
5796 |   // Refresco automático (~30 s): solo SELECT de movimientos. El sync global borra y reinserta CC/caja por cada orden;
5797 |   // hacerlo en cada tick vacía la tabla en lecturas concurrentes y parpadea los saldos sin que haya cambios reales.
5798 |   const debeCorrerSyncGlobalAntesDeLeer = !esRecargaPostSync && !silentCc && !skipSyncGlobal;
5799 |   if (debeCorrerSyncGlobalAntesDeLeer) {
5800 |     sincronizarCcYCajaParaTodasLasOrdenesConInstrumentacion()
5801 |       .catch(() => {})
5802 |       .then(() => {
5803 |         if (currentVistaId !== 'vista-cuenta-corriente') return;
5804 |         const ordenModal = document.getElementById('modal-orden-backdrop');
5805 |         if (ordenModal && ordenModal.classList.contains('activo')) return;
5806 |         const prevBg = pandiBackgroundRefreshActive;
5807 |         pandiBackgroundRefreshActive = true;
5808 |         loadCuentaCorriente({ esRecargaPostSync: true }).finally(() => { pandiBackgroundRefreshActive = prevBg; });
5809 |       });
5810 |   }
5811 | }

```

```

5812| // Primera pintada: fetch en paralelo al sync global; al terminar el sync, una recarga silenciosa (esRecargaPostSync) alinea movimientos con el modelo sin bloquear el primer
5813| render.
5814| return Promise.all([
5815|   client.from('clientes').select('id, nombre').order('nombre', { ascending: true }),
5816|   client.from('intermediarios').select('id, nombre').order('nombre', { ascending: true }),
5817|   client.from('movimientos_cuenta_corriente').select('id, cliente_id, orden_id, transaccion_id, transaccion_numero, fecha, moneda, monto, concepto, monto_usd, monto_ars, m
5818|   onto_eur, estado, incluir_en_detalle, es_movimiento_manual, manual_tip_movimiento, usuario_id' + CC_MOV_MANUAL_PAG_COB_COLS),
5819|   client.from('movimientos_cuenta_corriente_intermediario').select('id, intermediario_id, orden_id, transaccion_id, transaccion_numero, fecha, moneda, monto, concepto, mon
5820|   to_usd, monto_ars, monto_eur, estado, incluir_en_detalle, es_movimiento_manual, manual_tip_movimiento, usuario_id' + CC_MOV_MANUAL_PAG_COB_COLS),
5821| ])
5822| .then([rClientes, rInt, rMovCli, rMovInt]) => {
5823|   const clientes = rClientes.data || [];
5824|   const intermediarios = rInt.data || [];
5825|   const movCliRaw = rMovCli.data || [];
5826|   const movIntRaw = rMovInt.data || [];
5827|   const transaccionIds = [...new Set([...(movCliRaw.map(m) => m.transaccion_id), ...(movIntRaw.map(m) => m.transaccion_id)])].filter(Boolean)];
5828|   const ordenIds = [...new Set([...(movCliRaw.map(m) => m.orden_id), ...(movIntRaw.map(m) => m.orden_id)])].filter(Boolean)];
5829|   // Una sola carga de pendientes CC: instrumentación (global + por órdenes con movimientos) → órdenes → transacciones pendientes. Regla unificada para cliente e intermediar
5830|   io.
5831|   const promPendientesCcGlobal = Promise.all([
5832|     client.from('instrumentacion').select('id, orden_id',
5833|     ordenIds.length > 0 ? client.from('instrumentacion').select('id, orden_id').in('orden_id', ordenIds) : Promise.resolve({ data: [] })),
5834|   ]).then([rInstAll, rInstFromOrdenes]) => {
5835|     const instAll = (rInstAll.data || []).filter((i) => i.iid && i.id);
5836|     const instFromOrdenes = (rInstFromOrdenes.data || []).filter((i) => i.iid && i.id);
5837|     const instById = {};
5838|     instAll.forEach((i) => { instById[i.iid] = i; });
5839|     instFromOrdenes.forEach((i) => { instById[i.iid] = i; });
5840|     const inst = Object.values(instById);
5841|     const ordenIdsFromInst = [...new Set(inst.map((i) => i.orden_id).filter(Boolean))];
5842|     if (ordenIdsFromInst.length === 0) return { pendienteClienteAjusteByCli: {}, pendientePandyDebeIntByInt: {} };
5843|     return client.from('ordenes').select('id, cliente_id, intermediario_id, monto_recibido, monto_entregado, moneda_recibida, moneda_entregada, tipo_operacion_id, tipos_oper
5844|     acion(codigo, usa_intermediario)')
5845|     .in('id', ordenIdsFromInst)
5846|     .then((rO) => {
5847|       if (rO.error) return { pendienteClienteAjusteByCli: {}, pendientePandyDebeIntByInt: {} };
5848|       const ordenes = rO.data || [];
5849|       const ordenById = Object.fromEntries(ordenes.map(o => [o.id, o]));
5850|       const instIdSet = new Set(inst.map(i) => i.iid);
5851|       const instIdToOrdenId = Object.fromEntries(inst.map(i) => [i.iid, i.orden_id]);
5852|       return client.from('instrumentacion').select('orden_id, multicontraparte_manual').in('orden_id', ordenIdsFromInst).then((rMc) => {
5853|         const mcOrdenIds = new Set((rMc.data || []).filter((row) => row.multicontraparte_manual).map((row) => row.orden_id));
5854|         const ordenMcElegibleById = {};
5855|         ordenes.forEach(o => {
5856|           const toJ = o.tipos_operacion && (Array.isArray(o.tipos_operacion) ? o.tipos_operacion[0] : o.tipos_operacion);
5857|           ordenMcElegibleById[o.id] = mcOrdenIds.has(o.id) && esTipoOpMulticontraparteElegibleDesdeOrden(o, toJ);
5858|         });
5859|         // Cargar TODAS las transacciones pendientes y filtrar por instrumentación en JS (evita límite .in y RLS por orden).
5860|         return client.from('transacciones').select('id, instrumentacion_id, estado, pagador, cobrador, monto, moneda')
5861|         .eq('estado', 'pendiente')
5862|         .then((rTrx) => {
5863|           if (rTrx.error) return { pendienteClienteAjusteByCli: {}, pendientePandyDebeIntByInt: {} };
5864|           const pendientes = (rTrx.data || []).filter((t) => (t.estado || '').toString().toLowerCase() === 'pendiente');
5865|           const trx = pendientes.filter((t) => t.instrumentacion_id && instIdSet.has(t.instrumentacion_id));
5866|           const byInst = {};
5867|           trx.forEach((t) => {
5868|             const id = t.instrumentacion_id;
5869|             if (!byInst[id]) byInst[id] = [];
5870|             byInst[id].push(t);
5871|           });
5872|           const pendienteClienteAjusteByCli = {};
5873|           const pendientePandyDebeIntByInt = {};
5874|           trx.forEach((t) => {
5875|             const ordenId = instIdToOrdenId[t.instrumentacion_id];
5876|             const orden = ordenId ? ordenById[ordenId] : null;
5877|             if (!orden) return;
5878|             const transaccionesOrden = byInst[t.instrumentacion_id] || [];
5879|             const mcEl = !ordenMcElegibleById[ordenId];
5880|             const { cliente, intermediario } = contribucionPendienteCcUnificada(t, orden, transaccionesOrden, mcEl);
5881|             if (intermediario && intermediario.id) {
5882|               if (!pendientePandyDebeIntByInt[intermediario.id]) pendientePandyDebeIntByInt[intermediario.id] = { USD: 0, EUR: 0, ARS: 0 };
5883|               ['USD', 'EUR', 'ARS'].forEach(mon => { pendientePandyDebeIntByInt[intermediario.id][mon] += intermediario.delta[mon] || 0; });
5884|             }
5885|             if (cliente && cliente.id) {
5886|               if (!pendienteClienteAjusteByCli[cliente.id]) pendienteClienteAjusteByCli[cliente.id] = { USD: 0, EUR: 0, ARS: 0 };
5887|               ['USD', 'EUR', 'ARS'].forEach(mon => { pendienteClienteAjusteByCli[cliente.id][mon] += cliente.delta[mon] || 0; });
5888|             }
5889|           });
5890|           return { pendienteClienteAjusteByCli, pendientePandyDebeIntByInt };
5891|         });
5892|       });
5893|     });
5894|   ).catch(() => ({ pendienteClienteAjusteByCli: {}, pendientePandyDebeIntByInt: {} }));
5895|   return Promise.all([
5896|     transaccionIds.length > 0 ? client.from('transacciones').select('id, estado, tipo, pagador, cobrador, monto, moneda, pagador_cliente_id, cobrador_cliente_id, pagador_int
5897|     ermediario_id, cobrador_intermediario_id').in('id', transaccionIds) : Promise.resolve({ data: [] }),
5898|     ordenIds.length > 0 ? client.from('instrumentacion').select('id, orden_id').in('orden_id', ordenIds) : Promise.resolve({ data: [] }),
5899|     ordenIds.length > 0 ? client.from('ordenes').select('id, numero, cliente_id, intermediario_id, tipo_operacion_id, tipos_operacion(codigo, nombre, icono_modo, icono_url_p
5900|     ublica, moneda_in, moneda_out, usa_intermediario), monto_recibido, monto_entregado, moneda_recibida, moneda_entregada, tasa_descuento_intermediario').in('id', ordenIds) : Prom
5901|     ise.resolve({ data: [] })
5902|   ]).then([rTr, rInst, rOrdenes, pendientesResult]) => {
5903|     const trById = {};
5904|     const trPagadorById = {};
5905|     const trCobradorById = {};
5906|     const trTipoById = {};

```

```

5901     const trMontoById = {};
5902     (rTr.data || []).forEach((t) => {
5903         if (t.id == null || t.id === '') return;
5904         const id = String(t.id);
5905         trById[id] = t.estado;
5906         trPagadorById[id] = (t.pagador || '').toString().toLowerCase();
5907         if (t.cobrador != null) trCobradorById[id] = String(t.cobrador).toLowerCase();
5908         if (t.tipo != null) trTipoById[id] = String(t.tipo).toLowerCase();
5909         trMontoById[id] = { monto: t.monto, moneda: t.moneda };
5910     });
5911     const instByOrden = {};
5912     (rInst.data || []).forEach((i) => { instByOrden[i.orden_id] = i.id; });
5913     const ordenNumeroById = Object.fromEntries((rOrdenes.data || []).map((o) => [o.id, o.numero]));
5914     const ordenById = Object.fromEntries((rOrdenes.data || []).map((o) => [o.id, o]));
5915     const instIds = (rInst.data || []).map((i) => i.id).filter(Boolean);
5916     const promTrInst = instIds.length > 0
5917         ? client.from('transacciones').select('id, instrumentacion_id, estado, tipo, pagador, cobrador, monto, moneda, pagador_cliente_id, cobrador_cliente_id, pagador_interme
diario_id, cobrador_intermediario_id').in('instrumentacion_id', instIds)
5918         : Promise.resolve({ data: [] });
5919     return promTrInst.then((rTrInst) => ({
5920         rTrInst,
5921         rTrMovsRaw: rTr.data || [],
5922         pendientePandyDebeIntByInt: (pendientesResult && pendientesResult.pendientePandyDebeIntByInt) || {},
5923         pendienteClienteAjusteByCli: (pendientesResult && pendientesResult.pendienteClienteAjusteByCli) || {},
5924         trById,
5925         instByOrden,
5926         ordenNumeroById,
5927         trPagadorById,
5928         trCobradorById,
5929         trTipoById,
5930         ordenById,
5931         trMontoById,
5932     }));
5933     }).then(({ rTrInst, rTrMovsRaw, pendientePandyDebeIntByInt, pendienteClienteAjusteByCli, trById, instByOrden, ordenNumeroById, trPagadorById, trCobradorById, trTipoById, o
rdenById, trMontoById }) => {
5934         const pagadorClienteIdByTrx = {};
5935         const cobradorClienteIdByTrx = {};
5936         const pagadorIntermediarioIdByTrx = {};
5937         const cobradorIntermediarioIdByTrx = {};
5938         function ingestTrxParticipanteIds(t) {
5939             if (!t || t.id == null || t.id === '') return;
5940             const kid = String(t.id);
5941             if (t.pagador_cliente_id) pagadorClienteIdByTrx[kid] = t.pagador_cliente_id;
5942             if (t.cobrador_cliente_id) cobradorClienteIdByTrx[kid] = t.cobrador_cliente_id;
5943             if (t.pagador_intermediario_id) pagadorIntermediarioIdByTrx[kid] = t.pagador_intermediario_id;
5944             if (t.cobrador_intermediario_id) cobradorIntermediarioIdByTrx[kid] = t.cobrador_intermediario_id;
5945         }
5946         (rTrMovsRaw || []).forEach(ingestTrxParticipanteIds);
5947         (rTrInst.data || []).forEach(ingestTrxParticipanteIds);
5948         ccAbsorberMetaTransaccionesEnMaps(trPagadorById, trCobradorById, trTipoById, rTrInst.data || []);
5949         const trParticipanteIdsByTrx = {
5950             pagadorClienteIdByTrx,
5951             cobradorClienteIdByTrx,
5952             pagadorIntermediarioIdByTrx,
5953             cobradorIntermediarioIdByTrx,
5954         };
5955         const orderHasEjecutada = {};
5956         (rTrInst.data || []).forEach((t) => {
5957             const ordenId = Object.keys(instByOrden || {}).find((oid) => instByOrden[oid] === t.instrumentacion_id);
5958             if (ordenId && t.estado === 'ejecutada') orderHasEjecutada[ordenId] = true;
5959         });
5960         return { rTrInst, instByOrden, trById, orderHasEjecutada, ordenNumeroById, trPagadorById, trCobradorById, trTipoById, ordenById, trMontoById, pendientePandyDebeIntByInt,
pendienteClienteAjusteByCli, trParticipanteIdsByTrx };
5961     }).then(({ rTrInst, instByOrden, trById, orderHasEjecutada, ordenNumeroById, trPagadorById, trCobradorById, trTipoById, ordenById, trMontoById, pendientePandyDebeIntByInt,
pendienteClienteAjusteByCli, trParticipanteIdsByTrx }) => {
5962         // Incluir en saldo: compromisos (Compromiso de Pago / Compromiso a Cobrar o legacy "Compromiso") y realizados (Cobro/Pago Realizado o legacy "Compromiso Saldado").
5963         function incluirEnSaldo(m, trEstados, ordEjecutada) {
5964             if (m.estado === 'anulado') return false;
5965             const concepto = (m.concepto || '').toString();
5966             if (concepto.includes('Cobro Realizado') || concepto.includes('Pago Realizado')) return true;
5967             if (concepto.includes('Compromiso de Pago') || concepto.includes('Compromiso a Cobrar')) return true;
5968             if (concepto.includes('Compromiso Saldado')) return true;
5969             if (concepto.includes('Compromiso') && !concepto.includes('Compromiso Saldado')) return true;
5970             return true;
5971         }
5972         const movCli = movCliRaw.filter((m) => incluirEnSaldo(m, trById, orderHasEjecutada));
5973         const movInt = movIntRaw.filter((m) => incluirEnSaldo(m, trById, orderHasEjecutada));
5974         // Para intermediario: usar monto real de la transacción en "Compromiso a Cobrar" pendiente (Int-Pandy), así el resumen refleja lo que el intermediario aún debe.
5975         const movIntEnriched = (movInt || []).map((m) => {
5976             if (m.estado === 'pendiente' && (m.concepto || '').includes('Compromiso a Cobrar')) {
5977                 const t = trMontoById[ccTrxMapKey(m.transaccion_id)];
5978                 if (t != null && t.monto != null) {
5979                     const mon = ((t.moneda || m.moneda || 'ARS') || '').toString().toUpperCase();
5980                     const val = Number(t.monto) || 0;
5981                     return {
5982                         ...m,
5983                         monto: val,
5984                         monto_usd: mon === 'USD' ? val : 0,
5985                         monto_ars: mon === 'ARS' ? val : 0,
5986                         monto_eur: mon === 'EUR' ? val : 0,
5987                     };
5988                 }
5989             }
5990             return m;
5991         });
5992         // Saldos por modelo CC (docs/CC_MODELO_REFERENCIA.md): por orden, modelo ARS-ARS+int o suma de movimientos; responde a cualquier cambio y orden.
5993         // Saldo por orden (precomp): suma de **todos** los movimientos no anulados (coherente con lo que se ve en Movimientos).

```

```

5994 function sumMovsPorMoneda(movs) {
5995   const acc = { USD: 0, EUR: 0, ARS: 0 };
5996   (movs || []).forEach((m) => {
5997     if ((m.estado || '').toString().toLowerCase() === 'anulado') return;
5998     const has = m.monto_usd != null || m.monto_ars != null || m.monto_eur != null;
5999     if (has) {
6000       acc.USD += Number(m.monto_usd) || 0;
6001       acc.ARS += Number(m.monto_ars) || 0;
6002       acc.EUR += Number(m.monto_eur) || 0;
6003     } else {
6004       const mon = (m.moneda || 'ARS').toString().toUpperCase();
6005       const val = Number(m.monto) || 0;
6006       if (mon === 'USD') acc.USD += val; else if (mon === 'EUR') acc.EUR += val; else acc.ARS += val;
6007     }
6008   });
6009   return acc;
6010 }
6011 const transaccionesByOrdenId = {};
6012 (rTrInst.data || []).forEach((t) => {
6013   const ordenId = t.instrumentacion_id ? Object.keys(instByOrden || {}).find((oid) => instByOrden[oid] === t.instrumentacion_id) : null;
6014   if (!ordenId) return;
6015   if (!transaccionesByOrdenId[ordenId]) transaccionesByOrdenId[ordenId] = [];
6016   transaccionesByOrdenId[ordenId].push(t);
6017 });
6018 const movsCliByOrdenCliente = {};
6019 (movCli || []).forEach((m) => {
6020   const oid = m.orden_id;
6021   const cid = m.cliente_id;
6022   if (!oid || !cid) return;
6023   if (!movsCliByOrdenCliente[oid]) movsCliByOrdenCliente[oid] = {};
6024   if (!movsCliByOrdenCliente[oid][cid]) movsCliByOrdenCliente[oid][cid] = [];
6025   movsCliByOrdenCliente[oid][cid].push(m);
6026 });
6027 const movsIntByOrdenInt = {};
6028 (movIntEnriched || []).forEach((m) => {
6029   const oid = m.orden_id;
6030   const iid = m.intermediario_id;
6031   if (!oid || !iid) return;
6032   if (!movsIntByOrdenInt[oid]) movsIntByOrdenInt[oid] = {};
6033   if (!movsIntByOrdenInt[oid][iid]) movsIntByOrdenInt[oid][iid] = [];
6034   movsIntByOrdenInt[oid][iid].push(m);
6035 });
6036 const saldoClienteByCliPrecomp = {};
6037 const saldoIntByIdPrecomp = {};
6038 Object.keys(ordenById || {}).forEach((ordenId) => {
6039   const orden = ordenById[ordenId];
6040   if (!orden) return;
6041   const metaPre = tiposOperacionNestedMeta(orden.tipos_operacion);
6042   const codigo = metaPre.codigo !== '-' ? metaPre.codigo : null;
6043   const toJoin = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
6044   const esModelo = esTipoOperacionChequeArs(codigo, toJoin ? moneda_in : moneda_out) && orden.intermediario_id;
6045   const transacciones = transaccionesByOrdenId[ordenId] || [];
6046   let contribCliente = { USD: 0, EUR: 0, ARS: 0 };
6047   let contribInt = { USD: 0, EUR: 0, ARS: 0 };
6048   if (esModelo) {
6049     contribCliente = contribucionSaldoClienteModeloCc(orden, transacciones);
6050     contribInt = contribucionSaldoIntermediarioModeloCc(orden, transacciones);
6051   } else {
6052     if (orden.cliente_id) contribCliente = sumMovsPorMoneda(movsCliByOrdenCliente[ordenId] ? [orden.cliente_id]);
6053     if (orden.intermediario_id) contribInt = sumMovsPorMoneda(movsIntByOrdenInt[ordenId] ? [orden.intermediario_id]);
6054   }
6055   if (orden.cliente_id) {
6056     if (!saldoClienteByCliPrecomp[orden.cliente_id]) saldoClienteByCliPrecomp[orden.cliente_id] = { USD: 0, EUR: 0, ARS: 0 };
6057     ['USD', 'EUR', 'ARS'].forEach((mon) => { saldoClienteByCliPrecomp[orden.cliente_id][mon] += contribCliente[mon] || 0; });
6058   }
6059   if (orden.intermediario_id) {
6060     if (!saldoIntByIdPrecomp[orden.intermediario_id]) saldoIntByIdPrecomp[orden.intermediario_id] = { USD: 0, EUR: 0, ARS: 0 };
6061     ['USD', 'EUR', 'ARS'].forEach((mon) => { saldoIntByIdPrecomp[orden.intermediario_id][mon] += contribInt[mon] || 0; });
6062   }
6063 });
6064 const chequeCliIngChequePendienteByCli = ccMapClienteChequeIngresoPrincipalPendiente(ordenById || {}, transaccionesByOrdenId);
6065 const chequeCliEgresoPandyClientePendienteByCli = ccMapClienteChequeEgresoPandyClientePendiente(ordenById || {}, transaccionesByOrdenId);
6066 const chequeIntIngresoIntPandyPendienteByInt = ccMapIntermediarioChequeIngresoIntPandyPendiente(ordenById || {}, transaccionesByOrdenId);
6067 const chequeIntEgresoPandyIntPendienteByInt = ccMapIntermediarioChequeEgresoPandyIntPendiente(ordenById || {}, transaccionesByOrdenId);
6068 const usdUsdIntCliIngPendienteByCli = ccMapClienteUsdUsdIntIngresoClientePandyPendiente(ordenById || {}, transaccionesByOrdenId);
6069 const usdUsdIntCliEgresoPandyClientePendienteByCli = ccMapClienteUsdUsdIntEgresoPandyClientePendiente(ordenById || {}, transaccionesByOrdenId);
6070 return delayMinLoadingSiNoEsBackground(loadingShownAtCc).then(() => {
6071   if (miTicket !== ccCargaSerial) return;
6072   buildCcResumenRows(clientes, intermediarios, movCli, movInt, loadingEl, contenido, tbody, ordenNumeroById, trPagadorById || {}, trTipoById || {}, trCobradorByI
d || {}, ordenById || {}, pendientePandyDebeIntByInt || {}, pendienteClienteAjusteByCli || {}, saldoClienteByCliPrecomp, saldoIntByIdPrecomp, chequeCliIngChequePendienteByCli,
chequeCliEgresoPandyClientePendienteByCli, chequeIntIngresoIntPandyPendienteByInt, chequeIntEgresoPandyIntPendienteByInt, usdUsdIntCliIngPendienteByCli, usdUsdIntCliEgresoPan
dyClientePendienteByCli, trParticipanteIdsByTrx || {});
6073 });
6074 });
6075 }).catch((err) => {
6076   if (miTicket !== ccCargaSerial) return;
6077   if (loadingEl) loadingEl.style.display = 'none';
6078   if (!silentCc) {
6079     contenido.style.display = 'block';
6080     syncCcPestañasYPaneles();
6081   }
6082 });
6083 }
6084
6085 function buildCcResumenRows(clientes, intermediarios, movCli, movInt, loadingEl, contenido, tbody, ordenNumeroById, trPagadorById, trTipoById, trCobradorById, ordenById, pendi
entePandyDebeIntByInt, pendienteClienteAjusteByCli, saldoClienteByCliPrecomp, saldoIntByIdPrecomp, chequeCliIngChequePendienteByCli, chequeCliEgresoPandyClientePendienteByCli,
chequeIntIngresoIntPandyPendienteByInt, chequeIntEgresoPandyIntPendienteByInt, usdUsdIntCliIngPendienteByCli, usdUsdIntCliEgresoPandyClientePendienteByCli, trParticipanteIdsByTrx

```



```

yTrx) {
6086   if (loadingEl) loadingEl.style.display = 'none';
6087   ordenNumeroById = ordenNumeroById || {};
6088   trPagadorById = trPagadorById || {};
6089   trTipoById = trTipoById || {};
6090   trCobradorById = trCobradorById || {};
6091   ordenById = ordenById || {};
6092   trParticipanteIdsByTrx = trParticipanteIdsByTrx || {};
6093   pendientePandyDebeIntByInt = pendientePandyDebeIntByInt || {};
6094   pendienteClienteAjusteByCli = pendienteClienteAjusteByCli || {};
6095   saldoClienteByCliPrecomp = saldoClienteByCliPrecomp || null;
6096   saldoIntByIdPrecomp = saldoIntByIdPrecomp || null;
6097   chequeCliIngChequePendienteByCli = chequeCliIngChequePendienteByCli || {};
6098   chequeCliEgresoPandyClientePendienteByCli = chequeCliEgresoPandyClientePendienteByCli || {};
6099   chequeIntIngresoIntPandyPendienteByInt = chequeIntIngresoIntPandyPendienteByInt || {};
6100   chequeIntEgresoPandyIntPendienteByInt = chequeIntEgresoPandyIntPendienteByInt || {};
6101   usdUsdIntCliIngPendienteByCli = usdUsdIntCliIngPendienteByCli || {};
6102   usdUsdIntCliEgresoPandyClientePendienteByCli = usdUsdIntCliEgresoPandyClientePendienteByCli || {};
6103   const ordenTipoOpMetaById = {};
6104   Object.keys(ordenById).forEach((oid) => {
6105     ordenTipoOpMetaById[oid] = tiposOperacionNestedMeta(ordenById[oid]?.tipos_operacion);
6106   });
6107   function parseOrdenNumero(concepto) {
6108     const txt = (concepto || '').toString();
6109     let m = txt.match(/(?:ORDEN|NRO ORDEN)\s*(\d+)/i);
6110     if (m && m[1]) return m[1];
6111     m = txt.match(/nro\s*orden\s*(\d+)/i);
6112     if (m && m[1]) return m[1];
6113     return null;
6114   }
6115   function parseMonedaDesdeConcepto(concepto) {
6116     const txt = (concepto || '').toString().toUpperCase();
6117     const m = txt.match(/b(USD|ARS|EUR)\b/);
6118     return m ? m[1] : null;
6119   }
6120   function getMontosPorMoneda(m) {
6121     const hasPorMoneda = m.monto_usd != null || m.monto_ars != null || m.monto_eur != null;
6122     if (hasPorMoneda) {
6123       return {
6124         USD: Number(m.monto_usd) || 0,
6125         ARS: Number(m.monto_ars) || 0,
6126         EUR: Number(m.monto_eur) || 0,
6127       };
6128     }
6129     const mon = (m.moneda || '').toString().toUpperCase();
6130     const val = Number(m.monto) || 0;
6131     return {
6132       USD: mon === 'USD' ? val : 0,
6133       ARS: mon === 'ARS' ? val : 0,
6134       EUR: mon === 'EUR' ? val : 0,
6135     };
6136   }
6137   /**
6138    * Saldo CC resumen: suma algebraica por moneda de todos los movimientos no anulados (igual criterio que la lista Movimientos).
6139    */
6140   function saldosDesdeMovimientosPorOrden(movs) {
6141     const acc = { USD: 0, EUR: 0, ARS: 0 };
6142     (movs || []).forEach((m) => {
6143       if ((m.estado || '').toString().toLowerCase() === 'anulado') return;
6144       const montos = getMontosPorMoneda(m);
6145       acc.USD += montos.USD;
6146       acc.ARS += montos.ARS;
6147       acc.EUR += montos.EUR;
6148     });
6149     return acc;
6150   }
6151   const movsCliById = {};
6152   (movCli || []).forEach((m) => {
6153     const id = m.cliente_id;
6154     if (!id) return;
6155     if (!movsCliById[id]) movsCliById[id] = [];
6156     movsCliById[id].push(m);
6157   });
6158   const movsIntById = {};
6159   (movInt || []).forEach((m) => {
6160     const id = m.intermediario_id;
6161     if (!id) return;
6162     if (!movsIntById[id]) movsIntById[id] = [];
6163     movsIntById[id].push(m);
6164   });
6165   const clientesById = Object.fromEntries((clientes || []).map((c) => [c.id, c]));
6166   const intermediariosById = Object.fromEntries((intermediarios || []).map((i) => [i.id, i]));
6167   const rows = [];
6168   const addedCli = new Set();
6169   const addedInt = new Set();
6170   // Pendiente "ajuste" global (legacy/UI): no reemplaza la suma de movimientos del resumen.
6171   function pendienteClienteAjusteForCli(clienteId) {
6172     const p = pendienteClienteAjusteByCli(clienteId);
6173     if (!p) return null;
6174     const has = Math.abs(p.USD || 0) + Math.abs(p.EUR || 0) + Math.abs(p.ARS || 0);
6175     return has >= 1e-6 ? p : null;
6176   }
6177   // Saldo = suma de movimientos no anulados (coherente con listado).
6178   const saldosCliente = (id) => saldosDesdeMovimientosPorOrden(movsCliById[id] || []);
6179   function mergePendienteClaseChequeArsCliente(clienteId, saldos, pendienteEnMoneda, pendienteClasePorMoneda) {
6180
6181

```

```

6182|     const ing = chequeCliIngChequePendienteByCli[clienteId];
6183|     const pagoInst = chequeCliEgresoPandyClientePendienteByCli[clienteId];
6184|     if (!ing && !pagoInst) return pendienteClasePorMoneda;
6185|     const pendA = pendienteEnMoneda.ARS;
6186|     const absS = Math.abs(Number(saldos.ARS) || 0);
6187|     if (!pendA && absS < 1e-6) return pendienteClasePorMoneda;
6188|     const cur = pendienteClasePorMoneda.ARS;
6189|     if (ing && pagoInst) return { ...pendienteClasePorMoneda, ARS: 'mixto' };
6190|     if (ing) {
6191|       if (cur === 'pago' || cur === 'mixto') return { ...pendienteClasePorMoneda, ARS: 'mixto' };
6192|       return { ...pendienteClasePorMoneda, ARS: 'cobro' };
6193|     }
6194|     if (pagoInst) {
6195|       if (cur === 'cobro' || cur === 'mixto') return { ...pendienteClasePorMoneda, ARS: 'mixto' };
6196|       return { ...pendienteClasePorMoneda, ARS: 'pago' };
6197|     }
6198|     return pendienteClasePorMoneda;
6199|   }
6200|   function mergePendienteClaseChequeArsIntermediario(intId, saldos, pendienteEnMoneda, pendienteClasePorMoneda) {
6201|     const ingInt = chequeIntIngresoIntPandyPendienteByInt[intId];
6202|     const egresoPandyInt = chequeIntEgresoPandyIntPendienteByInt[intId];
6203|     if (!ingInt && !egresoPandyInt) return pendienteClasePorMoneda;
6204|     const pendA = pendienteEnMoneda.ARS;
6205|     const absS = Math.abs(Number(saldos.ARS) || 0);
6206|     if (!pendA && absS < 1e-6) return pendienteClasePorMoneda;
6207|     const cur = pendienteClasePorMoneda.ARS;
6208|     if (ingInt && egresoPandyInt) return { ...pendienteClasePorMoneda, ARS: 'mixto' };
6209|     if (ingInt) {
6210|       if (cur === 'pago' || cur === 'mixto') return { ...pendienteClasePorMoneda, ARS: 'mixto' };
6211|       return { ...pendienteClasePorMoneda, ARS: 'cobro' };
6212|     }
6213|     if (egresoPandyInt) {
6214|       if (cur === 'cobro' || cur === 'mixto') return { ...pendienteClasePorMoneda, ARS: 'mixto' };
6215|       return { ...pendienteClasePorMoneda, ARS: 'pago' };
6216|     }
6217|     return pendienteClasePorMoneda;
6218|   }
6219|   clientes.forEach((c) => {
6220|     const saldos = saldosCliente(c.id);
6221|     const movsC = movsCliById[c.id] || [];
6222|     const pendienteEnMoneda = ccPendientePorMonedaDesdeMovs(movsC);
6223|     let pendienteClasePorMoneda = ccPendienteClasePorMonedaDesdeMovs(movsC, trTipoById, trPagadorById, trCobradorById, c.id, ordenById, trParticipanteIdsByTrx);
6224|     pendienteClasePorMoneda = mergePendienteClaseChequeArsCliente(c.id, saldos, pendienteEnMoneda, pendienteClasePorMoneda);
6225|     pendienteClasePorMoneda = mergePendienteClaseUsdUsdIntCliente(c.id, saldos, pendienteEnMoneda, pendienteClasePorMoneda, usdUsdIntCliIngPendienteByCli, usdUsdIntCliEgresoPandyClientePendienteByCli);
6226|     rows.push({ tipo: 'cliente', id: c.id, nombre: c.nombre, saldos, pendienteEnMoneda, pendienteClasePorMoneda, pendienteClienteAjuste: pendienteClienteAjusteForCli(c.id) });
6227|     addedCli.add(c.id);
6228|   });
6229|   Object.keys(movsCliById || {}).forEach((id) => {
6230|     if (addedCli.has(id)) return;
6231|     const c = clientesById[id];
6232|     const saldos = saldosCliente(id);
6233|     const movsC = movsCliById[id] || [];
6234|     const pendienteEnMoneda = ccPendientePorMonedaDesdeMovs(movsC);
6235|     let pendienteClasePorMoneda = ccPendienteClasePorMonedaDesdeMovs(movsC, trTipoById, trPagadorById, trCobradorById, id, ordenById, trParticipanteIdsByTrx);
6236|     pendienteClasePorMoneda = mergePendienteClaseChequeArsCliente(id, saldos, pendienteEnMoneda, pendienteClasePorMoneda);
6237|     pendienteClasePorMoneda = mergePendienteClaseUsdUsdIntCliente(id, saldos, pendienteEnMoneda, pendienteClasePorMoneda, usdUsdIntCliIngPendienteByCli, usdUsdIntCliEgresoPandyClientePendienteByCli);
6238|     rows.push({ tipo: 'cliente', id, nombre: (c && c.nombre) || '-', saldos, pendienteEnMoneda, pendienteClasePorMoneda, pendienteClienteAjuste: pendienteClienteAjusteForCli(id) });
6239|     addedCli.add(id);
6240|   });
6241|   Object.keys(pendienteClienteAjusteByCli || {}).forEach((id) => {
6242|     if (addedCli.has(id)) return;
6243|     const c = clientesById[id];
6244|     const pendienteClienteAjuste = pendienteClienteAjusteForCli(id);
6245|     if (!pendienteClienteAjuste) return;
6246|     const movsC = movsCliById[id] || [];
6247|     const saldos = { USD: 0, EUR: 0, ARS: 0 };
6248|     const pendienteEnMoneda = ccPendientePorMonedaDesdeMovs(movsC);
6249|     let pendienteClasePorMoneda = ccPendienteClasePorMonedaDesdeMovs(movsC, trTipoById, trPagadorById, trCobradorById, id, ordenById, trParticipanteIdsByTrx);
6250|     pendienteClasePorMoneda = mergePendienteClaseChequeArsCliente(id, saldos, pendienteEnMoneda, pendienteClasePorMoneda);
6251|     pendienteClasePorMoneda = mergePendienteClaseUsdUsdIntCliente(id, saldos, pendienteEnMoneda, pendienteClasePorMoneda, usdUsdIntCliIngPendienteByCli, usdUsdIntCliEgresoPandyClientePendienteByCli);
6252|     rows.push({ tipo: 'cliente', id, nombre: (c && c.nombre) || '-', saldos, pendienteEnMoneda, pendienteClasePorMoneda, pendienteClienteAjuste });
6253|     addedCli.add(id);
6254|   });
6255|   function pendientePandyDebeForInt(intId) {
6256|     const p = pendientePandyDebeIntByInt[intId];
6257|     if (!p) return null;
6258|     const has = Math.abs(p.USD || 0) + Math.abs(p.EUR || 0) + Math.abs(p.ARS || 0);
6259|     return has >= 1e-6 ? p : null;
6260|   }
6261|   const saldosInt = (id) => saldosDesdeMovimientosPorOrden(movsIntById[id] || []);
6262|   intermediarios.forEach((i) => {
6263|     const saldos = saldosInt(i.id);
6264|     const movsI = movsIntById[i.id] || [];
6265|     const pendienteEnMoneda = ccPendientePorMonedaDesdeMovs(movsI);
6266|     let pendienteClasePorMoneda = ccPendienteClasePorMonedaDesdeMovs(movsI, trTipoById, trPagadorById, trCobradorById, null, null, null);
6267|     pendienteClasePorMoneda = mergePendienteClaseChequeArsIntermediario(i.id, saldos, pendienteEnMoneda, pendienteClasePorMoneda);
6268|     pendienteClasePorMoneda = mergePendienteClaseUsdUsdIntIntermediario(i.id, saldos, pendienteEnMoneda, pendienteClasePorMoneda, movsI, ordenById);
6269|     pendienteClasePorMoneda = mergePendienteClaseChequeArsIntermediarioCerradoPago(i.id, saldos, pendienteEnMoneda, pendienteClasePorMoneda, movsI, ordenById);
6270|     rows.push({ tipo: 'intermediario', id: i.id, nombre: i.nombre, saldos, pendienteEnMoneda, pendienteClasePorMoneda, pendientePandyDebe: pendientePandyDebeForInt(i.id) });
6271|     addedInt.add(i.id);
6272|   });
6273|   Object.keys(movsIntById || {}).forEach((id) => {
6274|     if (addedInt.has(id)) return;

```

```

6275|     const i = intermediariosById[id];
6276|     const saldos = saldosInt(id);
6277|     const movsI = movsIntById[id] || [];
6278|     const pendienteEnMoneda = ccPendientePorMonedaDesdeMovs(movsI);
6279|     let pendienteClasePorMoneda = ccPendienteClasePorMonedaDesdeMovs(movsI, trTipoById, trPagadorById, trCobradorById, null, null, null);
6280|     pendienteClasePorMoneda = mergePendienteClaseChequeArsIntermediario(id, saldos, pendienteEnMoneda, pendienteClasePorMoneda);
6281|     pendienteClasePorMoneda = mergePendienteClaseUsdUsdIntIntermediario(id, saldos, pendienteEnMoneda, pendienteClasePorMoneda, movsI, ordenById);
6282|     pendienteClasePorMoneda = mergePendienteClaseChequeArsIntermediarioCerradoPago(id, saldos, pendienteEnMoneda, pendienteClasePorMoneda, movsI, ordenById);
6283|     rows.push({ tipo: 'intermediario', id, nombre: (i && i.nombre) || '-', saldos, pendienteEnMoneda, pendienteClasePorMoneda, pendientePandyDebe: pendientePandyDebeForInt(id)
});
6284|   addedInt.add(id);
6285| });
6286| Object.keys(pendientePandyDebeIntByInt || {}).forEach((id) => {
6287|   if (addedInt.has(id)) return;
6288|   const i = intermediariosById[id];
6289|   const pendientePandyDebe = pendientePandyDebeForInt(id);
6290|   if (!pendientePandyDebe) return;
6291|   const movsI = movsIntById[id] || [];
6292|   const saldos = { USD: 0, EUR: 0, ARS: 0 };
6293|   const pendienteEnMoneda = ccPendientePorMonedaDesdeMovs(movsI);
6294|   let pendienteClasePorMoneda = ccPendienteClasePorMonedaDesdeMovs(movsI, trTipoById, trPagadorById, trCobradorById, null, null, null);
6295|   pendienteClasePorMoneda = mergePendienteClaseChequeArsIntermediario(id, saldos, pendienteEnMoneda, pendienteClasePorMoneda);
6296|   pendienteClasePorMoneda = mergePendienteClaseUsdUsdIntIntermediario(id, saldos, pendienteEnMoneda, pendienteClasePorMoneda, movsI, ordenById);
6297|   pendienteClasePorMoneda = mergePendienteClaseChequeArsIntermediarioCerradoPago(id, saldos, pendienteEnMoneda, pendienteClasePorMoneda, movsI, ordenById);
6298|   rows.push({ tipo: 'intermediario', id, nombre: (i && i.nombre) || '-', saldos, pendienteEnMoneda, pendienteClasePorMoneda, pendientePandyDebe });
6299|   addedInt.add(id);
6300| });
6301| ccResumenRowsTodos = [...rows].sort((a, b) => (a.nombre || '').localeCompare(b.nombre || ''));
6302|
6303| // Detalle = solo movimientos con incluir_en_detalle (si la columna no existe, se muestran todos).
6304| const detalleList = [];
6305| (movCli || []).forEach((m) => {
6306|   if (m.incluir_en_detalle === false) return;
6307|   const metaC = ordenTipoOpMetaById[m.orden_id];
6308|   detalleList.push({
6309|     ...m,
6310|     tipo: 'cliente',
6311|     nombre: (clientesById[m.cliente_id] && clientesById[m.cliente_id].nombre) || '-',
6312|     ...ccNombresPagadorCobradorMovimiento(m, 'cliente', ordenById, clientesById, intermediariosById, trTipoById, trPagadorById, trCobradorById, trParticipanteIdsByTrx),
6313|     orden_numero: ordenNumeroById[m.orden_id] != null ? ordenNumeroById[m.orden_id] : (m.orden_numero != null ? m.orden_numero : null),
6314|     tipo_operacion: metaC ? metaC.codigo : '-',
6315|     tipo_op_nombre: metaC ? metaC.nombre : '',
6316|     tipo_op_icono_modo: metaC ? metaC.icono_modo : 'auto',
6317|     tipo_op_icono_url: metaC ? metaC.icono_url_publica : '',
6318|     tipo_op_usa_intermediario: metaC ? metaC.usa_intermediario === true : false,
6319|   });
6320| });
6321| (movInt || []).forEach((m) => {
6322|   if (m.incluir_en_detalle === false) return;
6323|   const metaI = ordenTipoOpMetaById[m.orden_id];
6324|   detalleList.push({
6325|     ...m,
6326|     tipo: 'intermediario',
6327|     nombre: (intermediariosById[m.intermediario_id] && intermediariosById[m.intermediario_id].nombre) || '-',
6328|     ...ccNombresPagadorCobradorMovimiento(m, 'intermediario', ordenById, clientesById, intermediariosById, trTipoById, trPagadorById, trCobradorById, trParticipanteIdsByTrx),
6329|     orden_numero: ordenNumeroById[m.orden_id] != null ? ordenNumeroById[m.orden_id] : (m.orden_numero != null ? m.orden_numero : null),
6330|     tipo_operacion: metaI ? metaI.codigo : '-',
6331|     tipo_op_nombre: metaI ? metaI.nombre : '',
6332|     tipo_op_icono_modo: metaI ? metaI.icono_modo : 'auto',
6333|     tipo_op_icono_url: metaI ? metaI.icono_url_publica : '',
6334|     tipo_op_usa_intermediario: metaI ? metaI.usa_intermediario === true : false,
6335|   });
6336| });
6337| // Corazón de la app: un movimiento lógico = una fila. Manuales sin orden comparten orden/trans null y concepto parecido: dedupe por id para no colapsar patas distintas del mismo grupo.
6338| const detalleKey = (a) => {
6339|   if (a.es_movimiento_manual) {
6340|     return [a.tipo, 'manual', a.id != null ? String(a.id) : '', String(a.monto ?? ''), (a.concepto || '').slice(0, 80)].join('\t');
6341|   }
6342|   return [a.tipo, a.orden_id, a.transaccion_id, a.monto, (a.concepto || '').slice(0, 60)].join('\t');
6343| };
6344| const seenDetalle = new Set();
6345| const detalleListUnicos = detalleList.filter((a) => {
6346|   const k = detalleKey(a);
6347|   if (seenDetalle.has(k)) return false;
6348|   seenDetalle.add(k);
6349|   return true;
6350| });
6351| ccMovimientosDetalleList = detalleListUnicos.sort((a, b) => {
6352|   const fa = (a.fecha || '').toString();
6353|   const fb = (b.fecha || '').toString();
6354|   if (fb !== fa) return fb.localeCompare(fa);
6355|   return (b.id || 0) - (a.id || 0);
6356| });
6357|
6358| Promise.all([
6359|   hayTipoOperacionActivoConMoneda('USD'),
6360|   hayTipoOperacionActivoConMoneda('ARS'),
6361|   hayTipoOperacionActivoConMoneda('EUR'),
6362| ]).then(([, a, e]) => {
6363|   if (!ccDetalleMovimientosRangoInicializado) {
6364|     ccMovimientosMostrarTodoHistorial = false;
6365|     const hoyCc = fechaHoyYYYYMMDDArgentina();
6366|     ccDetalleDesde = hoyCc;
6367|     ccDetalleHasta = hoyCc;
6368|     const desdeEl0 = document.getElementById('cc-detalle-desde');

```

```

6369|     const hastaEl0 = document.getElementById('cc-detalle-hasta');
6370|     if (desdeEl0) desdeEl0.value = hoyCc;
6371|     if (hastaEl0) hastaEl0.value = hoyCc;
6372|     ccDetalleMovimientosRangoInicializado = true;
6373|   }
6374|   aplicarVisibilidadMonedasCuentaCorriente({ USD: !!u, ARS: !!a, EUR: !!e });
6375|   poblarSelectCcDetalleEntidad();
6376|   aplicarFiltroCcResumen();
6377| });
6378| }
6379|
6380| /** Rellena el combo Cliente/Intermediario en Movimientos según movimientos cargados y tipo actual. */
6381| function poblarSelectCcDetalleEntidad() {
6382|   const sel = document.getElementById('cc-detalle-entidad-select');
6383|   if (!sel) return;
6384|   const idKey = ccFiltroTipo === 'cliente' ? 'cliente_id' : 'intermediario_id';
6385|   const movs = ccMovimientosDetalleList.filter((m) => m.tipo === ccFiltroTipo);
6386|   const map = new Map();
6387|   movs.forEach((m) => {
6388|     const id = m[idKey];
6389|     if (!id) return;
6390|     const nombre = ((m.nombre || '') + ' ').trim() || '-';
6391|     if (!map.has(id)) map.set(id, nombre);
6392|   });
6393|   const sorted = [...map.entries()].sort((a, b) => a[1].localeCompare(b[1], 'es'));
6394|   const prev = ccDetalleFiltroEntidadId;
6395|   sel.innerHTML = '<option value="">' + (ccFiltroTipo === 'cliente' ? 'Todos los clientes' : 'Todos los intermediarios') + '</option>'
6396|     + sorted.map(([id, nom]) => '<option value="' + escapeHtml(String(id)) + '>' + escapeHtml(nom) + '</option>').join('');
6397|   if (prev && sorted.some([id] => id === prev)) {
6398|     sel.value = prev;
6399|     ccDetalleFiltroEntidadId = prev;
6400|   } else {
6401|     ccDetalleFiltroEntidadId = '';
6402|     sel.value = '';
6403|   }
6404|   const ariaEnt = ccFiltroTipo === 'cliente' ? 'Filtrar movimientos por cliente' : 'Filtrar movimientos por intermediario';
6405|   sel.setAttribute('aria-label', ariaEnt);
6406| }
6407|
6408| /** Muestra el panel Saldos o Movimientos y actualiza solapas (clase activo y aria-selected). */
6409| function syncCcPestañasYPaneles() {
6410|   const panelSaldos = document.getElementById('cc-panel-saldos');
6411|   const panelMov = document.getElementById('cc-panel-movimientos');
6412|   const toggleEl = document.getElementById('cc-vista-toggle');
6413|   const esDetalle = ccVistaToggle === 'detalle';
6414|   if (panelSaldos) panelSaldos.style.display = esDetalle ? 'none' : 'block';
6415|   if (panelMov) panelMov.style.display = esDetalle ? 'block' : 'none';
6416|   if (toggleEl) {
6417|     toggleEl.querySelectorAll('button[data-vista]').forEach((b) => {
6418|       const v = b.getAttribute('data-vista');
6419|       const on = (esDetalle && v === 'detalle') || (!esDetalle && v === 'resumen');
6420|       b.classList.toggle('activo', on);
6421|       b.setAttribute('aria-selected', on ? 'true' : 'false');
6422|     });
6423|   }
6424| }
6425|
6426| function aplicarFiltroCcResumen() {
6427|   const contenidoEl = document.getElementById('cc-contenido');
6428|   const detalleWrap = document.getElementById('cc-detalle-wrap');
6429|
6430|   if (ccVistaToggle === 'detalle') {
6431|     syncCcPestañasYPaneles();
6432|     if (contenidoEl) contenidoEl.style.display = 'none';
6433|     if (detalleWrap) detalleWrap.style.display = 'block';
6434|     const rangoWrap = document.getElementById('cc-detalle-rango-wrap');
6435|     if (rangoWrap) rangoWrap.style.display = 'flex';
6436|     let filtrados = ccMovimientosDetalleList.filter((m) => m.tipo === ccFiltroTipo);
6437|     if (ccDetalleFiltroEntidadId) {
6438|       if (ccFiltroTipo === 'cliente') {
6439|         filtrados = filtrados.filter((m) => m.cliente_id === ccDetalleFiltroEntidadId);
6440|       } else {
6441|         filtrados = filtrados.filter((m) => m.intermediario_id === ccDetalleFiltroEntidadId);
6442|       }
6443|     }
6444|     if (!ccMovimientosMostrarTodoHistorial) {
6445|       const hoyStr = fechaHoyYYYYMMDDArgentina();
6446|       const desde = ccDetalleDesde || hoyStr;
6447|       const hasta = ccDetalleHasta || hoyStr;
6448|       filtrados = filtrados.filter((m) => {
6449|         const f = (m.fecha || '').toString().slice(0, 10);
6450|         if (desde && f < desde) return false;
6451|         if (hasta && f > hasta) return false;
6452|         return true;
6453|       });
6454|     }
6455|     actualizarRangoDetalleDefaults();
6456|     renderCcVistaDetalle(filtrados);
6457|     return;
6458|   }
6459|   syncCcPestañasYPaneles();
6460|   const rangoWrap = document.getElementById('cc-detalle-rango-wrap');
6461|   if (rangoWrap) rangoWrap.style.display = 'none';
6462|   if (contenidoEl) contenidoEl.style.display = 'block';
6463|   if (detalleWrap) detalleWrap.style.display = 'none';
6464|   // Resumen: solo filas con saldo ≠ 0 (saldo = suma movimientos no anulados).
6465|   const EPSILON_SALDO = 1e-6;

```

```

6466| const conSaldo = (r) => Math.abs(Number(r.saldos.USD) || 0) >= EPSILON_SALDO || Math.abs(Number(r.saldos.EUR) || 0) >= EPSILON_SALDO || Math.abs(Number(r.saldos.ARS) || 0) >
= EPSILON_SALDO;
6467| const filtrados = ccResumenRowsTodos.filter((r) => r.tipo === ccFiltroTipo).filter(conSaldo);
6468| renderCcResumenTable(filtrados);
6469| }
6470|
6471| function renderCcResumenTable(rows) {
6472| const tbody = document.getElementById('cc-resumen-tbody');
6473| if (!tbody) return;
6474|
6475| const monedas = MONEDAS_CC_UI;
6476| // Resumen: importe con ccSaldoDisplayOpticaResumen; leyenda alineada al signo mostrado (verde/rojo).
6477| tbody.innerHTML = rows
6478|   .map((row) => {
6479|     let tr = '<tr><td>' + escapeHtml(row.nombre || '-') + '</td>';
6480|     const pendMon = row.pendienteEnMoneda || { USD: false, EUR: false, ARS: false };
6481|     const pendClase = row.pendienteClasePorMoneda || { USD: 'ninguno', EUR: 'ninguno', ARS: 'ninguno' };
6482|     monedas.forEach((mon) => {
6483|       const sAlg = Number(row.saldos[mon]) || 0;
6484|       const sDisp = ccSaldoDisplayOpticaResumen(sAlg, pendClase[mon]);
6485|       const val = sDisp !== 0 ? formatMonto(sDisp, mon) : '-';
6486|       const cls = sDisp > 0 ? 'valor-positivo' : (sDisp < 0 ? 'valor-negativo' : '');
6487|       const leyenda = ccLeyendaSaldoResumenHtml(mon, pendMon, pendClase[mon], sDisp);
6488|       tr += '<td data-cc-monedas-col="' + mon + '><span class="cc-saldo-celda-wrap"><span class="' + cls + '>' + val + '</span>' + leyenda + '</span></td>';
6489|     });
6490|     tr += '<td><button type="button" class="btn-ver-detalle" data-tipo="' + escapeHtml(row.tipo) + '" data-id="' + escapeHtml(row.id) + '" data-nombre="' + escapeHtml(row.no
mbre || '-') + '" title="Ver detalle de movimientos" aria-label="Ver detalle de movimientos"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" s
troke-width="1.5" stroke-linecap="round" stroke-linejoin="round"><path d="M1 12s4-8 11-8 11 8-4 8-11 8-11-8-11-8z"/><circle cx="12" cy="12" r="2.5" fill="none"/></svg></s
pan></button></td></tr>';
6491|     return tr;
6492|   })
6493|   .join('');
6494|
6495| if (rows.length === 0) {
6496|   const tipoLabel = ccFiltroTipo === 'cliente' ? 'clientes' : 'intermediarios';
6497|   const colspan = ccColspanResumenSaldosVacio();
6498|   tbody.innerHTML = '<tr><td colspan="' + colspan + '>No hay ' + tipoLabel + ' con saldo distinto de cero.</td></tr>';
6499| } else {
6500|   tbody.querySelectorAll('.btn-ver-detalle').forEach((btn) => {
6501|     btn.addEventListener('click', () => {
6502|       const tipo = btn.getAttribute('data-tipo');
6503|       const id = btn.getAttribute('data-id');
6504|       const nombre = btn.getAttribute('data-nombre') || '';
6505|       if (tipo && id) openModalCcDetalle(tipo, id, nombre);
6506|     });
6507|   });
6508| }
6509| reaplicarVisibilidadMonedasCuentaCorrienteDom();
6510| }
6511|
6512| /** Sincroniza inputs Desde/Hasta con el estado (no expande solo al rango de filas visibles). */
6513| function actualizarRangoDetalleDefaults() {
6514|   const desdeEl = document.getElementById('cc-detalle-desde');
6515|   const hastaEl = document.getElementById('cc-detalle-hasta');
6516|   if (!desdeEl || !hastaEl) return;
6517|   if (ccMovimientosMostrarTodoHistorial) {
6518|     desdeEl.value = '';
6519|     hastaEl.value = '';
6520|     return;
6521|   }
6522|   desdeEl.value = ccDetalleDesde || '';
6523|   hastaEl.value = ccDetalleHasta || '';
6524| }
6525|
6526| /** Comparador para ordenar filas de detalle CC por columna (fecha, nroOrden, nroTrans, concepto, usd, ars, eur, estado, nombre). */
6527| function compareCcDetalleRow(a, b, col, dir) {
6528|   let va, vb;
6529|   switch (col) {
6530|     case 'fecha':
6531|       va = (a.fecha || '').toString().slice(0, 10);
6532|       vb = (b.fecha || '').toString().slice(0, 10);
6533|       return dir * (va < vb ? -1 : va > vb ? 1 : 0);
6534|     case 'nroOrden':
6535|       va = a.orden_numero != null ? Number(a.orden_numero) : -Infinity;
6536|       vb = b.orden_numero != null ? Number(b.orden_numero) : -Infinity;
6537|       return dir * (va - vb);
6538|     case 'nroTrans':
6539|       va = a.transaccion_numero != null ? Number(a.transaccion_numero) : -Infinity;
6540|       vb = b.transaccion_numero != null ? Number(b.transaccion_numero) : -Infinity;
6541|       return dir * (va - vb);
6542|     case 'tipo_operacion':
6543|       va = (a.tipo_operacion || '-').toString();
6544|       vb = (b.tipo_operacion || '-').toString();
6545|       return dir * (va.localeCompare(vb));
6546|     case 'concepto':
6547|       va = (a.concepto || '').toString();
6548|       vb = (b.concepto || '').toString();
6549|       return dir * (va.localeCompare(vb));
6550|     case 'usd': case 'ars': case 'eur':
6551|       const key = col === 'usd' ? 'monto_usd' : (col === 'ars' ? 'monto_ars' : 'monto_eur');
6552|       const mon = col === 'usd' ? 'USD' : (col === 'ars' ? 'ARS' : 'EUR');
6553|       const tieneA = a.monto_usd != null || a.monto_ars != null || a.monto_eur != null;
6554|       const tieneB = b.monto_usd != null || b.monto_ars != null || b.monto_eur != null;
6555|       va = tieneA && a[key] != null ? Number(a[key]) : (a.moneda === mon ? Number(a.monto) || 0 : -Infinity);
6556|       vb = tieneB && b[key] != null ? Number(b[key]) : (b.moneda === mon ? Number(b.monto) || 0 : -Infinity);
6557|       return dir * (va - vb);
6558|   }

```

```

6559 |     case 'estado':
6560 |         va = (a.estado || '').toString();
6561 |         vb = (b.estado || '').toString();
6562 |         return dir * (va.localeCompare(vb));
6563 |     case 'nombre':
6564 |         va = (a.nombre || '').toString();
6565 |         vb = (b.nombre || '').toString();
6566 |         return dir * (va.localeCompare(vb));
6567 |     case 'ccPagador':
6568 |         va = (a.ccPagador || '').toString();
6569 |         vb = (b.ccPagador || '').toString();
6570 |         return dir * (va.localeCompare(vb));
6571 |     case 'ccCobrador':
6572 |         va = (a.ccCobrador || '').toString();
6573 |         vb = (b.ccCobrador || '').toString();
6574 |         return dir * (va.localeCompare(vb));
6575 |     default:
6576 |         return 0;
6577 | }
6578 | }
6579 |
6580 | /** Renderiza la tabla de la vista "Detalle de movimientos". Columnas: Fecha, Orden, Trans., Concepto, USD, ARS, EUR, Estado, Pagador, Cobrador (transacción). */
6581 | function renderCcVistaDetalle(filtrados) {
6582 |     const tbody = document.getElementById('cc-vista-detalle-tbody');
6583 |     const thEntity = document.getElementById('cc-detalle-th-entity');
6584 |     const thOriginante = document.getElementById('cc-detalle-th-originante');
6585 |     if (!tbody) return;
6586 |     if (thEntity) thEntity.textContent = 'Pagador';
6587 |     if (thOriginante) thOriginante.textContent = 'Cobrador';
6588 |     ccDetalleVistaRowsActual = filtrados;
6589 |
6590 |     tbody.innerHTML = filtrados
6591 |         .map(m) => {
6592 |             // Un movimiento = una moneda (la de la transacción). Mostrar solo esa columna; el resto "-".
6593 |             const mon = (m.moneda || 'USD').toUpperCase();
6594 |             const valUsd = mon === 'USD' ? (m.monto_usd !== null ? Number(m.monto_usd) : Number(m.monto) || 0) : null;
6595 |             const valArs = mon === 'ARS' ? (m.monto_ars !== null ? Number(m.monto_ars) : Number(m.monto) || 0) : null;
6596 |             const valEur = mon === 'EUR' ? (m.monto_eur !== null ? Number(m.monto_eur) : Number(m.monto) || 0) : null;
6597 |             const celdaUsd = mon === 'USD' ? formatearCeldaMonedaConSigno(valUsd, 'USD') : '-';
6598 |             const celdaArs = mon === 'ARS' ? formatearCeldaMonedaConSigno(valArs, 'ARS') : '-';
6599 |             const celdaEur = mon === 'EUR' ? formatearCeldaMonedaConSigno(valEur, 'EUR') : '-';
6600 |             const estadoLabel = (m.estado === 'pendiente' ? 'Pendiente' : (m.estado === 'cerrado' ? 'Cerrado' : (m.estado || '-')));
6601 |             const nroOrden = m.orden_numero !== null ? String(m.orden_numero) : '-';
6602 |             const nroTrans = m.transaccion_numero !== null ? String(m.transaccion_numero) : '-';
6603 |             const tipoOp = (m.tipo_operacion !== null && m.tipo_operacion !== '-') ? String(m.tipo_operacion) : '-';
6604 |             const tipoOpHtml = m.es_movimiento_manual
6605 |                 ? '<span class="cc-tipo-op-manual" title="Sin orden">Manual</span>'
6606 |                 : (tipoOp === '-' ? '-' : htmlTipoOperacionIconos(tipoOp, m.tipo_op_nombre || '', { iconoModo: m.tipo_op_icono_modo, iconoUrlPublica: m.tipo_op_icono_url, usaIntermediario: m.tipo_op_usa_intermediario === true }));
6607 |             return `<tr>
6608 |                 <td class="cc-col-fija cc-col-fija-1">${(m.fecha || '').toString().slice(0, 10)}</td>
6609 |                 <td class="cc-col-fija cc-col-fija-2 cc-col-tipo-op-iconos">${tipoOpHtml}</td>
6610 |                 <td>${escapeHtml(nroOrden)}</td>
6611 |                 <td>${escapeHtml(nroTrans)}</td>
6612 |                 <td class="td-concepto">${escapeHtml(m.concepto || '-')}</td>
6613 |                 <td data-cc-moneda-col="USD">${celdaUsd}</td>
6614 |                 <td data-cc-moneda-col="ARS">${celdaArs}</td>
6615 |                 <td data-cc-moneda-col="EUR">${celdaEur}</td>
6616 |                 <td>${escapeHtml(estadoLabel)}</td>
6617 |                 <td>${escapeHtml(m.ccPagador || '-')}</td>
6618 |                 <td>${escapeHtml(m.ccCobrador || '-')}</td>
6619 |                 <td class="cc-col-acciones-manual">${htmlCcAccionesMovimientoManualRow(m)}</td>
6620 |             </tr>`;
6621 |         })
6622 |         .join('');
6623 |
6624 |     if (filtrados.length === 0) {
6625 |         tbody.innerHTML = `<tr><td colspan="${ccColspanVistaDetalleMovimientos()}">No hay movimientos para el filtro seleccionado.</td></tr>`;
6626 |     } else {
6627 |         setupCcDetalleVistaSortHeaders();
6628 |     }
6629 |     reaplicarVisibilidadMonedasCuentaCorrienteDom();
6630 |     setupDelegacionAccionesCcManual();
6631 | }
6632 |
6633 | function setupCcDetalleVistaSortHeaders() {
6634 |     const table = document.getElementById('tabla-cc-detalle');
6635 |     if (!table || !table.closest('#cc-detalle-wrap')) return;
6636 |     const thead = table.querySelector('thead tr');
6637 |     if (!thead) return;
6638 |     thead.querySelectorAll('th[data-sort]').forEach((th) => {
6639 |         const col = th.getAttribute('data-sort');
6640 |         const indicator = th.querySelector('.sort-indicator') || (() => { const s = document.createElement('span'); s.className = 'sort-indicator'; th.appendChild(s); return s; })();
6641 |         indicator.textContent = ccDetalleSortCol === col ? (ccDetalleSortDir === 1 ? '▲' : '▼') : '';
6642 |         th.onclick = () => {
6643 |             if (ccDetalleVistaRowsActual.length === 0) return;
6644 |             if (ccDetalleSortCol === col) ccDetalleSortDir *= -1;
6645 |             else { ccDetalleSortCol = col; ccDetalleSortDir = 1; }
6646 |             const sorted = [...ccDetalleVistaRowsActual].sort(({a, b} => compareCcDetalleRow(a, b, col, ccDetalleSortDir));
6647 |             ccDetalleVistaRowsActual = sorted;
6648 |             renderCcVistaDetalle(sorted);
6649 |         };
6650 |     });
6651 | }
6652 |
6653 | /** Exporta la tabla actual de cuenta corriente (resumen o detalle según vista, filtro tipo e incluir cero). */

```

```

6654 function exportarCcResumenExcel() {
6655   if (ccVistaToggle === 'detalle') {
6656     let filtrados = ccMovimientosDetalleList.filter((m) => m.tipo === ccFiltroTipo);
6657     if (ccDetalleFiltroEntidadId) {
6658       if (ccFiltroTipo === 'cliente') {
6659         filtrados = filtrados.filter((m) => m.cliente_id === ccDetalleFiltroEntidadId);
6660       } else {
6661         filtrados = filtrados.filter((m) => m.intermediario_id === ccDetalleFiltroEntidadId);
6662       }
6663     }
6664     if (!ccMovimientosMostrarTodoHistorial) {
6665       const hoyStr = fechaHoyYYYYMMDDArgentina();
6666       const desde = ccDetalleDesde || hoyStr;
6667       const hasta = ccDetalleHasta || hoyStr;
6668       filtrados = filtrados.filter((m) => {
6669         const f = (m.fecha || '').toString().slice(0, 10);
6670         if (desde && f < desde) return false;
6671         if (hasta && f > hasta) return false;
6672         return true;
6673       });
6674     }
6675     if (filtrados.length === 0) {
6676       showToast('No hay movimientos para exportar.', 'info');
6677       return;
6678     }
6679     const idsUsuario = filtrados.map((m) => m.usuario_id).filter(Boolean);
6680     fetchMapsEmailUsuarios(idsUsuario).then((emailPorUsuario) => {
6681       const monedasExp = MONEDAS_CC_MOVIMIENTOS_COLS.filter((mon) => ccUiMonedasVisibles[mon]);
6682       const header = ['Fecha', 'Tipo op.', 'Orden', 'Trans.', 'Concepto', ...monedasExp, 'Estado', 'Pagador', 'Cobrador', 'Usuario (registro)'];
6683       const rows = filtrados.map((m) => {
6684         const tienePorMoneda = m.monto_usd != null || m.monto_ars != null || m.monto_eur != null;
6685         let usd = null, ars = null, eur = null;
6686         if (tienePorMoneda) {
6687           usd = m.monto_usd != null ? Number(m.monto_usd) : null;
6688           ars = m.monto_ars != null ? Number(m.monto_ars) : null;
6689           eur = m.monto_eur != null ? Number(m.monto_eur) : null;
6690         } else if (m.moneda && m.monto != null) {
6691           const val = Number(m.monto);
6692           if (m.moneda === 'USD') usd = val;
6693           else if (m.moneda === 'ARS') ars = val;
6694           else if (m.moneda === 'EUR') eur = val;
6695         }
6696         const porMon = { USD: usd, ARS: ars, EUR: eur };
6697         const estado = (m.estado === 'pendiente' ? 'Pendiente' : (m.estado === 'cerrado' ? 'Cerrado' : (m.estado || '-')));
6698         const nroOrden = m.orden_numero != null ? Number(m.orden_numero) : null;
6699         const nroTrans = m.transaccion_numero != null ? Number(m.transaccion_numero) : null;
6700         const tipoOp = m.es_movimiento_manual ? 'Manual' : ((m.tipo_operacion != null && m.tipo_operacion !== '-') ? String(m.tipo_operacion) : '-');
6701         const celdasMon = monedasExp.map((mon) => porMon[mon]);
6702         const uid = m.usuario_id != null ? String(m.usuario_id) : '';
6703         const mailReg = uid ? (emailPorUsuario[uid] || '') : '';
6704         return [(m.fecha || '').toString().slice(0, 10), tipoOp, nroOrden, nroTrans, m.concepto || '', ...celdasMon, estado, m.ccPagador || '', m.ccCobrador || '', mailReg || '-'];
6705       });
6706       const aoa = aoaExcelConMetaExportacion(header, rows);
6707       const ws = XLSX.utils.aoa_to_sheet(aoa);
6708       const wb = XLSX.utils.book_new();
6709       XLSX.utils.book_append_sheet(wb, ws, 'CC detalle movimientos');
6710       const nombreArchivo = 'cc_detalle_movimientos_' + fechaHoyYYYYMMDDArgentina() + '.xlsx';
6711       XLSX.writeFile(wb, nombreArchivo);
6712       showToast('Exportado: ' + nombreArchivo, 'success');
6713     });
6714     return;
6715   }
6716   const EPSILON_SALDO = 1e-6;
6717   const conSaldo = (r) => Math.abs(Number(r.saldos.USD) || 0) >= EPSILON_SALDO || Math.abs(Number(r.saldos.EUR) || 0) >= EPSILON_SALDO || Math.abs(Number(r.saldos.ARS) || 0) >= EPSILON_SALDO;
6718   const conSaldoList = ccResumenRowsTodos.filter(conSaldo);
6719   const filtrados = conSaldoList.filter((r) => r.tipo === ccFiltroTipo);
6720   if (filtrados.length === 0) {
6721     showToast('No hay filas para exportar.', 'info');
6722     return;
6723   }
6724   const monedasExp = MONEDAS_CC_UI.filter((mon) => ccUiMonedasVisibles[mon]);
6725   const header = ['Nombre', 'Tipo', ...monedasExp];
6726   const rows = filtrados.map((r) => {
6727     const tipoLabel = r.tipo === 'cliente' ? 'Cliente' : 'Intermediario';
6728     const cels = [r.nombre || '', tipoLabel];
6729     const pendClase = r.pendienteClasePorMoneda || { USD: 'ninguno', EUR: 'ninguno', ARS: 'ninguno' };
6730     monedasExp.forEach((mon) => {
6731       const sDisp = ccSaldoDisplayOpticaResumen(Number(r.saldos[mon]) || 0, pendClase[mon]);
6732       cels.push(sDisp);
6733     });
6734     return cels;
6735   });
6736   const aoa = aoaExcelConMetaExportacion(header, rows);
6737   const ws = XLSX.utils.aoa_to_sheet(aoa);
6738   const wb = XLSX.utils.book_new();
6739   XLSX.utils.book_append_sheet(wb, ws, 'Cuenta corriente');
6740   const nombreArchivo = 'cuenta_corriente_' + fechaHoyYYYYMMDDArgentina() + '.xlsx';
6741   XLSX.writeFile(wb, nombreArchivo);
6742   showToast('Exportado: ' + nombreArchivo, 'success');
6743 }
6744
6745 /** Compromiso por moneda desde órdenes (solo no anuladas y no ejecutadas): +monto_recibido en moneda_recibida, -monto_entregado en moneda_entregada. Las ejecutadas ya están r
6746 ealizadas en movimientos (incl. comisión/ganancia). */
6747 function compromisoDesdeOrdenes(ordenes, entityId, campoId) {
6748   const comp = { USD: 0, EUR: 0, ARS: 0 };

```

```

6748 | (ordenes || []).forEach((o) => {
6749 |   if (o.estado === 'orden_ejecutada') return;
6750 |   if (o[campoId] !== entityId) return;
6751 |   const monR = o.moneda_recibida;
6752 |   const monE = o.moneda_entregada;
6753 |   const mR = Number(o.monto_recibido) || 0;
6754 |   const mE = Number(o.monto_entregado) || 0;
6755 |   if (comp[monR] != null) comp[monR] += mR;
6756 |   if (monE && comp[monE] != null) comp[monE] -= mE;
6757 | });
6758 | return comp;
6759 | }
6760 |
6761 | /** Devuelve Promise<{ movimientos, saldos, ordenes, pendienteClasePorMoneda }>. Saldo = suma de movimientos no anulados (coherente con resumen CC). */
6762 | function fetchMovimientosCpPorEntidad(tipo, entityId) {
6763 |   const campoId = tipo === 'cliente' ? 'cliente_id' : 'intermediario_id';
6764 |   const tablaMov = tipo === 'cliente' ? 'movimientos_cuenta_corriente' : 'movimientos_cuenta_corriente_intermediario';
6765 |   const filtroMov = tipo === 'cliente' ? { cliente_id: entityId } : { intermediario_id: entityId };
6766 |   const selectMov = tipo === 'cliente'
6767 |     ? 'id, moneda, monto, concepto, fecha, estado, estado_fecha, monto_usd, monto_ars, monto_eur, orden_id, transaccion_id, transaccion_numero, incluir_en_detalle, es_movimien
6768 |       to_manual, manual_tip_movimiento, usuario_id' + CC.MOV_MANUAL_PAG_COB_COLS
6769 |       : 'id, moneda, monto, concepto, fecha, estado, estado_fecha, monto_usd, monto_ars, monto_eur, orden_id, transaccion_id, transaccion_numero, incluir_en_detalle, es_movimien
6770 |       to_manual, manual_tip_movimiento, usuario_id' + CC.MOV_MANUAL_PAG_COB_COLS;
6771 |   return Promise.all([
6772 |     client.from(tablaMov).select(selectMov).match(filtroMov).order('fecha', { ascending: false }).order('created_at', { ascending: false }),
6773 |     client.from('ordenes').select(ordenesTieneNumeroColumn ? 'id, numero, cliente_id, intermediario_id, fecha, estado, moneda_recibida, monto_recibido, moneda_entregada, monto
6774 |       entregado, tipo_operacion(codigo, nombre, icono_modo, icono_url_publica, moneda_in, moneda_out, usa_intermediario)' : 'id, cliente_id, intermediario_id, f
6775 |       echa, estado, moneda_recibida, monto_recibido, moneda_entregada, monto_entregado, tipo_operacion(codigo, nombre, icono_modo, icono_url_publica, moneda_in,
6776 |       moneda_out, usa_intermediario)').neg('estado', 'anulada').match({ [campoId]: entityId }),
6777 |   ]).then(([rMov, rOrd]) => {
6778 |     if (rMov.error) return Promise.reject(rMov.error);
6779 |     if (rOrd.error) return Promise.reject(rOrd.error);
6780 |     const movimientos = rMov.data || [];
6781 |     const ordenes = rOrd.data || [];
6782 |     const transaccionIds = [...new Set((movimientos.map((m) => m.transaccion_id)).filter(Boolean))];
6783 |     const ordenIds = [...new Set((ordenes.map((o) => o.orden_id)).filter(Boolean))];
6784 |     return Promise.all([
6785 |       transaccionIds.length > 0 ? client.from('transacciones').select('id, estado, tipo, pagador, cobrador, pagador_cliente_id, cobrador_cliente_id, pagador_intermediario_id,
6786 |         cobrador_intermediario_id').in('id', transaccionIds) : Promise.resolve({ data: [] }),
6787 |       ordenIds.length > 0 ? client.from('instrumentacion').select('id, orden_id').in('orden_id', ordenIds) : Promise.resolve({ data: [] }),
6788 |     ]).then(([rTr, rInst]) => {
6789 |       const trById = {};
6790 |       const trTipoById = {};
6791 |       const trPagadorById = {};
6792 |       const trCobradorById = {};
6793 |       (rTr.data || []).forEach((t) => {
6794 |         if (t.id == null || t.id === '') return;
6795 |         const id = String(t.id);
6796 |         trById[id] = t.estado;
6797 |         if (t.tipo != null) trTipoById[id] = String(t.tipo).toLowerCase();
6798 |         trPagadorById[id] = (t.pagador || '').toString().toLowerCase();
6799 |         if (t.cobrador != null) trCobradorById[id] = String(t.cobrador).toLowerCase();
6800 |       });
6801 |       const instByOrden = {};
6802 |       (rInst.data || []).forEach((i) => { instByOrden[i.orden_id] = i.id; });
6803 |       const instIds = (rInst.data || []).map((i) => i.id).filter(Boolean);
6804 |       const promTrInst = instIds.length > 0
6805 |         ? client.from('transacciones').select('id, instrumentacion_id, estado, tipo, pagador, cobrador, monto, moneda, pagador_cliente_id, cobrador_cliente_id, pagador_interme
6806 |           diario_id, cobrador_intermediario_id').in('instrumentacion_id', instIds)
6807 |         : Promise.resolve({ data: [] });
6808 |       return promTrInst.then((rTrInst) => ({ rTrInst, rTrMovsRaw: rTr.data || [], trById, instByOrden, trTipoById, trPagadorById, trCobradorById }));
6809 |     }).then(({ rTrInst, rTrMovsRaw, trById, instByOrden, trTipoById, trPagadorById, trCobradorById }) => {
6810 |       const pagadorClienteIdByTrx = {};
6811 |       const cobradorClienteIdByTrx = {};
6812 |       const pagadorIntermediarioIdByTrx = {};
6813 |       const cobradorIntermediarioIdByTrx = {};
6814 |       function ingestTrxParticipanteIdsModal(t) {
6815 |         if (!t || t.id == null || t.id === '') return;
6816 |         const kid = String(t.id);
6817 |         if (t.pagador_cliente_id) pagadorClienteIdByTrx[kid] = t.pagador_cliente_id;
6818 |         if (t.cobrador_cliente_id) cobradorClienteIdByTrx[kid] = t.cobrador_cliente_id;
6819 |         if (t.pagador_intermediario_id) pagadorIntermediarioIdByTrx[kid] = t.pagador_intermediario_id;
6820 |         if (t.cobrador_intermediario_id) cobradorIntermediarioIdByTrx[kid] = t.cobrador_intermediario_id;
6821 |       }
6822 |       (rTrMovsRaw || []).forEach(ingestTrxParticipanteIdsModal);
6823 |       (rTrInst.data || []).forEach(ingestTrxParticipanteIdsModal);
6824 |       cc.AbsorberMetaTransaccionesEnMaps(trPagadorById, trCobradorById, trTipoById, rTrInst.data || []);
6825 |       const trParticipanteIdsByTrx = {
6826 |         pagadorClienteIdByTrx,
6827 |         cobradorClienteIdByTrx,
6828 |         pagadorIntermediarioIdByTrx,
6829 |         cobradorIntermediarioIdByTrx,
6830 |       };
6831 |       const orderHasEjecutada = {};
6832 |       const transaccionesByOrdenId = {};
6833 |       (rTrInst.data || []).forEach((t) => {
6834 |         const ordenId = Object.keys(instByOrden || {}).find((oid) => instByOrden[oid] === t.instrumentacion_id);
6835 |         if (!ordenId) return;
6836 |         if (!transaccionesByOrdenId[ordenId]) transaccionesByOrdenId[ordenId] = [];
6837 |         transaccionesByOrdenId[ordenId].push(t);
6838 |         if (t.estado === 'ejecutada') orderHasEjecutada[ordenId] = true;
6839 |       });
6840 |       return { trById, orderHasEjecutada, trTipoById, trPagadorById, trCobradorById, transaccionesByOrdenId, trParticipanteIdsByTrx };
6841 |     }).then(({ trById, orderHasEjecutada, trTipoById, trPagadorById, trCobradorById, transaccionesByOrdenId, trParticipanteIdsByTrx }) => {
6842 |       // Saldo modal detalle: suma de todos los movimientos no anulados (misma lógica que resumen CC).
6843 |       function incluirEnSaldo(m) {
6844 |         if ((m.estado || '').toString().toLowerCase() === 'anulado') return false;

```



```

6838 |     return true;
6839 | }
6840 | const sumAll = { USD: 0, EUR: 0, ARS: 0 };
6841 | movimientos.forEach((m) => {
6842 |   if (!incluirEnSaldo(m)) return;
6843 |   if (m.monto_usd != null || m.monto_ars != null || m.monto_eur != null) {
6844 |     if (m.monto_usd != null) sumAll.USD += Number(m.monto_usd);
6845 |     if (m.monto_ars != null) sumAll.ARS += Number(m.monto_ars);
6846 |     if (m.monto_eur != null) sumAll.EUR += Number(m.monto_eur);
6847 |   } else if (m.moneda && sumAll[m.moneda] != null) {
6848 |     sumAll[m.moneda] += Number(m.monto);
6849 |   }
6850 | });
6851 | const saldos = { USD: sumAll.USD, EUR: sumAll.EUR, ARS: sumAll.ARS };
6852 | const ordenByIdFetch = Object.fromEntries(ordenes.map(o => [o.id, o]));
6853 | const pendMonFull = ccPendientePorMonedaDesdeMovs(movimientos);
6854 | let pendienteClasePorMoneda = ccPendienteClasePorMonedaDesdeMovs(
6855 |   movimientos,
6856 |   trTipoById,
6857 |   trPagadorById,
6858 |   trCobradorById,
6859 |   tipo === 'cliente' ? entityId : null,
6860 |   ordenByIdFetch,
6861 |   trParticipanteIdsByTrx
6862 | );
6863 | if (tipo === 'cliente') {
6864 |   const chequeIng = ccMapClienteChequeIngresoPrincipalPendiente(ordenByIdFetch, transaccionesByOrdenId);
6865 |   const chequePago = ccMapClienteChequeEgresoPandyClientePendiente(ordenByIdFetch, transaccionesByOrdenId);
6866 |   const pendA = pendMonFull.ARS;
6867 |   const absS = Math.abs(Number(saldos.ARS) || 0);
6868 |   const relevant = pendA || absS >= 1e-6;
6869 |   const curArs = pendienteClasePorMoneda.ARS;
6870 |   if (relevant && chequeIng[entityId] && chequePago[entityId]) {
6871 |     pendienteClasePorMoneda = { ...pendienteClasePorMoneda, ARS: 'mixto' };
6872 |   } else if (relevant && chequeIng[entityId]) {
6873 |     if (curArs === 'pago' || curArs === 'mixto') {
6874 |       pendienteClasePorMoneda = { ...pendienteClasePorMoneda, ARS: 'mixto' };
6875 |     } else {
6876 |       pendienteClasePorMoneda = { ...pendienteClasePorMoneda, ARS: 'cobro' };
6877 |     }
6878 |   } else if (relevant && chequePago[entityId]) {
6879 |     if (curArs === 'cobro' || curArs === 'mixto') {
6880 |       pendienteClasePorMoneda = { ...pendienteClasePorMoneda, ARS: 'mixto' };
6881 |     } else {
6882 |       pendienteClasePorMoneda = { ...pendienteClasePorMoneda, ARS: 'pago' };
6883 |     }
6884 |   }
6885 |   const usdIng = ccMapClienteUsdUsdIntIngresoClientePandyPendiente(ordenByIdFetch, transaccionesByOrdenId);
6886 |   const usdPago = ccMapClienteUsdUsdIntEgresoPandyClientePendiente(ordenByIdFetch, transaccionesByOrdenId);
6887 |   pendienteClasePorMoneda = mergePendienteClaseUsdUsdIntCliente(entityId, saldos, pendMonFull, pendienteClasePorMoneda, usdIng, usdPago);
6888 | } else if (tipo === 'intermediario') {
6889 |   const ingInt = ccMapIntermediarioChequeIngresoIntPandyPendiente(ordenByIdFetch, transaccionesByOrdenId);
6890 |   const egresoPandyInt = ccMapIntermediarioChequeEgresoPandyIntPendiente(ordenByIdFetch, transaccionesByOrdenId);
6891 |   const pendA = pendMonFull.ARS;
6892 |   const absS = Math.abs(Number(saldos.ARS) || 0);
6893 |   const relevant = pendA || absS >= 1e-6;
6894 |   const curArs = pendienteClasePorMoneda.ARS;
6895 |   if (relevant && ingInt[entityId] && egresoPandyInt[entityId]) {
6896 |     pendienteClasePorMoneda = { ...pendienteClasePorMoneda, ARS: 'mixto' };
6897 |   } else if (relevant && ingInt[entityId]) {
6898 |     if (curArs === 'pago' || curArs === 'mixto') {
6899 |       pendienteClasePorMoneda = { ...pendienteClasePorMoneda, ARS: 'mixto' };
6900 |     } else {
6901 |       pendienteClasePorMoneda = { ...pendienteClasePorMoneda, ARS: 'cobro' };
6902 |     }
6903 |   } else if (relevant && egresoPandyInt[entityId]) {
6904 |     if (curArs === 'cobro' || curArs === 'mixto') {
6905 |       pendienteClasePorMoneda = { ...pendienteClasePorMoneda, ARS: 'mixto' };
6906 |     } else {
6907 |       pendienteClasePorMoneda = { ...pendienteClasePorMoneda, ARS: 'pago' };
6908 |     }
6909 |   }
6910 | }
6911 | // Detalle = solo movimientos que deben mostrarse en el listado (incluir_en_detalle). Si la columna no existe, mostrar todos.
6912 | const movimientosParaDetalle = movimientos.filter((m) => m.incluir_en_detalle !== false);
6913 | const idsCliExtra = new Set(ordenes.map(o => o.cliente_id).filter(Boolean));
6914 | Object.values(trParticipanteIdsByTrx && trParticipanteIdsByTrx.pagadorClienteIdByTrx) || {}).forEach((id) => { if (id) idsCliExtra.add(id); });
6915 | Object.values(trParticipanteIdsByTrx && trParticipanteIdsByTrx.cobradorClienteIdByTrx) || {}).forEach((id) => { if (id) idsCliExtra.add(id); });
6916 | const clienteIdsFetch = [...idsCliExtra];
6917 | const idsIntExtra = new Set(ordenes.map(o => o.intermediario_id).filter(Boolean));
6918 | Object.values(trParticipanteIdsByTrx && trParticipanteIdsByTrx.pagadorIntermediarioIdByTrx) || {}).forEach((id) => { if (id) idsIntExtra.add(id); });
6919 | Object.values(trParticipanteIdsByTrx && trParticipanteIdsByTrx.cobradorIntermediarioIdByTrx) || {}).forEach((id) => { if (id) idsIntExtra.add(id); });
6920 | const intIdsFetch = [...idsIntExtra];
6921 | return Promise.all([
6922 |   clienteIdsFetch.length ? client.from('clientes').select('id, nombre').in('id', clienteIdsFetch) : Promise.resolve({ data: [] }),
6923 |   intIdsFetch.length ? client.from('intermediarios').select('id, nombre').in('id', intIdsFetch) : Promise.resolve({ data: [] }),
6924 | ]).then(([rCliDet, rIntDet]) => {
6925 |   const clientesByIdDet = Object.fromEntries((rCliDet.data || []).map((c) => [c.id, c]));
6926 |   const intermediariosByIdDet = Object.fromEntries((rIntDet.data || []).map((i) => [i.id, i]));
6927 |   const enriched = movimientosParaDetalle.map(m) => ({
6928 |     ...m,
6929 |     ...ccNombresPagadorCobradorMovimiento(m, tipo, ordenByIdFetch, clientesByIdDet, intermediariosByIdDet, trTipoById, trPagadorById, trCobradorById, trParticipanteIdsBy
Trx),
6930 |   });
6931 |   return { movimientos: enriched, saldos, ordenes, pendienteEnMoneda: pendMonFull, pendienteClasePorMoneda };
6932 | });
6933 | });

```

```

6934 | });
6935 | }
6936 |
6937 | function openModalCcDetalle(tipo, id, nombre) {
6938 |     const backdrop = document.getElementById('modal-cc-detalle-backdrop');
6939 |     const tituloEl = document.getElementById('modal-cc-detalle-titulo');
6940 |     const entityEl = document.getElementById('modal-cc-detalle-entity');
6941 |     const saldosWrap = document.getElementById('modal-cc-detalle-saldos');
6942 |     const loadingEl = document.getElementById('modal-cc-detalle-loading');
6943 |     const tablaWrap = document.getElementById('modal-cc-detalle-tabla-wrap');
6944 |     if (!backdrop || !tituloEl) return;
6945 |
6946 |     ccDetalleTipo = tipo;
6947 |     ccDetalleId = id;
6948 |     const tipoLabel = tipo === 'intermediario' ? 'Intermediario' : 'Cliente';
6949 |     tituloEl.textContent = 'Detalle de movimientos - ' + (nombre || '');
6950 |     entityEl.innerHTML = '';
6951 |     const strong = document.createElement('strong');
6952 |     strong.textContent = tipoLabel + ': ' + (nombre || '-');
6953 |     entityEl.appendChild(strong);
6954 |     saldosWrap.innerHTML = '';
6955 |     loadingEl.style.display = 'block';
6956 |     tablaWrap.style.display = 'none';
6957 |     backdrop.classList.add('activo');
6958 |
6959 |     const timeoutMs = 12000;
6960 |     const promFetch = fetchMovimientosCcPorEntidad(tipo, id);
6961 |     const promTimeout = new Promise((_, reject) => setTimeout(() => reject(new Error('Tiempo de espera agotado')), timeoutMs));
6962 |     Promise.race([promFetch, promTimeout]).then(({ movimientos, saldos, ordenes, pendienteEnMoneda, pendienteClasePorMoneda }) => {
6963 |         ccDetalleMovimientosList = ccDetalleRowsConTipoOpDesdeOrdenes(movimientos, ordenes);
6964 |         ccDetalleOrdenesList = ordenes || [];
6965 |         if (loadingEl) loadingEl.style.display = 'none';
6966 |
6967 |         const pendMonModal = pendienteEnMoneda || ccPendientePorMonedaDesdeMovs(movimientos);
6968 |         saldosWrap.innerHTML = htmlCcModalSaldosCards(saldos, pendMonModal, pendienteClasePorMoneda);
6969 |         replicarVisibilidadMonedasCuentaCorrienteDom();
6970 |
6971 |         renderCcDetalleTable();
6972 |         tablaWrap.style.display = 'block';
6973 |         renderCcDetalleOperaciones();
6974 |         const operacionesWrap = document.getElementById('modal-cc-detalle-operaciones-wrap');
6975 |         if (operacionesWrap) operacionesWrap.style.display = ccDetalleOrdenesList.length > 0 ? 'block' : 'none';
6976 |     }).catch((err) => {
6977 |         if (loadingEl) loadingEl.style.display = 'none';
6978 |         if (tablaWrap) tablaWrap.style.display = 'block';
6979 |         const msg = (err && (err.message || err.details || String(err))) || 'Error al cargar';
6980 |         showToast('Detalle CC: ' + msg, 'error', 6000);
6981 |         ccDetalleMovimientosList = [];
6982 |         ccDetalleOrdenesList = [];
6983 |         renderCcDetalleTable();
6984 |         replicarVisibilidadMonedasCuentaCorrienteDom();
6985 |     });
6986 | }
6987 |
6988 | function renderCcDetalleOperaciones() {
6989 |     const wrap = document.getElementById('modal-cc-detalle-operaciones-wrap');
6990 |     const tbody = document.getElementById('cc-detalle-operaciones-tbody');
6991 |     if (!wrap || !tbody) return;
6992 |     const todas = ccDetalleOrdenesList || [];
6993 |     const ordenes = todas.filter(o => o.estado !== 'orden ejecutada');
6994 |     const estadoLabel = (e) => ({ pendiente_instrumentar: 'Pend. Instrumentar', instrumentacion_parcial: 'Instrumentación Parcial', instrumentacion_cerrada_ejecucion: 'Cerrada e
n Ejecución', orden_ejecutada: 'Orden Ejecutada' }[e] || (e || '-'));
6995 |     if (wrap) wrap.style.display = ordenes.length > 0 ? 'block' : 'none';
6996 |     if (ordenes.length === 0) {
6997 |         tbody.innerHTML = '<tr><td colspan="7">No hay órdenes vinculadas a este cliente/intermediario.</td></tr>';
6998 |         return;
6999 |     }
7000 |     tbody.innerHTML = ordenes
7001 |         .map(o) => {
7002 |             const fecha = (o.fecha || '').toString().slice(0, 10);
7003 |             const monR = o.moneda_recibida || '-';
7004 |             const mR = formatMonto(Number(o.monto_recibido) || 0, o.moneda_recibida);
7005 |             const monE = o.moneda_entregada || '-';
7006 |             const mE = formatMonto(Number(o.monto_entregado) || 0, o.moneda_entregada);
7007 |             const est = estadoLabel(o.estado);
7008 |             return `<tr><td>${fecha}</td><td>${o.numero !== null ? '#' + o.numero : '<code>' + escapeHtml(o.id || '').slice(0, 8) + '</code>'}</td><td>${escapeHtml(monR)}</td><td>${
{mR}</td><td>${escapeHtml(monE)}</td><td>${mE}</td><td>${escapeHtml(est)}</td></tr>`;
7009 |         })
7010 |         .join('');
7011 | }
7012 |
7013 | function closeModalCcDetalle() {
7014 |     const backdrop = document.getElementById('modal-cc-detalle-backdrop');
7015 |     if (backdrop) backdrop.classList.remove('activo');
7016 |     ccDetalleTipo = null;
7017 |     ccDetalleId = null;
7018 |     ccDetalleMovimientosList = [];
7019 |     ccDetalleOrdenesList = [];
7020 |     ccDetalleModalSortCol = null;
7021 |     ccDetalleModalSortDir = 1;
7022 | }
7023 |
7024 | function renderCcDetalleTable() {
7025 |     const tbody = document.getElementById('cc-detalle-tbody');
7026 |     const tfoot = document.getElementById('cc-detalle-tfoot');
7027 |     if (!tbody) return;
7028 |

```

```

7029 | const filtrados = ccDetalleMovimientosList;
7030 | tbody.innerHTML = filtrados
7031 |   .map(m) => {
7032 |     // Un movimiento = una moneda (la de la transacción). Mostrar solo esa columna; el resto "-".
7033 |     const mon = (m.moneda || 'USD').toUpperCase();
7034 |     const valUsd = mon === 'USD' ? (m.monto.usd !== null ? Number(m.monto.usd) : Number(m.monto) || 0) : null;
7035 |     const valArs = mon === 'ARS' ? (m.monto.ars !== null ? Number(m.monto.ars) : Number(m.monto) || 0) : null;
7036 |     const valEur = mon === 'EUR' ? (m.monto.eur !== null ? Number(m.monto.eur) : Number(m.monto) || 0) : null;
7037 |     const celdaUsd = mon === 'USD' ? formatearCeldaMonedaConSigno(valUsd, 'USD') : '-';
7038 |     const celdaArs = mon === 'ARS' ? formatearCeldaMonedaConSigno(valArs, 'ARS') : '-';
7039 |     const celdaEur = mon === 'EUR' ? formatearCeldaMonedaConSigno(valEur, 'EUR') : '-';
7040 |     const estadoLabel = (m.estado === 'pendiente' ? 'Pendiente' : (m.estado === 'cerrado' ? 'Cerrado' : (m.estado || '-')));
7041 |     const nroOrden = m.orden.numero !== null ? String(m.orden.numero) : '-';
7042 |     const nroTrans = m.transaccion.numero !== null ? String(m.transaccion.numero) : '-';
7043 |     const tipoOp = (m.tipo_operación !== null && m.tipo_operacion !== '-') ? String(m.tipo_operacion) : '-';
7044 |     const tipoOpHtml = m.es_movimiento_manual
7045 |       ? '<span class="cc-tipo-op-manual" title="Sin orden">Manual</span>'
7046 |       : (tipoOp === '-' ? '-' : htmlTipoOperacionIconos(tipoOp, m.tipo_op_nombre || '', { iconoModo: m.tipo_op_icono_modo, iconoUrlPublica: m.tipo_op_icono_url, usaIntermediario: m.tipo_op_usa_intermediario === true }));
7047 |     return `<tr>
7048 |       <td class="cc-col-fija cc-col-fija-1">${(m.fecha || '').toString().slice(0, 10)}</td>
7049 |       <td class="cc-col-fija cc-col-fija-2 cc-col-tipo-op-icomas">${tipoOpHtml}</td>
7050 |       <td>${escapeHtml(nroOrden)}</td>
7051 |       <td>${escapeHtml(nroTrans)}</td>
7052 |       <td class="td-concepto">${escapeHtml(m.concepto || '-')}</td>
7053 |       <td data-cc-moneda-col="USD">${celdaUsd}</td>
7054 |       <td data-cc-moneda-col="ARS">${celdaArs}</td>
7055 |       <td data-cc-moneda-col="EUR">${celdaEur}</td>
7056 |       <td>${escapeHtml(estadoLabel)}</td>
7057 |       <td>${escapeHtml(m.ccPagador || '-')}</td>
7058 |       <td>${escapeHtml(m.ccCobrador || '-')}</td>
7059 |       <td class="cc-col-acciones-manual">${htmlCcAccionesMovimientoManualRow(m)}</td>
7060 |     </tr>;
7061 |   }
7062 |   .join('');
7063 |
7064 | if (tfoot) tfoot.innerHTML = filtrados.length === 0 ? '' : '';
7065 | if (filtrados.length === 0) tbody.innerHTML = '<tr><td colspan="1" + ccColspanModalDetalleMovimientos() + ">No hay movimientos.</td></tr>';
7066 | else setupCcDetalleModalSortHeaders();
7067 | reaplicarVisibilidadMonedasCuentaCorrienteDom();
7068 | setupDelegacionAccionesCcManual();
7069 | }
7070 |
7071 | function setupCcDetalleModalSortHeaders() {
7072 |   const table = document.getElementById('tabla-cc-detalle-modal');
7073 |   if (!table) return;
7074 |   const thead = table.querySelector('thead tr');
7075 |   if (!thead) return;
7076 |   thead.querySelectorAll('th[data-sort]').forEach((th) => {
7077 |     const col = th.getAttribute('data-sort');
7078 |     const indicator = th.querySelector('.sort-indicator') || (() => { const s = document.createElement('span'); s.className = 'sort-indicator'; th.appendChild(s); return s; })();
7079 |     indicator.textContent = ccDetalleModalSortCol === col ? (ccDetalleModalSortDir === 1 ? '▲' : '▼') : '';
7080 |     th.onclick = () => {
7081 |       if (ccDetalleMovimientosList.length === 0) return;
7082 |       if (ccDetalleModalSortCol === col) ccDetalleModalSortDir *= -1;
7083 |       else { ccDetalleModalSortCol = col; ccDetalleModalSortDir = 1; }
7084 |       ccDetalleMovimientosList = [...ccDetalleMovimientosList].sort((a, b) => compareCcDetalleRow(a, b, col, ccDetalleModalSortDir));
7085 |       renderCcDetalleTable();
7086 |     };
7087 |   });
7088 | }
7089 |
7090 | /** Carga CC de un intermediario: misma regla que resumen - saldo = solo movimientos cerrados (se excluye anulado y pendiente). */
7091 | function loadCuentaCorrienteIntermediario(intermediarioId) {
7092 |   const loadingEl = document.getElementById('cc-loading');
7093 |   const wrapEl = document.getElementById('cc-tabla-wrap');
7094 |   const tbody = document.getElementById('cc-tbody');
7095 |   if (!loadingEl || !wrapEl || !tbody) return;
7096 |
7097 |   loadingEl.style.display = 'block';
7098 |   wrapEl.style.display = 'none';
7099 |
7100 |   fetchMovimientosCcPorEntidad('intermediario', intermediarioId)
7101 |     .then(({ movimientos, saldos }) => {
7102 |       loadingEl.style.display = 'none';
7103 |       ccMovimientosList = movimientos || [];
7104 |       setCcSaldoCards(saldos || { USD: 0, EUR: 0, ARS: 0 });
7105 |       renderCcTable();
7106 |       wrapEl.style.display = 'block';
7107 |     })
7108 |     .catch((err) => {
7109 |       loadingEl.style.display = 'none';
7110 |       tbody.innerHTML = '<tr><td colspan="9">Error: ' + (err && err.message ? err.message : 'No se pudo cargar.') + '</td></tr>';
7111 |       wrapEl.style.display = 'block';
7112 |     });
7113 | }
7114 |
7115 | /** Carga CC de un cliente: saldo = solo movimientos cerrados (se excluye anulado y pendiente). */
7116 | function loadCuentaCorrienteCliente(clienteId) {
7117 |   const loadingEl = document.getElementById('cc-loading');
7118 |   const wrapEl = document.getElementById('cc-tabla-wrap');
7119 |   const tbody = document.getElementById('cc-tbody');
7120 |   if (!loadingEl || !wrapEl || !tbody) return;
7121 |
7122 |   loadingEl.style.display = 'block';
7123 |   wrapEl.style.display = 'none';

```

```

7124 | fetchMovimientosCcPorEntidad('cliente', clienteId)
7125 | .then(({ movimientos, saldos }) => {
7126 |   loadingEl.style.display = 'none';
7127 |   ccMovimientosList = movimientos || [];
7128 |   setCcSaldoCards(saldos || { USD: 0, EUR: 0, ARS: 0 });
7129 |   renderCcTable();
7130 |   wrapEl.style.display = 'block';
7131 | })
7132 | .catch((err) => {
7133 |   loadingEl.style.display = 'none';
7134 |   tbody.innerHTML = '<tr><td colspan="9">Error: ' + (err && err.message ? err.message : 'No se pudo cargar.') + '</td></tr>';
7135 |   wrapEl.style.display = 'block';
7136 | });
7137 | }
7138 |
7139 | function renderCcTable() {
7140 |   const tbody = document.getElementById('cc-tbody');
7141 |   const tfoot = document.getElementById('cc-tfoot');
7142 |   const moneda = ccMonedaActual === 'TODAS' ? null : ccMonedaActual;
7143 |   const filtrados = moneda ? ccMovimientosList.filter((m) => m.moneda === moneda) : ccMovimientosList;
7144 |   const canAbmCc = !ccEsIntermediario && (
7145 |     userPermissions.includes('editar_transacciones')
7146 |     || userPermissions.includes('editar_movimiento_cc_manual')
7147 |   );
7148 |   if (!tbody) return;
7149 |
7150 |   const totals = { USD: { debe: 0, haber: 0 }, EUR: { debe: 0, haber: 0 }, ARS: { debe: 0, haber: 0 } };
7151 |   filtrados.forEach((m) => {
7152 |     const n = Number(m.monto);
7153 |     const mon = m.moneda;
7154 |     if (totals[mon]) {
7155 |       if (n > 0) totals[mon].debe += n;
7156 |       else totals[mon].haber += -n;
7157 |     }
7158 |   });
7159 |
7160 |   tbody.innerHTML = filtrados
7161 |     .map((m) => {
7162 |       const n = Number(m.monto);
7163 |       const mon = m.moneda;
7164 |       const debeUsd = mon === 'USD' ? (n > 0 ? formatMonto(n, 'USD') : '-') : '-';
7165 |       const haberUsd = mon === 'USD' ? (n <= 0 ? formatMonto(-n, 'USD') : '-') : '-';
7166 |       const debeEur = mon === 'EUR' ? (n > 0 ? formatMonto(n, 'EUR') : '-') : '-';
7167 |       const haberEur = mon === 'EUR' ? (n <= 0 ? formatMonto(-n, 'EUR') : '-') : '-';
7168 |       const debeArs = mon === 'ARS' ? (n > 0 ? formatMonto(n, 'ARS') : '-') : '-';
7169 |       const haberArs = mon === 'ARS' ? (n <= 0 ? formatMonto(-n, 'ARS') : '-') : '-';
7170 |       return `<tr>
7171 |         <td>${(m.fecha || '').toString().slice(0, 10)}</td>
7172 |         <td>${escapeHtml(m.concepto || '-')}</td>
7173 |         <td>${debeUsd}</td><td>${haberUsd}</td>
7174 |         <td>${debeEur}</td><td>${haberEur}</td>
7175 |         <td>${debeArs}</td><td>${haberArs}</td>
7176 |         <td>${canAbmCc && !m.es_movimiento_manual && userPermissions.includes('editar_transacciones') ? `<button type="button" class="btn-editar btn-editar-cc" data-id="${m.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M11 4H4a2 2 0 0-2 2v14a2 2 0 00 2h14a2 2 0 00 2v-7"></path>
7177 |         d="M18.5 2.5a2.121 2.121 0 013.112 15.4 1 1-4 9.5-9.5z"></svg></span>Editar</button>` : ''}</td>
7178 |       </tr>`;
7179 |     })
7180 |     .join('');
7181 |
7182 |   if (tfoot) {
7183 |     tfoot.innerHTML = filtrados.length === 0
7184 |       ? `
7185 |         : <tr>
7186 |           <td colspan="2">Total</td>
7187 |           <td>${formatMonto(totals.USD.debe, 'USD')}</td><td>${formatMonto(totals.USD.haber, 'USD')}</td>
7188 |           <td>${formatMonto(totals.EUR.debe, 'EUR')}</td><td>${formatMonto(totals.EUR.haber, 'EUR')}</td>
7189 |           <td>${formatMonto(totals.ARS.debe, 'ARS')}</td><td>${formatMonto(totals.ARS.haber, 'ARS')}</td>
7190 |         </tr>`;
7191 |       : ``;
7192 |   }
7193 |
7194 |   if (filtrados.length === 0) tbody.innerHTML = '<tr><td colspan="9">No hay movimientos.</td></tr>';
7195 |   else if (canAbmCc) {
7196 |     tbody.querySelectorAll('.btn-editar-cc').forEach((btn) => {
7197 |       btn.addEventListener('click', () => {
7198 |         const id = btn.getAttribute('data-id');
7199 |         const mov = ccMovimientosList.find((x) => x.id === id);
7200 |         if (mov) openModalMovimientoCc(mov);
7201 |       });
7202 |     });
7203 |   }
7204 | }
7205 |
7206 | function openModalMovimientoCc(mov) {
7207 |   if (mov && mov.es_movimiento_manual) {
7208 |     showToast('Los movimientos manuales se editan desde la solapa Movimientos (botón lápiz en la fila).', 'info');
7209 |     return;
7210 |   }
7211 |   const backdrop = document.getElementById('modal-movimiento-cc-backdrop');
7212 |   const idEl = document.getElementById('mov-cc-id');
7213 |   const conceptoEl = document.getElementById('mov-cc-concepto');
7214 |   const fechaEl = document.getElementById('mov-cc-fecha');
7215 |   if (!backdrop || !idEl) return;
7216 |   idEl.value = mov.id;
7217 |   conceptoEl.value = mov.concepto || '';
7218 |   fechaEl.value = (mov.fecha || '').toString().slice(0, 10);

```

```

7219 | backdrop.classList.add('activo');
7220 | }
7221 |
7222 | function closeModalMovimientoCc() {
7223 |   const backdrop = document.getElementById('modal-movimiento-cc-backdrop');
7224 |   if (backdrop) backdrop.classList.remove('activo');
7225 | }
7226 |
7227 | function saveMovimientoCc() {
7228 |   const idEl = document.getElementById('mov-cc-id');
7229 |   const id = idEl && idEl.value ? idEl.value.trim() : '';
7230 |   const movLegacy = (ccMovimientosList || []).find(x => String(x.id) === String(id));
7231 |   if (movLegacy && movLegacy.es_movimiento_manual) {
7232 |     showToast('Los movimientos manuales se editan desde la solapa Movimientos.', 'info');
7233 |     return;
7234 |   }
7235 |   const concepto = document.getElementById('mov-cc-concepto').value.trim() || null;
7236 |   const fecha = document.getElementById('mov-cc-fecha').value;
7237 |   if (!id || !fecha) {
7238 |     showToast('Falta fecha.', 'error');
7239 |     return;
7240 |   }
7241 |   client
7242 |     .from('movimientos_cuenta_corriente')
7243 |     .update({ concepto, fecha })
7244 |     .eq('id', id)
7245 |     .then((res) => {
7246 |       if (res.error) {
7247 |         showToast('Error: ' + (res.error.message || 'No se pudo guardar.'), 'error');
7248 |         return;
7249 |       }
7250 |       closeModalMovimientoCc();
7251 |       if (ccDetalleId && ccDetalleTipo) {
7252 |         fetchMovimientosCcPorEntidad(ccDetalleTipo, ccDetalleId).then(({ movimientos, saldos, ordenes, pendienteEnMoneda, pendienteClasePorMoneda }) => {
7253 |           ccDetalleMovimientosList = ccDetalleRowsConTipoOpDesdeOrdenes(movimientos, ordenes);
7254 |           ccDetalleOrdenesList = ordenes || [];
7255 |           renderCcDetalleTable();
7256 |           const saldosWrap = document.getElementById('modal-cc-detalle-saldos');
7257 |           if (saldosWrap && saldos) {
7258 |             const pendMonModal = pendienteEnMoneda || ccPendientePorMonedaDesdeMovs(movimientos);
7259 |             saldosWrap.innerHTML = htmlCcModalSaldosCards(saldos, pendMonModal, pendienteClasePorMoneda);
7260 |             reaplicarVisibilidadMonedasCuentaCorrienteDom();
7261 |           }
7262 |           renderCcDetalleOperaciones();
7263 |           const operacionesWrap = document.getElementById('modal-cc-detalle-operaciones-wrap');
7264 |           if (operacionesWrap) operacionesWrap.style.display = (ccDetalleOrdenesList.length > 0) ? 'block' : 'none';
7265 |         });
7266 |       }
7267 |       loadCuentaCorriente();
7268 |     });
7269 | }
7270 |
7271 | function setupModalMovimientoCc() {
7272 |   const backdrop = document.getElementById('modal-movimiento-cc-backdrop');
7273 |   const btnClose = document.getElementById('modal-movimiento-cc-close');
7274 |   const btnCancel = document.getElementById('modal-movimiento-cc-cancelar');
7275 |   const form = document.getElementById('form-movimiento-cc');
7276 |   if (btnClose) btnClose.addEventListener('click', closeModalMovimientoCc);
7277 |   if (btnCancel) btnCancel.addEventListener('click', closeModalMovimientoCc);
7278 |   if (backdrop) setupBackdropCloseOnlyOnRealClick(backdrop, closeModalMovimientoCc);
7279 |   if (form) form.addEventListener('submit', (e) => { e.preventDefault(); saveMovimientoCc(); });
7280 | }
7281 |
7282 | /** Mensaje claro si falla insert CC manual por orden_id NOT NULL en Supabase. */
7283 | function mensajeErrorCcInsertSupabase(msg) {
7284 |   const m = String(msg || '').toLowerCase();
7285 |   if (m.includes('orden_id') && (m.includes('not-null') || m.includes('violates not-null'))) {
7286 |     return 'La tabla de CC exige orden en tu base; los manuales van sin orden. Ejecutá en Supabase: sql/migracion_cc_movimientos_orden_id_nullable.sql (o el bloque $4 de migra
7287 |   }
7288 |   return msg || '';
7289 | }
7290 |
7291 | function ccManualLeerRolRadio(name) {
7292 |   const r = document.querySelector('input[name="' + name + ""]:checked');
7293 |   const v = r && r.value;
7294 |   if (v === 'cliente' || v === 'intermediario' || v === 'pandy') return v;
7295 |   return 'cliente';
7296 | }
7297 |
7298 | /** Dinero Pagador → Cobrador: si paga la empresa → egreso caja; si cobra la empresa → ingreso. */
7299 | function ccManualDireccionCajaDesdeFlujo(pagRol, cobRol) {
7300 |   if (pagRol === 'pandy' && cobRol !== 'pandy') return 'egreso';
7301 |   if (cobRol === 'pandy' && pagRol !== 'pandy') return 'ingreso';
7302 |   return null;
7303 | }
7304 |
7305 | function ccManualParticipaEmpresa(pagRol, cobRol) {
7306 |   return pagRol === 'pandy' || cobRol === 'pandy';
7307 | }
7308 |
7309 | function ccManualNuevoGrupoId() {
7310 |   if (typeof crypto !== 'undefined' && typeof crypto.randomUUID === 'function') return crypto.randomUUID();
7311 |   return 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, (c) => {
7312 |     const n = Math.random() * 16 | 0;
7313 |     return (c === 'x' ? n : (n & 0x3 | 0x8)).toString(16);
7314 |   });

```

```

7315 | }
7316 |
7317 | /** Texto plano para concepto (nombre de marca o opción del combo). */
7318 | function ccManualTextoLugarRol(rol, lado) {
7319 |     if (rol === 'pandy') {
7320 |         const sp = document.querySelector('.js-marca-sistema-nombre');
7321 |         const t = sp && sp.textContent && sp.textContent.trim();
7322 |         return t || 'Empresa';
7323 |     }
7324 |     const selId = lado === 'pag'
7325 |         ? (rol === 'cliente' ? 'cc-manual-pagador-cliente' : 'cc-manual-pagador-intermediario')
7326 |         : (rol === 'cliente' ? 'cc-manual-cobrador-cliente' : 'cc-manual-cobrador-intermediario');
7327 |     const s = document.getElementById(selId);
7328 |     if (!s || !s.value || !s.options[s.selectedIndex]) return rol === 'cliente' ? 'Cliente' : 'Intermediario';
7329 |     return (s.options[s.selectedIndex].text || '').trim() || '-';
7330 | }
7331 |
7332 | /**
7333 |  * Arma las patas a insertar en tablas CC (cliente ↔ empresa / intermediario ↔ empresa).
7334 |  * Pagador no-empresa + Cobrador no-empresa → dos movimientos enlazados (mismo grupo).
7335 |  */
7336 | function ccManualConstruirLegs(pagRol, cobRol, pagCli, pagInt, cobCli, cobInt) {
7337 |     if (pagRol === 'pandy' && cobRol === 'pandy') {
7338 |         return { error: 'Pagador y cobrador no pueden ser ambos la empresa.' };
7339 |     }
7340 |     const legs = [];
7341 |     if (pagRol === 'cliente' && cobRol === 'cliente') {
7342 |         if (!pagCli || !cobCli || pagCli === cobCli) return { error: 'Elegi dos clientes distintos como pagador y cobrador.' };
7343 |         legs.push({ kind: 'cliente', cliente_id: pagCli, tip: 'cobro_entidad_pandy' });
7344 |         legs.push({ kind: 'cliente', cliente_id: cobCli, tip: 'pago_pandy_entidad' });
7345 |         return { legs };
7346 |     }
7347 |     if (pagRol === 'intermediario' && cobRol === 'intermediario') {
7348 |         if (!pagInt || !cobInt || pagInt === cobInt) return { error: 'Elegi dos intermediarios distintos.' };
7349 |         legs.push({ kind: 'intermediario', intermediario_id: pagInt, tip: 'cobro_entidad_pandy' });
7350 |         legs.push({ kind: 'intermediario', intermediario_id: cobInt, tip: 'pago_pandy_entidad' });
7351 |         return { legs };
7352 |     }
7353 |     if (pagRol === 'cliente' && cobRol === 'intermediario') {
7354 |         if (!pagCli || !cobInt) return { error: 'Completá cliente pagador e intermediario cobrador.' };
7355 |         legs.push({ kind: 'cliente', cliente_id: pagCli, tip: 'cobro_entidad_pandy' });
7356 |         legs.push({ kind: 'intermediario', intermediario_id: cobInt, tip: 'pago_pandy_entidad' });
7357 |         return { legs };
7358 |     }
7359 |     if (pagRol === 'intermediario' && cobRol === 'cliente') {
7360 |         if (!pagInt || !cobCli) return { error: 'Completá intermediario pagador y cliente cobrador.' };
7361 |         legs.push({ kind: 'intermediario', intermediario_id: pagInt, tip: 'cobro_entidad_pandy' });
7362 |         legs.push({ kind: 'cliente', cliente_id: cobCli, tip: 'pago_pandy_entidad' });
7363 |         return { legs };
7364 |     }
7365 |     if (pagRol === 'cliente' && cobRol === 'pandy') {
7366 |         if (!pagCli) return { error: 'Elegi el cliente pagador.' };
7367 |         legs.push({ kind: 'cliente', cliente_id: pagCli, tip: 'cobro_entidad_pandy' });
7368 |         return { legs };
7369 |     }
7370 |     if (pagRol === 'pandy' && cobRol === 'cliente') {
7371 |         if (!cobCli) return { error: 'Elegi el cliente cobrador.' };
7372 |         legs.push({ kind: 'cliente', cliente_id: cobCli, tip: 'pago_pandy_entidad' });
7373 |         return { legs };
7374 |     }
7375 |     if (pagRol === 'intermediario' && cobRol === 'pandy') {
7376 |         if (!pagInt) return { error: 'Elegi el intermediario pagador.' };
7377 |         legs.push({ kind: 'intermediario', intermediario_id: pagInt, tip: 'cobro_entidad_pandy' });
7378 |         return { legs };
7379 |     }
7380 |     if (pagRol === 'pandy' && cobRol === 'intermediario') {
7381 |         if (!cobInt) return { error: 'Elegi el intermediario cobrador.' };
7382 |         legs.push({ kind: 'intermediario', intermediario_id: cobInt, tip: 'pago_pandy_entidad' });
7383 |         return { legs };
7384 |     }
7385 |     return { error: 'Combinación pagador/cobrador no válida.' };
7386 | }
7387 |
7388 | function ccManualPoblarSelectEntidades(sel, rows) {
7389 |     if (!sel) return;
7390 |     const sorted = [...(rows || [])].sort((a, b) => String(a.nombre || '').localeCompare(String(b.nombre || ''), 'es'));
7391 |     sel.innerHTML = '<option value="">Elegir...</option>';
7392 |     + sorted.map((row) => '<option value="' + escapeHtml(String(row.id)) + '">' + escapeHtml(row.nombre || '-') + '</option>').join('');
7393 | }
7394 |
7395 | function ccManualPoblarTodosLosSelects(clientes, intermediarios) {
7396 |     ccManualPoblarSelectEntidades(document.getElementById('cc-manual-pagador-cliente'), clientes);
7397 |     ccManualPoblarSelectEntidades(document.getElementById('cc-manual-pagador-intermediario'), intermediarios);
7398 |     ccManualPoblarSelectEntidades(document.getElementById('cc-manual-cobrador-cliente'), clientes);
7399 |     ccManualPoblarSelectEntidades(document.getElementById('cc-manual-cobrador-intermediario'), intermediarios);
7400 | }
7401 |
7402 | function ccManualSyncVisibilidadRoles() {
7403 |     const pagRol = ccManualLeerRolRadio('cc-manual-pagador-rol');
7404 |     const cobRol = ccManualLeerRolRadio('cc-manual-cobrador-rol');
7405 |     const wpc = document.getElementById('cc-manual-wrap-pag-cliente');
7406 |     const wpi = document.getElementById('cc-manual-wrap-pag-inter');
7407 |     const wcc = document.getElementById('cc-manual-wrap-cob-cliente');
7408 |     const wci = document.getElementById('cc-manual-wrap-cob-inter');
7409 |     if (wpc) wpc.style.display = pagRol === 'cliente' ? 'block' : 'none';
7410 |     if (wpi) wpi.style.display = pagRol === 'intermediario' ? 'block' : 'none';
7411 |     if (wcc) wcc.style.display = cobRol === 'cliente' ? 'block' : 'none';

```

```

7412|   if (wci) wci.style.display = cobRol === 'intermediario' ? 'block' : 'none';
7413| }
7414|
7415| function ccManualNombreTipoCajaFijoPorDireccion(dir) {
7416|   const d = String(dir || '').toLowerCase();
7417|   return d === 'egreso' ? CC_MANUAL_TIPO_CAJA_NOMBRE_EGRESO : CC_MANUAL_TIPO_CAJA_NOMBRE_INGRESO;
7418| }
7419|
7420| /** Resuelve id en tipos movimiento caja para CC manual (efectivo); null si no existe en catálogo. */
7421| function ccManualTipoMovimientoCajaIdPorDireccion(dir) {
7422|   const d = String(dir || '').toLowerCase();
7423|   const nombreEsperado = ccManualNombreTipoCajaFijoPorDireccion(d);
7424|   const row = (tiposMovimientoCaja || []).find((t) => t.activo !== false
7425|     && String(t.direccion || '').toLowerCase() === d
7426|     && String(t.nombre || '').trim() === nombreEsperado);
7427|   return row ? row.id : null;
7428| }
7429|
7430| function ccManualActualizarTextoCajaTipoFijo() {
7431|   const p = document.getElementById('cc-manual-caja-tipo-fijo-info');
7432|   if (!p) return;
7433|   const pagRol = ccManualLeerRolRadio('cc-manual-pagador-rol');
7434|   const cobRol = ccManualLeerRolRadio('cc-manual-cobrador-rol');
7435|   const dir = ccManualDireccionCajaDesdeFlujo(pagRol, cobRol);
7436|   if (!dir) {
7437|     p.textContent = '';
7438|     return;
7439|   }
7440|   const nombre = ccManualNombreTipoCajaFijoPorDireccion(dir);
7441|   p.innerHTML = 'En caja (efectivo) se registrará como tipo <strong>' + escapeHtml(nombre) + '</strong>.';
7442| }
7443|
7444| function ccManualSyncWrapCaja() {
7445|   const wrap = document.getElementById('cc-manual-wrap-caja');
7446|   if (!wrap) return;
7447|   const pagRol = ccManualLeerRolRadio('cc-manual-pagador-rol');
7448|   const cobRol = ccManualLeerRolRadio('cc-manual-cobrador-rol');
7449|   const modoEl = document.getElementById('cc-manual-modo-pago');
7450|   const modo = modoEl ? modoEl.value : 'efectivo';
7451|   const on = ccManualParticipaEmpresa(pagRol, cobRol) && modo === 'efectivo';
7452|   wrap.style.display = on ? 'block' : 'none';
7453|   if (on) ccManualActualizarTextoCajaTipoFijo();
7454| }
7455|
7456| function actualizarCcManualResumenDinamico() {
7457|   const el = document.getElementById('cc-manual-resumen-dinamico');
7458|   if (!el) return;
7459|   ccManualSyncVisibilidadRoles();
7460|   const marca = etiquetaRolParticipanteUi('pandy');
7461|   const pagRol = ccManualLeerRolRadio('cc-manual-pagador-rol');
7462|   const cobRol = ccManualLeerRolRadio('cc-manual-cobrador-rol');
7463|   const pagCli = (document.getElementById('cc-manual-pagador-cliente') || {}).value || '';
7464|   const pagInt = (document.getElementById('cc-manual-pagador-intermediario') || {}).value || '';
7465|   const cobCli = (document.getElementById('cc-manual-cobrador-cliente') || {}).value || '';
7466|   const cobInt = (document.getElementById('cc-manual-cobrador-intermediario') || {}).value || '';
7467|   const tmpCli = {};
7468|   const tmpInt = {};
7469|   ['cc-manual-pagador-cliente', 'cc-manual-cobrador-cliente'].forEach((id) => {
7470|     const s = document.getElementById(id);
7471|     if (s && s.value && s.options[s.selectedIndex]) tmpCli[s.value] = { nombre: s.options[s.selectedIndex].text };
7472|   });
7473|   ['cc-manual-pagador-intermediario', 'cc-manual-cobrador-intermediario'].forEach((id) => {
7474|     const s = document.getElementById(id);
7475|     if (s && s.value && s.options[s.selectedIndex]) tmpInt[s.value] = { nombre: s.options[s.selectedIndex].text };
7476|   });
7477|   const nomCliPorIdRes = {};
7478|   Object.keys(tmpCli).forEach((k) => { nomCliPorIdRes[k] = (tmpCli[k] && tmpCli[k].nombre) || ''; });
7479|   const nomIntPorIdRes = {};
7480|   Object.keys(tmpInt).forEach((k) => { nomIntPorIdRes[k] = (tmpInt[k] && tmpInt[k].nombre) || ''; });
7481|   const nomPag = ccManualNombreDisplayRol(pagRol, pagCli, pagInt, tmpCli, tmpInt);
7482|   const nomCob = ccManualNombreDisplayRol(cobRol, cobCli, cobInt, tmpCli, tmpInt);
7483|   const moneda = (document.getElementById('cc-manual-monedas') && document.getElementById('cc-manual-monedas').value) || 'USD';
7484|   const montoAbs = parseImporteInput((document.getElementById('cc-manual-monto') && document.getElementById('cc-manual-monto').value) || '');
7485|   const modoPago = (document.getElementById('cc-manual-modo-pago') && document.getElementById('cc-manual-modo-pago').value) || 'efectivo';
7486|   const legsRes = ccManualConstruirLegs(pagRol, cobRol, pagCli, pagInt, cobCli, cobInt);
7487|   ccManualSyncWrapCaja();
7488|
7489|   let html = '<strong>Resumen</strong><br>';
7490|   html += 'Flujo: <strong>' + nomPag + '</strong> paga → <strong>' + nomCob + '</strong> cobra. Monto referencia: <strong>' + escapeHtml(moneda) + '</strong>';
7491|   if (montoAbs > 0) html += ' <strong>' + formatImporteDisplay(montoAbs) + '</strong>';
7492|   html += '. Modo: <strong>' + escapeHtml(modoPago === 'banco' ? 'Banco/transferencia' : (modoPago === 'cheque' ? 'Cheque' : 'Efectivo')) + '</strong>.<br>';
7493|
7494|   if (legsRes.error) {
7495|     html += '<span style="color:#b45309;">' + escapeHtml(legsRes.error) + '</span><br>';
7496|   } else if (legsRes.legs && montoAbs > 0) {
7497|     html += 'En CC se registrará:<br><ul style="margin:0.35rem 0 0.5rem 1.1rem;">';
7498|     legsRes.legs.forEach((leg) => {
7499|       const ent = leg.kind === 'cliente' ? 'Cliente' : 'Intermediario';
7500|       const signed = montoCuentaCorrienteManualSigno(leg, montoAbs, nomCliPorIdRes, nomIntPorIdRes);
7501|       html += '<li>' + ent + ': <strong>' + formatImporteDisplay(signed) + ' ' + escapeHtml(moneda) + '</strong> (' + escapeHtml(leg.tip === 'cobro_entidad_pandy' ? 'entrega d
el tercero hacia la empresa (CC)' : 'entrega de la empresa hacia el tercero (CC)') + ')</li>';
7502|     });
7503|     html += '</ul>';
7504|   } else if (legsRes.legs) {
7505|     html += 'Completá importe para ver el desglose con signo por cuenta.<br>';
7506|   }
7507|

```

```

7508| const emp = ccManualParticipaEmpresa(pagRol, cobRol);
7509| if (emp && modoPago === 'efectivo') {
7510|   if (!userPermissions.includes('abm_movimientos_caja')) {
7511|     html += '<span style="color:#b45309;">Caja (efectivo): falta permiso <em>abm_movimientos_caja</em> para guardar.</span>';
7512|   } else {
7513|     const dir = ccManualDireccionCajaDesdeFlujo(pagRol, cobRol);
7514|     const nombreTipo = dir ? ccManualNombreTipoCajaFijoPorDireccion(dir) : '-';
7515|     html += '<br/>Caja: <strong>efectivo</strong>, <strong>' + escapeHtml(nombreTipo) + '</strong>, <strong>' + formatImporteDisplay(montoAbs) + ' ' + escapeHtml(moneda) + '
</strong>.';
7516|   }
7517| } else if (emp && modoPago !== 'efectivo') {
7518|   html += '<br/>Caja: <strong>no</strong> (solo se registra efectivo cuando participa la empresa). El modo elegido queda en concepto.';
7519| } else {
7520|   html += '<br/>Caja: <strong>no</strong> (sin movimiento de efectivo de la empresa en este registro).';
7521| }
7522| el.innerHTML = html;
7523| }
7524|
7525| function closeModalCcMovimientoManual() {
7526|   const backdrop = document.getElementById('modal-cc-movimiento-manual-backdrop');
7527|   if (backdrop) backdrop.classList.remove('activo');
7528| }
7529|
7530| function openModalCcMovimientoManual() {
7531|   if (!puedeRegistrarMovCcManual()) {
7532|     showToast('No tenés permiso para registrar movimientos manuales de cuenta corriente.', 'error');
7533|     return;
7534|   }
7535|   const backdrop = document.getElementById('modal-cc-movimiento-manual-backdrop');
7536|   if (!backdrop) return;
7537|
7538|   const rp = document.querySelector('input[name="cc-manual-pagador-rol"][value="cliente"]');
7539|   const rpe = document.querySelector('input[name="cc-manual-cobrador-rol"][value="pandy"]');
7540|   if (rp) rp.checked = true;
7541|   if (rpe) rpe.checked = true;
7542|   ccManualSyncVisibilidadRoles();
7543|
7544|   const fechaEl = document.getElementById('cc-manual-fecha');
7545|   if (fechaEl) fechaEl.value = fechaHoyYYYYMMDDArgentina();
7546|   const montoEl = document.getElementById('cc-manual-monto');
7547|   if (montoEl) montoEl.value = '';
7548|   const conceptoEl = document.getElementById('cc-manual-concepto');
7549|   if (conceptoEl) conceptoEl.value = '';
7550|   const monedaEl = document.getElementById('cc-manual-moneda');
7551|   if (monedaEl) monedaEl.value = 'USD';
7552|   const modoEl = document.getElementById('cc-manual-modo-pago');
7553|   if (modoEl) modoEl.value = 'efectivo';
7554|
7555|   Promise.all([
7556|     client.from('clientes').select('id, nombre').order('nombre', { ascending: true }),
7557|     client.from('intermediarios').select('id, nombre').order('nombre', { ascending: true })
7558|   ]).then(([rCli, rInt]) => {
7559|     if (rCli.error || rInt.error) {
7560|       showToast('Error al cargar clientes/intermediarios.', 'error');
7561|       return;
7562|     }
7563|     ccManualPoblarTodosLosSelects(rCli.data || [], rInt.data || []);
7564|     ccManualSyncWrapCaja();
7565|     actualizarCcManualResumenDinamico();
7566|     backdrop.classList.add('activo');
7567|     if (montoEl) setupInputImporte(montoEl);
7568|   });
7569| }
7570|
7571| function ccManualRollbackCcIds(idsCli, idsInt) {
7572|   const ps = [];
7573|   if (idsCli && idsCli.length) ps.push(client.from('movimientos_cuenta_corriente').delete().in('id', idsCli));
7574|   if (idsInt && idsInt.length) ps.push(client.from('movimientos_cuenta_corriente_intermediario').delete().in('id', idsInt));
7575|   return ps.length ? Promise.all(ps) : Promise.resolve();
7576| }
7577|
7578| function saveCcMovimientoManual() {
7579|   if (!currentUserId) {
7580|     showToast('Sesión no válida.', 'error');
7581|     return;
7582|   }
7583|   if (!puedeRegistrarMovCcManual()) {
7584|     showToast('Sin permiso para registrar este movimiento.', 'error');
7585|     return;
7586|   }
7587|
7588|   const pagRol = ccManualLeerRolRadio('cc-manual-pagador-rol');
7589|   const cobRol = ccManualLeerRolRadio('cc-manual-cobrador-rol');
7590|   const pagCli = (document.getElementById('cc-manual-pagador-cliente') || {}).value || '';
7591|   const pagInt = (document.getElementById('cc-manual-pagador-intermediario') || {}).value || '';
7592|   const cobCli = (document.getElementById('cc-manual-cobrador-cliente') || {}).value || '';
7593|   const cobInt = (document.getElementById('cc-manual-cobrador-intermediario') || {}).value || '';
7594|
7595|   const legsRes = ccManualConstruirLegs(pagRol, cobRol, pagCli, pagInt, cobCli, cobInt);
7596|   if (legsRes.error) {
7597|     showToast(legsRes.error, 'error');
7598|     return;
7599|   }
7600|   const legs = legsRes.legs;
7601|
7602|   const moneda = ((document.getElementById('cc-manual-moneda') || {}).value || 'USD').toUpperCase();
7603|   const fecha = (document.getElementById('cc-manual-fecha') || {}).value;

```



```

7604 | if (!fecha) {
7605 |     showToast('Indicá la fecha.', 'error');
7606 |     return;
7607 | }
7608 | const montoAbs = parseImporteInput((document.getElementById('cc-manual-monto') || {}).value || '');
7609 | if (isNaN(montoAbs) || montoAbs <= 0) {
7610 |     showToast('El importe debe ser un número mayor que cero.', 'error');
7611 |     return;
7612 | }
7613 | const modoPago = ((document.getElementById('cc-manual-modo-pago') || {}).value || 'efectivo').toLowerCase();
7614 | const conceptoUsuario = ((document.getElementById('cc-manual-concepto') || {}).value || '').trim();
7615 | const sufixoModo = modoPago === 'efectivo' ? '' : (modoPago === 'banco' ? '[Banco]' : '[Cheque]');
7616 | const conceptoBase = conceptoUsuario || ('CC manual sin orden ' + ccManualTextoLugarRol(pagRol, 'pag') + ' ' + ccManualTextoLugarRol(cobRol, 'cob'));
7617 | const conceptoCc = conceptoBase + sufixoModo;
7618 |
7619 | const grupoId = legs.length > 1 ? ccManualNuevoGrupoId() : null;
7620 | const metaManual = {
7621 |     manual_grupo_id: grupoId,
7622 |     manual_pagador_rol: pagRol,
7623 |     manual_cobrador_rol: cobRol,
7624 |     manual_pagador_cliente_id: pagRol === 'cliente' ? pagCli : null,
7625 |     manual_pagador_intermediario_id: pagRol === 'intermediario' ? pagInt : null,
7626 |     manual_cobrador_cliente_id: cobRol === 'cliente' ? cobCli : null,
7627 |     manual_cobrador_intermediario_id: cobRol === 'intermediario' ? cobInt : null,
7628 | };
7629 |
7630 | const ahora = new Date().toISOString();
7631 | const participaEmp = ccManualParticipaEmpresa(pagRol, cobRol);
7632 | const requiereCaja = participaEmp && modoPago === 'efectivo';
7633 | if (requiereCaja) {
7634 |     if (!userPermissions.includes('abm_movimientos_caja')) {
7635 |         showToast('Para registrar efectivo en caja necesitas permiso abm_movimientos_caja.', 'error');
7636 |         return;
7637 |     }
7638 | }
7639 |
7640 | const idsCliLeg = [...new Set(legs.filter((l) => l.kind === 'cliente' && l.cliente_id).map((l) => l.cliente_id))];
7641 | const idsIntLeg = [...new Set(legs.filter((l) => l.kind === 'intermediario' && l.intermediario_id).map((l) => l.intermediario_id))];
7642 |
7643 | function ejecutarGuardadoCcManual(tipoMovCajaIdFinal, nomCliPorId, nomIntPorId) {
7644 |     const idsCli = [];
7645 |     const idsInt = [];
7646 |
7647 |     function insertNextLeg(idx) {
7648 |         if (idx >= legs.length) {
7649 |             if (!requiereCaja) {
7650 |                 closeModalCcMovimientoManual();
7651 |                 showToast(legs.length > 1 ? 'Movimientos registrados en cuenta corriente (enlace por grupo).': 'Movimiento registrado en cuenta corriente.', 'success');
7652 |                 loadCuentaCorriente();
7653 |                 return;
7654 |             }
7655 |             const tipoRow = (tiposMovimientoCaja || []).find((t) => String(t.id) === String(tipoMovCajaIdFinal));
7656 |             const signoCaja = tipoRow && String(tipoRow.direccion || '').toLowerCase() === 'egreso' ? -1 : 1;
7657 |             const montoCaja = montoAbs * signoCaja;
7658 |             const conceptoCaja = conceptoUsuario || ('CC manual efectivo ' + conceptoCc);
7659 |             return client.from('movimientos_caja').insert({
7660 |                 moneda,
7661 |                 monto: montoCaja,
7662 |                 tipo_movimiento_id: tipoMovCajaIdFinal,
7663 |                 orden_id: null,
7664 |                 transaccion_id: null,
7665 |                 caja_tipo: 'efectivo',
7666 |                 concepto: conceptoCaja,
7667 |                 fecha,
7668 |                 usuario_id: currentUserId,
7669 |                 estado: 'cerrado',
7670 |                 estado_fecha: ahora,
7671 |             }).select('id').single().then((rCaja) => {
7672 |                 if (rCaja.error) {
7673 |                     return ccManualRollbackCcIds(idsCli, idsInt).then(() => {
7674 |                         showToast('No se pudo registrar caja; se revirtieron los movimientos de CC. ' + (rCaja.error.message || ''), 'error');
7675 |                     });
7676 |                 }
7677 |                 const cajaNuevoId = rCaja.data && rCaja.data.id;
7678 |                 const enlazarCaja = () => {
7679 |                     if (!cajaNuevoId) {
7680 |                         closeModalCcMovimientoManual();
7681 |                         showToast('Movimientos registrados en CC y en caja (efectivo).', 'success');
7682 |                         loadCuentaCorriente();
7683 |                         if (typeof loadCajas === 'function') loadCajas();
7684 |                         return;
7685 |                     }
7686 |                     const ps = [];
7687 |                     idsCli.forEach((cid) => {
7688 |                         ps.push(client.from('movimientos_cuenta_corriente').update({ movimiento_caja_id: cajaNuevoId }).eq('id', cid));
7689 |                     });
7690 |                     idsInt.forEach((iid) => {
7691 |                         ps.push(client.from('movimientos_cuenta_corriente_intermediario').update({ movimiento_caja_id: cajaNuevoId }).eq('id', iid));
7692 |                     });
7693 |                     return Promise.all(ps).then((updRes) => {
7694 |                         const bad = (updRes || []).find((r) => r && r.error);
7695 |                         if (bad && bad.error) {
7696 |                             showToast('Caja registrada; no se pudo enlazar el id en CC: ' + (bad.error.message || 'ejecutó la migración movimiento_caja_id.'), 'error');
7697 |                         }
7698 |                         closeModalCcMovimientoManual();
7699 |                         showToast('Movimientos registrados en CC y en caja (efectivo).', 'success');
7700 |                         loadCuentaCorriente();

```

```

7701         if (typeof loadCajas === 'function') loadCajas();
7702     });
7703     };
7704     return enlazarCaja();
7705     });
7706 }
7707 const leg = legs[idx];
7708 const signedCc = montoCuentaCorrienteManualSigno(leg, montoAbs, nomCliPorId, nomIntPorId);
7709 const montos = montosCcPorMoneda(moneda, signedCc);
7710 const payloadCc = {
7711     orden_id: null,
7712     transaccion_id: null,
7713     transaccion_numero: null,
7714     concepto: conceptoCc,
7715     fecha,
7716     usuario_id: currentUserId,
7717     estado: 'cerrado',
7718     estado_fecha: ahora,
7719     incluir_en_detalle: true,
7720     es_movimiento_manual: true,
7721     manual_tip_movimiento: leg.tip,
7722     moneda,
7723     monto: signedCc,
7724     ...montos,
7725     ...metaManual,
7726 };
7727 const q = leg.kind === 'cliente'
7728   ? client.from('movimientos_cuenta_corriente').insert({ ...payloadCc, cliente_id: leg.cliente_id }).select('id').single()
7729   : client.from('movimientos_cuenta_corriente_intermediario').insert({ ...payloadCc, intermediario_id: leg.intermediario_id }).select('id').single();
7730 return q.then((r) => {
7731     if (r.error) {
7732         return ccManualRollbackCcIds(idsCli, idsInt).then(() => {
7733             showToast('Error en cuenta corriente: ' + mensajeErrorCcInsertSupabase(r.error.message || ''), 'error');
7734         });
7735     }
7736     const id = r.data && r.data.id;
7737     if (leg.kind === 'cliente') idsCli.push(id);
7738     else idsInt.push(id);
7739     return insertNextLeg(idx + 1);
7740 });
7741 }
7742 insertNextLeg(0);
7743 }
7744 }
7745 function iniciarGuardadoConNombres(nomCliPorId, nomIntPorId) {
7746     if (!requiereCaja) {
7747         ejecutarGuardadoCcManual(null, nomCliPorId, nomIntPorId);
7748         return;
7749     }
7750     const dirCaja = ccManualDireccionCajaDesdeFlujo(pagRol, cobRol);
7751     function cargarTipoYGuardarCcManual() {
7752         loadTiposMovimientoCaja().then(() => {
7753             const tipoId = ccManualTipoMovimientoCajaIdPorDireccion(dirCaja);
7754             if (!tipoId) {
7755                 const nom = ccManualNombreTipoCajaFijoPorDireccion(dirCaja);
7756                 showToast('No existe el tipo de caja «' + nom + '». Ejecutá sql/migracion_tipos_caja_cc_manual.sql en Supabase o creá ese tipo en Cajas → Tipos de movimiento.', 'err
7757 or');
7758                 return;
7759             }
7760             ejecutarGuardadoCcManual(tipoId, nomCliPorId, nomIntPorId);
7761         });
7762     }
7763     if (dirCaja === 'egreso') {
7764         fetchMovimientosCajaCerradosSinSync().then((list) => {
7765             const v = validarSaldoCajaParaEgresoLista(list, { cajaTipo: 'efectivo', moneda, montoAbs });
7766             if (!v.ok) {
7767                 showToast(v.mensaje, 'error');
7768                 return;
7769             }
7770             cargarTipoYGuardarCcManual();
7771         });
7772         return;
7773     }
7774     cargarTipoYGuardarCcManual();
7775 }
7776 const promCliNombres = idsCliLeg.length
7777   ? client.from('clientes').select('id, nombre').in('id', idsCliLeg)
7778   : Promise.resolve({ data: [], error: null });
7779 const promIntNombres = idsIntLeg.length
7780   ? client.from('intermediarios').select('id, nombre').in('id', idsIntLeg)
7781   : Promise.resolve({ data: [], error: null });
7782 Promise.all([promCliNombres, promIntNombres]).then(([rCli, rInt]) => {
7783     if (rCli.error || rInt.error) {
7784         showToast('Error al resolver nombres para el signo en cuenta corriente.', 'error');
7785         return;
7786     }
7787     const nomCliPorId = {};
7788     (rCli.data || []).forEach((r) => { nomCliPorId[String(r.id)] = r.nombre; });
7789     const nomIntPorId = {};
7790     (rInt.data || []).forEach((r) => { nomIntPorId[String(r.id)] = r.nombre; });
7791     iniciarGuardadoConNombres(nomCliPorId, nomIntPorId);
7792 });
7793 }
7794 }
7795 }
7796

```

```

7797 function setupModalCcMovimientoManual() {
7798   const backdrop = document.getElementById('modal-cc-movimiento-manual-backdrop');
7799   if (!backdrop || backdrop.dataset.ccManualBound === '1') return;
7800   backdrop.dataset.ccManualBound = '1';
7801
7802   const btnClose = document.getElementById('modal-cc-movimiento-manual-close');
7803   const btnCancel = document.getElementById('modal-cc-movimiento-manual-cancelar');
7804   const form = document.getElementById('form-cc-movimiento-manual');
7805   if (btnClose) btnClose.addEventListener('click', closeModalCcMovimientoManual);
7806   if (btnCancel) btnCancel.addEventListener('click', closeModalCcMovimientoManual);
7807   setupBackdropCloseOnlyOnRealClick(backdrop, closeModalCcMovimientoManual);
7808   if (form) form.addEventListener('submit', (e) => { e.preventDefault(); saveCcMovimientoManual(); });
7809
7810   document.querySelectorAll('input[name="cc-manual-pagador-rol"]').forEach((inp) => {
7811     inp.addEventListener('change', () => { ccManualSyncWrapCaja(); actualizarCcManualResumenDinamico(); });
7812   });
7813   document.querySelectorAll('input[name="cc-manual-cobrador-rol"]').forEach((inp) => {
7814     inp.addEventListener('change', () => { ccManualSyncWrapCaja(); actualizarCcManualResumenDinamico(); });
7815   });
7816
7817   ['cc-manual-pagador-cliente', 'cc-manual-pagador-intermediario', 'cc-manual-cobrador-cliente', 'cc-manual-cobrador-intermediario', 'cc-manual-moneda', 'cc-manual-modo-pago']
7818   .forEach((id) => {
7819     const el = document.getElementById(id);
7820     if (el) el.addEventListener('change', () => actualizarCcManualResumenDinamico());
7821   });
7822   const montoEl = document.getElementById('cc-manual-monto');
7823   if (montoEl) montoEl.addEventListener('input', () => actualizarCcManualResumenDinamico());
7824
7825   document.addEventListener('keydown', (e) => {
7826     if (e.key !== 'Escape') return;
7827     if (!backdrop.classList.contains('activo')) return;
7828     closeModalCcMovimientoManual();
7829   });
7830 }
7831 /**
7832  * Cierra el modal solo cuando el usuario hizo clic realmente en el backdrop (mousedown + click en backdrop).
7833  * Evita que se cierre al elegir una opción de un <select> u otro control nativo, donde el click
7834  * a veces se reporta en el backdrop.
7835  */
7836 function setupBackdropCloseOnlyOnRealClick(backdrop, onClose) {
7837   if (!backdrop || typeof onClose !== 'function') return;
7838   let mousedownOnBackdrop = false;
7839   backdrop.addEventListener('mousedown', function _md(e) {
7840     mousedownOnBackdrop = (e.target === backdrop);
7841   }, true);
7842   backdrop.addEventListener('click', function _cl(e) {
7843     if (e.target === backdrop && mousedownOnBackdrop) onClose();
7844     mousedownOnBackdrop = false;
7845   }, true);
7846 }
7847
7848 /** Modales: arrastrar por el header para mover; al cerrar se resetea la posición. */
7849 function setupModalesDraggable() {
7850   let dragState = null;
7851
7852   document.querySelectorAll('.modal').forEach((modal) => {
7853     const header = modal.querySelector('.modal-header');
7854     if (!header) return;
7855     header.addEventListener('mousedown', (e) => {
7856       if (e.target.closest('.modal-close')) return;
7857       e.preventDefault();
7858       const rect = modal.getBoundingClientRect();
7859       modal.style.position = 'fixed';
7860       modal.style.left = rect.left + 'px';
7861       modal.style.top = rect.top + 'px';
7862       modal.style.margin = '0';
7863       dragState = { modal, startX: e.clientX, startY: e.clientY, startLeft: rect.left, startTop: rect.top };
7864     });
7865   });
7866
7867   document.addEventListener('mousemove', (e) => {
7868     if (!dragState) return;
7869     const dx = e.clientX - dragState.startX;
7870     const dy = e.clientY - dragState.startY;
7871     dragState.modal.style.left = (dragState.startLeft + dx) + 'px';
7872     dragState.modal.style.top = (dragState.startTop + dy) + 'px';
7873   });
7874
7875   document.addEventListener('mouseup', () => { dragState = null; });
7876
7877   document.querySelectorAll('.modal-backdrop').forEach((backdrop) => {
7878     const modal = backdrop.querySelector('.modal');
7879     if (!modal) return;
7880     const observer = new MutationObserver(() => {
7881       if (!backdrop.classList.contains('activo')) {
7882         modal.style.position = '';
7883         modal.style.left = '';
7884         modal.style.top = '';
7885         modal.style.margin = '';
7886       }
7887     });
7888     observer.observe(backdrop, { attributes: true, attributeFilter: ['class'] });
7889   });
7890 }
7891
7892 function setupCuentaCorriente() {

```

```

7893| const backdropDetalle = document.getElementById('modal-cc-detalle-backdrop');
7894| const btnCloseDetalle = document.getElementById('modal-cc-detalle-close');
7895| if (btnCloseDetalle) btnCloseDetalle.addEventListener('click', closeModalCcDetalle);
7896| if (backdropDetalle) backdropDetalle.addEventListener('click', closeModalCcDetalle);
7897|
7898| const ccVistaToggleEl = document.getElementById('cc-vista-toggle');
7899| if (ccVistaToggleEl) {
7900|   ccVistaToggleEl.querySelectorAll('button[data-vista]').forEach((btn) => {
7901|     btn.addEventListener('click', () => {
7902|       const vista = btn.getAttribute('data-vista');
7903|       if (!vista || vista === ccVistaToggle) return;
7904|       ccVistaToggle = vista;
7905|       aplicarFiltroCcResumen();
7906|     });
7907|   });
7908| }
7909| const ccFiltroTipoEl = document.getElementById('cc-filtro-tipo');
7910| if (ccFiltroTipoEl) {
7911|   ccFiltroTipoEl.querySelectorAll('button').forEach((btn) => {
7912|     btn.addEventListener('click', () => {
7913|       const tipo = btn.getAttribute('data-tipo');
7914|       if (!tipo) return;
7915|       ccFiltroTipo = tipo;
7916|       ccFiltroTipoEl.querySelectorAll('button').forEach((b) => b.classList.remove('activo'));
7917|       btn.classList.add('activo');
7918|       ccDetalleFiltroEntidadId = '';
7919|       poblarSelectCcDetalleEntidad();
7920|       aplicarFiltroCcResumen();
7921|     });
7922|   });
7923| }
7924|
7925| const ccDetalleEntidadSel = document.getElementById('cc-detalle-entidad-select');
7926| if (ccDetalleEntidadSel) {
7927|   ccDetalleEntidadSel.addEventListener('change', () => {
7928|     ccDetalleFiltroEntidadId = ccDetalleEntidadSel.value || '';
7929|     aplicarFiltroCcResumen();
7930|   });
7931| }
7932|
7933| function onCcRefrescarClick() {
7934|   showToast('Sincronizando CC y caja desde órdenes.', 'info');
7935|   sincronizarCcCajaParaTodasLasOrdenesConInstrumentacion()
7936|     .then(() => {
7937|       loadCuentaCorriente({ skipSyncGlobal: true });
7938|       showToast('Cuenta corriente actualizada.', 'success');
7939|       setTimeout(() => {
7940|         const tbody = document.getElementById('cc-vista-detalle-tbody');
7941|         const sinMovimientos = tbody && tbody.querySelector('td[colspan]') && tbody.textContent.includes('No hay movimientos');
7942|         if (sinMovimientos) {
7943|           showToast('Si tenés órdenes con transacciones ejecutadas y no ves movimientos: verificá que existan filas en reglas_de_negocio para el tipo de operación (y usa_int
7944|           ermediario) de cada orden, o revisá permisos/sync. Volvó a Refrescar.', 'info', 12000);
7945|         }, 800);
7946|       });
7947|     })
7948|     .catch((e) => {
7949|       showToast(e && (e.message || String(e)) || 'Error al sincronizar', 'error');
7950|     });
7951| }
7952| const ccBtnRefrescar = document.getElementById('cc-btn-refrescar');
7953| const ccBtnRefrescarMov = document.getElementById('cc-btn-refrescar-movimientos');
7954| if (ccBtnRefrescar) ccBtnRefrescar.addEventListener('click', onCcRefrescarClick);
7955| if (ccBtnRefrescarMov) ccBtnRefrescarMov.addEventListener('click', onCcRefrescarClick);
7956|
7957| function onCcExportarClick() {
7958|   exportarCcResumenExcel();
7959| }
7960| const ccBtnExportar = document.getElementById('cc-btn-exportar-excel');
7961| const ccBtnExportarMov = document.getElementById('cc-btn-exportar-movimientos');
7962| if (ccBtnExportar) ccBtnExportar.addEventListener('click', onCcExportarClick);
7963| if (ccBtnExportarMov) ccBtnExportarMov.addEventListener('click', onCcExportarClick);
7964| const ccBtnMovManual = document.getElementById('cc-btn-movimiento-manual');
7965| const ccBtnMovManualMov = document.getElementById('cc-btn-movimiento-manual-mov');
7966| if (ccBtnMovManual) ccBtnMovManual.addEventListener('click', () => openModalCcMovimientoManual());
7967| if (ccBtnMovManualMov) ccBtnMovManualMov.addEventListener('click', () => openModalCcMovimientoManual());
7968| const ccDetalleDesdeEl = document.getElementById('cc-detalle-desde');
7969| const ccDetalleHastaEl = document.getElementById('cc-detalle-hasta');
7970| function aplicarRangoDetalle() {
7971|   ccDetalleDesde = (ccDetalleDesdeEl && ccDetalleDesdeEl.value) || '';
7972|   ccDetalleHasta = (ccDetalleHastaEl && ccDetalleHastaEl.value) || '';
7973|   if (ccDetalleDesde === '' && ccDetalleHasta === '') {
7974|     ccMovimientosMostrarTodoHistorial = true;
7975|   } else {
7976|     ccMovimientosMostrarTodoHistorial = false;
7977|   }
7978|   aplicarFiltroCcResumen();
7979| }
7980| if (ccDetalleDesdeEl) ccDetalleDesdeEl.addEventListener('change', aplicarRangoDetalle);
7981| if (ccDetalleHastaEl) ccDetalleHastaEl.addEventListener('change', aplicarRangoDetalle);
7982| const ccBtnDetalleHoy = document.getElementById('cc-detalle-btn-hoy');
7983| const ccBtnDetalleTodo = document.getElementById('cc-detalle-btn-todo-historial');
7984| if (ccBtnDetalleHoy) {
7985|   ccBtnDetalleHoy.addEventListener('click', () => {
7986|     ccMovimientosMostrarTodoHistorial = false;
7987|     const h = fechaHoyYYYYMMDDArgentina();
7988|     ccDetalleDesde = h;
7989|     ccDetalleHasta = h;

```

```

7989|         if (ccDetalleDesdeEl) ccDetalleDesdeEl.value = h;
7990|         if (ccDetalleHastaEl) ccDetalleHastaEl.value = h;
7991|         aplicarFiltroCcResumen();
7992|     });
7993| }
7994| if (ccBtnDetalleTodo) {
7995|     ccBtnDetalleTodo.addEventListener('click', () => {
7996|         ccMovimientosMostrarTodoHistorial = true;
7997|         ccDetalleDesde = '';
7998|         ccDetalleHasta = '';
7999|         if (ccDetalleDesdeEl) ccDetalleDesdeEl.value = '';
8000|         if (ccDetalleHastaEl) ccDetalleHastaEl.value = '';
8001|         aplicarFiltroCcResumen();
8002|     });
8003| }
8004|
8005| document.querySelectorAll('.link-inicio').forEach((a) => {
8006|     a.addEventListener('click', (e) => {
8007|         e.preventDefault();
8008|         const vistaId = a.getAttribute('data-vista');
8009|         const titles = { 'vista-inicio': 'Panel de Control', 'vista-ordenes': 'Órdenes', 'vista-cajas': 'Cajas', 'vista-clientes': 'Clientes', 'vista-cuenta-corriente': 'Cuenta
corriente' };
8010|         showView(vistaId, titles[vistaId] || vistaId);
8011|     });
8012| });
8013| }
8014|
8015| function loadTiposMovimientoCajaTable() {
8016|     const loadingEl = document.getElementById('tipos-mov-loading');
8017|     const wrapEl = document.getElementById('tipos-mov-tabla-wrap');
8018|     const tbody = document.getElementById('tipos-movimiento-tbody');
8019|     const btnNuevo = document.getElementById('btn-nuevo-tipo-movimiento');
8020|     if (!loadingEl || !wrapEl || !tbody) return;
8021|
8022|     const canAbm = userPermissions.includes('abm_tipos_movimiento_caja');
8023|     if (btnNuevo.style.display = canAbm ? '' : 'none';
8024|
8025|     loadingEl.style.display = 'block';
8026|     wrapEl.style.display = 'none';
8027|     tbody.innerHTML = '';
8028|
8029|     client
8030|         .from('tipos_movimiento_caja')
8031|         .select('id, nombre, direccion, activo, incluye_gp_operativo')
8032|         .order('nombre')
8033|         .then((res) => {
8034|             loadingEl.style.display = 'none';
8035|             if (res.error) {
8036|                 tbody.innerHTML = '<tr><td colspan="5">Error: ' + (res.error.message || '') + '</td></tr>';
8037|                 wrapEl.style.display = 'block';
8038|                 return;
8039|             }
8040|             const list = res.data || [];
8041|             function celdaToggleTipoMovCaja(checked, dataTipoId, claseInput, labelAccesible) {
8042|                 const dis = canAbm ? '' : 'disabled';
8043|                 const aria = escapeHtml(labelAccesible || '');
8044|                 return '<td class="tipo-mov-caja-toggle-cell"><span class="toggle-switch">
<input type="checkbox" class="${claseInput}" data-tipo-id="${dataTipoId}"${checked ? ' checked' : ''}${dis} title="${aria}" aria-label="${aria}" />
<span class="slider"></span>
</span></td>';
8047|             }
8048|             tbody.innerHTML = list
8049|                 .map((t) => {
8050|                     const nom = escapeHtml(t.nombre || 'tipo');
8051|                     const actOn = t.activo !== false;
8052|                     const gpOn = t.incluye_gp_operativo !== false;
8053|                     return '<tr data-id="${t.id}">
<td>${escapeHtml(t.nombre)}</td>
<td>${t.direccion === 'egreso' ? 'Egreso' : 'Ingreso'}</td>
${celdaToggleTipoMovCaja(actOn, t.id, 'tipo-mov-caja-toggle-activo', 'Activo: ${nom})}}
${celdaToggleTipoMovCaja(gpOn, t.id, 'tipo-mov-caja-toggle-gp', 'Incluye en G/P: ${nom})}}
<td>${canAbm ? '<button type="button" class="btn-editar" data-id="${t.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stro
ke-width="2"><path d="M11 4H4a2 2 0 0-2 2v14a2 2 0 02 2h14a2 2 0 02-2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013 1.14 9.5-9.5z"/></svg></span>Editar</button>' :
''}</td>
</tr>';
8061|                 });
8062|             .join('');
8063|             tbody.querySelectorAll('.tipo-mov-caja-toggle-activo').forEach((inp) => {
8064|                 inp.addEventListener('change', function () {
8065|                     if (!canAbm) return;
8066|                     const tid = this.getAttribute('data-tipo-id');
8067|                     const val = this.checked;
8068|                     client
8069|                         .from('tipos_movimiento_caja')
8070|                         .update({ activo: val })
8071|                         .eq('id', tid)
8072|                         .then((r) => {
8073|                             if (r.error) {
8074|                                 showToast('Error: ' + (r.error.message || 'No se pudo guardar.'), 'error');
8075|                                 this.checked = !val;
8076|                                 return;
8077|                             }
8078|                             showToast(val ? 'Tipo activo.' : 'Tipo inactivo.', 'success');
8079|                             loadTiposMovimientoCaja().then(() => {});
8080|                         });
8081|                 });
8082|             });

```

```

8083|     tbody.querySelectorAll('.tipo-mov-caja-toggle-gp').forEach((inp) => {
8084|         inp.addEventListener('change', function () {
8085|             if (!canAbm) return;
8086|             const tid = this.getAttribute('data-tipo-id');
8087|             const val = this.checked;
8088|             client
8089|                 .from('tipos_movimiento_caja')
8090|                 .update({ incluye_gp_operativo: val })
8091|                 .eq('id', tid)
8092|                 .then((r) => {
8093|                     if (r.error) {
8094|                         showToast('Error: ' + (r.error.message || 'No se pudo guardar.'), 'error');
8095|                         this.checked = !val;
8096|                         return;
8097|                     }
8098|                     showToast(val ? 'G/P activado para este tipo.' : 'G/P desactivado para este tipo.', 'success');
8099|                 });
8100|         });
8101|     });
8102|     tbody.querySelectorAll('.btn-editar').forEach((btn) => {
8103|         btn.addEventListener('click', () => {
8104|             const id = btn.getAttribute('data-id');
8105|             const row = list.find((r) => r.id === id);
8106|             if (row) openModalTipoMovimientoCaja(row);
8107|         });
8108|     });
8109|     if (list.length === 0) tbody.innerHTML = '<tr><td colspan="5">No hay tipos cargados. Agregá uno para usar en movimientos manuales.</td></tr>';
8110|     wrapEl.style.display = 'block';
8111| });
8112| }
8113|
8114| function openModalTipoMovimientoCaja(registro) {
8115|     const backdrop = document.getElementById('modal-tipo-movimiento-backdrop');
8116|     const titulo = document.getElementById('modal-tipo-movimiento-titulo');
8117|     const idEl = document.getElementById('tipo-movimiento-id');
8118|     const form = document.getElementById('form-tipo-movimiento');
8119|     if (!backdrop || !titulo || !idEl || !form) return;
8120|
8121|     if (registro) {
8122|         titulo.textContent = 'Editar tipo de movimiento';
8123|         idEl.value = registro.id;
8124|         document.getElementById('tipo-movimiento-nombre').value = registro.nombre || '';
8125|         document.getElementById('tipo-movimiento-direccion').value = registro.direccion === 'egreso' ? 'egreso' : 'ingreso';
8126|         document.getElementById('tipo-movimiento-activo').checked = registro.activo !== false;
8127|         const gpChk = document.getElementById('tipo-movimiento-incluye-gp');
8128|         if (gpChk) gpChk.checked = registro.incluye_gp_operativo !== false;
8129|     } else {
8130|         titulo.textContent = 'Nuevo tipo de movimiento';
8131|         idEl.value = '';
8132|         form.reset();
8133|         document.getElementById('tipo-movimiento-direccion').value = 'ingreso';
8134|         document.getElementById('tipo-movimiento-activo').checked = true;
8135|         const gpChkN = document.getElementById('tipo-movimiento-incluye-gp');
8136|         if (gpChkN) gpChkN.checked = true;
8137|     }
8138|     backdrop.classList.add('activo');
8139| }
8140|
8141| function closeModalTipoMovimientoCaja() {
8142|     const backdrop = document.getElementById('modal-tipo-movimiento-backdrop');
8143|     if (backdrop) backdrop.classList.remove('activo');
8144| }
8145|
8146| function saveTipoMovimientoCaja() {
8147|     const idEl = document.getElementById('tipo-movimiento-id');
8148|     const id = idEl && idEl.value ? idEl.value.trim() : '';
8149|     const nombre = document.getElementById('tipo-movimiento-nombre').value.trim();
8150|     if (!nombre) {
8151|         showToast('El nombre es obligatorio.', 'error');
8152|         return;
8153|     }
8154|     const direccion = document.getElementById('tipo-movimiento-direccion').value;
8155|     const activo = document.getElementById('tipo-movimiento-activo').checked;
8156|     const gpEl = document.getElementById('tipo-movimiento-incluye-gp');
8157|     const incluye_gp_operativo = gpEl ? gpEl.checked : true;
8158|     const payload = { nombre, direccion: direccion || 'ingreso', activo, incluye_gp_operativo };
8159|     const prom = id
8160|         ? client.from('tipos_movimiento_caja').update(payload).eq('id', id)
8161|         : client.from('tipos_movimiento_caja').insert(payload);
8162|     prom.then((res) => {
8163|         if (res.error) {
8164|             showToast('Error: ' + (res.error.message || 'No se pudo guardar.'), 'error');
8165|             return;
8166|         }
8167|         closeModalTipoMovimientoCaja();
8168|         loadTiposMovimientoCajaTable();
8169|         loadTiposMovimientoCaja().then(() => {}); // refresh dropdown en modal movimiento
8170|     });
8171| }
8172|
8173| function setupCajasToggle() {
8174|     const toggleMoneda = document.getElementById('cajas-toggle-monedas');
8175|     if (!toggleMoneda) return;
8176|     toggleMoneda.querySelectorAll('button').forEach((btn) => {
8177|         btn.addEventListener('click', () => {
8178|             const cajasMonedaActual = btn.getAttribute('data-monedas');
8179|             toggleMoneda.querySelectorAll('button').forEach((b) => b.classList.remove('activo'));

```

```

8180|         btn.classList.add('activo');
8181|         loadCajas({ soloFiltros: true });
8182|     });
8183| });
8184| }
8185|
8186| /** Muestra el panel Movimientos o Tipos según cajasVistaSolapa (alineado a solapas CC). */
8187| function syncCajasPaneles() {
8188|     const panelMov = document.getElementById('cajas-panel-movimientos');
8189|     const panelTipos = document.getElementById('cajas-panel-tipos');
8190|     const tablist = document.getElementById('cajas-vista-tabs');
8191|     if (!panelMov || !panelTipos || !tablist) return;
8192|     const esMov = cajasVistaSolapa === 'movimientos';
8193|     panelMov.style.display = esMov ? '' : 'none';
8194|     panelTipos.style.display = esMov ? 'none' : '';
8195|     tablist.querySelectorAll('button[data-cajas-vista]').forEach((b) => {
8196|         const v = b.getAttribute('data-cajas-vista');
8197|         const on = (v === 'movimientos' && esMov) || (v === 'tipos' && !esMov);
8198|         b.classList.toggle('activo', on);
8199|         b.setAttribute('aria-selected', on ? 'true' : 'false');
8200|     });
8201| }
8202|
8203| function setupCajasMovFiltrosYTabs() {
8204|     if (cajasMovFiltrosListenersAttached) return;
8205|     const dEl = document.getElementById('cajas-mov-desde');
8206|     const hEl = document.getElementById('cajas-mov-hasta');
8207|     function aplicarRangoCajasMov() {
8208|         cajasMovFechaDesde = (dEl && dEl.value) || '';
8209|         cajasMovFechaHasta = (hEl && hEl.value) || '';
8210|         if (cajasMovFechaDesde === '' && cajasMovFechaHasta === '') {
8211|             cajasMovMostrarTodoHistorial = true;
8212|         } else {
8213|             cajasMovMostrarTodoHistorial = false;
8214|         }
8215|         loadCajas({ soloFiltros: true });
8216|     }
8217|     if (dEl) dEl.addEventListener('change', aplicarRangoCajasMov);
8218|     if (hEl) hEl.addEventListener('change', aplicarRangoCajasMov);
8219|     const btnHoy = document.getElementById('cajas-mov-btn-hoy');
8220|     const btnTodo = document.getElementById('cajas-mov-btn-todo');
8221|     if (btnHoy) {
8222|         btnHoy.addEventListener('click', () => {
8223|             cajasMovMostrarTodoHistorial = false;
8224|             const h = fechaHoyYYYYMMDDArgentina();
8225|             cajasMovFechaDesde = h;
8226|             cajasMovFechaHasta = h;
8227|             syncCajasMovFechaInputs();
8228|             loadCajas({ soloFiltros: true });
8229|         });
8230|     }
8231|     if (btnTodo) {
8232|         btnTodo.addEventListener('click', () => {
8233|             cajasMovMostrarTodoHistorial = true;
8234|             cajasMovFechaDesde = '';
8235|             cajasMovFechaHasta = '';
8236|             syncCajasMovFechaInputs();
8237|             loadCajas({ soloFiltros: true });
8238|         });
8239|     }
8240|     document.querySelectorAll('[data-caja-tab]').forEach((btn) => {
8241|         btn.addEventListener('click', () => {
8242|             const v = (btn.getAttribute('data-caja-tab') || 'todo').toLowerCase();
8243|             cajasMovCajaTipoTab = v === 'efectivo' || v === 'banco' || v === 'cheque' ? v : 'todo';
8244|             document.querySelectorAll('[data-caja-tab]').forEach((b) => {
8245|                 b.classList.remove('activo');
8246|                 b.setAttribute('aria-pressed', 'false');
8247|             });
8248|             btn.classList.add('activo');
8249|             btn.setAttribute('aria-pressed', 'true');
8250|             loadCajas({ soloFiltros: true });
8251|         });
8252|     });
8253|     const cajasVistaTabsEl = document.getElementById('cajas-vista-tabs');
8254|     if (cajasVistaTabsEl) {
8255|         cajasVistaTabsEl.querySelectorAll('button[data-cajas-vista]').forEach((btn) => {
8256|             btn.addEventListener('click', () => {
8257|                 const v = btn.getAttribute('data-cajas-vista');
8258|                 if (!v || v === cajasVistaSolapa) return;
8259|                 cajasVistaSolapa = v;
8260|                 syncCajasPaneles();
8261|                 if (v === 'tipos') loadTiposMovimientoCajaTable();
8262|             });
8263|         });
8264|     }
8265|     syncCajasPaneles();
8266|     const btnExp = document.getElementById('cajas-btn-exportar-excel');
8267|     if (btnExp) {
8268|         btnExp.addEventListener('click', () => {
8269|             exportarMovimientosCajaExcel();
8270|         });
8271|     }
8272|     cajasMovFiltrosListenersAttached = true;
8273| }
8274|
8275| function loadTiposMovimientoCaja() {
8276|     return client

```

```

8277 |     .from('tipos_movimiento_caja')
8278 |     .select('id, nombre, direccion')
8279 |     .eq('activo', true)
8280 |     .order('nombre')
8281 |     .then((res) => {
8282 |         tiposMovimientoCaja = res.data || [];
8283 |         return tiposMovimientoCaja;
8284 |     });
8285 | }
8286 |
8287 | function openModalMovimientoCaja(registro) {
8288 |     const backdrop = document.getElementById('modal-movimiento-caja-backdrop');
8289 |     const titulo = document.getElementById('modal-movimiento-caja-titulo');
8290 |     const form = document.getElementById('form-movimiento-caja');
8291 |     const idEl = document.getElementById('mov-caja-id');
8292 |     const ordenIdEl = document.getElementById('mov-caja-orden-id');
8293 |     const hintOrden = document.getElementById('mov-caja-hint-orden');
8294 |     const wrapTipoCaja = document.getElementById('mov-caja-wrap-tipo-caja');
8295 |     const selTipoCaja = document.getElementById('mov-caja-tipo-caja');
8296 |     const selTipo = document.getElementById('mov-caja-tipo');
8297 |     const selMoneda = document.getElementById('mov-caja-moneda');
8298 |     const inputMonto = document.getElementById('mov-caja-monto');
8299 |     const inputConcepto = document.getElementById('mov-caja-concepto');
8300 |     const fechaEl = document.getElementById('mov-caja-fecha');
8301 |     if (!backdrop || !form || !selTipo) return;
8302 |
8303 |     loadTiposMovimientoCaja().then(() => {
8304 |         selTipo.innerHTML = tiposMovimientoCaja
8305 |             .map((t) => `<option value="${t.id}" data-direccion="${t.direccion}">${escapeHtml(t.nombre)} (${t.direccion})</option>`)
8306 |             .join('');
8307 |         if (tiposMovimientoCaja.length === 0) selTipo.innerHTML = `<option value="">No hay tipos cargados</option>`;
8308 |
8309 |         const esOrden = registro && registro.orden_id;
8310 |         if (idEl) idEl.value = registro ? registro.id : '';
8311 |         if (ordenIdEl) ordenIdEl.value = registro && registro.orden_id ? registro.orden_id : '';
8312 |         if (titulo) titulo.textContent = registro ? 'Editar movimiento' : 'Nuevo movimiento de caja';
8313 |         if (hintOrden) hintOrden.style.display = esOrden ? 'block' : 'none';
8314 |
8315 |         if (wrapTipoCaja) wrapTipoCaja.style.display = esOrden ? 'none' : 'block';
8316 |         if (selTipoCaja) {
8317 |             selTipoCaja.disabled = esOrden;
8318 |             const cajaTipo = (registro && registro.caja_tipo) ? String(registro.caja_tipo).toLowerCase() : 'efectivo';
8319 |             selTipoCaja.value = (cajaTipo === 'banco') ? 'banco' : 'efectivo';
8320 |         }
8321 |
8322 |         selMoneda.disabled = esOrden;
8323 |         selTipo.disabled = esOrden;
8324 |         inputMonto.disabled = esOrden;
8325 |
8326 |         if (registro) {
8327 |             selMoneda.value = registro.moneda || 'USD';
8328 |             fechaEl.value = (registro.fecha || '').toString().slice(0, 10);
8329 |             inputConcepto.value = registro.concepto || '';
8330 |             inputMonto.value = formatImporteParaInput(Math.abs(Number(registro.monto)));
8331 |             if (!esOrden) selTipo.value = registro.tipo_movimiento_id || '';
8332 |         } else {
8333 |             const hoy = fechaHoyYYYYMMDDArgentina();
8334 |             fechaEl.value = hoy;
8335 |             selMoneda.value = cajasMonedaActual;
8336 |             inputConcepto.value = '';
8337 |             inputMonto.value = '';
8338 |         }
8339 |         backdrop.classList.add('activo');
8340 |         setupInputImporte(inputMonto);
8341 |     });
8342 | }
8343 |
8344 | function closeModalMovimientoCaja() {
8345 |     const backdrop = document.getElementById('modal-movimiento-caja-backdrop');
8346 |     if (backdrop) backdrop.classList.remove('activo');
8347 | }
8348 |
8349 | function saveMovimientoCaja() {
8350 |     const idEl = document.getElementById('mov-caja-id');
8351 |     const ordenIdEl = document.getElementById('mov-caja-orden-id');
8352 |     const id = idEl && idEl.value ? idEl.value.trim() : '';
8353 |     const esDeOrden = ordenIdEl && ordenIdEl.value && ordenIdEl.value.trim() !== '';
8354 |     const concepto = document.getElementById('mov-caja-concepto').value.trim() || null;
8355 |     const fecha = document.getElementById('mov-caja-fecha').value;
8356 |
8357 |     if (id && esDeOrden) {
8358 |         const payload = { concepto, fecha: fecha || fechaHoyYYYYMMDDArgentina() };
8359 |         client
8360 |             .from('movimientos_caja')
8361 |             .update(payload)
8362 |             .eq('id', id)
8363 |             .then((res) => {
8364 |                 if (res.error) {
8365 |                     showToast('Error: ' + (res.error.message || 'No se pudo guardar.'), 'error');
8366 |                     return;
8367 |                 }
8368 |                 closeModalMovimientoCaja();
8369 |                 loadCajas();
8370 |             });
8371 |     }
8372 |     return;
8373 | }

```



```

8374 | const moneda = document.getElementById('mov-caja-moneda').value;
8375 | const tipoId = document.getElementById('mov-caja-tipo').value;
8376 | const montoInput = parseFloat(document.getElementById('mov-caja-monto').value);
8377 | if (!tipoId || isNaN(montoInput) || montoInput <= 0) {
8378 |   showToast('Completa tipo y monto (número positivo).', 'error');
8379 |   return;
8380 | }
8381 | const tipo = tiposMovimientoCaja.find((t) => t.id === tipoId);
8382 | const signo = tipo && tipo.direccion === 'egreso' ? -1 : 1;
8383 | const monto = montoInput * signo;
8384 | const cajaTipoEl = document.getElementById('mov-caja-tipo-caja');
8385 | const cajaTipo = (cajaTipoEl && cajaTipoEl.value) ? cajaTipoEl.value : 'efectivo';
8386 | const ahora = new Date().toISOString();
8387 | const payloadBase = {
8388 |   moneda,
8389 |   monto,
8390 |   tipo_movimiento_id: tipoId,
8391 |   orden_id: null,
8392 |   caja_tipo: cajaTipo,
8393 |   concepto,
8394 |   fecha: fecha || fechaHoyYYYYMMDDArgentina(),
8395 |   usuario_id: currentUserId,
8396 | };
8397 | const payload = id ? payloadBase : { ...payloadBase, estado: 'cerrado', estado_fecha: ahora };
8398 |
8399 | function ejecutarGuardadoMovimientoCajaManual() {
8400 |   function guardarCajaInsUpd() {
8401 |     if (id) {
8402 |       client
8403 |         .from('movimientos_caja')
8404 |         .update(payloadBase)
8405 |         .eq('id', id)
8406 |         .then((res) => {
8407 |           if (res.error) {
8408 |             showToast('Error: ' + (res.error.message || 'No se pudo guardar.'), 'error');
8409 |             return;
8410 |           }
8411 |           closeModalMovimientoCaja();
8412 |           loadCajas();
8413 |         });
8414 |     } else {
8415 |       client
8416 |         .from('movimientos_caja')
8417 |         .insert(payload)
8418 |         .then((res) => {
8419 |           if (res.error) {
8420 |             showToast('Error: ' + (res.error.message || 'No se pudo guardar.'), 'error');
8421 |             return;
8422 |           }
8423 |           closeModalMovimientoCaja();
8424 |           loadCajas();
8425 |         });
8426 |     }
8427 |   }
8428 |   if (!id && signo < 0) {
8429 |     fetchMovimientosCajaCerradosSinSync().then((list) => {
8430 |       const v = validarSaldoCajaParaEgresoLista(list, { cajaTipo, moneda, montoAbs: montoInput });
8431 |       if (!v.ok) {
8432 |         showToast(v.mensaje, 'error');
8433 |         return;
8434 |       }
8435 |       guardarCajaInsUpd();
8436 |     });
8437 |     return;
8438 |   }
8439 |   guardarCajaInsUpd();
8440 | }
8441 |
8442 | if (id) {
8443 |   ejecutarGuardadoMovimientoCajaManual();
8444 | } else {
8445 |   showConfirm(
8446 |     'Este movimiento solo modifica la caja (efectivo, banco o cheque según elegiste). No genera ningún movimiento en las cuentas corrientes de clientes ni de intermediarios.
8447 |     \n\nSi lo que necesitas es registrar algo que impacte esas cuentas, hacelo desde el menú Cuenta corriente (p. ej. movimiento manual en CC).\n\n¿Guardar solo en caja?',
8448 |     'Si, guardar en caja',
8449 |     ejecutarGuardadoMovimientoCajaManual,
8450 |     undefined,
8451 |     'Cancelar',
8452 |     'Movimiento solo en caja'
8453 |   );
8454 | }
8455 | }
8456 |
8457 | function escapeHtml(s) {
8458 |   if (s == null) return '';
8459 |   return String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
8460 | }
8461 |
8462 | /** Icono de moneda en columnas angostas del listado de órdenes y modal pendientes. */
8463 | function htmlIconoMonedaCeldaOrden(moneda) {
8464 |   const mon = String(moneda || '').toUpperCase().trim();
8465 |   const src = URL_ICONO_MONEDA_ASSETS[mon];
8466 |   if (!src) return '<span class="td-orden-moneda-vacio" aria-hidden="true"></span>';
8467 |   return '';
8468 | }
8469 |
8470 | /** Devuelve HTML con clase por participante: operador azul negrita, Cliente negro, Intermediario violeta. */

```

```

8470 function participantLabelHtml(role) {
8471   const r = (role || '').toLowerCase();
8472   let label;
8473   if (r === 'pandy') label = etiquetaRolParticipanteUi('pandy');
8474   else if (r === 'cliente') label = 'Cliente';
8475   else if (r === 'intermediario') label = 'Intermediario';
8476   else label = role || '-';
8477   const cls = r === 'pandy' ? 'participante-pandy' : (r === 'cliente' ? 'participante-cliente' : 'participante-intermediario');
8478   return '<span class="' + cls + '">' + escapeHtml(label) + '</span>';
8479 }
8480
8481 /**
8482  * Nombres de clientes/intermediarios referidos por transacciones + orden (para columnas Pagador/Cobrador).
8483  * @returns {Promise<{ clientesById: Record<string,string>, intermediariosById: Record<string,string> }>}
8484  */
8485 function fetchMapsNombresParticipantesTransacciones(orden, lista) {
8486   const idsCli = new Set();
8487   const idsInt = new Set();
8488   if (orden && orden.cliente_id) idsCli.add(orden.cliente_id);
8489   if (orden && orden.intermediario_id) idsInt.add(orden.intermediario_id);
8490   (lista || []).forEach((t) => {
8491     if (t.pagador_cliente_id) idsCli.add(t.pagador_cliente_id);
8492     if (t.cobrador_cliente_id) idsCli.add(t.cobrador_cliente_id);
8493     if (t.pagador_intermediario_id) idsInt.add(t.pagador_intermediario_id);
8494     if (t.cobrador_intermediario_id) idsInt.add(t.cobrador_intermediario_id);
8495   });
8496   const arrC = [...idsCli];
8497   const arrI = [...idsInt];
8498   const pC = arrC.length ? client.from('clientes').select('id, nombre').in('id', arrC) : Promise.resolve({ data: [] });
8499   const pI = arrI.length ? client.from('intermediarios').select('id, nombre').in('id', arrI) : Promise.resolve({ data: [] });
8500   return Promise.all([pC, pI]).then(([rC, rI]) => {
8501     const clientesById = {};
8502     (rC.data || []).forEach((c) => { clientesById[c.id] = (c.nombre != null ? String(c.nombre) : '').trim(); });
8503     const intermediariosById = {};
8504     (rI.data || []).forEach((i) => { intermediariosById[i.id] = (i.nombre != null ? String(i.nombre) : '').trim(); });
8505     return { clientesById, intermediariosById };
8506   });
8507 }
8508
8509 /**
8510  * Celda listado: nombre concreto (cliente/intermediario) cuando se puede resolver; si no, etiqueta de rol.
8511  * @param {'pagador'|'cobrador'} lado
8512  * @param {{ clientesById?: Record<string,string>, intermediariosById?: Record<string,string> }} maps
8513  */
8514 function transaccionParticipanteCeldaHtml(t, orden, lado, maps) {
8515   const m = maps || {};
8516   const clientesById = m.clientesById || {};
8517   const intermediariosById = m.intermediariosById || {};
8518   const roleRaw = lado === 'pagador'
8519     ? (t.pagador || (t.tipo === 'egreso' ? t.owner : 'pandy'))
8520     : (t.cobrador || (t.tipo === 'ingreso' ? t.owner : 'pandy'));
8521   const r = String(roleRaw || '').toLowerCase();
8522   const o = orden || null;
8523   if (r === 'pandy') return participantLabelHtml('pandy');
8524   if (r === 'cliente') {
8525     const id = lado === 'pagador'
8526       ? (t.pagador_cliente_id || o?.cliente_id)
8527       : (t.cobrador_cliente_id || o?.cliente_id);
8528     let nombre = id ? (clientesById[id] || ' ') : '';
8529     if (!nombre && o && o.cliente_id && id && String(o.cliente_id) === String(id)) {
8530       const nested = o.clientes;
8531       nombre = (nested && !Array.isArray(nested) ? nested.nombre : (Array.isArray(nested) ? nested[0]?.nombre : '')) || '';
8532       nombre = String(nombre).trim();
8533     }
8534     if (nombre) return '<span class="participante-cliente">' + escapeHtml(nombre) + '</span>';
8535     return participantLabelHtml('cliente');
8536   }
8537   if (r === 'intermediario') {
8538     const id = lado === 'pagador'
8539       ? (t.pagador_intermediario_id || o?.intermediario_id)
8540       : (t.cobrador_intermediario_id || o?.intermediario_id);
8541     let nombre = id ? (intermediariosById[id] || ' ') : '';
8542     if (!nombre && o && o.intermediario_id && id && String(o.intermediario_id) === String(id)) {
8543       const nested = o.intermediarios;
8544       nombre = (nested && !Array.isArray(nested) ? nested.nombre : (Array.isArray(nested) ? nested[0]?.nombre : '')) || '';
8545       nombre = String(nombre).trim();
8546     }
8547     if (nombre) return '<span class="participante-intermediario">' + escapeHtml(nombre) + '</span>';
8548     return participantLabelHtml('intermediario');
8549   }
8550   return participantLabelHtml(roleRaw);
8551 }
8552
8553 /** Devuelve HTML para tipo de transacción: Ingreso verde negrita, Egreso rojo negrita. */
8554 function tipoTransaccionHtml(tipo) {
8555   const esIngreso = (tipo || '').toLowerCase() === 'ingreso';
8556   const label = esIngreso ? 'Ingreso' : 'Egreso';
8557   const cls = esIngreso ? 'tipo-ingreso' : 'tipo-egreso';
8558   return '<span class="' + cls + '">' + escapeHtml(label) + '</span>';
8559 }
8560
8561 /** Orden para listas de transacciones en la UI: siempre ingreso y su contrapartida de egreso; por pagador: primero cliente, segundo Pandy, tercero intermediario (cualquier ti
8562 po de operación). */
8563 function ordenPagador(p) {
8564   const q = (p || '').toLowerCase();
8565   if (q === 'cliente') return 0;
8566   if (q === 'pandy') return 1;

```

```

8566 |   if (q === 'intermediario') return 2;
8567 |   return 3;
8568 | }
8569 | function sortTransaccionesIngresosPrimero(lista) {
8570 |   return (lista || []).slice().sort((a, b) => {
8571 |     const pagadorA = ordenPagador(a.pagador);
8572 |     const pagadorB = ordenPagador(b.pagador);
8573 |     if (pagadorA !== pagadorB) return pagadorA - pagadorB;
8574 |     const aEsIngreso = (a.tipo || '').toLowerCase() === 'ingreso';
8575 |     const bEsIngreso = (b.tipo || '').toLowerCase() === 'ingreso';
8576 |     const tipoA = aEsIngreso ? 0 : 1;
8577 |     const tipoB = bEsIngreso ? 0 : 1;
8578 |     if (tipoA !== tipoB) return tipoA - tipoB;
8579 |     const cobradorA = ordenPagador(a.cobrador);
8580 |     const cobradorB = ordenPagador(b.cobrador);
8581 |     return cobradorA - cobradorB;
8582 |   });
8583 | }
8584 |
8585 | /** Ordena transacciones por numero (1, 2, 3, 4...) para que en instrumentación la fila 1 sea siempre Tx1, la 2 sea Tx2, etc. */
8586 | function sortTransaccionesPorNumero(lista) {
8587 |   return (lista || []).slice().sort((a, b) => {
8588 |     const na = a.numero !== null ? Number(a.numero) : Infinity;
8589 |     const nb = b.numero !== null ? Number(b.numero) : Infinity;
8590 |     return na - nb;
8591 |   });
8592 | }
8593 |
8594 | // --- Formato importes: miles con punto, decimales con coma (es-AR) ---
8595 | function parseImporteInput(str) {
8596 |   if (str == null || typeof str !== 'string') return NaN;
8597 |   const s = str.trim().replace(/\.g, '').replace(',', '.');
8598 |   const n = parseFloat(s, 10);
8599 |   return isNaN(n) ? NaN : n;
8600 | }
8601 |
8602 | /** Guarda solo el monto de una transacción. Si está ejecutada, reajusta CC y caja. Órdenes con filas en `reglas_de_negocio`: solo actualiza la trx y `sincronizarCcYCajaDesde0
rden` (sin insertar "Cancelación de deuda" legacy). Resto: lógica momento cero / Cancelación / caja heredada. Llama onSuccess() tras guardar. */
8603 | function guardarSoloMontoTransaccion(transaccionId, valorInput, onSuccess) {
8604 |   const newMonto = parseImporteInput(typeof valorInput === 'string' ? valorInput : (valorInput && valorInput.value));
8605 |   if (isNaN(newMonto) || newMonto <= 0) {
8606 |     showToast('Monto inválido.', 'error');
8607 |     return Promise.resolve();
8608 |   }
8609 |   if (!currentUserId) return Promise.resolve();
8610 |   return client.from('transacciones').select('id, numero, estado, monto, tipo, instrumentacion_id, modo_pago_id, moneda, cobrador, pagador, concepto, tipo_cambio, owner').eq('
id', transaccionId).single().then((rTr) => {
8611 |     const t = rTr.data;
8612 |     if (!t) return Promise.resolve();
8613 |     const oldMonto = Number(t.monto) || 0;
8614 |     const esEjecutada = (t.estado || '').toLowerCase() === 'ejecutada';
8615 |     if (!esEjecutada) {
8616 |       return client.from('transacciones').update({ monto: newMonto, updated_at: new Date().toISOString() }).eq('id', transaccionId).then((r) => {
8617 |         if (r.error) { showToast('Error al actualizar monto: ' + (r.error?.message || ''), 'error'); return; }
8618 |         if (onSuccess) onSuccess();
8619 |       });
8620 |     }
8621 |     const instrumentacionId = t.instrumentacion_id;
8622 |     if (!instrumentacionId) { showToast('Falta instrumentación.', 'error'); return Promise.resolve(); }
8623 |     return client.from('instrumentacion').select('orden_id').eq('id', instrumentacionId).single().then((rInst) => {
8624 |       const ordenId = rInst.data && rInst.data.orden_id;
8625 |       if (!ordenId) { showToast('No se encontró la orden.', 'error'); return Promise.resolve(); }
8626 |       return client.from('ordenes').select('cliente_id, intermediario_id, moneda_recibida, monto_recibido, moneda_entregada, monto_entregado, numero, tipos_operacion(codigo, u
sa_intermediario)').eq('id', ordenId).single().then((rOrd) => {
8627 |         const orden = rOrd.data || {};
8628 |         const clienteId = orden.cliente_id || null;
8629 |         const mr = Number(orden.monto_recibido) || 0;
8630 |         const me = Number(orden.monto_entregado) || 0;
8631 |         const monR = orden.moneda_recibida || 'USD';
8632 |         const monE = orden.moneda_entregada || 'USD';
8633 |         const ordenLabel = orden.numero !== null ? 'nro orden ' + orden.numero : 'nro orden ' + (ordenId || '').toString().slice(0, 8);
8634 |         const fecha = fechaHoyYYYYMMDDArgentina();
8635 |         const ahora = new Date().toISOString();
8636 |         const toJoinOrd = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
8637 |         const codigoTipoRaw = (toJoinOrd && toJoinOrd.codigo) || null;
8638 |         const codigoTipoNorm = normalizarCodigoTipoOperacion(codigoTipoRaw) || codigoTipoRaw;
8639 |         const usaIntermediarioOrd =
8640 |           (toJoinOrd && toJoinOrd.usa_intermediario === true) || !!orden.intermediario_id;
8641 |         return getReglasDeNegocio(codigoTipoNorm, usaIntermediarioOrd).then((reglasDeNegocio) => {
8642 |           const usarMotorReglasNegocio = Array.isArray(reglasDeNegocio) && reglasDeNegocio.length > 0;
8643 |           if (usarMotorReglasNegocio) {
8644 |             const tHypo = { ...t, monto: newMonto };
8645 |             return validarSaldoCajaAntesMarcarTransaccionEjecutada(tHypo, instrumentacionId, ordenId).then((vSaldo) => {
8646 |               if (!vSaldo.ok) {
8647 |                 showToast(vSaldo.mensaje || 'Saldo de caja insuficiente.', 'error');
8648 |                 return Promise.resolve();
8649 |               }
8650 |               return client
8651 |                 .from('transacciones')
8652 |                 .update({ monto: newMonto, updated_at: new Date().toISOString() })
8653 |                 .eq('id', transaccionId)
8654 |                 .then((rUp) => {
8655 |                   if (rUp.error) {
8656 |                     showToast('Error al actualizar monto: ' + (rUp.error?.message || ''), 'error');
8657 |                     return Promise.resolve();
8658 |                   }
8659 |                   return sincronizarCcYCajaDesdeOrden(ordenId);

```

```

8660 |         })
8661 |         .then(() => {
8662 |             if (onSuccess) onSuccess();
8663 |         })
8664 |         .catch((err) => {
8665 |             showToast('Error al reajustar CC/caja: ' + (err?.message || String(err)), 'error');
8666 |         });
8667 |     });
8668 | }
8669 | return Promise.all([
8670 |     client.from('movimientos_cuenta_corriente').select('id, transaccion_id, concepto, monto_usd, monto_ars, monto_eur').eq('orden_id', ordenId).eq('cliente_id', clienteI
d),
8671 |     client.from('transacciones').select('id, tipo, monto, estado, cobrador, pagador').eq('instrumentacion_id', instrumentacionId),
8672 | ]).then([({Mov, rTrxList}) => {
8673 |     const rows = rMov.data || [];
8674 |     const listTrx = rTrxList.data || [];
8675 |     const sumIngresosOthers = listTrx.filter((tr) => tr.tipo === 'ingreso' && tr.pagador === 'cliente' && tr.estado === 'ejecutada' && tr.id !== transaccionId).reduce((s
, tr) => s + Number(tr.monto), 0);
8676 |     const sumEgresosOthers = listTrx.filter((tr) => tr.tipo === 'egreso' && tr.cobrador === 'cliente' && tr.estado === 'ejecutada' && tr.id !== transaccionId).reduce((s
, tr) => s + Number(tr.monto), 0);
8677 |     const sumIngresosClienteEjecutados = sumIngresosOthers + (t.tipo === 'ingreso' && t.pagador === 'cliente' ? newMonto : 0);
8678 |     const sumEgresosClienteEjecutados = sumEgresosOthers + (t.tipo === 'egreso' && t.cobrador === 'cliente' ? newMonto : 0);
8679 |     const cancelacionIds = rows.filter((m) => m.transaccion_id === transaccionId && (m.concepto || '').includes('Cancelación de deuda') || (m.concepto || '').includes('
Contraparte cancelación')).map((m) => m.id);
8680 |     const rowDebe = rows.find((r) => { const c = (r.concepto || '').toUpperCase(); return c.includes('DEBE') || c.includes('COMPROMISO DE PAGO'); });
8681 |     const rowComp = rows.find((r) => { const c = (r.concepto || '').normalize('NFD').replace(/u0301/g, '').toUpperCase(); return c.includes('COMPENSACION') || c.include
s('COMPROMISO A COBRAR'); });
8682 |     const tieneMomentoCero = rowDebe && rowComp && (rowDebe.monto_usd != null || rowDebe.monto_ars != null || rowDebe.monto_eur != null);
8683 |     let prom = Promise.resolve();
8684 |     if (cancelacionIds.length > 0) prom = prom.then(() => Promise.all(cancelacionIds.map((id) => client.from('movimientos_cuenta_corriente').delete().eq('id', id))));
8685 |     prom = prom.then(() => client.from('movimientos_caja').delete().eq('transaccion_id', transaccionId));
8686 |     prom = prom.then(() => client.from('transacciones').update({ monto: newMonto, updated_at: ahora }).eq('id', transaccionId));
8687 |     const esIngreso = t.tipo === 'ingreso';
8688 |     const delta = oldMonto - newMonto;
8689 |     if (tieneMomentoCero && clienteId) {
8690 |         const amountDebeMonR = rowDebe && (monR === 'USD' ? rowDebe.monto_usd : (monR === 'ARS' ? rowDebe.monto_ars : rowDebe.monto_eur));
8691 |         const amountCompMonE = rowComp && (monE === 'USD' ? rowComp.monto_usd : (monE === 'ARS' ? rowComp.monto_ars : rowComp.monto_eur));
8692 |         const restoEnFila = esIngreso ? Math.abs(Number(amountDebeMonR) | 0) : Math.abs(Number(amountCompMonE) | 0);
8693 |         const restoTrxId = esIngreso ? (rowDebe.transaccion_id || null) : (rowComp.transaccion_id || null);
8694 |         // Si la fila es nuestra (ejecutamos todo sin split), el nuevo resto es solo delta. Si hay otra transacción pendiente, nuevo resto = restoEnFila + delta.
8695 |         const newRestoMonto = (restoTrxId && restoTrxId !== transaccionId) ? Math.max(0, restoEnFila + delta) : Math.max(0, delta);
8696 |         if (restoTrxId && restoTrxId !== transaccionId && newRestoMonto >= 1e-6) {
8697 |             prom = prom.then(() => client.from('transacciones').update({ monto: newRestoMonto, updated_at: ahora }).eq('id', restoTrxId));
8698 |             const difUsd = esIngreso ? (monR === 'USD' ? -newRestoMonto : (monE === 'USD' ? -ratioCc(me * newRestoMonto, mr, newRestoMonto) : 0)) : (monR === 'USD' ? -ratioC
c(me * newRestoMonto, me, newRestoMonto) : (monE === 'USD' ? -newRestoMonto : 0));
8699 |             const difArs = esIngreso ? (monR === 'ARS' ? -newRestoMonto : (monE === 'ARS' ? -ratioCc(me * newRestoMonto, mr, newRestoMonto) : 0)) : (monR === 'ARS' ? -ratioC
c(me * newRestoMonto, me, newRestoMonto) : (monE === 'ARS' ? -newRestoMonto : 0));
8700 |             const difEur = esIngreso ? (monR === 'EUR' ? -newRestoMonto : (monE === 'EUR' ? -ratioCc(me * newRestoMonto, mr, newRestoMonto) : 0)) : (monR === 'EUR' ? -ratioC
c(me * newRestoMonto, me, newRestoMonto) : (monE === 'EUR' ? -newRestoMonto : 0));
8701 |             const remUsd = esIngreso ? (monR === 'USD' ? newRestoMonto : (monE === 'USD' ? ratioCc(me * newRestoMonto, mr, newRestoMonto) : 0)) : (monR === 'USD' ? ratioCc(m
r * newRestoMonto, me, newRestoMonto) : (monE === 'USD' ? newRestoMonto : 0));
8702 |             const remArs = esIngreso ? (monR === 'ARS' ? newRestoMonto : (monE === 'ARS' ? ratioCc(me * newRestoMonto, mr, newRestoMonto) : 0)) : (monR === 'ARS' ? ratioCc(m
r * newRestoMonto, me, newRestoMonto) : (monE === 'ARS' ? newRestoMonto : 0));
8703 |             const remEur = esIngreso ? (monR === 'EUR' ? newRestoMonto : (monE === 'EUR' ? ratioCc(me * newRestoMonto, mr, newRestoMonto) : 0)) : (monR === 'EUR' ? ratioCc(m
r * newRestoMonto, me, newRestoMonto) : (monE === 'EUR' ? newRestoMonto : 0));
8704 |             prom = prom.then(() => client.from('movimientos_cuenta_corriente').update({
8705 |                 monto_usd: esIngreso ? difUsd : remUsd, monto_ars: esIngreso ? difArs : remArs, monto_eur: esIngreso ? difEur : remEur, estado_fecha: ahora,
8706 |             }).eq('id', rowDebe.id));
8707 |             prom = prom.then(() => client.from('movimientos_cuenta_corriente').update({
8708 |                 monto_usd: esIngreso ? remUsd : difUsd, monto_ars: esIngreso ? remArs : difArs, monto_eur: esIngreso ? remEur : difEur, estado_fecha: ahora,
8709 |             }).eq('id', rowComp.id));
8710 |         } else if (newRestoMonto > 1e-6) {
8711 |             prom = prom.then(() => client.from('transacciones').insert({
8712 |                 instrumentacion_id: instrumentacionId, tipo: t.tipo, modo_pago_id: t.modo_pago_id, moneda: t.moneda || (esIngreso ? monR : monE),
8713 |                 monto: newRestoMonto, cobrador: t.cobrador, pagador: t.pagador, owner: t.owner || 'pandy', estado: 'pendiente',
8714 |                 concepto: t.concepto || '', tipo_cambio: t.tipo_cambio, updated_at: ahora,
8715 |             }).select('id').single()).then((rNew) => {
8716 |                 const newId = rNew.data && rNew.data.id;
8717 |                 if (!newId) return;
8718 |                 const difUsd = esIngreso ? (monR === 'USD' ? -newRestoMonto : (monE === 'USD' ? -ratioCc(me * newRestoMonto, mr, newRestoMonto) : 0)) : (monR === 'USD' ? -rati
oCc(me * newRestoMonto, me, newRestoMonto) : (monE === 'USD' ? -newRestoMonto : 0));
8719 |                 const difArs = esIngreso ? (monR === 'ARS' ? -newRestoMonto : (monE === 'ARS' ? -ratioCc(me * newRestoMonto, mr, newRestoMonto) : 0)) : (monR === 'ARS' ? -rati
oCc(me * newRestoMonto, me, newRestoMonto) : (monE === 'ARS' ? -newRestoMonto : 0));
8720 |                 const difEur = esIngreso ? (monR === 'EUR' ? -newRestoMonto : (monE === 'EUR' ? -ratioCc(me * newRestoMonto, mr, newRestoMonto) : 0)) : (monR === 'EUR' ? -rati
oCc(me * newRestoMonto, me, newRestoMonto) : (monE === 'EUR' ? -newRestoMonto : 0));
8721 |                 const remUsd = esIngreso ? (monR === 'USD' ? newRestoMonto : (monE === 'USD' ? ratioCc(me * newRestoMonto, mr, newRestoMonto) : 0)) : (monR === 'USD' ? ratioCc
(mr * newRestoMonto, me, newRestoMonto) : (monE === 'USD' ? newRestoMonto : 0));
8722 |                 const remArs = esIngreso ? (monR === 'ARS' ? newRestoMonto : (monE === 'ARS' ? ratioCc(me * newRestoMonto, mr, newRestoMonto) : 0)) : (monR === 'ARS' ? ratioCc
(mr * newRestoMonto, me, newRestoMonto) : (monE === 'ARS' ? newRestoMonto : 0));
8723 |                 const remEur = esIngreso ? (monR === 'EUR' ? newRestoMonto : (monE === 'EUR' ? ratioCc(me * newRestoMonto, mr, newRestoMonto) : 0)) : (monR === 'EUR' ? ratioCc
(mr * newRestoMonto, me, newRestoMonto) : (monE === 'EUR' ? newRestoMonto : 0));
8724 |                 return Promise.all([
8725 |                     client.from('movimientos_cuenta_corriente').update({
8726 |                         transaccion_id: newId, monto_usd: esIngreso ? difUsd : remUsd, monto_ars: esIngreso ? difArs : remArs, monto_eur: esIngreso ? difEur : remEur, estado_fecha
: ahora,
8727 |                     }).eq('id', rowDebe.id),
8728 |                     client.from('movimientos_cuenta_corriente').update({
8729 |                         monto_usd: esIngreso ? remUsd : difUsd, monto_ars: esIngreso ? remArs : difArs, monto_eur: esIngreso ? remEur : difEur, estado_fecha: ahora,
8730 |                     }).eq('id', rowComp.id),
8731 |                 ]);
8732 |             });
8733 |         } else if (newRestoMonto < 1e-6) {
8734 |             prom = prom.then(() => Promise.all([
8735 |                 client.from('movimientos_cuenta_corriente').update({ estado: 'cerrado', estado_fecha: ahora }).eq('id', rowDebe.id),
8736 |                 client.from('movimientos_cuenta_corriente').update({ estado: 'cerrado', estado_fecha: ahora }).eq('id', rowComp.id),
8737 |             ]));
8738 |         }
8739 |     });

```

```

8739 |     }
8740 |     // Sin momento cero: solo crear "resto" si tras este cambio la suma ejecutada del cliente sigue siendo menor que mr/me.
8741 |     const faltaIngreso = mr - sumIngresosClienteEjecutados > 1e-6;
8742 |     const faltaEgreso = me - sumEgresosClienteEjecutados > 1e-6;
8743 |     const splitSinMc = !tieneMomentoCero && clienteId && (
8744 |       (esIngreso && t.pagador === 'cliente' && mr > 1e-6 && newMonto > 1e-6 && faltaIngreso) ||
8745 |       (!esIngreso && t.cobrador === 'cliente' && me > 1e-6 && newMonto > 1e-6 && faltaEgreso)
8746 |     );
8747 |     if (splitSinMc) {
8748 |       const diferencia = esIngreso ? (mr - sumIngresosClienteEjecutados) : (me - sumEgresosClienteEjecutados);
8749 |       if (diferencia >= 1e-6) {
8750 |         prom = prom.then(() => client.from('transacciones').insert({
8751 |           instrumentacion_id: instrumentacionId,
8752 |           tipo: t.tipo,
8753 |           modo_pago_id: t.modo_pago_id,
8754 |           moneda: t.moneda || (esIngreso ? monR : monE),
8755 |           monto: diferencia,
8756 |           cobrador: t.cobrador,
8757 |           pagador: t.pagador,
8758 |           owner: t.owner || 'pandy',
8759 |           estado: 'pendiente',
8760 |           concepto: t.concepto || '',
8761 |           tipo_cambio: t.tipo_cambio || null,
8762 |           updated_at: ahora,
8763 |         })).then(() => {}));
8764 |       }
8765 |     }
8766 |     function montosCancelacionItem(montoTrx, pagadorCliente) {
8767 |       if (monR === monE) {
8768 |         const signo = pagadorCliente ? 1 : -1;
8769 |         return { monto_usd: numCc(monR === 'USD' ? signo * montoTrx : 0), monto_ars: numCc(monR === 'ARS' ? signo * montoTrx : 0), monto_eur: numCc(monR === 'EUR' ? signo
8770 | o * montoTrx : 0) };
8771 |       }
8772 |       if (pagadorCliente) {
8773 |         const enMonE = ratioCc(montoTrx * me, mr, montoTrx);
8774 |         return { monto_usd: numCc(monR === 'USD' ? montoTrx : (monE === 'USD' ? enMonE : 0)), monto_ars: numCc(monR === 'ARS' ? montoTrx : (monE === 'ARS' ? enMonE : 0))
8775 | , monto_eur: numCc(monR === 'EUR' ? montoTrx : (monE === 'EUR' ? enMonE : 0)) };
8776 |       }
8777 |       const enMonR = ratioCc(montoTrx * mr, me, montoTrx);
8778 |       return { monto_usd: numCc(monR === 'USD' ? -montoTrx : 0), monto_ars: numCc(monR === 'ARS' ? -enMonR : (monE === 'ARS' ? -montoTrx : 0
8779 | )), monto_eur: numCc(monR === 'EUR' ? -enMonR : (monE === 'EUR' ? -montoTrx : 0)) };
8780 |     }
8781 |     if (clienteId) {
8782 |       prom = prom.then(() => {
8783 |         if (monR !== monE && t.pagador === 'cliente') {
8784 |           const enMonEval = ratioCc(newMonto * me, mr, newMonto);
8785 |           const cancelacion = { monto_usd: numCc(monR === 'USD' ? newMonto : 0), monto_ars: numCc(monR === 'ARS' ? newMonto : 0), monto_eur: numCc(monR === 'EUR' ? newMo
8786 | nto : 0) };
8787 |           const contraparte = { monto_usd: numCc(monR === 'USD' ? -newMonto : (monE === 'USD' ? -enMonEval : 0)), monto_ars: numCc(monR === 'ARS' ? -newMonto : (monE ===
8788 | 'ARS' ? -enMonEval : 0)), monto_eur: numCc(monR === 'EUR' ? -newMonto : (monE === 'EUR' ? -enMonEval : 0)) };
8789 |           return client.from('movimientos_cuenta_corriente').insert({
8790 |             cliente_id: clienteId, orden_id: ordenId, transaccion_id: transaccionId, concepto: 'Cancelación de deuda ' + ordenLabel,
8791 |             fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora, monto_usd: cancelacion.monto_usd, monto_ars: cancelacion.monto_ars, monto_eur: canc
8792 | elacion.monto_eur, moneda: monR, monto: 0,
8793 |           }).then(() => client.from('movimientos_cuenta_corriente').insert({
8794 |             cliente_id: clienteId, orden_id: ordenId, transaccion_id: transaccionId, concepto: 'Contraparte cancelación ' + ordenLabel,
8795 |             fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora, monto_usd: contraparte.monto_usd, monto_ars: contraparte.monto_ars, monto_eur: cont
8796 | raparte.monto_eur, moneda: monE, monto: 0,
8797 |           }));
8798 |         }
8799 |         const montos = montosCancelacionItem(newMonto, t.pagador === 'cliente');
8800 |         return client.from('movimientos_cuenta_corriente').insert({
8801 |           cliente_id: clienteId, orden_id: ordenId, transaccion_id: transaccionId, concepto: 'Cancelación de deuda ' + ordenLabel,
8802 |           fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora, monto_usd: montos.monto_usd, monto_ars: montos.monto_ars, monto_eur: montos.monto_eur
8803 | , moneda: esIngreso ? monR : monE, monto: 0,
8804 |         });
8805 |       });
8806 |     }
8807 |     prom = prom.then(() => client.from('modos_pago').select('codigo').eq('id', t.modo_pago_id).single()).then((rModo) => {
8808 |       const codigo = (rModo.data && rModo.codigo) || '';
8809 |       const cajaTipo = codigoCajaTipoDesdeCodigo(codigo);
8810 |       const signoCaja = (t.cobrador || '') === 'pandy' ? 1 : -1;
8811 |       const concepto = conceptoCajaTransaccion((t.cobrador || '') === 'pandy', t.moneda || 'USD', newMonto, orden.numero, t.numero);
8812 |       return client.from('movimientos_caja').insert({
8813 |         moneda: t.moneda || 'USD', monto: signoCaja * newMonto, caja_tipo: cajaTipo, transaccion_id: transaccionId,
8814 |         orden_numero: orden.numero !== null ? orden.numero : null, transaccion_numero: t.numero !== null ? t.numero : null,
8815 |         concepto, fecha, usuario_id: currentUserId,
8816 |       });
8817 |     });
8818 |     return prom.then(() => { if (onSuccess) onSuccess(); }).catch((err) => { showToast('Error al reajustar CC/caja: ' + (err?.message || ''), 'error'); });
8819 |   });
8820 | });
8821 | /** Guarda solo el modo de pago de una transacción. Si está ejecutada, reajusta caja (borra movimiento anterior e inserta uno con el nuevo modo). Llama onSuccess() tras guarda
8822 | r, onFailure() opcional si no se puede guardar (ej. operación CHEQUE con modo Cheque). */
8823 | function guardarSoloModoPago(transaccionId, modoPagoId, onSuccess, onFailure) {
8824 |   if (!modoPagoId || !transaccionId || !currentUserId) return Promise.resolve();
8825 |   return client.from('transacciones').select('id, numero, estado, monto, moneda, concepto, instrumentacion_id, modo_pago_id, cobrador').eq('id', transaccionId).single().then((
8826 | rTr) => {
8827 |     const t = rTr.data;
8828 |     if (!t) return Promise.resolve();
8829 |     if (t.modo_pago_id === modoPagoId) {
8830 |       if (onSuccess) onSuccess();
8831 |     }
8832 |   });
8833 | }

```

```

8826 |     return Promise.resolve();
8827 | }
8828 | return client.from('instrumentacion').select('orden_id').eq('id', t.instrumentacion_id).single().then((rInst) => {
8829 |     const ordenId = rInst.data && rInst.data.orden_id;
8830 |     if (!ordenId) return Promise.resolve();
8831 |     return Promise.all([
8832 |         client.from('ordenes').select('numero, tipos_operacion(codigo, moneda_in, moneda_out)').eq('id', ordenId).single(),
8833 |         client.from('modos_pago').select('codigo').eq('id', t.modopago_id).single(),
8834 |     ]).then(([rOrd, rModoActual]) => {
8835 |         const to = rOrd.data?.tipos_operacion;
8836 |         const tipoCodigo = to?.codigo || '';
8837 |         const modoActualCodigo = (rModoActual.data && rModoActual.data.codigo) || '';
8838 |         if (esTipoOperacionChequeArs(tipoCodigo, to?.moneda_in, to?.moneda_out) && modoActualCodigo === 'cheque') {
8839 |             showToast('En operación CHEQUE no se puede cambiar el modo de pago de las transacciones generadas con Cheque.', 'error');
8840 |             if (onFailure) onFailure();
8841 |             return Promise.resolve();
8842 |         }
8843 |         const esEjecutada = (t.estado || '').toLowerCase() === 'ejecutada';
8844 |         return client.from('transacciones').update({ modo_pago_id: modoPagoId, updated_at: new Date().toISOString() }).eq('id', transaccionId).then((r) => {
8845 |             if (r.error) {
8846 |                 showToast('Error al actualizar modo de pago: ' + (r.error?.message || ''), 'error');
8847 |                 return;
8848 |             }
8849 |             if (!esEjecutada) {
8850 |                 if (onSuccess) onSuccess();
8851 |                 return;
8852 |             }
8853 |             const fecha = fechaHoyYYYYMMDDArgentina();
8854 |             const nroOrden = rOrd.data && rOrd.data.numero;
8855 |             return client.from('movimientos_caja').delete().eq('transaccion_id', transaccionId).then(() =>
8856 |                 client.from('modos_pago').select('codigo').eq('id', modoPagoId).single()
8857 |             ).then((rModo) => {
8858 |                 const codigo = (rModo.data && rModo.data.codigo) || '';
8859 |                 const cajaTipo = codigoCajaTipoDesdeCodigo(codigo);
8860 |                 const signoCaja = (t.cobrador || '') === 'pandy' ? 1 : -1;
8861 |                 const concepto = conceptoCajaTransaccion((t.cobrador || '') === 'pandy', t.moneda || 'USD', Number(t.monto) || 0, nroOrden, t.numero);
8862 |                 return client.from('movimientos_caja').insert({
8863 |                     moneda: t.moneda || 'USD', monto: signoCaja * (Number(t.monto) || 0), caja_tipo: cajaTipo, transaccion_id: transaccionId,
8864 |                     orden_numero: nroOrden != null ? nroOrden : null, transaccion_numero: t.numero != null ? t.numero : null,
8865 |                     concepto, fecha, usuario_id: currentUserId,
8866 |                 });
8867 |             }).then((rIns) => {
8868 |                 if (rIns.error) showToast('Error al actualizar caja: ' + (rIns.error?.message || ''), 'error');
8869 |                 if (onSuccess) onSuccess();
8870 |             });
8871 |         });
8872 |     });
8873 | });
8874 | });
8875 | }
8876 |
8877 | /**
8878 |  * Formato en tiempo real: solo dígitos y una coma (decimal).
8879 |  * - Parte entera: se agregan puntos como separador de miles al cumplirse (ej. 1000 → 1.000).
8880 |  * - Parte decimal: solo si el usuario escribe una coma; después de la coma, solo dígitos (sin puntos).
8881 |  * No se agregan decimales automáticamente; el separador decimal es la coma al escribir.
8882 |  */
8883 | function formatImporteInputOnType(inputEl, maxDecimales, soloComaDecimal) {
8884 |     if (!inputEl) return;
8885 |     const maxDec = (typeof maxDecimales === 'number' && maxDecimales >= 0) ? maxDecimales : 2;
8886 |     const oldValue = inputEl.value;
8887 |     const cursorPos = inputEl.selectionStart ?? oldValue.length;
8888 |
8889 |     // Extraer parte entera (solo dígitos; los puntos son miles y se quitan) y parte decimal (solo si el usuario escribió una coma)
8890 |     let raw = oldValue.replace(/,/g, '');
8891 |     if (soloComaDecimal && raw.includes(',')) {
8892 |         // Aceptar punto como decimal y convertirlo a coma (teclado en inglés): "2.5" → "2,5"
8893 |         const partes = raw.split('.');
8894 |         if (partes.length === 2 && /^d*$/i.test(partes[0]) && /^d*$/i.test(partes[1]) && partes[1].length <= maxDec) raw = partes[0] + ',' + partes[1];
8895 |         else if (partes.length === 2 && partes[1] === '' && /^d*$/i.test(partes[0])) raw = partes[0] + ',';
8896 |     }
8897 |     if (!soloComaDecimal) raw = raw.replace(/./g, ''); // quitar puntos de miles
8898 |     const idxComa = raw.indexOf(',');
8899 |     const tieneDecimal = idxComa >= 0;
8900 |     let parteEnteraStr = tieneDecimal ? raw.slice(0, idxComa).replace(/\D/g, '') : raw.replace(/\D/g, '');
8901 |     const valorPrevio = inputEl.importeValorPrevio;
8902 |     inputEl.importeValorPrevio = oldValue; // guardar para la próxima tecla
8903 |     // Si el usuario escribe sobre "0" (modal recién abierto): "01" → "1", y "10" (cursor al inicio) → "1"
8904 |     if (parteEnteraStr.length > 1) {
8905 |         if (parteEnteraStr[0] === '0') {
8906 |             parteEnteraStr = parteEnteraStr.replace(/^0+/, '') || '0';
8907 |         } else if ((valorPrevio === '0' || valorPrevio === '') && parteEnteraStr === '10') {
8908 |             parteEnteraStr = '1';
8909 |         }
8910 |     }
8911 |     const parteDecimalStr = tieneDecimal ? raw.slice(idxComa + 1).replace(/\D/g, '').slice(0, maxDec) : '';
8912 |
8913 |     // Formatear parte entera con puntos (miles); parte decimal sin puntos, solo tras coma. Mantener la coma aunque aún no haya decimales (ej. "2,") para que el usuario pueda se
8914 |     guir escribiendo.
8915 |     const formattedEntera = parteEnteraStr === '' ? '' : parteEnteraStr.replace(/B(?=(\d{3})+(?!\d))/g, '.');
8916 |     const formatted = tieneDecimal ? (formattedEntera + ',' + parteDecimalStr) : formattedEntera;
8917 |
8918 |     // Si el valor ya está igual al formateado, es reentrada (el navegador disparó input al cambiar value).
8919 |     // Solo fijar cursor al final para no dejarlo tras el primer dígito (que haría que el siguiente 0 → "10.000").
8920 |     if (formatted === oldValue) {
8921 |         inputEl.setSelectionRange(formatted.length, formatted.length);
8922 |         return;

```

```

8922 | }
8923 |
8924 | // Cuántos "caracteres lógicos" (dígitos o una coma) hay antes del cursor en el valor actual
8925 | const antesDeCursor = oldValue.slice(0, cursorPos);
8926 | let cuentaAntes = 0;
8927 | let viComa = false;
8928 | for (let i = 0; i < antesDeCursor.length; i++) {
8929 |   const c = antesDeCursor[i];
8930 |   if (/\\d/.test(c)) cuentaAntes++;
8931 |   else if ((c === ',' || c === '.') && !viComa) { viComa = true; cuentaAntes++; }
8932 | }
8933 |
8934 | inputEl.value = formatted;
8935 | inputEl._importeValorPrevio = formatted;
8936 |
8937 | // Restaurar cursor: misma cantidad de dígitos (y una coma) que antes
8938 | let newPos = formatted.length;
8939 | if (cuentaAntes > 0) {
8940 |   let count = 0;
8941 |   for (let i = 0; i < formatted.length; i++) {
8942 |     if (/\\d/.test(formatted[i])) count++;
8943 |     else if (formatted[i] === ',') count++;
8944 |     if (count === cuentaAntes) {
8945 |       newPos = i + 1;
8946 |       break;
8947 |     }
8948 |   }
8949 | } else {
8950 |   newPos = 0;
8951 | }
8952 | inputEl.setSelectionRange(newPos, newPos);
8953 | }
8954 |
8955 | function setupInputImporte(inputEl, maxDecimales, soloComaDecimal) {
8956 |   if (!inputEl) return;
8957 |   // Evita acumular listeners si el mismo input se configura en cada apertura de modal (p. ej. transaccion-monto).
8958 |   if (inputEl.dataset.importeInputBound === '1') {
8959 |     inputEl._importeValorPrevio = inputEl.value;
8960 |     return;
8961 |   }
8962 |   inputEl.dataset.importeInputBound = '1';
8963 |   inputEl.addEventListener('focus', () => {
8964 |     if (inputEl.value === '0' || inputEl.value === '') inputEl._importeValorPrevio = inputEl.value || '0';
8965 |   });
8966 |   // Capture: corre antes que otros listeners en bubble (p. ej. actualizarMontoCalculado en transaccion-monto)
8967 |   // para que miles/decimales queden aplicados antes de parsear el valor.
8968 |   inputEl.addEventListener('input', () => formatImporteInputOnType(inputEl, maxDecimales, soloComaDecimal), true);
8969 |   inputEl.addEventListener('blur', () => {
8970 |     const val = inputEl.value.trim();
8971 |     const n = parseImporteInput(val);
8972 |     if (isNaN(n) || val === '') return;
8973 |     if (n === 0) {
8974 |       inputEl.value = '0';
8975 |       inputEl._importeValorPrevio = '0';
8976 |       return;
8977 |     }
8978 |     // No agregar ",00" si el usuario no escribió coma: solo parte entera con miles
8979 |     const usuarioEscribioComa = val.indexOf(',') >= 0;
8980 |     if (soloComaDecimal) {
8981 |       inputEl.value = Number(n).toFixed(2).replace('.', ',');
8982 |     } else if (!usuarioEscribioComa && Number.isInteger(n)) {
8983 |       const entera = String(Math.floor(n)).replace(/\\B(?=(\\d{3})+?!\\d))/g, '.');
8984 |       inputEl.value = entera;
8985 |     } else {
8986 |       inputEl.value = formatImporteDisplay(n);
8987 |     }
8988 |   });
8989 | }
8990 |
8991 | /** Muestra una notificación toast integrada (reemplazo de alert para mensajes de éxito/info). type: 'success' | 'info' | 'error'. Duración en ms; 0 = no auto-cerrar. */
8992 | function showToast(mensaje, type = 'success', duracionMs = 4500) {
8993 |   const container = document.getElementById('toast-container');
8994 |   if (!container) return;
8995 |   const toast = document.createElement('div');
8996 |   toast.className = 'toast toast-' + (type === 'error' ? 'error' : type === 'info' ? 'info' : 'success');
8997 |   const icon = type === 'error'
8998 |     ? '<svg class="toast-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="12" cy="12" r="10"/><line x1="12" y1="8" x2="12" y2="12"/></svg>'
8999 |     : '<svg class="toast-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M22 11.08V12a10 10 0 1 1-5.93-9.14"/><polyline points="22 4 12 14.01 9 11.01"/></svg>';
9000 |   toast.innerHTML = icon + '<span class="toast-text">' + escapeHtml(mensaje) + '</span><button type="button" class="toast-close" aria-label="Cerrar">x</button>';
9001 |   const close = () => {
9002 |     toast.style.opacity = '0';
9003 |     toast.style.transform = 'translateY(-8px)';
9004 |     setTimeout(() => toast.remove(), 200);
9005 |   };
9006 |   toast.querySelector('.toast-close').addEventListener('click', close);
9007 |   if (duracionMs > 0) setTimeout(close, duracionMs);
9008 |   container.appendChild(toast);
9009 | }
9010 |
9011 | /** Cierra todos los toasts visibles (p. ej. al cerrar un modal para no dejar mensajes de error). */
9012 | function dismissAllToasts() {
9013 |   const container = document.getElementById('toast-container');
9014 |   if (!container) return;
9015 |   container.querySelectorAll('.toast').forEach((toast) => {
9016 |     toast.style.opacity = '0';

```

```

9017| toast.style.transform = 'translateY(-8px)';
9018| setTimeout(() => toast.remove(), 200);
9019| });
9020| }
9021|
9022| /** Confirmación con mensajería interna (no usar confirm() del navegador). Muestra un modal con mensaje y botones; llama onConfirm al aceptar, onCancel al cancelar o cerrar. t
extoCancelar y titulo opcionales. */
9023| function showConfirm(mensaje, textoConfirmar, onConfirm, onCancel, textoCancelar, tituloModal) {
9024|     const backdrop = document.getElementById('modal-confirm-backdrop');
9025|     const titulo = document.getElementById('modal-confirm-titulo');
9026|     const texto = document.getElementById('modal-confirm-mensaje');
9027|     const btnAceptar = document.getElementById('modal-confirm-aceptar');
9028|     const btnCancel = document.getElementById('modal-confirm-cancelar');
9029|     const btnCerrar = document.getElementById('modal-confirm-cerrar');
9030|     if (!backdrop || !texto || !btnAceptar || !btnCancelar) return;
9031|     texto.textContent = mensaje;
9032|     if (titulo) titulo.textContent = (tituloModal !== undefined && tituloModal !== null && tituloModal !== '') ? tituloModal : 'Confirmar';
9033|     btnAceptar.textContent = textoConfirmar || 'Confirmar';
9034|     btnCancel.textContent = (textoCancelar !== undefined && textoCancelar !== null) ? textoCancelar : 'Cancelar';
9035|     backdrop.classList.add('activo');
9036|     backdrop.setAttribute('aria-hidden', 'false');
9037|     const cerrar = (ejecutado) => {
9038|         backdrop.classList.remove('activo');
9039|         backdrop.setAttribute('aria-hidden', 'true');
9040|         btnAceptar.onclick = null;
9041|         btnCancel.onclick = null;
9042|         btnCerrar.onclick = null;
9043|         if (backdrop._confirmMouseDown) backdrop.removeEventListener('mousedown', backdrop._confirmMouseDown, true);
9044|         if (backdrop._confirmAbort) backdrop.removeEventListener('click', backdrop._confirmAbort, true);
9045|         backdrop._confirmMouseDown = null;
9046|         backdrop._confirmAbort = null;
9047|         if (!ejecutado && typeof onCancel === 'function') onCancel();
9048|     };
9049|     btnAceptar.onclick = () => { if (typeof onConfirm === 'function') onConfirm(); cerrar(true); };
9050|     btnCancel.onclick = () => cerrar(false);
9051|     btnCerrar.onclick = () => cerrar(false);
9052|     backdrop._confirmMouseDown = (e) => { backdrop._confirmMouseOnBackdrop = (e.target === backdrop); };
9053|     backdrop._confirmAbort = (e) => {
9054|         if (e.target === backdrop && backdrop._confirmMouseOnBackdrop) cerrar(false);
9055|         backdrop._confirmMouseOnBackdrop = false;
9056|     };
9057|     backdrop.addEventListener('mousedown', backdrop._confirmMouseDown, true);
9058|     backdrop.addEventListener('click', backdrop._confirmAbort, true);
9059| }
9060|
9061| function setupModalMovimientoCaja() {
9062|     const backdrop = document.getElementById('modal-movimiento-caja-backdrop');
9063|     const btnClose = document.getElementById('modal-movimiento-caja-close');
9064|     const btnCancel = document.getElementById('modal-movimiento-caja-cancelar');
9065|     const form = document.getElementById('form-movimiento-caja');
9066|     const btnNuevo = document.getElementById('btn-nuevo-movimiento-caja');
9067|     if (btnClose) btnClose.addEventListener('click', closeModalMovimientoCaja);
9068|     if (btnCancel) btnCancel.addEventListener('click', closeModalMovimientoCaja);
9069|     if (backdrop) setupBackdropCloseOnlyOnRealClick(backdrop, closeModalMovimientoCaja);
9070|     if (form) form.addEventListener('submit', (e) => { e.preventDefault(); saveMovimientoCaja(); });
9071|     if (btnNuevo) btnNuevo.addEventListener('click', () => openModalMovimientoCaja(null));
9072|     setupInputImporte(document.getElementById('mov-caja-monto'));
9073| }
9074|
9075| function setupModalTipoMovimientoCaja() {
9076|     const backdrop = document.getElementById('modal-tipo-movimiento-backdrop');
9077|     const btnClose = document.getElementById('modal-tipo-movimiento-close');
9078|     const btnCancel = document.getElementById('modal-tipo-movimiento-cancelar');
9079|     const form = document.getElementById('form-tipo-movimiento');
9080|     const btnNuevo = document.getElementById('btn-nuevo-tipo-movimiento');
9081|     if (btnClose) btnClose.addEventListener('click', closeModalTipoMovimientoCaja);
9082|     if (btnCancel) btnCancel.addEventListener('click', closeModalTipoMovimientoCaja);
9083|     if (backdrop) setupBackdropCloseOnlyOnRealClick(backdrop, closeModalTipoMovimientoCaja);
9084|     if (form) form.addEventListener('submit', (e) => { e.preventDefault(); saveTipoMovimientoCaja(); });
9085|     if (btnNuevo) btnNuevo.addEventListener('click', () => openModalTipoMovimientoCaja(null));
9086| }
9087|
9088| // --- Órdenes ---
9089| let ordenesVistaList = [];
9090| let ordenesVistaClientesMap = {};
9091| let ordenesVistaTiposOpMap = {};
9092| let ordenesVistaIntermediariosMap = {};
9093| let ordenesFiltrosListenersAttached = false;
9094|
9095| /** tiposMap[id] = { codigo, nombre, icono_modo?, icono_url_publica?, usa_intermediario? } o legacy string (solo código). */
9096| function htmlCeldaTipoOperacionDesdeMap(tipoOpId, tiposMap) {
9097|     const t = tipoOpId ? tiposMap[tipoOpId] : null;
9098|     if (t === null) return htmlTipoOperacionIconos('');
9099|     if (typeof t === 'string') return htmlTipoOperacionIconos(t);
9100|     return htmlTipoOperacionIconos(t.codigo || '', t.nombre || '', { iconoModo: t.icono_modo, iconoUrlPublica: t.icono_url_publica, usaIntermediario: t.usa_intermediario === true
9101| e });
9102| }
9103|
9104| function clearOrdenTipoOperacionListBoxFixedStyles(list) {
9105|     if (!list) return;
9106|     list.style.position = '';
9107|     list.style.left = '';
9108|     list.style.top = '';
9109|     list.style.right = '';
9110|     list.style.width = '';
9111|     list.style.maxHeight = '';
9112|     list.style.zIndex = '';

```



```

9112 | }
9113 |
9114 | /** Evita que el listBox quede recortado por overflow del modal (.modal-body / .modal). */
9115 | function positionOrdenTipoOperacionListBox() {
9116 |     const btn = document.getElementById('orden-tipo-operacion-combo-btn');
9117 |     const list = document.getElementById('orden-tipo-operacion-listbox');
9118 |     if (!btn || !list || !list.hidden) return;
9119 |     const r = btn.getBoundingClientRect();
9120 |     const gap = 4;
9121 |     const margin = 8;
9122 |     const vw = window.innerWidth;
9123 |     const vh = window.innerHeight;
9124 |     let left = r.left;
9125 |     let width = r.width;
9126 |     if (left + width > vw - margin) {
9127 |         width = Math.max(200, vw - margin * 2);
9128 |         left = Math.min(left, vw - margin - width);
9129 |     }
9130 |     left = Math.max(margin, left);
9131 |     const spaceBelow = vh - r.bottom - gap - margin;
9132 |     const maxH = Math.min(280, Math.max(100, spaceBelow));
9133 |     list.style.position = 'fixed';
9134 |     list.style.left = `${left}px`;
9135 |     list.style.top = `${r.bottom + gap}px`;
9136 |     list.style.width = `${width}px`;
9137 |     list.style.maxHeight = `${maxH}px`;
9138 |     list.style.right = 'auto';
9139 |     list.style.zIndex = '10050';
9140 | }
9141 |
9142 | function closeOrdenTipoOperacionListBox() {
9143 |     const list = document.getElementById('orden-tipo-operacion-listbox');
9144 |     const btn = document.getElementById('orden-tipo-operacion-combo-btn');
9145 |     if (list) {
9146 |         list.hidden = true;
9147 |         list.classList.remove('is-open');
9148 |         clearOrdenTipoOperacionListBoxFixedStyles(list);
9149 |     }
9150 |     if (btn) btn.setAttribute('aria-expanded', 'false');
9151 | }
9152 |
9153 | /** Rellena el listBox custom desde las <option> del select nativo (mantiene data-codigo para E2E). */
9154 | function rebuildOrdenTipoOperacionListBox() {
9155 |     const sel = document.getElementById('orden-tipo-operacion');
9156 |     const list = document.getElementById('orden-tipo-operacion-listbox');
9157 |     if (!sel || !list) return;
9158 |     const current = sel.value || '';
9159 |     const opts = Array.from(sel.querySelectorAll('option'));
9160 |     list.innerHTML = opts
9161 |         .map((opt) => {
9162 |             const v = opt.value !== null ? String(opt.value) : '';
9163 |             const codigo = (opt.getAttribute('data-codigo') || '').trim();
9164 |             const nombrePlano = (opt.textContent || '').trim();
9165 |             const nombreBase = (opt.getAttribute('data-nombre-base') || '').trim() || nombrePlano.replace(/s+\.s*Int\.s*$/i, '').trim();
9166 |             const modoRaw = (opt.getAttribute('data-icone-modo') || 'auto');
9167 |             const modo = modoRaw === 'cheque' || modoRaw === 'custom' ? modoRaw : 'auto';
9168 |             const url = (opt.getAttribute('data-icone-url') || '');
9169 |             const usaInt = opt.getAttribute('data-usa-intermediario') === 'true';
9170 |             const escV = escapeHtml(v).replace(/"/g, '&quot;');
9171 |             const selected = v === current ? 'true' : 'false';
9172 |             if (!v) {
9173 |                 return '<li role="presentation"><button type="button" class="orden-tipo-operacion-option" role="option" data-value="" aria-selected="${selected}"><span class="orden-ti
po-operacion-placeholder">Elegir...</span></button></li>';
9174 |             }
9175 |             const iconsHtml = codigo
9176 |                 ? htmlTipoOperacionIconos(codigo, nombreBase || nombrePlano, { iconoModo: modo, iconoUrlPublica: url, usaIntermediario: usaInt })
9177 |                 : '<span class="tipo-op-iconos">${escapeHtml(nombreBase || nombrePlano)}</span>';
9178 |             const labelHtml = htmlOrdenTipoOperacionEtiquetaVisible(nombreBase, usaInt);
9179 |             return '<li role="presentation"><button type="button" class="orden-tipo-operacion-option" role="option" data-value="${escV}" aria-selected="${selected}">${iconsHtml}<spa
n class="orden-tipo-operacion-option-label">${labelHtml}</span></button></li>';
9180 |         })
9181 |         .join('');
9182 | }
9183 |
9184 | function syncOrdenTipoOperacionIconosPreview() {
9185 |     const sel = document.getElementById('orden-tipo-operacion');
9186 |     const display = document.getElementById('orden-tipo-operacion-combo-display');
9187 |     if (!display) return;
9188 |     if (!sel || !sel.value) {
9189 |         display.innerHTML = '<span class="orden-tipo-operacion-placeholder">Elegir...</span>';
9190 |         return;
9191 |     }
9192 |     const opt = sel.selectedOptions[0] || sel.selectedOptions[0];
9193 |     const codigo = opt ? (opt.getAttribute('data-codigo') || '').trim() : '';
9194 |     const nombrePlano = opt && opt.textContent ? opt.textContent.trim() : '';
9195 |     const nombreBaseAttr = opt ? (opt.getAttribute('data-nombre-base') || '').trim() : '';
9196 |     const usaIntSel = opt && opt.getAttribute('data-usa-intermediario') === 'true';
9197 |     let base = nombreBaseAttr;
9198 |     if (!base) base = usaIntSel ? nombrePlano.replace(/s+\.s*Int\.s*$/i, '').trim() : nombrePlano;
9199 |     const modoRaw = (opt && opt.getAttribute('data-icone-modo') || 'auto');
9200 |     const modo = modoRaw === 'cheque' || modoRaw === 'custom' ? modoRaw : 'auto';
9201 |     const url = (opt && opt.getAttribute('data-icone-url') || '');
9202 |     const usaInt = !usaIntSel;
9203 |     const ic = codigo ? htmlTipoOperacionIconos(codigo, base || nombrePlano, { iconoModo: modo, iconoUrlPublica: url, usaIntermediario: usaInt }) : '';
9204 |     const nombreHtml = htmlOrdenTipoOperacionEtiquetaVisible(base, usaInt);
9205 |     display.innerHTML = '<span class="orden-tipo-operacion-combo-display-inner">${ic}<span class="orden-tipo-operacion-combo-nombre">${nombreHtml}</span></span>';
9206 | }

```

```

9207 | function clearOrdenOfflineTipoOperacionListboxFixedStyles(list) {
9208 |     if (!list) return;
9209 |     list.style.position = '';
9210 |     list.style.left = '';
9211 |     list.style.top = '';
9212 |     list.style.right = '';
9213 |     list.style.width = '';
9214 |     list.style.maxHeight = '';
9215 |     list.style.zIndex = '';
9216 | }
9217 |
9218 | function positionOrdenOfflineTipoOperacionListbox() {
9219 |     const btn = document.getElementById('orden-offline-tipo-combo-btn');
9220 |     const list = document.getElementById('orden-offline-tipo-listbox');
9221 |     if (!btn || !list || list.hidden) return;
9222 |     const r = btn.getBoundingClientRect();
9223 |     const gap = 4;
9224 |     const margin = 8;
9225 |     const vw = window.innerWidth;
9226 |     const vh = window.innerHeight;
9227 |     let left = r.left;
9228 |     let width = r.width;
9229 |     if (left + width > vw - margin) {
9230 |         width = Math.max(200, vw - margin * 2);
9231 |         left = Math.min(left, vw - margin - width);
9232 |     }
9233 |     left = Math.max(margin, left);
9234 |     const spaceBelow = vh - r.bottom - gap - margin;
9235 |     const maxH = Math.min(280, Math.max(100, spaceBelow));
9236 |     list.style.position = 'fixed';
9237 |     list.style.left = `${left}px`;
9238 |     list.style.top = `${r.bottom + gap}px`;
9239 |     list.style.width = `${width}px`;
9240 |     list.style.maxHeight = `${maxH}px`;
9241 |     list.style.right = 'auto';
9242 |     list.style.zIndex = '10050';
9243 | }
9244 |
9245 | function closeOrdenOfflineTipoOperacionListbox() {
9246 |     const list = document.getElementById('orden-offline-tipo-listbox');
9247 |     const btn = document.getElementById('orden-offline-tipo-combo-btn');
9248 |     if (list) {
9249 |         list.hidden = true;
9250 |         list.classList.remove('is-open');
9251 |         clearOrdenOfflineTipoOperacionListboxFixedStyles(list);
9252 |     }
9253 |     if (btn) btn.setAttribute('aria-expanded', 'false');
9254 | }
9255 |
9256 | function rebuildOrdenOfflineTipoOperacionListbox() {
9257 |     const sel = document.getElementById('orden-offline-tipo');
9258 |     const list = document.getElementById('orden-offline-tipo-listbox');
9259 |     if (!sel || !list) return;
9260 |     const current = sel.value || '';
9261 |     const opts = Array.from(sel.querySelectorAll('option'));
9262 |     list.innerHTML = opts
9263 |         .map((opt) => {
9264 |             const v = opt.value !== null ? String(opt.value) : '';
9265 |             const codigo = (opt.getAttribute('data-codigo') || '').trim();
9266 |             const nombrePlano = (opt.textContent || '').trim();
9267 |             const nombreBase = (opt.getAttribute('data-nombre-base') || '').trim() || nombrePlano.replace(/s+~s*Int\\.s*$/i, '').trim();
9268 |             const modoRaw = (opt.getAttribute('data-icone-modo')) || 'auto';
9269 |             const modo = modoRaw === 'cheque' || modoRaw === 'custom' ? modoRaw : 'auto';
9270 |             const url = (opt.getAttribute('data-icone-url')) || '';
9271 |             const usaInt = opt.getAttribute('data-usa-intermediario') === 'true';
9272 |             const escV = escapeHtml(v).replace(/"/g, '&quot;');
9273 |             const selected = v === current ? 'true' : 'false';
9274 |             if (!v) {
9275 |                 return `<li role="presentation"><button type="button" class="orden-tipo-operacion-option" role="option" data-value="" aria-selected="${selected}"><span class="orden-ti
9276 | po-operacion-placeholder">Elegir...</span></button></li>`;
9277 |             }
9278 |             const iconsHtml = codigo
9279 |                 ? htmlTipoOperacionIconos(codigo, nombreBase || nombrePlano, { iconoModo: modo, iconoUrlPublica: url, usaIntermediario: usaInt })
9280 |                 : `<span class="tipo-op-icone">${escapeHtml(nombreBase || nombrePlano)}</span>`;
9281 |             const labelHtml = htmlOrdenTipoOperacionEtiquetaVisible(nombreBase, usaInt);
9282 |             return `<li role="presentation"><button type="button" class="orden-tipo-operacion-option" role="option" data-value="${escV}" aria-selected="${selected}">${iconsHtml}<spa
n class="orden-tipo-operacion-option-label">${labelHtml}</span></button></li>`;
9283 |         })
9284 |         .join('');
9285 | }
9286 |
9287 | function syncOrdenOfflineTipoOperacionIconosPreview() {
9288 |     const sel = document.getElementById('orden-offline-tipo');
9289 |     const display = document.getElementById('orden-offline-tipo-combo-display');
9290 |     if (!display) return;
9291 |     if (!sel || !sel.value) {
9292 |         display.innerHTML = `<span class="orden-tipo-operacion-placeholder">Elegir...</span>`;
9293 |         return;
9294 |     }
9295 |     const opt = sel.selectedOptions && sel.selectedOptions[0];
9296 |     const codigo = opt ? (opt.getAttribute('data-codigo') || '').trim() : '';
9297 |     const nombrePlano = opt && opt.textContent ? opt.textContent.trim() : '';
9298 |     const nombreBaseAttr = opt ? (opt.getAttribute('data-nombre-base') || '').trim() : '';
9299 |     const usaIntSel = opt && opt.getAttribute('data-usa-intermediario') === 'true';
9300 |     let base = nombreBaseAttr;
9301 |     if (!base) base = usaIntSel ? nombrePlano.replace(/s+~s*Int\\.s*$/i, '').trim() : nombrePlano;

```

```

9300 const modoRow = (opt && opt.getAttribute('data-icono-modo')) || 'auto';
9301 const modo = modoRow === 'cheque' || modoRow === 'custom' ? modoRow : 'auto';
9302 const url = (opt && opt.getAttribute('data-icono-url')) || '';
9303 const usaInt = !usaIntSel;
9304 const ic = codigo ? htmlTipoOperacionIconos[codigo, base || nombrePlano, { iconoModo: modo, iconoUrlPublica: url, usaIntermediario: usaInt }] : '';
9305 const nombreHtml = htmlOrdenTipoOperacionEtiquetaVisible(base, usaInt);
9306 display.innerHTML = `<span class="orden-tipo-operacion-combo-display-inner">${ic}<span class="orden-tipo-operacion-combo-nombre">${nombreHtml}</span></span>`;
9307 }
9308
9309 function renderOrdenesTabla(list) {
9310   const tbody = document.getElementById('ordenes-tbody');
9311   const wrapEl = document.getElementById('ordenes-tabla-wrap');
9312   if (!tbody || !wrapEl) return;
9313   const canEditarOrden = userPermissions.includes('editar_orden');
9314   const canAnularOrden = userPermissions.includes('anular_orden');
9315   const canIngresarTransacciones = userPermissions.includes('ingresar_transacciones');
9316   const canEditarTransacciones = userPermissions.includes('editar_transacciones');
9317   const canEliminarTransacciones = userPermissions.includes('eliminar_transacciones');
9318   const canVerAccionesOrden = canEditarOrden || canAnularOrden || userPermissions.includes('editar_estado_orden') || canIngresarTransacciones || canEditarTransacciones || canEliminarTransacciones;
9319   const clientesMap = ordenesVistaClientesMap;
9320   const tiposOpMap = ordenesVistaTiposOpMap;
9321   const intermediariosMap = ordenesVistaIntermediariosMap;
9322   const estadoLabel = (e) => ({ pendienteInstrumentar: 'Pendiente Instrumentar', instrumentacionParcial: 'Instrumentación Parcial', instrumentacionCerradaEjecucion: 'Cerrada en Ejecución', ordenEjecutada: 'Orden Ejecutada', anulada: 'Anulada', cotizacion: 'Cotización', concertada: 'Concertada' })[e] || (e ? String(e) : '-');
9323   const estadoBadgeClass = (e) => (e && 'pendienteInstrumentar', 'instrumentacionParcial', 'instrumentacionCerradaEjecucion', 'ordenEjecutada', 'anulada', 'cotizacion', 'concertada').includes(e) ? `badge badge-${e.replace(/_/g, '-')}` : '';
9324   if (list.length === 0) {
9325     tbody.innerHTML = `<tr><td colspan="11">No hay órdenes con los filtros aplicados.</td></tr>`;
9326     wrapEl.style.display = 'block';
9327     return;
9328   }
9329   tbody.innerHTML = list
9330     .map(
9331       (o) => {
9332         const estado = o.estado || '';
9333         const badgeClass = estadoBadgeClass(estado);
9334         const estadoHtml = badgeClass ? `<span class="${badgeClass}">${estadoLabel(estado)}</span>` : estadoLabel(estado);
9335         const peligroAnular = estado === 'ordenEjecutada';
9336         const clsAnularTabla = peligroAnular ? 'btn-anular-orden-tabla btn-anular-orden-tabla-peligro' : 'btn-secondary btn-anular-orden-tabla';
9337         const titAnularTabla = peligroAnular ? 'Anular orden ejecutada (acción grave)' : 'Anular orden';
9338         const btnAnularTabla =
9339           canAnularOrden && estado !== 'anulada'
9340             ? `<button type="button" class="${clsAnularTabla}" btn-icon-only data-id="${o.id}" title="${titAnularTabla}" aria-label="${titAnularTabla}" style="margin-left:0.25rem;"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="12" cy="12" r="10"/></span></button>`
9341             : `<span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="12" cy="12" r="10"/></span>`;
9342         const puedeEditarEstaOrden = canEditarOrden && estado !== 'anulada';
9343         const toolbarTransaccionesPanel =
9344           estado === 'anulada'
9345             ? `<p class="orden-detalle-transacciones-solo-info" style="margin:0 0.75rem;font-size:0.9rem;color:#64748b;">Transacciones solo lectura: la orden está anulada y no puede modificarse.</p>`
9346             : `<div class="vista-toolbar" style="margin-bottom:0.75rem;">
9347               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="12" y1="5" x2="12" y2="19"/></span></button>
9348               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9349               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9350               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9351               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9352               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9353               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9354               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9355               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9356               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9357               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9358               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9359               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9360               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9361               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9362               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9363               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9364               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9365               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9366               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9367               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9368               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9369               <button type="button" class="btn-nuevo btn-nueva-transaccion-panel" data-orden-id="${o.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="5" y1="12" x2="19" y2="12"/></span></button>
9370               <button type="button" class="btn-nuevo btn-nueva-transaccion
```

```

9386 | });
9387 | tbody.querySelectorAll('.btn-transacciones').forEach((btn) => {
9388 |   btn.addEventListener('click', () => {
9389 |     const id = btn.getAttribute('data-id');
9390 |     if (id) expandOrdenTransacciones(id, list.find((r) => r.id === id));
9391 |   });
9392 | });
9393 | tbody.querySelectorAll('.btn-anular-orden-tabla').forEach((btn) => {
9394 |   btn.addEventListener('click', () => {
9395 |     const id = btn.getAttribute('data-id');
9396 |     if (id) solicitarConfirmacionYAnularOrden(id);
9397 |   });
9398 | });
9399 | wrapEl.style.display = 'block';
9400 | }
9401 |
9402 | function aplicarFiltrosOrdenesVista() {
9403 |   const selCliente = document.getElementById('ordenes-filtro-cliente');
9404 |   const selIntermediario = document.getElementById('ordenes-filtro-intermediario');
9405 |   const selEstado = document.getElementById('ordenes-filtro-estado');
9406 |   const clienteId = selCliente && selCliente.value ? selCliente.value.trim() : '';
9407 |   const intermediarioId = selIntermediario && selIntermediario.value ? selIntermediario.value.trim() : '';
9408 |   const estadoRaw = selEstado && selEstado.value != null ? String(selEstado.value).trim() : '__activas__';
9409 |   const filtered = ordenesVistaList.filter((o) => {
9410 |     if (clienteId && o.cliente_id !== clienteId) return false;
9411 |     if (intermediarioId && o.intermediario_id !== intermediarioId) return false;
9412 |     if (estadoRaw === 'activas') {
9413 |       const es = (o.estado || '').toString();
9414 |       if (es === 'orden_ejecutada' || es === 'anulada') return false;
9415 |     } else if (estadoRaw !== '' && (o.estado || '') !== estadoRaw) return false;
9416 |     return true;
9417 |   });
9418 |   renderOrdenesTabla(filtered);
9419 | }
9420 |
9421 | /** Mensaje en la grilla cuando falla el SELECT de órdenes (sin mostrar TypeError crudo en pantalla). */
9422 | function pandiHtmlTablaOrdenesMensajeError(msgRaw, silentOrd) {
9423 |   if (silentOrd) return '<tr><td colspan="11"></td></tr>';
9424 |   const msg = String(msgRaw || '');
9425 |   const esRed =
9426 |     /failed to fetch|networkerror|fetcherror|err_network|timeout|load failed|aborted|network request failed/i.test(
9427 |       msg.toLowerCase(),
9428 |     );
9429 |   const nCola = pandiOfflineQueueRead().length;
9430 |   if (esRed) {
9431 |     let linea =
9432 |       'No hay conexión con el servidor: acá solo se listan órdenes ya guardadas en Supabase.';
9433 |     if (nCola > 0) {
9434 |       linea +=
9435 |         'La cola local (' +
9436 |         nCola +
9437 |         ' orden(es)) no aparece en esta tabla hasta importarla: abrí «Orden en cola local» para ver el resumen o sumar borradores; cuando vuelva el servicio usá «Enviar cola l
9438 | ocal» o el aviso de importación.';
9439 |     } else {
9440 |       linea += ' Cuando vuelva la conexión, tocá Reintentar en el aviso superior o esperá el chequeo automático.';
9441 |     }
9442 |     return '<tr><td colspan="11" style="padding:1rem 0.75rem;line-height:1.5;">' + escapeHtml(linea) + '</td></tr>';
9443 |   }
9444 |   return '<tr><td colspan="11">Error: ' + escapeHtml(msg) + '</td></tr>';
9445 | }
9446 |
9447 | function loadOrdenes() {
9448 |   // No recargar la lista mientras el modal de orden está abierto: evita que la vista atrás cargue y el modal pierda el foco.
9449 |   if (document.getElementById('modal-orden-backdrop')?.classList?.contains('activo')) return Promise.resolve();
9450 |   const loadingEl = document.getElementById('ordenes-loading');
9451 |   const wrapEl = document.getElementById('ordenes-tabla-wrap');
9452 |   const tbody = document.getElementById('ordenes-tbody');
9453 |   const filtrosWrap = document.getElementById('ordenes-filtros-wrap');
9454 |   if (!loadingEl || !wrapEl || !tbody) return Promise.resolve();
9455 |   const canIngresarOrden = userPermissions.includes('ingresar_orden');
9456 |   const canEditarOrden = userPermissions.includes('editar_orden');
9457 |   pandiUpdateOfflineToolbarButtons();
9458 |
9459 |   const silentOrd = isPandiBackgroundRefresh();
9460 |   const loadingShownAtOrdenes = silentOrd ? 0 : Date.now();
9461 |   if (!silentOrd) {
9462 |     loadingEl.style.display = 'block';
9463 |     if (filtrosWrap) filtrosWrap.style.display = 'none';
9464 |     wrapEl.style.display = 'none';
9465 |     tbody.innerHTML = '';
9466 |   }
9467 |
9468 |   const selectBase = 'id, cliente_id, fecha, estado, tipo_operacion_id, operacion_directa, intermediario_id, moneda_recibida, moneda_entregada, monto_recibido, monto_entregado
9469 | , cotizacion, tasa_descuento_intermediario, observaciones';
9470 |   const selectConNumero = 'id, numero, cliente_id, fecha, estado, tipo_operacion_id, operacion_directa, intermediario_id, moneda_recibida, moneda_entregada, monto_recibido, mo
9471 | nto_entregado, cotizacion, tasa_descuento_intermediario, observaciones';
9472 |   function runLoadOrdenes(selectCols) {
9473 |     return client
9474 |       .from('ordenes')
9475 |       .select(selectCols)
9476 |       .order('fecha', { ascending: false })
9477 |       .order('created at', { ascending: false })
9478 |       .then((res) => {
9479 |         if (res.error) {

```

```

9480 | if ((msg.includes('numero') || msg.includes('does not exist')) && selectCols === selectConNumero) {
9481 |   ordenesTieneNumeroColumn = false;
9482 |   return runLoadOrdenes(selectBase);
9483 | }
9484 | return delayMinLoadingSiNoEsBackground(loadingShownAtOrdenes).then(() => {
9485 |   loadingEl.style.display = 'none';
9486 |   if (silentOrd) {
9487 |     showToast('Error al actualizar órdenes: ' + (msg || ''), 'error');
9488 |     return;
9489 |   }
9490 |   const esRed =
9491 |     /failed to fetch|networkerror|fetcherror|err_network|timeout|load failed|aborted|network request failed/i.test(
9492 |       String(msg || '').toLowerCase(),
9493 |     );
9494 |   if (esRed) {
9495 |     ordenesVistaList = [];
9496 |     ordenesVistaClientesMap = {};
9497 |     ordenesVistaTiposOpMap = {};
9498 |     ordenesVistaIntermediariosMap = {};
9499 |   }
9500 |   tbody.innerHTML = pandiHtmlTablaOrdenesMensajeError(msg, silentOrd);
9501 |   wrapEl.style.display = 'block';
9502 |   pandiUpdateOfflineToolBarButtons();
9503 | });
9504 |
9505 | const list = res.data || [];
9506 | const clienteIds = [...new Set(list.map((o) => o.cliente_id).filter(Boolean))];
9507 | const tipoOpIds = [...new Set(list.map(o => o.tipo_operacion_id).filter(Boolean))];
9508 | const intIds = [...new Set(list.map(o => o.intermediario_id).filter(Boolean))];
9509 | return Promise.all([
9510 |   client.from('clientes').select('id, nombre').eq('activo', true).order('nombre', { ascending: true }),
9511 |   tipoOpIds.length ? client.from('tipos_operacion').select('id, nombre, codigo, moneda_in, moneda_out, usa_intermediario, icono_mod, icono_url_publica').in('id', tipo
OpIds) : Promise.resolve({ data: [] }),
9512 |   client.from('intermediarios').select('id, nombre').eq('activo', true).order('nombre', { ascending: true }),
9513 | ]).then(([{crClientes, tr, crInt}] => {
9514 |   const clientesMap = {};
9515 |   (crClientes.data || []).forEach((c) => { clientesMap[c.id] = c.nombre || ''; });
9516 |   const tiposOpMap = {};
9517 |   (tr.data || []).forEach((t) => {
9518 |     tiposOpMap[t.id] = {
9519 |       codigo: t.codigo || '',
9520 |       nombre: t.nombre || '',
9521 |       moneda_in: (t.moneda_in || '').toString().trim(),
9522 |       moneda_out: (t.moneda_out || '').toString().trim(),
9523 |       usa_intermediario: t.usa_intermediario === true,
9524 |       icono_mod: (t.icono_mod || 'auto').toString().trim().toLowerCase(),
9525 |       icono_url_publica: (t.icono_url_publica || '').toString().trim(),
9526 |     };
9527 |   });
9528 |   const intermediariosMap = {};
9529 |   (crInt.data || []).forEach((i) => { intermediariosMap[i.id] = i.nombre || ''; });
9530 |   ordenesVistaList = list;
9531 |   ordenesVistaClientesMap = clientesMap;
9532 |   ordenesVistaTiposOpMap = tiposOpMap;
9533 |   ordenesVistaIntermediariosMap = intermediariosMap;
9534 |   pandiTrySaveOfflineCatalogosCache(crClientes.data || [], crInt.data || []);
9535 |
9536 |   const selCliente = document.getElementById('ordenes-filtro-cliente');
9537 |   const selIntermediario = document.getElementById('ordenes-filtro-intermediario');
9538 |   if (selCliente) {
9539 |     selCliente.innerHTML = '<option value="">Todos</option>' + (crClientes.data || []).map((c) => `<option value="${c.id}">${escapeHtml(c.nombre || '')}</option>`).join('');
9540 |   }
9541 |   if (selIntermediario) {
9542 |     selIntermediario.innerHTML = '<option value="">Todos</option>' + (crInt.data || []).map((i) => `<option value="${i.id}">${escapeHtml(i.nombre || '')}</option>`).join('');
9543 |   }
9544 |   if (filtrosWrap) filtrosWrap.style.display = 'flex';
9545 |   return delayMinLoadingSiNoEsBackground(loadingShownAtOrdenes).then(() => {
9546 |     loadingEl.style.display = 'none';
9547 |     if (!ordenesFiltrosListenersAttached) {
9548 |       const selC = document.getElementById('ordenes-filtro-cliente');
9549 |       const selI = document.getElementById('ordenes-filtro-intermediario');
9550 |       const selE = document.getElementById('ordenes-filtro-estado');
9551 |       if (selC) selC.addEventListener('change', aplicarFiltrosOrdenesVista);
9552 |       if (selI) selI.addEventListener('change', aplicarFiltrosOrdenesVista);
9553 |       if (selE) selE.addEventListener('change', aplicarFiltrosOrdenesVista);
9554 |       ordenesFiltrosListenersAttached = true;
9555 |     }
9556 |     aplicarFiltrosOrdenesVista();
9557 |     pandiUpdateOfflineToolBarButtons();
9558 |   });
9559 | });
9560 | });
9561 | }
9562 |
9563 | // Sincronizar CC/caja sin bloquear el listado: la grilla solo necesita `ordenes` + catálogos (rápido con pocas filas).
9564 | // Antes se encadenaba sync -> runLoadOrdenes; el sync recorre cada orden con instrumentación en serie ('sincronizarCcYcajaDesdeOrden')
9565 | // y domina el tiempo aunque haya 7 órdenes - no es un tema de índices en `ordenes`. CC y Cajas siguen llamando al mismo sync al entrar.
9566 | sincronizarCcYcajaTodasLasOrdenesConInstrumentacion().catch(() => {});
9567 | return runLoadOrdenes(selectConNumero).catch((err) => {
9568 |   const msg = err && err.message != null ? String(err.message) : String(err || '');
9569 |   ordenesVistaList = [];
9570 |   ordenesVistaClientesMap = {};
9571 |   ordenesVistaTiposOpMap = {};
9572 |   ordenesVistaIntermediariosMap = {};
9573 |   return delayMinLoadingSiNoEsBackground(loadingShownAtOrdenes).then(() => {

```

```

9574 |         loadingEl.style.display = 'none';
9575 |         if (silentOrd) {
9576 |             showToast('Error al actualizar órdenes: ' + (msg || ''), 'error');
9577 |         } else {
9578 |             tbody.innerHTML = pandiHtmlTablaOrdenesMensajeError(msg, silentOrd);
9579 |             wrapEl.style.display = 'block';
9580 |         }
9581 |         pandiUpdateOfflineToolBarButtons();
9582 |     });
9583 | });
9584 | }
9585 |
9586 | /** Crea una orden borrador (mínima) para "Nueva orden". Si el usuario cierra sin guardar, se elimina en closeModalOrden. */
9587 | function crearOrdenBorrador() {
9588 |     const fecha = fechaHoyYYYYMMDDArgentina();
9589 |     const payload = {
9590 |         fecha,
9591 |         estado: 'pendiente_instrumentar',
9592 |         moneda_recibida: 'USD',
9593 |         moneda_entregada: 'USD',
9594 |         monto_recibido: 0,
9595 |         monto_entregado: 0,
9596 |         usuario_id: currentUserId,
9597 |         updated_at: new Date().toISOString(),
9598 |     };
9599 |     const selectCols = ordenesTieneNumeroColumn ? 'id, numero, fecha, estado, moneda_recibida, moneda_entregada, monto_recibido, monto_entregado' : 'id, fecha, estado, moneda_re
9600 | cibida, moneda_entregada, monto_recibido, monto_entregado';
9601 |     return client.from('ordenes').insert(payload).select(selectCols).single().then((r) => {
9602 |         if (r.error || !r.data) return Promise.reject(new Error(r.error?.message || 'No se pudo crear la orden'));
9603 |         return r.data;
9604 |     });
9605 | }
9606 | function syncOrdenIntPatronInstrumentacionWrap() {
9607 |     const wrap = document.getElementById('orden-wrap-int-patron-instrumentacion');
9608 |     if (!wrap) return;
9609 |     const opt = document.getElementById('orden-tipo-operacion')?.selectedOptions?.[0];
9610 |     if (!opt) return;
9611 |     const codigo = opt.getAttribute('data-codigo') || '';
9612 |     const mi = opt.getAttribute('data-moneda-in') || '';
9613 |     const mo = opt.getAttribute('data-moneda-out') || '';
9614 |     const usaInt = opt.getAttribute('data-usa-intermediario') === 'true';
9615 |     const intVal = document.getElementById('orden-intermediario')?.value?.trim();
9616 |     const esCheque = esTipoOperacionChequeArs(codigo, mi, mo);
9617 |     const show = usaInt && !intVal && !esCheque;
9618 |     wrap.style.display = show ? 'block' : 'none';
9619 |     applyOrdenUsdIntPostPatronVisibility();
9620 | }
9621 |
9622 | function setupOrdenIntPatronRadiosOnce() {
9623 |     if (typeof document === 'undefined' || document.body.dataset.ordenIntPatronRadiosBound === '1') return;
9624 |     document.body.dataset.ordenIntPatronRadiosBound = '1';
9625 |     document.querySelectorAll('input[name="orden-int-patron-radio"]').forEach((radio) => {
9626 |         radio.addEventListener('change', () => {
9627 |             const hid = document.getElementById('orden-int-patron');
9628 |             if (hid && radio.checked) hid.value = radio.value;
9629 |             applyOrdenUsdIntPostPatronVisibility();
9630 |             syncOrdenWizardUsdIntComisionUi();
9631 |         });
9632 |     });
9633 | }
9634 |
9635 | function setupOrdenNachoComisionRadiosOnce() {
9636 |     if (typeof document === 'undefined' || document.body.dataset.ordenNachoComisionRadiosBound === '1') return;
9637 |     document.body.dataset.ordenNachoComisionRadiosBound = '1';
9638 |     document.querySelectorAll('input[name="orden-usd-nacho-comision-usd"]').forEach((radio) => {
9639 |         radio.addEventListener('change', () => {
9640 |             if (radio.checked) persistirPreferenciaComisionFijaInterUsd(radio.value);
9641 |             document.getElementById('orden-importe-cheque')?.dispatchEvent(new Event('input', { bubbles: true }));
9642 |         });
9643 |     });
9644 | }
9645 |
9646 | function resetOrdenIntPatronUi() {
9647 |     const hid = document.getElementById('orden-int-patron');
9648 |     const r0 = document.querySelector('input[name="orden-int-patron-radio"][value="cp_ic"]');
9649 |     if (r0) r0.checked = true;
9650 |     if (hid) hid.value = 'cp_ic';
9651 | }
9652 |
9653 | /** Nombre visible del intermediario elegido en el wizard de orden. */
9654 | function ordenWizardIntermediarioNombreVisible() {
9655 |     const sel = document.getElementById('orden-intermediario');
9656 |     if (!sel || !sel.value) return '';
9657 |     const opt = sel.selectedOptions && sel.selectedOptions[0];
9658 |     return opt ? String(opt.textContent || '').trim() : '';
9659 | }
9660 |
9661 | /**
9662 |  * USD-USD + int + cp_ic + intermediario con comisión fija (Seguridad o nombre «nacho» legacy): persiste split sin depender de esWizardUsdUsdConIntermediario.
9663 |  */
9664 | function ordenComisionFijaNachoUsdParaPersistir(tipoCodigoRaw, intermediarioId) {
9665 |     if (!intermediarioId) return false;
9666 |     if (String(tipoCodigoRaw || '').trim().toUpperCase() !== 'USD-USD') return false;
9667 |     const pat = String(document.getElementById('orden-int-patron')?.value || '').trim().toLowerCase();
9668 |     if (pat !== 'cp_ic') return false;
9669 |     return intermediarioTieneComisionFijaUsdAcordada(intermediarioId, ordenWizardIntermediarioNombreVisible());

```

```

9670 }
9671
9672 function esWizardUsdUsdConIntermediario() {
9673     const opt = document.getElementById('orden-tipo-operacion')?.selectedOptions?.[0];
9674     const codigo = String(opt?.getAttribute('data-codigo') || '').trim().toUpperCase();
9675     const usaInt = opt?.getAttribute('data-usa-intermediario') === 'true';
9676     const intVal = document.getElementById('orden-intermediario')?.value?.trim();
9677     return codigo === 'USD-USD' && usaInt && !!intVal;
9678 }
9679
9680 /** Tasa % intermediario junto a tasa cliente (USD-USD+int): ci_pc o cp_ic sin comisión fija parametrizada. */
9681 function ordenUsdIntMostrarTasaIntermediarioEnWizard() {
9682     if (!esWizardUsdUsdConIntermediario()) return false;
9683     const pat = String(document.getElementById('orden-int-patron')?.value || '').trim().toLowerCase();
9684     if (pat === 'ci_pc') return true;
9685     const intId = document.getElementById('orden-intermediario')?.value?.trim();
9686     const nom = ordenWizardIntermediarioNombreVisible();
9687     if (pat === 'cp_ic' && !intermediarioTieneComisionFijaUsdAcordada(intId, nom)) return true;
9688     return false;
9689 }
9690
9691 function ordenUsdIntMostrarComisionFijaNacho() {
9692     if (!esWizardUsdUsdConIntermediario()) return false;
9693     const pat = String(document.getElementById('orden-int-patron')?.value || '').trim().toLowerCase();
9694     const intId = document.getElementById('orden-intermediario')?.value?.trim();
9695     return pat === 'cp_ic' && intermediarioTieneComisionFijaUsdAcordada(intId, ordenWizardIntermediarioNombreVisible());
9696 }
9697
9698 function ordenIntPatronExplicitoElegido() {
9699     const v = String(document.getElementById('orden-int-patron')?.value || '').trim().toLowerCase();
9700     return v === 'cp_ic' || v === 'ci_pc';
9701 }
9702
9703 function toastElegirComisionFijaIntermediarioUsd() {
9704     const a = Number(usdUsdComisionFijaConfig.opcion_a) || 50;
9705     const b = Number(usdUsdComisionFijaConfig.opcion_b) || 75;
9706     showToast('Elegí la comisión fija del intermediario USD ' + formatImporteDisplay(a) + ' o USD ' + formatImporteDisplay(b) + ' ', 'error');
9707 }
9708
9709 /** Tras elegir patrón, muestra el resto del paso Detalles (USD-USD con intermediario, orden nueva). */
9710 function applyOrdenUsdIntPostPatronVisibility() {
9711     const wrapTras = document.getElementById('orden-wrap-detalles-tras-patron');
9712     if (!wrapTras) return;
9713     const ordenId = document.getElementById('orden-id')?.value?.trim();
9714     if (!esWizardUsdUsdConIntermediario()) {
9715         wrapTras.style.display = '';
9716         return;
9717     }
9718     const patronVal = String(document.getElementById('orden-int-patron')?.value || '').trim();
9719     if (ordenId || patronVal === 'cp_ic' || patronVal === 'ci_pc') {
9720         wrapTras.style.display = '';
9721     } else {
9722         wrapTras.style.display = 'none';
9723     }
9724 }
9725
9726 /** USD-USD + int orden nueva: obliga a elegir instrumentación sugerida antes del resto. */
9727 function resetOrdenIntPatronRequiereEleccionUsdInt() {
9728     const hid = document.getElementById('orden-int-patron');
9729     document.querySelectorAll('input[name="orden-int-patron-radio"]').forEach((r) => { r.checked = false; });
9730     if (hid) hid.value = '';
9731     document.querySelectorAll('input[name="orden-usd-nacho-comision-usd"]').forEach((r) => { r.checked = false; });
9732     applyOrdenUsdIntPostPatronVisibility();
9733 }
9734
9735 function syncOrdenWizardUsdUsdIntComisionUi() {
9736     aplicarValoresRadiosComisionFijaUsdWizard();
9737     const wrapNacho = document.getElementById('orden-wrap-usd-nacho-comision-fija');
9738     if (wrapNacho) {
9739         wrapNacho.style.display = ordenUsdIntMostrarComisionFijaNacho() ? 'block' : 'none';
9740     }
9741     const wrapTasa = document.getElementById('orden-wrap-tasa-descuento-intermediario');
9742     const opt = document.getElementById('orden-tipo-operacion')?.selectedOptions?.[0];
9743     const codigo = String(opt?.getAttribute('data-codigo') || '').trim().toUpperCase();
9744     if (wrapTasa && codigo === 'USD-USD') {
9745         if (esWizardUsdUsdConIntermediario() && ordenUsdIntMostrarTasaIntermediarioEnWizard()) wrapTasa.style.display = 'block';
9746         else if (esWizardUsdUsdConIntermediario()) wrapTasa.style.display = 'none';
9747     }
9748     const imp = document.getElementById('orden-importe-cheque');
9749     if (imp) imp.dispatchEvent(new Event('input', { bubbles: true }));
9750 }
9751
9752 /** Si ya hay 2 transacciones, alinea radios con patrón detectado (solo UI; no persiste en orden). */
9753 function syncPatronDesdeTransaccionesOrdenInt(ordenId) {
9754     if (!ordenId || !client) return Promise.resolve();
9755     return client.from('instrumentacion').select('id').eq('orden_id', ordenId).maybeSingle()
9756         .then((rInst) => {
9757             const instId = rInst.data && rInst.data.id;
9758             if (!instId) return Promise.resolve();
9759             return client.from('transacciones').select('tipo, pagador, cobrador, numero, created_at').eq('instrumentacion_id', instId).order('numero', { ascending: true }).order('created_at', { ascending: true });
9760         })
9761         .then((rTr) => {
9762             const list = (rTr && rTr.data) || [];
9763             if (list.length < 2) return;
9764             const t1 = list[0];
9765             const cob1 = String(t1.cobrador || '').toLowerCase();

```

```

9766 |     const esIngreso = (tl.tipo || '').toLowerCase() === 'ingreso';
9767 |     const hid = document.getElementById('orden-int-patron');
9768 |     const rCi = document.querySelector('input[name="orden-int-patron-radio"][value="ci_pc"]');
9769 |     const rCp = document.querySelector('input[name="orden-int-patron-radio"][value="cp_ic"]');
9770 |     if (esIngreso && cobl === 'intermediario') {
9771 |         if (hid) hid.value = 'ci_pc';
9772 |         if (rCi) rCi.checked = true;
9773 |     } else {
9774 |         if (hid) hid.value = 'cp_ic';
9775 |         if (rCp) rCp.checked = true;
9776 |     }
9777 |     applyOrdenUsdIntPostPatronVisibility();
9778 |     syncOrdenWizardUsdIntComisionUi();
9779 | })
9780 | .catch(() => {});
9781 | }
9782 |
9783 | function openModalOrden(registro) {
9784 |     const backdrop = document.getElementById('modal-orden-backdrop');
9785 |     const titulo = document.getElementById('modal-orden-titulo');
9786 |     const idEl = document.getElementById('orden-id');
9787 |     const form = document.getElementById('form-orden');
9788 |     const wizard = document.getElementById('orden-wizard');
9789 |     const stepParticipantes = document.getElementById('orden-step-participantes');
9790 |     const stepDetalles = document.getElementById('orden-step-detalles');
9791 |     const btnNext = document.getElementById('orden-btn-next');
9792 |     const btnBack = document.getElementById('orden-btn-back');
9793 |     const btnIrInst = document.getElementById('orden-btn-ir-instrumentacion');
9794 |     const btnBackDetalles = document.getElementById('orden-btn-back-detalles');
9795 |     const btnNuevaTr = document.getElementById('orden-btn-nueva-transaccion');
9796 |     if (!backdrop || !titulo || !idEl || !form) return;
9797 |     if (registro && String(registro.estado || '') === 'anulada') {
9798 |         showToast('La orden está anulada; no se puede editar.', 'info');
9799 |         return;
9800 |     }
9801 |     ordenModalLoadSeq += 1;
9802 |     const modalLoadSeq = ordenModalLoadSeq;
9803 |     if (registro == null) {
9804 |         idEl.value = '';
9805 |         ordenWizardOrdenIdActual = null;
9806 |         ordenWizardInstrumentacionIdActual = null;
9807 |         form.reset();
9808 |         document.querySelectorAll('input[name="orden-int-patron-radio"]').forEach((r) => { r.checked = false; });
9809 |         const hidPatEarly = document.getElementById('orden-int-patron');
9810 |         if (hidPatEarly) hidPatEarly.value = '';
9811 |         document.querySelectorAll('input[name="orden-usd-nacho-comision-usd"]').forEach((r) => { r.checked = false; });
9812 |         closeOrdenTipoOperacionListbox();
9813 |         titulo.textContent = 'Nueva orden';
9814 |         syncOrdenTipoOperacionIconosPreview();
9815 |         rebuildOrdenTipoOperacionListbox();
9816 |         applyOrdenUsdIntPostPatronVisibility();
9817 |     }
9818 |     const montoEntregadoInput = document.getElementById('orden-monto-entregado');
9819 |     if (montoEntregadoInput && !montoEntregadoInput.dataset.cursorInicio) {
9820 |         montoEntregadoInput.dataset.cursorInicio = '1';
9821 |         montoEntregadoInput.addEventListener('focus', () => {
9822 |             // Solo al inicio del flujo (valor vacío o cero): en móvil, forzar cursor 0,0 en cada focus
9823 |             // puede interferir con el teclado virtual o con edición parcial.
9824 |             const v = (montoEntregadoInput.value || '').trim().replace(/s/g, '');
9825 |             const esInicio =
9826 |                 v === '' || v === '0' || v === '0.00' || v === '0.00' || v === '0,0' || v === '0.';
9827 |             if (!esInicio) return;
9828 |             const ponerCursorAlInicio = () => {
9829 |                 try {
9830 |                     montoEntregadoInput.setSelectionRange(0, 0);
9831 |                 } catch (_) { /* noop */ }
9832 |             };
9833 |             requestAnimationFrame(() => {
9834 |                 ponerCursorAlInicio();
9835 |                 requestAnimationFrame(ponerCursorAlInicio);
9836 |             });
9837 |         });
9838 |     }
9839 | }
9840 |
9841 | const tiposOrdenCols = 'id, codigo, nombre, moneda_in, moneda_out, usa_intermediario, icono_moda, icono_url_publica';
9842 | const promDatos = Promise.all([
9843 |     client.from('clientes').select('id, nombre').eq('activo', true).order('nombre', { ascending: true }),
9844 |     tiposOperacionFetchConFallbackOrdenVisual(
9845 |         () => client.from('tipos_operacion').select(tiposOrdenCols + ', orden_visual').eq('activo', true).order('orden_visual', { ascending: true }).order('codigo').order('usa_i
9846 |         ntermediario').order('id'),
9847 |         () => client.from('tipos_operacion').select(tiposOrdenCols).eq('activo', true).order('codigo').order('usa_intermediario').order('id'),
9848 |     ),
9849 |     client.from('intermediarios').select('id, nombre').eq('activo', true).order('nombre', { ascending: true })
9850 | ]);
9851 | const promRegistro = registro
9852 |     ? Promise.resolve(registro)
9853 |     : Promise.resolve({
9854 |         fecha: fechaHoyYYYYMMDDArgentina(),
9855 |         estado: 'pendiente instrumentar',
9856 |         moneda_recibida: 'USD',
9857 |         moneda_entregada: 'USD',
9858 |         monto_recibido: 0,
9859 |         monto_entregado: 0,
9860 |     });
9861 |
9862 | Promise.all([promDatos, promRegistro])
9863 |     .then([[[rClientes, rTipos, rInt], registroActual]] => {

```



```

9862 |     if (modalLoadSeq !== ordenModalLoadSeq) return;
9863 |     const clientes = (rClientes.data || []);
9864 |     const tipos = ordenarTiposOperacionListaParaOrden(rTipos.data || []);
9865 |     const intermediarios = (rInt.data || []);
9866 |     if (rTipos.error) {
9867 |       showToast('Error al cargar tipos de operación: ' + (rTipos.error.message || ''), 'error');
9868 |     }
9869 |
9870 |     const selCliente = document.getElementById('orden-cliente');
9871 |     const selTipo = document.getElementById('orden-tipo-operacion');
9872 |     const selInt = document.getElementById('orden-intermediario');
9873 |     if (selCliente) selCliente.innerHTML = '<option value="">Sin asignar</option>' + clientes.map((c) => `<option value="${c.id}">${escapeHtml(c.nombre)}</option>`).join('');
9874 |     if (selTipo) {
9875 |       const escUrl = (s) => String(s || '').replace(/&g, '&amp;').replace(/"/g, '&quot;');
9876 |       const escAttr = (s) => String(s || '').replace(/&g, '&amp;').replace(/"/g, '&quot;');
9877 |       const im = (m) => (m === 'cheque' || m === 'custom' ? m : 'auto');
9878 |       selTipo.innerHTML = '<option value="">Elegir...</option>' + tipos.map((t) => {
9879 |         const modo = im((t.icono_modo || 'auto').toString().trim().toLowerCase());
9880 |         const baseNombre = t.nombre !== null ? String(t.nombre).trim() : '';
9881 |         const etiqueta = nombreTipoOperacionOrdenUi(t);
9882 |         return `<option value="${t.id}" data-nombre-base="${escAttr(baseNombre)}" data-codigo="${escapeHtml(t.codigo || '')}" data-icono-modo="${escapeHtml(modo)}" data-icono-
url="${escUrl(t.icono_url_publica || '')}" data-monedas-in="${escapeHtml((t.moneda_in || '').toUpperCase())}" data-monedas-out="${escapeHtml((t.moneda_out || '').toUpperCase())}"
" data-usa-intermediario="${t.usa_intermediario === true ? 'true' : 'false'}">${escapeHtml(etiqueta)}</option>`;
9883 |       }).join('');
9884 |     }
9885 |     syncOrdenTipoOperacionIconosPreview();
9886 |     rebuildOrdenTipoOperacionListbox();
9887 |     if (selInt) selInt.innerHTML = '<option value="">Sin asignar</option>' + intermediarios.map((i) => `<option value="${i.id}">${escapeHtml(i.nombre)}</option>`).join('');
9888 |
9889 |     const selTipoEl = document.getElementById('orden-tipo-operacion');
9890 |     const selIntEl = document.getElementById('orden-intermediario');
9891 |     const wrapIntermediario = document.getElementById('orden-wrap-intermediario');
9892 |     const wrapSplit = document.getElementById('orden-wrap-comision-split');
9893 |     const pctPandyEl = document.getElementById('orden-comision-pandy-pct');
9894 |     const pctIntEl = document.getElementById('orden-comision-intermediario-pct');
9895 |
9896 |     function showStep(which) {
9897 |       if (!wizard || !stepParticipantes || !stepDetalles) return;
9898 |       wizard.style.display = 'block';
9899 |       stepParticipantes.style.display = which === 'participantes' ? 'block' : 'none';
9900 |       stepDetalles.style.display = which === 'detalles' ? 'block' : 'none';
9901 |     }
9902 |
9903 |     function toggleComisionSplit() {
9904 |       if (!wrapSplit || !selTipoEl) return;
9905 |       const opt = selTipoEl.selectedOptions && selTipoEl.selectedOptions[0];
9906 |       const codigo = opt ? (opt.getAttribute('data-codigo') || '') : '';
9907 |       const tieneIntermediario = !(selIntEl && selIntEl.value.trim());
9908 |       wrapSplit.style.display = 'none';
9909 |       if (codigo === 'USD-USD') {
9910 |         if (!tieneIntermediario) {
9911 |           if (pctPandyEl) { pctPandyEl.value = '100'; pctPandyEl.disabled = true; }
9912 |           if (pctIntEl) { pctIntEl.value = '0'; pctIntEl.disabled = true; }
9913 |         } else {
9914 |           if (pctPandyEl) pctPandyEl.disabled = false;
9915 |           if (pctIntEl) pctIntEl.disabled = false;
9916 |         }
9917 |       }
9918 |       syncOrdenIntPatronInstrumentacionWrap();
9919 |     }
9920 |
9921 |     function syncComisionPctOtro(campoCambiado) {
9922 |       if (!pctPandyEl || !pctIntEl || wrapSplit?.style?.display === 'none') return;
9923 |       const p = Number(parseImporteInput(pctPandyEl.value));
9924 |       const i = Number(parseImporteInput(pctIntEl.value));
9925 |       const clamp = (n) => Math.max(0, Math.min(100, isNaN(n) ? 0 : n));
9926 |       if (campoCambiado === 'pandy') {
9927 |         const otro = clamp(100 - (isNaN(p) ? 0 : p));
9928 |         pctIntEl.value = formatImporteDisplay(otro);
9929 |       } else {
9930 |         const otro = clamp(100 - (isNaN(i) ? 0 : i));
9931 |         pctPandyEl.value = formatImporteDisplay(otro);
9932 |       }
9933 |     }
9934 |
9935 |     function onTipoChange() {
9936 |       const opt = selTipoEl && selTipoEl.selectedOptions && selTipoEl.selectedOptions[0];
9937 |       const codigo = opt ? (opt.getAttribute('data-codigo') || '') : '';
9938 |       const tipoId = selTipoEl ? (selTipoEl.value || '') : '';
9939 |       if (codigo) {
9940 |         if (wizard) wizard.style.display = 'block';
9941 |         if (selCliente) selCliente.disabled = false;
9942 |         adaptarFormularioOrden(codigo, tipos, tipoId);
9943 |         showStep('participantes');
9944 |         toggleComisionSplit();
9945 |       } else {
9946 |         if (wizard) wizard.style.display = 'none';
9947 |         if (selCliente) selCliente.disabled = true;
9948 |         if (wrapIntermediario) wrapIntermediario.style.display = 'none';
9949 |       }
9950 |       syncOrdenTipoOperacionIconosPreview();
9951 |       rebuildOrdenTipoOperacionListbox();
9952 |     }
9953 |     if (selTipoEl) selTipoEl.onchange = onTipoChange;
9954 |     if (selIntEl) {
9955 |       selIntEl.addEventListener('change', () => {
9956 |         toggleComisionSplit();

```

```

9957|         syncOrdenIntPatronInstrumentacionWrap();
9958|         syncOrdenWizardUsdUsdIntComisionUi();
9959|     });
9960| }
9961| if (pctPandyEl) pctPandyEl.addEventListener('change', () => syncComisionPctOtros('pandy'));
9962| if (pctPandyEl) pctPandyEl.addEventListener('input', () => syncComisionPctOtros('pandy'));
9963| if (pctIntEl) pctIntEl.addEventListener('change', () => syncComisionPctOtros('intermediario'));
9964| if (pctIntEl) pctIntEl.addEventListener('input', () => syncComisionPctOtros('intermediario'));
9965| if (btnNext) btnNext.onclick = () => {
9966|     const optTipo = document.getElementById('orden-tipo-operacion')?.selectedOptions?.[0];
9967|     const usaIntermediario = optTipo ? (optTipo.getAttribute('data-usa-intermediario') === 'true') : false;
9968|     const valorIntermediario = document.getElementById('orden-intermediario')?.value?.trim() || '';
9969|     if (usaIntermediario && !valorIntermediario) {
9970|         showToast('Para este tipo de operación es obligatorio elegir un intermediario.', 'error');
9971|         return;
9972|     }
9973|     showOrdenWizardStep('detalles');
9974|     const opt = optTipo;
9975|     const codigo = opt?.getAttribute('data-codigo') || '';
9976|     const esUsdUsdNext = codigo === 'USD-USD';
9977|     const intNext = document.getElementById('orden-intermediario')?.value?.trim() || '';
9978|     if (esUsdUsdNext && usaIntermediario && intNext && !document.getElementById('orden-id')?.value?.trim()) {
9979|         resetOrdenIntPatronRequiereEleccionUsdInt();
9980|     } else {
9981|         applyOrdenUsdIntPostPatronVisibility();
9982|     }
9983|     const miN = (opt?.getAttribute('data-monedas-in') || '').trim().toUpperCase();
9984|     const moN = (opt?.getAttribute('data-monedas-out') || '').trim().toUpperCase();
9985|     const patronNext = patronTipoCambioOrden(miN, moN);
9986|     if (esPatronCompraFiatConTc(patronNext)) {
9987|         setTimeout(() => {
9988|             const el = document.getElementById('orden-monto-entregado');
9989|             if (el) {
9990|                 const vz = (el.value || '').trim();
9991|                 if (vz === '0' || vz === '0,00' || vz === '0.00') el.value = '';
9992|                 el.focus();
9993|                 setTimeout(() => { el.setSelectionRange(0, 0); }, 0);
9994|                 el.dispatchEvent(new Event('input', { bubbles: true }));
9995|             }
9996|         }, 150);
9997|     } else if (esPatronVendeFiatConTc(patronNext)) {
9998|         setTimeout(() => {
9999|             const el = document.getElementById('orden-monto-recibido');
10000|             if (el) {
10001|                 const v = (el.value || '').trim();
10002|                 if (v === '0' || v === '0,00' || v === '0.00') el.value = '';
10003|                 el.focus();
10004|                 setTimeout(() => { el.setSelectionRange(0, 0); }, 0);
10005|                 el.dispatchEvent(new Event('input', { bubbles: true }));
10006|             }
10007|         }, 150);
10008|     } else if (esTipoOperacionChequeArs(codigo, opt?.getAttribute('data-monedas-in'), opt?.getAttribute('data-monedas-out')) || codigo === 'USD-USD') {
10009|         setTimeout(() => {
10010|             const el = document.getElementById('orden-importe-cheque');
10011|             if (el) {
10012|                 el.focus();
10013|                 el.classList.add('orden-field-editing');
10014|             }
10015|         }, 150);
10016|     }
10017| };
10018| if (btnBack) btnBack.onclick = () => showOrdenWizardStep('participantes');
10019| if (btnBackDetalles) btnBackDetalles.onclick = () => showOrdenWizardStep('detalles');
10020| // Listo/Cerrar del paso instrumentación se asignan en setupModalOrden para que respondan siempre
10021| if (btnIrInst) btnIrInst.onclick = () => {
10022|     const loadingInstEl = document.getElementById('orden-inst-loading');
10023|     const wrapInstEl = document.getElementById('orden-inst-tabla-wrap');
10024|     guardarOrdenDesdeWizard().then((ordenId) => {
10025|         if (!ordenId) return;
10026|         showOrdenWizardStep('instrumentacion');
10027|         if (loadingInstEl) {
10028|             loadingInstEl.textContent = 'Cargando instrumentación...';
10029|             loadingInstEl.style.display = 'block';
10030|         }
10031|         if (wrapInstEl) wrapInstEl.style.display = 'none';
10032|         ensureInstrumentacionForOrden(ordenId).then((instId) => {
10033|             if (!instId) return;
10034|             ordenWizardInstrumentacionIdActual = instId;
10035|             renderOrdenWizardInstrumentacion(instId);
10036|             if (btnNuevaTr) {
10037|                 const canIngresarTr = userPermissions.includes('ingresar_transacciones');
10038|                 btnNuevaTr.style.display = canIngresarTr ? '' : 'none';
10039|                 btnNuevaTr.onclick = () => openModalTransaccion(null, instId);
10040|             }
10041|         });
10042|     });
10043| };
10044|
10045| let promContinuar = Promise.resolve();
10046| if (registroActual && registroActual.id) {
10047|     titulo.textContent = registroActual.numero != null ? 'Editar orden #' + registroActual.numero : 'Editar orden';
10048|     idEl.value = registroActual.id;
10049|     document.getElementById('orden-cliente').value = registroActual.cliente_id || '';
10050|     document.getElementById('orden-fecha').value = (registroActual.fecha || '').toString().slice(0, 10);
10051|     document.getElementById('orden-tipo-operacion').value = registroActual.tipo_operacion_id || '';
10052|     document.getElementById('orden-intermediario').value = registroActual.intermediario_id || '';
10053|     document.getElementById('orden-monedas-recibida').value = registroActual.moneda_recibida || 'USD';

```

```

10054 | document.getElementById('orden-monto-recibido').value = formatImporteParaInput(registroActual.monto_recibido);
10055 | document.getElementById('orden-monedas-entregada').value = registroActual.monedas_entregada || 'USD';
10056 | document.getElementById('orden-monto-entregado').value = formatImporteParaInput(registroActual.monto_entregado);
10057 | document.getElementById('orden-cotizacion').value = formatImporteParaInput(registroActual.cotizacion);
10058 | const tasaIntEl = document.getElementById('orden-tasa-descuento-intermediario');
10059 | if (tasaIntEl) tasaIntEl.value = (registroActual.tasa_descuento_intermediario != null && Number(registroActual.tasa_descuento_intermediario) > 0) ? formatImporteDisplay(
Number(registroActual.tasa_descuento_intermediario) * 100) : '';
10060 | const mr = registroActual.monto_recibido != null ? Number(registroActual.monto_recibido) : null;
10061 | const me = registroActual.monto_entregado != null ? Number(registroActual.monto_entregado) : null;
10062 | const tipoRowReg = tipos.find((t) => t.id === registroActual.tipo_operacion_id);
10063 | const tipoCodigoReg = tipoRowReg?.codigo || '';
10064 | const esChequeArsReg = esTipoOperacionChequeArs(tipoCodigoReg, tipoRowReg?.moneda_in, tipoRowReg?.moneda_out);
10065 | if ((esChequeArsReg || tipoCodigoReg === 'USD-USD') && me != null && me > 0 && mr != null && mr > 0) {
10066 |     const importeChequeEl = document.getElementById('orden-importe-cheque');
10067 |     const tasaClienteEl = document.getElementById('orden-tasa-descuento-cliente');
10068 |     if (importeChequeEl) importeChequeEl.value = formatImporteDisplay(mr);
10069 |     if (tasaClienteEl) tasaClienteEl.value = formatImporteDisplay((1 - me / mr) * 100);
10070 | }
10071 | document.getElementById('orden-estado').value = (registroActual.estado && ['pendiente_instrumentar', 'instrumentacion_parcial', 'instrumentacion_cerrada_ejecucion', 'ord
en_ejecutada', 'anulada'].includes(registroActual.estado)) ? registroActual.estado : 'pendiente_instrumentar';
10072 | document.getElementById('orden-observaciones').value = registroActual.observaciones || '';
10073 | onTipoChange();
10074 | ordenWizardOrdenIdActual = registroActual.id;
10075 | const btnAnular = document.getElementById('orden-btn-anular');
10076 | if (btnAnular) {
10077 |     const puedeAnular = userPermissions.includes('anular_orden');
10078 |     const peligroModal = registroActual.estado === 'orden_ejecutada';
10079 |     if (registroActual.estado !== 'anulada' && puedeAnular) {
10080 |         btnAnular.style.display = '';
10081 |         btnAnular.classList.toggle('btn-anular-orden-peligro', peligroModal);
10082 |         btnAnular.classList.toggle('btn-secondary', !peligroModal);
10083 |         btnAnular.title = peligroModal ? 'Anular orden ejecutada (acción grave)' : 'Anular esta orden';
10084 |         btnAnular.setAttribute('aria-label', btnAnular.title);
10085 |         btnAnular.onclick = () => {
10086 |             solicitarConfirmacionYAnularOrden(registroActual.id, {
10087 |                 onExito: () => refrescarVistasTrasAnularOrden(true),
10088 |             });
10089 |         };
10090 |     } else {
10091 |         btnAnular.style.display = 'none';
10092 |         btnAnular.classList.remove('btn-anular-orden-peligro');
10093 |         btnAnular.classList.add('btn-secondary');
10094 |     }
10095 | }
10096 | promContinuar = client.from('comisiones_orden').select('beneficiario, monto').eq('orden_id', registroActual.id).then((rCom) => {
10097 |     const rows = rCom.data || [];
10098 |     let pandyMonto = 0, interMonto = 0;
10099 |     rows.forEach((row) => {
10100 |         if (row.beneficiario === 'pandy') pandyMonto += Number(row.monto) || 0;
10101 |         else if (row.beneficiario === 'intermediario') interMonto += Number(row.monto) || 0;
10102 |     });
10103 |     const total = pandyMonto + interMonto;
10104 |     if (total > 1e-6 && pctPandyEl && pctIntEl) {
10105 |         const pctP = (pandyMonto / total) * 100;
10106 |         const pctI = (interMonto / total) * 100;
10107 |         pctPandyEl.value = formatImporteDisplay(pctP);
10108 |         pctIntEl.value = formatImporteDisplay(pctI);
10109 |     }
10110 |     const tipoRowCom = tipos.find((t) => t.id === registroActual.tipo_operacion_id);
10111 |     const codCom = tipoRowCom?.codigo || '';
10112 |     if (codCom === 'USD-USD' && registroActual.intermediario_id && interMonto > 1e-6) {
10113 |         marcarRadioComisionFijaUsdSegunMontoGuardado(interMonto);
10114 |     }
10115 |     });
10116 | } else {
10117 |     titulo.textContent = registroActual && registroActual.numero != null ? 'Orden #' + registroActual.numero : 'Nueva orden';
10118 |     idEl.value = (registroActual && registroActual.id) ? registroActual.id : '';
10119 |     // Sin id es alta nueva: el stub siempre es truthy; hacia que nunca se ejecutara reset y quedaran datos de la orden anterior.
10120 |     if (!(registroActual && registroActual.id)) form.reset();
10121 |     document.getElementById('orden-fecha').value = (registroActual && registroActual.fecha) ? (registroActual.fecha || '').toString().slice(0, 10) : fechaHoyYYYYMMDDArgentina
a());
10122 | document.getElementById('orden-estado').value = 'pendiente_instrumentar';
10123 | if (registroActual) {
10124 |     document.getElementById('orden-monedas-recibida').value = registroActual.monedas_recibida || 'USD';
10125 |     document.getElementById('orden-monedas-entregada').value = registroActual.monedas_entregada || 'USD';
10126 |     document.getElementById('orden-monto-recibido').value = formatImporteParaInput(registroActual.monto_recibido);
10127 |     document.getElementById('orden-monto-entregado').value = formatImporteParaInput(registroActual.monto_entregado);
10128 | }
10129 | const esNuevaOrden = !registroActual || !registroActual.id;
10130 | if (wizard) wizard.style.display = esNuevaOrden ? 'none' : 'block';
10131 | if (selCliente) selCliente.disabled = esNuevaOrden;
10132 | if (esNuevaOrden && wrapIntermediario) wrapIntermediario.style.display = 'none';
10133 | ordenWizardOrdenIdActual = registroActual ? registroActual.id : null;
10134 | const btnAnular = document.getElementById('orden-btn-anular');
10135 | if (btnAnular) {
10136 |     btnAnular.style.display = 'none';
10137 |     btnAnular.classList.remove('btn-anular-orden-peligro');
10138 |     btnAnular.classList.add('btn-secondary');
10139 | }
10140 | }
10141 | promContinuar.then(() => {
10142 |     if (modalLoadSeq !== ordenModalLoadSeq) return;
10143 |     if (pctPandyEl && !pctPandyEl.value) pctPandyEl.value = '100';
10144 |     if (pctIntEl && !pctIntEl.value) pctIntEl.value = '0';
10145 |     setupOrdenIntPatronRadiosOnce();
10146 |     setupOrdenNachoComisionRadiosOnce();
10147 |     resetOrdenIntPatronUi();

```

```

10148 |     if (registroActual && registroActual.id) {
10149 |         syncPatronDesdeTransaccionesOrdenInt(registroActual.id).then(() => {
10150 |             if (modalLoadSeq !== ordenModalLoadSeq) return;
10151 |             syncOrdenIntPatronInstrumentacionWrap();
10152 |         });
10153 |     } else {
10154 |         syncOrdenIntPatronInstrumentacionWrap();
10155 |     }
10156 |     const inpParticipanteMarca = document.getElementById('orden-participante-nombre-marca');
10157 |     if (inpParticipanteMarca) inpParticipanteMarca.value = nombreMarcaSistema();
10158 |     backdrop.classList.add('activo');
10159 |     document.body.classList.add('modal-orden-abierto');
10160 |     showOrdenWizardStep('participantes');
10161 |     if (wizard && wizard.style.display === 'none') {
10162 |         setTimeout(() => { document.getElementById('orden-tipo-operacion-combo-btn')?.focus(); }, 120);
10163 |     }
10164 |     closeOrdenTipoOperacionListbox();
10165 |     rebuildOrdenTipoOperacionListbox();
10166 |     syncOrdenTipoOperacionIconosPreview();
10167 |     setupInputImporte(document.getElementById('orden-monto-recibido'));
10168 |     setupInputImporte(document.getElementById('orden-monto-entregado'));
10169 |     setupInputImporte(document.getElementById('orden-tasa-descuento-intermediario'), 2, true);
10170 |     setupInputImporte(document.getElementById('orden-importe-cheque'));
10171 |     setupInputImporte(document.getElementById('orden-tasa-descuento-cliente'), 2, true);
10172 |     setupInputImporte(document.getElementById('orden-comision-pandy-pct'));
10173 |     setupInputImporte(document.getElementById('orden-comision-intermediario-pct'));
10174 | });
10175 | });
10176 | .catch((err) => {
10177 |     if (modalLoadSeq !== ordenModalLoadSeq) return;
10178 |     if (!registro) showToast('Error al crear la orden: ' + (err && err.message ? err.message : ''), 'error');
10179 | });
10180 | }
10181 |
10182 | function adaptarFormularioOrden(codigo, tipos, tipoIdSeleccionado) {
10183 |     // Prioridad: moneda_in/moneda_out del tipo (tabla); fallback: parsear código (primera = recibida, segunda = entregada)
10184 |     let recibidaDesdeTipo = null;
10185 |     let entregadaDesdeTipo = null;
10186 |     const tipo = Array.isArray(tipos)
10187 |         ? (
10188 |             (tipoIdSeleccionado ? tipos.find((t) => String(t.id || '')) === String(tipoIdSeleccionado)) : null)
10189 |             || (codigo ? tipos.find((t) => (t.codigo || '')) === codigo) : null)
10190 |         : null;
10191 |     const usaIntermediario = tipo && tipo.usa_intermediario === true;
10192 |
10193 |     const wrapIntermediario = document.getElementById('orden-wrap-intermediario');
10194 |     const selIntermediario = document.getElementById('orden-intermediario');
10195 |     if (wrapIntermediario) wrapIntermediario.style.display = usaIntermediario ? 'block' : 'none';
10196 |     if (selIntermediario && !usaIntermediario) selIntermediario.value = '';
10197 |
10198 |     if (tipo && (tipo.moneda_in || tipo.moneda_out)) {
10199 |         recibidaDesdeTipo = (tipo.moneda_in || 'T').trim().toUpperCase() || null;
10200 |         entregadaDesdeTipo = (tipo.moneda_out || 'I').trim().toUpperCase() || null;
10201 |     }
10202 |
10203 |     if (recibidaDesdeTipo == null || entregadaDesdeTipo == null) {
10204 |         const partes = (codigo || '').split('-');
10205 |         const primera = (partes[0] || '').trim().toUpperCase();
10206 |         const segunda = (partes[1] || '').trim().toUpperCase();
10207 |         const normalizarMoneda = (s) => (s === 'DOLAR' ? 'USD' : s);
10208 |         if (recibidaDesdeTipo == null) recibidaDesdeTipo = primera && segunda ? normalizarMoneda(primera) : null;
10209 |         if (entregadaDesdeTipo == null) entregadaDesdeTipo = (segunda === 'CHEQUE' ? 'ARS' : (primera && segunda ? normalizarMoneda(segunda) : null));
10210 |     }
10211 |
10212 |     const recNorm = monedaCatalogoParaOrden(recibidaDesdeTipo);
10213 |     const entNorm = monedaCatalogoParaOrden(entregadaDesdeTipo);
10214 |     const isUsdUsd = codigo === 'USD-USD';
10215 |     const isArsArs = esTipoOperacionChequeArs(codigo, tipo?.moneda_in, tipo?.moneda_out);
10216 |     const patronTcForm = esPatronTipoCambioOrden(recNorm, entNorm);
10217 |     const isTcComprUsd = esPatronComprFiatConTc(patronTcForm);
10218 |     const isTcVendeUsd = esPatronVendeFiatConTc(patronTcForm);
10219 |     const isTipoConTc = !!(patronTcForm);
10220 |     const isTipoDosMonedas = !(recNorm && entNorm && recNorm !== entNorm);
10221 |     const monedaRecibida = document.getElementById('orden-moneda-recibida');
10222 |     const monedaEntregada = document.getElementById('orden-moneda-entregada');
10223 |     const labelMontoRecibido = document.getElementById('orden-label-monto-recibido');
10224 |     const labelMontoEntregado = document.getElementById('orden-label-monto-entregado');
10225 |     const wrapComision = document.getElementById('orden-wrap-comision');
10226 |     const wrapCotizacion = document.getElementById('orden-wrap-cotizacion');
10227 |     const labelCotizacion = document.getElementById('orden-label-cotizacion');
10228 |     const inputCotizacion = document.getElementById('orden-cotizacion');
10229 |     const comisionDisplay = document.getElementById('orden-comision-display');
10230 |     const estadoSelect = document.getElementById('orden-estado');
10231 |
10232 |     if (monedaRecibida) {
10233 |         if (recibidaDesdeTipo && ['USD', 'EUR', 'ARS', 'CHEQUE'].includes(recibidaDesdeTipo)) {
10234 |             monedaRecibida.value = monedaCatalogoParaOrden(recibidaDesdeTipo);
10235 |             monedaRecibida.disabled = true;
10236 |         } else {
10237 |             monedaRecibida.disabled = false;
10238 |         }
10239 |     }
10240 |
10241 |     if (monedaEntregada) {
10242 |         if (entregadaDesdeTipo && ['USD', 'EUR', 'ARS', 'CHEQUE'].includes(entregadaDesdeTipo)) {
10243 |             monedaEntregada.value = monedaCatalogoParaOrden(entregadaDesdeTipo);
10244 |             monedaEntregada.disabled = true;
10245 |         } else {

```

```

10245 |     monedaEntregada.disabled = false;
10246 | }
10247 | }
10248 | if (labelMontoRecibido) {
10249 |     if (isTipoDosMonedas) {
10250 |         if (isTcVendeUsd) labelMontoRecibido.textContent = 'El cliente vende ' + (recibidaDesdeTipo || '') + ' *';
10251 |         else labelMontoRecibido.innerHTML = 'El cliente <span class="orden-label-verb-destacado">entregará</span>' + escapeHtml(recibidaDesdeTipo || '') + ' (calculado)';
10252 |     } else labelMontoRecibido.textContent = (isUsdUsd || isTipoConTc || isArsArs) ? 'Monto a Recibir *' : 'Monto recibido *';
10253 | }
10254 | if (labelMontoEntregado) {
10255 |     if (isTipoDosMonedas) {
10256 |         if (isTcVendeUsd) labelMontoEntregado.innerHTML = 'El cliente <span class="orden-label-verb-destacado">recibirá</span>' + escapeHtml(entregadaDesdeTipo || '') + ' (calculado)';
10257 |         else labelMontoEntregado.textContent = 'El cliente compra ' + (entregadaDesdeTipo || '') + ' *';
10258 |     } else labelMontoEntregado.textContent = (isUsdUsd || isTipoConTc || isArsArs) ? 'Monto a Entregar *' : 'Monto entregado *';
10259 | }
10260 | const wrapTasaDescuentoInt = document.getElementById('orden-wrap-tasa-descuento-intermediario');
10261 | const wrapComisionSplit = document.getElementById('orden-wrap-comision-split');
10262 | const isTipoSinComision = isTipoConTc;
10263 | if (wrapComision) wrapComision.style.display = (isUsdUsd || (isTipoConTc && !isTipoSinComision) || isArsArs) ? 'block' : 'none';
10264 | if (wrapTasaDescuentoInt) {
10265 |     const tasaIntUsdWizard = isUsdUsd && usaIntermediario && ordenUsdIntMostrarTasaIntermediarioEnWizard();
10266 |     wrapTasaDescuentoInt.style.display = ((isArsArs && usaIntermediario) || tasaIntUsdWizard) ? 'block' : 'none';
10267 | }
10268 | if (wrapComisionSplit) wrapComisionSplit.style.display = 'none';
10269 | const fechaOrdenEl = document.getElementById('orden-fecha');
10270 | if (fechaOrdenEl) {
10271 |     if (isUsdUsd) {
10272 |         fechaOrdenEl.readOnly = true;
10273 |         fechaOrdenEl.value = fechaHoyYYYYMMDDArgentina();
10274 |     } else {
10275 |         fechaOrdenEl.readOnly = false;
10276 |     }
10277 | }
10278 | if (wrapCotizacion) {
10279 |     wrapCotizacion.style.display = (isUsdUsd || isArsArs) ? 'none' : 'block';
10280 |     if (labelCotizacion) labelCotizacion.textContent = isTipoDosMonedas ? 'Tipo de cambio *' : (isTipoConTc ? 'Tipo de cambio del acuerdo *' : 'Cotización (opcional)');
10281 |     if (inputCotizacion) inputCotizacion.required = !!isTipoConTc;
10282 | }
10283 | if (isTipoDosMonedas) {
10284 |     const formResto = document.getElementById('orden-form-resto');
10285 |     const wrapCotizacionEl = document.getElementById('orden-wrap-cotizacion');
10286 |     const rowEntregado = document.getElementById('orden-monto-entregado')?.closest('.form-row');
10287 |     const rowRecibido = document.getElementById('orden-monto-recibido')?.closest('.form-row');
10288 |     const fechaGroup = document.getElementById('orden-fecha')?.closest('.form-group');
10289 |     if (formResto && wrapCotizacionEl && rowEntregado && rowRecibido && fechaGroup) {
10290 |         if (isTcVendeUsd) {
10291 |             formResto.insertBefore(rowRecibido, wrapCotizacionEl);
10292 |             formResto.insertBefore(rowEntregado, fechaGroup);
10293 |         } else {
10294 |             formResto.insertBefore(rowEntregado, wrapCotizacionEl);
10295 |             formResto.insertBefore(rowRecibido, fechaGroup);
10296 |         }
10297 |     }
10298 |     [rowRecibido, rowEntregado].forEach((row) => {
10299 |         const firstGroup = row?.querySelector('.form-group:first-child');
10300 |         if (firstGroup) firstGroup.style.display = 'none';
10301 |     });
10302 |     const montoRecibidoElEarly = document.getElementById('orden-monto-recibido');
10303 |     if (montoRecibidoElEarly && !isTcVendeUsd) {
10304 |         montoRecibidoElEarly.readOnly = true;
10305 |         montoRecibidoElEarly.style.background = '#eee';
10306 |         montoRecibidoElEarly.style.color = '#555';
10307 |     }
10308 |     if (isTcVendeUsd && montoRecibidoElEarly) {
10309 |         const v = (montoRecibidoElEarly.value || '').trim();
10310 |         if (v === '0' || v === '0,00' || v === '0.00') montoRecibidoElEarly.value = '';
10311 |     }
10312 | } else {
10313 |     const rowRecibidoRestore = document.getElementById('orden-monto-recibido')?.closest('.form-row');
10314 |     const rowEntregadoRestore = document.getElementById('orden-monto-entregado')?.closest('.form-row');
10315 |     [rowRecibidoRestore, rowEntregadoRestore].forEach((row) => {
10316 |         const firstGroup = row?.querySelector('.form-group:first-child');
10317 |         if (firstGroup) firstGroup.style.display = '';
10318 |     });
10319 |     const montoRecibidoRestore = document.getElementById('orden-monto-recibido');
10320 |     if (montoRecibidoRestore) {
10321 |         montoRecibidoRestore.readOnly = false;
10322 |         montoRecibidoRestore.style.background = '';
10323 |         montoRecibidoRestore.style.color = '';
10324 |     }
10325 | }
10326 | const labelComision = document.querySelector('#orden-wrap-comision label[for="orden-comision-display"]');
10327 | if (labelComision) labelComision.textContent = isArsArs ? 'Beneficio del Acuerdo' : 'Comisión a Recibir';
10328 | if (comisionDisplay) {
10329 |     comisionDisplay.value = '';
10330 |     if (isArsArs) comisionDisplay.classList.add('orden-beneficio-acuerdo');
10331 |     else comisionDisplay.classList.remove('orden-beneficio-acuerdo');
10332 | }
10333 |
10334 | if (estadoSelect) {
10335 |     const optPI = estadoSelect.querySelector('option[value="pendiente_instrumentar"]');
10336 |     if (optPI) optPI.textContent = 'Pendiente Instrumentar';
10337 | }
10338 |
10339 | function actualizarComisionUsdUsd() {
10340 |     if (!isUsdUsd || !comisionDisplay) return;

```

```

10341|     const r = parseImporteInput(document.getElementById('orden-monto-recibido').value);
10342|     const e = parseImporteInput(document.getElementById('orden-monto-entregado').value);
10343|     const comision = (typeof r === 'number' && !isNaN(r) && typeof e === 'number' && !isNaN(e) && r > e) ? r - e : null;
10344|     comisionDisplay.value = comision !== null ? formatImporteDisplay(comision) : '';
10345| }
10346| function actualizarComisionUsdArs() {
10347|     if (!isTcVendeUsd || !comisionDisplay) return;
10348|     const r = parseImporteInput(document.getElementById('orden-monto-recibido').value);
10349|     const e = parseImporteInput(document.getElementById('orden-monto-entregado').value);
10350|     const tc = parseImporteInput(document.getElementById('orden-cotizacion').value);
10351|     let comision = null;
10352|     if (typeof r === 'number' && !isNaN(r) && r > 0 && typeof tc === 'number' && !isNaN(tc) && tc > 0 && typeof e === 'number' && !isNaN(e) && e >= 0) {
10353|         const usdEquivEntregado = e / tc;
10354|         if (r > usdEquivEntregado) comision = r - usdEquivEntregado;
10355|     }
10356|     comisionDisplay.value = comision !== null ? formatImporteDisplay(comision) + ' USD' : '';
10357| }
10358| function actualizarComisionArsArs() {
10359|     if (!isArsArs || !comisionDisplay) return;
10360|     const r = parseImporteInput(document.getElementById('orden-monto-recibido').value);
10361|     const e = parseImporteInput(document.getElementById('orden-monto-entregado').value);
10362|     const comision = (typeof r === 'number' && !isNaN(r) && typeof e === 'number' && !isNaN(e) && r > e) ? r - e : null;
10363|     comisionDisplay.value = comision !== null ? formatImporteDisplay(comision) + ' ARS' : '';
10364| }
10365| const montoRecibidoEl = document.getElementById('orden-monto-recibido');
10366| const montoEntregadoEl = document.getElementById('orden-monto-entregado');
10367| if (montoRecibidoEl) {
10368|     montoRecibidoEl.removeEventListener('input', actualizarComisionUsdUsd); montoRecibidoEl.removeEventListener('change', actualizarComisionUsdUsd);
10369|     montoRecibidoEl.removeEventListener('input', actualizarComisionUsdArs); montoRecibidoEl.removeEventListener('change', actualizarComisionUsdArs);
10370|     montoRecibidoEl.removeEventListener('input', actualizarComisionArsArs); montoRecibidoEl.removeEventListener('change', actualizarComisionArsArs);
10371| }
10372| if (montoEntregadoEl) {
10373|     montoEntregadoEl.removeEventListener('input', actualizarComisionUsdUsd); montoEntregadoEl.removeEventListener('change', actualizarComisionUsdUsd);
10374|     montoEntregadoEl.removeEventListener('input', actualizarComisionUsdArs); montoEntregadoEl.removeEventListener('change', actualizarComisionUsdArs);
10375|     montoEntregadoEl.removeEventListener('input', actualizarComisionArsArs); montoEntregadoEl.removeEventListener('change', actualizarComisionArsArs);
10376| }
10377| if (inputCotizacion) {
10378|     inputCotizacion.removeEventListener('input', actualizarComisionUsdArs); inputCotizacion.removeEventListener('change', actualizarComisionUsdArs);
10379| }
10380| let _actualizandoMontosTc = false;
10381| function actualizarMontosDesdeTc(origen) {
10382|     if (_actualizandoMontosTc || !inputCotizacion || !montoRecibidoEl || !montoEntregadoEl) return;
10383|     const tc = parseImporteInput(inputCotizacion.value);
10384|     if (typeof tc !== 'number' || isNaN(tc) || tc <= 0) return;
10385|     const r = parseImporteInput(montoRecibidoEl.value);
10386|     const e = parseImporteInput(montoEntregadoEl.value);
10387|     const tieneRecibir = typeof r === 'number' && !isNaN(r) && r > 0;
10388|     const tieneEntregar = typeof e === 'number' && !isNaN(e) && e > 0;
10389|     if (isTcCompraUsd) {
10390|         if (origen === 'tc' || origen === 'entregar') {
10391|             const baseEntregar = tieneEntregar ? e : 1;
10392|             if (tieneEntregar || origen === 'tc') {
10393|                 _actualizandoMontosTc = true;
10394|                 if (!tieneEntregar) montoEntregadoEl.value = formatImporteDisplay(1);
10395|                 montoRecibidoEl.value = formatImporteDisplay(baseEntregar * tc);
10396|                 _actualizandoMontosTc = false;
10397|             }
10398|         } else if (origen === 'recibir' && tieneRecibir) {
10399|             _actualizandoMontosTc = true;
10400|             montoEntregadoEl.value = formatImporteDisplay(r / tc);
10401|             _actualizandoMontosTc = false;
10402|         }
10403|     } else if (isTcVendeUsd) {
10404|         if (origen === 'tc') {
10405|             _actualizandoMontosTc = true;
10406|             if (tieneRecibir) montoEntregadoEl.value = formatImporteDisplay(r * tc);
10407|             else montoEntregadoEl.value = '';
10408|             _actualizandoMontosTc = false;
10409|         } else if (origen === 'entregar') {
10410|             if (tieneEntregar) {
10411|                 _actualizandoMontosTc = true;
10412|                 montoRecibidoEl.value = (e / tc === 0 || !Number.isFinite(e / tc)) ? '' : formatImporteParaInput(e / tc);
10413|                 _actualizandoMontosTc = false;
10414|             }
10415|         } else if (origen === 'recibir' && tieneRecibir) {
10416|             _actualizandoMontosTc = true;
10417|             montoEntregadoEl.value = formatImporteDisplay(r * tc);
10418|             _actualizandoMontosTc = false;
10419|         }
10420|     }
10421| }
10422| if (isUsdUsd) {
10423|     if (montoRecibidoEl) { montoRecibidoEl.addEventListener('input', actualizarComisionUsdUsd); montoRecibidoEl.addEventListener('change', actualizarComisionUsdUsd); }
10424|     if (montoEntregadoEl) { montoEntregadoEl.addEventListener('input', actualizarComisionUsdUsd); montoEntregadoEl.addEventListener('change', actualizarComisionUsdUsd); }
10425|     actualizarComisionUsdUsd();
10426| } else if (isTipoConTc) {
10427|     if (inputCotizacion) {
10428|         inputCotizacion.addEventListener('input', () => actualizarMontosDesdeTc('tc'));
10429|         inputCotizacion.addEventListener('change', () => actualizarMontosDesdeTc('tc'));
10430|     }
10431|     if (montoEntregadoEl) {
10432|         montoEntregadoEl.addEventListener('input', () => actualizarMontosDesdeTc('entregar'));
10433|         montoEntregadoEl.addEventListener('change', () => actualizarMontosDesdeTc('entregar'));
10434|     }
10435|     if (montoRecibidoEl) {
10436|         montoRecibidoEl.addEventListener('input', () => actualizarMontosDesdeTc('recibir'));
10437|         montoRecibidoEl.addEventListener('change', () => actualizarMontosDesdeTc('recibir'));
10438|     }
10439| }

```

```

10438 |     }
10439 |     setTimeout(() => actualizarMontosDesdeTc('tc'), 0);
10440 | } else if (isArsArs) {
10441 |   if (montoRecibidoEl) { montoRecibidoEl.addEventListener('input', actualizarComisionArsArs); montoRecibidoEl.addEventListener('change', actualizarComisionArsArs); }
10442 |   if (montoEntregadoEl) { montoEntregadoEl.addEventListener('input', actualizarComisionArsArs); montoEntregadoEl.addEventListener('change', actualizarComisionArsArs); }
10443 |   actualizarComisionArsArs();
10444 | }
10445 |
10446 | // Flujo primeros datos ARS-ARS / USD-USD: Importe + Tasa descuento -> solo Ir a instrumentación. Dos monedas (ARS-USD, etc.): también solo Instrumentación.
10447 | const isTipoPrimerosDatos = isArsArs || isUsdUsd;
10448 | const soloInstrumentacion = isTipoPrimerosDatos || isTipoDosMonedas;
10449 | const wrapPrimerosDatos = document.getElementById('orden-wrap-primeros-datos');
10450 | const labelImporteCheque = document.getElementById('orden-label-importe-cheque');
10451 | const importeChequeEl = document.getElementById('orden-importe-cheque');
10452 | const tasaDescuentoClienteEl = document.getElementById('orden-tasa-descuento-cliente');
10453 | const wrapMontosCalculados = document.getElementById('orden-wrap-montos-calculados');
10454 | const montoEntregadoDisplay = document.getElementById('orden-monto-entregado-display');
10455 | const montoRecibidoDisplay = document.getElementById('orden-monto-recibido-display');
10456 | const fechaEl = document.getElementById('orden-fecha');
10457 | const observacionesEl = document.getElementById('orden-observaciones');
10458 | const btnGuardar = document.getElementById('orden-btn-guardar');
10459 | const btnIrInst = document.getElementById('orden-btn-ir-instrumentacion');
10460 |
10461 | if (wrapPrimerosDatos) wrapPrimerosDatos.style.display = isTipoPrimerosDatos ? 'block' : 'none';
10462 | const rowMonedaMontoRec = document.getElementById('orden-row-moneda-monto-recibido');
10463 | const rowMonedaMontoEnt = document.getElementById('orden-row-moneda-monto-entregado');
10464 | if (rowMonedaMontoRec) rowMonedaMontoRec.style.display = isTipoPrimerosDatos ? 'none' : '';
10465 | if (rowMonedaMontoEnt) rowMonedaMontoEnt.style.display = isTipoPrimerosDatos ? 'none' : '';
10466 | if (labelImporteCheque) labelImporteCheque.textContent = isArsArs ? 'Importe en Cheque (ARS) *' : (isUsdUsd ? 'Importe (USD) *' : 'Importe *');
10467 | if (btnGuardar) btnGuardar.style.display = soloInstrumentacion ? 'none' : '';
10468 | const iconInstrumentacion = '<span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M14 2H6a2 2 0 0-2 2v16a2 2 0 00 2h12a2 2 0 00 2-2V8z"/><path d="M14 2v6h6"/><path d="M16 13H8"/><path d="M16 17H8"/><path d="M10 9H8"/></svg></span>';
10469 | if (btnIrInst) {
10470 |   btnIrInst.style.display = '';
10471 |   if (soloInstrumentacion) {
10472 |     btnIrInst.innerHTML = iconInstrumentacion + (isTipoPrimerosDatos ? 'Ir a instrumentación' : 'Instrumentación');
10473 |     btnIrInst.classList.remove('btn-secondary');
10474 |     btnIrInst.classList.add('btn-primary');
10475 |   } else {
10476 |     btnIrInst.innerHTML = iconInstrumentacion + 'Instrumentación';
10477 |     btnIrInst.classList.remove('btn-primary');
10478 |     btnIrInst.classList.add('btn-secondary');
10479 |   }
10480 | }
10481 |
10482 | function setRestoOrdenEditable(editable) {
10483 |   const el = (id) => document.getElementById(id);
10484 |   const tieneIntermediario = !(el('orden-intermediario')?.value?.trim());
10485 |   if (isUsdUsd) {
10486 |     [fechaEl, estadoSelect, observacionesEl].forEach((e) => { if (e) e.disabled = true; });
10487 |     if (montoRecibidoEl) montoRecibidoEl.readOnly = true;
10488 |     if (montoEntregadoEl) montoEntregadoEl.readOnly = true;
10489 |     if (monedaRecibida) monedaRecibida.disabled = true;
10490 |     if (monedaEntregada) monedaEntregada.disabled = true;
10491 |     const tasaIntUsd = el('orden-tasa-descuento-intermediario');
10492 |     const wrapTasaIntUsd = document.getElementById('orden-wrap-tasa-descuento-intermediario');
10493 |     const tasaIntUsdOn = wrapTasaIntUsd && (wrapTasaIntUsd.style.display === 'block' || (typeof window !== 'undefined' && window.getComputedStyle && window.getComputedStyle(wrapTasaIntUsd).display !== 'none'));
10494 |     // No atar a 'editable': si no, la tasa int. queda grisada hasta tener valor válido (imposible cargarla).
10495 |     if (tasaIntUsd) tasaIntUsd.disabled = !(tieneIntermediario && tasaIntUsdOn);
10496 |     // Nacho 50/75: no usar 'editable' aquí - si no, quedan grisados hasta elegir radio (imposible).
10497 |     document.querySelectorAll('input[name="orden-usd-nacho-comision-usd"]').forEach((r) => {
10498 |       r.disabled = !ordenUsdIntMostrarComisionFijaNacho();
10499 |     });
10500 |   } else if (isArsArs) {
10501 |     // ARS-ARS: todo el resto es solo informativo (grisado) salvo tasa de descuento del intermediario si hay intermediario
10502 |     [fechaEl, estadoSelect, observacionesEl].forEach((e) => { if (e) e.disabled = true; });
10503 |     if (montoRecibidoEl) montoRecibidoEl.readOnly = true;
10504 |     if (montoEntregadoEl) montoEntregadoEl.readOnly = true;
10505 |     if (monedaRecibida) monedaRecibida.disabled = true;
10506 |     if (monedaEntregada) monedaEntregada.disabled = true;
10507 |     const tasaInt = el('orden-tasa-descuento-intermediario');
10508 |     if (tasaInt) tasaInt.disabled = !(editable && tieneIntermediario);
10509 |   } else if (isTipoDosMonedas) {
10510 |     // compra usd: editable monto entregado (USD); vende_usd: editable monto recibido (USD); la otra moneda calculada con tc
10511 |     if (fechaEl) fechaEl.disabled = true;
10512 |     if (estadoSelect) estadoSelect.disabled = true;
10513 |     if (observacionesEl) observacionesEl.disabled = !editable;
10514 |     if (isTcVendeUsd) {
10515 |       if (montoRecibidoEl) {
10516 |         montoRecibidoEl.readOnly = !editable;
10517 |         montoRecibidoEl.style.background = '';
10518 |         montoRecibidoEl.style.color = '';
10519 |       }
10520 |       if (montoEntregadoEl) {
10521 |         montoEntregadoEl.readOnly = true;
10522 |         montoEntregadoEl.style.background = '#eee';
10523 |         montoEntregadoEl.style.color = '#555';
10524 |       }
10525 |     } else {
10526 |       if (montoRecibidoEl) {
10527 |         montoRecibidoEl.readOnly = true;
10528 |         montoRecibidoEl.style.background = '#eee';
10529 |         montoRecibidoEl.style.color = '#555';
10530 |       }
10531 |       if (montoEntregadoEl) {
10532 |         montoEntregadoEl.readOnly = !editable;

```

```

10533|         montoEntregadoEl.style.background = '';
10534|         montoEntregadoEl.style.color = '';
10535|     }
10536| }
10537| const cotEl = el('orden-cotizacion');
10538| if (cotEl) cotEl.readOnly = !editable;
10539| if (monedaRecibida) monedaRecibida.disabled = true;
10540| if (monedaEntregada) monedaEntregada.disabled = true;
10541| } else {
10542| [fechaEl, estadoSelect, observacionesEl].forEach((e) => { if (e) e.disabled = !editable; });
10543| if (wrapTasaDescuentoInt && isArsArs) {
10544|     const input = el('orden-tasa-descuento-intermediario');
10545|     if (input) input.disabled = !editable;
10546| }
10547| if (montoRecibidoEl) montoRecibidoEl.readOnly = !editable;
10548| if (montoEntregadoEl) montoEntregadoEl.readOnly = !editable;
10549| if (monedaRecibida) monedaRecibida.disabled = !editable;
10550| if (monedaEntregada) monedaEntregada.disabled = !editable;
10551| }
10552| }
10553|
10554| function actualizarPrimerosDatos() {
10555|     if (!isTipoPrimerosDatos || !importeChequeEl || !tasaDescuentoClienteEl) return;
10556|     const wrapTrasPat = document.getElementById('orden-wrap-detalles-tras-patron');
10557|     if (isUsdUsd && esWizardUsdUsdConIntermediario() && wrapTrasPat.style.display === 'none') {
10558|         setRestoOrdenEditable(false);
10559|         if (wrapMontosCalculados) wrapMontosCalculados.style.display = 'none';
10560|         if (comisionDisplay) comisionDisplay.value = '';
10561|         return;
10562|     }
10563|     const importe = parseImporteInput(importeChequeEl.value);
10564|     const tasaPct = parseImporteInput(tasaDescuentoClienteEl.value);
10565|     const importeOk = typeof importe === 'number' && !isNaN(importe) && importe > 0;
10566|     const tasaOk = typeof tasaPct === 'number' && !isNaN(tasaPct) && tasaPct > 0 && tasaPct < 100;
10567|     if (!importeOk || !tasaOk) {
10568|         setRestoOrdenEditable(false);
10569|         if (wrapMontosCalculados) wrapMontosCalculados.style.display = 'none';
10570|         if (montoRecibidoEl) montoRecibidoEl.value = '';
10571|         if (montoEntregadoEl) montoEntregadoEl.value = '';
10572|         if (comisionDisplay) comisionDisplay.value = '';
10573|         return;
10574|     }
10575|     const montoRecibir = importe;
10576|     const tasaIntEl = document.getElementById('orden-tasa-descuento-intermediario');
10577|     const wrapTasaInt = document.getElementById('orden-wrap-tasa-descuento-intermediario');
10578|     const tasaIntVisible = wrapTasaInt && (wrapTasaInt.style.display === 'block' || (typeof window !== 'undefined' && window.getComputedStyle && window.getComputedStyle(wrapTasaInt).display !== 'none'));
10579|     let montoEntregar;
10580|     if (isUsdUsd && esWizardUsdUsdConIntermediario() && tasaIntVisible) {
10581|         const tasaIntPct = parseImporteInput(tasaIntEl.value);
10582|         // La tasa del intermediario es acuerdo Pandy-intermediario (reparto de comisiones orden), no descuenta otra vez sobre lo que el cliente entrega.
10583|         if (!(typeof tasaIntPct === 'number' && !isNaN(tasaIntPct) && tasaIntPct > 0 && tasaIntPct < 100)) {
10584|             setRestoOrdenEditable(false);
10585|             if (wrapMontosCalculados) wrapMontosCalculados.style.display = 'none';
10586|             if (montoRecibidoEl) montoRecibidoEl.value = '';
10587|             if (montoEntregadoEl) montoEntregadoEl.value = '';
10588|             if (comisionDisplay) comisionDisplay.value = formatImporteDisplay(importe * (tasaPct / 100));
10589|             return;
10590|         }
10591|         montoEntregar = importe * (1 - tasaPct / 100);
10592|     } else if (isUsdUsd && ordenUsdIntMostrarComisionFijaNacho()) {
10593|         const nachoInp = document.querySelector('input[name="orden-usd-nacho-comision-usd"]:checked');
10594|         if (!nachoInp) {
10595|             setRestoOrdenEditable(false);
10596|             if (wrapMontosCalculados) wrapMontosCalculados.style.display = 'none';
10597|             if (montoRecibidoEl) montoRecibidoEl.value = '';
10598|             if (montoEntregadoEl) montoEntregadoEl.value = '';
10599|             // Spread total = importe x tasa cliente % (mismo que mr-me al elegir 50/75); evita valor viejo en pantalla.
10600|             if (comisionDisplay) comisionDisplay.value = formatImporteDisplay(importe * (tasaPct / 100));
10601|             return;
10602|         }
10603|         montoEntregar = importe * (1 - tasaPct / 100);
10604|         const comSpread = importe - montoEntregar;
10605|         const fija = Number(nachoInp.value) || 0;
10606|         if (fija > comSpread + 1e-6) {
10607|             setRestoOrdenEditable(false);
10608|             if (wrapMontosCalculados) wrapMontosCalculados.style.display = 'none';
10609|             if (montoRecibidoEl) montoRecibidoEl.value = '';
10610|             if (montoEntregadoEl) montoEntregadoEl.value = '';
10611|             if (comisionDisplay) comisionDisplay.value = '';
10612|             return;
10613|         }
10614|     } else {
10615|         montoEntregar = importe * (1 - tasaPct / 100);
10616|     }
10617|     if (montoRecibidoEl) { montoRecibidoEl.value = formatImporteDisplay(montoRecibir); montoRecibidoEl.readOnly = true; }
10618|     if (montoEntregadoEl) { montoEntregadoEl.value = formatImporteDisplay(montoEntregar); montoEntregadoEl.readOnly = true; }
10619|     if (montoRecibidoDisplay) montoRecibidoDisplay.value = formatImporteDisplay(montoRecibir);
10620|     if (montoEntregadoDisplay) montoEntregadoDisplay.value = formatImporteDisplay(montoEntregar);
10621|     if (wrapMontosCalculados) wrapMontosCalculados.style.display = 'flex';
10622|     setRestoOrdenEditable(true);
10623|     if (isUsdUsd) actualizarComisionUsdUsd();
10624|     if (isArsArs) actualizarComisionArsArs();
10625|     if (isArsArs) {
10626|         const tasaInt = document.getElementById('orden-tasa-descuento-intermediario');
10627|         if (tasaInt) tasaInt.disabled = false;
10628|     } else if (isUsdUsd && tasaIntVisible) {

```



```

10629|         if (tasaIntEl) tasaIntEl.disabled = false;
10630|     }
10631| }
10632|
10633| if (isTipoPrimerosDatos) {
10634|     setRestoOrdenEditable(false);
10635|     if (wrapMontosCalculados) wrapMontosCalculados.style.display = 'none';
10636|     if (importeChequeEl) {
10637|         importeChequeEl.removeEventListener('input', actualizarPrimerosDatos);
10638|         importeChequeEl.removeEventListener('change', actualizarPrimerosDatos);
10639|         importeChequeEl.addEventListener('input', actualizarPrimerosDatos);
10640|         importeChequeEl.addEventListener('change', actualizarPrimerosDatos);
10641|     }
10642|     if (tasaDescuentoClienteEl) {
10643|         tasaDescuentoClienteEl.removeEventListener('input', actualizarPrimerosDatos);
10644|         tasaDescuentoClienteEl.removeEventListener('change', actualizarPrimerosDatos);
10645|         tasaDescuentoClienteEl.addEventListener('input', actualizarPrimerosDatos);
10646|         tasaDescuentoClienteEl.addEventListener('change', actualizarPrimerosDatos);
10647|     }
10648|     const tasaIntPrimerosUsd = document.getElementById('orden-tasa-descuento-intermediario');
10649|     if (tasaIntPrimerosUsd && isUsdUsd) {
10650|         tasaIntPrimerosUsd.removeEventListener('input', actualizarPrimerosDatos);
10651|         tasaIntPrimerosUsd.removeEventListener('change', actualizarPrimerosDatos);
10652|         tasaIntPrimerosUsd.addEventListener('input', actualizarPrimerosDatos);
10653|         tasaIntPrimerosUsd.addEventListener('change', actualizarPrimerosDatos);
10654|     }
10655| } else {
10656|     setRestoOrdenEditable(true);
10657|     // No forzar readOnly=false en ambos montos si es cruce con TC (ARS-USD, etc.):
10658|     // setRestoOrdenEditable ya dejó solo el campo operativo editable; si ambos quedan
10659|     // editables, el usuario suele escribir en el gris (calculado) sin TC y parece "bloqueado".
10660|     if (!isTipoDosMonedas) {
10661|         if (montoRecibidoEl) montoRecibidoEl.readOnly = false;
10662|         if (montoEntregadoEl) montoEntregadoEl.readOnly = false;
10663|     }
10664| }
10665| if (isTipoPrimerosDatos) actualizarPrimerosDatos();
10666| const lblIntTxt = document.getElementById('orden-label-intermediario-texto');
10667| if (lblIntTxt) lblIntTxt.textContent = usaIntermediario ? 'Intermediario *' : 'Intermediario (opcional)';
10668| syncOrdenIntPatronInstrumentacionWrap();
10669| applyOrdenUsdIntPostPatronVisibilidad();
10670| syncOrdenWizardUsdIntComisionUi();
10671| }
10672|
10673| /** Cierra el modal de orden (sin validar instrumentación). Usar solicitarCierreModalOrden desde la UI del wizard. */
10674| function ejecutarCierreModalOrden() {
10675|     ordenModalLoadSeq += 1;
10676|     const idBorrador = ordenIdBorradorParaEliminar;
10677|     const instId = ordenWizardInstrumentacionIdActual;
10678|     ordenIdBorradorParaEliminar = null;
10679|     ordenWizardOrdenIdActual = null;
10680|     ordenWizardInstrumentacionIdActual = null;
10681|     const backdrop = document.getElementById('modal-orden-backdrop');
10682|     function doClose() {
10683|         closeOrdenTipoOperacionListbox();
10684|         dismissAllToasts();
10685|         document.body.classList.remove('modal-orden-abierto');
10686|         if (backdrop) backdrop.classList.remove('activo');
10687|         if (idBorrador) client.from('ordenes').delete().eq('id', idBorrador).then(() => loadOrdenes());
10688|     }
10689|     // Cerrar el modal de inmediato para que Listo/Cerrar respondan al instante.
10690|     // Sincronizar montos editados en la tabla en segundo plano (sin bloquear).
10691|     if (instId && backdrop && backdrop.classList.contains('activo')) {
10692|         const inputs = backdrop.querySelectorAll('.input-monto-transaccion-tabla');
10693|         const pendientes = [];
10694|         inputs.forEach((input) => {
10695|             const id = input.getAttribute('data-id');
10696|             if (id) pendientes.push({ id, value: input.value });
10697|         });
10698|         doClose();
10699|         if (pendientes.length > 0) {
10700|             let prom = Promise.resolve();
10701|             pendientes.forEach(p => { prom = prom.then(() => guardarSoloMontoTransaccion(p.id, p.value)); });
10702|             prom.catch(() => {});
10703|         }
10704|     } else {
10705|         doClose();
10706|     }
10707| }
10708|
10709| function closeModalOrden() {
10710|     ejecutarCierreModalOrden();
10711| }
10712|
10713| function showOrdenWizardStep(which) {
10714|     const stepParticipantes = document.getElementById('orden-step-participantes');
10715|     const stepDetalles = document.getElementById('orden-step-detalles');
10716|     const stepInst = document.getElementById('orden-step-instrumentacion');
10717|     if (stepParticipantes) stepParticipantes.style.display = which === 'participantes' ? 'block' : 'none';
10718|     if (stepDetalles) stepDetalles.style.display = which === 'detalles' ? 'block' : 'none';
10719|     if (stepInst) stepInst.style.display = which === 'instrumentacion' ? 'block' : 'none';
10720|     const modalOrden = document.querySelector('#modal-orden-backdrop .modal-orden');
10721|     if (modalOrden) {
10722|         if (which === 'detalles' || which === 'instrumentacion') modalOrden.classList.add('modal-orden-con-instrumentacion');
10723|         else modalOrden.classList.remove('modal-orden-con-instrumentacion');
10724|     }
10725|     // Al mostrar instrumentación: scroll a Listo/Cerrar y dar foco al botón Listo para que el modal no pierda foco

```

```

10726 | if (which === 'instrumentacion') {
10727 |     const btnCerrar = document.getElementById('orden-btn-cerrar-wizard');
10728 |     if (btnCerrar) {
10729 |         requestAnimationFrame(() => {
10730 |             btnCerrar.scrollIntoView({ block: 'nearest', behavior: 'smooth' });
10731 |             btnCerrar.focus({ preventScroll: true });
10732 |         });
10733 |     }
10734 | }
10735 | }
10736 |
10737 | /** Devuelve el próximo número de orden: MAX(numero)+1 (o 1 si no hay órdenes). Solo si ordenesTieneNumeroColumn. */
10738 | function getProximoNumeroOrden() {
10739 |     if (!ordenesTieneNumeroColumn) return Promise.resolve(1);
10740 |     return client.from('ordenes').select('numero').order('numero', { ascending: false }).limit(1).maybeSingle()
10741 |         .then((r) => {
10742 |             if (r.error) return 1;
10743 |             const max = r.data && r.data.numero != null ? Number(r.data.numero) : 0;
10744 |             return (typeof max === 'number' && !isNaN(max) ? max : 0) + 1;
10745 |         });
10746 | }
10747 |
10748 | /** Inserta una orden con numero = MAX+1 atómico (función en DB con lock). Si la columna numero no existe o la RPC no está en Supabase, hace INSERT con getProximoNumeroOrden.
10749 | */
10750 | function insertOrdenConProximoNumero(payload) {
10751 |     if (!ordenesTieneNumeroColumn) {
10752 |         return client.from('ordenes').insert(payload).select('id, numero');
10753 |     }
10754 |     return client.rpc('ordenes_insertar_con_proximo_numero', {
10755 |         p_cliente_id: payload.cliente_id,
10756 |         p_fecha: payload.fecha,
10757 |         p_estado: payload.estado,
10758 |         p_tipo_operacion_id: payload.tipo_operacion_id,
10759 |         p_operacion_directa: payload.operacion_directa,
10760 |         p_intermediario_id: payload.intermediario_id,
10761 |         p_moneda_recibida: payload.moneda_recibida,
10762 |         p_moneda_entregada: payload.moneda_entregada,
10763 |         p_monto_recibido: payload.monto_recibido,
10764 |         p_monto_entregado: payload.monto_entregado,
10765 |         p_cotizacion: payload.cotizacion,
10766 |         p_tasa_descuento_intermediario: payload.tasa_descuento_intermediario,
10767 |         p_observaciones: payload.observaciones,
10768 |         p_usuario_id: payload.usuario_id,
10769 |         p_updated_at: payload.updated_at,
10770 |     }).then((res) => {
10771 |         if (!res.error) {
10772 |             const row = res.data && (Array.isArray(res.data) ? res.data[0] : res.data);
10773 |             return { data: row ? [row] : [], error: null };
10774 |         }
10775 |         const msg = (res.error.message || '').toLowerCase();
10776 |         const rpcNoDisponible = msg.includes('could not find the function') || msg.includes('schema cache') || (msg.includes('function') && msg.includes('does not exist'));
10777 |         if (!rpcNoDisponible) return res;
10778 |         return getProximoNumeroOrden().then((nextNum) => {
10779 |             const p = { ...payload, numero: nextNum };
10780 |             return client.from('ordenes').insert(p).select('id, numero').then((insertRes) => {
10781 |                 if (insertRes.error) return insertRes;
10782 |                 showToast('Orden creada. Para asignación atómica de número, ejecutó sql/ordenes_insertar_con_proximo_numero.sql en Supabase.', 'info');
10783 |                 return { data: insertRes.data, error: null };
10784 |             });
10785 |         });
10786 |     });
10787 | }
10788 |
10789 | /** El split USD-USD/CHEQUE usa display:flex|none; no confundir style.display === '' con visible (rompe guardar e instrumentación). */
10790 | function ordenWrapComisionSplitEsVisible() {
10791 |     const wrapSplitEl = document.getElementById('orden-wrap-comision-split');
10792 |     if (!wrapSplitEl) return false;
10793 |     try {
10794 |         return typeof window !== 'undefined' && window.getComputedStyle
10795 |             ? window.getComputedStyle(wrapSplitEl).display !== 'none'
10796 |             : (wrapSplitEl.style.display !== 'none' && wrapSplitEl.style.display !== '');
10797 |     } catch (e) {
10798 |         return wrapSplitEl.style.display !== 'none' && wrapSplitEl.style.display !== '';
10799 |     }
10800 | }
10801 |
10802 | /** Guarda la orden según el form, pero sin cerrar el modal. Devuelve Promise<ordenId>. */
10803 | function guardarOrdenDesdeWizard(opcionGuardarConComisionCero = false) {
10804 |     const idEl = document.getElementById('orden-id');
10805 |     const id = idEl && idEl.value ? idEl.value.trim() : '';
10806 |     const clienteId = document.getElementById('orden-cliente').value.trim() || null;
10807 |     const fecha = document.getElementById('orden-fecha').value;
10808 |     const tipoOperacionId = document.getElementById('orden-tipo-operacion')?.value?.trim() || null;
10809 |     const selTipoOptGuardar = document.getElementById('orden-tipo-operacion')?.selectedOptions?.[0];
10810 |     const usaIntermediarioTipo = selTipoOptGuardar ? (selTipoOptGuardar.getAttribute('data-usa-intermediario') === 'true') : false;
10811 |     let intermediarioId = document.getElementById('orden-intermediario')?.value?.trim() || null;
10812 |     if (!usaIntermediarioTipo) intermediarioId = null;
10813 |     const operacionDirecta = !intermediarioId;
10814 |     const monedaRecibida = document.getElementById('orden-moneda-recibida').value;
10815 |     const monedaEntregada = document.getElementById('orden-moneda-entregada').value;
10816 |     const montoRecibido = parseImporteInput(document.getElementById('orden-monto-recibido').value);
10817 |     const montoEntregado = parseImporteInput(document.getElementById('orden-monto-entregado').value);
10818 |     const cotizacionRaw = document.getElementById('orden-cotizacion').value.trim();
10819 |     const cotizacion = cotizacionRaw ? parseImporteInput(cotizacionRaw) : null;
10820 |     const estado = document.getElementById('orden-estado').value;
10821 |     const observaciones = document.getElementById('orden-observaciones').value.trim() || null;

```

```

10822| if (!clienteId && !intermediarioId) {
10823|   showToast('Definí participantes: elegí un cliente, un intermediario o ambos.', 'error');
10824|   return Promise.resolve(null);
10825| }
10826| if (!tipoOperacionId) {
10827|   showToast('Elegí un tipo de operación.', 'error');
10828|   return Promise.resolve(null);
10829| }
10830| if (usaIntermediarioTipo && !intermediarioId) {
10831|   showToast('Para este tipo de operación es obligatorio elegir un intermediario.', 'error');
10832|   return Promise.resolve(null);
10833| }
10834| const selTipoOptPre = document.getElementById('orden-tipo-operacion')?.selectedOptions?.[0];
10835| const tipoCodigoPre = selTipoOptPre ? (selTipoOptPre.getAttribute('data-codigo') || '') : '';
10836| if (String(tipoCodigoPre).trim().toUpperCase() === 'USD-USD' && usaIntermediarioTipo && intermediarioId && !id && !ordenIntPatronExplicitoElegido()) {
10837|   showToast('Elegí la instrumentación sugerida (con intermediario) antes de guardar o ir a instrumentación.', 'error');
10838|   return Promise.resolve(null);
10839| }
10840| if (!fecha || isNaN(montoRecibido) || montoRecibido <= 0 || isNaN(montoEntregado) || montoEntregado <= 0) {
10841|   showToast('Completá fecha, monto recibido y monto entregado (números positivos).', 'error');
10842|   return Promise.resolve(null);
10843| }
10844| }
10845| const selTipoOpt = document.getElementById('orden-tipo-operacion')?.selectedOptions?.[0];
10846| const tipoCodigo = selTipoOpt ? (selTipoOpt.getAttribute('data-codigo') || '') : '';
10847| const tipoCodigoU = String(tipoCodigo).trim().toUpperCase();
10848| const esChequeArsOrden = esChequeArsDesdeSeleccionOption(selTipoOpt);
10849| const patronTcGuardar = patronTipoCambioOrden(monedaRecibida, monedaEntregada);
10850| if (patronTcGuardar && (!cotizacion || !(cotizacion > 0))) {
10851|   showToast('En operaciones con cruce con tipo de cambio (USD o EUR contra otra moneda) el tipo de cambio del acuerdo es obligatorio y debe ser mayor a cero.', 'error');
10852|   return Promise.resolve(null);
10853| }
10854| if (tipoCodigoU === 'USD-USD' && montoRecibido <= montoEntregado) {
10855|   showToast('En USD-USD el monto a recibir debe ser mayor al monto a entregar (la diferencia es la comisión).', 'error');
10856|   return Promise.resolve(null);
10857| }
10858| if (esChequeArsOrden) {
10859|   if (montoRecibido <= montoEntregado) {
10860|     showToast('En operación con cheque (CHEQUE-ARS) el monto a recibir debe ser mayor al monto a entregar (descuento acuerdo).', 'error');
10861|     return Promise.resolve(null);
10862|   }
10863|   const tasaPctRaw = document.getElementById('orden-tasa-descuento-intermediario')?.value?.trim() || '';
10864|   const tasaPct = tasaPctRaw ? parseImporteInput(tasaPctRaw) : null;
10865|   if (typeof tasaPct !== 'number' || isNaN(tasaPct) || tasaPct <= 0 || tasaPct >= 100) {
10866|     showToast('En operación con cheque (CHEQUE-ARS) la tasa de descuento del intermediario es obligatoria (ej. 1 para 1%, entre 0 y 100).', 'error');
10867|     return Promise.resolve(null);
10868|   }
10869| }
10870| if (tipoCodigoU === 'USD-USD' && intermediarioId && ordenComisionFijaNachoUsdParaPersistir(tipoCodigo, intermediarioId)) {
10871|   const nr = document.querySelector('input[name="orden-usd-nacho-comision-usd"]:checked');
10872|   if (!nr) {
10873|     toastElegirComisionFijaIntermediarioUsd();
10874|     return Promise.resolve(null);
10875|   }
10876| }
10877| if (tipoCodigoU === 'USD-USD' && intermediarioId && ordenUsdIntMostrarTasaIntermediarioEnWizard()) {
10878|   const rawUsdT = document.getElementById('orden-tasa-descuento-intermediario')?.value?.trim() || '';
10879|   const tUsdInt = rawUsdT ? parseImporteInput(rawUsdT) : null;
10880|   if (typeof tUsdInt !== 'number' || isNaN(tUsdInt) || tUsdInt <= 0 || tUsdInt >= 100) {
10881|     showToast('En USD-USD con intermediario completá la tasa del intermediario (% sobre el importe).', 'error');
10882|     return Promise.resolve(null);
10883|   }
10884| }
10885| const comisionUsd = tipoCodigoU === 'USD-USD' ? montoRecibido - montoEntregado
10886|   : (esChequeArsOrden ? montoRecibido - montoEntregado
10887|     : (esPatronCompraFiatConTc(patronTcGuardar) && cotizacion > 0 ? (montoRecibido / cotizacion) - montoEntregado
10888|       : (esPatronVendeFiatConTc(patronTcGuardar) && cotizacion > 0 ? montoRecibido - (montoEntregado / cotizacion) : null)));
10889| if (tipoCodigoU === 'USD-USD' && intermediarioId && ordenComisionFijaNachoUsdParaPersistir(tipoCodigo, intermediarioId) && comisionUsd !== null) {
10890|   const nchkW = document.querySelector('input[name="orden-usd-nacho-comision-usd"]:checked');
10891|   const fjW = nchkW ? Number(nchkW.value) || 0 : 0;
10892|   if (fjW > comisionUsd + 1e-6) {
10893|     showToast('La comisión fija del intermediario no puede superar el beneficio del acuerdo.', 'error');
10894|     return Promise.resolve(null);
10895|   }
10896| }
10897| const pctPandy = parseImporteInput(document.getElementById('orden-comision-pandy-pct')?.value || '100');
10898| const pctInt = parseImporteInput(document.getElementById('orden-comision-intermediario-pct')?.value || '0');
10899| const tieneSplitVisible = ordenWrapComisionSplitEsVisible();
10900| if ((tipoCodigoU === 'USD-USD' || patronTcGuardar || esChequeArsOrden) && intermediarioId && tieneSplitVisible) {
10901|   const a = Number(pctPandy);
10902|   const b = Number(pctInt);
10903|   if (isNaN(a) || isNaN(b) || a < 0 || b < 0 || a > 100 || b > 100 || Math.abs((a + b) - 100) > 1e-6) {
10904|     showToast('La distribución de comisión debe sumar 100% (Pandy + Intermediario).', 'error');
10905|     return Promise.resolve(null);
10906|   }
10907| }
10908| if (intermediarioId && tieneSplitVisible && (Number(pctInt) || 0) < 1e-6 && !opcionGuardarConComisionCero) {
10909|   showConfirm('La comisión del intermediario es cero. ¿Deseás guardar la orden igual?', 'Si, guardar', () => guardarOrdenDesdeWizard(true));
10910|   return Promise.resolve(null);
10911| }
10912| }
10913| const tasaDescuentoIntPct = document.getElementById('orden-tasa-descuento-intermediario')?.value?.trim();
10914| let tasaDescuentoIntermediario = null;
10915| if (esChequeArsOrden && tasaDescuentoIntPct) {
10916|   tasaDescuentoIntermediario = parseImporteInput(tasaDescuentoIntPct) / 100;
10917| } else if (tipoCodigoU === 'USD-USD' && intermediarioId && ordenUsdIntMostrarTasaIntermediarioEnWizard() && tasaDescuentoIntPct) {
10918|   tasaDescuentoIntermediario = parseImporteInput(tasaDescuentoIntPct) / 100;

```

```

10919 | }
10920 | const estadoFinal = id ? estado : 'pendiente_instrumentar';
10921 | const payload = {
10922 |   cliente_id: clienteId,
10923 |   fecha,
10924 |   estado: estadoFinal,
10925 |   tipoOperacion id: tipoOperacionId || null,
10926 |   operacion directa: operacionDirecta,
10927 |   intermediario id: intermediarioId,
10928 |   moneda_recibida: monedaRecibida,
10929 |   moneda_entregada: monedaEntregada,
10930 |   monto_recibido: montoRecibido,
10931 |   monto_entregado: montoEntregado,
10932 |   cotizacion: cotizacion,
10933 |   tasa_descuento_intermediario: tasaDescuentoIntermediario,
10934 |   observaciones,
10935 |   usuario_id: currentUserId,
10936 |   updated_at: new Date().toISOString(),
10937 | };
10938 |
10939 | function hacerUpdate(estadoPersistir) {
10940 |   const p = { ...payload, estado: estadoPersistir };
10941 |   return id ? client.from('ordenes').update(p).eq('id', id) : client.from('ordenes').insert(p).select('id');
10942 | }
10943 |
10944 | const prom = id
10945 |   ? client.from('instrumentacion').select('id, multicontraparte_manual').eq('orden_id', id).maybeSingle().then((rInst) => {
10946 |     const instId = rInst.data && rInst.data.id;
10947 |     const mcFlag = !(rInst.data && rInst.data.multicontraparte_manual);
10948 |     const promTr = instId ? client.from('transacciones').select('id, estado, tipo, moneda, monto, cobrador, pagador, pagador_cliente_id, cobrador_cliente_id').eq('instrume
ntacion_id', instId) : Promise.resolve({ data: [] });
10949 |     const promTipo = tipoOperacionId ? client.from('tipos_operacion').select('codigo, usa_intermediario').eq('id', tipoOperacionId).single() : Promise.resolve({ data: null
});
10950 |     return Promise.all([promTr, promTipo]).then(([rTr, rTipo]) => {
10951 |       const list = rTr.data || [];
10952 |       const toJ = rTipo.data;
10953 |       const ordenParaCalc = {
10954 |         cliente_id: clienteId,
10955 |         intermediario_id: intermediarioId,
10956 |         moneda_recibida: monedaRecibida,
10957 |         monto_recibido: montoRecibido,
10958 |         moneda_entregada: monedaEntregada,
10959 |         monto_entregado: montoEntregado,
10960 |         cotizacion,
10961 |       };
10962 |       const totMc = mcFlag && esTipoOpMulticontraparteElegibleDesdeOrden(ordenParaCalc, toJ);
10963 |       const { estado: estadoCalculado } = calcularEstadoOrden(list, ordenParaCalc, { totalesMulticontraparte: totMc });
10964 |       return hacerUpdate(estadoCalculado);
10965 |     });
10966 |   })
10967 |   : InsertOrdenConProximoNumero(payload);
10968 |
10969 | return prom.then((res) => {
10970 |   if (res.error) {
10971 |     showToast('Error: ' + (res.error.message || 'No se pudo guardar.'), 'error');
10972 |     return null;
10973 |   }
10974 |   const ordenId = id || (res.data && res.data[0] && res.data[0].id);
10975 |   if (!ordenId) return null;
10976 |   if (ordenIdBorradorParaEliminar === ordenId) ordenIdBorradorParaEliminar = null;
10977 |   if (!idEl.value) idEl.value = ordenId;
10978 |   ordenWizardOrdenIdActual = ordenId;
10979 |   function guardarComision() {
10980 |     const conceptoComision = tipoCodigo === 'USD-ARS' ? 'Comisión USD-ARS' : (esChequeArsOrden ? 'Comisión ARS-ARS' : 'Comisión USD-USD');
10981 |     const comisionMoneda = esChequeArsOrden ? 'ARS' : 'USD';
10982 |     if (!((tipoCodigoU === 'USD-USD' || esChequeArsOrden) && comisionUsd != null && comisionUsd > 0)) return Promise.resolve();
10983 |     return client.from('comisiones_orden').delete().eq('orden_id', ordenId).then(() => {
10984 |       let montoPandy;
10985 |       let montoInter;
10986 |       if (tipoCodigoU === 'USD-USD' && intermediarioId) {
10987 |         const importeAc = parseImporteInput(document.getElementById('orden-importe-cheque')?.value);
10988 |         if (ordenComisionFijaNachoUsdParaPersistir(tipoCodigo, intermediarioId)) {
10989 |           const n = document.querySelector('input[name="orden-usd-nacho-comision-usd"]:checked');
10990 |           const fija = n ? Number(n.value) || 0 : 0;
10991 |           montoInter = fija;
10992 |           montoPandy = comisionUsd - fija;
10993 |           persistirPreferenciaComisionFijaInterUsd(fija);
10994 |         } else if (ordenUsdIntMostrarTasaIntermediarioEnWizard() && typeof importeAc === 'number' && !isNaN(importeAc) && importeAc > 0) {
10995 |           const tI = parseImporteInput(document.getElementById('orden-tasa-descuento-intermediario')?.value);
10996 |           if (typeof tI === 'number' && !isNaN(tI)) {
10997 |             const totalCom = comisionUsd;
10998 |             const brutoInter = importeAc * (tI / 100);
10999 |             montoInter = Math.min(Math.max(0, brutoInter), Math.max(0, totalCom));
11000 |             montoPandy = Math.max(0, totalCom - montoInter);
11001 |           } else {
11002 |             montoPandy = comisionUsd;
11003 |             montoInter = 0;
11004 |           }
11005 |         } else {
11006 |           montoPandy = comisionUsd;
11007 |           montoInter = 0;
11008 |         }
11009 |       } else {
11010 |         const a = intermediarioId ? Number(pctPandy) : 100;
11011 |         const b = intermediarioId ? Number(pctInt) : 0;
11012 |         montoPandy = comisionUsd * (a / 100);
11013 |         montoInter = comisionUsd * (b / 100);

```

```

11014 |     }
11015 |     if (montoPandy < -1e-6) {
11016 |       showToast('La comision fija del intermediario no puede superar el beneficio del acuerdo.', 'error');
11017 |       return Promise.reject(new Error('comision_invalida'));
11018 |     }
11019 |     const rows = [
11020 |       { orden_id: ordenId, moneda: comisionMoneda, monto: montoPandy, concepto: conceptoComision, beneficiario: 'pandy', intermediario_id: null },
11021 |     ];
11022 |     if (intermediarioId && montoInter > 0) rows.push({ orden_id: ordenId, moneda: comisionMoneda, monto: montoInter, concepto: conceptoComision, beneficiario: 'intermediar
io', intermediario_id: intermediarioId });
11023 |     return client.from('comisiones_orden').insert(rows).then(() => {});
11024 |   });
11025 | }
11026 | if (esChequeArsOrden && intermediarioId && tasaDescuentoIntermediario != null) {
11027 |   return guardarComision().then(() =>
11028 |     actualizarTasaTransaccionIngresoIntermediarioCheque(ordenId, {
11029 |       monto_recibido: montoRecibido,
11030 |       tasa_descuento_intermediario: tasaDescuentoIntermediario,
11031 |       intermediario_id: intermediarioId,
11032 |     })
11033 |   ).then(() => ordenId).catch(() => null);
11034 | }
11035 | return guardarComision().then(() => ordenId).catch(() => null);
11036 | });
11037 | }
11038 |
11039 | function ensureInstrumentacionForOrden(ordenId) {
11040 |   if (!ordenId) return Promise.resolve(null);
11041 |   return client.from('instrumentacion').select('id').eq('orden_id', ordenId).maybeSingle().then((r) => {
11042 |     const instId = r.data && r.data.id;
11043 |     if (instId) return instId;
11044 |     return client.from('instrumentacion').insert({ orden_id: ordenId }).select('id').single().then((ins) => (ins.data ? ins.data.id : null));
11045 |   });
11046 | }
11047 |
11048 | function renderOrdenWizardInstrumentacion(instId) {
11049 |   const loadingEl = document.getElementById('orden-inst-loading');
11050 |   const wrapEl = document.getElementById('orden-inst-tabla-wrap');
11051 |   const tbody = document.getElementById('orden-inst-tbody');
11052 |   const acuerdoTexto = document.getElementById('orden-inst-acuerdo-texto');
11053 |   const instrumentadoTexto = document.getElementById('orden-inst-instrumentado-texto');
11054 |   const acuerdoAviso = document.getElementById('orden-inst-acuerdo-aviso');
11055 |   if (!loadingEl || !wrapEl || !tbody || !instId) return;
11056 |   const msgActualizando = document.getElementById('orden-inst-actualizando-msg');
11057 |   if (msgActualizando) msgActualizando.style.display = 'none';
11058 |   loadingEl.textContent = 'Cargando instrumentación...';
11059 |   loadingEl.style.display = 'block';
11060 |   wrapEl.style.display = 'none';
11061 |   tbody.innerHTML = '';
11062 |   if (acuerdoTexto) acuerdoTexto.textContent = '-';
11063 |   if (instrumentadoTexto) instrumentadoTexto.textContent = '-';
11064 |   if (acuerdoAviso) acuerdoAviso.textContent = '';
11065 |
11066 |   client.from('instrumentacion').select('orden_id, multicontraparte_manual').eq('id', instId).single().then((rInst) => {
11067 |     const multicontraparteManualInst = !!rInst.data && rInst.data.multicontraparte_manual;
11068 |     const ordenId = rInst.data && rInst.data.orden_id;
11069 |     if (!ordenId) {
11070 |       loadingEl.style.display = 'none';
11071 |       if (acuerdoTexto) acuerdoTexto.textContent = '-';
11072 |       if (instrumentadoTexto) instrumentadoTexto.textContent = '-';
11073 |       return;
11074 |     }
11075 |     Promise.all([
11076 |       client.from('ordenes').select('id, cliente_id, tipo_operacion_id, intermediario_id, moneda_recibida, monto_recibido, moneda_entregada, monto_entregado, cotizacion, tasa_
descuento_intermediario, estado, clientes(nombre), intermediarios(nombre), tipos_operacion(codigo, usa_intermediario)').eq('id', ordenId).single(),
11077 |       client.from('transacciones').select('id, numero, tipo, modo_pago_id, moneda, monto, cobrador, pagador, owner, estado, concepto, tipo_cambio, pagador_cliente_id, cobrador
_cliente_id, pagador_intermediario_id, cobrador_intermediario_id').eq('instrumentacion_id', instId).order('created_at', { ascending: true }),
11078 |       client.from('modos_pago').select('id, codigo, nombre'),
11079 |     ]).then(([rOrd, resTr, rModos]) => {
11080 |       loadingEl.style.display = 'none';
11081 |       wrapEl.style.display = 'block';
11082 |       const orden = rOrd.data || null;
11083 |       const ordenAnuladaWiz = orden && String(orden.estado || '') === 'anulada';
11084 |       const btnNuevaTrWiz = document.getElementById('orden-btn-nueva-transaccion');
11085 |       if (btnNuevaTrWiz) {
11086 |         btnNuevaTrWiz.style.display = (userPermissions.includes('ingresar_transacciones') && !ordenAnuladaWiz) ? '' : 'none';
11087 |       }
11088 |       const participantesEl = document.getElementById('orden-inst-participantes-texto');
11089 |       if (participantesEl) {
11090 |         if (orden) {
11091 |           const nombreCliente = orden.clientes?.nombre ?? (orden.cliente_id ? '-' : null);
11092 |           const nombreIntermediario = orden.intermediarios?.nombre ?? (orden.intermediario_id ? '-' : null);
11093 |           const partes = [];
11094 |           if (nombreCliente) partes.push('Cliente: ' + (nombreCliente || '-'));
11095 |           else if (orden.intermediario_id) partes.push('Cliente: Sin asignar');
11096 |           if (nombreIntermediario) partes.push('Intermediario: ' + nombreIntermediario);
11097 |           else partes.push('Intermediario: Sin asignar');
11098 |           participantesEl.textContent = partes.join(' · ');
11099 |         } else {
11100 |           participantesEl.textContent = '-';
11101 |         }
11102 |       }
11103 |       if (orden) {
11104 |         const monR = orden.moneda_recibida || 'USD';
11105 |         const monE = orden.moneda_entregada || 'USD';
11106 |         const mr = Number(orden.monto_recibido) || 0;
11107 |         const me = Number(orden.monto_entregado) || 0;

```

```

11108 |         if (acuerdoTexto) acuerdoTexto.textContent = `Recibir ${formatImporteDisplay(mr)} ${monR} · Entregar ${formatImporteDisplay(me)} ${monE}.`;
11109 |     } else {
11110 |         if (acuerdoTexto) acuerdoTexto.textContent = '-';
11111 |     }
11112 |
11113 |     if (resTr.error) {
11114 |         tbody.innerHTML = `<tr><td colspan="9">Error: ' + (resTr.error.message || '') + '</td></tr>`;
11115 |         if (instrumentadoTexto) instrumentadoTexto.textContent = '-';
11116 |         return;
11117 |     }
11118 |     let list = resTr.data || [];
11119 |
11120 |     const toJoinWizard = orden && orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
11121 |     const totalesOptsWizard = multicontraparteManualInst && orden && esTipoOpMulticontraparteElegibleDesdeOrden(orden, toJoinWizard) ? { totalesMulticontraparte: true } : un
defined;
11122 |
11123 |     const wrapMcInst = document.getElementById('orden-inst-multicontraparte-wrap');
11124 |     const chkMcInst = document.getElementById('orden-inst-multicontraparte-manual');
11125 |     if (wrapMcInst && chkMcInst) {
11126 |         const eligMcInst = !(orden && esTipoOpMulticontraparteElegibleDesdeOrden(orden, toJoinWizard));
11127 |         wrapMcInst.style.display = eligMcInst ? 'flex' : 'none';
11128 |         chkMcInst.checked = multicontraparteManualInst;
11129 |         chkMcInst.disabled = !ordenAnuladaWiz;
11130 |         chkMcInst.onchange = ordenAnuladaWiz
? null
11131 |         : () => {
11132 |             const checked = chkMcInst.checked;
11133 |             if (!checked) {
11134 |                 client.from('instrumentacion').update({ multicontraparte_manual: false, updated_at: new Date().toISOString() }).eq('id', instId).then((rUp) => {
11135 |                     if (rUp.error) {
11136 |                         chkMcInst.checked = true;
11137 |                         showToast('No se pudo guardar la opción multicontraparte.', 'error');
11138 |                         return;
11139 |                     }
11140 |                     showToast('Multicontraparte manual desactivada.', 'info', 4500);
11141 |                     renderOrdenWizardInstrumentacion(instId);
11142 |                 });
11143 |                 return;
11144 |             }
11145 |             if (!eligMcInst) return;
11146 |             if (!userPermissions.includes('eliminar_transacciones')) {
11147 |                 chkMcInst.checked = false;
11148 |                 showToast('Para activar multicontraparte manual hace falta permiso para eliminar transacciones: se quitan las sugeridas por el sistema si siguen con el esquema C
liente-Pandy (recibida) y Pandy-Cliente (entregada).', 'error');
11149 |                 return;
11150 |             }
11151 |             showConfirm(
11152 |                 'Se eliminarán, si existen, las transacciones sugeridas: ingreso del cliente a Pandy en la moneda recibida y egreso de Pandy al cliente en la moneda entregada. S
11153 | i ya las cambiaste (por ejemplo otro cobrador), no se borran. Luego podés cargar N pagos y contrapartes explícitas. ¿Activar multicontraparte manual?',
11154 |                 'Activar',
11155 |                 () => {
11156 |                     chkMcInst.disabled = true;
11157 |                     client.from('instrumentacion').update({ multicontraparte_manual: true, updated_at: new Date().toISOString() }).eq('id', instId).then((rUp) => {
11158 |                         if (rUp.error) {
11159 |                             chkMcInst.checked = false;
11160 |                             chkMcInst.disabled = false;
11161 |                             showToast('No se pudo guardar la opción multicontraparte.', 'error');
11162 |                             return;
11163 |                         }
11164 |                         return borrarTransaccionesPlantillaEstandarParaMulticontraparte(instId, orden)
11165 |                             .then((nDel) =>
11166 |                                 sincronizarCcYCajaDesdeOrden(ordenId)
11167 |                                     .then(() => actualizarEstadoOrden(ordenId))
11168 |                                     .then(() => {
11169 |                                         chkMcInst.disabled = false;
11170 |                                         showToast(
11171 |                                             nDel > 0
11172 |                                             ? `Multicontraparte manual activada. Se eliminaron ${nDel} transacción(es) sugerida(s) por el sistema.`
11173 |                                             : 'Multicontraparte manual activada. No había transacciones en el esquema sugerido (o ya las habías modificado). Podés cargar N transacciones y c
ontrapartes explícitas.',
11174 |                                             'success',
11175 |                                             5500
11176 |                                         );
11177 |                                         const vistaCc = document.getElementById('vista-cuenta-corriente');
11178 |                                         if (vistaCc && vistaCc.style.display !== 'none') loadCuentaCorriente();
11179 |                                         renderOrdenWizardInstrumentacion(instId);
11180 |                                     })
11181 |                                 )
11182 |                             .catch((err) => {
11183 |                                 chkMcInst.checked = false;
11184 |                                 chkMcInst.disabled = false;
11185 |                                 client.from('instrumentacion').update({ multicontraparte_manual: false, updated_at: new Date().toISOString() }).eq('id', instId).then(() => {
11186 |                                     renderOrdenWizardInstrumentacion(instId);
11187 |                                 });
11188 |                                 showToast('No se pudo completar la activación: ' + (err && (err.message || err.code) ? String(err.message || err.code) : 'error'), 'error');
11189 |                             });
11190 |                         });
11191 |                     },
11192 |                     () => { chkMcInst.checked = false; }
11193 |                 );
11194 |             };
11195 |         }
11196 |
11197 |     function renderWizardList(lista) {
11198 |         const { totalRecibido, totalEntregado } = totalesInstrumentacion(lista, orden, totalesOptsWizard);
11199 |         const labelEl = document.getElementById('orden-inst-instrumentado-label');
11200 |         if (instrumentadoTexto && orden) {

```

```

11201     const monR = orden.moneda_recibida || 'USD';
11202     const monE = orden.moneda_entregada || 'USD';
11203     const ejecutada = orden.estado === 'orden_ejecutada';
11204     if (labelEL labelEL.textContent = ejecutada ? 'Instrumentado:' : 'Instrumentación:');
11205     if (ejecutada) {
11206         instrumentadoTexto.textContent = 'Recibido ${formatImporteDisplay(totalRecibido)} ${monR} · Entregado ${formatImporteDisplay(totalEntregado)} ${monE}.';
11207     } else {
11208         const mr = Number(orden.monto_recibido) || 0;
11209         const me = Number(orden.monto_entregado) || 0;
11210         instrumentadoTexto.textContent = 'A recibir ${formatImporteDisplay(mr)} ${monR} - A entregar ${formatImporteDisplay(me)} ${monE}.';
11211     }
11212 } else {
11213     if (labelEL) labelEL.textContent = 'Instrumentado: ';
11214     if (instrumentadoTexto) instrumentadoTexto.textContent = '-';
11215 }
11216 if (acuerdoAviso && orden) {
11217     const avisoInst = textoAvisoFalta0ExcesoInstrumentacion(orden, lista, totalesOptsWizard);
11218     acuerdoAviso.textContent = avisoInst ? ` (${avisoInst})` : '';
11219 }
11220 const modosMap = {};
11221 (rModos.data || []).forEach((m) => { modosMap[m.id] = m; });
11222 const esc = (s) => (s == null ? '' : String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;'));
11223 const canEditarTr = userPermissions.includes('editar_transacciones') && !ordenAnuladaWiz;
11224 const estadoTrxCombo = (t) => {
11225     if (String(t.estado || '').toLowerCase() === 'anulada') return '<span class="badge badge-estado-anulada">Anulada</span>';
11226     const est = t.estado === 'ejecutada' ? 'ejecutada' : 'pendiente';
11227     return '<select class="combo-estado-transaccion-combo-estado-${est}" data-id="${t.id}" aria-label="Estado"><option value="pendiente">pendiente</option><option value="ejecutada">ejecutada</option></select>';
11228 };
11229 const estadoTexto = (t) => (String(t.estado || '').toLowerCase() === 'anulada' ? 'Anulada' : (t.estado === 'ejecutada' ? 'Ejecutada' : 'Pendiente'));
11230 const listaSorted = sortTransaccionesPorNumero(lista);
11231 const selTrxWizardCols = 'id, numero, tipo, modo_pago_id, moneda, monto, cobrador, pagador, owner, estado, concepto, tipo_cambio, pagador_cliente_id, cobrador_cliente_id, pagador_intermediario_id, cobrador_intermediario_id';
11232 function paintWizardTabla(maps) {
11233     const cobradorL = (t) => transaccionParticipanteCeldaHtml(t, orden, 'cobrador', maps);
11234     const pagadorL = (t) => transaccionParticipanteCeldaHtml(t, orden, 'pagador', maps);
11235     if (listaSorted.length === 0) {
11236         tbody.innerHTML = '<tr><td colspan="9">Todavía no hay transacciones.</td></tr>';
11237         return;
11238     }
11239     const montoCell = (t) => {
11240         if (!canEditarTr) return '<td>${formatImporteDisplay(t.monto)}</td>';
11241         const val = formatImporteParaInput(t.monto);
11242         return '<td><input type="text" class="input-monto-transaccion-tabla" data-id="${esc(t.id)}" value="${esc(val)}" inputmode="decimal" aria-label="Monto ${esc(t.moneda)}></td>';
11243     };
11244     const modoPagoCell = (t) => {
11245         if (!canEditarTr) {
11246             const modo = modosMap[t.modo_pago_id];
11247             return '<td>${esc(modo ? modo.nombre : '-')}</td>';
11248         }
11249         const opciones = (rModos.data || []).map((m) => '<option value="${m.id}"${t.modo_pago_id === m.id ? ' selected' : ''}>${esc(m.nombre)}</option>').join('');
11250         return '<td><select class="combo-modo-pago-transaccion-tabla" data-id="${esc(t.id)}" aria-label="Modo de pago">${opciones}</select></td>';
11251     };
11252     tbody.innerHTML = listaSorted.map((t) => {
11253         return '<tr data-id="${t.id}" data-numero="${t.numero != null ? esc(String(t.numero)) : ''}">
11254             <td>${t.numero != null ? esc(String(t.numero)) : '-'}</td>
11255             <td>${tipoTransaccionHtml(t.tipo)}</td>
11256             ${modoPagoCell(t)}
11257             <td>${esc(t.moneda)}</td>
11258             ${montoCell(t)}
11259             <td>${pagadorL(t)}</td>
11260             <td>${cobradorL(t)}</td>
11261             <td>${canEditarTr ? estadoTrxCombo(t) : estadoTexto(t)}</td>
11262             <td>${canEditarTr ? '<button type="button" class="btn-editar-transaccion-ordenwizard" data-id="${t.id}" title="Editar concepto y demás campos">Editar</button>' : ''}</td>
11263         </tr>';
11264     }).join('');
11265     if (canEditarTr) {
11266         tbody.querySelectorAll('.combo-estado-transaccion').forEach((sel) => {
11267             sel.addEventListener('change', function() { cambiarEstadoTransaccion(this.getAttribute('data-id'), this.value, instId, this); });
11268         });
11269         tbody.querySelectorAll('.combo-modo-pago-transaccion-tabla').forEach((sel) => {
11270             sel.addEventListener('change', function() {
11271                 const id = this.getAttribute('data-id');
11272                 const prev = lista.find((r) => r.id === id);
11273                 if (!prev || this.value === prev.modo_pago_id) return;
11274                 const selEl = this;
11275                 guardarSoloModoPagoTransaccion(id, this.value, () => {
11276                     client.from('transacciones').select(selTrxWizardCols).eq('instrumentacion_id', instId).order('created_at', { ascending: true }).then((r2) => {
11277                         list = r2.data || [];
11278                         renderWizardList(list);
11279                     });
11280                 }, () => { selEl.value = prev.modo_pago_id; });
11281             });
11282         });
11283         tbody.querySelectorAll('.input-monto-transaccion-tabla').forEach((input) => {
11284             input.addEventListener('blur', function() {
11285                 const id = this.getAttribute('data-id');
11286                 const prev = lista.find((r) => r.id === id);
11287                 if (!prev || parseFloat(this.value) === Number(prev.monto)) return;
11288                 guardarSoloMontoTransaccion(id, this.value, () => {
11289                     client.from('transacciones').select(selTrxWizardCols).eq('instrumentacion_id', instId).order('created_at', { ascending: true }).then((r2) => {
11290                         list = r2.data || [];
11291                         renderWizardList(list);
11292                     });
11293                 });
11294             });
11295         });

```

```

11294 |     });
11295 |   });
11296 |   tbody.querySelectorAll('.btn-editar-transaccion-ordenwizard').forEach((btn) => {
11297 |     btn.addEventListener('click', () => {
11298 |       const row = lista.find((r) => r.id === btn.getAttribute('data-id'));
11299 |       if (row) openModalTransaccion(row, instId);
11300 |     });
11301 |   });
11302 | }
11303 |
11304 | if (listaSorted.length === 0) {
11305 |   paintWizardTabla({ clientesById: {}, intermediariosById: {} });
11306 | } else {
11307 |   fetchMapsNombresParticipantesTransacciones(orden, listaSorted).then(paintWizardTabla);
11308 | }
11309 |
11310 |
11311 | if (list.length === 0 && orden && orden.tipo_operacion_id && !ordenAnuladaWiz) {
11312 |   client.from('tipos_operacion').select('codigo, moneda_in, moneda_out').eq('id', orden.tipo_operacion_id).single().then((rTipo) => {
11313 |     const row = rTipo.data || {};
11314 |     const codigo = row.codigo || '';
11315 |     const miW = (row.moneda_in || '').toString().toUpperCase().trim();
11316 |     const moW = (row.moneda_out || '').toString().toUpperCase().trim();
11317 |     const patronTcW = patronTipoCambioOrden(miW, moW);
11318 |     const esUsdUsdW = miW === 'USD' && moW === 'USD';
11319 |     if (esTipoOperacionChequeArs(codigo, row.moneda_in, row.moneda_out) && orden.intermediario_id) {
11320 |       return autoCompleteLetarInstrumentacionChequeConIntermediario(ordenId, instId, orden);
11321 |     }
11322 |     const autoDosTxSinInt =
11323 |       !multicontraparteManualInst &&
11324 |       !orden.intermediario_id &&
11325 |       (esUsdUsdW || patronTcW || esTipoOperacionChequeArs(codigo, row.moneda_in, row.moneda_out));
11326 |     if (autoDosTxSinInt) {
11327 |       return autoCompleteLetarInstrumentacionSinIntermediario(ordenId, instId, orden);
11328 |     }
11329 |     if (orden.intermediario_id && (esUsdUsdW || patronTcW) && !esTipoOperacionChequeArs(codigo, row.moneda_in, row.moneda_out)) {
11330 |       return autoCompleteLetarInstrumentacionUsdUsdConIntermediario(ordenId, instId, orden);
11331 |     }
11332 |     return Promise.resolve();
11333 |   }).then(() =>
11334 |     client.from('transacciones').select('id, numero, tipo, modo_pago_id, moneda, monto, cobrador, pagador, owner, estado, concepto, tipo_cambio').eq('instrumentacion_id'
, instId).order('created_at', { ascending: true })
11335 |     ).then((r2) => {
11336 |       list = r2.data || [];
11337 |       renderWizardList(list);
11338 |     });
11339 |   } else {
11340 |     renderWizardList(list);
11341 |   }
11342 | });
11343 | });
11344 | }
11345 |
11346 | function saveOrden(aceptaComisionCero = false) {
11347 |   const idEl = document.getElementById('orden-id');
11348 |   const id = idEl && idEl.value ? idEl.value.trim() : '';
11349 |   const canIngresarOrden = userPermissions.includes('ingresar_orden');
11350 |   const canEditarOrden = userPermissions.includes('editar_orden');
11351 |   const canEditarEstadoOrden = userPermissions.includes('editar_estado_orden');
11352 |   const estado = document.getElementById('orden-estado')?.value || '';
11353 |   if (id) {
11354 |     if (!canEditarOrden && !canEditarEstadoOrden && !(estado === 'anulada' && userPermissions.includes('anular_orden'))) {
11355 |       showToast('No tenés permiso para editar órdenes.', 'error');
11356 |       return;
11357 |     }
11358 |   } else {
11359 |     if (!canIngresarOrden) {
11360 |       showToast('No tenés permiso para crear órdenes.', 'error');
11361 |       return;
11362 |     }
11363 |   }
11364 |   if (id && estado === 'anulada') {
11365 |     if (!userPermissions.includes('anular_orden')) {
11366 |       showToast('No tenés permiso para anular órdenes.', 'error');
11367 |       return;
11368 |     }
11369 |     solicitarConfirmacionYAnularOrden(id, { onExito: () => refrescarVistasTrasAnularOrden(true) });
11370 |     return;
11371 |   }
11372 |   const clienteId = document.getElementById('orden-cliente').value.trim() || null;
11373 |   const fecha = document.getElementById('orden-fecha').value;
11374 |   const tipoOperacionId = document.getElementById('orden-tipo-operacion')?.value?.trim() || null;
11375 |   const selTipoOptSave = document.getElementById('orden-tipo-operacion')?.selectedOptions?.[0];
11376 |   const usaIntermediarioSave = selTipoOptSave ? (selTipoOptSave.getAttribute('data-usa-intermediario') === 'true') : false;
11377 |   let intermediarioId = document.getElementById('orden-intermediario')?.value?.trim() || null;
11378 |   if (!usaIntermediarioSave) intermediarioId = null;
11379 |   const operacionDirecta = !intermediarioId;
11380 |   const monedaRecibida = document.getElementById('orden-moneda-recibida').value;
11381 |   const monedaEntregada = document.getElementById('orden-moneda-entregada').value;
11382 |   const montoRecibido = parseImporteInput(document.getElementById('orden-monto-recibido').value);
11383 |   const montoEntregado = parseImporteInput(document.getElementById('orden-monto-entregado').value);
11384 |   const cotizacionRaw = document.getElementById('orden-cotizacion').value.trim();
11385 |   const cotizacion = cotizacionRaw ? parseImporteInput(cotizacionRaw) : null;
11386 |   const observaciones = document.getElementById('orden-observaciones').value.trim() || null;
11387 |
11388 |   if (!clienteId && !intermediarioId) {
11389 |     showToast('Definí participantes: elegí un cliente, un intermediario o ambos.', 'error');

```



```

11390|     return;
11391| }
11392| if (!tipoOperacionId) {
11393|     showToast('Elegí un tipo de operación.', 'error');
11394|     return;
11395| }
11396| if (usaIntermediarioSave && !intermediarioId) {
11397|     showToast('Para este tipo de operación es obligatorio elegir un intermediario.', 'error');
11398|     return;
11399| }
11400| const tipoCodigoSavePre = selTipoOptSave ? (selTipoOptSave.getAttribute('data-codigo') || '') : '';
11401| if (String(tipoCodigoSavePre).trim().toUpperCase() === 'USD-USD' && usaIntermediarioSave && intermediarioId && !id && !ordenIntPatronExplicitoElegido()) {
11402|     showToast('Elegí la instrumentación sugerida (con intermediario) antes de guardar.', 'error');
11403|     return;
11404| }
11405| if (!fecha || isNaN(montoRecibido) || montoRecibido <= 0 || isNaN(montoEntregado) || montoEntregado <= 0) {
11406|     showToast('Completá fecha, monto recibido y monto entregado (números positivos).', 'error');
11407|     return;
11408| }
11409|
11410| const selTipoOpt = document.getElementById('orden-tipo-operacion')?.selectedOptions?.[0];
11411| const tipoCodigo = selTipoOpt ? (selTipoOpt.getAttribute('data-codigo') || '') : '';
11412| const tipoCodigoU = String(tipoCodigo).trim().toUpperCase();
11413| const esChequeArsSave = esChequeArsDesdeSelectOption(selTipoOpt);
11414| const patronTcSaveOrd = patronTipoCambioOrden(monedaRecibida, monedaEntregada);
11415| if (patronTcSaveOrd && (!cotizacion || !(cotizacion > 0))) {
11416|     showToast('En operaciones con cruce contra USD el tipo de cambio del acuerdo es obligatorio y debe ser mayor a cero.', 'error');
11417|     return;
11418| }
11419| if (tipoCodigoU === 'USD-USD' && montoRecibido <= montoEntregado) {
11420|     showToast('En USD-USD el monto a recibir debe ser mayor al monto a entregar (la diferencia es la comisión).', 'error');
11421|     return;
11422| }
11423| if (esChequeArsSave) {
11424|     if (montoRecibido <= montoEntregado) {
11425|         showToast('En operación con cheque (CHEQUE-ARS) el monto a recibir debe ser mayor al monto a entregar (descuento acuerdo).', 'error');
11426|         return;
11427|     }
11428|     const tasaPctRaw = document.getElementById('orden-tasa-descuento-intermediario')?.value?.trim() || '';
11429|     const tasaPct = tasaPctRaw ? parseImporteInput(tasaPctRaw) : null;
11430|     if (typeof tasaPct !== 'number' || isNaN(tasaPct) || tasaPct <= 0 || tasaPct >= 100) {
11431|         showToast('En operación con cheque (CHEQUE-ARS) la tasa de descuento del intermediario es obligatoria (ej. 1 para 1%, entre 0 y 100).', 'error');
11432|         return;
11433|     }
11434| }
11435| if (tipoCodigoU === 'USD-USD' && intermediarioId && ordenComisionFijaNachoUsdParaPersistir(tipoCodigo, intermediarioId)) {
11436|     const nr = document.querySelector('input[name="orden-usd-nacho-comision-usd"]:checked');
11437|     if (!nr) {
11438|         toastElegirComisionFijaIntermediarioUsd();
11439|         return;
11440|     }
11441| }
11442| if (tipoCodigoU === 'USD-USD' && intermediarioId && ordenUsdIntMostrarTasaIntermediarioEnWizard()) {
11443|     const rawUsdTs = document.getElementById('orden-tasa-descuento-intermediario')?.value?.trim() || '';
11444|     const tUsdInts = rawUsdTs ? parseImporteInput(rawUsdTs) : null;
11445|     if (typeof tUsdInts !== 'number' || isNaN(tUsdInts) || tUsdInts <= 0 || tUsdInts >= 100) {
11446|         showToast('En USD-USD con intermediario completá la tasa del intermediario (% sobre el importe).', 'error');
11447|         return;
11448|     }
11449| }
11450| const comisionUsd = tipoCodigoU === 'USD-USD' ? montoRecibido - montoEntregado : (esChequeArsSave ? montoRecibido - montoEntregado : (esPatronCompraFiatConTc(patronTcSaveOrd
) && cotizacion > 0 ? (montoRecibido / cotizacion) - montoEntregado : (esPatronVendeFiatConTc(patronTcSaveOrd) && cotizacion > 0 ? montoRecibido - (montoEntregado / cotizacion
) : null)));
11451| if (tipoCodigoU === 'USD-USD' && intermediarioId && ordenComisionFijaNachoUsdParaPersistir(tipoCodigo, intermediarioId) && comisionUsd !== null) {
11452|     const nchkS = document.querySelector('input[name="orden-usd-nacho-comision-usd"]:checked');
11453|     const fjs = nchkS ? Number(nchkS.value) || 0 : 0;
11454|     if (fjs > comisionUsd + 1e-6) {
11455|         showToast('La comisión fija del intermediario no puede superar el beneficio del acuerdo.', 'error');
11456|         return;
11457|     }
11458| }
11459|
11460| const pctPandy = parseImporteInput(document.getElementById('orden-comision-pandy-pct')?.value || '100');
11461| const pctInt = parseImporteInput(document.getElementById('orden-comision-intermediario-pct')?.value || '0');
11462| const tieneSplitVisible = ordenWrapComisionSplitEsVisible();
11463| if ((tipoCodigoU === 'USD-USD' || patronTcSaveOrd || esChequeArsSave) && intermediarioId && tieneSplitVisible) {
11464|     const a = Number(pctPandy);
11465|     const b = Number(pctInt);
11466|     if (isNaN(a) || isNaN(b) || a < 0 || b < 0 || a > 100 || b > 100 || Math.abs((a + b) - 100) > 1e-6) {
11467|         showToast('La distribución de comisión debe sumar 100% (Pandy + Intermediario).', 'error');
11468|         return;
11469|     }
11470| }
11471| if (intermediarioId && tieneSplitVisible && (Number(pctInt) || 0) < 1e-6 && !aceptaComisionCero) {
11472|     showConfirm('La comisión del intermediario es cero. ¿Deseás guardar la orden igual?', 'Si, guardar', () => saveOrden(true));
11473|     return;
11474| }
11475|
11476| const tasaDescuentoIntPctSave = document.getElementById('orden-tasa-descuento-intermediario')?.value?.trim();
11477| let tasaDescuentoIntermediarioSave = null;
11478| if (esChequeArsSave && tasaDescuentoIntPctSave) {
11479|     tasaDescuentoIntermediarioSave = parseImporteInput(tasaDescuentoIntPctSave) / 100;
11480| } else if (tipoCodigoU === 'USD-USD' && intermediarioId && ordenUsdIntMostrarTasaIntermediarioEnWizard() && tasaDescuentoIntPctSave) {
11481|     tasaDescuentoIntermediarioSave = parseImporteInput(tasaDescuentoIntPctSave) / 100;
11482| }
11483| const estadoFinal = id ? estado : 'pendiente_instrumentar';
11484| const payload = {

```

```

11485 |     cliente_id: clienteId,
11486 |     fecha,
11487 |     estado: estadoFinal,
11488 |     tipo_operacion_id: tipoOperacionId || null,
11489 |     operacion_directa: operacionDirecta,
11490 |     intermediario_id: intermediarioId,
11491 |     moneda_recibida: monedaRecibida,
11492 |     moneda_entregada: monedaEntregada,
11493 |     monto_recibido: montoRecibido,
11494 |     monto_entregado: montoEntregado,
11495 |     cotizaci3n: cotizaci3n,
11496 |     tasa_descuento_intermediario: tasaDescuentoIntermediarioSave,
11497 |     observaciones,
11498 |     usuario_id: currentUserId,
11499 |     updated_at: new Date().toISOString(),
11500 | };
11501 |
11502 | function hacerUpdateOrden(estadoPersistir) {
11503 |     const p = { ...payload, estado: estadoPersistir };
11504 |     return id ? client.from('ordenes').update(p).eq('id', id) : client.from('ordenes').insert(p).select('id');
11505 | }
11506 |
11507 | const prom = id
11508 |   ? client
11509 |     .from('ordenes')
11510 |     .select('estado')
11511 |     .eq('id', id)
11512 |     .single()
11513 |     .then((rOrdActual) => {
11514 |       if (rOrdActual.data && rOrdActual.data.estado === 'anulada') {
11515 |         return hacerUpdateOrden('anulada');
11516 |       }
11517 |       return client.from('instrumentacion').select('id, multicontraparte_manual').eq('orden_id', id).maybeSingle().then((rInst) => {
11518 |         const instId = rInst.data && rInst.data.id;
11519 |         const mcFlag = !(rInst.data && rInst.data.multicontraparte_manual);
11520 |         const promTr = instId
11521 |           ? client
11522 |             .from('transacciones')
11523 |             .select('id, estado, tipo, moneda, monto, cobrador, pagador, pagador_cliente_id, cobrador_cliente_id')
11524 |             .eq('instrumentacion_id', instId)
11525 |             : Promise.resolve({ data: [] });
11526 |         const promTipo = tipoOperacionId ? client.from('tipos_operacion').select('codigo, usa_intermediario').eq('id', tipoOperacionId).single() : Promise.resolve({ data:
11527 | null });
11528 |         return Promise.all([promTr, promTipo]).then(([rTr, rTipo]) => {
11529 |           const list = rTr.data || [];
11530 |           const toJ = rTipo.data;
11531 |           const ordenParaCalc = {
11532 |             cliente_id: clienteId,
11533 |             intermediario_id: intermediarioId,
11534 |             moneda_recibida: monedaRecibida,
11535 |             monto_recibido: montoRecibido,
11536 |             moneda_entregada: monedaEntregada,
11537 |             monto_entregado: montoEntregado,
11538 |             cotizaci3n,
11539 |           };
11540 |           const totMc = mcFlag && esTipoOpMulticontraparteElegibleDesdeOrden(ordenParaCalc, toJ);
11541 |           const { estado: estadoCalculado } = calcularEstadoOrden(list, ordenParaCalc, { totalesMulticontraparte: totMc });
11542 |           return hacerUpdateOrden(estadoCalculado);
11543 |         });
11544 |       });
11545 |     : insertOrdenConProximoNumero(payload);
11546 |
11547 | prom.then((res) => {
11548 |   if (res.error) {
11549 |     showToast('Error: ' + (res.error.message || 'No se pudo guardar.'), 'error');
11550 |     return;
11551 |   }
11552 |   const ordenId = id || (res.data && res.data[0] && res.data[0].id);
11553 |   if (!ordenId) {
11554 |     ejecutarCierreModalOrden();
11555 |     loadOrdenes();
11556 |     return;
11557 |   }
11558 |   if (ordenIdBorradorParaEliminar === ordenId) ordenIdBorradorParaEliminar = null;
11559 |   const conceptoComision = tipoCodigo === 'USD-ARS' ? 'Comisi3n ARS-ARS' : 'Comisi3n USD-ARS' : (esChequeArsSave ? 'Comisi3n ARS-ARS' : 'Comisi3n USD-USD');
11560 |   const comisionMoneda = esChequeArsSave ? 'ARS' : 'USD';
11561 |   function guardarComisionYContinuar(continuar) {
11562 |     if ((tipoCodigoU === 'USD-USD' || esChequeArsSave) && comisionUsd != null && comisionUsd > 0) {
11563 |       client.from('comisiones_orden').delete().eq('orden_id', ordenId).then(() => {
11564 |         let montoPandy;
11565 |         let montoInter;
11566 |         if (tipoCodigoU === 'USD-USD' && intermediarioId) {
11567 |           const importeAc = parseImporteInput(document.getElementById('orden-importe-cheque')?.value);
11568 |           if (ordenComisionFijaNachoUsdParaPersistir(tipoCodigo, intermediarioId)) {
11569 |             const n = document.querySelector('input[name="orden-usd-nacho-comision-usd"]:checked');
11570 |             const fija = n ? Number(n.value) || 0 : 0;
11571 |             montoInter = fija;
11572 |             montoPandy = comisionUsd - fija;
11573 |             persistirPreferenciaComisionFijaInterUsd(fija);
11574 |           } else if (ordenUsdIntMostrarTasaIntermediarioEnWizard()) && typeof importeAc === 'number' && !isNaN(importeAc) && importeAc > 0) {
11575 |             const tI = parseImporteInput(document.getElementById('orden-tasa-descuento-intermediario')?.value);
11576 |             if (typeof tI === 'number' && !isNaN(tI)) {
11577 |               const totalCom = comisionUsd;
11578 |               const brutoInter = importeAc * (tI / 100);
11579 |               montoInter = Math.min(Math.max(0, brutoInter), Math.max(0, totalCom));
11580 |               montoPandy = Math.max(0, totalCom - montoInter);

```

```

11581 |         } else {
11582 |             montoPandy = comisionUsd;
11583 |             montoInter = 0;
11584 |         }
11585 |     } else {
11586 |         montoPandy = comisionUsd;
11587 |         montoInter = 0;
11588 |     }
11589 | } else {
11590 |     const a = intermediarioId ? Number(pctPandy) : 100;
11591 |     const b = intermediarioId ? Number(pctInt) : 0;
11592 |     montoPandy = comisionUsd * (a / 100);
11593 |     montoInter = comisionUsd * (b / 100);
11594 | }
11595 | if (montoPandy < -1e-6) {
11596 |     showToast('La comisión fija del intermediario no puede superar el beneficio del acuerdo.', 'error');
11597 |     return;
11598 | }
11599 | const rows = [
11600 |     { orden_id: ordenId, moneda: comisionMoneda, monto: Math.max(0, montoPandy), concepto: conceptoComision, beneficiario: 'pandy', intermediario_id: null },
11601 | ];
11602 | if (intermediarioId && montoInter > 0) {
11603 |     rows.push({ orden_id: ordenId, moneda: comisionMoneda, monto: montoInter, concepto: conceptoComision, beneficiario: 'intermediario', intermediario_id: intermediari
oid });
11604 | }
11605 | client.from('comisiones_orden').insert(rows).then((rCom) => {
11606 |     if (rCom.error) console.warn('Comisión no guardada:', rCom.error.message);
11607 |     continuar();
11608 | });
11609 | });
11610 | } else continuar();
11611 | }
11612 |
11613 | // Orden nueva: crear instrumentación (1:1 con la orden)
11614 | if (!id) {
11615 |     guardarComisionYContinuar(() => {
11616 |         client.from('instrumentacion').insert({ orden_id: ordenId }).then((rInst) => {
11617 |             if (rInst.error) console.warn('Instrumentación no creada:', rInst.error.message);
11618 |             ejecutarCierreModalOrden();
11619 |             loadOrdenes();
11620 |         });
11621 |     });
11622 |     return;
11623 | }
11624 | // Al editar, también guardar comisión; si es CHEQUE con intermediario, actualizar monto 4.ª transacción por tasa; cierre y refresco (reversión CC/caja solo vía Anular ord
en).
11625 | guardarComisionYContinuar(() => {
11626 |     const promTasa = (esChequeArsSave && intermediarioId && tasaDescuentoIntermediarioSave != null)
11627 |         ? actualizarTasaTransaccionIngresoIntermediarioCheque(ordenId, { monto_recibido: montoRecibido, tasa_descuento_intermediario: tasaDescuentoIntermediarioSave, intermedi
ario_id: intermediarioId })
11628 |         : Promise.resolve();
11629 |     promTasa.then(() => {
11630 |         ejecutarCierreModalOrden();
11631 |         loadOrdenes();
11632 |         loadCajas();
11633 |         const vistaCcPostSave = document.getElementById('vista-cuenta-corriente');
11634 |         if (vistaCcPostSave && vistaCcPostSave.style.display !== 'none') loadCuentaCorriente();
11635 |     });
11636 | });
11637 | });
11638 | }
11639 |
11640 | function setupModalOrden() {
11641 |     const backdrop = document.getElementById('modal-orden-backdrop');
11642 |     const btnClose = document.getElementById('modal-orden-close');
11643 |     const btnCancel = document.getElementById('modal-orden-cancelar');
11644 |     const form = document.getElementById('form-orden');
11645 |     const btnNuevo = document.getElementById('btn-nueva-orden');
11646 |     if (btnClose) btnClose.addEventListener('click', () => solicitarCierreModalOrden({ modo: 'salir', refrescarOrdenes: true }));
11647 |     if (btnCancel) btnCancel.addEventListener('click', () => {
11648 |         const ordenId = (document.getElementById('orden-id') && document.getElementById('orden-id').value || '').trim();
11649 |         const instId = ordenWizardInstrumentacionIdActual;
11650 |         if (ordenId && instId) {
11651 |             solicitarCierreModalOrden({ modo: 'salir', refrescarOrdenes: true });
11652 |             return;
11653 |         }
11654 |         showConfirm('Los datos se perderán. ¿Continuar?', 'Sí, salir', () => {
11655 |             ejecutarCierreModalOrden();
11656 |             loadOrdenes();
11657 |         });
11658 |     });
11659 |     // Listo / Cerrar del paso instrumentación: solo delegación en document (evita doble disparo con addEventListener en el botón).
11660 |     // Delegación en document (capture): click y mousedown por si el click no llega (p. ej. en algunos entornos)
11661 |     function ordenWizardCerrar(btnId, back) {
11662 |         try {
11663 |             if (btnId === 'orden-btn-cerrar-wizard') solicitarCierreModalOrden({ modo: 'listo', refrescarOrdenes: true });
11664 |             else if (btnId === 'orden-btn-cancelar-wizard') solicitarCierreModalOrden({ modo: 'salir', refrescarOrdenes: true });
11665 |         } catch (err) {
11666 |             if (typeof console !== 'undefined' && console.error) console.error('ordenWizardCerrar', err);
11667 |             if (back) { back.classList.remove('activo'); document.body.classList.remove('modal-orden-abierto'); }
11668 |         }
11669 |     }
11670 |     function ordenWizardCerrarHandler(evName) {
11671 |         return function (e) {
11672 |             const btn = e.target && e.target.closest ? e.target.closest('button') : null;
11673 |             if (!btn || (btn.id !== 'orden-btn-cerrar-wizard' && btn.id !== 'orden-btn-cancelar-wizard')) return;
11674 |             const back = document.getElementById('modal-orden-backdrop');

```

```

11675 |         if (!back || !back.classList.contains('activo')) return;
11676 |         e.preventDefault();
11677 |         e.stopPropagation();
11678 |         ordenWizardCerrar(btn.id, back);
11679 |     });
11680 | }
11681 | document.addEventListener('click', ordenWizardCerrarHandler('click'), true);
11682 | document.addEventListener('keydown', function (e) {
11683 |     if (e.key !== 'Escape') return;
11684 |     const back = document.getElementById('modal-orden-backdrop');
11685 |     if (!back || !back.classList.contains('activo')) return;
11686 |     const lb = document.getElementById('orden-tipo-operacion-listbox');
11687 |     if (lb && !lb.hidden) {
11688 |         e.preventDefault();
11689 |         e.stopPropagation();
11690 |         closeOrdenTipoOperacionListbox();
11691 |         return;
11692 |     }
11693 |     e.preventDefault();
11694 |     solicitarCierreModalOrden({ modo: 'salir', refrescarOrdenes: true });
11695 | }, true);
11696 | if (backdrop) setupBackdropCloseOnlyOnRealClick(backdrop, () => solicitarCierreModalOrden({ modo: 'salir', refrescarOrdenes: true }));
11697 | if (form) form.addEventListener('submit', (e) => { e.preventDefault(); saveOrden(); });
11698 | if (btnNuevo) btnNuevo.addEventListener('click', () => openModalOrden(null));
11699 | ['orden-cotizacion', 'orden-monto-recibido', 'orden-monto-entregado', 'orden-importe-cheque', 'orden-tasa-descuento-cliente', 'orden-tasa-descuento-intermediario', 'orden-co
11700 | mision-pandy-pct', 'orden-comision-intermediario-pct'].forEach((id) => {
11701 |     const el = document.getElementById(id);
11702 |     if (el) {
11703 |         el.addEventListener('focus', () => el.classList.add('orden-field-editing'));
11704 |         el.addEventListener('blur', () => el.classList.remove('orden-field-editing'));
11705 |     }
11706 | });
11707 | const comboUi = document.querySelector('.orden-tipo-operacion-combo-ui');
11708 | const comboBtn = document.getElementById('orden-tipo-operacion-combo-btn');
11709 | const comboList = document.getElementById('orden-tipo-operacion-listbox');
11710 | const comboWrap = document.querySelector('.orden-tipo-operacion-combo-wrap');
11711 | if (comboUi && comboBtn && comboList && comboWrap && comboUi.dataset.comboBound !== '1') {
11712 |     comboUi.dataset.comboBound = '1';
11713 |     function reposicionarListboxTipoOpSiAbierto() {
11714 |         if (!comboList || comboList.hidden) return;
11715 |         positionOrdenTipoOperacionListbox();
11716 |     }
11717 |     window.addEventListener('scroll', reposicionarListboxTipoOpSiAbierto, true);
11718 |     window.addEventListener('resize', reposicionarListboxTipoOpSiAbierto);
11719 |     comboBtn.addEventListener('click', (ev) => {
11720 |         ev.preventDefault();
11721 |         ev.stopPropagation();
11722 |         if (!comboList.hidden) {
11723 |             closeOrdenTipoOperacionListbox();
11724 |             return;
11725 |         }
11726 |         rebuildOrdenTipoOperacionListbox();
11727 |         comboList.hidden = false;
11728 |         comboBtn.setAttribute('aria-expanded', 'true');
11729 |         requestAnimationFrame(() => {
11730 |             positionOrdenTipoOperacionListbox();
11731 |         });
11732 |     });
11733 |     comboList.addEventListener('click', (ev) => {
11734 |         const optBtn = ev.target && ev.target.closest('.orden-tipo-operacion-option') : null;
11735 |         if (!optBtn) return;
11736 |         ev.preventDefault();
11737 |         ev.stopPropagation();
11738 |         const val = optBtn.getAttribute('data-value');
11739 |         const sel = document.getElementById('orden-tipo-operacion');
11740 |         closeOrdenTipoOperacionListbox();
11741 |         if (sel && val !== null) {
11742 |             sel.value = val;
11743 |             sel.dispatchEvent(new Event('change', { bubbles: true }));
11744 |         }
11745 |         comboBtn.focus();
11746 |     });
11747 |     document.addEventListener(
11748 |         'mousedown',
11749 |         (ev) => {
11750 |             if (!comboList || comboList.hidden) return;
11751 |             if (comboWrap.contains(ev.target)) return;
11752 |             closeOrdenTipoOperacionListbox();
11753 |         },
11754 |         true
11755 |     );
11756 | }
11757 | }
11758 |
11759 | // --- Cargar orden por chat (solo local / MVP) ---
11760 | let chatOrdenClientes = [];
11761 | let chatOrdenTipos = [];
11762 | let chatOrdenUltimaInterpretacion = null;
11763 | let chatOrdenAbiertoDesdePanel = false;
11764 |
11765 | function interpretarTextoOrden(texto, clientes, tipos) {
11766 |     const t = (texto || '').trim();
11767 |     if (!t) return { error: 'Escribí algo para interpretar.' };
11768 |
11769 |     const normalizarMoneda = (s) => {
11770 |         if (!s) return null;

```

```

11771|     const u = (s + '').toUpperCase().replace(/0/g, '0');
11772|     if (u.includes('ARS')) || u.includes('PES0')) return 'ARS';
11773|     if (u.includes('USD')) || u.includes('DOLAR')) return 'USD';
11774|     if (u.includes('EUR')) || u.includes('EURO')) return 'EUR';
11775|     return null;
11776| };
11777|
11778| const parseNum = (str) => {
11779|   if (!str) return NaN;
11780|   const s = (str + '').trim().replace(/\. /g, '').replace(',', '.');
11781|   const n = parseFloat(s, 10);
11782|   return isNaN(n) ? NaN : n;
11783| };
11784|
11785| // Tipo de cambio: "a tc 1500", "tc 1500", "a 1500", "cotización 1500" (evitar capturar el monto de "recibo 3000")
11786| let cotizacion = null;
11787| const reTc = /(?:a\s+tc\s+|tc\s+|tipo\s+(?:de\s+cambio\s+)?|cotización\s+|a\s+)(\d[\d.,]*)/gi;
11788| const matchTc = reTc.exec(t);
11789| if (matchTc && matchTc[1]) {
11790|   const n = parseImporteInput(matchTc[1]);
11791|   if (!isNaN(n) && n > 0) cotizacion = n;
11792| }
11793|
11794| // "recibo 3000 usd" / "recibimos 1500000 ars"
11795| const reRecibo = /\brecib(o|imos|i|en)?s+(\d[\d.,]*)\s*(ARS|USD|EUR|pesos?|dólares?|dolares?|euros?)/gi;
11796| const reciboMatch = reRecibo.exec(t);
11797| let montoRecibido = null, monedaRecibida = null;
11798| if (reciboMatch && reciboMatch[2]) {
11799|   montoRecibido = parseNum(reciboMatch[2]);
11800|   monedaRecibida = normalizarMoneda(reciboMatch[3] || '');
11801| }
11802|
11803| // "entrego 4500000 ars" / "entregamos ars" (sin número: se calcula con TC)
11804| const reEntregoMonto = /\bentreg(o|amos|á|an)?s+(\d[\d.,]*)\s*(ARS|USD|EUR|pesos?|dólares?|dolares?|euros?)/gi;
11805| const reEntregoSoloMoneda = /\bentreg(o|amos|á|an)?s+(ARS|USD|EUR|pesos?|dólares?|dolares?|euros?)\b/gi;
11806| let montoEntregado = null, monedaEntregada = null;
11807| const entregoMontoMatch = reEntregoMonto.exec(t);
11808| const entregoSoloMatch = reEntregoSoloMoneda.exec(t);
11809| if (entregoMontoMatch && entregoMontoMatch[2]) {
11810|   montoEntregado = parseNum(entregoMontoMatch[2]);
11811|   monedaEntregada = normalizarMoneda(entregoMontoMatch[3] || '');
11812| } else if (entregoSoloMatch && entregoSoloMatch[2]) {
11813|   monedaEntregada = normalizarMoneda(entregoSoloMatch[2] || '');
11814| }
11815|
11816| // Si tenemos recibo + entrego (moneda) pero sin monto entregado, intentar con TC
11817| if (montoRecibido !== null && montoRecibido > 0 && monedaRecibida && monedaEntregada && (montoEntregado == null || montoEntregado <= 0) && cotizacion !== null && cotizacion > 0) {
11818|   const pChat = patronTipoCambioOrden(monedaRecibida, monedaEntregada);
11819|   if (esPatronVendeFiatConTc(pChat)) montoEntregado = montoRecibido * cotizacion;
11820|   else if (esPatronCompraFiatConTc(pChat)) montoEntregado = montoRecibido / cotizacion;
11821| }
11822|
11823| // Fallback: dos números en el texto (primero = recibo, segundo = entregado)
11824| if ((montoRecibido == null || montoEntregado == null) || (!monedaRecibida || !monedaEntregada)) {
11825|   const reMonto = /(\d[\d.,]*)\s*(ARS|USD|EUR|pesos?|dólares?|dolares?|euros?)/gi;
11826|   const montos = [];
11827|   let match;
11828|   while ((match = reMonto.exec(t)) !== null) {
11829|     const n = parseNum(match[1]);
11830|     if (!isNaN(n) && n > 0) {
11831|       const moneda = normalizarMoneda(match[2] || '');
11832|       montos.push({ monto: n, moneda: moneda || null });
11833|     }
11834|   }
11835|   if (montos.length >= 2 && !monedaRecibida) {
11836|     montoRecibido = montos[0].monto;
11837|     monedaRecibida = montos[0].moneda;
11838|     montoEntregado = montos[1].monto;
11839|     monedaEntregada = montos[1].moneda;
11840|   }
11841| }
11842|
11843| if (!montoRecibido || montoRecibido <= 0 || !montoEntregado || montoEntregado <= 0) {
11844|   return { error: 'Indicá qué recibís y qué entregás (montos y monedas). Si ponés un solo monto y tipo de cambio (ej. "a tc 1500"), calculo el otro.' };
11845| }
11846| if (!monedaRecibida || !monedaEntregada) {
11847|   return { error: 'Indicá las monedas (ARS, USD o EUR) para recibir y entregar.' };
11848| }
11849|
11850| const codigoTipo = monedaRecibida + '-' + monedaEntregada;
11851| const tipo = Array.isArray(tipos) && tipos.find((x) => (x.codigo || '')) === codigoTipo;
11852| if (!tipo) {
11853|   return { error: 'No hay tipo de operación ' + codigoTipo + '. Revisá las monedas.' };
11854| }
11855|
11856| const requiereCotizacion = !patronTipoCambioOrden(monedaRecibida, monedaEntregada);
11857| if (requiereCotizacion && (!cotizacion || cotizacion <= 0)) {
11858|   return { error: 'Para el cruce ' + codigoTipo + ' indicá el tipo de cambio (ej. "a tc 1500").' };
11859| }
11860|
11861| let cliente_id = null;
11862| let cliente_nombre = '';
11863| const paraMatch = t.match(/bpara\s+([\^.\d]+)?(?:\s*\s+|s+recib|s+entrega$)/i);
11864| const clienteMatch = t.match(/bcliente\s+([\^.\d]+)?(?:\s*\s+|s+recib|s+entrega$)/i);
11865| const nombreBuscar = (paraMatch && paraMatch[1].trim()) || (clienteMatch && clienteMatch[1].trim());
11866| if (nombreBuscar && Array.isArray(clientes) && clientes.length) {

```

```

11867 |     const nombreNorm = nombreBuscar.toLowerCase().trim();
11868 |     const encontrado = clientes.find((c) => (c.nombre || '').toLowerCase().trim() === nombreNorm ||
11869 |       (c.nombre || '').toLowerCase().includes(nombreNorm) ||
11870 |       nombreNorm.includes((c.nombre || '').toLowerCase().trim()));
11871 |     if (encontrado) {
11872 |       cliente_id = encontrado.id;
11873 |       cliente_nombre = encontrado.nombre || '';
11874 |     }
11875 |   }
11876 |
11877 |   let fecha = new Date();
11878 |   if (/bhoy\b/i.test(t)) fecha = new Date();
11879 |   else if (/bmañana\b/i.test(t)) { fecha = new Date(); fecha.setDate(fecha.getDate() + 1); }
11880 |   const fechaStr = fecha.getFullYear() + '-' + String(fecha.getMonth() + 1).padStart(2, '0') + '-' + String(fecha.getDate()).padStart(2, '0');
11881 |
11882 |   return {
11883 |     cliente_id,
11884 |     cliente_nombre,
11885 |     tipo_operacion_id: tipo.id,
11886 |     tipo_codigo: tipo.codigo,
11887 |     moneda_recibida: monedaRecibida,
11888 |     moneda_entregada: monedaEntregada,
11889 |     monto_recibido: montoRecibido,
11890 |     monto_entregado: montoEntregado,
11891 |     fecha: fechaStr,
11892 |     cotizacion: cotizacion,
11893 |   };
11894 | }
11895 |
11896 | function openModalChatOrden() {
11897 |   const backdrop = document.getElementById('modal-chat-orden-backdrop');
11898 |   const logEl = document.getElementById('chat-orden-log');
11899 |   const previewEl = document.getElementById('chat-orden-preview');
11900 |   const confirmWrap = document.getElementById('chat-orden-confirmar-wrap');
11901 |   const inputEl = document.getElementById('chat-orden-input');
11902 |   if (!backdrop || !logEl) return;
11903 |   logEl.innerHTML = '';
11904 |   previewEl.style.display = 'none';
11905 |   previewEl.innerHTML = '';
11906 |   confirmWrap.style.display = 'none';
11907 |   chatOrdenUltimaInterpretacion = null;
11908 |   if (inputEl) inputEl.value = '';
11909 |   backdrop.classList.add('activo');
11910 |   if (inputEl) inputEl.focus();
11911 |   Promise.all([
11912 |     client.from('clientes').select('id, nombre').eq('activo', true).order('nombre', { ascending: true }),
11913 |     tiposOperacionFetchConFallbackOrdenVisual(
11914 |       () => client.from('tipos_operacion').select('id, codigo, nombre, orden_visual').eq('activo', true).order('orden_visual', { ascending: true }).order('codigo').order('usa_
intermediario').order('id'),
11915 |       () => client.from('tipos_operacion').select('id, codigo, nombre').eq('activo', true).order('codigo').order('usa_intermediario').order('id'),
11916 |     ),
11917 |   ]).then(([rC, rT]) => {
11918 |     chatOrdenClientes = rC.data || [];
11919 |     chatOrdenTipos = rT.data || [];
11920 |     if (rT.error) showToast('Error al cargar tipos de operación: ' + (rT.error.message || ''), 'error');
11921 |   });
11922 | }
11923 |
11924 | function closeModalChatOrden() {
11925 |   const backdrop = document.getElementById('modal-chat-orden-backdrop');
11926 |   if (backdrop) backdrop.classList.remove('activo');
11927 | }
11928 |
11929 | function setupModalChatOrden() {
11930 |   const backdrop = document.getElementById('modal-chat-orden-backdrop');
11931 |   const btnClose = document.getElementById('modal-chat-orden-close');
11932 |   const btnEnviar = document.getElementById('chat-orden-enviar');
11933 |   const inputEl = document.getElementById('chat-orden-input');
11934 |   const logEl = document.getElementById('chat-orden-log');
11935 |   const previewEl = document.getElementById('chat-orden-preview');
11936 |   const confirmWrap = document.getElementById('chat-orden-confirmar-wrap');
11937 |   const btnConfirmar = document.getElementById('chat-orden-confirmar');
11938 |   const btnAbrirChat = document.getElementById('btn-orden-por-chat');
11939 |
11940 |   if (btnAbrirChat) btnAbrirChat.addEventListener('click', () => { chatOrdenAbiertoDesdePanel = false; openModalChatOrden(); });
11941 |   const btnChatInicio = document.getElementById('btn-orden-por-chat-inicio');
11942 |   if (btnChatInicio) btnChatInicio.addEventListener('click', () => { chatOrdenAbiertoDesdePanel = true; openModalChatOrden(); });
11943 |   if (btnClose) btnClose.addEventListener('click', closeModalChatOrden);
11944 |   if (backdrop) setupBackdropCloseOnlyOnRealClick(backdrop, closeModalChatOrden);
11945 |
11946 |   function appendMsg(role, text, label) {
11947 |     if (!logEl) return;
11948 |     const div = document.createElement('div');
11949 |     div.className = 'chat-msg ' + role;
11950 |     if (label) {
11951 |       const l = document.createElement('div');
11952 |       l.className = 'chat-msg-label';
11953 |       l.textContent = label;
11954 |       div.appendChild(l);
11955 |     }
11956 |     div.appendChild(document.createTextNode(text));
11957 |     logEl.appendChild(div);
11958 |     logEl.scrollTop = logEl.scrollHeight;
11959 |   }
11960 |
11961 |   if (btnEnviar && inputEl) {
11962 |     btnEnviar.addEventListener('click', () => {

```

```

11964 const texto = inputEl.value.trim();
11965 if (!texto) return;
11966 appendMsg('user', texto, 'Vos');
11967 inputEl.value = '';
11968 const result = interpretarTextoOrden(texto, chatOrdenClientes, chatOrdenTipos);
11969 if (result.error) {
11970   appendMsg('bot', result.error, 'Sistema');
11971   previewEl.style.display = 'none';
11972   confirmWrap.style.display = 'none';
11973   chatOrdenUltimaInterpretacion = null;
11974   return;
11975 }
11976 const clienteTexto = result.cliente_nombre || (result.cliente_id ? 'Cliente asignado' : 'Sin cliente');
11977 let previewTexto = 'Cliente: ' + clienteTexto + ' · Recibimos ' + result.moneda_recibida + ' ' + formatMonto(result.monto_recibido) + ' · Entregamos ' + result.moneda_en
11978 tregada + ' ' + formatMonto(result.monto_entregado);
11979 if (result.cotizacion !== null && result.cotizacion > 0) previewTexto += ' · TC: ' + formatMonto(result.cotizacion);
11980 previewTexto += ' · Fecha: ' + result.fecha;
11981 appendMsg('bot', previewTexto, 'Preview');
11982 previewEl.textContent = previewTexto;
11983 previewEl.style.display = 'block';
11984 confirmWrap.style.display = 'block';
11985 chatOrdenUltimaInterpretacion = result;
11986 logEl.scrollTop = logEl.scrollHeight;
11987 });
11988 }
11989 if (inputEl) {
11990   inputEl.addEventListener('keydown', (e) => {
11991     if (e.key === 'Enter' && !e.shiftKey) {
11992       e.preventDefault();
11993       if (btnEnviar) btnEnviar.click();
11994     }
11995   });
11996 }
11997 if (btnConfirmar && confirmWrap) {
11998   btnConfirmar.addEventListener('click', () => {
11999     const r = chatOrdenUltimaInterpretacion;
12000     if (!r) return;
12001     if (!userPermissions.includes('ingresar_orden')) {
12002       showToast('No tenés permiso para crear órdenes.', 'error');
12003       return;
12004     }
12005     if (!r.cliente_id) {
12006       showToast('Indicá un cliente en el mensaje (ej. "para Adriana").', 'error');
12007       return;
12008     }
12009     if (!r.tipo_operacion_id) {
12010       showToast('No se pudo identificar el tipo de operación. Indicá en el mensaje (ej. "recibo USD y entrego ARS" o "ARS-USD").', 'error');
12011       return;
12012     }
12013     const requiereTc = !!patronTipoCambioOrden(r.moneda_recibida, r.moneda_entregada);
12014     if (requiereTc && (!r.cotizacion || r.cotizacion <= 0)) {
12015       showToast('Para este cruce con USD el tipo de cambio es obligatorio.', 'error');
12016       return;
12017     }
12018     const payload = {
12019       cliente_id: r.cliente_id || null,
12020       fecha: r.fecha,
12021       estado: 'pendiente_instrumentar',
12022       tipo_operacion_id: r.tipo_operacion_id,
12023       operacion_directa: true,
12024       intermediario_id: null,
12025       moneda_recibida: r.moneda_recibida,
12026       moneda_entregada: r.moneda_entregada,
12027       monto_recibido: r.monto_recibido,
12028       monto_entregado: r.monto_entregado,
12029       cotizacion: r.cotizacion || null,
12030       tasa_descuento_intermediario: null,
12031       observaciones: null,
12032       usuario_id: currentUser.id,
12033       updated_at: new Date().toISOString(),
12034     };
12035     insertOrdenConProximoNumero(payload).then((res) => {
12036       if (res.error) {
12037         showToast('Error al crear la orden: ' + (res.error.message || ''), 'error');
12038         return;
12039       }
12040       const ordenId = res.data && res.data[0] && res.data[0].id;
12041       if (!ordenId) {
12042         showToast('Error: no se obtuvo el id de la orden.', 'error');
12043         return;
12044       }
12045       const ordenParaAuto = {
12046         intermediario_id: null,
12047         tipo_operacion_id: r.tipo_operacion_id,
12048         monto_recibido: r.monto_recibido,
12049         monto_entregado: r.monto_entregado,
12050         moneda_recibida: r.moneda_recibida,
12051         moneda_entregada: r.moneda_entregada,
12052         cotizacion: r.cotizacion || null,
12053       };
12054       client.from('instrumentacion').insert({ orden_id: ordenId }).select('id').then((rInst) => {
12055         function alFinalizar() {
12056           closeModalChatOrden();
12057           loadOrdenes();
12058           if (chatOrdenAbiertoDesdePanel) {

```

```

12059|         chatOrdenAbiertoDesdePanel = false;
12060|         showView('vista-ordenes', 'Órdenes');
12061|     }
12062| }
12063| if (rInst.error) {
12064|     showToast('Orden creada pero falló la instrumentación: ' + (rInst.error.message || ''), 'error');
12065|     alFinalizar();
12066|     return;
12067| }
12068| const instId = rInst.data && rInst.data[0] && rInst.data[0].id;
12069| if (!instId) {
12070|     showToast('Orden e instrumentación creadas.', 'success');
12071|     alFinalizar();
12072|     return;
12073| }
12074| autoCompletarInstrumentacionSinIntermediario(ordenId, instId, ordenParaAuto).then(() => {
12075|     showToast('Orden e instrumentación creadas (con transacciones).', 'success');
12076|     alFinalizar();
12077| }).catch(() => {
12078|     showToast('Orden e instrumentación creadas; no se pudieron generar las transacciones automáticas.', 'info');
12079|     alFinalizar();
12080| });
12081| });
12082| });
12083| });
12084| }
12085| }
12086| }
12087| // --- Transacciones (panel debajo de la orden) ---
12088| let transaccionesOrdenIdActual = null;
12089| /** Sólo el último openModalTransaccion aplica UI: si hay doble click o reentradas, un callback tardío no debe ejecutar form.reset() pisando el formulario ya cargado (E2E 03 U
SD-ARS E,P: modal quedaba en Ingreso + modo Ninguno). */
12090| let openModalTransaccionSeq = 0;
12091|
12092| /** Convierte monto a la moneda destino. tipo_cambio = unidades ARS por 1 unidad de la otra (ej. ARS por 1 USD). */
12093| function convertirAMonedaOrden(monto, monedaOrigen, monedaDestino, tipoCambio) {
12094|     if (monedaOrigen === monedaDestino) return Number(monto);
12095|     const m = Number(monto);
12096|     const tc = tipoCambio !== null && !isNaN(tipoCambio) && tipoCambio > 0 ? Number(tipoCambio) : null;
12097|     if (monedaOrigen === 'ARS' && monedaDestino !== 'ARS' && tc) return m / tc;
12098|     if (monedaOrigen !== 'ARS' && monedaDestino === 'ARS' && tc) return m * tc;
12099|     if (monedaOrigen === 'EUR' && monedaDestino === 'USD') return m;
12100|     if (monedaOrigen === 'USD' && monedaDestino === 'EUR') return m;
12101|     return m;
12102| }
12103|
12104| /**
12105|  * Totales de transacciones en moneda del acuerdo. orden: { moneda_recibida, monto_recibido, moneda_entregada, monto_entregado, cotizacion?, cliente_id }.
12106|  * options.totalesMulticontraparte: solo suman al acuerdo ingresos donde el pagador (rol cliente) es el cliente del acuerdo y egresos donde el cobrador (rol cliente) es el cli
ente del acuerdo (docs/INSTRUMENTACION_MANUAL_MULTICONTRAPARTE.md).
12107|  */
12108| function totalesInstrumentacion(transacciones, orden, options) {
12109|     if (!orden) return { totalRecibido: 0, totalEntregado: 0 };
12110|     const requiereCliente = !orden.cliente_id;
12111|     const totalesMc = options && options.totalesMulticontraparte === true;
12112|     const cidAcuerdo = orden.cliente_id || null;
12113|     const monedaRecibida = orden.moneda_recibida || 'USD';
12114|     const monedaEntregada = orden.moneda_entregada || 'USD';
12115|     const tcAcuerdo = orden.cotizacion !== null && !isNaN(Number(orden.cotizacion)) && Number(orden.cotizacion) > 0 ? Number(orden.cotizacion) : null;
12116|     let totalRecibido = 0;
12117|     let totalEntregado = 0;
12118|     (transacciones || []).forEach((t) => {
12119|         if (totalesMc && cidAcuerdo) {
12120|             const tipoL = (t.tipo || '').toString().toLowerCase();
12121|             if (tipoL === 'ingreso') {
12122|                 const { pag } = pagCobEfectivosTransaccionSync(t);
12123|                 if (pag !== 'cliente' || idClientePagadorEfectivoMulticontraparte(t, orden) !== cidAcuerdo) return;
12124|             } else {
12125|                 const { cob } = pagCobEfectivosTransaccionSync(t);
12126|                 if (cob !== 'cliente' || idClienteCobradorEfectivoMulticontraparte(t, orden) !== cidAcuerdo) return;
12127|             }
12128|         } else {
12129|             // Para comparar con el acuerdo del cliente, solo cuentan transacciones donde participa el cliente (rol genérico).
12130|             // Las transacciones Pandý + Intermediario se permiten para conciliar cuentas, pero no deben bloquear el acuerdo.
12131|             if (requiereCliente && !(t.cobrador === 'cliente' || t.pagador === 'cliente')) return;
12132|         }
12133|         const monto = Number(t.monto);
12134|         const tcTrx = t.tipo_cambio !== null && !isNaN(t.tipo_cambio) ? Number(t.tipo_cambio) : null;
12135|         const tc = (tcTrx !== null && tcTrx > 0) ? tcTrx : tcAcuerdo;
12136|         if (t.tipo === 'ingreso') {
12137|             totalRecibido += convertirAMonedaOrden(monto, t.moneda, monedaRecibida, tc);
12138|         } else {
12139|             totalEntregado += convertirAMonedaOrden(monto, t.moneda, monedaEntregada, tc);
12140|         }
12141|     });
12142|     return { totalRecibido, totalEntregado };
12143| }
12144|
12145| /** Valida que los totales no superen el acuerdo. Devuelve { ok, mensaje }. Si ya se completó el acuerdo, mensaje específico; si no, indica en cuánto se excede. */
12146| function validarTotalesVsAcuerdo(transacciones, orden, transaccionExcluirId, transaccionAgregar, totalesOpts) {
12147|     const list = (transacciones || []).filter((t) => t.id !== transaccionExcluirId);
12148|     const listConNueva = transaccionAgregar ? [...list, transaccionAgregar] : list;
12149|     const { totalRecibido: totalRecSin, totalEntregado: totalEntSin } = totalesInstrumentacion(list, orden, totalesOpts);
12150|     const { totalRecibido: totalRecCon, totalEntregado: totalEntCon } = totalesInstrumentacion(listConNueva, orden, totalesOpts);
12151|     const montoRecibido = Number(orden.monto_recibido) || 0;
12152|     const montoEntregado = Number(orden.monto_entregado) || 0;
12153|     const monedaRecibida = (orden.moneda_recibida || 'USD').trim().toUpperCase();

```



```

12154|     const monedaEntregada = (orden.moneda_entregada || 'USD').trim().toUpperCase();
12155|     const tol = 1e-6;
12156|
12157|     if (totalRecCon > montoRecibido + tol) {
12158|         const yaCompleto = totalRecSin >= montoRecibido - tol;
12159|         const mensaje = yaCompleto
12160|             ? 'No se puede cargar una transacción de ingreso dado que ya se completó el acuerdo.'
12161|             : `La transacción excede el acuerdo en ingresos en ${formatImporteDisplay(totalRecCon - montoRecibido)} ${monedaRecibida}. El máximo permitido es ${formatImporteDisplay(
montoRecibido)} ${monedaRecibida}.`;
12162|         return { ok: false, mensaje };
12163|     }
12164|     if (totalEntCon > montoEntregado + tol) {
12165|         const yaCompleto = totalEntSin >= montoEntregado - tol;
12166|         const mensaje = yaCompleto
12167|             ? 'No se puede cargar una transacción de egreso dado que ya se completó el acuerdo.'
12168|             : `La transacción excede el acuerdo en egresos en ${formatImporteDisplay(totalEntCon - montoEntregado)} ${monedaEntregada}. El máximo permitido es ${formatImporteDisplay(
montoEntregado)} ${monedaEntregada}.`;
12169|         return { ok: false, mensaje };
12170|     }
12171|     return { ok: true };
12172| }
12173|
12174| /** Indica si los totales de transacciones coinciden con el acuerdo (dentro de tolerancia). */
12175| function estaConciliada(transacciones, orden, totalesOpts) {
12176|     if (!orden) return false;
12177|     const { totalRecibido, totalEntregado } = totalesInstrumentacion(transacciones || [], orden, totalesOpts);
12178|     const montoRecibido = Number(orden.monto_recibido) || 0;
12179|     const montoEntregado = Number(orden.monto_entregado) || 0;
12180|     const tol = 1e-6;
12181|     return Math.abs(totalRecibido - montoRecibido) <= tol && Math.abs(totalEntregado - montoEntregado) <= tol;
12182| }
12183|
12184| /** Texto corto para UI: faltan ingresos/egresos por N o exceso respecto del acuerdo. */
12185| function textoAvisoFalta0ExcesoInstrumentacion(orden, transacciones, totalesOpts) {
12186|     if (!orden) return '';
12187|     const { totalRecibido, totalEntregado } = totalesInstrumentacion(transacciones || [], orden, totalesOpts);
12188|     const mr = Number(orden.monto_recibido) || 0;
12189|     const me = Number(orden.monto_entregado) || 0;
12190|     const monR = (orden.moneda_recibida || 'USD').trim();
12191|     const monE = (orden.moneda_entregada || 'USD').trim();
12192|     const tol = 1e-6;
12193|     if (estaConciliada(transacciones, orden, totalesOpts)) return '';
12194|     const partes = [];
12195|     if (totalRecibido < mr - tol) partes.push(`Faltan ingresos por ${formatImporteDisplay(mr - totalRecibido)} ${monR}`);
12196|     else if (totalRecibido > mr + tol) partes.push(`Los ingresos superan el acuerdo en ${formatImporteDisplay(totalRecibido - mr)} ${monR}`);
12197|     if (totalEntregado < me - tol) partes.push(`Faltan egresos por ${formatImporteDisplay(me - totalEntregado)} ${monE}`);
12198|     else if (totalEntregado > me + tol) partes.push(`Los egresos superan el acuerdo en ${formatImporteDisplay(totalEntregado - me)} ${monE}`);
12199|     return partes.join(' ');
12200| }
12201|
12202| function fetchOrdenYTransaccionesParaValidarCierreWizard(ordenId, instId) {
12203|     if (!ordenId || !instId) return Promise.resolve({ orden: null, transacciones: [], totalesOpts: undefined });
12204|     return client.from('ordenes').select('id, estado, cliente_id, intermediario_id, moneda_entregada, monto_entregado, cotizacion, tipos_operaci
on(codigo, usa_intermediario)').eq('id', ordenId).single().Then((rOrd) => {
12205|         const orden = rOrd.data;
12206|         if (!orden) return { orden: null, transacciones: [], totalesOpts: undefined };
12207|         return Promise.all([
12208|             client.from('transacciones').select('id, tipo, moneda, monto, cobrador, pagador, tipo_cambio, estado, pagador_cliente_id, cobrador_cliente_id, pagador_intermediario_id,
cobrador_intermediario_id').eq('instrumentacion_id', instId).order('created_at', { ascending: true }),
12209|             client.from('instrumentacion').select('multicontraparte_manual').eq('id', instId).single(),
12210|         ]).then(([, rTr, rInst]) => {
12211|             const toJoin = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
12212|             const mc = !(rInst.data && rInst.data.multicontraparte_manual);
12213|             const totalesOpts = mc && esTipoOpMulticontraparteElegibleDesdeOrden(orden, toJoin) ? { totalesMulticontraparte: true } : undefined;
12214|             return {
12215|                 orden,
12216|                 transacciones: rTr.data || [],
12217|                 totalesOpts,
12218|             };
12219|         });
12220|     });
12221| }
12222|
12223| /** Aplica montos editados en la tabla del wizard (inputs sin blur) antes de validar cierre. */
12224| function aplicarMontosDesdeInputsWizardInstrumentacion(transacciones, rootEl) {
12225|     if (!rootEl || !transacciones || !transacciones.length) return transacciones || [];
12226|     const map = {};
12227|     rootEl.querySelectorAll('input-monto-transaccion-tabla').forEach((input) => {
12228|         const id = input.getAttribute('data-id');
12229|         if (!id) return;
12230|         const v = parseImporteInput(input.value);
12231|         if (typeof v === 'number' && !isNaN(v)) map[id] = v;
12232|     });
12233|     return transacciones.map((t) => (map[t.id] != null ? { ...t, monto: map[t.id] } : t));
12234| }
12235|
12236| /**
12237|  * Listo: no cierra hasta estar conciliada (toast + scroll al acuerdo).
12238|  * Salir (X, Cerrar, Escape, backdrop, Cancelar con instrumentación abierta): confirmación si no está conciliada.
12239|  */
12240| function solicitarCierreModalOrden(options) {
12241|     const modo = options && options.modo === 'listo' ? 'listo' : 'salir';
12242|     const refrescarOrdenes = options && options.refrescarOrdenes !== false;
12243|     const backdrop = document.getElementById('modal-orden-backdrop');
12244|     if (!backdrop || !backdrop.classList.contains('activo')) return;
12245|
12246|     const ordenId = (document.getElementById('orden-id') && document.getElementById('orden-id').value || '').trim();

```

```

12247| const instId = ordenWizardInstrumentacionIdActual;
12248|
12249| function finalizar() {
12250|     ejecutarCierreModalOrden();
12251|     if (refrescarOrdenes && typeof loadOrdenes === 'function') loadOrdenes();
12252| }
12253|
12254| if (!ordenId || !instId) {
12255|     finalizar();
12256|     return;
12257| }
12258|
12259| fetchOrdenYTransaccionesParaValidarCierreWizard(ordenId, instId).then(({ orden, transacciones, totalesOpts }) => {
12260|     const tx = aplicarMontosDesdeInputsWizardInstrumentacion(transacciones, backdrop);
12261|     if (!orden) {
12262|         finalizar();
12263|         return;
12264|     }
12265|     if (orden.estado === 'anulada') {
12266|         finalizar();
12267|         return;
12268|     }
12269|     if (estaConciliada(tx, orden, totalesOpts)) {
12270|         finalizar();
12271|         return;
12272|     }
12273|     const aviso = textoAvisoFalta0ExcesoInstrumentacion(orden, tx, totalesOpts);
12274|     if (modo === 'listo') {
12275|         showToast(
12276|             'Completá la instrumentación para cerrar con «Listo». ' + (aviso || 'Los totales no coinciden con el acuerdo.'),
12277|             'error'
12278|         );
12279|         const blk = document.getElementById('orden-inst-acuerdo-block');
12280|         const scrollRoot = document.querySelector('#orden-step-instrumentacion .orden-inst-contenido-scroll');
12281|         if (blk) {
12282|             if (scrollRoot && scrollRoot.contains(blk)) {
12283|                 const top = blk.offsetTop - 8;
12284|                 scrollRoot.scrollTo({ top: Math.max(0, top), behavior: 'smooth' });
12285|             } else {
12286|                 blk.scrollToView({ behavior: 'smooth', block: 'nearest' });
12287|             }
12288|         }
12289|         return;
12290|     }
12291|     const cuerpo = aviso
12292|         ? `${aviso} ¿Salir igual? La orden quedará pendiente de instrumentar.`
12293|         : 'La instrumentación no coincide con el acuerdo. ¿Salir igual? La orden quedará pendiente de instrumentar.';
12294|     showConfirm(cuerpo, 'Sí, salir', () => finalizar());
12295| }).catch(() => {
12296|     showToast('No se pudo verificar la instrumentación. Intentá de nuevo.', 'error');
12297| });
12298| }
12299|
12300| /** Calcula el estado de la orden según transacciones y acuerdo. Devuelve { estado, conciliada, todasEjecutadas }.
12301| * Orden Ejecutada solo cuando TODAS las transacciones (incl. compensaciones Pandy-Intermediario) están ejecutadas.
12302| * options.totalesMulticontraparte: ver `totalesInstrumentacion`. */
12303| function calcularEstadoOrden(transacciones, orden, options) {
12304|     const requiereCliente = !orden?.cliente_id;
12305|     const listAll = (transacciones || []).filter((t) => String(t.estado || '').toLowerCase() !== 'anulada');
12306|     const listCliente = requiereCliente ? listAll.filter((t) => t.cobrador === 'cliente' || t.pagador === 'cliente') : listAll;
12307|     const totalTodas = listAll.length;
12308|     const ejecutadasTodas = listAll.filter((t) => t.estado === 'ejecutada').length;
12309|     const todasEjecutadas = totalTodas > 0 && ejecutadasTodas === totalTodas;
12310|     const totOpts = options && options.totalesMulticontraparte ? { totalesMulticontraparte: true } : undefined;
12311|     const conciliada = estaConciliada(listCliente, orden, totOpts);
12312|
12313|     let estado = 'pendiente_instrumentar';
12314|     if (totalTodas === 0) estado = 'pendiente_instrumentar';
12315|     else if (todasEjecutadas) estado = 'orden_ejecutada';
12316|     else if (conciliada) estado = 'instrumentacion_cerrada_ejecucion';
12317|     else estado = 'instrumentacion_parcial';
12318|
12319|     return { estado, conciliada, todasEjecutadas };
12320| }
12321|
12322| /**
12323| * Si existiera ingreso instrumentado Intermediario-Pandy (legacy), actualizaría su monto por la tasa.
12324| * Con el modelo actual (solo Cliente-Pandy en `transacciones`) no hay fila que actualizar.
12325| */
12326| function actualizarTasaTransaccionIngresoIntermediarioCheque(ordenId, orden) {
12327|     if (!ordenId || !orden || !orden.intermediario_id || orden.tasa_descuento_intermediario == null) return Promise.resolve();
12328|     const mr = Number(orden.monto_recibido) || 0;
12329|     const tasa = Number(orden.tasa_descuento_intermediario);
12330|     const montoEfectivoInt = (typeof tasa === 'number' && !isNaN(tasa) && tasa >= 0 && tasa < 1) ? mr * (1 - tasa) : mr;
12331|     return client.from('instrumentacion').select('id').eq('orden_id', ordenId).maybeSingle().then((rInst) => {
12332|         const instId = rInst.data && rInst.data.id;
12333|         if (!instId) return Promise.resolve();
12334|         return client.from('transacciones').select('id, tipo, modo_pago_id, cobrador, pagador').eq('instrumentacion_id', instId).then((rTr) => {
12335|             const list = rTr.data || [];
12336|             if (list.length === 0) return Promise.resolve();
12337|             return client.from('modos_pago').select('id, codigo').then((rModos) => {
12338|                 const byId = {};
12339|                 (rModos.data || []).forEach((m) => { byId[m.id] = m.codigo; });
12340|                 const trx = list.find((t) => t.tipo === 'ingreso' && t.cobrador === 'pandy' && t.pagador === 'intermediario' && byId[t.modo_pago_id] === 'efectivo');
12341|                 if (!trx) return Promise.resolve();
12342|                 return client.from('transacciones').update({ monto: montoEfectivoInt, updated_at: new Date().toISOString() }).eq('id', trx.id);
12343|             });
12344|         });
12345|     });

```

```

12344 |     });
12345 | });
12346 | }
12347 |
12348 | /**
12349 |  * Inserta movimientos en cuenta corriente (cliente e intermediario) para una transacción.
12350 |  * Regla: solo se impacta cuenta corriente con transacciones EJECUTADAS (no pendientes).
12351 |  * No hay cuenta corriente cliente-intermediario: los flujos entre cliente e intermediario se reflejan en Pandy-Intermediario.
12352 |  * orden: { cliente_id, intermediario_id }; t: { cobrador, pagador, moneda, monto }; estadoTransaccion: 'pendiente' | 'ejecutada'.
12353 |  */
12354 | function insertarMovimientosCcParaTransaccion(transaccionId, orden, t, estadoTransaccion) {
12355 |   if (!transaccionId || !currentUserId || estadoTransaccion !== 'ejecutada') return Promise.resolve();
12356 |   const ordenId = orden && orden.id;
12357 |   if (!ordenId) return Promise.resolve();
12358 |   const fecha = fechaHoyYYYYMMDDArgentina();
12359 |   const ahora = new Date().toISOString();
12360 |   const monto = Number(t.monto) || 0;
12361 |   const cob = t.cobrador;
12362 |   const pag = t.pagador;
12363 |   const mon = t.moneda || 'USD';
12364 |   const clienteId = orden.cliente_id || null;
12365 |   const intermediarioId = orden.intermediario_id || null;
12366 |   const monR = orden.moneda_recibida || 'USD';
12367 |   const monE = orden.moneda_entregada || 'USD';
12368 |   const mr = Number(orden.monto_recibido) || 0;
12369 |   const me = Number(orden.monto_entregado) || 0;
12370 |   // CC cliente: ingreso misma moneda + -me (Pandy debe el valor de su transacción). Egreso + +monto. Otras + -monto / +monto por transacción.
12371 |   const esIngresoMismaMoneda = (monR === monE && String(cob || '').toLowerCase() === 'pandy' && pag === 'cliente');
12372 |   const montoCobroCc = esIngresoMismaMoneda ? me : monto;
12373 |   const montosCobro = montosCcPorMoneda(mon, -montoCobroCc);
12374 |   const montosDeuda = montosCcPorMoneda(mon, monto);
12375 |   const inserts = [];
12376 |   const transNumero = t.numero !== null ? t.numero : null;
12377 |   const ordenNumero = orden.numero !== null ? orden.numero : null;
12378 |   if (pag === 'cliente' && cob !== 'intermediario' && clienteId) {
12379 |     inserts.push(client.from('movimientos_cuenta_corriente').insert({
12380 |       cliente_id: clienteId, moneda: mon, monto: -montoCobroCc, orden_id: ordenId, transaccion_id: transaccionId, transaccion_numero: transNumero,
12381 |       concepto: conceptoCcLeyenda('cobro_realizado', ordenNumero, transNumero), fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
12382 |       ...montosCobro,
12383 |     }));
12384 |   }
12385 |   if (cob === 'cliente' && pag !== 'intermediario' && clienteId) {
12386 |     inserts.push(client.from('movimientos_cuenta_corriente').insert({
12387 |       cliente_id: clienteId, moneda: mon, monto, orden_id: ordenId, transaccion_id: transaccionId, transaccion_numero: transNumero,
12388 |       concepto: conceptoCcLeyenda('compromiso_pago', ordenNumero, transNumero), fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
12389 |       ...montosDeuda,
12390 |     }));
12391 |   }
12392 |   // Pandy ↔ Intermediario: Cobro = intermediario pagó; Deuda = Pandy debe al intermediario.
12393 |   const esPandyInt = (cob === 'pandy' && pag === 'intermediario') || (cob === 'intermediario' && pag === 'pandy');
12394 |   if (esPandyInt && cob === 'pandy' && intermediarioId) {
12395 |     inserts.push(
12396 |       client.from('movimientos_cuenta_corriente_intermediario').update({ estado: 'cerrado', estado_fecha: ahora })
12397 |       .eq('orden_id', ordenId).eq('intermediario_id', intermediarioId).eq('transaccion_id', transaccionId).eq('estado', 'pendiente')
12398 |       .then(() => client.from('movimientos_cuenta_corriente_intermediario').insert({
12399 |         intermediario_id: intermediarioId, moneda: mon, monto, orden_id: ordenId, transaccion_id: transaccionId, transaccion_numero: transNumero,
12400 |         concepto: conceptoCcLeyenda('cobro_realizado', ordenNumero, transNumero), fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
12401 |         ...montosCcPorMoneda(mon, monto),
12402 |       }));
12403 |     );
12404 |   }
12405 |   if (esPandyInt && cob === 'intermediario' && intermediarioId) {
12406 |     const instrumentacionId = t.instrumentacion_id || null;
12407 |     const tasa = Number(orden.tasa_descuento_intermediario) || 0;
12408 |     const montoEfectivoInt = (typeof tasa === 'number' && !isNaN(tasa) && tasa >= 0 && tasa < 1) ? mr * (1 - tasa) : mr;
12409 |     const monInt = orden.moneda_recibida || mon || 'ARS';
12410 |     inserts.push(
12411 |       client.from('movimientos_cuenta_corriente_intermediario').select('id').eq('orden_id', ordenId).eq('intermediario_id', intermediarioId).eq('transaccion_id', transaccionId)
12412 |       ).maybeSingle()
12413 |       .then((r) => {
12414 |         if (r.data && r.data.id) return client.from('movimientos_cuenta_corriente_intermediario').update({ estado: 'cerrado', estado_fecha: ahora }).eq('id', r.data.id);
12415 |         return Promise.resolve();
12416 |       })
12417 |       .then(() => client.from('movimientos_cuenta_corriente_intermediario').insert({
12418 |         intermediario_id: intermediarioId, orden_id: ordenId, transaccion_id: transaccionId, transaccion_numero: transNumero, moneda: monInt, monto: -montoEfectivoInt,
12419 |         concepto: conceptoCcLeyenda('compromiso_cobrar', ordenNumero, transNumero), fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
12420 |         ...montosCcPorMoneda(monInt, -montoEfectivoInt),
12421 |       }));
12422 |     .then(() => {
12423 |       if (!instrumentacionId) return Promise.resolve();
12424 |       return client.from('comisiones_orden').select('moneda, monto').eq('orden_id', ordenId).eq('beneficiario', 'intermediario').maybeSingle()
12425 |       .then((rCom) => {
12426 |         const comMonto = rCom.data && (Number(rCom.data.monto) || 0);
12427 |         if (comMonto >= 1e-6) {
12428 |           const monCom = (rCom.data.moneda || 'ARS').toUpperCase();
12429 |           // Según Excel: comisión intermediario en CC con signo invertido (-3000).
12430 |           return client.from('movimientos_cuenta_corriente_intermediario').insert({
12431 |             intermediario_id: intermediarioId, orden_id: ordenId, transaccion_id: transaccionId, transaccion_numero: transNumero, moneda: monCom, monto: -comMonto,
12432 |             concepto: conceptoCcLeyenda('comision_acuerdo', ordenNumero, transNumero), fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
12433 |             ...montosCcPorMoneda(monCom, -comMonto),
12434 |           }).then(() => asegurarComisionIntermediario(ordenId, instrumentacionId, intermediarioId, comMonto, monCom, ordenNumero, transNumero));
12435 |         }
12436 |         return Promise.resolve();
12437 |       });
12438 |     });
12439 |   }

```

```

12440 // Cliente ↔ Intermediario: se refleja en Pandy-Intermediario
12441 if (cob === 'cliente' && pag === 'intermediario' && intermediarioId) {
12442   inserts.push(client.from('movimientos_cuenta_corriente_intermediario').insert({
12443     intermediario_id: intermediarioId, moneda: mon, monto: -monto, orden_id: ordenId, transaccion_id: transaccionId, transaccion_numero: transNumero,
12444     concepto: conceptoCcLeyenda('compromiso_pago', ordenNumero, transNumero), fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
12445     ...montosCcPorMoneda(mon, -monto),
12446   }));
12447 }
12448 if (cob === 'intermediario' && pag === 'cliente' && intermediarioId) {
12449   inserts.push(client.from('movimientos_cuenta_corriente_intermediario').insert({
12450     intermediario_id: intermediarioId, moneda: mon, monto: -monto, orden_id: ordenId, transaccion_id: transaccionId, transaccion_numero: transNumero,
12451     concepto: conceptoCcLeyenda('pago_realizado', ordenNumero, transNumero), fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
12452     ...montosCcPorMoneda(mon, -monto),
12453   }));
12454 }
12455 if (inserts.length === 0) return Promise.resolve();
12456 return Promise.all(inserts);
12457 }
12458
12459 /**
12460  * Sincroniza CC (cliente e intermediario) y caja desde la orden y sus transacciones.
12461  * Regla única: orden + instrumentación son la fuente de verdad; CC y caja se recalculan por derivación.
12462  * Se borran todos los movimientos de esta orden y se vuelven a insertar según el estado actual de las transacciones.
12463  * Fecha contable y estado_fecha de cada fila derivada se anclan a la transacción ('fecha_ejecucion', 'updated_at'); ver `fechaYEstadoFechaMovimientoCcCajaDesdeTransaccion`.
12464  * @param {string} ordenId
12465  * @returns {Promise<void>}
12466  */
12467 function sincronizarCcYCajaDesdeOrden(ordenId) {
12468   if (!ordenId || !currentUserId) return Promise.resolve();
12469   const fecha = fechaHoyYYYYMMDDArgentina();
12470   const ahora = new Date().toISOString();
12471
12472   return client.from('ordenes').select('id, numero, cliente_id, intermediario_id, tipo_operacion_id, tipos_operacion(codigo, moneda_in, moneda_out, usa_intermediario), moneda_recibida, moneda_entregada, monto_recibido, monto_entregado, tasa_descuento_intermediario, intermediarios(nombre)').eq('id', ordenId).single()
12473     .then((rOrd) => {
12474       if (rOrd.error || !rOrd.data) return Promise.resolve();
12475       const orden = rOrd.data;
12476       const toJoin = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
12477       const codigoOrdenRaw = (toJoin && toJoin.codigo) || null;
12478       const codigoOrden = normalizarCodigoTipoOperacion(codigoOrdenRaw) || codigoOrdenRaw;
12479       // 'usa_intermediario' del catálogo o orden con intermediario: debe coincidir con el fetch a `reglas_de_negocio`.
12480       const usaIntermediario =
12481         (toJoin && toJoin.usa_intermediario === true) || !orden.intermediario_id;
12482       const codNorm = String(codigoOrden || '').toUpperCase();
12483       const miOrd = (toJoin && toJoin.moneda_in || '').toString().toUpperCase().trim();
12484       const moOrd = (toJoin && toJoin.moneda_out || '').toString().toUpperCase().trim();
12485       const patronCajaCruceUsd = patronTipoCambioOrden(miOrd, moOrd);
12486       return getReglasDeNegocio(codigoOrden, usaIntermediario).then((reglasDeNegocio) => {
12487         const tieneReglasNeg = Array.isArray(reglasDeNegocio) && reglasDeNegocio.length > 0;
12488         /** Una sola regla: si hay filas para este tipo + intermediario, el motor CC es solo `reglas_de_negocio` (sin cc_modelo_reglas). */
12489         const usarMotorReglasNegocio = tieneReglasNeg;
12490         return client.from('instrumentacion').select('id, multicontraparte_manual').eq('orden_id', ordenId).maybeSingle().then((rInst) => {
12491           if (!rInst.data || !rInst.data.id) return Promise.resolve();
12492           const instId = rInst.data.id;
12493           const multicontraparteManual = !(rInst.data && rInst.data.multicontraparte_manual);
12494           return Promise.all([
12495             client.from('transacciones').select('id, numero, tipo, monto, moneda, cobrador, pagador, estado, modo_pago_id, concepto, instrumentacion_id, pagador_cliente_id, cobrador_cliente_id, pagador_intermediario_id, cobrador_intermediario_id, fecha_ejecucion, updated_at').eq('instrumentacion_id', instId),
12496             client.from('comisiones_orden').select('moneda, monto, beneficiario').eq('orden_id', ordenId),
12497             client.from('modos_pago').select('id, codigo'),
12498             orden.intermediario_id ? client.from('intermediarios').select('nombre').eq('id', orden.intermediario_id).maybeSingle() : Promise.resolve({ data: null }),
12499           ]).then(([rTr, rCom, rModos, rIntNom]) => {
12500             const transacciones = rTr.data || [];
12501             const comisiones = rCom.data || [];
12502             const modosMap = {};
12503             (rModos.data || []).forEach((m) => { modosMap[m.id] = m.codigo || 'efectivo'; });
12504             const ordenLabel = orden.numero != null ? 'nro orden ' + orden.numero : 'nro orden ' + (ordenId || '').toString().slice(0, 8);
12505             const clienteId = orden.cliente_id || null;
12506             const intermediarioId = orden.intermediario_id || null;
12507             const intJoinRowSync = orden.intermediarios && (Array.isArray(orden.intermediarios) ? orden.intermediarios[0] : orden.intermediarios);
12508             let nombreInterSync = intJoinRowSync ? String(intJoinRowSync.nombre || '').trim() : '';
12509             if (!nombreInterSync && rIntNom && rIntNom.data && rIntNom.data.nombre != null) {
12510               nombreInterSync = String(rIntNom.data.nombre).trim();
12511             }
12512             const patronIntUsdSync = patronInstrumentacionIntDesdeTransacciones(transacciones);
12513             // Comisión fija 50/75 (Seguridad < «nacho» solo aplica a **cp_ic** en **ci_pc** siempre tasas % sobre el acuerdo (ver ordenComisionFijaNachoUsdParaPersistir). Si
12514             // acá fuera true en ci_pc, se bloqueaba mrxtasa y quedaba comisionInt=0 sin fila en CC int.
12515             const esUsdUsdInterComisionFijaSync =
12516               codNorm === 'USD-USD' &&
12517               intermediarioId &&
12518               patronIntUsdSync === 'cp_ic' &&
12519               intermediarioIdTieneComisionFijaUsdAcordada(intermediarioId, nombreInterSync);
12520             const monR = (orden.moneda_recibida || 'USD').toUpperCase();
12521             const monE = (orden.moneda_entregada || 'USD').toUpperCase();
12522             const mr = Number(orden.monto_recibido) || 0;
12523             const me = Number(orden.monto_entregado) || 0;
12524             const tasa = Number(orden.tasa_descuento_intermediario);
12525             const comisionInt = comisiones.find((c) => String(c.beneficiario || '').toLowerCase() === 'intermediario');
12526             let comisionIntMonto = comisionInt ? Number(comisionInt.monto) || 0 : 0;
12527             let comisionIntMon = (comisionInt && comisionInt.moneda) ? comisionInt.moneda.toUpperCase() : 'ARS';
12528             // Si no hay comisión en comisiones_orden pero la orden tiene tasa descuento intermediario, derivar: mr * tasa (ej. 50.000 * 0,015 = 750).
12529             // Con comision fija parametrizada **solo en cp_ic** (esUsdUsdInterComisionFijaSync no usar mrxtasa ni duplicar con inferencia 50/75; en **ci_pc** siempre se permit
12530             e mrxtasa aunque el intermediario esté en Seguridad para Fija en cp_ic.
12531             if (intermediarioId && comisionIntMonto < 1e-6 && !esUsdUsdInterComisionFijaSync && typeof tasa === 'number' && !isNaN(tasa) && tasa >= 0 && tasa < 1 && tasa > 1e-12
12532               && mr >= 1e-6) {
12533               comisionIntMonto = mr * tasa;
12534               comisionIntMon = monR;
12535             }

```

```

12532 |     }
12533 |     const comisionPandy = comisiones.find((c) => (c.beneficiario || '').toString().toLowerCase() === 'pandy');
12534 |     let comisionPandyMonto = comisionPandy ? Number(comisionPandy.monto) || 0 : 0;
12535 |     const comisionPandyMon = (comisionPandy && comisionPandy.moneda) ? String(comisionPandy.moneda).toUpperCase() : (orden.moneda_recibida || 'ARS').toUpperCase();
12536 |     // CHEQUE-ARS + intermediario: sin fila Pandy en comisiones_orden el motor no inserta la linea es_comision y el saldo cliente queda en -(mr-me). Misma idea que la de
12537 |     // comision int. por tasa (arriba); completar con el resto del spread del acuerdo.
12538 |     if (codNorm === 'CHEQUE-ARS' && intermediarioId && comisionPandyMonto < 1e-6 && mr > me + 1e-6) {
12539 |         const spreadAcuerdo = mr - me;
12540 |         comisionPandyMonto = Math.max(0, spreadAcuerdo - (comisionIntMonto > 1e-6 ? comisionIntMonto : 0));
12541 |     }
12542 |     // USD-USD cp_ic + Nacho: fila int. faltante o sin comisiones_orden; inferir 50/75 y repartir spread (también si no hay fila Pandy: asumir spread = parte Pandy impli
12543 |     cita).
12544 |     if (codNorm === 'USD-USD' && intermediarioId && patronIntUsdSync === 'cp_ic' && comisionIntMonto < 1e-6 && mr > me + 1e-6) {
12545 |         const spreadUsdAc = mr - me;
12546 |         const oaCfg = Number(usdUsdComisionFijaConfig.opcion_a) || 50;
12547 |         const obCfg = Number(usdUsdComisionFijaConfig.opcion_b) || 75;
12548 |         const minFijaUsd = Math.min(oaCfg, obCfg);
12549 |         if (spreadUsdAc >= minFijaUsd - 1e-6 && intermediarioTieneComisionFijaUsdAcordada(intermediarioId, nombreInterSync)) {
12550 |             let pandyEff = comisionPandyMonto;
12551 |             if (pandyEff < 1e-6 && spreadUsdAc > 1e-6) pandyEff = spreadUsdAc;
12552 |             const gapIntUsd = spreadUsdAc - pandyEff;
12553 |             let intInf = 0;
12554 |             if (gapIntUsd > 1e-6 && (Math.abs(gapIntUsd - oaCfg) <= 0.01 || Math.abs(gapIntUsd - obCfg) <= 0.01)) {
12555 |                 intInf = gapIntUsd;
12556 |             } else if (Math.abs(gapIntUsd) <= 0.02 && pandyEff >= spreadUsdAc - 0.02) {
12557 |                 const pref = preferenciaComisionFijaInterUsdDesdeStorage();
12558 |                 if (Math.abs(pref - obCfg) < 0.01 && spreadUsdAc >= obCfg - 1e-6) intInf = obCfg;
12559 |                 else intInf = Math.min(oaCfg, spreadUsdAc);
12560 |             }
12561 |             if (intInf >= 1e-6 && intInf <= spreadUsdAc + 1e-6) {
12562 |                 comisionIntMonto = intInf;
12563 |                 comisionIntMon = monR;
12564 |                 comisionPandyMonto = Math.max(0, spreadUsdAc - intInf);
12565 |             }
12566 |         }
12567 |     }
12568 |     // Tras inferir comision int. por tasa o por fija cp_ic: si Pandy tenía todo el spread en comisiones_orden y suma > mr-me, ajustar parte Pandy.
12569 |     if (codNorm === 'USD-USD' && intermediarioId && mr > me + 1e-6) {
12570 |         const spUsd = mr - me;
12571 |         if (comisionIntMonto >= 1e-6 && comisionPandyMonto + comisionIntMonto > spUsd + 0.02) {
12572 |             comisionPandyMonto = Math.max(0, spUsd - comisionIntMonto);
12573 |         }
12574 |     }
12575 |     const spreadAcuerdoChequeInt =
12576 |     codNorm === 'CHEQUE-ARS' && intermediarioId && mr > me + 1e-6 ? mr - me : 0;
12577 |     const montoEfectivoInt = (typeof tasa === 'number' && !isNaN(tasa) && tasa >= 0 && tasa < 1) ? mr * (1 - tasa) : mr;
12578 |     const usarMultiContraparteSync = multiContraparteManual && esTipoOpMultiContraparteElegibleDesdeOrden(orden, toJoin);
12579 |     const usarMotorEfectivo = usarMotorReglasNegocio && !usarMultiContraparteSync;
12580 |
12581 |     const rowsCcCliente = [];
12582 |     const rowsCcInt = [];
12583 |     const rowsCaja = [];
12584 |
12585 |     // Regla simple e infalible (docs/REGLA_CC_SIMPLE_INFALIBLE.md): solo transacciones ejecutadas; un movimiento por transacción por entidad; signo = pagador entidad →
12586 |     +monto, cobrador entidad → -monto. Caja: Pandy cobra +, Pandy paga -.
12587 |     // Instrumentación multiContraparte manual (docs/INSTRUMENTACION_MANUAL_MULTICONTRAPARTE.md): sin motor reglas; CC por entidad explícita; caja solo Pandy + efectivo.
12588 |     transacciones.forEach((t) => {
12589 |         if (t.estado !== 'ejecutada') return;
12590 |         const feT = fechaYEstadoFechaMovimientoCcCajaDesdeTransaccion(t, fecha, ahora);
12591 |         const transaccionId = t.id;
12592 |         const monto = Number(t.monto) || 0;
12593 |         const mon = (t.moneda || 'USD').toUpperCase();
12594 |         const cob = String(t.cobrador !== null ? t.cobrador : (t.tipo === 'ingreso' ? 'pandy' : 'cliente')).toLowerCase();
12595 |         const pag = String(t.pagador !== null ? t.pagador : (t.tipo === 'egreso' ? 'pandy' : 'cliente')).toLowerCase();
12596 |         const esGananciaTrx = (t.concepto || '').includes('Ganancia del acuerdo');
12597 |         const esComisionPandyTrx = comisionPandyMonto >= 1e-6 && (t.tipo || '').toLowerCase() === 'ingreso' && pag === 'cliente' && cob === 'pandy' && Math.abs(monto - com
12598 | isionPandyMonto) < 1e-6;
12599 |
12600 |         if (usarMultiContraparteSync && !esGananciaTrx && !esComisionPandyTrx) {
12601 |             const codigoModoMc = (modosMap[t.modo_pago_id] || 'efectivo').toLowerCase();
12602 |             // Caja solo con roles explícitos en BD: los defaults de pag/cob (p. ej, egreso sin pagador → «pandy»)
12603 |             // inventarian salida de caja de la casa en patas cliente=cliente o intermediario=cliente.
12604 |             const pagDb =
12605 |             t.pagador !== null && String(t.pagador).trim() !== '' ? String(t.pagador).toLowerCase() : null;
12606 |             const cobDb =
12607 |             t.cobrador !== null && String(t.cobrador).trim() !== '' ? String(t.cobrador).toLowerCase() : null;
12608 |             if (pagDb && cobDb && (pagDb === 'pandy' || cobDb === 'pandy') && codigoModoMc === 'efectivo') {
12609 |                 const signoCaja = cobDb === 'pandy' ? 1 : -1;
12610 |                 const conceptoCaja = conceptoCajaTransaccion(cobDb === 'pandy', mon, monto, orden.numero, t.numero);
12611 |                 rowsCaja.push({
12612 |                     moneda: mon,
12613 |                     monto: signoCaja * monto,
12614 |                     caja_tipo: 'efectivo',
12615 |                     transaccion_id: transaccionId,
12616 |                     orden_id: ordenId,
12617 |                     orden_numero: orden.numero !== null ? orden.numero : null,
12618 |                     transaccion_numero: t.numero !== null ? t.numero : null,
12619 |                     concepto: conceptoCaja,
12620 |                     fecha: feT.fecha,
12621 |                     usuario_id: currentUserId,
12622 |                 });
12623 |             }
12624 |             // CC multiContraparte: un solo lote tras el forEach (solo si hay alguna ejecutada; pendientes van en el mismo lote).
12625 |             return;
12626 |         }
12627 |     });
12628 |     // Caja solo se mueve cuando participa Pandy (pagador o cobrador).

```

```

12625|         if (pag === 'pandy' || cob === 'pandy') {
12626|             const codigoModo = modosMap[t.modo_pago_id] || 'efectivo';
12627|             const cajaTipo = codigoCajaTipoDesdeCodigo(codigoModo);
12628|             const signoCaja = cob === 'pandy' ? 1 : -1;
12629|             const conceptoCaja = esGananciaTrx ? conceptoCajaTransaccionEspecial(t.concepto, mon, monto, orden.numero, t.numero) : conceptoCajaTransaccion(cob === 'pandy', m
on, monto, orden.numero, t.numero);
12630|             rowsCaja.push({
12631|                 moneda: mon,
12632|                 monto: signoCaja * monto,
12633|                 caja_tipo: cajaTipo,
12634|                 transaccion_id: transaccionId,
12635|                 orden_id: ordenId,
12636|                 orden_numero: orden.numero !== null ? orden.numero : null,
12637|                 transaccion_numero: t.numero !== null ? t.numero : null,
12638|                 concepto: conceptoCaja,
12639|                 fecha: feT.fecha,
12640|                 usuario_id: currentUserId,
12641|             });
12642|         }
12643|         // Egreso Intermediario-Cliente (cp_ic y cruces con int.): **no** movimiento de caja de Pandy.
12644|         // El efectivo lo entrega el intermediario; la caja de la casa solo refleja transacciones donde participa Pandy
12645|         // (ingreso Cliente-Pandy, egreso Pandy-Cliente, etc.). Ver docs/CORAZON_SISTEMA_CC_Y_CAJA.md y USD_USD_CON_INTERMEDIARIO.md.
12646|         // Ingreso Cliente-Intermediario (ejecutado): no genera movimiento de caja de Pandy – el efectivo lo cobra el intermediario, no la caja de la casa.
12647|
12648|         // CC: con motor `reglas_de_negocio` se arma después; si no hay filas, lógica legacy por transacción.
12649|         if (!usarMotorEfectivo && clienteId && !esGananciaTrx && !esComisionPandyTrx && incluirEnMovimientosCcClienteModelo(orden, t)) {
12650|             if (pag === 'cliente' && cob !== 'intermediario') {
12651|                 const codigo = (toJoin && toJoin.codigo) || null;
12652|                 const esModeloCliente = esTipoOperacionChequeArs(codigo, toJoin?.moneda_in, toJoin?.moneda_out) && orden.intermediario_id;
12653|                 const esIngresoMismaMoneda = (t.tipo || '').toLowerCase() === 'ingreso' && monR === monE;
12654|                 const montoIngresoCc = esModeloCliente ? mr : (esIngresoMismaMoneda ? me : monto);
12655|                 rowsCcCliente.push({
12656|                     cliente_id: clienteId,
12657|                     orden_id: ordenId,
12658|                     transaccion_id: transaccionId,
12659|                     transaccion_numero: t.numero !== null ? t.numero : null,
12660|                     concepto: conceptoCcLeyenda('cobro_realizado', orden.numero, t.numero),
12661|                     fecha: feT.fecha,
12662|                     usuario_id: currentUserId,
12663|                     moneda: mon,
12664|                     monto: -montoIngresoCc,
12665|                     estado: 'cerrado',
12666|                     estado_fecha: feT.estado_fecha,
12667|                     ...montosCcPorMoneda(mon, -montoIngresoCc),
12668|                 });
12669|             }
12670|             if (cob === 'cliente' && pag !== 'intermediario') {
12671|                 rowsCcCliente.push({
12672|                     cliente_id: clienteId,
12673|                     orden_id: ordenId,
12674|                     transaccion_id: transaccionId,
12675|                     transaccion_numero: t.numero !== null ? t.numero : null,
12676|                     concepto: conceptoCcLeyenda('compromiso_pago', orden.numero, t.numero),
12677|                     fecha: feT.fecha,
12678|                     usuario_id: currentUserId,
12679|                     moneda: mon,
12680|                     monto: monto,
12681|                     estado: 'cerrado',
12682|                     estado_fecha: feT.estado_fecha,
12683|                     ...montosCcPorMoneda(mon, monto),
12684|                 });
12685|             }
12686|         }
12687|     }
12688|     if (!usarMotorEfectivo && intermediarioId && incluirEnMovimientosCcIntermediarioModelo(orden, t)) {
12689|         // Pandy-Int (egreso): solo -monto. La comisión se suma cuando se ejecuta Int-Pandy (ida y vuelta cerrada).
12690|         // Según modelo (Excel): egreso Pandy-Int en CC Intermediario = +200.000 (CC_INTERMEDIARIO positivo). Signo vital para reversa.
12691|         if (cob === 'intermediario' && pag === 'pandy') {
12692|             const monInt = orden.moneda_recibida || mon || 'ARS';
12693|             rowsCcInt.push({
12694|                 intermediario_id: intermediarioId,
12695|                 orden_id: ordenId,
12696|                 transaccion_id: transaccionId,
12697|                 transaccion_numero: t.numero !== null ? t.numero : null,
12698|                 moneda: monInt,
12699|                 monto: monto,
12700|                 concepto: conceptoCcLeyenda('pago_realizado', orden.numero, t.numero),
12701|                 fecha: feT.fecha,
12702|                 usuario_id: currentUserId,
12703|                 estado: 'cerrado',
12704|                 estado_fecha: feT.estado_fecha,
12705|                 ...montosCcPorMoneda(monInt, monto),
12706|             });
12707|         }
12708|         // Int-Pandy (ingreso): +montoEfectivoInt. La comisión intermediario se agrega en el bloque único después del forEach (INCLUIR EN MOV DE CC INTERMEDIARIO = Y, ce
12709|         // leste).
12710|         if (cob === 'pandy' && pag === 'intermediario') {
12711|             const monInt = orden.moneda_recibida || mon || 'ARS';
12712|             rowsCcInt.push({
12713|                 intermediario_id: intermediarioId,
12714|                 orden_id: ordenId,
12715|                 transaccion_id: transaccionId,
12716|                 transaccion_numero: t.numero !== null ? t.numero : null,
12717|                 moneda: monInt,
12718|                 monto: montoEfectivoInt,
12719|                 concepto: conceptoCcLeyenda('cobro_realizado', orden.numero, t.numero),
12720|                 fecha: feT.fecha,

```

```

12720|         usuario_id: currentUserId,
12721|         estado: 'cerrado',
12722|         estado_fecha: feT.estado_fecha,
12723|         ...montosCcPorMoneda(monInt, montoEfectivoInt),
12724|     });
12725| }
12726| if (cob === 'cliente' && pag === 'intermediario') {
12727|     rowsCcInt.push({
12728|         intermediario_id: intermediarioId,
12729|         orden_id: ordenId,
12730|         transaccion_id: transaccionId,
12731|         transaccion_numero: t.numero != null ? t.numero : null,
12732|         moneda: mon,
12733|         monto: -monto,
12734|         concepto: conceptoCcLeyenda('compromiso_pago', orden.numero, t.numero),
12735|         fecha: feT.fecha,
12736|         usuario_id: currentUserId,
12737|         estado: 'cerrado',
12738|         estado_fecha: feT.estado_fecha,
12739|         ...montosCcPorMoneda(mon, -monto),
12740|     });
12741| }
12742| if (cob === 'intermediario' && pag === 'cliente') {
12743|     rowsCcInt.push({
12744|         intermediario_id: intermediarioId,
12745|         orden_id: ordenId,
12746|         transaccion_id: transaccionId,
12747|         transaccion_numero: t.numero != null ? t.numero : null,
12748|         moneda: mon,
12749|         monto: -monto,
12750|         concepto: conceptoCcLeyenda('pago_realizado', orden.numero, t.numero),
12751|         fecha: feT.fecha,
12752|         usuario_id: currentUserId,
12753|         estado: 'cerrado',
12754|         estado_fecha: feT.estado_fecha,
12755|         ...montosCcPorMoneda(mon, -monto),
12756|     });
12757| }
12758| });
12759| });
12760|
12761| if (usarMulticontraparteSync) {
12762|     aplicarCcMulticontraparteManualConciliacionCompleta(transacciones, orden, ordenId, orden.numero, fecha, ahora, rowsCcCliente, rowsCcInt);
12763| }
12764|
12765| // Motor único: filas en `reglas_de_negocio` para (codigo, usa_intermediario). Multicontraparte manual no usa motor (evita duplicar CC).
12766| if (usarMotorEfectivo) {
12767|     aplicarMotorCcDesdeReglasDeNegocio({
12768|         tipoOperacionCodigo: codNorm,
12769|         transacciones:
12770|             reglasDeNegocio,
12771|         orden,
12772|         clienteId,
12773|         intermediarioId,
12774|         ordenId,
12775|         rowsCcCliente,
12776|         rowsCcInt,
12777|         fecha,
12778|         ahora,
12779|         comisionPandyMonto,
12780|         comisionPandyMon,
12781|         comisionIntMonto,
12782|         comisionIntMon,
12783|         montoEfectivoInt,
12784|         comisionSpreadAcuerdoClienteCheque: spreadAcuerdoChequeInt,
12785|     });
12786| } else if (esTipoOperacionChequeArs(codigoOrdenRaw, toJoin?.moneda_in, toJoin?.moneda_out) && orden.intermediario_id) {
12787|     const nroTransComisionChequeFb = nroTransIngresoClientePandyPrincipalParaComisionConcepto(transacciones, comisionPandyMonto);
12788|     const feSynthChequeFb = nroTransComisionChequeFb != null
12789|       ? fechaYEstadoFechaMovimientoCcCajaDesdeNumeroTransaccion(transacciones, nroTransComisionChequeFb, fecha, ahora)
12790|       : fechaYEstadoFechaMovimientoCcCajaDesdeUltimaEjecutada(transacciones, fecha, ahora);
12791|     const tx1EjecutadaFb = transacciones.some((t) => (t.tipo || '').toLowerCase() === 'ingreso' && String(t.pagador || '').toLowerCase() === 'cliente' && String(t.cobrador || '').toLowerCase() === 'pandy' && (t.estado || '').toLowerCase() === 'ejecutada'));
12792|     const tx2EjecutadaFb = transacciones.some((t) => (t.tipo || '').toLowerCase() === 'egreso' && String(t.pagador || '').toLowerCase() === 'pandy' && String(t.cobrado r || '').toLowerCase() === 'cliente' && (t.estado || '').toLowerCase() === 'ejecutada'));
12793|     const parClienteCerradoFb = tx1EjecutadaFb && tx2EjecutadaFb;
12794|     const montoComisionCcClienteChequeFb =
12795|       spreadAcuerdoChequeInt >= 1e-6 ? spreadAcuerdoChequeInt : comisionPandyMonto;
12796|     if (clienteId && montoComisionCcClienteChequeFb >= 1e-6 && !parClienteCerradoFb) {
12797|       rowsCcCliente.push({ cliente_id: clienteId, orden_id: ordenId, transaccion_id: null, transaccion_numero: null, concepto: conceptoCcLeyenda('comision_acuerdo', orden.numero, nroTransComisionChequeFb), fecha: feSynthChequeFb.fecha, usuario_id: currentUserId, moneda: comisionPandyMon, monto: montoComisionCcClienteChequeFb, estado: 'cerrado', estado_fecha: feSynthChequeFb.estado_fecha, ...montosCcPorMoneda(comisionPandyMon, montoComisionCcClienteChequeFb) });
12798|     }
12799|     const hayTx3Ejecutada = transacciones.some((t) => (t.tipo || '').toLowerCase() === 'egreso' && String(t.pagador || '').toLowerCase() === 'pandy' && String(t.cobrador or || '').toLowerCase() === 'intermediario' && (t.estado || '').toLowerCase() === 'ejecutada'));
12800|     const hayTx4Ejecutada = transacciones.some((t) => (t.tipo || '').toLowerCase() === 'ingreso' && String(t.pagador || '').toLowerCase() === 'intermediario' && String(t.cobrador || '').toLowerCase() === 'pandy' && (t.estado || '').toLowerCase() === 'ejecutada'));
12801|     if (intermediarioId && comisionIntMonto >= 1e-6 && (hayTx3Ejecutada || hayTx4Ejecutada || parClienteCerradoFb)) {
12802|       rowsCcInt.push({ intermediario_id: intermediarioId, orden_id: ordenId, transaccion_id: null, transaccion_numero: null, concepto: conceptoCcLeyenda('comision_acuerdo', orden.numero, nroTransComisionChequeFb), fecha: feSynthChequeFb.fecha, usuario_id: currentUserId, moneda: comisionIntMon, monto: -comisionIntMonto, estado: 'cerrado', estado_fecha: feSynthChequeFb.estado_fecha, ...montosCcPorMoneda(comisionIntMon, -comisionIntMonto) });
12803|     }
12804|     if (intermediarioId && comisionIntMonto >= 1e-6 && parClienteCerradoFb) {
12805|       const monComCajaFb = (comisionIntMon || 'ARS').toUpperCase();
12806|       rowsCaja.push({
12807|         orden_id: ordenId,
12808|         transaccion_id: null,

```

```

12809|         moneda: monComCajaFb,
12810|         monto: -comisionIntMonto,
12811|         caja_tipo: 'efectivo',
12812|         orden_numero: orden.numero != null ? orden.numero : null,
12813|         transaccion_numero: nroTransComisionChequeFb,
12814|         concepto: conceptoCajaTransaccionEspecial('Comisión del acuerdo', monComCajaFb, comisionIntMonto, orden.numero, nroTransComisionChequeFb),
12815|         fecha: feSynthChequeFb.fecha,
12816|         usuario_id: currentUserId,
12817|     });
12818| }
12819| }
12820|
12821| // Cierre sintético dos monedas (CC cliente): +montoRecibido en monR y -montoEntregado en monE cuando el "par cliente" está ejecutado.
12822| // Ingreso: Cliente-Pandy o Cliente-Intermediario (misma semántica que ingresoDesdeClienteHaciaPandyOIntermediarioEjecutado).
12823| // Egreso entrega al cliente: Pandy-Cliente **o** Intermediario-Cliente (cp ic: cobro a Pandy + entrega vía int.).
12824| // Solo **legacy** si NO corre el motor ('reglas_de_negocio'): con 'aplicarMotorCcDesdeReglasDeNegocio' el cierre duplicaría monR/monE y rompe el cierre (ej. ARS-USD
+int E,E: +5M ARS y -5k USD extra). Multicontraparte manual: CC la arma aplicarCcMulticontraparteManualConciliacionCompleta (no duplicar cierre aquí).
12825| if (clienteId && monR !== monE && !usarMotorEfectivo && !usarMulticontraparteSync) {
12826|     const trxEjecutadaCierre = (t) => (t.estado || '').toLowerCase() === 'ejecutada';
12827|     const ingresosCli = transacciones.filter((t) => {
12828|         const { pag, cob } = pagCobEfectivosTransaccionSync(t);
12829|         if (usarMulticontraparteSync) {
12830|             return (t.tipo || '').toLowerCase() === 'ingreso' && pag === 'cliente' && idClientePagadorEfectivoMulticontraparte(t, orden) === clienteId && trxEje
cutadaCierre(t);
12831|         }
12832|         return (t.tipo || '').toLowerCase() === 'ingreso' && pag === 'cliente' && (cob === 'pandy' || cob === 'intermediario') && trxEjecutadaCierre(t);
12833|     });
12834|     const egresosCli = transacciones.filter((t) => {
12835|         const { pag, cob } = pagCobEfectivosTransaccionSync(t);
12836|         if (usarMulticontraparteSync) {
12837|             return (t.tipo || '').toLowerCase() === 'egreso' && cob === 'cliente' && idClienteCobradorEfectivoMulticontraparte(t, orden) === clienteId && trxEje
cutadaCierre(t);
12838|         }
12839|         return (t.tipo || '').toLowerCase() === 'egreso' && cob === 'cliente' && (pag === 'pandy' || pag === 'intermediario') && trxEjecutadaCierre(t);
12840|     });
12841|     const montoRecibido = ingresosCli.reduce((s, t) => s + (Number(t.monto) || 0), 0);
12842|     const montoEntregado = egresosCli.reduce((s, t) => s + (Number(t.monto) || 0), 0);
12843|     const egresoRef = egresosCli.slice().sort((a, b) => (Number(a.numero) || 0) - (Number(b.numero) || 0)).pop() || null;
12844|     if (montoRecibido >= 1e-6 && montoEntregado >= 1e-6) {
12845|         const conceptoCierre = 'Cierre orden ' + (orden.numero != null ? orden.numero : ordenId);
12846|         const feCierre = fechaYEstadoFechaMovimientoCcCajaDesdeTransaccion(egresoRef, fecha, ahora);
12847|         // Saldo por moneda: -montoRecibido (cobro) + montoEntregado (compromiso) + cierre +montoRecibido en monR -montoEntregado en monE -> 0 en ambas.
12848|         // Varios ingresos/egresos parciales (ej. 5k efectivo + 5k transferencia): se suman todos; ref. de nro. trans. = último egreso.
12849|         rowsCcCliente.push({
12850|             cliente_id: clienteId,
12851|             orden_id: ordenId,
12852|             transaccion_id: egresoRef != null ? egresoRef.id : null,
12853|             transaccion_numero: egresoRef && egresoRef.numero != null ? egresoRef.numero : null,
12854|             concepto: conceptoCierre,
12855|             fecha: feCierre.fecha,
12856|             usuario_id: currentUserId,
12857|             moneda: monR,
12858|             monto: montoRecibido,
12859|             estado: 'cerrado',
12860|             estado_fecha: feCierre.estado_fecha,
12861|             ...montosCcPorMoneda(monR, montoRecibido),
12862|         });
12863|         rowsCcCliente.push({
12864|             cliente_id: clienteId,
12865|             orden_id: ordenId,
12866|             transaccion_id: egresoRef != null ? egresoRef.id : null,
12867|             transaccion_numero: egresoRef && egresoRef.numero != null ? egresoRef.numero : null,
12868|             concepto: conceptoCierre,
12869|             fecha: feCierre.fecha,
12870|             usuario_id: currentUserId,
12871|             moneda: monE,
12872|             monto: -montoEntregado,
12873|             estado: 'cerrado',
12874|             estado_fecha: feCierre.estado_fecha,
12875|             ...montosCcPorMoneda(monE, -montoEntregado),
12876|         });
12877|     }
12878| }
12879|
12880| // Misma moneda con comisión implícita (mr > me): el ingreso ya aportó -me y el egreso +me, saldo = 0. No se añade movimiento extra en CC.
12881| // La caja ya tiene +mr (ingreso) y -me (egreso); la diferencia es la comisión. Opcional: movimiento caja explícito por comisión si se desea desglose.
12882|
12883| // Evitar duplicados: clave debe distinguir dos líneas válidas con mismo monto (p. ej. ARS-USD inversa P,E: +me USD en Tx1 pendiente y en Tx2 ejecutada).
12884| const seenCli = new Set();
12885| const rowsCcClienteUnicos = (rowsCcCliente || []).filter((r) => {
12886|     const key = [
12887|         r.cliente_id,
12888|         r.orden_id,
12889|         r.transaccion_id || '',
12890|         r.transaccion_numero != null && r.transaccion_numero !== '' ? String(r.transaccion_numero) : '',
12891|         (r.moneda || '').toUpperCase(),
12892|         r.monto,
12893|         (r.concepto || '').slice(0, 72),
12894|     ].join('\t');
12895|     if (seenCli.has(key)) return false;
12896|     seenCli.add(key);
12897|     return true;
12898| });
12899| const seenInt = new Set();
12900| const rowsCcIntUnicos = (rowsCcInt || []).filter((r) => {
12901|     const key = [
12902|         r.intermediario_id,

```



```

12903 |         r.orden_id,
12904 |         r.transaccion_id || '',
12905 |         r.transaccion_numero != null && r.transaccion_numero !== '' ? String(r.transaccion_numero) : '',
12906 |         (r.moneda || '').toUpperCase(),
12907 |         r.monto,
12908 |         (r.concepto || '').slice(0, 72),
12909 |     ].join('\t');
12910 |     if (seenInt.has(key)) return false;
12911 |     seenInt.add(key);
12912 |     return true;
12913 | });
12914 | const idsTrx = transacciones.map((t) => t.id).filter(Boolean);
12915 | return client.rpc('sync_cc_caja_orden', {
12916 |   p_orden_id: ordenId,
12917 |   p_usuario_id: currentUserId,
12918 |   p_rows_cc_cliente: rowsCcClienteUnicos,
12919 |   p_rows_cc_int: rowsCcIntUnicos,
12920 |   p_rows_caja: rowsCaja,
12921 | }).then((rRpc) => {
12922 |   if (rRpc.error) {
12923 |     console.warn('sync_cc_caja_orden:', rRpc.error.message || rRpc.error);
12924 |     return fallbackSyncCcCaja(ordenId, idsTrx, rowsCcClienteUnicos, rowsCcIntUnicos, rowsCaja)
12925 |       .catch((e) => {
12926 |         showToast('Error al guardar movimientos de CC (orden ' + (orden.numero || ordenId) + '): ' + (e && (e.message || e.details) || String(e)), 'error', 8000);
12927 |         throw e;
12928 |       });
12929 |   }
12930 |   return Promise.resolve();
12931 | });
12932 | });
12933 | });
12934 | });
12935 | });
12936 | .catch((err) => {
12937 |   console.warn('sincronizarCcYCajaDesdeOrden:', err && (err.message || err.code));
12938 |   showToast('Error al sincronizar CC para una orden: ' + (err && (err.message || err.details) || String(err)), 'error', 6000);
12939 |   return Promise.resolve();
12940 | });
12941 | }
12942 |
12943 | /** Fallback cuando la RPC sync_cc_caja_orden no existe o falla: delete + insert desde el cliente (mismo resultado, sin atomicidad en una transacción). */
12944 | function fallbackSyncCcCaja(ordenId, idsTrx, rowsCcCliente, rowsCcInt, rowsCaja) {
12945 |   const promDelCc = client.from('movimientos_cuenta_corriente').delete().eq('orden_id', ordenId);
12946 |   const promDelCcInt = client.from('movimientos_cuenta_corriente_intermediario').delete().eq('orden_id', ordenId);
12947 |   const promDelCajaTrx = (idsTrx && idsTrx.length > 0)
12948 |     ? client.from('movimientos_caja').delete().in('transaccion_id', idsTrx)
12949 |     : Promise.resolve();
12950 |   const promDelCajaComision = client.from('movimientos_caja').delete().eq('orden_id', ordenId).is('transaccion_id', null);
12951 |   return Promise.all([promDelCc, promDelCcInt, promDelCajaTrx, promDelCajaComision]).then(() => {
12952 |     const inserts = [];
12953 |     (rowsCcCliente || []).forEach((row) => inserts.push(client.from('movimientos_cuenta_corriente').insert(row)));
12954 |     (rowsCcInt || []).forEach((row) => inserts.push(client.from('movimientos_cuenta_corriente_intermediario').insert(row)));
12955 |     (rowsCaja || []).forEach((row) => inserts.push(client.from('movimientos_caja').insert(row)));
12956 |     if (inserts.length === 0) return Promise.resolve();
12957 |     return Promise.all(inserts);
12958 |   });
12959 | }
12960 |
12961 | /** Tabla orden_comisiones_generadas: una fila por (orden_id, tipo) evita duplicar Ganancia Pandy y Comisión intermediario al re-ejecutar. */
12962 |
12963 | function asegurarGananciaPandy(ordenId, instrumentacionId, orden, clienteId, comisionPandyMonto, listTrx) {
12964 |   if (!ordenId || !instrumentacionId || !clienteId || !comisionPandyMonto || comisionPandyMonto < 1e-6) return Promise.resolve();
12965 |   return client.from('orden_comisiones_generadas').select('id').eq('orden_id', ordenId).eq('tipo', 'ganancia_pandy').maybeSingle()
12966 |     .then((r) => {
12967 |       if (r.data && r.data.id) return Promise.resolve();
12968 |       const ahora = new Date().toISOString();
12969 |       const fecha = fechaHoyYYYYMMDDArgentina();
12970 |       const monedaCom = orden.moneda_recibida || 'ARS';
12971 |       return client.from('modos_pago').select('id').eq('codigo', 'efectivo').maybeSingle()
12972 |         .then((rModo) => {
12973 |           const modoId = (rModo.data && rModo.data.id) || null;
12974 |           return client.from('transacciones').insert({
12975 |             instrumentacion_id: instrumentacionId, tipo: 'ingreso', modo_pago_id: modoId, moneda: monedaCom, monto: comisionPandyMonto,
12976 |             cobrador: 'pandy', pagador: 'cliente', owner: 'pandy', estado: 'ejecutada', concepto: 'Ganancia del acuerdo',
12977 |             pagador_cliente_id: clienteId,
12978 |             tipo_cambio: null, fecha_ejecucion: fecha, usuario_id: currentUserId, updated_at: ahora,
12979 |           }).select('id, numero').single();
12980 |         })
12981 |         .then((rNew) => {
12982 |           const trId = rNew.data && rNew.data.id;
12983 |           const trNumero = rNew.data && rNew.data.numero;
12984 |           if (!trId) return Promise.resolve();
12985 |           const candidatos = (listTrx || []).filter((tr) => tr.tipo === 'ingreso' && tr.pagador === 'cliente' && tr.cobrador === 'pandy' && tr.estado === 'ejecutada' && !(tr.concepto || '').includes('Ganancia'));
12986 |           const trToReduce = candidatos.reduce((best, tr) => {
12987 |             const m = Number(tr.monto) || 0;
12988 |             if (m <= comisionPandyMonto + 1e-6) return best;
12989 |             if (!best || m > Number(best.monto || 0)) return tr;
12990 |             return best;
12991 |           }, null);
12992 |           const transaccionIdReducida = trToReduce && trToReduce.id ? trToReduce.id : null;
12993 |           const conceptoGanancia = conceptoCajaTransaccionEspecial('Ganancia del acuerdo', monedaCom, comisionPandyMonto, orden.numero, trNumero);
12994 |           return client.from('movimientos_caja').insert({
12995 |             moneda: monedaCom, monto: comisionPandyMonto, caja_tipo: 'efectivo', transaccion_id: trId,
12996 |             orden_numero: orden.numero !== null ? orden.numero : null, transaccion_numero: trNumero !== null ? trNumero : null,
12997 |             concepto: conceptoGanancia, fecha, usuario_id: currentUserId,
12998 |           }).then(() => {

```

```

12999| const monR = (orden.moneda_recibida || 'USD').toUpperCase();
13000| const monE = (orden.moneda_entregada || 'USD').toUpperCase();
13001| const montosPos = montosCcPorMoneda(monedaCom, comisionPandyMonto);
13002| const montosNeg = montosCcPorMoneda(monedaCom, -comisionPandyMonto);
13003| if (monR === monE) {
13004|   return client.from('movimientos_cuenta_corriente').insert({
13005|     cliente_id: clienteId, orden_id: ordenId, transaccion_id: trId, transaccion_numero: trNumero != null ? trNumero : null,
13006|     concepto: 'Ganancia del acuerdo',
13007|     moneda: monedaCom, monto: comisionPandyMonto, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
13008|     monto_usd: montosPos.monto_usd, monto_ars: montosPos.monto_ars, monto_eur: montosPos.monto_eur,
13009|   }).then(() => client.from('movimientos_cuenta_corriente').insert({
13010|     cliente_id: clienteId, orden_id: ordenId, transaccion_id: trId, transaccion_numero: trNumero != null ? trNumero : null,
13011|     concepto: conceptoCcLeyenda("comision acuerdo", orden.numero, trNumero),
13012|     moneda: monedaCom, monto: -comisionPandyMonto, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
13013|     monto_usd: montosNeg.monto_usd, monto_ars: montosNeg.monto_ars, monto_eur: montosNeg.monto_eur,
13014|   }));
13015| }
13016| return client.from('movimientos_cuenta_corriente').insert({
13017|   cliente_id: clienteId, orden_id: ordenId, transaccion_id: trId, transaccion_numero: trNumero != null ? trNumero : null,
13018|   concepto: 'Ganancia del acuerdo',
13019|   moneda: monedaCom, monto: comisionPandyMonto, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
13020|   monto_usd: montosPos.monto_usd, monto_ars: montosPos.monto_ars, monto_eur: montosPos.monto_eur,
13021| });
13022| }).then(() => client.from('orden_comisiones_generadas').insert({
13023|   orden_id: ordenId, tipo: 'ganancia_pandy', transaccion_id: trId, transaccion_id_reducida: transaccionIdReducida || null,
13024| })).then(() => {
13025|   if (!trToReduce || !transaccionIdReducida) return Promise.resolve();
13026|   const nuevoMonto = Math.max(0, Number(trToReduce.monto) - comisionPandyMonto);
13027|   if (nuevoMonto < 1e-6) return Promise.resolve();
13028|   return client.from('transacciones').update({ monto: nuevoMonto, updated_at: ahora }).eq('id', trToReduce.id)
13029|   .then(() => client.from('movimientos_caja').select('id, monto').eq('transaccion_id', trToReduce.id).limit(1).maybeSingle())
13030|   .then((rC) => {
13031|     if (rC.data) {
13032|       const signo = (Number(rC.data.monto) || 0) >= 0 ? 1 : -1;
13033|       return client.from('movimientos_caja').update({ monto: signo * Math.max(0, Math.abs(Number(rC.data.monto) || 0) - comisionPandyMonto) }).eq('id', rC.data.id)
13034|     };
13035|     return Promise.resolve();
13036|   })
13037|   .then(() => {
13038|     return client.from('movimientos_cuenta_corriente').select('id, monto_usd, monto_ars, monto_eur').eq('orden_id', ordenId).eq('cliente_id', clienteId).eq('transa
ccion_id', trToReduce.id).limit(1).maybeSingle()
13039|     .then((rCc) => {
13040|       if (!rCc.data) return Promise.resolve();
13041|       const monR = (orden.moneda_recibida || 'USD').toUpperCase();
13042|       const monE = (orden.moneda_entregada || 'USD').toUpperCase();
13043|       const restar = comisionPandyMonto;
13044|       const updates = {};
13045|       if (monR === 'USD' && (rCc.data.monto_usd != null || rCc.data.monto_usd === 0)) updates.monto_usd = Math.max(-1e9, (Number(rCc.data.monto_usd) || 0) - rest
ar);
13046|       if (monR === 'ARS' && (rCc.data.monto_ars != null || rCc.data.monto_ars === 0)) updates.monto_ars = Math.max(-1e9, (Number(rCc.data.monto_ars) || 0) - rest
ar);
13047|       if (monR === 'EUR' && (rCc.data.monto_eur != null || rCc.data.monto_eur === 0)) updates.monto_eur = Math.max(-1e9, (Number(rCc.data.monto_eur) || 0) - rest
ar);
13048|       if (Object.keys(updates).length === 0) return Promise.resolve();
13049|       return client.from('movimientos_cuenta_corriente').update(updates).eq('id', rCc.data.id);
13050|     });
13051|   });
13052| });
13053| });
13054| });
13055| }
13056|
13057| function revertirGananciaPandy(ordenId, orden, clienteId, comisionPandyMonto) {
13058|   if (!ordenId) return Promise.resolve();
13059|   return client.from('orden_comisiones_generadas').select('transaccion_id, transaccion_id_reducida').eq('orden_id', ordenId).eq('tipo', 'ganancia_pandy').maybeSingle()
13060|   .then((r) => {
13061|     if (!r.data || !r.data.transaccion_id) return Promise.resolve();
13062|     const trId = r.data.transaccion_id;
13063|     const trReducidaId = r.data.transaccion_id_reducida || null;
13064|     const monR = orden && (orden.moneda_recibida || 'USD').toUpperCase();
13065|     const monE = orden && (orden.moneda_entregada || 'USD').toUpperCase();
13066|     const restaurar = trReducidaId && orden && clienteId && comisionPandyMonto >= 1e-6
13067|     ? client.from('transacciones').select('monto').eq('id', trReducidaId).maybeSingle()
13068|     .then((rTr) => {
13069|       if (!rTr.data) return Promise.resolve();
13070|       const montoActual = Number(rTr.data.monto) || 0;
13071|       return client.from('transacciones').update({ monto: montoActual + comisionPandyMonto, updated_at: new Date().toISOString() }).eq('id', trReducidaId)
13072|       .then(() => client.from('movimientos_caja').select('id, monto').eq('transaccion_id', trReducidaId).limit(1).maybeSingle())
13073|       .then((rC) => {
13074|         if (rC.data) {
13075|           const signo = (Number(rC.data.monto) || 0) >= 0 ? 1 : -1;
13076|           return client.from('movimientos_caja').update({ monto: signo * (Math.abs(Number(rC.data.monto) || 0) + comisionPandyMonto) }).eq('id', rC.data.id);
13077|         }
13078|         return Promise.resolve();
13079|       })
13080|       .then(() => {
13081|         return client.from('movimientos_cuenta_corriente').select('id, monto_usd, monto_ars, monto_eur').eq('orden_id', ordenId).eq('cliente_id', clienteId).eq('tran
saccion_id', trReducidaId).limit(1).maybeSingle()
13082|         .then((rCc) => {
13083|           if (!rCc.data) return Promise.resolve();
13084|           const monR = orden && (orden.moneda_recibida || 'USD').toUpperCase();
13085|           const sumar = comisionPandyMonto;
13086|           const updates = {};
13087|           if (monR === 'USD' && (rCc.data.monto_usd != null || rCc.data.monto_usd === 0)) updates.monto_usd = (Number(rCc.data.monto_usd) || 0) + sumar;
13088|           if (monR === 'ARS' && (rCc.data.monto_ars != null || rCc.data.monto_ars === 0)) updates.monto_ars = (Number(rCc.data.monto_ars) || 0) + sumar;
13089|           if (monR === 'EUR' && (rCc.data.monto_eur != null || rCc.data.monto_eur === 0)) updates.monto_eur = (Number(rCc.data.monto_eur) || 0) + sumar;

```

```

13090 |         if (Object.keys(updates).length === 0) return Promise.resolve();
13091 |         return client.from('movimientos_cuenta_corriente').update(updates).eq('id', rCc.data.id);
13092 |     });
13093 | });
13094 |     });
13095 |     : Promise.resolve();
13096 |     return restaurar.then(() => client.from('movimientos_cuenta_corriente').delete().eq('transaccion_id', trId))
13097 |     .then(() => client.from('movimientos_caja').delete().eq('transaccion_id', trId))
13098 |     .then(() => client.from('transacciones').delete().eq('id', trId))
13099 |     .then(() => client.from('orden_comisiones_generadas').delete().eq('orden_id', ordenId).eq('tipo', 'ganancia_pandy'));
13100 | });
13101 | }
13102 |
13103 | /** Comisión intermediario: solo caja (sin fila en `transacciones`). La CC de comisión ya la inserta el flujo que ejecuta la transacción cliente=Pandy. Requiere migración `sql
13104 | /migracion_orden_comisiones_movimiento_caja.sql` para `movimiento_caja_id` y `transaccion_id` nullable. */
13105 | function asegurarComisionIntermediario(ordenId, instrumentacionId, intermediarioId, montoCom, monCom, ordenNumero, transaccionNumeroRef) {
13106 |     void instrumentacionId;
13107 |     if (!ordenId || !intermediarioId || !montoCom || montoCom < 1e-6) return Promise.resolve();
13108 |     const ordNum = ordenNumero != null ? ordenNumero : null;
13109 |     const nroRef = transaccionNumeroRef != null ? transaccionNumeroRef : null;
13110 |     return client.from('orden_comisiones_generadas').select('id').eq('orden_id', ordenId).eq('tipo', 'comision_intermediario').maybeSingle()
13111 |     .then((r) => {
13112 |         if (r.data && r.data.id) return Promise.resolve();
13113 |         const fecha = fechaHoyYYYYMMDDArgentina();
13114 |         const conceptoCom = conceptoCajaTransaccionEspecial('Comisión del acuerdo', monCom, montoCom, ordNum, nroRef);
13115 |         return client.from('movimientos_caja').insert({
13116 |             orden_id: ordenId,
13117 |             moneda: monCom,
13118 |             monto: -montoCom,
13119 |             caja_tipo: 'efectivo',
13120 |             transaccion_id: null,
13121 |             orden_numero: ordNum,
13122 |             transaccion_numero: nroRef,
13123 |             concepto: conceptoCom,
13124 |             fecha,
13125 |             usuario_id: currentUserId,
13126 |         }).select('id').single()
13127 |         .then((rCaja) => {
13128 |             const mcId = rCaja.data && rCaja.data.id;
13129 |             const row = { orden_id: ordenId, tipo: 'comision_intermediario', transaccion_id: null };
13130 |             if (mcId) row.movimiento_caja_id = mcId;
13131 |             return client.from('orden_comisiones_generadas').insert(row);
13132 |         });
13133 |     });
13134 | }
13135 |
13136 | function revertirComisionIntermediario(ordenId) {
13137 |     if (!ordenId) return Promise.resolve();
13138 |     return client.from('orden_comisiones_generadas').select('transaccion_id, movimiento_caja_id').eq('orden_id', ordenId).eq('tipo', 'comision_intermediario').maybeSingle()
13139 |     .then((r) => {
13140 |         if (!r.data) return Promise.resolve();
13141 |         const trId = r.data.transaccion_id;
13142 |         const mcId = r.data.movimiento_caja_id;
13143 |         const delRow = () => client.from('orden_comisiones_generadas').delete().eq('orden_id', ordenId).eq('tipo', 'comision_intermediario');
13144 |         if (trId) {
13145 |             return client.from('movimientos_cuenta_corriente_intermediario').delete().eq('transaccion_id', trId)
13146 |             .then(() => client.from('movimientos_caja').delete().eq('transaccion_id', trId))
13147 |             .then(() => client.from('transacciones').delete().eq('id', trId))
13148 |             .then(() => delRow());
13149 |         }
13150 |         if (mcId) {
13151 |             return client.from('movimientos_caja').delete().eq('id', mcId).then(() => delRow());
13152 |         }
13153 |         return delRow();
13154 |     });
13155 | }
13156 |
13157 | /**
13158 | * Momento cero CC intermediario (CHEQUE + int.): referencia transacciones Cliente=Pandy (Tx1/Tx2) para conceptos;
13159 | * el circuito Pandy=Intermediario (Tx3/Tx4) se instrumenta aparte.
13160 | * transIngresoClienteId = ingreso cheque Cliente=Pandy; transEgresoClienteId = egreso efectivo Pandy-Cliente.
13161 | */
13162 | function insertarMovimientosCcMomentoCeroIntermediario(ordenId, orden, transIngresoClienteId, transEgresoClienteId) {
13163 |     const intermediarioId = orden.intermediario_id;
13164 |     if (!intermediarioId || !ordenId || !currentUserId || !transIngresoClienteId || !transEgresoClienteId) return Promise.resolve();
13165 |     const mr = Number(orden.monto_recibido) || 0;
13166 |     const monR = orden.moneda_recibida || 'ARS';
13167 |     if (mr < 1e-6) return Promise.resolve();
13168 |     const fecha = fechaHoyYYYYMMDDArgentina();
13169 |     return client.from('ordenes').select('numero').eq('id', ordenId).single()
13170 |     .then((rOrd) => rOrd.data?.numero)
13171 |     .then((ordenNum) =>
13172 |         client.from('transacciones').select('id, numero').in('id', [transIngresoClienteId, transEgresoClienteId]).then((rTr) => {
13173 |             const trs = rTr.data || [];
13174 |             const nroIngresoCli = trs.find((x) => x.id === transIngresoClienteId)?.numero;
13175 |             const nroEgresoCli = trs.find((x) => x.id === transEgresoClienteId)?.numero;
13176 |             const conceptoDebe = conceptoCcLeyenda('compromiso_pago', ordenNum, nroIngresoCli);
13177 |             const conceptoComp = conceptoCcLeyenda('compromiso_cobrar', ordenNum, nroEgresoCli);
13178 |             const row1 = {
13179 |                 intermediario_id: intermediarioId,
13180 |                 orden_id: ordenId,
13181 |                 transaccion_id: transIngresoClienteId,
13182 |                 transaccion_numero: nroIngresoCli != null ? nroIngresoCli : null,
13183 |                 concepto: conceptoDebe,
13184 |                 moneda: monR,
13185 |                 monto: -mr,
13186 |                 monto_usd: numCc(monR === 'USD' ? -mr : 0),

```

```

13186|     monto_ars: numCc(monR === 'ARS' ? -mr : 0),
13187|     monto_eur: numCc(monR === 'EUR' ? -mr : 0),
13188|     fecha,
13189|     usuario_id: currentUserId,
13190|     estado: 'pendiente',
13191|   };
13192|   const row2 = {
13193|     intermediario_id: intermediarioId,
13194|     orden_id: ordenId,
13195|     transaccion_id: transEgresoClienteId,
13196|     transaccion_numero: nroEgresoCli !== null ? nroEgresoCli : null,
13197|     concepto: conceptoComp,
13198|     moneda: monR,
13199|     monto: mr,
13200|     monto_usd: numCc(monR === 'USD' ? mr : 0),
13201|     monto_ars: numCc(monR === 'ARS' ? mr : 0),
13202|     monto_eur: numCc(monR === 'EUR' ? mr : 0),
13203|     fecha,
13204|     usuario_id: currentUserId,
13205|     estado: 'pendiente',
13206|   };
13207|   return Promise.all([
13208|     client.from('movimientos_cuenta_corriente_intermediario').insert(row1),
13209|     client.from('movimientos_cuenta_corriente_intermediario').insert(row2),
13210|   ]);
13211| }
13212| ).then(() => {}).catch((err) => {
13213|   console.warn('insertarMovimientosCcMomentoCeroIntermediario:', err && (err.message || err.code));
13214|   throw err;
13215| });
13216| }
13217|
13218| /**
13219|  * Operación CHEQUE (ARS) con intermediario: 4 transacciones por defecto (instrumentación explícita).
13220|  * 1) Ingreso cheque Cliente-Pandy · 2) Egreso efectivo Pandy-Cliente · 3) Egreso cheque Pandy-Intermediario ·
13221|  * 4) Ingreso efectivo Intermediario-Pandy (monto neto con tasa). No son compensatorias automáticas al editar.
13222|  * Momento cero CC cliente e intermediario anclado a Tx1/Tx2.
13223|  */
13224| function autoCompleteInstrumentacionChequeConIntermediario(ordenId, instrumentacionId, orden) {
13225|   if (!ordenId || !instrumentacionId || !orden || !orden.intermediario_id || !orden.tipo_operacion_id) return Promise.resolve();
13226|   const mr = Number(orden.monto_recibido) || 0;
13227|   const me = Number(orden.monto_entregado) || 0;
13228|   const tasa = Number(orden.tasa_descuento_intermediario);
13229|   const montoEfectivoInt = (typeof tasa === 'number' && !isNaN(tasa) && tasa >= 0 && tasa < 1) ? mr * (1 - tasa) : mr;
13230|   return client.from('modos_pago').select('id, codigo').in('codigo', ['efectivo', 'cheque']).then((rModos) => {
13231|     const byCodigo = {};
13232|     (rModos.data || []).forEach((m) => { byCodigo[m.codigo] = m.id; });
13233|     const modoEfectivoId = byCodigo.efectivo;
13234|     const modoChequeId = byCodigo.cheque;
13235|     if (!modoEfectivoId || !modoChequeId) return Promise.resolve();
13236|     const ahora = new Date().toISOString();
13237|     const row1 = { instrumentacion_id: instrumentacionId, tipo: 'ingreso', modo_pago_id: modoChequeId, moneda: 'ARS', monto: mr, cobrador: 'pandy', pagador: 'cliente', owner:
13238| 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: null, updated_at: ahora };
13239|     const row2 = { instrumentacion_id: instrumentacionId, tipo: 'egreso', modo_pago_id: modoEfectivoId, moneda: 'ARS', monto: me, cobrador: 'cliente', pagador: 'pandy', owner:
13240| 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: null, updated_at: ahora };
13241|     const row3 = { instrumentacion_id: instrumentacionId, tipo: 'egreso', modo_pago_id: modoChequeId, moneda: 'ARS', monto: mr, cobrador: 'intermediario', pagador: 'pandy', ow
13242| ner: 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: null, updated_at: ahora };
13243|     const row4 = { instrumentacion_id: instrumentacionId, tipo: 'ingreso', modo_pago_id: modoEfectivoId, moneda: 'ARS', monto: montoEfectivoInt, cobrador: 'pandy', pagador: 'i
13244| ntermediario', owner: 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: null, updated_at: ahora };
13245|     const rows = [row1, row2, row3, row4];
13246|     return rows.reduce((prom, row) => prom.then((ids) => {
13247|       client.from('transacciones').insert(row).select('id').single().then((rIns) => {
13248|         const id = rIns.data && rIns.data.id;
13249|         return id ? ids.concat([id]) : ids;
13250|       })
13251|     })), Promise.resolve([])).then((ids) => {
13252|       const id1 = ids[0];
13253|       const id2 = ids[1];
13254|       if ((id1 || id2) && ids.length < 4) return actualizarEstadoOrden(ordenId);
13255|       return insertarMovimientosCcMomentoCero(ordenId, orden, id1, id2)
13256|         .then(() => insertarMovimientosCcMomentoCeroIntermediario(ordenId, orden, id1, id2))
13257|         .then(() => actualizarEstadoOrden(ordenId));
13258|     });
13259|   });
13260| }
13261|
13262| /**
13263|  * Inserta en movimientos_cuenta_corriente los registros "momento cero" (regla simple):
13264|  * un registro por moneda del compromiso, concepto "Compromiso - Orden Nro X y Trans Nro Y".
13265|  * Cada fila impacta solo una moneda. orden: { cliente_id, numero, moneda_recibida, moneda_entregada, monto_recibido, monto_entregado }.
13266|  */
13267| function insertarMovimientosCcMomentoCero(ordenId, orden, ingresoId, egresoId) {
13268|   const clienteId = orden.cliente_id;
13269|   if (!clienteId || !ordenId || !currentUserId) return Promise.resolve();
13270|   const monR = orden.moneda_recibida || 'USD';
13271|   const monE = orden.moneda_entregada || 'USD';
13272|   const mr = Number(orden.monto_recibido) || 0;
13273|   const me = Number(orden.monto_entregado) || 0;
13274|   const fecha = fechaHoyYYYYMMDDArgentina();
13275|   const ahora = new Date().toISOString();
13276|
13277|   return client.from('transacciones').select('id, numero').in('id', [ingresoId, egresoId]).then((rTr) => {
13278|     const trs = rTr.data || [];
13279|     const nroIngreso = trs.find((x) => x.id === ingresoId)?.numero;
13280|     const nroEgreso = trs.find((x) => x.id === egresoId)?.numero;
13281|     const ordenNum = orden.numero !== null ? orden.numero : null;
13282|     // Alineado a reglas_de_negocio + motor: moneda recibida (ingreso C-Pandy pendiente) = «Compromiso a Cobrar» con monto positivo (pendiente de cobro); moneda entregada (e

```

```

13279 greso Pandy-C pendiente) = «Compromiso de Pago» con -me.
13280 const conceptoMonR = conceptoCcLeyenda('compromiso_cobrar', ordenNum, nroIngreso);
13281 const conceptoMonE = conceptoCcLeyenda('compromiso_pago', ordenNum, nroEgreso);
13282 // Una fila por moneda: solo esa moneda con valor; el resto 0.
13283 const rowMonR = {
13284   cliente_id: clienteId,
13285   orden_id: ordenId,
13286   transaccion_id: ingresoId,
13287   transaccion_numero: nroIngreso !== null ? nroIngreso : null,
13288   concepto: conceptoMonR,
13289   fecha,
13290   usuario_id: currentUserId,
13291   moneda: monR,
13292   monto: mr,
13293   monto_usd: numCc(monR === 'USD' ? mr : 0),
13294   monto_ars: numCc(monR === 'ARS' ? mr : 0),
13295   monto_eur: numCc(monR === 'EUR' ? mr : 0),
13296 };
13297 const rowMonE = {
13298   cliente_id: clienteId,
13299   orden_id: ordenId,
13300   transaccion_id: egresoId,
13301   transaccion_numero: nroEgreso !== null ? nroEgreso : null,
13302   concepto: conceptoMonE,
13303   fecha,
13304   usuario_id: currentUserId,
13305   moneda: monE,
13306   monto: -me,
13307   monto_usd: numCc(monE === 'USD' ? -me : 0),
13308   monto_ars: numCc(monE === 'ARS' ? -me : 0),
13309   monto_eur: numCc(monE === 'EUR' ? -me : 0),
13310 };
13311 return Promise.all([
13312   client.from('movimientos_cuenta_corriente').insert(rowMonR),
13313   client.from('movimientos_cuenta_corriente').insert(rowMonE),
13314 ]);
13315 }).catch((err) => {
13316   console.warn('insertarMovimientosCcMomentoCero:', err && (err.message || err.code));
13317   return Promise.resolve();
13318 });
13319 }
13320 /**
13321  * Al activar multicontraparte manual (ARS-USD / USD-ARS sin int.): quita la pareja sugerida por el sistema
13322  * (ingreso Cliente-Pandy en monR + egreso Pandy-Cliente en monE) para evitar que queden vivas y se editen
13323  * mezcladas con patas libres. No borra ingresos con cobrador distinto de Pandy (p. ej. pago a tercero).
13324  */
13325 function borrarTransaccionesPlantillaEstandarParaMulticontraparte(instrumentacionId, orden) {
13326   if (!instrumentacionId || !orden) return Promise.resolve(0);
13327   const monR = (orden.moneda_recibida || 'USD').toUpperCase();
13328   const monE = (orden.moneda_entregada || 'USD').toUpperCase();
13329   return client
13330     .from('transacciones')
13331     .select('id, tipo, moneda, cobrador, pagador')
13332     .eq('instrumentacion_id', instrumentacionId)
13333     .then((r) => {
13334       if (r.error) return Promise.reject(r.error);
13335       const list = r.data || [];
13336       const ids = [];
13337       list.forEach((t) => {
13338         const tipo = (t.tipo || '').toLowerCase();
13339         const mon = (t.moneda || 'USD').toUpperCase();
13340         const { pag, cob } = pagCobEfectivosTransaccionSync(t);
13341         if (tipo === 'ingreso' && pag === 'cliente' && cob === 'pandy' && mon === monR) ids.push(t.id);
13342         else if (tipo === 'egreso' && pag === 'pandy' && cob === 'cliente' && mon === monE) ids.push(t.id);
13343       });
13344       if (ids.length === 0) return Promise.resolve(0);
13345       const borrarUna = (trId) =>
13346         Promise.all([
13347           client.from('movimientos_cuenta_corriente').delete().eq('transaccion_id', trId),
13348           client.from('movimientos_cuenta_corriente_intermediario').delete().eq('transaccion_id', trId),
13349           client.from('movimientos_caja').delete().eq('transaccion_id', trId),
13350         ]).then((results) => {
13351           const errDel = results.find((x) => x.error);
13352           if (errDel && errDel.error) return Promise.reject(errDel.error);
13353           return client.from('transacciones').delete().eq('id', trId).then((rDel) => {
13354             if (rDel.error) return Promise.reject(rDel.error);
13355           });
13356         });
13357       return ids.reduce((p, id) => p.then(() => borrarUna(id)), Promise.resolve()).then(() => ids.length);
13358     });
13359 }
13360 /**
13361  * Si la orden es sin intermediario y la instrumentación está vacía, crea dos transacciones por defecto
13362  * (ingreso moneda recibida, egreso moneda entregada). Por **monedas del tipo** (IN/OUT), no solo por código:
13363  * USD-USD, cualquier cruce fiat+USD (ARS, EUR, ... vía patronTipoCambioOrden) y CHEQUE-ARS (ARS/ARS vía cheque).
13364  * Modo de pago efectivo, estado pendiente.
13365  */
13366 function autoCompletarInstrumentacionSinIntermediario(ordenId, instrumentacionId, orden) {
13367   if (!ordenId || !instrumentacionId || !orden || !orden.tipo_operacion_id) return Promise.resolve();
13368   return client.from('tipos_operacion').select('codigo, moneda_in, moneda_out').eq('id', orden.tipo_operacion_id).single().then((rTipo) => {
13369     const row = rTipo.data[0] || {};
13370     const codigo = row.codigo || '';
13371     // Con intermediario no entra aquí (par+USD+int → autoCompletarInstrumentacionUsdUsdConIntermediario; CHEQUE-ARS+int → cheque 4 tx).
13372     if (orden.intermediario_id) return Promise.resolve();
13373     const mi = (row.moneda_in || '').toString().toUpperCase().trim();

```

```

13375|     const mo = (row.moneda_out || '').toString().toUpperCase().trim();
13376|     const patronTc = patronTipoCambioOrden(mi, mo);
13377|     const esUsdUsd = mi === 'USD' && mo === 'USD';
13378|     const esArsArs = esTipoOperacionChequeArs(codigo, row.moneda_in, row.moneda_out);
13379|     if (!esUsdUsd && !esArsArs && !patronTc) return Promise.resolve();
13380|     if (!patronTc && !(Number(ordenes.cotizacion) > 0)) return Promise.resolve();
13381|     return client.from('modos_pago').select('id').eq('codigo', 'efectivo').maybeSingle().then((rModo) => {
13382|       const modoPagoEfectivoId = (rModo.data && rModo.data.id) || null;
13383|       if (!modoPagoEfectivoId) return Promise.resolve();
13384|       const mr = Number(ordenes.monto_recibido) || 0;
13385|       const me = Number(ordenes.monto_entregado) || 0;
13386|       const monR = ordenes.moneda_recibida || 'USD';
13387|       const monE = ordenes.moneda_entregada || 'USD';
13388|       const cotizacion = Number(ordenes.cotizacion) || null;
13389|       const ahora = new Date().toISOString();
13390|       const rows = [];
13391|       if (esUsdUsd) {
13392|         rows.push({ instrumentacion_id: instrumentacionId, tipo: 'ingreso', modo_pago_id: modoPagoEfectivoId, moneda: monR, monto: mr, cobrador: 'pandy', pagador: 'cliente', o
wner: 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: null, updated_at: ahora });
13393|         rows.push({ instrumentacion_id: instrumentacionId, tipo: 'egreso', modo_pago_id: modoPagoEfectivoId, moneda: monE, monto: me, cobrador: 'cliente', pagador: 'pandy', ow
ner: 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: null, updated_at: ahora });
13394|       } else if (esArsArs) {
13395|         rows.push({ instrumentacion_id: instrumentacionId, tipo: 'ingreso', modo_pago_id: modoPagoEfectivoId, moneda: 'ARS', monto: mr, cobrador: 'pandy', pagador: 'cliente',
owner: 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: null, updated_at: ahora });
13396|         rows.push({ instrumentacion_id: instrumentacionId, tipo: 'egreso', modo_pago_id: modoPagoEfectivoId, moneda: 'ARS', monto: me, cobrador: 'cliente', pagador: 'pandy', o
wner: 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: null, updated_at: ahora });
13397|       } else if (esPatronCompraFiatConTc(patronTc)) {
13398|         rows.push({ instrumentacion_id: instrumentacionId, tipo: 'ingreso', modo_pago_id: modoPagoEfectivoId, moneda: monR, monto: mr, cobrador: 'pandy', pagador: 'cliente', o
wner: 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: cotizacion, updated_at: ahora });
13399|         rows.push({ instrumentacion_id: instrumentacionId, tipo: 'egreso', modo_pago_id: modoPagoEfectivoId, moneda: monE, monto: me, cobrador: 'cliente', pagador: 'pandy', ow
ner: 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: null, updated_at: ahora });
13400|       } else if (esPatronVendeFiatConTc(patronTc)) {
13401|         rows.push({ instrumentacion_id: instrumentacionId, tipo: 'ingreso', modo_pago_id: modoPagoEfectivoId, moneda: monR, monto: mr, cobrador: 'pandy', pagador: 'cliente', o
wner: 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: null, updated_at: ahora });
13402|         rows.push({ instrumentacion_id: instrumentacionId, tipo: 'egreso', modo_pago_id: modoPagoEfectivoId, moneda: monE, monto: me, cobrador: 'cliente', pagador: 'pandy', ow
ner: 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: cotizacion, updated_at: ahora });
13403|       }
13404|       // Orden por pagador (Cliente, Pandy). Insertar en secuencia para que numero quede 1, 2.
13405|       return rows.reduce((prom, row) => prom.then(() => client.from('transacciones').insert(row).select('id').single()), Promise.resolve()).then(() => {
13406|         return actualizarEstadoOrden(ordenesId);
13407|       });
13408|     });
13409|   });
13410| }
13411|
13412| /**
13413|  * Patrón de instrumentación con intermediario elegido en el wizard ('cp_ic' | 'ci_pc'). Fuera del wizard (panel) se fuerza 'cp_ic' al autocompletar.
13414|  */
13415| function getOrdenPatronInstrumentacionInt() {
13416|   const hid = document.getElementById('orden-int-patron');
13417|   const v = hid && String(hid.value || '').trim().toLowerCase();
13418|   return v === 'ci_pc' ? 'ci_pc' : 'cp_ic';
13419| }
13420|
13421| /**
13422|  * Par cliente con intermediario: 2 transacciones (mismo patrón que USD-USD+int) para USD-USD o cualquier cruce fiat+USD según moneda_in/out.
13423|  * - 'cp_ic' (defecto): ingreso Cliente-Pandy (mr), egreso Intermediario-Cliente (me).
13424|  * - 'ci_pc': ingreso Cliente-Intermediario (mr), egreso Pandy-Cliente (me).
13425|  * Cruce con USD: 'tipo_cambio' en la pata que corresponde (compra usd / vende usd). No aplica a CHEQUE-ARS.
13426|  * @param {string} [patron] Si no se pasa, se usa 'getOrdenPatronInstrumentacionInt()' en contexto wizard.
13427|  */
13428| function autoCompletarInstrumentacionUsdUsdConIntermediario(ordenesId, instrumentacionId, orden, patron) {
13429|   if (!ordenId || !instrumentacionId || !orden || !orden.intermediario_id || !orden.tipo_operacion_id) return Promise.resolve();
13430|   const pat = patron || (typeof document !== 'undefined' ? getOrdenPatronInstrumentacionInt() : 'cp_ic');
13431|   return client.from('tipos_operacion').select('codigo, moneda_in, moneda_out').eq('id', orden.tipo_operacion_id).single().then((rTipo) => {
13432|     const rowTipo = rTipo.data || {};
13433|     const codigoTipo = rowTipo.codigo || '';
13434|     if (esTipoOperacionChequeArs(codigoTipo, rowTipo.moneda_in, rowTipo.moneda_out)) return Promise.resolve();
13435|     const mi = (rowTipo.moneda_in || '').toString().toUpperCase().trim();
13436|     const mo = (rowTipo.moneda_out || '').toString().toUpperCase().trim();
13437|     const patronTc = patronTipoCambioOrden(mi, mo);
13438|     const esUsdUsd = mi === 'USD' && mo === 'USD';
13439|     if (!esUsdUsd && !patronTc) return Promise.resolve();
13440|     const cotizacion = Number(ordenes.cotizacion) || null;
13441|     if (patronTc && !(cotizacion > 0)) return Promise.resolve();
13442|     let tcIngreso = null;
13443|     let tcEgreso = null;
13444|     if (esPatronCompraFiatConTc(patronTc)) tcIngreso = cotizacion;
13445|     else if (esPatronVendeFiatConTc(patronTc)) tcEgreso = cotizacion;
13446|     return client.from('modos_pago').select('id').eq('codigo', 'efectivo').maybeSingle().then((rModo) => {
13447|       const modoPagoEfectivoId = (rModo.data && rModo.data.id) || null;
13448|       if (!modoPagoEfectivoId) return Promise.resolve();
13449|       const mr = Number(ordenes.monto_recibido) || 0;
13450|       const me = Number(ordenes.monto_entregado) || 0;
13451|       const monR = ordenes.moneda_recibida || 'USD';
13452|       const monE = ordenes.moneda_entregada || 'USD';
13453|       const ahora = new Date().toISOString();
13454|       const rows = pat === 'ci_pc'
13455|         ? [
13456|           { instrumentacion_id: instrumentacionId, tipo: 'ingreso', modo_pago_id: modoPagoEfectivoId, moneda: monR, monto: mr, cobrador: 'intermediario', pagador: 'cliente',
owner: 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: tcIngreso, updated_at: ahora },
13457|           { instrumentacion_id: instrumentacionId, tipo: 'egreso', modo_pago_id: modoPagoEfectivoId, moneda: monE, monto: me, cobrador: 'cliente', pagador: 'pandy', owner: '
pandy', estado: 'pendiente', concepto: '', tipo_cambio: tcEgreso, updated_at: ahora },
13458|         ]
13459|         : [
13460|           { instrumentacion_id: instrumentacionId, tipo: 'ingreso', modo_pago_id: modoPagoEfectivoId, moneda: monR, monto: mr, cobrador: 'pandy', pagador: 'cliente', owner:
'pandy', estado: 'pendiente', concepto: '', tipo_cambio: tcIngreso, updated_at: ahora },

```

```

13461 |         { instrumentacion_id: instrumentacionId, tipo: 'egreso', modo_pago_id: modoPagoEfectivoId, moneda: monE, monto: me, cobrador: 'cliente', pagador: 'intermediario',
owner: 'pandy', estado: 'pendiente', concepto: '', tipo_cambio: tcEgreso, updated_at: ahora },
13462 |     ];
13463 |     return rows.reduce((prom, row) => prom.then(() => client.from('transacciones').insert(row).select('id').single()), Promise.resolve()).then(() => actualizarEstadoOrden(or
denId));
13464 |     });
13465 |     });
13466 | }
13467 |
13468 | function expandOrdenTransacciones(ordenId, orden) {
13469 |     const detailRow = document.getElementById('orden-detalle-' + ordenId);
13470 |     if (!detailRow) return;
13471 |     const wasOpen = detailRow.style.display !== 'none';
13472 |     document.querySelectorAll('.orden-detalle-tr').forEach((tr) => { tr.style.display = 'none'; });
13473 |     if (wasOpen) {
13474 |         transaccionesOrdenIdActual = null;
13475 |         return;
13476 |     }
13477 |     transaccionesOrdenIdActual = ordenId;
13478 |     detailRow.style.display = 'table-row';
13479 |     const panel = document.getElementById('panel-orden-' + ordenId);
13480 |     const encabezado = panel.querySelector('.orden-detalle-encabezado');
13481 |     const loadingEl = panel.querySelector('.orden-detalle-loading');
13482 |     const contentEl = panel.querySelector('.orden-detalle-content');
13483 |     const tbody = panel.querySelector('.orden-detalle-tbody');
13484 |     if (!encabezado || !loadingEl || !contentEl || !tbody) return;
13485 |     const ordenAnulada = orden && String(orden.estado || '') === 'anulada';
13486 |     const canIngresarTr = userPermissions.includes('ingresar transacciones') && !ordenAnulada;
13487 |     const canEditarTr = userPermissions.includes('editar transacciones') && !ordenAnulada;
13488 |     const canEliminarTr = userPermissions.includes('eliminar transacciones') && !ordenAnulada;
13489 |     const btnNuevaTr = panel.querySelector('.btn-nueva-transaccion-panel');
13490 |     if (btnNuevaTr) btnNuevaTr.style.display = canIngresarTr ? '' : 'none';
13491 |
13492 |     const estadoLabelOrd = (e) => ({ pendiente_instrumentar: 'Pendiente Instrumentar', instrumentacion_parcial: 'Instrumentación Parcial', instrumentacion_cerrada_ejecucion: 'Ce
rrada en Ejecución', orden_ejecutada: 'Orden Ejecutada', anulada: 'Anulada' }[e] || (e || '-'));
13493 |     const estadoBadgeOrd = (e) => (e && ['pendiente_instrumentar', 'instrumentacion_parcial', 'instrumentacion_cerrada_ejecucion', 'orden_ejecutada', 'anulada'].includes(e) ? `<
span class="badge badge-estado-${e.replace(/_/g, ' ')}">${estadoLabelOrd(e)}</span>` : estadoLabelOrd(e));
13494 |     encabezado.innerHTML = orden
13495 |     ? `<div class="orden-detalle-resumen"><strong>Orden ${orden.numero != null ? '#' + orden.numero : ''}</strong> ${orden.fecha || ''}.toString().slice(0, 10)} · Estado: ${e
stadoBadgeOrd(orden.estado)} · ${orden.moneda_recibida} ${formatImporteDisplay(orden.monto_recibido)} → ${orden.moneda_entregada} ${formatImporteDisplay(orden.monto_entregado)}</div>
` : '';
13496 |     loadingEl.style.display = 'block';
13497 |     contentEl.style.display = 'none';
13498 |     tbody.innerHTML = '';
13499 |
13500 |     client
13501 |     .from('instrumentacion')
13502 |     .select('id, multicontraparte_manual')
13503 |     .eq('orden_id', ordenId)
13504 |     .maybeSingle()
13505 |     .then((r) => {
13506 |         const rowEx = r.data;
13507 |         const instId = rowEx && rowEx.id;
13508 |         if (!instId) {
13509 |             if (ordenAnulada) {
13510 |                 return Promise.resolve({ skipInstrumentacion: true });
13511 |             }
13512 |             return client.from('instrumentacion').insert({ orden_id: ordenId }).select('id, multicontraparte_manual').single().then((ins) => ({
13513 |                 instrumentacionId: ins.data ? ins.data.id : null,
13514 |                 mcRow: ins.data || null,
13515 |             }));
13516 |         }
13517 |         return { instrumentacionId: instId, mcRow: rowEx };
13518 |     })
13519 |     .then((ctx) => {
13520 |         if (ctx && ctx.skipInstrumentacion) {
13521 |             loadingEl.style.display = 'none';
13522 |             contentEl.style.display = 'block';
13523 |             tbody.innerHTML = `<tr><td colspan="9">No hay instrumentación para esta orden anulada.</td></tr>`;
13524 |             if (btnNuevaTr) btnNuevaTr.style.display = 'none';
13525 |             return;
13526 |         }
13527 |         const instrumentacionId = ctx && ctx.instrumentacionId;
13528 |         const mcRow = ctx && ctx.mcRow;
13529 |         if (!instrumentacionId) {
13530 |             loadingEl.style.display = 'none';
13531 |             tbody.innerHTML = `<tr><td colspan="9">No se pudo cargar la instrumentación.</td></tr>`;
13532 |             contentEl.style.display = 'block';
13533 |             return;
13534 |         }
13535 |         panel.dataset.instrumentacionId = instrumentacionId;
13536 |
13537 |         Promise.all([
13538 |             client.from('ordenes').select('id, cliente_id, intermediario_id, moneda_recibida, monto_recibido, moneda_entregada, monto_entregado, cotizacion, estado, tipos_operacio
n(codigo, usa_intermediario)').eq('id', ordenId).single(),
13539 |             client
13540 |             .from('transacciones')
13541 |             .select('id, numero, tipo, modo_pago_id, moneda, monto, cobrador, pagador, owner, estado, concepto, tipo_cambio, pagador_cliente_id, cobrador_cliente_id, pagador_int
ermediario_id, cobrador_intermediario_id')
13542 |             .eq('instrumentacion_id', instrumentacionId)
13543 |             .order('created_at', { ascending: true }),
13544 |         ]).then(([rOrdFull, res]) => {
13545 |             loadingEl.style.display = 'none';
13546 |             contentEl.style.display = 'block';
13547 |             const ordenTotales = rOrdFull.data || orden;
13548 |             const toJP = ordenTotales.tipos_operacion && (Array.isArray(ordenTotales.tipos_operacion) ? ordenTotales.tipos_operacion[0] : ordenTotales.tipos_operacion);
13549 |

```

```

13550|     const mcInstP = !(mcRow && mcRow.multicontraparte_manual);
13551|     const totalesOptsPanel = mcInstP && esTipoOpMulticontraparteElegibleDesdeOrden(ordenesTotales, toJP) ? { totalesMulticontraparte: true } : undefined;
13552|     if (res.error) {
13553|         tbody.innerHTML = '<tr><td colspan="9">Error: ' + (res.error.message || '') + '</td></tr>';
13554|         return;
13555|     }
13556|     let list = res.data || [];
13557|
13558|     function renderTransaccionesList(lista) {
13559|         const { totalRecibido, totalEntregado } = totalesInstrumentacion(lista, ordenTotales, totalesOptsPanel);
13560|         const totalesEl = panel.querySelector('.orden-detalle-totales');
13561|         if (totalesEl && orden) {
13562|             const mr = Number(orden.monto_recibido) || 0;
13563|             const me = Number(orden.monto_entregado) || 0;
13564|             const monR = orden.moneda_recibida || 'USD';
13565|             const monE = orden.moneda_entregada || 'USD';
13566|             const okRec = totalRecibido <= mr + 1e-6;
13567|             const okEnt = totalEntregado <= me + 1e-6;
13568|             const ejecutada = orden.estado === 'orden_ejecutada';
13569|             const textoInst = ejecutada
13570|                 ? 'Recibido ${formatImporteDisplay(totalRecibido)} ${monR} · Entregado ${formatImporteDisplay(totalEntregado)} ${monE}.'
13571|                 : 'Instrumentación: A recibir ${formatImporteDisplay(mr)} ${monR} - A entregar ${formatImporteDisplay(me)} ${monE}.';
13572|             totalesEl.innerHTML = <strong>Acuerdo:</strong> Recibir ${formatImporteDisplay(mr)} ${monR} · Entregar ${formatImporteDisplay(me)} ${monE}. &nbsp;&nbsp;& <strong>${tex
toInst}</strong>${(!okRec || !okEnt) ? ' <span style="color:#b91c1c;">(Supera acuerdo)</span>' : ''};
13573|         }
13574|         return Promise.all([
13575|             client.from('modos_pago').select('id, codigo, nombre'),
13576|             fetchMapsNombresParticipantesTransacciones(ordenesTotales, lista),
13577|         ]).then(([rModos, maps]) => {
13578|             const modosMap = {};
13579|             (rModos.data || []).forEach((m) => { modosMap[m.id] = m; });
13580|             const esc = (s) => (s == null ? '' : String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;').replace(/"/g, '&quot;'));
13581|             const esOrdenChequeWiz = esOrdenChequeArsDesdeOrden(orden);
13582|             const ingresoChequeCliPandy = lista.find((tr) => (tr.tipo || '').toLowerCase() === 'ingreso' && String(tr.pagador || '').toLowerCase() === 'cliente' && String(tr
.cobrador || '').toLowerCase() === 'pandy' && modosMap[tr.modo_pago_id]?.codigo === 'cheque');
13583|             const bloquearEjecutadaEgresoCheque = (t) => orden?.intermediario_id && (t.tipo || '').toLowerCase() === 'egreso' && String(t.pagador || '').toLowerCase() === 'p
andy' && String(t.cobrador || '').toLowerCase() === 'intermediario' && modosMap[t.modo_pago_id]?.codigo === 'cheque' && ingresoChequeCliPandy && (ingresoChequeCliPandy.estado
|| '').toLowerCase() !== 'ejecutada';
13584|             const cobradorL = (t) => transaccionParticipanteCeldaHtml(t, ordenTotales, 'cobrador', maps);
13585|             const pagadorL = (t) => transaccionParticipanteCeldaHtml(t, ordenTotales, 'pagador', maps);
13586|             const estadoTrxCombo = (t) => {
13587|                 if (String(t.estado || '').toLowerCase() === 'anulada') return '<span class="badge badge-estado-anulada">Anulada</span>';
13588|                 const est = t.estado === 'ejecutada' ? 'ejecutada' : 'pendiente';
13589|                 const bloquear = bloquearEjecutadaEgresoCheque(t);
13590|                 const title = bloquear ? 'No se puede marcar ejecutada hasta que el ingreso del cheque (Cliente-Pandy) esté ejecutado.' : '';
13591|                 return <select class="combo-estado-transaccion-combo-estado-${est}" data-id="${t.id}" aria-label="Estado" title="${esc(title)}"><option value="pendiente"${t.e
stado === 'pendiente' ? ' selected' : ''}>Pendiente</option><option value="ejecutada"${t.estado === 'ejecutada' ? ' selected' : ''}>Ejecutada</op
tion></select>';
13592|             };
13593|             const estadoTexto = (t) => (String(t.estado || '').toLowerCase() === 'anulada' ? 'Anulada' : (t.estado === 'ejecutada' ? 'Ejecutada' : 'Pendiente'));
13594|             const montoCell = (t) => {
13595|                 if (!canEditarTr) return <td>${formatImporteDisplay(t.monto)}</td>;
13596|                 const val = formatImporteParaInput(t.monto);
13597|                 return <td><input type="text" class="input-monto-transaccion-tabla" data-id="${esc(t.id)}" value="${esc(val)}" inputmode="decimal" aria-label="Monto ${esc(t.m
oneda)}"></td>;
13598|             };
13599|             const modoPagoCell = (t) => {
13600|                 const modo = modosMap[t.modo_pago_id];
13601|                 const modoChequeBloqueado = esOrdenChequeWiz && modo?.codigo === 'cheque';
13602|                 if (!canEditarTr || modoChequeBloqueado) {
13603|                     return <td>${esc(modo ? modo.nombre : '-')}</td>;
13604|                 }
13605|                 const opciones = (rModos.data || []).map((m) => <option value="${m.id}"${t.modo_pago_id === m.id ? ' selected' : ''}>${esc(m.nombre)}</option>).join('');
13606|                 return <td><select class="combo-modo-pago-transaccion-tabla" data-id="${esc(t.id)}" aria-label="Modo de pago">${opciones}</select></td>;
13607|             };
13608|             const listaSorted = sortTransaccionesIngresosPrimero(lista);
13609|             tbody.innerHTML = listaSorted
13610|                 .map(
13611|                     (t) => {
13612|                         return <tr data-id="${t.id}">
13613|                             <td>${t.numero != null ? esc(String(t.numero)) : '-'}</td>
13614|                             <td>${tipoTransaccionHtml(t.tipo)}</td>
13615|                             ${modoPagoCell(t)}
13616|                             <td>${esc(t.moneda)}</td>
13617|                             ${montoCell(t)}
13618|                             <td>${pagadorL(t)}</td>
13619|                             <td>${cobradorL(t)}</td>
13620|                             <td>${canEditarTr ? estadoTrxCombo(t) : estadoTexto(t)}</td>
13621|                             <td>${canEditarTr ? `<button type="button" class="btn-editar btn-editar-transaccion-panel" data-id="${t.id}" title="Editar concepto y demás campos">Edita
r</button>` : ''}${canEliminarTr ? `<button type="button" class="btn-secondary btn-eliminar-transaccion-panel" data-id="${t.id}" title="Dar de baja">Eliminar</button>` : ''}<
td>
13622|                             </tr>;
13623|                     }
13624|                 )
13625|                 .join('');
13626|             if (canEditarTr) {
13627|                 tbody.querySelectorAll('.combo-estado-transaccion').forEach((sel) => {
13628|                     sel.addEventListener('change', function() { cambiarEstadoTransaccion(this.getAttribute('data-id'), this.value, instrumentacionId, this); });
13629|                 });
13630|                 tbody.querySelectorAll('.combo-modo-pago-transaccion-tabla').forEach((sel) => {
13631|                     sel.addEventListener('change', function() {
13632|                         const id = this.getAttribute('data-id');
13633|                         const prev = lista.find((r) => r.id === id);
13634|                         if (!prev || this.value === prev.modo_pago_id) return;
13635|                         guardarSoloModoPagoTransaccion(id, this.value, () => refreshTransaccionesPanel(ordenId), () => { this.value = prev.modo_pago_id; });
13636|                     });
13637|                 });

```



```

13638 |         tbody.querySelectorAll('.input-monto-transaccion-tabla').forEach((input) => {
13639 |             input.addEventListener('blur', function() {
13640 |                 const id = this.getAttribute('data-id');
13641 |                 const prev = lista.find((r) => r.id === id);
13642 |                 if (!prev || parseInt(input.value) === Number(prev.monto)) return;
13643 |                 guardarSoloMontoTransaccion(id, this.value, () => refreshTransaccionesPanel(ordenId));
13644 |             });
13645 |         });
13646 |         tbody.querySelectorAll('.btn-editar-transaccion-panel').forEach((btn) => {
13647 |             btn.addEventListener('click', () => {
13648 |                 const row = lista.find((r) => r.id === btn.getAttribute('data-id'));
13649 |                 if (row) openModalTransaccion(row, instrumentacionId);
13650 |             });
13651 |         });
13652 |     });
13653 |     tbody.querySelectorAll('.btn-eliminar-transaccion-panel').forEach((btn) => {
13654 |         btn.addEventListener('click', () => { eliminarTransaccion(btn.getAttribute('data-id'), ordenId); });
13655 |     });
13656 | });
13657 | }
13658 |
13659 | if (list.length === 0 && ordenTotales && ordenTotales.tipo_operacion_id && String(ordenTotales.estado || '') !== 'anulada') {
13660 |     return client.from('tipos_operacion').select('codigo, moneda_in, moneda_out').eq('id', ordenTotales.tipo_operacion_id).single().then((rTipo) => {
13661 |         const row = rTipo.data || {};
13662 |         const codigo = row.codigo || '';
13663 |         const miP = (row.moneda_in || '').toString().toUpperCase().trim();
13664 |         const moP = (row.moneda_out || '').toString().toUpperCase().trim();
13665 |         const patronTcP = patronTipoCambioOrden(miP, moP);
13666 |         const esUsdUsdP = miP === 'USD' && moP === 'USD';
13667 |         if (esTipoOperacionChequeArs(codigo, row.moneda_in, row.moneda_out) && ordenTotales.intermediario_id) {
13668 |             return autoCompleteInstrumentacionChequeConIntermediario(ordenId, instrumentacionId, ordenTotales);
13669 |         }
13670 |         // USD-USD+int: cp_ic por defecto en panel. Cruce fiat+USD+int: ci_pc (misma lógica que ARS-USD/USD-ARS+int; motor CC reglas_de_negocio).
13671 |         if (ordenTotales.intermediario_id && (esUsdUsdP || patronTcP) && !esTipoOperacionChequeArs(codigo, row.moneda_in, row.moneda_out)) {
13672 |             const patronPanelPar = patronTcP ? 'ci_pc' : 'cp_ic';
13673 |             return autoCompleteInstrumentacionUsdUsdConIntermediario(ordenId, instrumentacionId, ordenTotales, patronPanelPar);
13674 |         }
13675 |         if (!mcInstP && !ordenTotales.intermediario_id && (esUsdUsdP || patronTcP || esTipoOperacionChequeArs(codigo, row.moneda_in, row.moneda_out))) {
13676 |             return autoCompleteInstrumentacionSinIntermediario(ordenId, instrumentacionId, ordenTotales);
13677 |         }
13678 |         return Promise.resolve();
13679 |     }).then(() =>
13680 |         client.from('transacciones').select('id, numero, tipo, modo_pago_id, moneda, monto, cobrador, pagador, owner, estado, concepto, tipo_cambio, pagador_cliente_id,
cobrador_cliente_id, pagador_intermediario_id, cobrador_intermediario_id').eq('instrumentacion_id', instrumentacionId).order('created_at', { ascending: true })
13681 |     ).then((r2) => {
13682 |         list = (r2.data || []);
13683 |         return renderTransaccionesList(list);
13684 |     });
13685 | }
13686 | return renderTransaccionesList(list);
13687 | })
13688 | .then(() => {
13689 |     const btnNueva = panel.querySelector('.btn-nueva-transaccion-panel');
13690 |     if (btnNueva) {
13691 |         if (ordenAnulada) {
13692 |             btnNueva.onclick = null;
13693 |             btnNueva.style.display = 'none';
13694 |         } else {
13695 |             btnNueva.onclick = () => openModalTransaccion(null, instrumentacionId);
13696 |         }
13697 |     }
13698 | });
13699 | });
13700 | }
13701 |
13702 | function refreshTransaccionesPanel(ordenId) {
13703 |     if (!ordenId) return;
13704 |     const panel = document.getElementById('panel-orden-' + ordenId);
13705 |     const tbody = panel?.querySelector('.orden-detalle-tbody');
13706 |     const instrumentacionId = panel?.dataset?.instrumentacionId;
13707 |     if (!panel || !tbody || !instrumentacionId) return;
13708 |     tbody.innerHTML = '';
13709 |     Promise.all([
13710 |         client.from('transacciones').select('id, numero, tipo, modo_pago_id, moneda, monto, cobrador, pagador, owner, estado, concepto, tipo_cambio, pagador_cliente_id, cobrador_c
liente_id, pagador_intermediario_id, cobrador_intermediario_id').eq('instrumentacion_id', instrumentacionId).order('created_at', { ascending: true }),
13711 |         client.from('ordenes').select('id, cliente_id, intermediario_id, moneda_recibida, monto_recibido, moneda_entregada, monto_entregado, estado, tipo_operacion_id, cotizacion,
tipos_operacion(codigo, moneda_in, moneda_out, usa_intermediario)').eq('id', ordenId).single(),
13712 |         client.from('instrumentacion').select('multicontraparte_manual').eq('id', instrumentacionId).maybeSingle(),
13713 |     ]).then(([resTr, resOrd, rInstMc]) => {
13714 |         const orden = resOrd?.data || null;
13715 |         const ordenAnuladaRf = orden && String(orden.estado || '') === 'anulada';
13716 |         const toJR = orden?.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
13717 |         const totMcRf = !(rInstMc.data && rInstMc.data.multicontraparte_manual) && esTipoOpMulticontraparteElegibleDesdeOrden(orden, toJR);
13718 |         const totalesOptsRf = totMcRf ? { totalesMulticontraparte: true } : undefined;
13719 |         if (resTr.error) {
13720 |             tbody.innerHTML = '<tr><td colspan="9">Error: ' + (resTr.error.message || '') + '</td></tr>';
13721 |             return;
13722 |         }
13723 |         const list = resTr.data || [];
13724 |         const { totalRecibido, totalEntregado } = totalesInstrumentacion(list, orden, totalesOptsRf);
13725 |         const totalesEl = panel.querySelector('.orden-detalle-totales');
13726 |         if (totalesEl && orden) {
13727 |             const mr = Number(orden.monto_recibido) || 0;
13728 |             const me = Number(orden.monto_entregado) || 0;
13729 |             const monR = orden.moneda_recibida || 'USD';
13730 |             const monE = orden.moneda_entregada || 'USD';
13731 |             const okRec = totalRecibido <= mr + 1e-6;

```

```

13732|     const okEnt = totalEntregado <= me + 1e-6;
13733|     const ejecutada = orden.estado === 'orden_ejecutada';
13734|     const textoInst = ejecutada
13735|       ? Recibido ${formatImporteDisplay(totalRecibido)} ${monR} · Entregado ${formatImporteDisplay(totalEntregado)} ${monE}·
13736|       : Instrumentación: A recibir ${formatImporteDisplay(mr)} ${monR} A entregar ${formatImporteDisplay(me)} ${monE}·;
13737|     talesEl.innerHTML = <strong>Acuerdo:</strong> Recibir ${formatImporteDisplay(mr)} ${monR} · Entregar ${formatImporteDisplay(me)} ${monE}. &nbsp;&nbsp;<strong>${textoInst}<
/strong>${(lokRec || !okEnt) ? ' <span style="color:#b91c1c;">(Supera acuerdo)</span>' : ''};
13738|   }
13739|   const btnNuevaRf = panel.querySelector('.btn-nueva-transaccion-panel');
13740|   if (btnNuevaRf) btnNuevaRf.style.display = (userPermissions.includes('ingresar transacciones') && !ordenAnuladaRf) ? '' : 'none';
13741|   const canEditarTr = userPermissions.includes('editar transacciones') && !ordenAnuladaRf;
13742|   const canEliminarTr = userPermissions.includes('eliminar transacciones') && !ordenAnuladaRf;
13743|   Promise.all([
13744|     client.from('modos_pago').select('id, codigo, nombre'),
13745|     fetchMapsNombresParticipantesTransacciones(orden, list),
13746|   ]).then(([rModos, maps]) => {
13747|     const modosMap = {};
13748|     (rModos.data || []).forEach((m) => { modosMap[m.id] = m; });
13749|     const esc = (s) => (s == null ? '' : String(s).replace(/&/g, '&amp;')).replace(/>/g, '&lt;').replace(/</g, '&gt;').replace(/"/g, '&quot;'));
13750|     const esOrdenCheque = esOrdenChequeArsDesdeOrden(orden);
13751|     const ingresoChequeCliPandy = list.find((tr) => (tr.tipo || '').toLowerCase() === 'ingreso' && String(tr.pagador || '').toLowerCase() === 'cliente' && String(tr.cobrador
|| '').toLowerCase() === 'pandy' && modosMap[tr.modopago_id]?.codigo === 'cheque'));
13752|     const bloquearEjecutadaEgresoCheque = (t) => orden?.intermediario_id && (t.tipo || '').toLowerCase() === 'egreso' && String(t.pagador || '').toLowerCase() === 'pandy' &&
String(t.cobrador || '').toLowerCase() === 'intermediario' && modosMap[t.modopago_id]?.codigo === 'cheque' && ingresoChequeCliPandy && (ingresoChequeCliPandy.estado || '').t
oLowerCase() !== 'ejecutada';
13753|     const cobradorL = (t) => transaccionParticipanteCeldaHtml(t, orden, 'cobrador', maps);
13754|     const pagadorL = (t) => transaccionParticipanteCeldaHtml(t, orden, 'pagador', maps);
13755|     const estadoTrxCombo = (t) => {
13756|       if (String(t.estado || '').toLowerCase() === 'anulada') return '<span class="badge badge-estado-anulada">Anulada</span>';
13757|       const est = t.estado === 'ejecutada' ? 'ejecutada' : 'pendiente';
13758|       const bloquear = bloquearEjecutadaEgresoCheque(t);
13759|       const title = bloquear ? 'No se puede marcar ejecutada hasta que el ingreso del cheque (Cliente-Pandy) esté ejecutado.' : '';
13760|       return '<select class="combo-estado-transaccion combo-estado-${est}" data-id="${t.id}" aria-label="Estado" title="${esc(title)}"><option value="pendiente"${t.estado ===
'pendiente' ? ' selected' : ''}>Pendiente</option><option value="ejecutada"${t.estado === 'ejecutada' ? ' selected' : ''}>Ejecutada</option></s
elect>';
13761|     };
13762|     const estadoTexto = (t) => (String(t.estado || '').toLowerCase() === 'anulada' ? 'Anulada' : (t.estado === 'ejecutada' ? 'Ejecutada' : 'Pendiente'));
13763|     const montoCell = (t) => {
13764|       if (!canEditarTr) return '<td>${formatImporteDisplay(t.monto)}</td>';
13765|       const val = formatImporteParaInput(t.monto);
13766|       return '<td><input type="text" class="input-monto-transaccion-tabla" data-id="${esc(t.id)}" value="${esc(val)}" inputmode="decimal" aria-label="Monto ${esc(t.moneda)}"
></td>';
13767|     };
13768|     const modoPagoCell = (t) => {
13769|       const modo = modosMap[t.modopago_id];
13770|       const modoChequeBloqueado = esOrdenCheque && modo?.codigo === 'cheque';
13771|       if (!canEditarTr || modoChequeBloqueado) {
13772|         return '<td>${esc(modo ? modo.nombre : '-')}</td>';
13773|       }
13774|       const opciones = (rModos.data || []).map(m) => '<option value="${m.id}"${t.modopago_id === m.id ? ' selected' : ''}>${esc(m.nombre)}</option>'.join('');
13775|       return '<td><select class="combo-modo-pago-transaccion-tabla" data-id="${esc(t.id)}" aria-label="Modo de pago">${opciones}</select></td>';
13776|     };
13777|     const listSorted = sortTransaccionesIngresosPrimero(list);
13778|     tbody.innerHTML = listSorted
13779|       .map(
13780|         (t) => {
13781|           return '<tr data-id="${t.id}">
13782|             <td>${t.numero != null ? esc(String(t.numero)) : '-'}</td>
13783|             <td>${t.tipoTransaccionHtml(t.tipo)}</td>
13784|             ${modoPagoCell(t)}
13785|             <td>${esc(t.moneda)}</td>
13786|             ${montoCell(t)}
13787|             <td>${pagadorL(t)}</td>
13788|             <td>${cobradorL(t)}</td>
13789|             <td>${canEditarTr ? estadoTrxCombo(t) : estadoTexto(t)}</td>
13790|             <td>${canEditarTr ? <button type="button" class="btn-editar btn-editar-transaccion-panel" data-id="${t.id}" title="Editar concepto y demás campos">Editar</butto
n> : ''}${canEliminarTr ? <button type="button" class="btn-secondary btn-eliminar-transaccion-panel" data-id="${t.id}" title="Dar de baja">Eliminar</button> : ''}</td>
</tr>';
13791|         }
13792|       )
13793|       .join('');
13794|     if (canEditarTr) {
13795|       tbody.querySelectorAll('.combo-estado-transaccion').forEach((sel) => {
13796|         sel.addEventListener('change', function() { cambiarEstadoTransaccion(this.getAttribute('data-id'), this.value, instrumentacionId, this); });
13797|       });
13798|       tbody.querySelectorAll('.combo-modo-pago-transaccion-tabla').forEach((sel) => {
13799|         sel.addEventListener('change', function() {
13800|           const id = this.getAttribute('data-id');
13801|           const prev = list.find((r) => r.id === id);
13802|           if (!prev || this.value === prev.modopago_id) return;
13803|           const selEl = this;
13804|           guardarSoloModoPagoTransaccion(id, this.value, () => refreshTransaccionesPanel(ordenId), () => { selEl.value = prev.modopago_id; });
13805|         });
13806|       });
13807|     };
13808|     tbody.querySelectorAll('.input-monto-transaccion-tabla').forEach((input) => {
13809|       input.addEventListener('blur', function() {
13810|         const id = this.getAttribute('data-id');
13811|         const prev = list.find((r) => r.id === id);
13812|         if (!prev || parseImporteInput(this.value) === Number(prev.monto)) return;
13813|         guardarSoloMontoTransaccion(id, this.value, () => refreshTransaccionesPanel(ordenId));
13814|       });
13815|     });
13816|     tbody.querySelectorAll('.btn-editar-transaccion-panel').forEach((btn) => {
13817|       btn.addEventListener('click', () => {
13818|         const row = list.find((r) => r.id === btn.getAttribute('data-id'));
13819|         if (row) openModalTransaccion(row, instrumentacionId);
13820|       });

```

```

13821 |     });
13822 |     }
13823 |     tbody.querySelectorAll('.btn-eliminar-transaccion-panel').forEach((btn) => {
13824 |         btn.addEventListener('click', () => { eliminarTransaccion(btn.getAttribute('data-id'), ordenId); });
13825 |     });
13826 | });
13827 | });
13828 | }
13829 |
13830 | /** Conversión inversa: si moneda es ARS → calculado = monto/TC (otra moneda); si moneda es USD/EUR → calculado = monto*TC (ARS). */
13831 | function actualizarMontoCalculado() {
13832 |     const backdrop = document.getElementById('modal-transaccion-backdrop');
13833 |     const selMoneda = document.getElementById('transaccion-moneda');
13834 |     const selTipo = document.getElementById('transaccion-tipo');
13835 |     const wrapConversion = document.getElementById('transaccion-wrap-conversion');
13836 |     if (!wrapConversion || wrapConversion.style.display !== 'block') return;
13837 |     const monedaRecibida = (backdrop?.dataset.monedaRecibida || '').toUpperCase();
13838 |     const monedaEntregada = (backdrop?.dataset.monedaEntregada || '').toUpperCase();
13839 |     const moneda = (selMoneda?.value || '').toUpperCase();
13840 |     const tipo = selTipo?.value || 'ingreso';
13841 |     const otraMoneda = moneda === 'ARS' ? (tipo === 'ingreso' ? monedaEntregada : monedaRecibida) : 'ARS';
13842 |     const lblCalculado = document.getElementById('transaccion-monto-calculado-currency-label');
13843 |     const display = document.getElementById('transaccion-monto-calculado-display');
13844 |     if (lblCalculado) lblCalculado.textContent = otraMoneda || 'USD';
13845 |     const montoRaw = document.getElementById('transaccion-monto')?.value?.trim() || '';
13846 |     const tcRaw = document.getElementById('transaccion-tipo-cambio')?.value?.trim() || '';
13847 |     const monto = parseImporteInput(montoRaw);
13848 |     const tc = parseImporteInput(tcRaw);
13849 |     let calculado = null;
13850 |     if (typeof monto === 'number' && !isNaN(monto) && typeof tc === 'number' && !isNaN(tc) && tc > 0) {
13851 |         calculado = moneda === 'ARS' ? monto / tc : monto * tc;
13852 |     }
13853 |     if (display) display.value = calculado !== null ? formatImporteDisplay(calculado) : '';
13854 | }
13855 |
13856 | function toggleTransaccionMonedaArs() {
13857 |     const selMoneda = document.getElementById('transaccion-moneda');
13858 |     const selTipo = document.getElementById('transaccion-tipo');
13859 |     const backdrop = document.getElementById('modal-transaccion-backdrop');
13860 |     const wrapConversion = document.getElementById('transaccion-wrap-conversion');
13861 |     const lblMonto = document.getElementById('transaccion-monto-currency-label');
13862 |     const esOrdenCheque = document.getElementById('transaccion-es-orden-cheque')?.value === '1';
13863 |     if (esOrdenCheque) {
13864 |         if (selMoneda) { selMoneda.value = 'ARS'; selMoneda.disabled = true; }
13865 |         if (wrapConversion) wrapConversion.style.display = 'none';
13866 |         if (lblMonto) lblMonto.textContent = 'ARS';
13867 |         return;
13868 |     }
13869 |     if (selMoneda) selMoneda.disabled = false;
13870 |     const moneda = (selMoneda?.value || 'USD').toUpperCase();
13871 |     const monedaRecibida = (backdrop?.dataset.monedaRecibida || '').toUpperCase();
13872 |     const monedaEntregada = (backdrop?.dataset.monedaEntregada || '').toUpperCase();
13873 |     const tipo = selTipo?.value || 'ingreso';
13874 |     const monedaTransaccion = tipo === 'ingreso' ? monedaRecibida : monedaEntregada;
13875 |     if (lblMonto) lblMonto.textContent = monedaTransaccion || moneda;
13876 |     const hayDosMonedas = monedaRecibida && monedaEntregada && monedaRecibida !== monedaEntregada;
13877 |     if (wrapConversion) wrapConversion.style.display = hayDosMonedas ? 'block' : 'none';
13878 |     if (hayDosMonedas) {
13879 |         const lblCalculado = document.getElementById('transaccion-monto-calculado-currency-label');
13880 |         const otraMoneda = moneda === 'ARS' ? (tipo === 'ingreso' ? monedaEntregada : monedaRecibida) : 'ARS';
13881 |         if (lblCalculado) lblCalculado.textContent = otraMoneda || 'USD';
13882 |         actualizarMontoCalculado();
13883 |     } else {
13884 |         const d = document.getElementById('transaccion-monto-calculado-display');
13885 |         if (d) d.value = '';
13886 |     }
13887 | }
13888 |
13889 | /** Según tipo de operación: Ingreso solo permite moneda recibida; Egreso solo moneda entregada. Gris opciones no permitidas. */
13890 | function adaptarTransaccionTipoYMoneda() {
13891 |     const backdrop = document.getElementById('modal-transaccion-backdrop');
13892 |     const selTipo = document.getElementById('transaccion-tipo');
13893 |     const selMoneda = document.getElementById('transaccion-moneda');
13894 |     if (!backdrop || !selTipo || !selMoneda) return;
13895 |     const monedaRecibida = (backdrop?.dataset.monedaRecibida || '').toUpperCase();
13896 |     const monedaEntregada = (backdrop?.dataset.monedaEntregada || '').toUpperCase();
13897 |     const tipo = selTipo.value;
13898 |     const opciones = Array.from(selMoneda.options);
13899 |     const monedasValidas = ['USD', 'EUR', 'ARS'];
13900 |     const restringir = monedasValidas.includes(monedaRecibida) && monedasValidas.includes(monedaEntregada);
13901 |     if (!restringir) {
13902 |         opciones.forEach((opt) => { opt.disabled = false; });
13903 |         return;
13904 |     }
13905 |     const monedaPermitida = tipo === 'ingreso' ? monedaRecibida : monedaEntregada;
13906 |     opciones.forEach((opt) => {
13907 |         opt.disabled = opt.value !== monedaPermitida;
13908 |     });
13909 |     const valorActual = selMoneda.value;
13910 |     if (valorActual !== monedaPermitida) {
13911 |         selMoneda.value = monedaPermitida;
13912 |         toggleTransaccionMonedaArs();
13913 |     }
13914 | }
13915 |
13916 | function actualizarVisibilidadFilasContraparteMulticontraparte() {
13917 |     const wrap = document.getElementById('transaccion-multicontraparte-contrapartes-wrap');

```

```

13918| if (!wrap || wrap.style.display === 'none') return;
13919| const pag = (document.getElementById('transaccion-pagador')?.value || '').toLowerCase();
13920| const cob = (document.getElementById('transaccion-cobrador')?.value || '').toLowerCase();
13921| const wPagCli = document.getElementById('transaccion-wrap-pagador-cliente-id');
13922| const wCobCli = document.getElementById('transaccion-wrap-cobrador-cliente-id');
13923| const wPagInt = document.getElementById('transaccion-wrap-pagador-intermediario-id');
13924| const wCobInt = document.getElementById('transaccion-wrap-cobrador-intermediario-id');
13925| if (wPagCli) wPagCli.style.display = pag === 'cliente' ? 'block' : 'none';
13926| if (wCobCli) wCobCli.style.display = cob === 'cliente' ? 'block' : 'none';
13927| if (wPagInt) wPagInt.style.display = pag === 'intermediario' ? 'block' : 'none';
13928| if (wCobInt) wCobInt.style.display = cob === 'intermediario' ? 'block' : 'none';
13929| }
13930|
13931| /** Pagador cliente del acuerdo: solo lectura (compromiso); opción explícita «Otro cliente es el pagador» abre el combo. */
13932| function aplicarModoPagadorClienteMulticontraparte(participantes, registro) {
13933|   const wrapPag = document.getElementById('transaccion-wrap-pagador-cliente-id');
13934|   const selPagCli = document.getElementById('transaccion-pagador-cliente-id');
13935|   const readP = document.getElementById('transaccion-pagador-cliente-readonly');
13936|   const btnOtro = document.getElementById('transaccion-pagador-otro-cliente-btn');
13937|   const btnVol = document.getElementById('transaccion-pagador-volver-acuerdo-btn');
13938|   if (!wrapPag || !selPagCli) return;
13939|   const acuerdoId = participantes.ordenClienteId || '';
13940|   const acuerdoNombre = (participantes.clienteNombre || 'Cliente del acuerdo').trim() || 'Cliente del acuerdo';
13941|   const listCli = participantes.listaClientes || [];
13942|   const idReg = registro?.pagador_cliente_id || null;
13943|   const otroGuardado = idReg && String(idReg) !== String(acuerdoId);
13944|
13945|   function setModoAcuerdo() {
13946|     wrapPag.dataset.pagadorClienteModo = 'acuerdo';
13947|     if (readP) {
13948|       readP.style.display = 'block';
13949|       readP.textContent = acuerdoNombre + ' (compromiso del acuerdo – no editable)';
13950|     }
13951|     selPagCli.style.display = 'none';
13952|     selPagCli.disabled = false;
13953|     selPagCli.innerHTML = '<option value=""></option>';
13954|     selPagCli.value = '';
13955|     if (btnOtro) btnOtro.style.display = 'inline-flex';
13956|     if (btnVol) btnVol.style.display = 'none';
13957|   }
13958|
13959|   function setModoOtro(preservarId) {
13960|     wrapPag.dataset.pagadorClienteModo = 'otro';
13961|     if (readP) readP.style.display = 'none';
13962|     selPagCli.style.display = 'block';
13963|     selPagCli.disabled = false;
13964|     let h = '<option value="">Elegí el cliente pagador...</option>';
13965|     listCli.forEach((c) => {
13966|       if (!c || !c.id) return;
13967|       if (acuerdoId && String(c.id) === String(acuerdoId)) return;
13968|       h += '<option value="${escapeHtml(String(c.id))}">${escapeHtml(c.nombre !== null ? String(c.nombre) : String(c.id))}</option>';
13969|     });
13970|     selPagCli.innerHTML = h;
13971|     if (preservarId && String(preservarId) !== String(acuerdoId)) {
13972|       selPagCli.value = String(preservarId);
13973|       if (selPagCli.value !== String(preservarId)) selPagCli.value = '';
13974|     }
13975|     if (btnOtro) btnOtro.style.display = 'none';
13976|     if (btnVol) btnVol.style.display = 'inline-flex';
13977|   }
13978|
13979|   if (otroGuardado) setModoOtro(idReg);
13980|   else setModoAcuerdo();
13981|
13982|   if (btnOtro && !btnOtro._wiredMcPag) {
13983|     btnOtro._wiredMcPag = true;
13984|     btnOtro.addEventListener('click', () => setModoOtro(null));
13985|   }
13986|   if (btnVol && !btnVol._wiredMcPag) {
13987|     btnVol._wiredMcPag = true;
13988|     btnVol.addEventListener('click', () => setModoAcuerdo());
13989|   }
13990| }
13991|
13992| /**
13993|  * Desplegable cobrador cliente (MC).
13994|  * - Pagador acuerdo + cobrador cliente: el cobrador es un tercero (no otra fila «acuerdo»).
13995|  * - Pagador *otro* cliente + cobrador cliente: el cobrador puede ser el *cliente del acuerdo* (recibe la entrega) o un tercero.
13996|  */
13997| function htmlOptionsCobradorClienteMulticontraparte(listCli, acuerdoId, acuerdoNombre, clienteACliente, pagadorEsOtroCliente) {
13998|   const soloTercerosComoCobrador = clienteACliente && !pagadorEsOtroCliente;
13999|   const valAcuerdo = acuerdoId ? String(acuerdoId) : '';
14000|   let h = soloTercerosComoCobrador
14001|     ? '<option value="">Elegí el cliente cobrador...</option>'
14002|     : valAcuerdo
14003|       ? '<option value="${escapeHtml(valAcuerdo)}">${escapeHtml(acuerdoNombre)} (acuerdo)</option>'
14004|       : '<option value="">Elegí el cliente cobrador...</option>';
14005|   (listCli || []).forEach((c) => {
14006|     if (!c || !c.id) return;
14007|     if (acuerdoId && String(c.id) === String(acuerdoId)) return;
14008|     h += '<option value="${escapeHtml(String(c.id))}">${escapeHtml(c.nombre !== null ? String(c.nombre) : String(c.id))}</option>';
14009|   });
14010|   return h;
14011| }
14012|
14013| function aplicarValorSelectCobradorClienteMulticontraparte(sel, htmlOpts, idGuardado, acuerdoId) {
14014|   if (!sel) return;

```

```

14015 |     sel.innerHTML = htmlOpts;
14016 |     const selVal = idGuardado ? String(idGuardado) : '';
14017 |     sel.value = selVal;
14018 |     if (selVal && sel.value !== selVal) sel.value = '';
14019 | }
14020 |
14021 | function refillMcCobradorClienteDropdown() {
14022 |     const wrap = document.getElementById('transaccion-multicontraparte-contrapartes-wrap');
14023 |     if (!wrap || wrap.style.display === 'none') return;
14024 |     const sel = document.getElementById('transaccion-cobrador-cliente-id');
14025 |     if (!sel) return;
14026 |     const bd = document.getElementById('modal-transaccion-backdrop');
14027 |     const listCli = bd?.mclistaClientes;
14028 |     if (!Array.isArray(listCli)) return;
14029 |     const acuerdoId = bd?.mcOrdenClienteId || '';
14030 |     const acuerdoNombre = (bd?.mcClienteNombre || 'Cliente del acuerdo').trim() || 'Cliente del acuerdo';
14031 |     const pag = (document.getElementById('transaccion-pagador')?.value || '').toLowerCase();
14032 |     const cob = (document.getElementById('transaccion-cobrador')?.value || '').toLowerCase();
14033 |     const wrapPagCli = document.getElementById('transaccion-wrap-pagador-cliente-id');
14034 |     const pagadorEsOtroCliente = !(wrapPagCli && wrapPagCli.dataset.pagadorClienteModo === 'otro');
14035 |     const clienteACliente = pag === 'cliente' && cob === 'cliente';
14036 |     const idPrev = sel.value || null;
14037 |     const h = htmlOptionsCobradorClienteMulticontraparte(listCli, acuerdoId, acuerdoNombre, clienteACliente, pagadorEsOtroCliente);
14038 |     aplicarValorSelectCobradorClienteMulticontraparte(sel, h, idPrev, acuerdoId);
14039 | }
14040 |
14041 | /** Rellena desplegables de cliente/intermediario concreto cuando la instrumentación tiene multicontraparte manual (ARS-USD / USD-ARS sin int.). */
14042 | function poblarSelectContrapartesMulticontraparteModal(participantes, registro, optsMc) {
14043 |     const wrap = document.getElementById('transaccion-multicontraparte-contrapartes-wrap');
14044 |     if (!wrap) return;
14045 |     const permite = participantes.permiteMulticontraparteUi === true;
14046 |     wrap.style.display = permite ? 'block' : 'none';
14047 |     if (!permite) return;
14048 |     const selCobCli = document.getElementById('transaccion-cobrador-cliente-id');
14049 |     const selPagInt = document.getElementById('transaccion-pagador-intermediario-id');
14050 |     const selCobInt = document.getElementById('transaccion-cobrador-intermediario-id');
14051 |     const acuerdoId = participantes.ordenClienteId || '';
14052 |     const acuerdoNombre = (participantes.clienteNombre || 'Cliente del acuerdo').trim() || 'Cliente del acuerdo';
14053 |     const listCli = participantes.listaClientes || [];
14054 |     const listInt = participantes.listaIntermediarios || [];
14055 |     const ordenIntId = participantes.ordenIntermediarioId || null;
14056 |     function fillCliCobrador(sel, idGuardado) {
14057 |         if (!sel) return;
14058 |         const pag = (document.getElementById('transaccion-pagador')?.value || '').toLowerCase();
14059 |         const cob = (document.getElementById('transaccion-cobrador')?.value || '').toLowerCase();
14060 |         const wrapPagCli = document.getElementById('transaccion-wrap-pagador-cliente-id');
14061 |         const pagadorEsOtroCliente = !(wrapPagCli && wrapPagCli.dataset.pagadorClienteModo === 'otro');
14062 |         const clienteACliente = pag === 'cliente' && cob === 'cliente';
14063 |         const h = htmlOptionsCobradorClienteMulticontraparte(listCli, acuerdoId, acuerdoNombre, clienteACliente, pagadorEsOtroCliente);
14064 |         aplicarValorSelectCobradorClienteMulticontraparte(sel, h, idGuardado, acuerdoId);
14065 |     }
14066 |     function fillInt(sel, idGuardado) {
14067 |         if (!sel) return;
14068 |         let h = '<option value="">- Intermediario de la orden -</option>';
14069 |         listInt.forEach(x => {
14070 |             if (!x || !x.id) return;
14071 |             h += '<option value="' + escapeHtml(String(x.id)) + ">" + escapeHtml(x.nombre !== null ? String(x.nombre) : String(x.id)) + "</option>";
14072 |         });
14073 |         sel.innerHTML = h;
14074 |         let selVal = '';
14075 |         if (idGuardado) {
14076 |             if (!ordenIntId || String(idGuardado) !== String(ordenIntId)) selVal = String(idGuardado);
14077 |         }
14078 |         sel.value = selVal;
14079 |         if (selVal && sel.value !== selVal) sel.value = '';
14080 |     }
14081 |     aplicarModoPagadorClienteMulticontraparte(participantes, registro);
14082 |     if (optsMc && optsMc.egresoMonEfaltanteMc && !registro) {
14083 |         const btnOtro = document.getElementById('transaccion-pagador-otro-cliente-btn');
14084 |         if (btnOtro) btnOtro.click();
14085 |     }
14086 |     const cobradorIdInicial =
14087 |         registro?.cobrador_cliente_id !== null
14088 |         ? registro.cobrador_cliente_id
14089 |         : optsMc && optsMc.egresoMonEfaltanteMc && acuerdoId
14090 |         ? acuerdoId
14091 |         : undefined;
14092 |     fillCliCobrador(selCobCli, cobradorIdInicial);
14093 |     fillInt(selPagInt, registro?.pagador_intermediario_id);
14094 |     fillInt(selCobInt, registro?.cobrador_intermediario_id);
14095 |     actualizarVisibilidadFilasContraparteMulticontraparte();
14096 | }
14097 |
14098 | /** Modal transacción: scroll al inicio del cuerpo y foco en el primer control editable (evita quedar abajo tras 1ª/2ª trx). */
14099 | function scrollModalTransaccionAcuerdoVisibleYFoco() {
14100 |     const bd = document.getElementById('modal-transaccion-backdrop');
14101 |     if (!bd || !bd.classList.contains('activo')) return;
14102 |     const scrollWrap = document.getElementById('transaccion-form-scroll');
14103 |     if (scrollWrap) scrollWrap.scrollTop = 0;
14104 |     else {
14105 |         const body = bd.querySelector('.modal-body');
14106 |         if (body) body.scrollTop = 0;
14107 |     }
14108 |     if (bd.dataset.transaccionAnulada === '1') return;
14109 |     const ids = [
14110 |         'transaccion-tipo',
14111 |         'transaccion-modo-pago',

```

```

14112 |     'transaccion-monedas',
14113 |     'transaccion-monto',
14114 |     'transaccion-pagador',
14115 |     'transaccion-cobrador',
14116 |     'transaccion-estado',
14117 |     'transaccion-concepto',
14118 | ];
14119 | for (let i = 0; i < ids.length; i++) {
14120 |     const el = document.getElementById(ids[i]);
14121 |     if (!el || el.disabled) continue;
14122 |     if (el.tagName === 'INPUT' && el.readOnly) continue;
14123 |     const st = typeof window !== 'undefined' && window.getComputedStyle ? window.getComputedStyle(el) : null;
14124 |     if (st && (st.display === 'none' || st.visibility === 'hidden')) continue;
14125 |     try {
14126 |         el.focus({ preventScroll: true });
14127 |         return;
14128 |     } catch (_) { /* noop */ }
14129 | }
14130 | }
14131 |
14132 | function openModalTransaccion(registro, instrumentacionId) {
14133 |     const backdrop = document.getElementById('modal-transaccion-backdrop');
14134 |     const titulo = document.getElementById('modal-transaccion-titulo');
14135 |     const idEl = document.getElementById('transaccion-id');
14136 |     const instIdEl = document.getElementById('transaccion-instrumentacion-id');
14137 |     const selMoneda = document.getElementById('transaccion-monedas');
14138 |     if (!backdrop || !titulo || !idEl || !instIdEl) return;
14139 |
14140 |     const seq = ++openModalTransaccionSeq;
14141 |
14142 |     function cargarParticipantesYOrden() {
14143 |         const vacio = {
14144 |             cliente: false, intermediario: false, cotizacion: null, esCheque: false, monedaRecibida: '', monedaEntregada: '', clienteNombre: '', montoRecibido: null, montoEntregado:
14145 |             null,
14146 |             multicontraparteManual: false, permiteMulticontraparteUi: false, ordenClienteId: null, ordenIntermediarioId: null, ordenId: null, listaClientes: [], listaIntermediarios:
14147 |             [],
14148 |         };
14149 |         if (!instrumentacionId) return Promise.resolve(vacio);
14150 |         return client.from('instrumentacion').select('orden_id, multicontraparte_manual').eq('id', instrumentacionId).single().then((rInst) => {
14151 |             const ordenId = rInst.data && rInst.data.orden_id;
14152 |             const multicontraparteManual = !(rInst.data && rInst.data.multicontraparte_manual);
14153 |             if (!ordenId) return { ...vacio, multicontraparteManual };
14154 |             return client.from('ordenes').select('id, cliente_id, intermediario_id, cotizacion, tipo_operacion_id, moneda_recibida, moneda_entregada, monto_recibido, monto_entregado
14155 |             , clientes(nombre), tipos_operacion(codigo, usa_intermediario, moneda_in, moneda_out)').eq('id', ordenId).single().then((rOrd) => {
14156 |                 const o = rOrd.data || {};
14157 |                 const cot = o.cotizacion != null && Number(o.cotizacion) > 0 ? Number(o.cotizacion) : null;
14158 |                 const monedaRecibida = (o.moneda_recibida || '').trim().toUpperCase() || '';
14159 |                 const monedaEntregada = (o.moneda_entregada || '').trim().toUpperCase() || '';
14160 |                 const clientesRef = o.clientes;
14161 |                 const clienteNombre = (clientesRef && (typeof clientesRef === 'object' && !Array.isArray(clientesRef) ? clientesRef.nombre : (Array.isArray(clientesRef) ? clientesRef[
14162 |                 0]?.nombre : null))) || '';
14163 |                 const montoRecibido = o.monto_recibido != null && !isNaN(Number(o.monto_recibido)) ? Number(o.monto_recibido) : null;
14164 |                 const montoEntregado = o.monto_entregado != null && !isNaN(Number(o.monto_entregado)) ? Number(o.monto_entregado) : null;
14165 |                 const toJoin = o.tipos_operacion && (Array.isArray(o.tipos_operacion) ? o.tipos_operacion[0] : o.tipos_operacion);
14166 |                 const ordenMini = { id: o.id, cliente_id: o.cliente_id, intermediario_id: o.intermediario_id };
14167 |                 const permiteMulticontraparteUi = multicontraparteManual && esTipoOpMulticontraparteElegibleDesdeOrden(ordenMini, toJoin);
14168 |                 const rowTipo = toJoin || {};
14169 |                 const esCheque = esTipoOperacionChequeArs(rowTipo.codigo, rowTipo.moneda_in, rowTipo.moneda_out);
14170 |                 return {
14171 |                     cliente: !!o.cliente_id,
14172 |                     intermediario: !!o.intermediario_id,
14173 |                     cotizacion: cot,
14174 |                     monedaRecibida,
14175 |                     monedaEntregada,
14176 |                     clienteNombre,
14177 |                     montoRecibido,
14178 |                     montoEntregado,
14179 |                     multicontraparteManual,
14180 |                     permiteMulticontraparteUi,
14181 |                     ordenClienteId: o.cliente_id || null,
14182 |                     ordenIntermediarioId: o.intermediario_id || null,
14183 |                     ordenId: o.id || null,
14184 |                     listaClientes: [],
14185 |                     listaIntermediarios: [],
14186 |                     esCheque,
14187 |                 };
14188 |             });
14189 |         }).catch(() => vacio);
14190 |     }
14191 |
14192 |     Promise.all([
14193 |         client.from('modos_pago').select('id, codigo, nombre').eq('activo', true).order('nombre'),
14194 |         cargarParticipantesYOrden(),
14195 |     ]).then(([r, participantes]) => {
14196 |         if (seq !== openModalTransaccionSeq) return;
14197 |
14198 |         function finishOpenModalTransaccion() {
14199 |             let mcSmartApplied = false;
14200 |             let mcEgresoFaltanteMonE = false;
14201 |             const formTr = document.getElementById('form-transaccion');
14202 |             if (formTr) {
14203 |                 formTr.querySelectorAll('input:not([type="hidden"]'), select, textarea').forEach((el) => { el.disabled = false; });
14204 |                 const subEn = formTr.querySelector('button[type="submit"]');
14205 |                 if (subEn) subEn.disabled = false;
14206 |             }
14207 |             const sel = document.getElementById('transaccion-modo_pago');
14208 |             if (sel) sel.innerHTML = '<option value="">Ninguno</option>' + (r.data || []).map((m) => `<option value="${m.id}" data-codigo="${escapeHtml(m.codigo)}">${escapeHtml(m.nomb

```

```

re)}</option>`)).join('');
14205
14206 const mapLabel = { pandy: nombreMarcaSistema(), cliente: 'Cliente', intermediario: 'Intermediario' };
14207 const permiteMcUi = participantes.permiteMulticontraparteUi === true;
14208 const allowed = [{ pandy: true, cliente: true, intermediario: true }, { pandy: false, cliente: true, intermediario: false }, { pandy: true, cliente: false, intermediario: true }, { pandy: false, cliente: false, intermediario: false }];
14209 const selCob = document.getElementById('transaccion-cobrador');
14210 const selPag = document.getElementById('transaccion-pagador');
14211 const extra = [];
14212 if (registro?.cobrador && !allowed.includes(registro.cobrador)) extra.push(registro.cobrador);
14213 if (registro?.pagador && !allowed.includes(registro.pagador)) extra.push(registro.pagador);
14214 const all = allowed.concat(extra.filter(x => x && !allowed.includes(x)));
14215 const optsHtml = all.map(v => `<option value="${escapeHtml(v)}">${escapeHtml(mapLabel[v] || v)}</option>`)).join('');
14216 if (selCob) selCob.innerHTML = optsHtml;
14217 if (selPag) selPag.innerHTML = optsHtml;
14218
14219 const esCheque = !!participantes?.esCheque;
14220 const elEsCheque = document.getElementById('transaccion-es-orden-cheque');
14221 if (elEsCheque) elEsCheque.value = esCheque ? '1' : '0';
14222
14223 if (sel) sel.disabled = false;
14224
14225 if (registro) {
14226   titulo.textContent = 'Editar transacción';
14227   idEl.value = registro.id;
14228   document.getElementById('transaccion-tipo').value = registro.tipo || 'ingreso';
14229   document.getElementById('transaccion-modo-pago').value = registro.modo_pago_id ? String(registro.modo_pago_id) : '';
14230   const modoRegistro = (r, data || []).find((m) => m.id === registro.modo_pago_id);
14231   if (esCheque && modoRegistro?.codigo === 'cheque' && sel) sel.disabled = true;
14232   document.getElementById('transaccion-monedas').value = esCheque ? 'ARS' : (registro.moneda || 'USD');
14233   document.getElementById('transaccion-monto').value = formatImporteParaInput(registro.monto);
14234   document.getElementById('transaccion-cobrador').value = registro.cobrador || 'pandy';
14235   document.getElementById('transaccion-pagador').value = registro.pagador || 'pandy';
14236   const selEstTr = document.getElementById('transaccion-estado');
14237   const estLo = String(registro.estado || '').toLowerCase();
14238   if (selEstTr) {
14239     selEstTr.value = estLo === 'anulada' ? 'anulada' : registro.estado === 'ejecutada' ? 'ejecutada' : 'pendiente';
14240   }
14241   document.getElementById('transaccion-concepto').value = registro.concepto || '';
14242   if (!esCheque) {
14243     const tc = registro.tipo_cambio !== null && Number(registro.tipo_cambio) > 0 ? Number(registro.tipo_cambio) : null;
14244     document.getElementById('transaccion-tipo-cambio').value = tc !== null ? formatImporteDisplay(tc) : '';
14245   } else {
14246     document.getElementById('transaccion-tipo-cambio').value = '';
14247   }
14248 } else {
14249   titulo.textContent = 'Nueva transacción';
14250   idEl.value = '';
14251   document.getElementById('form-transaccion').reset();
14252   if (elEsCheque) elEsCheque.value = esCheque ? '1' : '0';
14253   if (sel) sel.disabled = false;
14254   const monRui = (participantes?.monedaRecibida || '').toString().trim().toUpperCase() || 'USD';
14255   const monEui = (participantes?.monedaEntregada || '').toString().trim().toUpperCase() || 'USD';
14256   if (permiteMcUi && Array.isArray(participantes.transaccionesInst) && participantes.ordenClienteId) {
14257     const ordenMiniModal = {
14258       cliente_id: participantes.ordenClienteId,
14259       moneda_recibida: monRui,
14260       moneda_entregada: monEui,
14261       monto_recibido: participantes.montoRecibido,
14262       monto_entregado: participantes.montoEntregado,
14263       cotización: participantes.cotización,
14264     };
14265     const { totalRecibido: totR, totalEntregado: totE } = totalesInstrumentacion(participantes.transaccionesInst, ordenMiniModal, { totalesMulticontraparte: true });
14266     const mrN = Number(participantes.montoRecibido) || 0;
14267     const meN = Number(participantes.montoEntregado) || 0;
14268     const tolMcModal = 1e-6;
14269     const faltanR = Math.max(0, mrN - totR);
14270     const faltanE = Math.max(0, meN - totE);
14271     if (faltanR > tolMcModal) {
14272       document.getElementById('transaccion-tipo').value = 'ingreso';
14273       document.getElementById('transaccion-monedas').value = esCheque ? 'ARS' : monRui;
14274       document.getElementById('transaccion-pagador').value = 'cliente';
14275       document.getElementById('transaccion-cobrador').value = 'pandy';
14276       document.getElementById('transaccion-monto').value = formatImporteParaInput(faltanR);
14277       mcSmartApplied = true;
14278     } else if (faltanE > tolMcModal) {
14279       document.getElementById('transaccion-tipo').value = 'egreso';
14280       document.getElementById('transaccion-monedas').value = esCheque ? 'ARS' : monEui;
14281       if (permiteMcUi) {
14282         /* MC: no asumir Pandy-Cliente (generaba caja -m aunque un tercero/intermediario pague en la realidad). */
14283         document.getElementById('transaccion-pagador').value = 'cliente';
14284         document.getElementById('transaccion-cobrador').value = 'cliente';
14285         mcEgresoFaltanteMonE = true;
14286       } else {
14287         document.getElementById('transaccion-pagador').value = 'pandy';
14288         document.getElementById('transaccion-cobrador').value = 'cliente';
14289       }
14290       document.getElementById('transaccion-monto').value = formatImporteParaInput(faltanE);
14291       mcSmartApplied = true;
14292     }
14293   }
14294   if (!mcSmartApplied) {
14295     const pagIng = participantes?.cliente ? 'cliente' : (participantes?.intermediario ? 'intermediario' : 'pandy');
14296     document.getElementById('transaccion-tipo').value = 'ingreso';
14297     document.getElementById('transaccion-monedas').value = esCheque ? 'ARS' : 'USD';
14298     document.getElementById('transaccion-pagador').value = pagIng;
14299     document.getElementById('transaccion-cobrador').value = 'pandy';
14300     document.getElementById('transaccion-monto').value = '';

```

```

14301 |     }
14302 |     document.getElementById('transaccion-modo-pago').value = '';
14303 |     document.getElementById('transaccion-estado').value = 'pendiente';
14304 |     if (!esCheque) {
14305 |         const tcAcuerdo = participantes?.cotizacion;
14306 |         document.getElementById('transaccion-tipo-cambio').value = tcAcuerdo != null ? formatImporteDisplay(tcAcuerdo) : '';
14307 |     } else document.getElementById('transaccion-tipo-cambio').value = '';
14308 | }
14309 | if (backdrop) {
14310 |     backdrop.dataset.monedaRecibida = participantes?.monedaRecibida || '';
14311 |     backdrop.dataset.monedaEntregada = participantes?.monedaEntregada || '';
14312 | }
14313 | const elCliente = document.getElementById('transaccion-acuerdo-cliente');
14314 | const elMontoRec = document.getElementById('transaccion-acuerdo-monto-recibido');
14315 | const elMonRec = document.getElementById('transaccion-acuerdo-moneda-recibida');
14316 | const elMontoEnt = document.getElementById('transaccion-acuerdo-monto-entregado');
14317 | const elMonEnt = document.getElementById('transaccion-acuerdo-moneda-entregada');
14318 | if (elCliente) elCliente.textContent = participantes?.clienteNombre?.trim() || '-';
14319 | if (elMontoRec) elMontoRec.textContent = participantes?.montoRecibido != null ? formatImporteDisplay(participantes.montoRecibido) : '-';
14320 | if (elMonRec) elMonRec.textContent = participantes?.monedaRecibida ? String(participantes.monedaRecibida) : '';
14321 | if (elMontoEnt) elMontoEnt.textContent = participantes?.montoEntregado != null ? formatImporteDisplay(participantes.montoEntregado) : '-';
14322 | if (elMonEnt) elMonEnt.textContent = participantes?.monedaEntregada ? String(participantes.monedaEntregada) : '';
14323 | // form.reset() (rama "Nueva transacción") restaura el valor por defecto del hidden y borra el instrumentacion_id → segundo guardado fallaba / modal no cerraba.
14324 | instIdEl.value = instrumentacionId || '';
14325 | adaptarTransaccionTipoYMoneda();
14326 | toggleTransaccionMonedaArs();
14327 | // Nueva transacción: sugerir monto completo del acuerdo según tipo + moneda (MC manual ya fijó restante arriba).
14328 | if (!registro && !mcSmartApplied) {
14329 |     const montoEl = document.getElementById('transaccion-monto');
14330 |     const tipoSel = document.getElementById('transaccion-tipo')?.value;
14331 |     const monSel = (document.getElementById('transaccion-moneda')?.value || '').toString().toUpperCase().trim();
14332 |     const monR = (participantes?.monedaRecibida || '').toString().toUpperCase().trim();
14333 |     const monE = (participantes?.monedaEntregada || '').toString().toUpperCase().trim();
14334 |     const mRec = participantes?.montoRecibido;
14335 |     const mEnt = participantes?.montoEntregado;
14336 |     let sugg = null;
14337 |     if (tipoSel === 'ingreso' && monSel && monSel === monR && mRec != null && !isNaN(Number(mRec))) {
14338 |         sugg = Number(mRec);
14339 |     }
14340 |     if (tipoSel === 'egreso' && monSel && monSel === monE && mEnt != null && !isNaN(Number(mEnt))) {
14341 |         sugg = Number(mEnt);
14342 |     }
14343 |     if (montoEl && sugg != null) {
14344 |         montoEl.value = formatImporteParaInput(sugg);
14345 |     }
14346 | }
14347 | const montoFinal = document.getElementById('transaccion-monto');
14348 | if (montoFinal) {
14349 |     const raw = (montoFinal.value || '').trim();
14350 |     if (raw !== '') {
14351 |         const n = parseImporteInput(raw);
14352 |         if (!isNaN(n)) montoFinal.value = formatImporteParaInput(n);
14353 |     }
14354 |     montoFinal._importeValorPrevio = montoFinal.value;
14355 | }
14356 | poblarSelectContrapartesMulticontraparteModal(
14357 |     participantes,
14358 |     registro,
14359 |     mcEgresoFaltanteMonE ? { egresoMonEFaltanteMc: true } : undefined
14360 | );
14361 | const pagMcBind = document.getElementById('transaccion-pagador');
14362 | const cobMcBind = document.getElementById('transaccion-cobrador');
14363 | if (pagMcBind && !pagMcBind._bindMcContraparte) {
14364 |     pagMcBind._bindMcContraparte = true;
14365 |     pagMcBind.addEventListener('change', () => {
14366 |         refillMcCobradorClienteDropdown();
14367 |         actualizarVisibilidadFilasContraparteMulticontraparte();
14368 |     });
14369 | }
14370 | if (cobMcBind && !cobMcBind._bindMcContraparte) {
14371 |     cobMcBind._bindMcContraparte = true;
14372 |     cobMcBind.addEventListener('change', () => {
14373 |         refillMcCobradorClienteDropdown();
14374 |         actualizarVisibilidadFilasContraparteMulticontraparte();
14375 |     });
14376 | }
14377 |
14378 | const esTrxAnuladaModal = !(registro && String(registro.estado || '').toLowerCase() === 'anulada');
14379 | if (backdrop) backdrop.dataset.transaccionAnulada = esTrxAnuladaModal ? '1' : '';
14380 | if (formTr && esTrxAnuladaModal) {
14381 |     formTr.querySelectorAll('input:not([type="hidden"]'), select, textarea').forEach((el) => { el.disabled = true; });
14382 |     const subDis = formTr.querySelector('button[type="submit"]');
14383 |     if (subDis) subDis.disabled = true;
14384 | }
14385 |
14386 | if (backdrop) {
14387 |     if (permiteMcUi) {
14388 |         backdrop._mcListaClientes = participantes.listaClientes || [];
14389 |         backdrop._mcOrdenClienteId = participantes.ordenClienteId || '';
14390 |         backdrop._mcClienteNombre = participantes.clienteNombre || '';
14391 |     } else {
14392 |         delete backdrop._mcListaClientes;
14393 |         delete backdrop._mcOrdenClienteId;
14394 |         delete backdrop._mcClienteNombre;
14395 |     }
14396 | }
14397 | backdrop.classList.add('activo');

```



```

14398|     setupInputImporte(montoFinal);
14399|     actualizarMontoCalculado();
14400|     requestAnimationFrame(() => {
14401|         requestAnimationFrame(() => {
14402|             scrollModalTransaccionAcuerdoVisibleYFoco();
14403|         });
14404|     });
14405| }
14406|
14407| if (participantes.permiteMulticontraparteUi) {
14408|     const qTrxInst = !registro && instrumentacionId
14409|         ? client.from('transacciones').select('id, tipo, moneda, monto, cobrador, pagador, tipo_cambio, estado, pagador_cliente_id, cobrador_cliente_id, pagador_intermediario_
id, cobrador_intermediario_id').eq('instrumentacion_id', instrumentacionId).order('created_at', { ascending: true })
14410|         : Promise.resolve({ data: [] });
14411|     Promise.all([
14412|         client.from('clientes').select('id, nombre').order('nombre').limit(800),
14413|         client.from('intermediarios').select('id, nombre').order('nombre').limit(800),
14414|         qTrxInst,
14415|     ]).then(([rCli, rInt, rTrx]) => {
14416|         if (seq !== openModalTransaccionSeq) return;
14417|         participantes.listaClientes = rCli.data || [];
14418|         participantes.listaIntermediarios = rInt.data || [];
14419|         participantes.transaccionesInst = rTrx.data || [];
14420|         finishOpenModalTransaccion();
14421|     });
14422|     return;
14423| }
14424| finishOpenModalTransaccion();
14425| });
14426| }
14427|
14428| function closeModalTransaccion() {
14429|     dismissAllToasts();
14430|     const backdrop = document.getElementById('modal-transaccion-backdrop');
14431|     if (backdrop) {
14432|         backdrop.classList.remove('activo');
14433|         delete backdrop.dataset.transaccionAnulada;
14434|         delete backdrop._mListaClientes;
14435|         delete backdrop._mcOrdenClienteId;
14436|         delete backdrop._mcClienteNombre;
14437|     }
14438| }
14439|
14440| function codigoCajaTipo(modoPagoId) {
14441|     const sel = document.getElementById('transaccion-modo-pago');
14442|     if (!sel || !sel.selectedOptions.length) return 'efectivo';
14443|     const opt = sel.selectedOptions[0];
14444|     const codigo = opt.getAttribute('data-codigo') || '';
14445|     if (codigo === 'transferencia') return 'banco';
14446|     if (codigo === 'cheque') return 'cheque';
14447|     return 'efectivo';
14448| }
14449|
14450| function codigoCajaTipoDesdeCodigo(codigo) {
14451|     if (codigo === 'transferencia') return 'banco';
14452|     if (codigo === 'cheque') return 'cheque';
14453|     return 'efectivo';
14454| }
14455|
14456| /**
14457|  * Si esta transacción en estado ejecutada generaría en sync un egreso de caja de la empresa (monto negativo en movimientos_caja),
14458|  * devuelve { cajaTipo, moneda, montoAbs }. Si no aplica, null.
14459|  * Alineado a `sincronizarCcYCajaDesdeOrden` (multicontraparte manual vs estándar).
14460|  */
14461| function proyectarEgresoCajaEmpresaSiEjecutadaTransaccion(t, ctx) {
14462|     const EPS = EPS_SALDO_CAJA_VALIDACION;
14463|     const montoTrx = Number(t.monto) || 0;
14464|     if (montoTrx < EPS) return null;
14465|
14466|     const codigoModoRaw = ctx && ctx.codigoModo !== null ? String(ctx.codigoModo) : 'efectivo';
14467|     const codigoModo = codigoModoRaw.toLowerCase();
14468|
14469|     const esGananciaTrx = (t.concepto || '').includes('Ganancia del acuerdo');
14470|     const comP = ctx && ctx.comisionPandyMonto !== null ? Number(ctx.comisionPandyMonto) : null;
14471|     const esComisionPandyTrx =
14472|         comP !== null && comP >= EPS &&
14473|         (t.tipo || '').toLowerCase() === 'ingreso' &&
14474|         String(t.pagador || '').toLowerCase() === 'cliente' &&
14475|         String(t.cobrador || '').toLowerCase() === 'pandy' &&
14476|         Math.abs(montoTrx - comP) < 1e-6;
14477|
14478|     const ordenMini = (ctx && ctx.orden) || {};
14479|     const toJoin = ctx && ctx.tipoOperacionJoin;
14480|     const usarMc =
14481|         !(ctx && ctx.multicontraparteManual) &&
14482|         esTipoOpMulticontraparteElegibleDesdeOrden(ordenMini, toJoin);
14483|
14484|     const mon = (t.moneda || 'USD').toUpperCase();
14485|
14486|     if (usarMc && !esGananciaTrx && !esComisionPandyTrx) {
14487|         const pagDb =
14488|             t.pagador !== null && String(t.pagador).trim() !== ''
14489|             ? String(t.pagador).toLowerCase()
14490|             : null;
14491|         const cobDb =
14492|             t.cobrador !== null && String(t.cobrador).trim() !== ''
14493|             ? String(t.cobrador).toLowerCase()

```

```

14494 |     : null;
14495 |     if (!pagDb || !cobDb || (pagDb !== 'pandy' && cobDb !== 'pandy')) return null;
14496 |     if (codigoModo !== 'efectivo') return null;
14497 |     const signoCaja = cobDb === 'pandy' ? 1 : -1;
14498 |     if (signoCaja * montoTrx >= -EPS) return null;
14499 |     return { cajaTipo: 'efectivo', moneda: mon, montoAbs: montoTrx };
14500 | }
14501 |
14502 | const { pag, cob } = pagCobEfectivosTransaccionSync(t);
14503 | if (!(pag === 'pandy' || cob === 'pandy')) return null;
14504 | const signoCaja = cob === 'pandy' ? 1 : -1;
14505 | if (signoCaja * montoTrx >= -EPS) return null;
14506 | const cajaTipo = codigoCajaTipoDesdeCodigo(codigoModo);
14507 | return { cajaTipo, moneda: mon, montoAbs: montoTrx };
14508 | }
14509 |
14510 | /**
14511 |  * Antes de persistir estado ejecutada: comprueba saldo de caja empresa si la transacción implicaría egreso (sync).
14512 |  * @returns {Promise<{ ok: boolean, mensaje?: string }>}
14513 |  */
14514 | function validarSaldoCajaAntesMarcarTransaccionEjecutada(t, instrumentacionId, ordenId) {
14515 |     return Promise.all([
14516 |         client.from('instrumentacion').select('multicontraparte_manual').eq('id', instrumentacionId).single(),
14517 |         client.from('ordenes').select('cliente_id, intermediario_id, tipos_operacion(codigo, usa_intermediario)').eq('id', ordenId).single(),
14518 |         client.from('comisiones_orden').select('monto').eq('orden_id', ordenId).eq('beneficiario', 'pandy').maybeSingle(),
14519 |         fetchMovimientosCajaCerradosSinSync(),
14520 |         client.from('modos_pago').select('codigo').eq('id', t.modos_pago_id).maybeSingle(),
14521 |     ]).then(([rInstMc, rOrd, rCom, rListCaja, rModo]) => {
14522 |         const orden = rOrd.data || {};
14523 |         const toJoin = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
14524 |         const multicontraparteManual = !(rInstMc.data && rInstMc.data.multicontraparte_manual);
14525 |         const comisionPandyMonto = rCom.data != null ? Number(rCom.data.monto) : null;
14526 |         const codigoModo = (rModo.data && rModo.data.codigo) ? String(rModo.data.codigo) : 'efectivo';
14527 |         const proy = proyectarEgresoCajaEmpresaSiEjecutadaTransaccion(t, {
14528 |             multicontraparteManual,
14529 |             orden,
14530 |             tipoOperacionJoin: toJoin,
14531 |             comisionPandyMonto,
14532 |             codigoModo,
14533 |         });
14534 |         if (!proy) return { ok: true };
14535 |         return validarSaldoCajaParaEgresoLista(listCaja, proy);
14536 |     });
14537 | }
14538 |
14539 | /**
14540 |  * Cambia el estado de una transacción (pendiente => ejecutada) desde el combo en la tabla.
14541 |  * Actualiza CC y caja si pasa a ejecutada; luego actualiza estado de la orden y refresca la vista.
14542 |  * selectEl: opcional, el <select> que disparó el cambio; se usa para mostrar "Actualizando..." y deshabilitar combos.
14543 |  */
14544 | function cambiarEstadoTransaccion(transaccionId, nuevoEstado, instrumentacionId, selectEl) {
14545 |     if (!transaccionId || !instrumentacionId || !currentUserId) return Promise.resolve();
14546 |
14547 |     function showLoadingEstado() {
14548 |         if (!selectEl) return;
14549 |         const table = selectEl.closest('table');
14550 |         if (table) table.querySelectorAll('.combo-estado-transaccion').forEach((s) => { s.disabled = true; });
14551 |         const enWizard = selectEl.closest('#orden-inst-tabla-wrap');
14552 |         if (enWizard) {
14553 |             const msg = document.getElementById('orden-inst-actualizando-msg');
14554 |             if (msg) msg.style.display = 'inline';
14555 |         } else {
14556 |             showToast('Actualizando estado.', 'info');
14557 |         }
14558 |     }
14559 |     function hideLoadingEstado() {
14560 |         const msg = document.getElementById('orden-inst-actualizando-msg');
14561 |         if (msg) msg.style.display = 'none';
14562 |         if (selectEl) {
14563 |             const table = selectEl.closest('table');
14564 |             if (table) table.querySelectorAll('.combo-estado-transaccion').forEach((s) => { s.disabled = false; });
14565 |         }
14566 |     }
14567 |
14568 |     showLoadingEstado();
14569 |     const promCambioEstado = (async () => {
14570 |         const rInst = await client.from('instrumentacion').select('orden_id').eq('id', instrumentacionId).single();
14571 |         const ordenId = rInst.data && rInst.data.orden_id;
14572 |         if (!ordenId) return Promise.resolve();
14573 |         const rTr = await client.from('transacciones').select('tipo, numero, modos_pago_id, moneda, monto, cobrador, pagador, owner, concepto, estado, revertida_una_vez').eq('id', transaccionId).single();
14574 |         const t = rTr.data;
14575 |         if (!t) return Promise.resolve();
14576 |         if (String(t.estado || '').toLowerCase() === 'anulada') {
14577 |             hideLoadingEstado();
14578 |             if (selectEl) selectEl.disabled = true;
14579 |             showToast('Esta transacción está anulada junto con la orden.', 'info');
14580 |             return Promise.resolve();
14581 |         }
14582 |
14583 |         // Validación: no se puede marcar ejecutada el egreso de cheque (Pandy-Intermediario) si el ingreso de cheque (Cliente-Pandy) sigue pendiente. No podés entregar lo que no tenés
14584 |         if (nuevoEstado === 'ejecutada') {
14585 |             const esEgresoPandyInt = (t.tipo || '').toLowerCase() === 'egreso' && String(t.pagador || '').toLowerCase() === 'pandy' && String(t.cobrador || '').toLowerCase() === 'intermediario';
14586 |             if (esEgresoPandyInt) {
14587 |                 const [rModos, rOrden, rLista] = await Promise.all([

```

```

14588 |         client.from('modos_pago').select('id, codigo'),
14589 |         client.from('ordenes').select('id, intermediario_id, tipo_operacion_id').eq('id', ordenId).single(),
14590 |         client.from('transacciones').select('id, tipo, modo_pago_id, cobrador, pagador, estado').eq('instrumentacion_id', instrumentacionId),
14591 |     ]);
14592 |     const modosMap = Object.fromEntries((rModos.data || []).map((m) => [m.id, m]));
14593 |     const orden = rOrden?.data;
14594 |     const lista = rLista.data || [];
14595 |     const esCheque = (modosMap[t.modo_pago_id] && modosMap[t.modo_pago_id].codigo === 'cheque');
14596 |     const tieneIntermediario = orden && orden.intermediario_id;
14597 |     if (esCheque && tieneIntermediario) {
14598 |         const ingresoChequeCliPandy = lista.find((tr) => (tr.tipo || '').toLowerCase() === 'ingreso' && String(tr.pagador || '').toLowerCase() === 'cliente' && String(tr.cob
rador || '').toLowerCase() === 'pandy' && modosMap[tr.modo_pago_id] && modosMap[tr.modo_pago_id].codigo === 'cheque');
14599 |         if (ingresoChequeCliPandy && (ingresoChequeCliPandy.estado || '').toLowerCase() !== 'ejecutada') {
14600 |             hideLoadingEstado();
14601 |             if (selectEl) selectEl.value = 'pendiente';
14602 |             showToast('No se puede marcar ejecutada la entrega del cheque al intermediario si el ingreso del cheque (Cliente-Pandy) sigue pendiente.', 'error');
14603 |             return Promise.resolve();
14604 |         }
14605 |     }
14606 | }
14607 | const vSaldoCajaEj = await validarSaldoCajaAntesMarcarTransaccionEjecutada(t, instrumentacionId, ordenId);
14608 | if (!vSaldoCajaEj.ok) {
14609 |     hideLoadingEstado();
14610 |     if (selectEl) selectEl.value = 'pendiente';
14611 |     showToast(vSaldoCajaEj.mensaje || 'Saldo de caja insuficiente.', 'error');
14612 |     return Promise.resolve();
14613 | }
14614 | }
14615 |
14616 | // Reversión (ejecutada → pendiente): sin límite de veces. Siempre pedir confirmación.
14617 | if (nuevoEstado === 'pendiente' && t.estado === 'ejecutada') {
14618 |     const montoTrx = Number(t.monto) || 0;
14619 |     const monedaTrx = (t.moneda || 'ARS').toUpperCase();
14620 |     const esIngreso = (t.tipo || '').toLowerCase() === 'ingreso';
14621 |     const cobTrx = String(t.cobrador || '').toLowerCase();
14622 |     const pagTrx = String(t.pagador || '').toLowerCase();
14623 |     const textoImplicancia = montoTrx >= 1e-6
14624 |       ? (esIngreso && cobTrx === 'pandy'
14625 |         ? 'Reversar indica que no se recibieron los ' + formatMonto(montoTrx, monedaTrx) + ' (del cliente). '
14626 |         : (pagTrx === 'pandy'
14627 |           ? 'Reversar indica que no se entregaron los ' + formatMonto(montoTrx, monedaTrx) + '. '
14628 |           : ''))
14629 |       : '';
14630 |     return new Promise((resolve) => {
14631 |         showConfirm(
14632 |             textoImplicancia + '¿Reversar esta operación a pendiente? La cuenta corriente y la caja se actualizarán.',
14633 |             'Si, reversar',
14634 |             () => { continuarCambioEstado().then(resolve).catch(() => resolve()); },
14635 |             () => { hideLoadingEstado(); if (selectEl) selectEl.value = 'ejecutada'; resolve(); },
14636 |             'Cancelar',
14637 |             'Confirmar reversión'
14638 |         );
14639 |     });
14640 | }
14641 | return continuarCambioEstado();
14642 |
14643 | function continuarCambioEstado() {
14644 |     const montoActual = Number(t.monto) || 0;
14645 |     // Si el usuario cambió el monto en la tabla pero no hizo blur, sincronizar antes de ejecutar para que el split use el monto correcto.
14646 |     if (selectEl && nuevoEstado === 'ejecutada') {
14647 |         const row = selectEl.closest('tr');
14648 |         const inputMonto = row && row.querySelector('.input-monto-transaccion-tabla[data-id="' + transaccionId + '"]');
14649 |         if (inputMonto) {
14650 |             const val = parseImporteInput(inputMonto.value);
14651 |             if (!isNaN(val) && val > 0 && Math.abs(val - montoActual) > 1e-6) {
14652 |                 return guardarSoloMontoTransaccion(transaccionId, inputMonto.value).then(() => cambiarEstadoTransaccion(transaccionId, nuevoEstado, instrumentacionId, selectEl));
14653 |             }
14654 |         }
14655 |     }
14656 |     return client.from('ordenes').select('cliente_id, intermediario_id, monto_recibido, monto_entregado, moneda_recibida, moneda_entregada, cotizacion, numero, tasa_descuent
o_intermediario, tipos_operacion(codigo, usa_intermediario)').eq('id', ordenId).single().then((rOrd) => {
14657 |         const orden = rOrd.data || {};
14658 |         const clienteId = orden.cliente_id || null;
14659 |         const intermediarioId = orden.intermediario_id || null;
14660 |         const tipoCodigo = orden.tipos_operacion?.codigo || '';
14661 |         const cob = t.cobrador || (t.tipo === 'ingreso' ? (t.owner || 'pandy') : 'pandy');
14662 |         const pag = t.pagador || (t.tipo === 'egreso' ? (t.owner || 'pandy') : 'pandy');
14663 |         const montoRecibido = Number(orden.monto_recibido) || 0;
14664 |         const montoEntregado = Number(orden.monto_entregado) || 0;
14665 |         const monR = orden.moneda_recibida;
14666 |         const monE = orden.moneda_entregada;
14667 |         // No dividir la transacción: la comisión se trata en CC desde los datos del acuerdo (como en CC intermediario).
14668 |         const debeDividir = false;
14669 |         const comision = 0;
14670 |
14671 |         const payload = { estado: nuevoEstado, updated_at: new Date().toISOString() };
14672 |         if (nuevoEstado === 'ejecutada') {
14673 |             payload.fecha_ejecucion = fecha hoyYYYYMMDDArgentina();
14674 |             payload.usuario_id = currentUserid;
14675 |         }
14676 |         if (nuevoEstado === 'pendiente') payload.revertida_una_vez = true;
14677 |         if (debeDividir) payload.monto = montoEntregado;
14678 |
14679 |         const hacerUpdate = () => client.from('transacciones').update(payload).eq('id', transaccionId);
14680 |         const promUpdate = client.rpc('transacciones_cambiar_estado', {
14681 |             p_transaccion_id: transaccionId,
14682 |             p_estado: nuevoEstado,

```

```

14683|         p_fecha_ejecucion: nuevoEstado === 'ejecutada' ? payload.fecha_ejecucion : null,
14684|         p_usuario_id: nuevoEstado === 'ejecutada' ? currentUserId : null,
14685|         p_revertida_una_vez: nuevoEstado === 'pendiente' ? true : null,
14686|     }).then((rRpc) => {
14687|         if (rRpc.error) {
14688|             console.warn('transacciones_cambiar_estado:', rRpc.error.message || rRpc.error);
14689|             return hacerUpdate();
14690|         }
14691|         return { data: null, error: null };
14692|     });
14693|
14694|     return promUpdate.then((rUp) => {
14695|         if (rUp && rUp.error) {
14696|             showToast('Error al actualizar estado: ' + (rUp.error.message || ''), 'error');
14697|             return;
14698|         }
14699|         let promesaSiguiente = Promise.resolve(null);
14700|         if (debeDividir) {
14701|             promesaSiguiente = client.from('transacciones').insert({
14702|                 instrumentacion_id: instrumentacionId,
14703|                 tipo: 'ingreso',
14704|                 modo_pago_id: t.modo_pago_id,
14705|                 moneda: t.moneda,
14706|                 monto: comision,
14707|                 cobrador: 'pandy',
14708|                 pagador: 'cliente',
14709|                 owner: 'pandy',
14710|                 estado: 'ejecutada',
14711|                 concepto: 'Ganancia del acuerdo',
14712|                 tipo_cambio: null,
14713|                 fecha_ejecucion: payload.fecha_ejecucion,
14714|                 usuario_id: currentUserId,
14715|                 updated_at: new Date().toISOString(),
14716|             }).select('id, numero').single();
14717|         }
14718|         return promesaSiguiente.then((rNew) => {
14719|             const nuevaTrxId = rNew && rNew.data && rNew.data.id;
14720|             const nuevaTrxNumero = rNew && rNew.data && rNew.data.numero;
14721|             const fecha = fechaHoyYYYYMMDDArgentina();
14722|             const ahora = new Date().toISOString();
14723|             const monR = orden.moneda_recibida || 'USD';
14724|             const monE = orden.moneda_entregada || 'USD';
14725|             const mr = Number(orden.monto_recibido) || 0;
14726|             const me = Number(orden.monto_entregado) || 0;
14727|             const listaTrx = debeDividir
14728|                 ? [
14729|                     { id: transaccionId, numero: t.numero, monto: montoEntregado, moneda: t.moneda, modo_pago_id: t.modo_pago_id, concepto: conceptoCcMovimiento(t.moneda, montoE
ntregado, 'deuda'), cobrador: cob, pagador: pag },
14730|                     { id: nuevaTrxId, numero: nuevaTrxNumero, monto: comision, moneda: t.moneda, modo_pago_id: t.modo_pago_id, concepto: conceptoCcMovimiento(t.moneda, comision,
'comision'), cobrador: 'pandy', pagador: 'cliente' },
14731|                 ]
14732|                 : [{ id: transaccionId, numero: t.numero, monto: montoActual, moneda: t.moneda, modo_pago_id: t.modo_pago_id, concepto: conceptoCcMovimiento(t.moneda, montoActua
l, pag === 'cliente' ? 'cobro' : 'deuda'), cobrador: cob, pagador: pag }];
14733|
14734|             const idsTrx = listaTrx.map(i => i.id).filter(Boolean);
14735|             const estadoCc = nuevoEstado === 'ejecutada' ? 'cerrado' : 'pendiente';
14736|             const deletes = [];
14737|             if (nuevoEstado === 'pendiente') deletes.push(client.from('movimientos_caja').delete().eq('transaccion_id', transaccionId));
14738|             // Con instrumentación, derivar siempre CC y caja desde orden + transacciones (cualquier movimiento parcial se refleja).
14739|             if (instrumentacionId && !debeDividir) {
14740|                 const esIngresoClientePandy = (t.tipo || '').toLowerCase() === 'ingreso' && String(pag || '').toLowerCase() === 'cliente' && String(cob || '').toLowerCase() ===
'pandy';
14741|                 const comisionPandyMonto = (monR === monE & montoRecibido > montoEntregado) ? montoRecibido - montoEntregado : 0;
14742|                 const promRevertirGanancia = (nuevoEstado === 'pendiente' && esIngresoClientePandy && clienteId && comisionPandyMonto >= 1e-6)
14743|                     ? revertirGananciaPandy(ordenId, orden, clienteId, comisionPandyMonto)
14744|                     : Promise.resolve();
14745|                 // Regla simple: tras cambiar estado, siempre sync (recalcula CC y caja solo desde transacciones ejecutadas).
14746|                 const promRevertirComision = (nuevoEstado === 'pendiente' && cob === 'pandy' && pag === 'intermediario' && intermediarioId) ? revertirComisionIntermediario(orden
Id) : Promise.resolve();
14747|                 return Promise.all(deletes).then(() => promRevertirGanancia).then(() => promRevertirComision).then(() => sincronizarCc(CajaDesdeOrden(ordenId)).then(() => actual
izarEstadoOrden(ordenId)).then(() => ({ ordenId, instrumentacionId })));
14748|             }
14749|             const ordenLabel = orden.numero != null ? 'nro orden ' + orden.numero : 'nro orden ' + (ordenId || '').toString().slice(0, 8);
14750|             /** Cancelación: por el monto de esta transacción (item.monto). En misma moneda (monR === monE) solo ese monto en la moneda participante. */
14751|             function montosCancelacion(item) {
14752|                 const montoTrx = Number(item.monto) || 0;
14753|                 const esIngreso = item.pagador === 'cliente';
14754|                 if (monR === monE) {
14755|                     const signo = esIngreso ? 1 : -1;
14756|                     return {
14757|                         monto_usd: numCc(monR === 'USD' ? signo * montoTrx : 0),
14758|                         monto_ars: numCc(monR === 'ARS' ? signo * montoTrx : 0),
14759|                         monto_eur: numCc(monR === 'EUR' ? signo * montoTrx : 0),
14760|                     };
14761|                 }
14762|                 if (esIngreso) {
14763|                     const enMonE = ratioCc(montoTrx * me, mr, montoTrx);
14764|                     return {
14765|                         monto_usd: numCc(monR === 'USD' ? montoTrx : (monE === 'USD' ? enMonE : 0)),
14766|                         monto_ars: numCc(monR === 'ARS' ? montoTrx : (monE === 'ARS' ? enMonE : 0)),
14767|                         monto_eur: numCc(monR === 'EUR' ? montoTrx : (monE === 'EUR' ? enMonE : 0)),
14768|                     };
14769|                 }
14770|                 const enMonR = ratioCc(montoTrx * mr, me, montoTrx);
14771|                 return {
14772|                     monto_usd: numCc(monR === 'USD' ? -enMonR : (monE === 'USD' ? -montoTrx : 0)),
14773|                     monto_ars: numCc(monR === 'ARS' ? -enMonR : (monE === 'ARS' ? -montoTrx : 0)),

```

```

14774|         monto_eur: numCc(monR === 'EUR' ? -enMonR : (monE === 'EUR' ? -montoTrx : 0)),
14775|     });
14776| }
14777| return Promise.all(deletes).then(() => {
14778|     let promCcCliente = Promise.resolve({ data: [], idsTrxMomentoCero: [] });
14779|     if (clienteId) {
14780|         promCcCliente = client.from('movimientos_cuenta_corriente').select('id, transaccion_id, concepto, estado, monto_usd, monto_ars, monto_eur').eq('orden_id', orde
nId).eq('cliente_id', clienteId).then((rRows) => {
14781|             const rows = rRows.data || [];
14782|             const idsTrxMomentoCero = rows.filter((r) => r.monto_usd != null || r.monto_ars != null || r.monto_eur != null).map((r) => r.transaccion_id);
14783|             const updates = rows.map((row) => {
14784|                 const esDeEstaTrx = idsTrx.includes(row.transaccion_id);
14785|                 const payload = {
14786|                     estado_fecha: ahora,
14787|                     estado: esDeEstaTrx ? estadoCc : (row.estado || 'pendiente'),
14788|                 };
14789|                 return client.from('movimientos_cuenta_corriente').update(payload).eq('id', row.id);
14790|             });
14791|             return Promise.all(updates).then(() => ({ data: rows, idsTrxMomentoCero }));
14792|         });
14793|     }
14794|     let promCcInt = Promise.resolve({ data: [] });
14795|     if (intermediarioId && nuevoEstado === 'pendiente' && idsTrx.length > 0) {
14796|         // Misma regla que cliente: Debe (momento_cero: tiene monto_usd/ars/eur) -> solo estado pendiente; cobro (monto > 0) -> borrar.
14797|         promCcInt = client.from('movimientos_cuenta_corriente_intermediario').select('id, transaccion_id, monto, monto_usd, monto_ars, monto_eur').eq('orden_id', orden
Id).eq('intermediario_id', intermediarioId).in('transaccion_id', idsTrx).then((rRows) => {
14798|             const rows = rRows.data || [];
14799|             if (rows.length === 0) return Promise.resolve({ data: [] });
14800|             const promises = rows.map((row) => {
14801|                 const esMomentoCero = row.monto_usd != null || row.monto_ars != null || row.monto_eur != null;
14802|                 if (esMomentoCero) return client.from('movimientos_cuenta_corriente_intermediario').update({ estado: 'pendiente', estado_fecha: ahora }).eq('id', row.id);
14803|                 if (Number(row.monto) > 0) return client.from('movimientos_cuenta_corriente_intermediario').delete().eq('id', row.id);
14804|                 return Promise.resolve();
14805|             });
14806|             return Promise.all(promises).then(() => ({ data: rows }));
14807|         });
14808|     }
14809|     const promComisionPandy = client.from('comisiones_orden').select('monto').eq('orden_id', ordenId).eq('beneficiario', 'pandy').maybeSingle();
14810|     // Cargar transacciones después del update para que la suma ejecutada incluya esta transacción ya como ejecutada (evita crear resto erróneo al ejecutar el 10k).
14811|     const promTrxList = client.from('transacciones').select('id, tipo, monto, estado, cobrador, pagador').eq('instrumentacion_id', instrumentacionId);
14812|     return Promise.all([promCcCliente, promCcInt, promComisionPandy]).then(([rCc, rCcInt, rCom]) =>
14813|         promTrxList.then((rTrxList) => [rCc, rCcInt, rCom, rTrxList])
14814|     ).then(([rCc, rCcInt, rCom, rTrxList]) => {
14815|         const comisionPandyMonto = rCom.data != null ? Number(rCom.data.monto) : null;
14816|         const listTrx = rTrxList.data || [];
14817|         const sumIngresosClienteEjecutados = listTrx.filter((tr) => tr.tipo === 'ingreso' && tr.pagador === 'cliente' && tr.estado === 'ejecutada').reduce((s, tr) =>
14818|             s + Number(tr.monto), 0);
14819|         const sumEgresosClienteEjecutados = listTrx.filter((tr) => tr.tipo === 'egreso' && tr.cobrador === 'cliente' && tr.estado === 'ejecutada').reduce((s, tr) =>
14820|             s + Number(tr.monto), 0);
14821|         const idsTrxMomentoCero = rCc.idsTrxMomentoCero || [];
14822|         const rowsCc = rCc.data || [];
14823|         const rowDebe = rowsCc.find((r) => { const c = (r.concepto || '').toUpperCase(); return c.includes('DEBE') || c.includes('COMPROMISO DE PAGO'); });
14824|         const rowComp = rowsCc.find((r) => { const c = (r.concepto || '').normalize('NFD').replace(/u0301/g, '').toUpperCase(); return c.includes('COMPENSACION') ||
c.includes('COMPROMISO A COBRAR'); });
14825|         const tieneMomentoCero = !!(rowDebe && rowComp && (rowDebe.monto_usd != null || rowDebe.monto_ars != null || rowDebe.monto_eur != null) && (rowComp.monto_usd
!= null || rowComp.monto_ars != null || rowComp.monto_eur != null));
14826|         const ingresoId = rowDebe && rowDebe.transaccion_id;
14827|         const egresoId = rowComp && rowComp.transaccion_id;
14828|         const esIngresoEjecutada = t.tipo === 'ingreso' && nuevoEstado === 'ejecutada' && idsTrxMomentoCero.includes(transaccionId);
14829|         const esEgresoEjecutada = t.tipo === 'egreso' && nuevoEstado === 'ejecutada' && idsTrxMomentoCero.includes(transaccionId);
14830|         // Solo hacer split cuando la fila CC (Debe/Compensación) representa MÁS que este monto: así no duplicamos al ejecutar la transacción "diferencia" creada por
un split anterior.
14831|         const amountDebeMonR = rowDebe && (monR === 'USD' ? rowDebe.monto_usd : (monR === 'ARS' ? rowDebe.monto_ars : rowDebe.monto_eur));
14832|         const amountCompMonE = rowComp && (monE === 'USD' ? rowComp.monto_usd : (monE === 'ARS' ? rowComp.monto_ars : rowComp.monto_eur));
14833|         const splitIngreso = esIngresoEjecutada && mr > 1e-6 && montoActual > 1e-6 && (mr - montoActual) > 1e-6 && (Math.abs(Number(amountDebeMonR) || 0) - montoActu
al) > 1e-6;
14834|         const splitEgreso = esEgresoEjecutada && me > 1e-6 && montoActual > 1e-6 && (me - montoActual) > 1e-6 && (Math.abs(Number(amountCompMonE) || 0) - montoActu
al) > 1e-6;
14835|         const split = (splitIngreso || splitEgreso) && rowDebe && rowComp && clienteId;
14836|         // Split sin momento_cero: solo crear "resto" si tras esta ejecución el cliente aún no completó (suma ejecutada < mr/me). Así no se genera un resto erróneo a
l ejecutar la transacción de 10k que ya era el resto de un split anterior.
14837|         const faltaIngreso = mr - sumIngresosClienteEjecutados > 1e-6;
14838|         const faltaEgreso = me - sumEgresosClienteEjecutados > 1e-6;
14839|         const splitSinMomentoCero = !tieneMomentoCero && nuevoEstado === 'ejecutada' && clienteId && (
14840|             (t.tipo === 'ingreso' && pag === 'cliente' && mr > 1e-6 && montoActual > 1e-6 && faltaIngreso) ||
14841|             (t.tipo === 'egreso' && cob === 'cliente' && me > 1e-6 && montoActual > 1e-6 && faltaEgreso)
14842|         );
14843|         let promSplitSinMc = Promise.resolve(null);
14844|         if (splitSinMomentoCero) {
14845|             const diferencia = t.tipo === 'ingreso' ? (mr - sumIngresosClienteEjecutados) : (me - sumEgresosClienteEjecutados);
14846|             if (diferencia < 1e-6) { /* no crear resto si ya no hay diferencia */ } else {
14847|                 promSplitSinMc = client.from('transacciones').insert({
14848|                     instrumentacion_id: instrumentacionId,
14849|                     tipo: t.tipo,
14850|                     modo_pago_id: t.modo_pago_id,
14851|                     moneda: t.moneda || (t.tipo === 'ingreso' ? monR : monE),
14852|                     monto: diferencia,
14853|                     cobrador: cob,
14854|                     pagador: pag,
14855|                     owner: t.owner || 'pandy',
14856|                     estado: 'pendiente',
14857|                     concepto: t.concepto || '',
14858|                     tipo_cambio: t.tipo_cambio || null,
14859|                     updated_at: ahora,
14860|                 }).select('id').single().then((r) => r.data && r.data.id);
14861|             }
14862|         }
14863|     }

```

```

14861 | let promSplit = Promise.resolve(null);
14862 | if (split && splitIngreso) {
14863 |   const restoEnFila = Math.abs(Number(amountDebeMonR) || 0);
14864 |   const diferencia = restoEnFila - montoActual;
14865 |   if (diferencia >= 1e-6) {
14866 |     promSplit = client.from('transacciones').insert({
14867 |       instrumentacion_id: instrumentacionId,
14868 |       tipo: 'ingreso',
14869 |       modo_pago_id: t.modo_pago_id,
14870 |       moneda: t.moneda || monR,
14871 |       monto: diferencia,
14872 |       cobrador: cob,
14873 |       pagador: pag,
14874 |       owner: t.owner || 'pandy',
14875 |       estado: 'pendiente',
14876 |       concepto: t.concepto || '',
14877 |       tipo_cambio: t.tipo_cambio || null,
14878 |       updated_at: ahora,
14879 |     }).select('id').single().then((r) => r.data && r.data.id);
14880 |   }
14881 | } else if (split && splitEgreso) {
14882 |   const restoEnFila = Math.abs(Number(amountCompMonE) || 0);
14883 |   const diferencia = restoEnFila - montoActual;
14884 |   if (diferencia >= 1e-6) {
14885 |     promSplit = client.from('transacciones').insert({
14886 |       instrumentacion_id: instrumentacionId,
14887 |       tipo: 'egreso',
14888 |       modo_pago_id: t.modo_pago_id,
14889 |       moneda: t.moneda || monE,
14890 |       monto: diferencia,
14891 |       cobrador: cob,
14892 |       pagador: pag,
14893 |       owner: t.owner || 'pandy',
14894 |       estado: 'pendiente',
14895 |       concepto: t.concepto || '',
14896 |       tipo_cambio: t.tipo_cambio || null,
14897 |       updated_at: ahora,
14898 |     }).select('id').single().then((r) => r.data && r.data.id);
14899 |   }
14900 | }
14901 | const insertsCc = [];
14902 | const updatesCc = [];
14903 | let promBorrarCancelacion = Promise.resolve();
14904 | if (nuevoEstado === 'pendiente' && clienteId && idsTrx.length > 0) {
14905 |   promBorrarCancelacion = client.from('movimientos_cuenta_corriente').select('id, concepto').eq('orden_id', ordenId).eq('cliente_id', clienteId).in('transacc
ion_id', idsTrx).then((rDel) => {
14906 |     const rows = rDel.data || [];
14907 |     const idsCancel = rows.filter((r) => (r.concepto || '').includes('Cancelación de deuda') || (r.concepto || '').includes('Contraparte cancelación')).map((
x) => x.id);
14908 |     if (idsCancel.length > 0) return Promise.all(idsCancel.map((id) => client.from('movimientos_cuenta_corriente').delete().eq('id', id)));
14909 |   });
14910 | }
14911 | return promBorrarCancelacion.then(() => Promise.all([promSplit, promSplitSinMc])).then(([newTrxId]) => {
14912 |   if (nuevoEstado === 'ejecutada' && split && newTrxId != null) {
14913 |     if (splitIngreso) {
14914 |       const restoEnFilaIng = Math.abs(Number(amountDebeMonR) || 0);
14915 |       const diferencia = restoEnFilaIng - montoActual;
14916 |       const ejecutadoMr = montoActual;
14917 |       const ejecutadoMe = monR === monE ? montoActual : ratioCc(montoActual * me, mr, montoActual);
14918 |       const montoUsdCancel = numCc(monR === 'USD' ? ejecutadoMr : (monE === 'USD' ? ejecutadoMe : 0));
14919 |       const montoArsCancel = numCc(monR === 'ARS' ? ejecutadoMr : (monE === 'ARS' ? ejecutadoMe : 0));
14920 |       const montoEurCancel = numCc(monR === 'EUR' ? ejecutadoMr : (monE === 'EUR' ? ejecutadoMe : 0));
14921 |       const difUsd = monR === 'USD' ? -diferencia : (monE === 'USD' ? -ratioCc(me * diferencia, mr, diferencia) : 0);
14922 |       const difArs = monR === 'ARS' ? -diferencia : (monE === 'ARS' ? -ratioCc(me * diferencia, mr, diferencia) : 0);
14923 |       const difEur = monR === 'EUR' ? -diferencia : (monE === 'EUR' ? -ratioCc(me * diferencia, mr, diferencia) : 0);
14924 |       const remUsd = monR === 'USD' ? diferencia : (monE === 'USD' ? ratioCc(me * diferencia, mr, diferencia) : 0);
14925 |       const remArs = monR === 'ARS' ? diferencia : (monE === 'ARS' ? ratioCc(me * diferencia, mr, diferencia) : 0);
14926 |       const remEur = monR === 'EUR' ? diferencia : (monE === 'EUR' ? ratioCc(me * diferencia, mr, diferencia) : 0);
14927 |       insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
14928 |         cliente_id: clienteId, orden_id: ordenId, transaccion_id: transaccionId,
14929 |         concepto: 'Cancelación de deuda' + ordenLabel,
14930 |         fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
14931 |         monto_usd: montoUsdCancel, monto_ars: montoArsCancel, monto_eur: montoEurCancel,
14932 |         moneda: monR, monto: 0,
14933 |       }));
14934 |       updatesCc.push(client.from('movimientos_cuenta_corriente').update({
14935 |         transaccion_id: newTrxId,
14936 |         monto_usd: difUsd, monto_ars: difArs, monto_eur: difEur,
14937 |         estado_fecha: ahora,
14938 |       }).eq('id', rowDebe.id));
14939 |       updatesCc.push(client.from('movimientos_cuenta_corriente').update({
14940 |         monto_usd: remUsd, monto_ars: remArs, monto_eur: remEur,
14941 |         estado_fecha: ahora,
14942 |       }).eq('id', rowComp.id));
14943 |     } else {
14944 |       const restoEnFilaEgr = Math.abs(Number(amountCompMonE) || 0);
14945 |       const diferencia = restoEnFilaEgr - montoActual;
14946 |       const ejecutadoMe = montoActual;
14947 |       const ejecutadoMr = monR === monE ? montoActual : ratioCc(montoActual * mr, me, montoActual);
14948 |       const montoUsdCancel = numCc(monR === 'USD' ? -ejecutadoMr : (monE === 'USD' ? -ejecutadoMe : 0));
14949 |       const montoArsCancel = numCc(monR === 'ARS' ? -ejecutadoMr : (monE === 'ARS' ? -ejecutadoMe : 0));
14950 |       const montoEurCancel = numCc(monR === 'EUR' ? -ejecutadoMr : (monE === 'EUR' ? -ejecutadoMe : 0));
14951 |       // Egreso: diferencia está en monE; Debe (monR) debe llevar equivalente mr*diferencia/me
14952 |       const difUsd = numCc(monR === 'USD' ? -ratioCc(mr * diferencia, me, diferencia) : (monE === 'USD' ? -diferencia : 0));
14953 |       const difArs = numCc(monR === 'ARS' ? -ratioCc(mr * diferencia, me, diferencia) : (monE === 'ARS' ? -diferencia : 0));
14954 |       const difEur = numCc(monR === 'EUR' ? -ratioCc(mr * diferencia, me, diferencia) : (monE === 'EUR' ? -diferencia : 0));
14955 |       const remUsd = numCc(monR === 'USD' ? ratioCc(mr * diferencia, me, diferencia) : (monE === 'USD' ? diferencia : 0));

```

```

14956|         const remArs = numCc(monR === 'ARS' ? ratioCc(mr * diferencia, me, diferencia) : (monE === 'ARS' ? diferencia : 0));
14957|         const remEur = numCc(monR === 'EUR' ? ratioCc(mr * diferencia, me, diferencia) : (monE === 'EUR' ? diferencia : 0));
14958|         insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
14959|             cliente_id: clienteId, orden_id: ordenId, transaccion_id: transaccionId,
14960|             concepto: 'Cancelación de deuda ' + ordenLabel,
14961|             fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
14962|             monto_usd: montoUsdCancel, monto_ars: montoArsCancel, monto_eur: montoEurCancel,
14963|             moneda: monE, monto: 0,
14964|         }));
14965|         updatesCc.push(client.from('movimientos_cuenta_corriente').update({
14966|             transaccion_id: newTrxId,
14967|             monto_usd: remUsd, monto_ars: remArs, monto_eur: remEur,
14968|             estado_fecha: ahora,
14969|         }).eq('id', rowComp.id));
14970|         updatesCc.push(client.from('movimientos_cuenta_corriente').update({
14971|             monto_usd: difUsd, monto_ars: difArs, monto_eur: difEur,
14972|             estado_fecha: ahora,
14973|         }).eq('id', rowDebe.id));
14974|     }
14975| } else if (nuevoEstado === 'ejecutada') {
14976|     listaTrx.forEach((item) => {
14977|         if (!item.id) return;
14978|         if (idsTrxMomentoCero.includes(item.id) && clienteId && !split) {
14979|             if (monR !== monE && item.pagador === 'cliente') {
14980|                 const montoTrx = Number(item.monto) || 0;
14981|                 const enMonEval = ratioCc(montoTrx * me, mr, montoTrx);
14982|                 const cancelacion = { monto_usd: numCc(monR === 'USD' ? montoTrx : 0), monto_ars: numCc(monR === 'ARS' ? montoTrx : 0), monto_eur: numCc(monR === '
14983| EUR' ? montoTrx : 0) };
14984|                 const contraparte = { monto_usd: numCc(monR === 'USD' ? -montoTrx : (monE === 'USD' ? -enMonEval : 0)), monto_ars: numCc(monR === 'ARS' ? -montoTrx
14985| : (monE === 'ARS' ? -enMonEval : 0)), monto_eur: numCc(monR === 'EUR' ? -montoTrx : (monE === 'EUR' ? -enMonEval : 0)) };
14986|                 insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
14987|                     cliente_id: clienteId, orden_id: ordenId, transaccion_id: item.id,
14988|                     concepto: 'Cancelación de deuda ' + ordenLabel,
14989|                     fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
14990|                     monto_usd: cancelacion.monto_usd, monto_ars: cancelacion.monto_ars, monto_eur: cancelacion.monto_eur,
14991|                     moneda: item.moneda || 'USD', monto: 0,
14992|                 }));
14993|                 insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
14994|                     cliente_id: clienteId, orden_id: ordenId, transaccion_id: item.id,
14995|                     concepto: 'Contraparte cancelación ' + ordenLabel,
14996|                     fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
14997|                     monto_usd: contraparte.monto_usd, monto_ars: contraparte.monto_ars, monto_eur: contraparte.monto_eur,
14998|                     moneda: item.moneda || 'USD', monto: 0,
14999|                 }));
15000|             } else {
15001|                 const montos = montosCancelacion(item);
15002|                 insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
15003|                     cliente_id: clienteId, orden_id: ordenId, transaccion_id: item.id,
15004|                     concepto: 'Cancelación de deuda ' + ordenLabel,
15005|                     fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
15006|                     monto_usd: montos.monto_usd, monto_ars: montos.monto_ars, monto_eur: montos.monto_eur,
15007|                     moneda: item.moneda || 'USD', monto: 0,
15008|                 }));
15009|             }
15010|         }
15011|     });
15012|     const esPandyInt = (cob === 'pandy' && pag === 'intermediario') || (cob === 'intermediario' && pag === 'pandy');
15013|     listaTrx.forEach((item) => {
15014|         if (!item.id) return;
15015|         const needLegacyCc = !idsTrxMomentoCero.includes(item.id);
15016|         const montoItem = Number(item.monto) || 0;
15017|         const conceptoCc = item.concepto || conceptoCcMovimiento(item.moneda, item.monto, item.pagador === 'cliente' ? 'cobro' : 'deuda');
15018|         const montosCobroItem = montosCcPorMoneda(item.moneda || 'USD', montoItem);
15019|         const montosDeudaItem = montosCcPorMoneda(item.moneda || 'USD', -montoItem);
15020|         const esComisionPandyItem = item.pagador === 'cliente' && item.cobrador === 'pandy' && intermediarioId && comisionPandyMonto != null && Math.abs(montoI
15021| tem - comisionPandyMonto) < 1e-6;
15022|         if (needLegacyCc && item.pagador === 'cliente' && item.cobrador !== 'intermediario' && clienteId && !esComisionPandyItem) {
15023|             insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
15024|                 cliente_id: clienteId, moneda: item.moneda, monto: montoItem, orden_id: ordenId, transaccion_id: item.id,
15025|                 concepto: conceptoCc, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
15026|                 ...montosCobroItem,
15027|             }));
15028|         }
15029|         if (needLegacyCc && cob === 'cliente' && pag !== 'intermediario' && clienteId) {
15030|             insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
15031|                 cliente_id: clienteId, moneda: item.moneda, monto: -montoItem, orden_id: ordenId, transaccion_id: item.id,
15032|                 concepto: conceptoCc, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
15033|                 ...montosDeudaItem,
15034|             }));
15035|         }
15036|         if (esPandyInt && pag === 'intermediario' && intermediarioId) {
15037|             // Momento cero: ya existe fila Debe (-mr); solo actualizar a cerrado. No insertar para no duplicar.
15038|             insertsCc.push(
15039|                 client.from('movimientos_cuenta_corriente_intermediario').select('id, concepto, monto').eq('orden_id', ordenId).eq('intermediario_id', intermediari
15040| oId).eq('transaccion_id', item.id).then((rRows) => {
15041|                     const rows = rRows.data || [];
15042|                     const esMomentoCeroDebe = rows.some((r) => (Number(r.monto) || 0) < 0 && ((r.concepto || '').includes('Compromiso de Pago') || (r.concepto || ''
15043| .includes('Debe'))));
15044|                     const promUpdate = client.from('movimientos_cuenta_corriente_intermediario').update({ estado: 'cerrado', estado_fecha: ahora })
15045| .eq('orden_id', ordenId).eq('intermediario_id', intermediarioId).eq('transaccion_id', item.id).eq('estado', 'pendiente');
15046|                     if (esMomentoCeroDebe) return promUpdate;
15047|                     return promUpdate.then(() => client.from('movimientos_cuenta_corriente_intermediario').insert({
15048|                         intermediario_id: intermediarioId, moneda: item.moneda, monto: montoItem, orden_id: ordenId, transaccion_id: item.id,
15049|                         concepto: conceptoCc, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
15050|                         ...montosCcPorMoneda(item.moneda || 'USD', montoItem),
15051|                     }));
15052|                 })
15053|             );
15054|     }

```

```

15048 |         );
15049 |     }
15050 |     if (esPandyInt && cob === 'intermediario' && intermediarioId) {
15051 |         const tasa = Number(orden.tasa_descuento_intermediario) || 0;
15052 |         const montoEfectivoInt = (typeof tasa === 'number' && !isNaN(tasa) && tasa >= 0 && tasa < 1) ? mr * (1 - tasa) : mr;
15053 |         const monInt = orden.moneda_recibida || item.moneda || 'ARS';
15054 |         // Momento cero: ya existe Fila Compensación (+mr); solo actualizar a cerrado. Si no, insertar Cobro Realizado +montoEfectivoInt.
15055 |         insertsCc.push(
15056 |             client.from('movimientos_cuenta_corriente_intermediario').select('id, concepto, monto').eq('orden_id', ordenId).eq('intermediario_id', intermediari
oid).eq('transaccion_id', item.id).then((rRows) => {
15057 |                 const rows = rRows.data || [];
15058 |                 const esMomentoCeroComp = rows.some((r) => (Number(r.monto) || 0) > 0 && ((r.concepto || '').includes('Compromiso a Cobrar') || (r.concepto || ''
).includes('Compensación')));
15059 |                 const promUpdate = rows.length > 0
15060 |                     ? client.from('movimientos_cuenta_corriente_intermediario').update({ estado: 'cerrado', estado_fecha: ahora }).eq('orden_id', ordenId).eq('inte
rmediario_id', intermediarioId).eq('transaccion_id', item.id).eq('estado', 'pendiente')
15061 |                     : Promise.resolve();
15062 |                 if (esMomentoCeroComp) return promUpdate;
15063 |                 return promUpdate.then(() => client.from('movimientos_cuenta_corriente_intermediario').insert({
15064 |                     intermediario_id: intermediarioId, orden_id: ordenId, transaccion_id: item.id, transaccion_numero: item.numero != null ? item.numero : null,
15065 |                     moneda: monInt, monto: montoEfectivoInt,
15066 |                     concepto: conceptoCcLeyenda('cobro_realizado', orden && orden.numero != null ? orden.numero : null, item.numero), fecha, usuario_id: currentUse
rId, estado: 'cerrado', estado_fecha: ahora,
15067 |                     ...montosCcPorMoneda(monInt, montoEfectivoInt),
15068 |                 })).then(() => client.from('comisiones_orden').select('moneda, monto').eq('orden_id', ordenId).eq('beneficiario', 'intermediario').maybeSingle())
15069 |                     .then((rCom) => {
15070 |                         const comMonto = rCom.data && (Number(rCom.data.monto) || 0);
15071 |                         if (comMonto >= 1e-6) {
15072 |                             const monCom = (rCom.data.moneda || 'ARS').toUpperCase();
15073 |                             return client.from('movimientos_cuenta_corriente_intermediario').insert({
15074 |                                 intermediario_id: intermediarioId, orden_id: ordenId, transaccion_id: item.id, transaccion_numero: item.numero != null ? item.numero : nu
ll,
15075 |                                 moneda: monCom, monto: -comMonto,
15076 |                                 concepto: conceptoCcLeyenda('comision_acuerdo', orden && orden.numero != null ? orden.numero : null, item.numero), fecha, usuario_id: cur
rentUserId, estado: 'cerrado', estado_fecha: ahora,
15077 |                                 ...montosCcPorMoneda(monCom, -comMonto),
15078 |                             })).then(() => asegurarComisionIntermediario(ordenId, instrumentacionId, intermediarioId, comMonto, monCom, orden && orden.numero != null ?
orden.numero : null, item.numero != null ? item.numero : null));
15079 |                         }
15080 |                         return Promise.resolve();
15081 |                     });
15082 |                 })
15083 |             );
15084 |         }
15085 |         if (cob === 'cliente' && pag === 'intermediario' && intermediarioId) {
15086 |             insertsCc.push(client.from('movimientos_cuenta_corriente_intermediario').insert({
15087 |                 intermediario_id: intermediarioId, moneda: item.moneda, monto: -montoItem, orden_id: ordenId, transaccion_id: item.id,
15088 |                 transaccion_numero: item.numero != null ? item.numero : null,
15089 |                 concepto: conceptoCc, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
15090 |                 ...montosCcPorMoneda(item.moneda || 'USD', -montoItem),
15091 |             }));
15092 |         }
15093 |         if (cob === 'intermediario' && pag === 'cliente' && intermediarioId) {
15094 |             insertsCc.push(client.from('movimientos_cuenta_corriente_intermediario').insert({
15095 |                 intermediario_id: intermediarioId, moneda: item.moneda, monto: -montoItem, orden_id: ordenId, transaccion_id: item.id,
15096 |                 transaccion_numero: item.numero != null ? item.numero : null,
15097 |                 concepto: conceptoCcLeyenda('pago_realizado', orden && orden.numero != null ? orden.numero : null, item.numero), fecha, usuario_id: currentUserId,
estado: 'cerrado', estado_fecha: ahora,
15098 |                 ...montosCcPorMoneda(item.moneda || 'USD', -montoItem),
15099 |             }));
15100 |         }
15101 |     });
15102 | }
15103 | const promUpdatesCc = updatesCc.length > 0 ? Promise.all(updatesCc) : Promise.resolve();
15104 | function actualizarEstadoYConversion() {
15105 |     return actualizarEstadoOrden(ordenId).then((res) => {
15106 |         return res;
15107 |     }).then(() => ({ ordenId, instrumentacionId }));
15108 | }
15109 | function actualizarEstadoSinSync() {
15110 |     return actualizarEstadoOrden(ordenId).then(() => actualizarEstadoYConversion());
15111 | }
15112 | return Promise.all(insertsCc).then(() => promUpdatesCc).then(() => {
15113 |     if (nuevoEstado !== 'ejecutada') {
15114 |         let promReversa = Promise.resolve();
15115 |         if (cob === 'pandy' && pag === 'intermediario' && intermediarioId) {
15116 |             promReversa = promReversa.then(() => revertirComisionIntermediario(ordenId)).then(() =>
15117 |                 client.from('movimientos_cuenta_corriente_intermediario').select('id, concepto').eq('orden_id', ordenId).eq('intermediario_id', intermediarioId).eq
('transaccion_id', transaccionId).then((rRows) => {
15118 |                     const rows = rRows.data || [];
15119 |                     const idsBorrar = rows.filter((r) => (r.concepto || '').includes('Compromiso a Cobrar') || (r.concepto || '').includes('Comisión del acuerdo')).m
ap((r) => r.id);
15120 |                     const rowDebe = rows.find((r) => (r.concepto || '').toLowerCase().includes('compromiso a cobrar'));
15121 |                     const del = idsBorrar.length > 0 ? Promise.all(idsBorrar.map((id) => client.from('movimientos_cuenta_corriente_intermediario').delete().eq('id',
id))) : Promise.resolve();
15122 |                     const monR = orden.moneda_recibida || 'ARS';
15123 |                     const montoUsd = monR === 'USD' ? -mr : 0;
15124 |                     const montoArs = monR === 'ARS' ? -mr : 0;
15125 |                     const montoEur = monR === 'EUR' ? -mr : 0;
15126 |                     const upd = rowDebe && rowDebe.id ? client.from('movimientos_cuenta_corriente_intermediario').update({ estado: 'pendiente', estado_fecha: ahora,
monto_usd: montoUsd, monto_ars: montoArs, monto_eur: montoEur }).eq('id', rowDebe.id) : Promise.resolve();
15127 |                     return del.then(() => upd);
15128 |                 })
15129 |             );
15130 |         }
15131 |         if (cob === 'intermediario' && pag === 'pandy' && intermediarioId) {
15132 |             promReversa = promReversa.then(() =>

```



```

15133|         client.from('movimientos_cuenta_corriente_intermediario').select('id, concepto').eq('orden_id', ordenId).eq('intermediario_id', intermediarioId).eq
15134| ('transaccion_id', transaccionId).then((rRows) => {
15135|             const rows = rRows.data || [];
15136|             const idsBorrar = rows.filter((r) => (r.concepto || '').includes('Cobro') || (r.concepto || '').includes('Descuento')).map((r) => r.id);
15137|             const idCompensacion = rows.find((r) => (r.concepto || '').toLowerCase().includes('compensacion'));
15138|             const del = idsBorrar.length > 0 ? Promise.all(idsBorrar.map(id) => client.from('movimientos_cuenta_corriente_intermediario').delete().eq('id',
15139| id))) : Promise.resolve();
15140|             const upd = idCompensacion ? client.from('movimientos_cuenta_corriente_intermediario').update({ estado: 'pendiente', estado_fecha: ahora }).eq('i
15141| d', idCompensacion.id) : Promise.resolve();
15142|             return del.then(() => upd);
15143|         });
15144|     }
15145|     // Revertir Ganancia del acuerdo si esta transacción es ingreso cliente-Pandy y había comisión.
15146|     const esIngresoClientePandyLegacy = (t.tipo || '').toLowerCase() === 'ingreso' && pag === 'cliente' && cob === 'pandy';
15147|     const comisionPandyLegacy = (monR === monE && mr > me) ? mr - me : 0;
15148|     if (esIngresoClientePandyLegacy && clienteId && comisionPandyLegacy >= 1e-6) {
15149|         promReversa = promReversa.then(() => revertirGananciaPandy(ordenId, orden, clienteId, comisionPandyLegacy));
15150|     }
15151|     return promReversa.then(() => actualizarEstadoSinSync());
15152| }
15153| const pandyParticipa = cob === 'pandy' || pag === 'pandy';
15154| if (!pandyParticipa) return actualizarEstadoSinSync();
15155| let promCaja = Promise.resolve();
15156| listaTrx.forEach(item => {
15157|     if (!item.id || !item.modos_pago_id) return;
15158|     promCaja = promCaja.then(() => {
15159|         client.from('movimientos_caja').select('id').eq('transaccion_id', item.id).limit(1).then((rCaja) => {
15160|             if (rCaja.data && rCaja.data.length > 0) return;
15161|             return client.from('modos_pago').select('codigo').eq('id', item.modos_pago_id).single().then((rModo) => {
15162|                 const codigo = (rModo.data && rModo.data.codigo) || '';
15163|                 const cajaTipo = codigoCajaTipoDesdeCodigo(codigo);
15164|                 const signoCaja = (item.cobrador || '') === 'pandy' ? 1 : -1;
15165|                 const concepto = conceptoCajaTransaccion(item.cobrador || '') === 'pandy', item.moneda, Number(item.monto) || 0, orden.numero, item.numero);
15166|                 return client.from('movimientos_caja').insert({
15167|                     moneda: item.moneda, monto: signoCaja * Number(item.monto), caja tipo: cajaTipo, transaccion_id: item.id,
15168|                     orden_numero: orden.numero != null ? orden.numero : null, transaccion_numero: item.numero != null ? item.numero : null,
15169|                     concepto, fecha, usuario_id: currentUserId,
15170|                 });
15171|             });
15172|         });
15173|     });
15174| });
15175| return promCaja.then(() => actualizarEstadoSinSync()).then(() => {
15176|     if (ordenWizardInstrumentacionIdActual && instrumentacionId && ordenWizardInstrumentacionIdActual === instrumentacionId) renderOrdenWizardInstrumentaci
on(instrumentacionId);
15177|     refreshCvView();
15178| });
15179| });
15180| });
15181| });
15182| });
15183| });
15184| } } ());
15185| return promCambioEstado.then((ctx) => {
15186|     hideLoadingEstado();
15187|     if (ctx && ctx.ordenId) {
15188|         if (ordenWizardInstrumentacionIdActual === instrumentacionId) {
15189|             renderOrdenWizardInstrumentacion(instrumentacionId);
15190|             // No llamar loadOrdenes() con el modal abierto: la vista Órdenes cargando atrás hace que el modal pierda el foco.
15191|         } else {
15192|             refreshTransaccionesPanel(ctx.ordenId);
15193|             // No llamar loadOrdenes() para no colapsar la fila de transacciones expandida.
15194|         }
15195|         const vistaCc = document.getElementById('vista-cuenta-corriente');
15196|         if (vistaCc && vistaCc.style.display !== 'none') loadCuentaCorriente();
15197|         showToast('Estado de la transacción actualizado.', 'success');
15198|     }
15199| }).catch((err) => {
15200|     hideLoadingEstado();
15201|     if (err && err.message) showToast('Error: ' + err.message, 'error');
15202|     if (ordenWizardInstrumentacionIdActual === instrumentacionId && instrumentacionId) {
15203|         renderOrdenWizardInstrumentacion(instrumentacionId);
15204|     }
15205| });
15206| }
15207|
15208| function saveTransaccion() {
15209|     const backdropTr = document.getElementById('modal-transaccion-backdrop');
15210|     if (backdropTr && backdropTr.dataset.transaccionAnulada === '1') {
15211|         showToast('Esta transacción está anulada; no se puede guardar cambios.', 'info');
15212|         return;
15213|     }
15214|     const idEl = document.getElementById('transaccion-id');
15215|     const instIdEl = document.getElementById('transaccion-instrumentacion-id');
15216|     const id = idEl?.value?.trim() || '';
15217|     const instrumentacionId = instIdEl?.value?.trim();
15218|     if (!instrumentacionId) {
15219|         showToast('Falta instrumentación.', 'error');
15220|         return;
15221|     }
15222|     const esOrdenCheque = document.getElementById('transaccion-es-orden-cheque')?.value === '1';
15223|     const tipo = document.getElementById('transaccion-tipo').value;
15224|     const modoPagoId = (document.getElementById('transaccion-modo-pago').value || '').trim();
15225|     if (!modoPagoId) {

```

```

15226|     showToast('Elegí un modo de pago.', 'error');
15227|     return;
15228| }
15229| const moneda = esOrdenCheque ? 'ARS' : document.getElementById('transaccion-moneda').value;
15230| const monto = parseImporteInput(document.getElementById('transaccion-monto').value);
15231| const cobrador = document.getElementById('transaccion-cobrador').value;
15232| const pagador = document.getElementById('transaccion-pagador').value;
15233| const estado = document.getElementById('transaccion-estado').value;
15234| if (estado === 'anulada') {
15235|     showToast('El estado Anulada no se asigna desde este formulario (solo al anular la orden).', 'info');
15236|     return;
15237| }
15238| const conceptoRaw = document.getElementById('transaccion-concepto').value.trim();
15239| const concepto = conceptoRaw || null;
15240| const tipoCambioRaw = document.getElementById('transaccion-tipo-cambio').value.trim();
15241| const tipoCambio = esOrdenCheque ? null : (tipoCambioRaw ? parseImporteInput(tipoCambioRaw) : null);
15242|
15243| if (isNaN(monto) || monto <= 0) {
15244|     showToast(esOrdenCheque || moneda !== 'ARS' ? 'Monto debe ser un número positivo.' : 'Completá el monto en la moneda indicada; el tipo de cambio se toma del acuerdo.', 'error');
15245|     return;
15246| }
15247|
15248| /** @param {string|null|undefined} clienteIdAcuerdo - orden.cliente_id: obligatorio persistir si pagador = cliente del acuerdo (combo oculto), para CC Pagador/Cobrador y syn
c. */
15249| function leerIdsContraparteMulticontraparteForm(clienteIdAcuerdo) {
15250|     const v = (idEl) => {
15251|         const s = document.getElementById(idEl)?.value;
15252|         if (s === null || String(s).trim() === '') return null;
15253|         return String(s).trim();
15254|     };
15255|     const cidAc =
15256|         clienteIdAcuerdo != null && String(clienteIdAcuerdo).trim() !== '' ? String(clienteIdAcuerdo).trim() : null;
15257|     const pagadorRol = (document.getElementById('transaccion-pagador')?.value || '').toLowerCase();
15258|     const cobradorRol = (document.getElementById('transaccion-cobrador')?.value || '').toLowerCase();
15259|     const wrapPagCli = document.getElementById('transaccion-wrap-pagador-cliente-id');
15260|     const modoPagAcuerdo = wrapPagCli && wrapPagCli.dataset.pagadorClienteModo === 'acuerdo';
15261|     let pagadorClienteId = null;
15262|     if (pagadorRol === 'cliente') {
15263|         if (modoPagAcuerdo) {
15264|             pagadorClienteId = cidAc || v('transaccion-pagador-cliente-id');
15265|         } else {
15266|             pagadorClienteId = v('transaccion-pagador-cliente-id');
15267|         }
15268|     }
15269|     return {
15270|         pagador_cliente_id: pagadorClienteId,
15271|         cobrador_cliente_id: cobradorRol !== 'cliente' ? null : v('transaccion-cobrador-cliente-id'),
15272|         pagador_intermediario_id: pagadorRol !== 'intermediario' ? null : v('transaccion-pagador-intermediario-id'),
15273|         cobrador_intermediario_id: cobradorRol !== 'intermediario' ? null : v('transaccion-cobrador-intermediario-id'),
15274|     };
15275| }
15276|
15277| let ordenClienteIdParaIdsMc = null;
15278| /** Orden cargada en el flujo de guardado; sirve para completar UUIDs de pagador/cobrador en cualquier tipo de trx. */
15279| let ordenParaCompletarIdsTransaccion = null;
15280| const idsMcForm = leerIdsContraparteMulticontraparteForm(null);
15281| let transaccionProyectada = { tipo, moneda, monto, tipo_cambio: tipoCambio, cobrador, pagador, ...idsMcForm };
15282|
15283| client.from('instrumentacion').select('orden_id, multicontraparte_manual').eq('id', instrumentacionId).single().then((rInst) => {
15284|     const ordenId = rInst.data?.orden_id;
15285|     if (!ordenId) {
15286|         showToast('No se encontró la orden de esta instrumentación.', 'error');
15287|         return;
15288|     }
15289|     client.from('ordenes').select('id, cliente_id, intermediario_id, moneda_recibida, monto_recibido, moneda_entregada, monto_entregado, cotizacion, tipos_operacion(codigo, us
a_intermediario)').eq('id', ordenId).single().then((rOrd) => {
15290|         const orden = rOrd.data;
15291|         if (!orden) {
15292|             showToast('No se encontró la orden.', 'error');
15293|             return;
15294|         }
15295|         ordenClienteIdParaIdsMc = orden.cliente_id || null;
15296|         ordenParaCompletarIdsTransaccion = orden;
15297|         const toJoinSv = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
15298|         const totMcSv = !(rInst.data && rInst.data.multicontraparte_manual) && esTipoOpMulticontraparteElegibleDesdeOrden(orden, toJoinSv);
15299|         const totalesOptsSv = totMcSv ? { totalesMulticontraparte: true } : undefined;
15300|         transaccionProyectada = { ...transaccionProyectada, ...leerIdsContraparteMulticontraparteForm(orden.cliente_id) };
15301|         const wrapPagCliVal = document.getElementById('transaccion-wrap-pagador-cliente-id');
15302|         if (totMcSv) {
15303|             if (pagador === 'intermediario') {
15304|                 const idInt = transaccionProyectada.pagador_intermediario_id || orden.intermediario_id;
15305|                 if (!idInt) {
15306|                     showToast('Elegí el intermediario pagador.', 'error');
15307|                     return;
15308|                 }
15309|             }
15310|             if (cobrador === 'intermediario') {
15311|                 const idInt = transaccionProyectada.cobrador_intermediario_id || orden.intermediario_id;
15312|                 if (!idInt) {
15313|                     showToast('Elegí el intermediario cobrador.', 'error');
15314|                     return;
15315|                 }
15316|             }
15317|             if (pagador === 'cliente' && wrapPagCliVal && wrapPagCliVal.dataset.pagadorClienteModo === 'otro' && !transaccionProyectada.pagador_cliente_id) {
15318|                 showToast('Elegí el cliente pagador.', 'error');
15319|                 return;

```

```

15320 |     }
15321 |     if (pagador === 'cliente' && cobrador === 'cliente') {
15322 |         const cidPagRes = transaccionProyectada.pagador_cliente_id || orden.cliente_id;
15323 |         const cidCobRes = transaccionProyectada.cobrador_cliente_id;
15324 |         if (!cidCobRes) {
15325 |             showToast('En Cliente → Cliente elegí el cliente cobrador (puede ser el del acuerdo si recibe la entrega, u otro cliente).', 'error');
15326 |             return;
15327 |         }
15328 |         if (cidPagRes && String(cidCobRes) === String(cidPagRes)) {
15329 |             showToast('El cobrador no puede ser el mismo cliente que el pagador.', 'error');
15330 |             return;
15331 |         }
15332 |     }
15333 | }
15334 | const idsMismaEntidad = totMcSv
15335 | ? transaccionProyectada
15336 | : { pagador_cliente_id: null, cobrador_cliente_id: null, pagador_intermediario_id: null, cobrador_intermediario_id: null };
15337 | if (esMismoParticipantePagadorCobrador(pagador, cobrador, orden, idsMismaEntidad)) {
15338 |     showToast('El pagador y el cobrador no pueden ser la misma entidad.', 'error');
15339 |     return;
15340 | }
15341 | client.from('transacciones').select('id, tipo, modo_pago_id, moneda, monto, cobrador, pagador, owner, estado, concepto, tipo_cambio, pagador_cliente_id, cobrador_cliente
_id, pagador_intermediario_id, cobrador_intermediario_id').eq('instrumentacion_id', instrumentacionId).then((rTr) => {
15342 |     const list = rTr.data || [];
15343 |     const validacion = validarTotalesVsAcuerdo(list, orden, id || null, transaccionProyectada, totales0ptsSv);
15344 |     if (!validacion.ok) {
15345 |         showToast(validacion.mensaje, 'error');
15346 |         return;
15347 |     }
15348 |     const trActual = id ? list.find((t) => t.id === id) : null;
15349 |     const oldMonto = trActual != null && !isNaN(Number(trActual.monto)) ? Number(trActual.monto) : null;
15350 |     const bajandoImporte = id && oldMonto != null && monto < oldMonto - 1e-6;
15351 |     const diferenciaComp = bajandoImporte ? oldMonto - monto : null;
15352 |     const esParPandyIntermediario =
15353 |     (cobrador === 'pandy' && pagador === 'intermediario') || (cobrador === 'intermediario' && pagador === 'pandy');
15354 |     if (bajandoImporte && diferenciaComp > 0) {
15355 |         // Si es momento cero + guardamos ejecutada + monto menor: el split crea la diferencia; no ofrecer "compensatoria" para no duplicar.
15356 |         return client.from('movimientos_cuenta_corriente').select('id, monto_usd, monto_ars, monto_eur').eq('transaccion_id', id).then((rMov) => {
15357 |             const filas = rMov.data || [];
15358 |             const tieneMomentoCero = filas.some((m) => m.monto_usd != null || m.monto_ars != null || m.monto_eur != null);
15359 |             const mr = Number(orden.monto_recibido) || 0;
15360 |             const me = Number(orden.monto_entregado) || 0;
15361 |             const esSplit = tieneMomentoCero && estado === 'ejecutada' && (
15362 |                 (tipo === 'ingreso' && mr > 1e-6 && monto > 1e-6 && (mr - monto) > 1e-6) ||
15363 |                 (tipo === 'egreso' && me > 1e-6 && monto > 1e-6 && (me - monto) > 1e-6)
15364 |             );
15365 |             if (esSplit || esParPandyIntermediario) {
15366 |                 guardarTransaccionPayload();
15367 |             } else {
15368 |                 showConfirm(
15369 |                     'Estás bajando el importe. ¿Querés que el sistema genere una transacción compensatoria con los mismos datos por ${formatImporteDisplay(diferenciaComp)} ${moned
a) para cerrar el acuerdo?',
15370 |                     'Sí, generar compensatoria',
15371 |                     () => guardarTransaccionPayload(diferenciaComp),
15372 |                     () => guardarTransaccionPayload()
15373 |                 );
15374 |             }
15375 |             });
15376 |         }
15377 |         guardarTransaccionPayload();
15378 |     });
15379 | });
15380 | });
15381 |
15382 | function guardarTransaccionPayload(montoCompensatorio) {
15383 |     const wrapMcSave = document.getElementById('transaccion-multicontraparte-contrapartes-wrap');
15384 |     const userMcSave = wrapMcSave && wrapMcSave.style.display !== 'none';
15385 |     const idsMcSave = leerIdsContraparteMulticontraparteForm(ordenClienteIdParaIdsMc);
15386 |     const payload = {
15387 |         instrumentacion_id: instrumentacionId,
15388 |         tipo,
15389 |         modo_pago_id: modoPagoId,
15390 |         moneda,
15391 |         monto,
15392 |         cobrador,
15393 |         pagador,
15394 |         owner: cobrador,
15395 |         estado,
15396 |         concepto,
15397 |         tipo_cambio: tipoCambio,
15398 |         updated_at: new Date().toISOString(),
15399 |     };
15400 |     if (userMcSave) {
15401 |         payload.pagador_cliente_id = idsMcSave.pagador_cliente_id;
15402 |         payload.cobrador_cliente_id = idsMcSave.cobrador_cliente_id;
15403 |         payload.pagador_intermediario_id = idsMcSave.pagador_intermediario_id;
15404 |         payload.cobrador_intermediario_id = idsMcSave.cobrador_intermediario_id;
15405 |     } else {
15406 |         payload.pagador_cliente_id = null;
15407 |         payload.cobrador_cliente_id = null;
15408 |         payload.pagador_intermediario_id = null;
15409 |         payload.cobrador_intermediario_id = null;
15410 |     }
15411 |     completarPayloadIdsContraparteDesdeOrden(payload, ordenParaCompletarIdsTransaccion);
15412 |     if (estado === 'ejecutada') payload.fecha_ejecucion = fechaHoyYYYYMMDDArgentina();
15413 |     if (estado === 'ejecutada') payload.usuario_id = currentUserId;
15414 |

```

```

15415 | const prom = id
15416 | ? client.from('transacciones').update(payload).eq('id', id)
15417 | : client.from('transacciones').insert(payload).select('id, numero').single();
15418 |
15419 | function insertarCompensatoria() {
15420 |   if (!montoCompensatorio || montoCompensatorio <= 0) return Promise.resolve();
15421 |   const payloadComp = {
15422 |     instrumentacion_id: instrumentacionId,
15423 |     tipo,
15424 |     modo_pago_id: modoPagoId,
15425 |     moneda,
15426 |     monto: montoCompensatorio,
15427 |     cobrador,
15428 |     pagador,
15429 |     owner: cobrador,
15430 |     estado: 'pendiente',
15431 |     concepto: 'Compensación por reducción de importe',
15432 |     tipo_cambio: tipoCambio,
15433 |     updated_at: new Date().toISOString(),
15434 |     pagador_cliente_id: usarMcSave ? idsMcSave.pagador_cliente_id : null,
15435 |     cobrador_cliente_id: usarMcSave ? idsMcSave.cobrador_cliente_id : null,
15436 |     pagador_intermediario_id: usarMcSave ? idsMcSave.pagador_intermediario_id : null,
15437 |     cobrador_intermediario_id: usarMcSave ? idsMcSave.cobrador_intermediario_id : null,
15438 |   };
15439 |   completarPayloadIdsContraparteDesdeOrden(payloadComp, ordenParaCompletarIdsTransaccion);
15440 |   return client.from('transacciones').insert(payloadComp).then((r) => {
15441 |     if (r.error) showToast('Error al crear la transacción compensatoria: ' + (r.error.message || ''), 'error');
15442 |     else showToast('Transacción compensatoria creada por ' + formatImporteDisplay(montoCompensatorio) + ' ' + moneda + '.', 'success');
15443 |   });
15444 | }
15445 |
15446 | prom.then((res) => {
15447 |   if (res.error) {
15448 |     showToast('Error: ' + (res.error.message || 'No se pudo guardar.'), 'error');
15449 |     return;
15450 |   }
15451 |   const transaccionId = id || (res.data && (res.data.id || (res.data[0] && res.data[0].id)));
15452 |   const transaccionNumero = id ? null : (res.data && (res.data.numero != null ? res.data.numero : (res.data[0] && res.data[0].numero)));
15453 |   if (!transaccionId) {
15454 |     closeModalTransaccion();
15455 |     refreshTransaccionesModal();
15456 |     return;
15457 |   }
15458 |   function continuarFlujo() {
15459 |     const fecha = fechaHoyYYYYMMDDArgentina();
15460 |     const ahora = new Date().toISOString();
15461 |
15462 |     function refreshCcView() {
15463 |       const vistaCc = document.getElementById('vista-cuenta-corriente');
15464 |       if (vistaCc && vistaCc.style.display !== 'none') loadCuentaCorriente();
15465 |       if (ccDetalleId && ccDetalleTipo) {
15466 |         fetchMovimientosCcPorEntidad(ccDetalleTipo, ccDetalleId).then(({ movimientos, saldos, ordenes, pendienteEnMoneda, pendienteClasePorMoneda }) => {
15467 |           ccDetalleMovimientosList = ccDetalleRowsConTipoOpDesdeOrdenes(movimientos, ordenes);
15468 |           ccDetalleOrdenesList = ordenes || [];
15469 |           renderCcDetalleTable();
15470 |           const saldosWrap = document.getElementById('modal-cc-detalle-saldos');
15471 |           if (saldosWrap && saldos) {
15472 |             const pendMonModal = pendienteEnMoneda || ccPendientePorMonedaDesdeMovs(movimientos);
15473 |             saldosWrap.innerHTML = htmlCcModalSaldosCards(saldos, pendMonModal, pendienteClasePorMoneda);
15474 |             reemplazarVisibilidadMonedasCuentaCorrienteDom();
15475 |           }
15476 |           renderCcDetalleOperaciones();
15477 |           const operacionesWrap = document.getElementById('modal-cc-detalle-operaciones-wrap');
15478 |           if (operacionesWrap) operacionesWrap.style.display = (ccDetalleOrdenesList.length > 0) ? 'block' : 'none';
15479 |         });
15480 |       }
15481 |     }
15482 |
15483 |     function hacerCierre(ordenIdFromSave) {
15484 |       closeModalTransaccion();
15485 |       refreshTransaccionesModal();
15486 |       const ordenId = ordenIdFromSave || transaccionesOrdenIdActual;
15487 |       if (ordenWizardInstrumentacionIdActual && instrumentacionId && ordenWizardInstrumentacionIdActual === instrumentacionId) {
15488 |         renderOrdenWizardInstrumentacion(instrumentacionId);
15489 |       }
15490 |       if (ordenId) {
15491 |         actualizarEstadoOrden(ordenId)
15492 |           .then((res) => {
15493 |             mostrarMensajeSiInstrumentacionCerrada(res);
15494 |             return res;
15495 |           })
15496 |           .then((res) => {
15497 |             if (res && res.estado === 'instrumentacion_cerrada_ejecucion') {
15498 |               return client.from('ordenes').select('intermediario_id, tipos_operacion(codigo, moneda_in, moneda_out, usa_intermediario)').eq('id', ordenId).single().then((rOrd
15499 | ) => {
15500 |               const orden = rOrd.data || {};
15501 |               const toSt = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
15502 |               const usaIntermediarioTipo = !(toSt && toSt.usa_intermediario);
15503 |               const miSt = (toSt && toSt.moneda_in || '').toString().toUpperCase().trim();
15504 |               const moSt = (toSt && toSt.moneda_out || '').toString().toUpperCase().trim();
15505 |               const esDosMonedasConIntermediario = !!patronTipoCambioOrden(miSt, moSt) && usaIntermediarioTipo;
15506 |               if (esDosMonedasConIntermediario) return res;
15507 |               // No generar transacciones automáticas Pandy-Intermediario (compensación/comisiones en instrumentación): va por CC y carga manual si aplica.
15508 |               return res;
15509 |             }
15510 |           });
15511 |     }
15512 |   });
15513 | }
15514 | return res;

```

```

15511 |     })
15512 |     .then((res) => {
15513 |       // Con la lógica nueva de CC no se generan movimientos "Conversión de moneda"
15514 |       return res;
15515 |     })
15516 |     .then(() => {
15517 |       if (ordenWizardInstrumentacionIdActual && instrumentacionId && ordenWizardInstrumentacionIdActual === instrumentacionId) renderOrdenWizardInstrumentacion(instrumen
15518 | tacionId);
15519 |       refreshCcView();
15520 |     });
15521 |   } else {
15522 |     refreshCcView();
15523 |   }
15524 | } // continuarCambioEstado
15525 |
15526 | client.from('instrumentacion').select('orden_id').eq('id', instrumentacionId).single().then((rOrd) => {
15527 |   const ordenId = rOrd.data && rOrd.data.orden_id;
15528 |   if (!ordenId) {
15529 |     hacerCierre();
15530 |     return;
15531 |   }
15532 |   client.from('ordenes').select('cliente_id, intermediario_id, moneda_recibida, monto_recibido, moneda_entregada, monto_entregado, numero, tasa_descuento_intermediario, ti
15533 | pos_operacion(codigo, moneda_in, moneda_out, usa_intermediario)').eq('id', ordenId).single().then((rO) => {
15534 |     const orden = rO.data || {};
15535 |     const clienteId = orden.cliente_id || null;
15536 |     const intermediarioId = orden.intermediario_id || null;
15537 |     const mr = Number(orden.monto_recibido) || 0;
15538 |     const me = Number(orden.monto_entregado) || 0;
15539 |     const monR = orden.moneda_recibida || 'USD';
15540 |     const monE = orden.moneda_entregada || 'USD';
15541 |     const ordenLabel = orden.numero != null ? 'nro orden ' + orden.numero : 'nro orden ' + (ordenId || '').toString().slice(0, 8);
15542 |     return client.from('comisiones_orden').select('monto').eq('orden_id', ordenId).eq('beneficiario', 'pandy').maybeSingle().then((rCom) => {
15543 |       const comisionPandyMonto = rCom.data != null ? Number(rCom.data.monto) : null;
15544 |       return { orden, clienteId, intermediarioId, mr, me, monR, monE, ordenLabel, ordenId, comisionPandyMonto };
15545 |     });
15546 |   }).then(({ orden, clienteId, intermediarioId, mr, me, monR, monE, ordenLabel, ordenId, comisionPandyMonto }) => {
15547 |     const esOrdenCheque = esTipoOperacionChequeArsDesdeJoin(orden.tipos_operacion);
15548 |     // Cuenta corriente: si estamos editando (id), revertir Cancelación y caja de esta transacción para re-aplicar con los nuevos valores.
15549 |     let promRevert = Promise.resolve();
15550 |     if (id && clienteId) {
15551 |       promRevert = Promise.all([
15552 |         client.from('movimientos_cuenta_corriente').select('id').eq('orden_id', ordenId).eq('cliente_id', clienteId).eq('transaccion_id', transaccionId).like('concepto', '
15553 | Cancelación%'),
15554 |         client.from('movimientos_cuenta_corriente').select('id').eq('orden_id', ordenId).eq('cliente_id', clienteId).eq('transaccion_id', transaccionId).like('concepto', '
15555 | Contraparte cancelación%'),
15556 |       ]).then(([rDel1, rDel2]) => {
15557 |         const ids = [...(rDel1.data || []), ...(rDel2.data || [])].map((x) => x.id);
15558 |         const delCc = ids.length > 0 ? Promise.all(ids.map((idDel) => client.from('movimientos_cuenta_corriente').delete().eq('id', idDel))) : Promise.resolve();
15559 |         return delCc.then(() => client.from('movimientos_caja').delete().eq('transaccion_id', transaccionId));
15560 |       });
15561 |     }
15562 |     promRevert.then(() => Promise.all([
15563 |       client.from('movimientos_cuenta_corriente').select('id, transaccion_id, concepto, monto_usd, monto_ars, monto_eur').eq('orden_id', ordenId).eq('cliente_id', clienteI
15564 | d),
15565 |       client.from('transacciones').select('id, tipo, monto, estado, cobrador, pagador').eq('instrumentacion_id', instrumentacionId),
15566 |     ])).then(([rMov, rTrxList]) => {
15567 |       const filasCc = rMov.data || [];
15568 |       const listTrx = rTrxList.data || [];
15569 |       const sumIngresosOthers = listTrx.filter((tr) => tr.tipo === 'ingreso' && tr.pagador === 'cliente' && tr.estado === 'ejecutada' && tr.id !== transaccionId).reduce((s
15570 | , tr) => s + Number(tr.monto), 0);
15571 |       const sumEgresosOthers = listTrx.filter((tr) => tr.tipo === 'egreso' && tr.cobrador === 'cliente' && tr.estado === 'ejecutada' && tr.id !== transaccionId).reduce((s
15572 | , tr) => s + Number(tr.monto), 0);
15573 |       const sumIngresosClienteEjecutados = sumIngresosOthers + (tipo === 'ingreso' && pagador === 'cliente' && estado === 'ejecutada' ? monto : 0);
15574 |       const sumEgresosClienteEjecutados = sumEgresosOthers + (tipo === 'egreso' && cobrador === 'cliente' && estado === 'ejecutada' ? monto : 0);
15575 |       const filasDeEstaTrx = filasCc.filter((m) => m.transaccion_id === transaccionId);
15576 |       const rowDebe = filasCc.find((r) => { const c = (r.concepto || '').toUpperCase(); return c.includes('DEBE') || c.includes('COMPROMISO DE PAGO'); });
15577 |       const rowComp = filasCc.find((r) => { const c = (r.concepto || '').normalize('NFD').replace(/u0301/g, '').toUpperCase(); return c.includes('COMPENSACION') || c.incl
15578 | udes('COMPROMISO A COBRAR'); });
15579 |       const tieneMomentoCero = !(rowDebe && rowComp && (rowDebe.monto_usd != null || rowDebe.monto_ars != null || rowDebe.monto_eur != null) && (rowComp.monto_usd != null
15580 | || rowComp.monto_ars != null || rowComp.monto_eur != null));
15581 |       const idsLegacy = filasDeEstaTrx.filter((m) => m.monto_usd == null && m.monto_ars == null && m.monto_eur == null).map((m) => m.id);
15582 |       const amountDebeMonR = rowDebe && (monR === 'USD' ? rowDebe.monto_usd : (monR === 'ARS' ? rowDebe.monto_ars : rowDebe.monto_eur));
15583 |       const amountCompMonE = rowComp && (monE === 'USD' ? rowComp.monto_usd : (monE === 'ARS' ? rowComp.monto_ars : rowComp.monto_eur));
15584 |       const splitIngreso = tieneMomentoCero && estado === 'ejecutada' && id && tipo === 'ingreso' && mr > 1e-6 && monto > 1e-6 && (mr - monto) > 1e-6 && (Math.abs(Number(a
15585 | mountDebeMonR) || 0) - monto) > 1e-6;
15586 |       const splitEgreso = tieneMomentoCero && estado === 'ejecutada' && id && tipo === 'egreso' && me > 1e-6 && monto > 1e-6 && (me - monto) > 1e-6 && (Math.abs(Number(am
15587 | untCompMonE) || 0) - monto) > 1e-6;
15588 |       const split = (splitIngreso || splitEgreso) && rowDebe && rowComp && clienteId;
15589 |       const faltaIngreso = mr - sumIngresosClienteEjecutados > 1e-6;
15590 |       const faltaEgreso = me - sumEgresosClienteEjecutados > 1e-6;
15591 |       const splitSinMomentoCero = !tieneMomentoCero && estado === 'ejecutada' && clienteId && (
15592 |         (tipo === 'ingreso' && pagador === 'cliente' && mr > 1e-6 && monto > 1e-6 && faltaIngreso) ||
15593 |         (tipo === 'egreso' && cobrador === 'cliente' && me > 1e-6 && monto > 1e-6 && faltaEgreso)
15594 |       );
15595 |       let promSplitSinMc = Promise.resolve(null);
15596 |       if (splitSinMomentoCero) {
15597 |         const diferencia = tipo === 'ingreso' ? (mr - sumIngresosClienteEjecutados) : (me - sumEgresosClienteEjecutados);
15598 |         if (diferencia >= 1e-6) {
15599 |           const ahoraIns = new Date().toISOString();
15600 |           promSplitSinMc = client.from('transacciones').insert({
15601 |             instrumentacion_id: instrumentacionId,
15602 |             tipo,
15603 |             modo_pago_id: modoPagoId,
15604 |             moneda: moneda || (tipo === 'ingreso' ? monR : monE),
15605 |             monto: diferencia,
15606 |             cobrador,
15607 |             pagador,

```

```

15597 |         owner: cobrador,
15598 |         estado: 'pendiente',
15599 |         concepto: concepto || '',
15600 |         tipo_cambio: tipoCambio || null,
15601 |         updated_at: ahoraIns,
15602 |     }).select('id').single().then((r) => r.data && r.data.id);
15603 | }
15604 | }
15605 |
15606 | let promSplit = Promise.resolve(null);
15607 | if (split && splitIngreso) {
15608 |     const restoEnFila = Math.abs(Number(amountDebeMonR) || 0);
15609 |     const diferencia = restoEnFila - monto;
15610 |     const restoTrxId = rowDebe.transaccion_id && rowDebe.transaccion_id !== id ? rowDebe.transaccion_id : null;
15611 |     if (restoTrxId && diferencia >= 1e-6) {
15612 |         promSplit = client.from('transacciones').update({ monto: diferencia, updated_at: new Date().toISOString() }).eq('id', restoTrxId).then(() => restoTrxId);
15613 |     } else if (!restoTrxId && diferencia >= 1e-6) {
15614 |         promSplit = client.from('transacciones').insert({
15615 |             instrumentacion_id: instrumentacionId, tipo: 'ingreso', modo_pago_id: modoPagoId, moneda: monto: diferencia,
15616 |             cobrador, pagador, owner: cobrador, estado: 'pendiente', concepto: concepto || '', tipo_cambio: tipoCambio, updated_at: new Date().toISOString(),
15617 |         }).select('id').single().then((r) => r.data && r.data.id);
15618 |     }
15619 | } else if (split && splitEgreso) {
15620 |     const restoEnFila = Math.abs(Number(amountCompMonE) || 0);
15621 |     const diferencia = restoEnFila - monto;
15622 |     const restoTrxIdEgr = rowComp.transaccion_id && rowComp.transaccion_id !== id ? rowComp.transaccion_id : null;
15623 |     if (restoTrxIdEgr && diferencia >= 1e-6) {
15624 |         promSplit = client.from('transacciones').update({ monto: diferencia, updated_at: new Date().toISOString() }).eq('id', restoTrxIdEgr).then(() => restoTrxIdEgr);
15625 |     } else if (!restoTrxIdEgr && diferencia >= 1e-6) {
15626 |         promSplit = client.from('transacciones').insert({
15627 |             instrumentacion_id: instrumentacionId, tipo: 'egreso', modo_pago_id: modoPagoId, moneda: monto: diferencia,
15628 |             cobrador, pagador, owner: cobrador, estado: 'pendiente', concepto: concepto || '', tipo_cambio: tipoCambio, updated_at: new Date().toISOString(),
15629 |         }).select('id').single().then((r) => r.data && r.data.id);
15630 |     }
15631 | }
15632 |
15633 | const deleteCliente = idsLegacy.length > 0
15634 |   ? Promise.all(idsLegacy.map(idDel => client.from('movimientos_cuenta_corriente').delete().eq('id', idDel)))
15635 |   : Promise.resolve();
15636 | // CC intermediario: misma regla que reversa. No borrar la fila momento cero (Debe); poner estado pendiente. Borrar solo cobros (monto > 0).
15637 | const ahoraRev = new Date().toISOString();
15638 | const revertCcInt = intermediarioId
15639 |   ? client.from('movimientos_cuenta_corriente_intermediario').select('id, monto, monto_usd, monto_ars, monto_eur').eq('orden_id', ordenId).eq('intermediario_id', int
15640 |   ermediarioId).eq('transaccion_id', transaccionId).then((rRows) => {
15641 |       const rows = rRows.data || [];
15642 |       const promises = rows.map(row => {
15643 |           const esMomentoCero = row.monto_usd !== null || row.monto_ars !== null || row.monto_eur !== null;
15644 |           if (esMomentoCero) return client.from('movimientos_cuenta_corriente_intermediario').update({ estado: 'pendiente', estado_fecha: ahoraRev }).eq('id', row.id);
15645 |           if (Number(row.monto) > 0) return client.from('movimientos_cuenta_corriente_intermediario').delete().eq('id', row.id);
15646 |           return Promise.resolve();
15647 |       });
15648 |       return Promise.all(promises);
15649 |   })
15650 |   : Promise.resolve();
15651 | return Promise.all([
15652 |     deleteCliente,
15653 |     revertCcInt,
15654 |     ]).then(() => Promise.all([promSplit, promSplitSinMc])).then(([newTrxId]) => ({ tieneMomentoCero, split, splitIngreso, splitEgreso, rowDebe, rowComp, newTrxId, comis
15655 | ionPandyMonto }));
15656 | }).then(({ tieneMomentoCero, split, splitIngreso, splitEgreso, rowDebe, rowComp, newTrxId, comisionPandyMonto }) => {
15657 |     const insertsCc = [];
15658 |     const updatesCc = [];
15659 |     if (split && newTrxId !== null) {
15660 |         if (splitIngreso) {
15661 |             const amountDebeMonRLocal = rowDebe && (monR === 'USD' ? rowDebe.monto_usd : (monR === 'ARS' ? rowDebe.monto_ars : rowDebe.monto_eur));
15662 |             const restoEnFilaIng = Math.abs(Number(amountDebeMonRLocal) || 0);
15663 |             const diferencia = restoEnFilaIng - monto;
15664 |             const ejecutadoMr = monto;
15665 |             const montoUsdCancel = numCc(monR === 'USD' ? monto : ratioCc(monto * me, mr, monto));
15666 |             const montoArsCancel = numCc(monR === 'ARS' ? monto : ratioCc(monto * me, mr, monto));
15667 |             const montoEurCancel = numCc(monR === 'EUR' ? monto : ratioCc(monto * me, mr, monto));
15668 |             const difUsd = monR === 'USD' ? -diferencia : (monE === 'USD' ? -ratioCc(me * diferencia, mr, diferencia) : 0);
15669 |             const difArs = monR === 'ARS' ? -diferencia : (monE === 'ARS' ? -ratioCc(me * diferencia, mr, diferencia) : 0);
15670 |             const difEur = monR === 'EUR' ? -diferencia : (monE === 'EUR' ? -ratioCc(me * diferencia, mr, diferencia) : 0);
15671 |             const remUsd = monR === 'USD' ? diferencia : (monE === 'USD' ? ratioCc(me * diferencia, mr, diferencia) : 0);
15672 |             const remArs = monR === 'ARS' ? diferencia : (monE === 'ARS' ? ratioCc(me * diferencia, mr, diferencia) : 0);
15673 |             const remEur = monR === 'EUR' ? diferencia : (monE === 'EUR' ? ratioCc(me * diferencia, mr, diferencia) : 0);
15674 |             insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
15675 |                 cliente_id: clienteId, orden_id: ordenId, transaccion_id: transaccionId,
15676 |                 concepto: 'Cancelación de deuda ' + ordenLabel,
15677 |                 fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
15678 |                 monto_usd: montoUsdCancel, monto_ars: montoArsCancel, monto_eur: montoEurCancel,
15679 |                 moneda: monR, monto: 0,
15680 |             }));
15681 |             updatesCc.push(client.from('movimientos_cuenta_corriente').update({
15682 |                 transaccion_id: newTrxId,
15683 |                 monto_usd: difUsd, monto_ars: difArs, monto_eur: difEur,
15684 |                 estado_fecha: ahora,
15685 |             }).eq('id', rowDebe.id));
15686 |             updatesCc.push(client.from('movimientos_cuenta_corriente').update({
15687 |                 monto_usd: remUsd, monto_ars: remArs, monto_eur: remEur,
15688 |                 estado_fecha: ahora,
15689 |             }).eq('id', rowComp.id));
15690 |         } else {
15691 |             const amountCompMonELocal = rowComp && (monE === 'USD' ? rowComp.monto_usd : (monE === 'ARS' ? rowComp.monto_ars : rowComp.monto_eur));
15692 |             const restoEnFilaEgr = Math.abs(Number(amountCompMonELocal) || 0);

```

```

15692|         const diferencia = restoEnFilaEgr - monto;
15693|         const ejecutadoMe = monto;
15694|         const ejecutadoMr = monR === monE ? monto : ratioCc(monto * mr, me, monto);
15695|         const montoUsdCancel = numCc(monR === 'USD' ? -ejecutadoMr : (monE === 'USD' ? -ejecutadoMe : 0));
15696|         const montoArsCancel = numCc(monR === 'ARS' ? -ejecutadoMr : (monE === 'ARS' ? -ejecutadoMe : 0));
15697|         const montoEurCancel = numCc(monR === 'EUR' ? -ejecutadoMr : (monE === 'EUR' ? -ejecutadoMe : 0));
15698|         // Egreso: diferencia está en monE; Debe (monR) debe llevar equivalente mr*diferencia/me
15699|         const difUsd = numCc(monR === 'USD' ? -ratioCc(mr * diferencia, me, diferencia) : (monE === 'USD' ? -diferencia : 0));
15700|         const difArs = numCc(monR === 'ARS' ? -ratioCc(mr * diferencia, me, diferencia) : (monE === 'ARS' ? -diferencia : 0));
15701|         const difEur = numCc(monR === 'EUR' ? -ratioCc(mr * diferencia, me, diferencia) : (monE === 'EUR' ? -diferencia : 0));
15702|         const remUsd = numCc(monR === 'USD' ? ratioCc(mr * diferencia, me, diferencia) : (monE === 'USD' ? diferencia : 0));
15703|         const remArs = numCc(monR === 'ARS' ? ratioCc(mr * diferencia, me, diferencia) : (monE === 'ARS' ? diferencia : 0));
15704|         const remEur = numCc(monR === 'EUR' ? ratioCc(mr * diferencia, me, diferencia) : (monE === 'EUR' ? diferencia : 0));
15705|         insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
15706|             cliente_id: clienteId, orden_id: ordenId, transaccion_id: transaccionId,
15707|             concepto: 'Cancelación de deuda ' + ordenLabel,
15708|             fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
15709|             monto_usd: montoUsdCancel, monto_ars: montoArsCancel, monto_eur: montoEurCancel,
15710|             moneda: monE, monto: 0,
15711|         }));
15712|         updatesCc.push(client.from('movimientos_cuenta_corriente').update({
15713|             transaccion_id: newTrxId,
15714|             monto_usd: remUsd, monto_ars: remArs, monto_eur: remEur,
15715|             estado_fecha: ahora,
15716|         }).eq('id', rowComp.id));
15717|         updatesCc.push(client.from('movimientos_cuenta_corriente').update({
15718|             monto_usd: difUsd, monto_ars: difArs, monto_eur: difEur,
15719|             estado_fecha: ahora,
15720|         }).eq('id', rowDebe.id));
15721|     }
15722| } else if (!tieneMomentoCero) {
15723|     const montosCobro = montosCcPorMoneda(moneda || 'USD', monto);
15724|     const montosDeuda = montosCcPorMoneda(moneda || 'USD', -monto);
15725|     // No registrar "Cobro por" en CC cliente cuando es la comisión/ganancia de Pandy (ej. ARS-ARS CHEQUE): el cliente ya la pagó, no es deuda.
15726|     const esComisionPandy = pagador === 'cliente' && cobrador === 'pandy' && intermediarioId && comisionPandyMonto != null && Math.abs(monto - comisionPandyMonto) < 1e
-6;
15727|     if (pagador === 'cliente' && clienteId && !esComisionPandy) {
15728|         if (monR !== monE && (tipo || '').toLowerCase() === 'ingreso') {
15729|             const enMonEVal = ratioCc(monto * me, mr, monto);
15730|             const cancelacion = { monto_usd: numCc(monR === 'USD' ? monto : 0), monto_ars: numCc(monR === 'ARS' ? monto : 0), monto_eur: numCc(monR === 'EUR' ? monto : 0)
15731| };
15732|             const contraparte = { monto_usd: numCc(monR === 'USD' ? -monto : (monE === 'USD' ? -enMonEVal : 0)), monto_ars: numCc(monR === 'ARS' ? -monto : (monE === 'ARS'
? -enMonEVal : 0)), monto_eur: numCc(monR === 'EUR' ? -monto : (monE === 'EUR' ? -enMonEVal : 0)) };
15733|             insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
15734|                 cliente_id: clienteId, orden_id: ordenId, transaccion_id: transaccionId,
15735|                 concepto: 'Cancelación de deuda ' + ordenLabel,
15736|                 fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
15737|                 monto_usd: cancelacion.monto_usd, monto_ars: cancelacion.monto_ars, monto_eur: cancelacion.monto_eur, moneda: monR, monto: 0,
15738|             }));
15739|             insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
15740|                 cliente_id: clienteId, orden_id: ordenId, transaccion_id: transaccionId,
15741|                 concepto: 'Contraparte cancelación ' + ordenLabel,
15742|                 fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
15743|                 monto_usd: contraparte.monto_usd, monto_ars: contraparte.monto_ars, monto_eur: contraparte.monto_eur, moneda: monE, monto: 0,
15744|             }));
15745|         } else {
15746|             insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
15747|                 cliente_id: clienteId, moneda: monto, orden_id: ordenId, transaccion_id: transaccionId,
15748|                 concepto: conceptoCcMovimiento(moneda, monto, 'cobro'), fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
15749|                 ...montosCobro,
15750|             }));
15751|         }
15752|     }
15753|     if (cobrador === 'cliente' && clienteId) {
15754|         insertsCc.push(client.from('movimientos_cuenta_corriente').insert({
15755|             cliente_id: clienteId, moneda: monto, orden_id: ordenId, transaccion_id: transaccionId,
15756|             concepto: conceptoCcMovimiento(moneda, monto, 'deuda'), fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
15757|             ...montosDeuda,
15758|         }));
15759|     }
15760| }
15761| const esPandyIntermediario = (cobrador === 'pandy' && pagador === 'intermediario') || (cobrador === 'intermediario' && pagador === 'pandy');
15762| if (esPandyIntermediario && pagador === 'intermediario' && intermediarioId) {
15763|     insertsCc.push(
15764|         client.from('movimientos_cuenta_corriente_intermediario').update({ estado: 'cerrado', estado_fecha: ahora })
15765|         .eq('orden_id', ordenId).eq('intermediario_id', intermediarioId).eq('transaccion_id', transaccionId).eq('estado', 'pendiente')
15766|         .then(() => client.from('movimientos_cuenta_corriente_intermediario').insert({
15767|             intermediario_id: intermediarioId, moneda: monto, orden_id: ordenId, transaccion_id: transaccionId,
15768|             concepto: conceptoCcMovimiento(moneda, monto, 'cobro'), fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
15769|             ...montosCcPorMoneda(moneda || 'USD', monto),
15770|         }));
15771|     );
15772| }
15773| if (esPandyIntermediario && cobrador === 'intermediario' && intermediarioId) {
15774|     const tasa = Number(orden.tasa_descuento_intermediario) || 0;
15775|     const montoEfectivoInt = (typeof tasa === 'number' && !isNaN(tasa) && tasa >= 0 && tasa < 1) ? mr * (1 - tasa) : mr;
15776|     const monInt = orden.moneda_recibida || moneda || 'ARS';
15777|     insertsCc.push(
15778|         client.from('movimientos_cuenta_corriente_intermediario').select('id').eq('orden_id', ordenId).eq('intermediario_id', intermediarioId).eq('transaccion_id', trans
accionId).maybeSingle()
15779|         .then((r) => {
15780|             if (r.data && r.data.id)
15781|                 return client.from('movimientos_cuenta_corriente_intermediario').update({ estado: 'cerrado', estado_fecha: ahora }).eq('id', r.data.id);
15782|             return Promise.resolve();
15783|         })
15784|         .then(() => client.from('movimientos_cuenta_corriente_intermediario').insert({
15785|             intermediario_id: intermediarioId, orden_id: ordenId, transaccion_id: transaccionId, transaccion_numero: transaccionNumero != null ? transaccionNumero : null

```

```

15785 |         moneda: monInt, monto: -montoEfectivoInt,
15786 |         concepto: conceptoCcLeyenda('compromiso_cobrar', orden && orden.numero != null ? orden.numero : null, transaccionNumero), fecha, usuario_id: currentUserId, e
stado: 'cerrado', estado_fecha: ahora,
15787 |         ...montosCcPorMoneda(monInt, -montoEfectivoInt),
15788 |     })
15789 |     .then(() => client.from('comisiones_orden').select('moneda, monto').eq('orden_id', ordenId).eq('beneficiario', 'intermediario').maybeSingle())
15790 |     .then((rCom) => {
15791 |         const comMonto = rCom.data && (Number(rCom.data.monto) || 0);
15792 |         if (comMonto >= 1e-6) {
15793 |             const monCom = (rCom.data.moneda || 'ARS').toUpperCase();
15794 |             return client.from('movimientos_cuenta_corriente_intermediario').insert({
15795 |                 intermediario_id: intermediarioId, orden_id: ordenId, transaccion_id: transaccionId, transaccion_numero: transaccionNumero != null ? transaccionNumero :
null,
15796 |                 moneda: monCom, monto: -comMonto,
15797 |                 concepto: conceptoCcLeyenda('comision_acuerdo', orden && orden.numero != null ? orden.numero : null, transaccionNumero), fecha, usuario_id: currentUserId
, estado: 'cerrado', estado_fecha: ahora,
15798 |                 ...montosCcPorMoneda(monCom, -comMonto),
15799 |             }).then(() => asegurarComisionIntermediario(ordenId, instrumentacionId, intermediarioId, comMonto, monCom, orden && orden.numero != null ? orden.numero : n
ull, transaccionNumero != null ? transaccionNumero : null));
15800 |         }
15801 |         return Promise.resolve();
15802 |     })
15803 |     );
15804 | }
15805 | const promUpdatesCc = (updatesCc && updatesCc.length > 0) ? Promise.all(updatesCc) : Promise.resolve();
15806 | Promise.all([insertsCc].then(() => promUpdatesCc).then(() => {
15807 |     if (estado !== 'ejecutada') {
15808 |         let promReversa = Promise.resolve();
15809 |         if (cobrador === 'pandy' && pagador === 'intermediario' && intermediarioId) {
15810 |             promReversa = revertirComisionIntermediario(ordenId).then(() =>
15811 |                 client.from('movimientos_cuenta_corriente_intermediario').select('id, concepto').eq('orden_id', ordenId).eq('intermediario_id', intermediarioId).eq('transacc
ion_id', transaccionId).then((rRows) => {
15812 |                     const rows = rRows.data || [];
15813 |                     const idsBorrar = rows.filter((r) => (r.concepto || '').includes('Compromiso a Cobrar') || (r.concepto || '').includes('Comisión del acuerdo')).map((r) =>
r.id);
15814 |                     const rowDebe = rows.find((r) => (r.concepto || '').toLowerCase().includes('debe'));
15815 |                     const del = idsBorrar.length > 0 ? Promise.all(idsBorrar.map((id) => client.from('movimientos_cuenta_corriente_intermediario').delete().eq('id', id))) : Pr
omise.resolve();
15816 |                     const monR = orden.moneda_recibida || 'ARS';
15817 |                     const montoUsd = monR === 'USD' ? -mr : 0;
15818 |                     const montoArs = monR === 'ARS' ? -mr : 0;
15819 |                     const montoEur = monR === 'EUR' ? -mr : 0;
15820 |                     const upd = rowDebe && rowDebe.id ? client.from('movimientos_cuenta_corriente_intermediario').update({ estado: 'pendiente', estado_fecha: ahora, monto_usd:
montoUsd, monto_ars: montoArs, monto_eur: montoEur }).eq('id', rowDebe.id) : Promise.resolve();
15821 |                     return del.then(() => upd);
15822 |                 })
15823 |             );
15824 |         }
15825 |         if (cobrador === 'intermediario' && pagador === 'pandy' && intermediarioId) {
15826 |             promReversa = promReversa.then(() =>
15827 |                 client.from('movimientos_cuenta_corriente_intermediario').select('id, concepto').eq('orden_id', ordenId).eq('intermediario_id', intermediarioId).eq('transacc
ion_id', transaccionId).then((rRows) => {
15828 |                     const rows = rRows.data || [];
15829 |                     const idsBorrar = rows.filter((r) => (r.concepto || '').includes('Cobro') || (r.concepto || '').includes('Descuento')).map((r) => r.id);
15830 |                     const idCompensacion = rows.find((r) => (r.concepto || '').toLowerCase().includes('compensacion'));
15831 |                     const del = idsBorrar.length > 0 ? Promise.all(idsBorrar.map((id) => client.from('movimientos_cuenta_corriente_intermediario').delete().eq('id', id))) : Pr
omise.resolve();
15832 |                     const upd = idCompensacion ? client.from('movimientos_cuenta_corriente_intermediario').update({ estado: 'pendiente', estado_fecha: ahora }).eq('id', idComp
ensacion.id) : Promise.resolve();
15833 |                     return del.then(() => upd);
15834 |                 })
15835 |             );
15836 |         }
15837 |         // Revertir Ganancia del acuerdo si esta transacción es ingreso cliente-Pandy y había comisión (misma moneda mr > me o comisiones_orden).
15838 |         const esIngresoClientePandy = (tipo || '').toLowerCase() === 'ingreso' && pagador === 'cliente' && cobrador === 'pandy';
15839 |         const comisionPandyReversa = (monR === monE && mr > me) ? mr - me : (comisionPandyMonto || 0);
15840 |         if (esIngresoClientePandy && clienteId && comisionPandyReversa >= 1e-6) {
15841 |             promReversa = promReversa.then(() => revertirGananciaPandy(ordenId, orden, clienteId, comisionPandyReversa));
15842 |         }
15843 |         promReversa.then(() => { hacerCierre(ordenId); });
15844 |         return;
15845 |     }
15846 |     // La caja es siempre la de Pandy: solo impactamos caja cuando Pandy es cobrador o pagador.
15847 |     const pandyParticipa = cobrador === 'pandy' || pagador === 'pandy';
15848 |     if (!pandyParticipa) {
15849 |         hacerCierre(ordenId);
15850 |         loadCajas();
15851 |         return;
15852 |     }
15853 |     client.from('movimientos_caja').select('id').eq('transaccion_id', transaccionId).limit(1).then((r) => {
15854 |         if (r.data && r.data.length > 0) {
15855 |             hacerCierre(ordenId);
15856 |             loadCajas();
15857 |             return;
15858 |         }
15859 |         const promNroTrx = transaccionNumero != null ? Promise.resolve(transaccionNumero) : client.from('transacciones').select('numero').eq('id', transaccionId).single(
).then((rr) => rr.data && rr.data.numero);
15860 |         const promNroTrx.then((nroTrx) => {
15861 |             const cajaTipo = codigoCajaTipo(modopagoId);
15862 |             const signo = cobrador === 'pandy' ? 1 : -1; // Pandy cobra = ingreso; Pandy paga = egreso
15863 |             function insertarMovCajaTrasValidarSaldo() {
15864 |                 const conceptoMov = conceptoCajaTransaccion(cobrador === 'pandy', moneda, monto, orden.numero, nroTrx);
15865 |                 const movCaja = {
15866 |                     moneda, monto: signo * monto, caja_tipo: cajaTipo, transaccion_id: transaccionId,
15867 |                     orden_numero: orden.numero != null ? orden.numero : null, transaccion_numero: nroTrx != null ? nroTrx : null,
15868 |                     concepto: conceptoMov, fecha, usuario_id: currentUserId,

```



```

15869 | };
15870 | client.from('movimientos_caja').insert(movCaja).then((rCaja) => {
15871 |   if (rCaja.error) {
15872 |     showToast('Error al crear movimiento de caja: ' + (rCaja.error.message || ''), 'error');
15873 |     hacerCierre(ordenId);
15874 |     loadCajas();
15875 |     return;
15876 |   }
15877 |   if (!esOrdenCheque || !intermediarioId || !comisionPandyMonto || comisionPandyMonto < 1e-6) {
15878 |     hacerCierre(ordenId);
15879 |     loadCajas();
15880 |     return;
15881 |   }
15882 |   client.from('transacciones').select('id, tipo, monto, estado, cobrador, pagador, concepto').eq('instrumentacion_id', instrumentacionId).then((rList) => {
15883 |     const list = rList.data || [];
15884 |     const sumIngCli = list.filter((tr) => tr.tipo === 'ingreso' && tr.pagador === 'cliente' && tr.estado === 'ejecutada').reduce((s, tr) => s + Number(tr.mon
to), 0);
15885 |     const sumEgrCli = list.filter((tr) => tr.tipo === 'egreso' && tr.cobrador === 'cliente' && tr.estado === 'ejecutada').reduce((s, tr) => s + Number(tr.mon
to), 0);
15886 |     if (sumIngCli < mr - 1e-6 || sumEgrCli < me - 1e-6) {
15887 |       hacerCierre(ordenId);
15888 |       loadCajas();
15889 |       return;
15890 |     }
15891 |     hacerCierre(ordenId);
15892 |     loadCajas();
15893 |   });
15894 | });
15895 | }
15896 | if (estado === 'ejecutada' && signo < 0) {
15897 |   const tSynth = {
15898 |     tipo,
15899 |     moneda,
15900 |     monto,
15901 |     pagador: pagador,
15902 |     cobrador: cobrador,
15903 |     modo_pago_id: modoPagoId,
15904 |     concepto,
15905 |   };
15906 |   Promise.all([
15907 |     client.from('instrumentacion').select('multicontraparte_manual').eq('id', instrumentacionId).single(),
15908 |     fetchMovimientosCajaCerradosSinSync(),
15909 |     client.from('modos_pago').select('codigo').eq('id', modoPagoId).maybeSingle(),
15910 |   ]).then(([rInstMc, listCaja, rModo]) => {
15911 |     const multicontraparteManual = !!rInstMc.data && rInstMc.data.multicontraparte_manual;
15912 |     const toJoin = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
15913 |     const codigoModo = (rModo.data && rModo.data.codigo) ? String(rModo.data.codigo) : 'efectivo';
15914 |     const proy = proyectarEgresoCajaEmpresaSiEjecutadaTransaccion(tSynth, {
15915 |       multicontraparteManual,
15916 |       orden,
15917 |       tipoOperacionJoin: toJoin,
15918 |       comisionPandyMonto,
15919 |       codigoModo,
15920 |     });
15921 |     if (proy) {
15922 |       const v = validarSaldoCajaParaEgresoLista(listCaja, proy);
15923 |       if (!v.ok) {
15924 |         showToast(v.mensaje, 'error');
15925 |         hacerCierre(ordenId);
15926 |         loadCajas();
15927 |         return;
15928 |       }
15929 |     }
15930 |     insertarMovCajaTrasValidarSaldo();
15931 |   });
15932 |   return;
15933 | }
15934 | insertarMovCajaTrasValidarSaldo();
15935 | });
15936 | });
15937 | });
15938 | });
15939 | });
15940 | }
15941 | }
15942 | (montoCompensatorio ? insertarCompensatoria() : Promise.resolve()).then(continuarFlujo);
15943 | });
15944 | }
15945 | }
15946 |
15947 | function refreshTransaccionesModal() {
15948 |   const ordenId = transaccionesOrdenIdActual;
15949 |   if (ordenId) refreshTransaccionesPanel(ordenId);
15950 | }
15951 |
15952 | function closeModalTransacciones() {
15953 |   const backdrop = document.getElementById('modal-transacciones-backdrop');
15954 |   if (backdrop) backdrop.classList.remove('activo');
15955 |   transaccionesOrdenIdActual = null;
15956 | }
15957 |
15958 | function mostrarMensajeSiInstrumentacionCerrada(res) {
15959 |   if (res && (res.estado === 'orden ejecutada' || res.estado === 'instrumentacion cerrada ejecucion')) {
15960 |     if (res.estado === 'orden ejecutada') showToast('La orden ha quedado ejecutada.', 'success');
15961 |     else showToast('La instrumentación ha quedado cerrada. Orden en ejecución. Quedan transacciones por ejecutar.', 'info');
15962 |   }
15963 | }

```

```

15964 |
15965 | /**
15966 | * Calcula el neto por moneda que el intermediario desembolsó al cliente (egresos inter-cliente menos ingresos inter-cliente).
15967 | * Solo considera transacciones donde participa el cliente. Si el neto es positivo, Pandy debe compensar al intermediario.
15968 | */
15969 | function netoIntermediarioClientePorMoneda(transacciones) {
15970 |     const neto = {};
15971 |     (transacciones || []).forEach((t) => {
15972 |         if (t.cobrador !== 'cliente' && t.pagador !== 'cliente') return;
15973 |         const monto = Number(t.monto) || 0;
15974 |         const moneda = t.moneda || 'USD';
15975 |         if (!neto[moneda]) neto[moneda] = 0;
15976 |         if (t.tipo === 'egreso' && t.cobrador === 'cliente' && t.pagador === 'intermediario') neto[moneda] += monto;
15977 |         else if (t.tipo === 'ingreso' && t.cobrador === 'intermediario' && t.pagador === 'cliente') neto[moneda] -= monto;
15978 |     });
15979 |     return neto;
15980 | }
15981 |
15982 | /**
15983 | * Desactivado: no se crean transacciones automáticas de compensación Pandy*Intermediario.
15984 | * El acuerdo instrumentado es con el cliente; la relación con el intermediario se lleva en cuenta corriente.
15985 | */
15986 | function generarTransaccionesCompensacionPandyIntermediario(ordenId, instrumentacionId) {
15987 |     void ordenId;
15988 |     void instrumentacionId;
15989 |     return Promise.resolve();
15990 | }
15991 |
15992 | /**
15993 | * Genera movimientos en la cuenta corriente del cliente para que totalicen el acuerdo y queden en cero.
15994 | * - En la moneda del acuerdo (ej. USD): conversión de moneda por el faltante hasta monto_recibido (ej. 25.000 USD) + movimiento Comisión del acuerdo (Haber) por (monto_recibi
do - monto_entregado) para saldar.
15995 | * - En otras monedas: conversión de moneda por -saldo para anular.
15996 | * Se ejecuta una sola vez por orden (comprueba si ya existe movimiento de conversión).
15997 | */
15998 | function generarMovimientoConversionCc(ordenId) {
15999 |     if (!ordenId || !currentUser) return Promise.resolve();
16000 |     return client.from('ordenes').select('id, cliente_id, moneda_recibida, monto_recibido, moneda_entregada, monto_entregado, cotizacion').eq('id', ordenId).single().then((rOrd)
=> {
16001 |         const orden = rOrd.data;
16002 |         if (!orden || !orden.cliente_id) return;
16003 |         const clienteId = orden.cliente_id;
16004 |         const monedaRecibida = orden.moneda_recibida || 'USD';
16005 |         const monedaEntregada = orden.moneda_entregada || 'USD';
16006 |         const montoRecibidoOrden = Number(orden.monto_recibido) || 0;
16007 |         const montoEntregadoOrden = Number(orden.monto_entregado) || 0;
16008 |         const cotizacion = Number(orden.cotizacion) || 0;
16009 |         const mismaMoneda = monedaRecibida === monedaEntregada;
16010 |         const comisionOrden = mismaMoneda ? montoRecibidoOrden - montoEntregadoOrden : 0;
16011 |         let comisionUsd = null;
16012 |         if (!mismaMoneda && cotizacion > 1e-6) {
16013 |             if (monedaRecibida === 'ARS' && monedaEntregada === 'USD') comisionUsd = (montoRecibidoOrden / cotizacion) - montoEntregadoOrden;
16014 |             else if (monedaRecibida === 'USD' && monedaEntregada === 'ARS') comisionUsd = montoRecibidoOrden - (montoEntregadoOrden / cotizacion);
16015 |         }
16016 |         return client.from('instrumentacion').select('id').eq('orden_id', ordenId).maybeSingle().then((rInst) => {
16017 |             const instId = rInst.data && rInst.data.id;
16018 |             if (!instId) return;
16019 |             return client.from('movimientos_cuenta_corriente').select('id, concepto').eq('orden_id', ordenId).in('concepto', CONCEPTOS_CC_AUTOGENERADOS).then((rExist) => {
16020 |                 const existentes = rExist.data || [];
16021 |                 const tieneComision = existentes.some((m) => CONCEPTOS_CC_COMISION TODOS.includes(m.concepto));
16022 |                 if (tieneComision && existentes.length > 0) return;
16023 |                 const idsBorrar = existentes.map((m) => m.id);
16024 |                 const promBorrar = idsBorrar.length > 0 ? client.from('movimientos_cuenta_corriente').delete().in('id', idsBorrar) : Promise.resolve();
16025 |                 return promBorrar.then(() => client.from('transacciones').select('id').eq('instrumentacion_id', instId).eq('estado', 'ejecutada').then((rTr) => {
16026 |                     const trIds = (rTr.data || []).map((t) => t.id);
16027 |                     if (trIds.length === 0) return;
16028 |                     return client.from('movimientos_cuenta_corriente').select('moneda, monto').eq('cliente_id', clienteId).in('transaccion_id', trIds).then((rMov) => {
16029 |                         const totalDebe = { USD: 0, EUR: 0, ARS: 0 };
16030 |                         const totalHaber = { USD: 0, EUR: 0, ARS: 0 };
16031 |                         (rMov.data || []).forEach((m) => {
16032 |                             const mon = m.moneda;
16033 |                             const n = Number(m.monto);
16034 |                             if (totalDebe[mon] != null) {
16035 |                                 if (n > 0) totalDebe[mon] += n;
16036 |                                 else totalHaber[mon] += -n;
16037 |                             }
16038 |                         });
16039 |                         const inserts = [];
16040 |                         const fecha = fechaHoyYYYYMMDDArgentina();
16041 |                         const ahora = new Date().toISOString();
16042 |
16043 |                         if (montoRecibidoOrden > 0 && monedaRecibida) {
16044 |                             const recibidoEnMoneda = totalDebe[monedaRecibida] || 0;
16045 |                             const conversion = montoRecibidoOrden - recibidoEnMoneda;
16046 |                             if (conversion > 1e-6) {
16047 |                                 inserts.push(client.from('movimientos_cuenta_corriente').insert({
16048 |                                     cliente_id: clienteId, moneda: monedaRecibida, monto: conversion, orden_id: ordenId, transaccion_id: null,
16049 |                                     concepto: CONCEPTO_CC_CONVERSION, fecha, usuario_id: currentUser, estado: 'cerrado', estado_fecha: ahora,
16050 |                                     ...montosCcPorMoneda(monedaRecibida, conversion),
16051 |                                 }));
16052 |                             }
16053 |                             // Comisión en CC cliente para que la cuenta cierre: ARS-USD / USD-ARS / USD-USD en moneda correspondiente (Haber)
16054 |                             if (comisionUsd != null && comisionUsd > 1e-6) {
16055 |                                 if (!mismaMoneda && monedaRecibida === 'ARS' && monedaEntregada === 'USD') {
16056 |                                     inserts.push(client.from('movimientos_cuenta_corriente').insert({
16057 |                                         cliente_id: clienteId, moneda: 'USD', monto: -comisionUsd, orden_id: ordenId, transaccion_id: null,
16058 |                                         concepto: CONCEPTO_CC_COMISION, fecha, usuario_id: currentUser, estado: 'cerrado', estado_fecha: ahora,

```

```

16059 |         ...montosCcPorMoneda('USD', -comisionUsd),
16060 |     });
16061 | } else if (!mismaMoneda && monedaRecibida === 'USD' && monedaEntregada === 'ARS' && cotizacion > 1e-6) {
16062 |     // USD-ARS: comisión en ARS (Haber) para que cierre el saldo ARS: Debe = recibido*TC, Haber = entregado + comisión
16063 |     const comisionArs = comisionUsd * cotizacion;
16064 |     inserts.push(client.from('movimientos_cuenta_corriente').insert({
16065 |         cliente_id: clienteId, moneda: 'ARS', monto: -comisionArs, orden_id: ordenId, transaccion_id: null,
16066 |         concepto: CONCEPTO_CC_COMISION, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
16067 |         ...montosCcPorMoneda('ARS', -comisionArs),
16068 |     }));
16069 | } else {
16070 |     inserts.push(client.from('movimientos_cuenta_corriente').insert({
16071 |         cliente_id: clienteId, moneda: 'USD', monto: -comisionUsd, orden_id: ordenId, transaccion_id: null,
16072 |         concepto: CONCEPTO_CC_COMISION, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
16073 |         ...montosCcPorMoneda('USD', -comisionUsd),
16074 |     }));
16075 | }
16076 | } else if (comisionOrden > 1e-6) {
16077 |     inserts.push(client.from('movimientos_cuenta_corriente').insert({
16078 |         cliente_id: clienteId, moneda: monedaRecibida, monto: -comisionOrden, orden_id: ordenId, transaccion_id: null,
16079 |         concepto: CONCEPTO_CC_COMISION, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
16080 |         ...montosCcPorMoneda(monedaRecibida, -comisionOrden),
16081 |     }));
16082 | }
16083 | }
16084 |
16085 | ['USD', 'EUR', 'ARS'].forEach((moneda) => {
16086 |     const saldo = (totalDebe[moneda] || 0) - (totalHaber[moneda] || 0);
16087 |     let montoConversion = null;
16088 |     if (moneda === monedaRecibida) {
16089 |         // USD-ARS: en moneda recibida (USD) falta el Haber que compense el Debe; agregamos Conversión -montoRecibidoOrden
16090 |         if (!mismaMoneda && montoRecibidoOrden > 1e-6) {
16091 |             montoConversion = -montoRecibidoOrden;
16092 |         } else {
16093 |             return;
16094 |         }
16095 |     } else {
16096 |         montoConversion = -saldo;
16097 |         // En órdenes mixtas, en la moneda entregada usamos el equivalente de lo recibido para saldar
16098 |         if (moneda === monedaEntregada && !mismaMoneda && cotizacion > 1e-6) {
16099 |             if (monedaRecibida === 'ARS' && monedaEntregada === 'USD') montoConversion = montoRecibidoOrden / cotizacion;
16100 |             else if (monedaRecibida === 'USD' && monedaEntregada === 'ARS') montoConversion = montoRecibidoOrden * cotizacion;
16101 |         }
16102 |     }
16103 |     if (montoConversion == null || Math.abs(montoConversion) < 1e-6) return;
16104 |     inserts.push(client.from('movimientos_cuenta_corriente').insert({
16105 |         cliente_id: clienteId, moneda: moneda, monto: montoConversion, orden_id: ordenId, transaccion_id: null,
16106 |         concepto: CONCEPTO_CC_CONVERSION, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
16107 |         ...montosCcPorMoneda(moneda, montoConversion),
16108 |     }));
16109 | });
16110 |
16111 | if (inserts.length === 0) return;
16112 | return Promise.all(inserts).then(() => {});
16113 | });
16114 | });
16115 | });
16116 | });
16117 | });
16118 | }
16119 |
16120 | /**
16121 |  * Genera movimientos en la cuenta corriente del intermediario para saldar por orden ejecutada:
16122 |  * - "Conversión de moneda" por monto entregado (Debe).
16123 |  * - "Comisión del acuerdo" por la comisión con beneficiario=intermediario (Debe), para que el pago de comisión (Haber) quede saldado.
16124 |  * Si no hay comisiones_orden para intermediario, se usa un solo movimiento Conversión por el total (-saldo).
16125 |  * Siempre se recalculan y se reemplazan los movimientos existentes de esta orden para incluir transacciones nuevas (ej. comisión).
16126 |  */
16127 | function generarMovimientoConversionCcIntermediario(ordenId) {
16128 |     if (!ordenId || !currentUserId) return Promise.resolve();
16129 |     return client.from('ordenes').select('id, intermediario_id, moneda_entregada, monto_entregado').eq('id', ordenId).single().then((rOrd) => {
16130 |         const orden = rOrd.data;
16131 |         if (!orden || !orden.intermediario_id) return Promise.resolve();
16132 |         const intermediarioId = orden.intermediario_id;
16133 |         const monedaEntregada = orden.moneda_entregada || 'USD';
16134 |         const montoEntregado = Number(orden.monto_entregado) || 0;
16135 |         return client.from('instrumentacion').select('id').eq('orden_id', ordenId).maybeSingle().then((rInst) => {
16136 |             const instId = rInst.data && rInst.data.id;
16137 |             if (!instId) return Promise.resolve();
16138 |             const idsBorrar = client.from('movimientos_cuenta_corriente_intermediario').select('id').eq('orden_id', ordenId).eq('intermediario_id', intermediarioId).in('concepto', CONCEPTOS_CC_AUTOGENERADOS);
16139 |             return idsBorrar.then((rDel) => {
16140 |                 const ids = (rDel.data || []).map((m) => m.id);
16141 |                 const promBorrar = ids.length > 0 ? client.from('movimientos_cuenta_corriente_intermediario').delete().in('id', ids) : Promise.resolve();
16142 |                 return promBorrar.then(() => client.from('transacciones').select('id, cobrador, pagador').eq('instrumentacion_id', instId).eq('estado', 'ejecutada').then((rTr) => {
16143 |                     const list = (rTr.data || []).filter((t) => (t.cobrador === 'pandy' && t.pagador === 'intermediario') || (t.cobrador === 'intermediario' && t.pagador === 'pandy'));
16144 |                     const trIds = list.map((t) => t.id);
16145 |                     if (trIds.length === 0) return Promise.resolve();
16146 |                     return Promise.all([
16147 |                         client.from('movimientos_cuenta_corriente_intermediario').select('moneda, monto').eq('intermediario_id', intermediarioId).in('transaccion_id', trIds),
16148 |                         client.from('comisiones_orden').select('moneda, monto').eq('orden_id', ordenId).eq('beneficiario', 'intermediario'),
16149 |                     ]).then(([{rMov, rCom}]) => {
16150 |                         const saldos = { USD: 0, EUR: 0, ARS: 0 };
16151 |                         (rMov.data || []).forEach((m) => {
16152 |                             if (saldos[m.moneda] != null) saldos[m.moneda] += Number(m.monto);
16153 |                         });
16154 |                         const comisionPorMoneda = { USD: 0, EUR: 0, ARS: 0 };

```

```

16155 | (rCom.data || []).forEach((c) => {
16156 |   if (comisionPorMoneda[c.moneda] !== null) comisionPorMoneda[c.moneda] += Number(c.monto) || 0;
16157 | });
16158 | const fecha = fechaHoyYYYYMMDDArgentina();
16159 | const ahora = new Date().toISOString();
16160 | const inserts = [];
16161 | const tieneComisionInter = ['USD', 'EUR', 'ARS'].some((mon) => comisionPorMoneda[mon] > 1e-6);
16162 | if (tieneComisionInter && montoEntregado > 1e-6 && monedaEntregada) {
16163 |   const haberEnMonedaEntregada = Math.max(0, -(saldos[monedaEntregada] || 0));
16164 |   const totalDebePlaneado = montoEntregado + (comisionPorMoneda[monedaEntregada] || 0);
16165 |   const totalDebeACrear = totalDebePlaneado <= 1e-6 ? 0 : Math.min(totalDebePlaneado, haberEnMonedaEntregada);
16166 |   if (totalDebeACrear > 1e-6) {
16167 |     const estamosCap = totalDebeACrear < totalDebePlaneado - 1e-6;
16168 |     if (estamosCap) {
16169 |       // Un solo movimiento: comisión a cobrar (Debe) concilia con comisión cobrada (Haber)
16170 |       inserts.push(client.from('movimientos_cuenta_corriente_intermediario').insert({
16171 |         intermediario_id: intermediarioId, moneda: monedaEntregada, monto: totalDebeACrear, orden_id: ordenId, transaccion_id: null,
16172 |         concepto: CONCEPTO_CC_COMISION, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
16173 |       }));
16174 |     } else {
16175 |       const convMonto = Math.round(montoEntregado * 1e4) / 1e4;
16176 |       const comMonto = Math.round((comisionPorMoneda[monedaEntregada] || 0) * 1e4) / 1e4;
16177 |       if (convMonto > 1e-6) {
16178 |         inserts.push(client.from('movimientos_cuenta_corriente_intermediario').insert({
16179 |           intermediario_id: intermediarioId, moneda: monedaEntregada, monto: convMonto, orden_id: ordenId, transaccion_id: null,
16180 |           concepto: CONCEPTO_CC_CONVERSION, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
16181 |         }));
16182 |       }
16183 |       if (comMonto > 1e-6) {
16184 |         inserts.push(client.from('movimientos_cuenta_corriente_intermediario').insert({
16185 |           intermediario_id: intermediarioId, moneda: monedaEntregada, monto: comMonto, orden_id: ordenId, transaccion_id: null,
16186 |           concepto: CONCEPTO_CC_COMISION, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
16187 |         }));
16188 |       }
16189 |     }
16190 |   }
16191 |   // Otras monedas: limitar Debe al Haber; si hay tope, un solo movimiento "Comisión del acuerdo"
16192 |   ['USD', 'EUR', 'ARS'].forEach((moneda) => {
16193 |     if (moneda === monedaEntregada) return;
16194 |     const saldo = saldos[moneda];
16195 |     const haberEnMoneda = Math.max(0, -(saldo || 0));
16196 |     if (haberEnMoneda <= 1e-6) return;
16197 |     const convPlaneado = saldo < 0 ? -saldo : 0;
16198 |     const comPlaneado = comisionPorMoneda[moneda] || 0;
16199 |     const totalPlaneado = convPlaneado + comPlaneado;
16200 |     const totalACrear = totalPlaneado <= 1e-6 ? 0 : Math.min(totalPlaneado, haberEnMoneda);
16201 |     if (totalACrear <= 1e-6) return;
16202 |     const estamosCap = totalACrear < totalPlaneado - 1e-6;
16203 |     if (estamosCap) {
16204 |       inserts.push(client.from('movimientos_cuenta_corriente_intermediario').insert({
16205 |         intermediario_id: intermediarioId, moneda: moneda, monto: totalACrear, orden_id: ordenId, transaccion_id: null,
16206 |         concepto: CONCEPTO_CC_COMISION, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
16207 |       }));
16208 |     } else {
16209 |       if (convPlaneado > 1e-6) {
16210 |         inserts.push(client.from('movimientos_cuenta_corriente_intermediario').insert({
16211 |           intermediario_id: intermediarioId, moneda: moneda, monto: convPlaneado, orden_id: ordenId, transaccion_id: null,
16212 |           concepto: CONCEPTO_CC_CONVERSION, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
16213 |         }));
16214 |       }
16215 |       if (comPlaneado > 1e-6) {
16216 |         inserts.push(client.from('movimientos_cuenta_corriente_intermediario').insert({
16217 |           intermediario_id: intermediarioId, moneda: moneda, monto: comPlaneado, orden_id: ordenId, transaccion_id: null,
16218 |           concepto: CONCEPTO_CC_COMISION, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
16219 |         }));
16220 |       }
16221 |     }
16222 |   });
16223 | } else {
16224 |   ['USD', 'EUR', 'ARS'].forEach((moneda) => {
16225 |     const saldo = saldos[moneda];
16226 |     if (saldo === 0) return;
16227 |     inserts.push(client.from('movimientos_cuenta_corriente_intermediario').insert({
16228 |       intermediario_id: intermediarioId, moneda: moneda, monto: -saldo, orden_id: ordenId, transaccion_id: null,
16229 |       concepto: CONCEPTO_CC_CONVERSION, fecha, usuario_id: currentUserId, estado: 'cerrado', estado_fecha: ahora,
16230 |     }));
16231 |   });
16232 | }
16233 | if (inserts.length === 0) return Promise.resolve();
16234 | return Promise.all(inserts).then(() => {});
16235 | });
16236 | });
16237 | });
16238 | });
16239 | });
16240 | }
16241 |
16242 | /** Baja (elimina) una transacción: borra movimientos asociados, la transacción y recalcula estado de la orden.
16243 |  * No se permite dar de baja una transacción que tiene movimientos de momento cero (Debe/Compensación) en CC cliente o CC intermediario, para no desbalancear. */
16244 | function eliminarTransaccion(transaccionId, ordenId) {
16245 |   if (!transaccionId || !ordenId) return Promise.resolve();
16246 |   const canEliminarTr = userPermissions.includes('eliminar_transacciones');
16247 |   if (!canEliminarTr) {
16248 |     showToast('No tenés permiso para dar de baja transacciones.', 'error');
16249 |     return Promise.resolve();
16250 |   }
16251 |   Promise.all([

```

```

16252 |     client.from('movimientos_cuenta_corriente').select('id, monto_usd, monto_ars, monto_eur').eq('transaccion_id', transaccionId),
16253 |     client.from('movimientos_cuenta_corriente_intermediario').select('id, monto_usd, monto_ars, monto_eur').eq('transaccion_id', transaccionId),
16254 |   ]).then(([rMov, rMovInt]) => {
16255 |     const movs = rMov.data || [];
16256 |     const movsInt = rMovInt.data || [];
16257 |     const tieneMomentoCeroCliente = movs.some((m) => m.monto_usd != null || m.monto_ars != null || m.monto_eur != null);
16258 |     const tieneMomentoCeroInt = movsInt.some((m) => m.monto_usd != null || m.monto_ars != null || m.monto_eur != null);
16259 |     if (tieneMomentoCeroCliente || tieneMomentoCeroInt) {
16260 |       showToast('No se puede dar de baja esta transacción: forma parte del momento cero de la orden. Para deshacer los movimientos de la orden, usá "Anular orden."', 'error');
16261 |       return;
16262 |     }
16263 |     showConfirm('¿Dar de baja esta transacción? Se eliminarán también los movimientos de caja y cuenta corriente asociados.', 'Dar de baja', () => {
16264 |       const deletes = [
16265 |         client.from('movimientos_cuenta_corriente').delete().eq('transaccion_id', transaccionId),
16266 |         client.from('movimientos_cuenta_corriente_intermediario').delete().eq('transaccion_id', transaccionId),
16267 |         client.from('movimientos_caja').delete().eq('transaccion_id', transaccionId),
16268 |       ];
16269 |       Promise.all(deletes)
16270 |         .then(() => client.from('transacciones').delete().eq('id', transaccionId))
16271 |         .then((rDel) => {
16272 |           if (rDel.error) {
16273 |             showToast('Error al eliminar: ' + (rDel.error?.message || ''), 'error');
16274 |             return;
16275 |           }
16276 |           showToast('Transacción dada de baja.', 'success');
16277 |           return actualizarEstadoOrden(ordenId);
16278 |         })
16279 |         .then(() => {
16280 |           const vistaCc = document.getElementById('vista-cuenta-corriente');
16281 |           if (vistaCc && vistaCc.style.display !== 'none') loadCuentaCorriente();
16282 |         });
16283 |     });
16284 |   });
16285 | }
16286 |
16287 | /** Actualiza el estado de la orden según transacciones y acuerdo. Devuelve promesa con { estado, conciliada, todasEjecutadas } o undefined. */
16288 | function actualizarEstadoOrden(ordenId) {
16289 |   if (!ordenId) return Promise.resolve();
16290 |   return client.from('instrumentacion').select('id, multicontraparte_manual').eq('orden_id', ordenId).maybeSingle().then((r) => {
16291 |     const instId = r.data && r.data.id;
16292 |     if (!instId) return;
16293 |     const mcInst = !!r.data && r.data.multicontraparte_manual;
16294 |     return client.from('ordenes').select('id, cliente_id, intermediario_id, moneda_recibida, monto_recibido, moneda_entregada, monto_entregado, cotizacion, tipos_operacion(cod
16295 | igo, usa_intermediario)').eq('id', ordenId).single().then((r0rd) => {
16296 |       const orden = r0rd.data;
16297 |       if (!orden) return;
16298 |       return client.from('transacciones').select('id, tipo, moneda, monto, estado, tipo_cambio, cobrador, pagador, pagador_cliente_id, cobrador_cliente_id, pagador_intermediar
16299 | io_id, cobrador_intermediario_id').eq('instrumentacion_id', instId).then((res) => {
16300 |         const list = res.data || [];
16301 |         const toJ = orden.tipos_operacion && (Array.isArray(orden.tipos_operacion) ? orden.tipos_operacion[0] : orden.tipos_operacion);
16302 |         const totMc = mcInst && esTipoOpMulticontraparteElegibleDesdeOrden(orden, toJ);
16303 |         const { estado, conciliada, todasEjecutadas } = calcularEstadoOrden(list, orden, { totalesMulticontraparte: totMc });
16304 |         return client.from('ordenes').update({ estado, updated_at: new Date().toISOString() }).eq('id', ordenId).then(() => {
16305 |           const ordenIdAbierto = transaccionesOrdenIdActual;
16306 |           const prom = loadOrdenes();
16307 |           if (prom && ordenIdAbierto) {
16308 |             prom.then(() => {
16309 |               client.from('ordenes').select('id, cliente_id, fecha, estado, tipo_operacion_id, operacion_directa, intermediario_id, moneda_recibida, moneda_entregada, monto_re
16310 | cibido, monto_entregado, cotizacion, observaciones').eq('id', ordenIdAbierto).single().then((r0rd2) => {
16311 |                 if (r0rd2.data) expandOrdenTransacciones(ordenIdAbierto, r0rd2.data);
16312 |               });
16313 |             });
16314 |           }
16315 |           return { estado, conciliada, todasEjecutadas };
16316 |         });
16317 |       });
16318 |     });
16319 |   });
16320 | }
16321 |
16322 | function setupModalTransacciones() {
16323 |   const backdrop = document.getElementById('modal-transacciones-backdrop');
16324 |   const btnClose = document.getElementById('modal-transacciones-close');
16325 |   if (btnClose) btnClose.addEventListener('click', closeModalTransacciones);
16326 |   if (backdrop) setupBackdropCloseOnlyOnRealClick(backdrop, closeModalTransacciones);
16327 | }
16328 |
16329 | function setupModalTransaccion() {
16330 |   const backdrop = document.getElementById('modal-transaccion-backdrop');
16331 |   const btnClose = document.getElementById('modal-transaccion-close');
16332 |   const btnCancel = document.getElementById('modal-transaccion-cancelar');
16333 |   const form = document.getElementById('form-transaccion');
16334 |   const selMoneda = document.getElementById('transaccion-moneda');
16335 |   const montoEl = document.getElementById('transaccion-monto');
16336 |   if (btnClose) btnClose.addEventListener('click', closeModalTransaccion);
16337 |   if (btnCancel) btnCancel.addEventListener('click', closeModalTransaccion);
16338 |   if (backdrop) setupBackdropCloseOnlyOnRealClick(backdrop, closeModalTransaccion);
16339 |   if (form) form.addEventListener('submit', (e) => { e.preventDefault(); saveTransaccion(); });
16340 |   const selTipo = document.getElementById('transaccion-tipo');
16341 |   if (selTipo) selTipo.addEventListener('change', adaptarTransaccionTipoMoneda);
16342 |   if (selMoneda) selMoneda.addEventListener('change', toggleTransaccionMonedaArs);
16343 |   if (montoEl) { montoEl.addEventListener('input', actualizarMontoCalculado); montoEl.addEventListener('change', actualizarMontoCalculado); }
16344 | }
16345 |
16346 | // --- Clientes ABM ---
16347 | function loadClientes() {
16348 |   const loadingEl = document.getElementById('clientes-loading');

```

```

16346| const wrapEl = document.getElementById('clientes-tabla-wrap');
16347| const tbody = document.getElementById('clientes-tbody');
16348| const btnNuevo = document.getElementById('btn-nuevo-cliente');
16349| if (!loadingEl || !wrapEl || !tbody) return;
16350|
16351| const canAbm = userPermissions.includes('abm_clientes');
16352| if (btnNuevo) btnNuevo.style.display = canAbm ? '' : 'none';
16353|
16354| const silentCli = isPandiBackgroundRefresh();
16355| if (!silentCli) {
16356|   loadingEl.style.display = 'block';
16357|   wrapEl.style.display = 'none';
16358|   tbody.innerHTML = '';
16359| }
16360|
16361| return client
16362|   .from('clientes')
16363|   .select('id, nombre, documento, email, telefono, direccion, activo')
16364|   .order('nombre', { ascending: true })
16365|   .then((res) => {
16366|     loadingEl.style.display = 'none';
16367|     if (res.error) {
16368|       if (silentCli) showToast('Error al actualizar clientes: ' + (res.error.message || ''), 'error');
16369|       else {
16370|         tbody.innerHTML = '<tr><td colspan="6">Error: ' + (res.error.message || '') + '</td></tr>';
16371|         wrapEl.style.display = 'block';
16372|       }
16373|       return;
16374|     }
16375|     const list = res.data || [];
16376|     const esc = (s) => (s == null ? '' : String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;'));
16377|     const switchCellCliente = (id, checked) =>
16378|       `<div class="tipo-op-toggle-cell"><span class="toggle-switch"><input type="checkbox" class="cliente-activo-toggle" data-id="${id}"${checked ? ' checked' : ''}${canAbm
? '' : ' disabled'} /><span class="slider"></span></span></div>`;
16379|     tbody.innerHTML = list
16380|       .map(
16381|         (c) =>
16382|           `<tr data-id="${c.id}">
16383|             <td>${esc(c.nombre)}</td>
16384|             <td>${esc(c.documento)}</td>
16385|             <td>${esc(c.email)}</td>
16386|             <td>${esc(c.telefono)}</td>
16387|             <td>${switchCellCliente(c.id, c.activo !== false)}</td>
16388|             <td>${canAbm ? `<button type="button" class="btn-editar-cliente" data-id="${c.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke=
"currentColor" stroke-width="2"><path d="M11 4H4a2 2 0 0-2 2v14a2 2 0 02 2h14a2 2 0 02-2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013 3l12 15-4 1 1-4 9.5-9.5z"/></svg></span>
Editar</button>` : ''}</td>
16389|           `)
16390|       .join('');
16391|     tbody.querySelectorAll('.cliente-activo-toggle').forEach((chk) => {
16392|       if (chk.disabled) return;
16393|       chk.addEventListener('change', function () {
16394|         const id = this.getAttribute('data-id');
16395|         const newVal = this.checked;
16396|         client.from('clientes').update({ activo: newVal, updated_at: new Date().toISOString() }).eq('id', id).then((res) => {
16397|           if (res.error) showToast('Error: ' + (res.error.message || 'No se pudo actualizar.'), 'error');
16398|           else showToast('Actualizado.');
```

```

16440 | backdrop.classList.add('activo');
16441 | }
16442 |
16443 | function closeModalCliente() {
16444 |   const backdrop = document.getElementById('modal-cliente-backdrop');
16445 |   if (backdrop) backdrop.classList.remove('activo');
16446 | }
16447 |
16448 | function saveCliente() {
16449 |   const idEl = document.getElementById('cliente-id');
16450 |   const id = idEl && idEl.value ? idEl.value.trim() : '';
16451 |   const nombre = document.getElementById('cliente-nombre').value.trim();
16452 |   if (!nombre) {
16453 |     showToast('El nombre es obligatorio.', 'error');
16454 |     return;
16455 |   }
16456 |   const payload = {
16457 |     nombre,
16458 |     documento: document.getElementById('cliente-documento').value.trim() || null,
16459 |     email: document.getElementById('cliente-email').value.trim() || null,
16460 |     telefono: document.getElementById('cliente-telefono').value.trim() || null,
16461 |     direccion: document.getElementById('cliente-direccion').value.trim() || null,
16462 |     activo: document.getElementById('cliente-activo').checked,
16463 |   };
16464 |   const prom = id
16465 |     ? client.from('clientes').update({ ...payload, updated_at: new Date().toISOString() }).eq('id', id)
16466 |     : client.from('clientes').insert(payload);
16467 |   prom.then((res) => {
16468 |     if (res.error) {
16469 |       showToast('Error: ' + (res.error.message || 'No se pudo guardar.'), 'error');
16470 |       return;
16471 |     }
16472 |     closeModalCliente();
16473 |     loadClientes();
16474 |   });
16475 | }
16476 |
16477 | function setupModalCliente() {
16478 |   const backdrop = document.getElementById('modal-cliente-backdrop');
16479 |   const btnClose = document.getElementById('modal-cliente-close');
16480 |   const btnCancel = document.getElementById('modal-cliente-cancelar');
16481 |   const form = document.getElementById('form-cliente');
16482 |   const btnNuevo = document.getElementById('btn-nuevo-cliente');
16483 |
16484 |   if (btnClose) btnClose.addEventListener('click', closeModalCliente);
16485 |   if (btnCancel) btnCancel.addEventListener('click', closeModalCliente);
16486 |   if (backdrop) setupBackdropCloseOnlyOnRealClick(backdrop, closeModalCliente);
16487 |   if (form) {
16488 |     form.addEventListener('submit', (e) => {
16489 |       e.preventDefault();
16490 |       saveCliente();
16491 |     });
16492 |   }
16493 |   if (btnNuevo) {
16494 |     btnNuevo.addEventListener('click', () => openModalCliente(null));
16495 |   }
16496 | }
16497 |
16498 | // --- Intermediarios ABM ---
16499 | function loadIntermediarios() {
16500 |   const loadingEl = document.getElementById('intermediarios-loading');
16501 |   const wrapEl = document.getElementById('intermediarios-tabla-wrap');
16502 |   const tbody = document.getElementById('intermediarios-tbody');
16503 |   const btnNuevo = document.getElementById('btn-nuevo-intermediario');
16504 |   if (!loadingEl || !wrapEl || !tbody) return;
16505 |
16506 |   const canAbm = userPermissions.includes('abm_intermediarios');
16507 |   if (btnNuevo) btnNuevo.style.display = canAbm ? '' : 'none';
16508 |
16509 |   const silentInt = isPandiBackgroundRefresh();
16510 |   if (!silentInt) {
16511 |     loadingEl.style.display = 'block';
16512 |     wrapEl.style.display = 'none';
16513 |     tbody.innerHTML = '';
16514 |   }
16515 |
16516 |   return client
16517 |     .from('intermediarios')
16518 |     .select('id, nombre, documento, email, telefono, direccion, activo')
16519 |     .order('nombre', { ascending: true })
16520 |     .then((res) => {
16521 |       loadingEl.style.display = 'none';
16522 |       if (res.error) {
16523 |         if (silentInt) showToast('Error al actualizar intermediarios: ' + (res.error.message || ''), 'error');
16524 |         else {
16525 |           tbody.innerHTML = '<tr><td colspan="6">Error: ' + (res.error.message || '') + '</td></tr>';
16526 |           wrapEl.style.display = 'block';
16527 |         }
16528 |         return;
16529 |       }
16530 |       const list = res.data || [];
16531 |       const esc = (s) => (s == null ? '' : String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;'));
16532 |       const switchCellInt = (id, checked) =>
16533 |         <div class="tipo-op-toggle-cell"><span class="toggle-switch"><input type="checkbox" class="intermediario-activo-toggle" data-id="${id}"${checked ? ' checked' : ''}${c
anAbm ? '' : ' disabled'} /><span class="slider"></span></span></div>;
16534 |       tbody.innerHTML = list
16535 |         .map(

```

```

16536 |         (i) =>
16537 |             <tr data-id="${i.id}">
16538 |                 <td>${esc(i.nombre)}</td>
16539 |                 <td>${esc(i.documento)}</td>
16540 |                 <td>${esc(i.email)}</td>
16541 |                 <td>${esc(i.telefono)}</td>
16542 |                 <td>${switchCelInt(i.id, i.activo !== false)}</td>
16543 |                 <td>${canAbm ? `<button type="button" class="btn-editar btn-editar-intermediario" data-id="${i.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" s
troke="currentColor" stroke-width="2"><path d="M11 4H4a2 2 0 0-2 2v14a2 2 0 002 2h14a2 2 0 002 2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013 12 15l-4 1 1-4 9.5-9.5z"/></svg><
/span>Editar</button>` : ''}</td>
16544 |             </tr>
16545 |         );
16546 |         tbody.querySelectorAll('.intermediario-activo-toggle').forEach((chk) => {
16547 |             if (chk.disabled) return;
16548 |             chk.addEventListener('change', function () {
16549 |                 const id = this.getAttribute('data-id');
16550 |                 const newVal = this.checked;
16551 |                 client.from('intermediarios').update({ activo: newVal, updated_at: new Date().toISOString() }).eq('id', id).then((res) => {
16552 |                     if (res.error) showToast('Error: ' + (res.error.message || 'No se pudo actualizar.'), 'error');
16553 |                     else showToast('Actualizado.');

```



```

16631 | }
16632 |
16633 | function setupModalIntermediario() {
16634 |     const backdrop = document.getElementById('modal-intermediario-backdrop');
16635 |     const btnClose = document.getElementById('modal-intermediario-close');
16636 |     const btnCancel = document.getElementById('modal-intermediario-cancelar');
16637 |     const form = document.getElementById('form-intermediario');
16638 |     const btnNuevo = document.getElementById('btn-nuevo-intermediario');
16639 |
16640 |     if (btnClose) btnClose.addEventListener('click', closeModalIntermediario);
16641 |     if (btnCancel) btnCancel.addEventListener('click', closeModalIntermediario);
16642 |     if (backdrop) setupBackdropCloseOnlyOnRealClick(backdrop, closeModalIntermediario);
16643 |     if (form) form.addEventListener('submit', (e) => { e.preventDefault(); saveIntermediario(); });
16644 |     if (btnNuevo) btnNuevo.addEventListener('click', () => openModalIntermediario(null));
16645 | }
16646 |
16647 | // --- Tipos de operación (ABM)
16648 | async function subirIconoTipoOperacionStorage(file) {
16649 |     if (!file || !file.type.startsWith('image/')) {
16650 |         showToast('Elegí un archivo de imagen.', 'error');
16651 |         return null;
16652 |     }
16653 |     const safe = (file.name || 'img').replace(/[\w.\-]/g, '_').slice(0, 64);
16654 |     const path = `${Date.now()}-${Math.random().toString(36).slice(2, 10)}-${safe}`;
16655 |     const { data, error } = await client.storage.from(STORAGE_BUCKET_TIPO_OP_ICONOS).upload(path, file, { upsert: false, contentType: file.type || undefined, cacheControl: '3600'
16656 |     });
16657 |     if (error) {
16658 |         showToast('No se pudo subir: ' + (error.message || 'revisá bucket y permisos en Supabase. '), 'error');
16659 |         return null;
16660 |     }
16661 |     const { data: pub } = client.storage.from(STORAGE_BUCKET_TIPO_OP_ICONOS).getPublicUrl(data.path);
16662 |     return pub && pub.publicUrl ? pub.publicUrl : null;
16663 | }
16664 | /** Misma cuenta Storage que iconos de tipo de operación; rutas bajo empresa-marca/ para el logo de marca. */
16665 | async function subirLogoEmpresaStorage(file) {
16666 |     if (!file || !file.type.startsWith('image/')) {
16667 |         showToast('Elegí un archivo de imagen.', 'error');
16668 |         return null;
16669 |     }
16670 |     const safe = (file.name || 'img').replace(/[\w.\-]/g, '_').slice(0, 64);
16671 |     const path = `empresa-marca/${Date.now()}-${Math.random().toString(36).slice(2, 10)}-${safe}`;
16672 |     const { data, error } = await client.storage.from(STORAGE_BUCKET_TIPO_OP_ICONOS).upload(path, file, { upsert: false, contentType: file.type || undefined, cacheControl: '3600'
16673 |     });
16674 |     if (error) {
16675 |         showToast('No se pudo subir: ' + (error.message || 'revisá bucket tipo-operacion-iconos y permisos en Supabase. '), 'error');
16676 |         return null;
16677 |     }
16678 |     const { data: pub } = client.storage.from(STORAGE_BUCKET_TIPO_OP_ICONOS).getPublicUrl(data.path);
16679 |     return pub && pub.publicUrl ? pub.publicUrl : null;
16680 | }
16681 | function syncConfigEmpresaLogoPreview() {
16682 |     const inp = document.getElementById('config-empresa-logo-url');
16683 |     const prev = document.getElementById('config-empresa-logo-preview');
16684 |     if (!prev) return;
16685 |     const raw = (inp && inp.value || '').trim();
16686 |     let src = '/assets/favicon-192x192.png';
16687 |     if (raw) {
16688 |         if (raw.startsWith('/') && !raw.startsWith('//')) src = raw;
16689 |         else if (isHttpsUrlSegura(raw)) src = raw;
16690 |     }
16691 |     prev.src = src;
16692 |     prev.alt = nombreMarcaSistema();
16693 | }
16694 |
16695 | let configEmpresaMarcaUiBound = false;
16696 | function setupConfigEmpresaMarcaControles() {
16697 |     if (configEmpresaMarcaUiBound) return;
16698 |     const fileEl = document.getElementById('config-empresa-logo-file');
16699 |     const btnSubir = document.getElementById('config-empresa-logo-subir');
16700 |     const btnReset = document.getElementById('config-empresa-logo-restablecer');
16701 |     const urlInp = document.getElementById('config-empresa-logo-url');
16702 |     if (btnSubir && fileEl) {
16703 |         btnSubir.addEventListener('click', () => fileEl.click());
16704 |         fileEl.addEventListener('change', () => {
16705 |             const f = fileEl.files && fileEl.files[0];
16706 |             fileEl.value = '';
16707 |             if (!f) return;
16708 |             subirLogoEmpresaStorage(f).then((url) => {
16709 |                 if (!url) return;
16710 |                 if (urlInp) urlInp.value = url;
16711 |                 syncConfigEmpresaLogoPreview();
16712 |                 showToast('Logo subido; URL cargada. Recordá guardar la configuración.', 'success');
16713 |             });
16714 |         });
16715 |     }
16716 |     if (btnReset && urlInp) {
16717 |         btnReset.addEventListener('click', () => {
16718 |             urlInp.value = '';
16719 |             syncConfigEmpresaLogoPreview();
16720 |         });
16721 |     }
16722 |     if (urlInp) urlInp.addEventListener('input', () => syncConfigEmpresaLogoPreview());
16723 |     configEmpresaMarcaUiBound = true;
16724 | }
16725 |

```

```

16726 function updateTipoOperacionModalIconoCustomWrap() {
16727   const wrap = document.getElementById('tipo-operacion-icone-custom-wrap');
16728   const rad = document.querySelector('input[name="tipo-operacion-icone-modo"]:checked');
16729   const v = rad ? rad.value : 'auto';
16730   if (wrap) wrap.style.display = v === 'custom' ? 'block' : 'none';
16731 }
16732
16733 function syncTipoOperacionModalIconoPreview() {
16734   const wrap = document.getElementById('tipo-operacion-icone-preview');
16735   if (!wrap) return;
16736   const codEl = document.getElementById('tipo-operacion-codigo');
16737   const nomEl = document.getElementById('tipo-operacion-nombre');
16738   const urlEl = document.getElementById('tipo-operacion-icone-url');
16739   const codigo = (codEl && codEl.value || '').trim();
16740   const nombre = (nomEl && nomEl.value || '').trim();
16741   const rad = document.querySelector('input[name="tipo-operacion-icone-modo"]:checked');
16742   const modoRaw = rad ? rad.value : 'auto';
16743   const modo = modoRaw === 'cheque' || modoRaw === 'custom' ? modoRaw : 'auto';
16744   const url = (urlEl && urlEl.value || '').trim();
16745   const usaIntEl = document.getElementById('tipo-operacion-usa-intermediario');
16746   const usaInt = !!usaIntEl && usaIntEl.checked;
16747   wrap.innerHTML = htmlTipoOperacionIconos(codigo || '-', nombre, { iconoModo: modo, iconoUrlPublica: url, usaIntermediario: usaInt });
16748 }
16749
16750 /** True si el error de PostgREST/Postgres es por columna orden_visual inexistente (migración no ejecutada). */
16751 function supabaseErrorFaltaColumnaOrdenVisualTiposOperacion(err) {
16752   if (!err) return false;
16753   if (String(err.code) === '42703') return true;
16754   const msg = String(err.message || err.details || '');
16755   return /orden_visual/i.test(msg) && (/does not exist|no existe|undefined column/i.test(msg));
16756 }
16757
16758 /** Texto plano en <option> SR: variante con intermediario lleva sufijo ` - Int.`. */
16759 function nombreTipoOperacionOrdenUi(t) {
16760   const n = t && t.nombre !== null ? String(t.nombre).trim() : '';
16761   if (t && t.usa_intermediario === true) return n ? `${n} - Int.` : '- Int.';
16762   return n || '-';
16763 }
16764
16765 /** Etiqueta en combo custom (HTML): nombre base + ` - Int.` en azul negrita si aplica. */
16766 function htmlOrdenTipoOperacionEtiquetaVisible(nombreBase, usaInt) {
16767   const b = (nombreBase || '').trim();
16768   if (usaInt) {
16769     if (b) return `${escapeHtml(b)}<span class="tipo-op-sufijo-int"> - Int.</span>`;
16770     return '<span class="tipo-op-sufijo-int">- Int.</span>';
16771   }
16772   return escapeHtml(b || '-');
16773 }
16774
16775 /**
16776  * Orden estable igual que en Supabase: orden_visual (null al final), codigo, usa_intermediario, id.
16777  * Refuerzo en cliente por si la respuesta llegara desordenada.
16778  */
16779 function ordenarTiposOperacionListaParaOrden(arr) {
16780   const list = (arr || []).slice();
16781   const tieneOv = list.some(
16782     (t) => t && t.orden_visual !== null && String(t.orden_visual).trim() !== '' && Number.isFinite(Number(t.orden_visual)),
16783   );
16784   function claveOv(t) {
16785     const n = Number(t && t.orden_visual);
16786     if (Number.isFinite(n)) return n;
16787     return tieneOv ? 1e9 : 0;
16788   }
16789   list.sort((a, b) => {
16790     const da = claveOv(a);
16791     const db = claveOv(b);
16792     if (da !== db) return da - db;
16793     const sc = String((a && a.codigo) || '').localeCompare(String((b && b.codigo) || ''));
16794     if (sc !== 0) return sc;
16795     const ia = a && a.usa_intermediario === true ? 1 : 0;
16796     const ib = b && b.usa_intermediario === true ? 1 : 0;
16797     if (ia !== ib) return ia - ib;
16798     return String((a && a.id) || '').localeCompare(String((b && b.id) || ''));
16799   });
16800   return list;
16801 }
16802
16803 /**
16804  * Ejecuta consulta tipos_operacion con orden_visual; si la columna no existe, repite sin ella y avisa una vez por sesión (toast).
16805  * executeFull y executeLegacy deben devolver la promesa del .select()... (mismo client.from).
16806  */
16807 function tiposOperacionFetchConFallbackOrdenVisual(executeFull, executeLegacy) {
16808   return executeFull().then((res) => {
16809     if (!res.error || !supabaseErrorFaltaColumnaOrdenVisualTiposOperacion(res.error)) return res;
16810     if (typeof window !== 'undefined' && !window.__pandiOrdenVisualMigracionToastHecho) {
16811       window.__pandiOrdenVisualMigracionToastHecho = true;
16812       showToast('Falta la columna orden_visual en tipos_operacion. Ejecutó en Supabase: sql/migracion_tipos_operacion_orden_visual.sql. Hasta entonces se usa orden por código.', 'info');
16813     }
16814     return executeLegacy();
16815   });
16816 }
16817
16818 /** Intercambia orden_visual con el vecino en la lista ya ordenada (delta -1 = subir, +1 = bajar). */
16819 function intercambiarOrdenVisualTiposOperacion(listOrdenada, indiceActual, delta) {
16820   if (!userPermissions.includes('abm_tipos_operacion')) return;
16821   const j = indiceActual + delta;

```

```

16822 | if (j < 0 || j >= listOrdenada.length) return;
16823 | const a = listOrdenada[indiceActual];
16824 | const b = listOrdenada[j];
16825 | const oa = Number(a.orden_visual);
16826 | const ob = Number(b.orden_visual);
16827 | Promise.all([
16828 |   client.from('tipos_operacion').update({ orden_visual: ob }).eq('id', a.id),
16829 |   client.from('tipos_operacion').update({ orden_visual: oa }).eq('id', b.id),
16830 | ]).then(([{r1, r2}] => {
16831 |   if (r1.error || r2.error) {
16832 |     const e1 = r1.error;
16833 |     const e2 = r2.error;
16834 |     if (supabaseErrorFaltaColumnaOrdenVisualTiposOperacion(e1) || supabaseErrorFaltaColumnaOrdenVisualTiposOperacion(e2)) {
16835 |       showToast('No se puede reordenar: falta la columna orden_visual. Ejecutá sql/migracion_tipos_operacion_orden_visual.sql en Supabase.', 'error');
16836 |     } else {
16837 |       showToast('Error: ' + ((e1 && e1.message) || (e2 && e2.message) || 'No se pudo reordenar.'), 'error');
16838 |     }
16839 |     return;
16840 |   }
16841 |   showToast('Orden actualizado.', 'success');
16842 |   loadTiposOperacion();
16843 | });
16844 | }
16845 |
16846 | function loadTiposOperacion() {
16847 |   const loadingEl = document.getElementById('tipos-operacion-loading');
16848 |   const wrapEl = document.getElementById('tipos-operacion-tabla-wrap');
16849 |   const tbody = document.getElementById('tipos-operacion-tbody');
16850 |   const btnNuevo = document.getElementById('btn-nuevo-tipo-operacion');
16851 |   if (!loadingEl || !wrapEl || !tbody) return;
16852 |
16853 |   const canAbm = userPermissions.includes('abm_tipos_operacion');
16854 |   if (btnNuevo) btnNuevo.style.display = canAbm ? '' : 'none';
16855 |
16856 |   const silentTipos = isPandiBackgroundRefresh();
16857 |   if (!silentTipos) {
16858 |     loadingEl.style.display = 'block';
16859 |     wrapEl.style.display = 'none';
16860 |     tbody.innerHTML = '';
16861 |   }
16862 |
16863 |   const colsBase = 'id, codigo, nombre, moneda_in, moneda_out, usa_intermediario, activo, icono_modo, icono_url_publica';
16864 |   return tiposOperacionFetchConFallbackOrdenVisual(
16865 |     () => client.from('tipos_operacion').select(colsBase + ', orden_visual').order('orden_visual', { ascending: true }).order('codigo').order('usa_intermediario').order('id'),
16866 |     () => client.from('tipos_operacion').select(colsBase).order('codigo').order('usa_intermediario').order('id'),
16867 |   ).then((res) => {
16868 |     loadingEl.style.display = 'none';
16869 |     if (res.error) {
16870 |       const msg = res.error.message || 'No se pudieron cargar los tipos.';
16871 |       showToast('Error: ' + msg, 'error');
16872 |       if (!silentTipos) {
16873 |         tbody.innerHTML = '<tr><td colspan="9">Error: ' + escapeHtml(msg) + '</td></tr>';
16874 |         wrapEl.style.display = 'block';
16875 |       }
16876 |       return;
16877 |     }
16878 |     const list = res.data || [];
16879 |     const tieneOrdenVisual = list.length === 0 || (list[0] && Object.prototype.hasOwnProperty.call(list[0], 'orden_visual'));
16880 |     const switchCell = (id, field, checked) =>
16881 |       `<div class="tipo-op-toggle-cell"><span class="toggle-switch"><input type="checkbox" class="tipo-op-toggle" data-id="${id}" data-field="${field}"${checked ? ' checked' : ''}></span></div>`;
16882 |     if (canAbm ? '' : 'disabled') /><span class="slider"></span></div> ;
16883 |     tbody.innerHTML = list.map((t, idx) => {
16884 |       const monIn = (t.moneda_in || '').toUpperCase();
16885 |       const monOut = (t.moneda_out || '').toUpperCase();
16886 |       const ordenCelda = !tieneOrdenVisual
16887 |         ? (canAbm
16888 |           ? `<span style="font-size:0.8rem;color:#92400e;text-align:center;display:block;">Migración SQL pendiente</span>`
16889 |           : `<span style="color:#666;font-size:0.85rem;"></span>`)
16890 |         : `<div class="tipo-op-orden-botones" style="display:flex;gap:0.25rem;justify-content:center;align-items:center;flex-wrap:wrap;">
16891 |           <button type="button" class="btn-secondary btn-icon-only btn-tipo-op-orden-mover" data-id="${t.id}" data-dir="-1" title="Subir en el listado" aria-label="Subir en el listado">
16892 |             <path d="M18 15l-6 6 6 6"/></svg></span></button>
16893 |             <button type="button" class="btn-secondary btn-icon-only btn-tipo-op-orden-mover" data-id="${t.id}" data-dir="1" title="Bajar en el listado" aria-label="Bajar en el listado">
16894 |             <path d="M6 15l-6 6 6 6"/></svg></span></button>
16895 |           </div>`;
16896 |       const monInStyle = "color:#666;font-size:0.85rem;" + `${escapeHtml(String(Number.isFinite(Number(t.orden_visual)) ? t.orden_visual : '-'))}</span>`;
16897 |       return `<tr data-id="${t.id}">
16898 |         <td class="tipo-op-orden-celda">${ordenCelda}</td>
16899 |         <td class="td-tipo-op-iconos">${htmlTipoOperacionIconos(t.codigo || '', t.nombre || '', { iconoModo: t.icono_modo, iconoUrlPublica: t.icono_url_publica, usaIntermediario: t.usa_intermediario === true })}</td>
16900 |         <td>${escapeHtml(t.nombre || '')}</td>
16901 |         <td><code style="font-size:0.85rem;">${escapeHtml(t.codigo || '')}</code>${t.usa_intermediario === true ? ' <span style="color:#666;font-size:0.8rem;">(int.)</span>' : ''}</td>
16902 |         <td><span class="tipo-op-moneda-celda">${htmlIconoMonedaTipoOp(monIn, 18)} <span class="tipo-op-moneda-codigo">${escapeHtml(monIn)}</span></span></td>
16903 |         <td><span class="tipo-op-moneda-celda">${htmlIconoMonedaTipoOp(monOut, 18)} <span class="tipo-op-moneda-codigo">${escapeHtml(monOut)}</span></span></td>
16904 |         <td>${switchCell(t.id, 'usa_intermediario', t.usa_intermediario === true)}</td>
16905 |         <td>${switchCell(t.id, 'activo', t.activo !== false)}</td>
16906 |         <td>${canAbm ? `<button type="button" class="btn-editar btn-tipo-operacion" data-id="${t.id}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M11 4H4a2 2 0 0-2 2v14a2 2 0 0-2 2h14a2 2 0 0-2 2v-7"/></svg></span>` : `<button type="button"></button>`}</td>
16907 |       </tr>`;
16908 |     }).join('');
16909 |     tbody.querySelectorAll('.btn-tipo-op-orden-mover').forEach((btn) => {
16910 |       btn.addEventListener('click', () => {
16911 |         if (btn.disabled) return;

```

```

16910 |     const id = btn.getAttribute('data-id');
16911 |     const dir = Number(btn.getAttribute('data-dir'));
16912 |     const idx = list.findIndex(x => String(x.id) === String(id));
16913 |     if (idx < 0 || !Number.isFinite(dir)) return;
16914 |     intercambiarOrdenVisualTiposOperacion(list, idx, dir);
16915 |   });
16916 | });
16917 | tbody.querySelectorAll('.tipo-op-toggle').forEach((chk) => {
16918 |   if (chk.disabled) return;
16919 |   chk.addEventListener('change', function () {
16920 |     const id = this.getAttribute('data-id');
16921 |     const field = this.getAttribute('data-field');
16922 |     const newVal = this.checked;
16923 |     const row = list.find((r) => String(r.id) === String(id));
16924 |     const aplicar = () => {
16925 |       client.from('tipos_operacion').update([ {field: newVal }]).eq('id', id).then((resU) => {
16926 |         if (resU.error) showToast('Error: ' + (resU.error.message || 'No se pudo actualizar.'), 'error');
16927 |         else showToast('Actualizado.');
16928 |       });
16929 |     };
16930 |     if (field !== 'usa_intermediario' || !row || !row.codigo) {
16931 |       aplicar();
16932 |       return;
16933 |     }
16934 |     client.from('tipos_operacion').select('id').eq('codigo', row.codigo).eq('usa_intermediario', newVal).neq('id', id).limit(1).then((rDup) => {
16935 |       if (rDup.error) {
16936 |         showToast('Error: ' + (rDup.error.message || 'No se pudo validar.'), 'error');
16937 |         this.checked = !newVal;
16938 |         return;
16939 |       }
16940 |       if (rDup.data && rDup.data.length) {
16941 |         this.checked = !newVal;
16942 |         showToast('Ya existe un tipo con el mismo código y la misma opción Intermediario.', 'error');
16943 |         return;
16944 |       }
16945 |       aplicar();
16946 |     });
16947 |   });
16948 | });
16949 | tbody.querySelectorAll('.btn-editar-tipo-operacion').forEach((btn) => {
16950 |   btn.addEventListener('click', () => {
16951 |     const id = btn.getAttribute('data-id');
16952 |     const row = list.find((r) => String(r.id) === String(id));
16953 |     if (row) openModalTipoOperacion(row);
16954 |   });
16955 | });
16956 | wrapEl.style.display = 'block';
16957 | }).catch(() => {
16958 |   loadingEl.style.display = 'none';
16959 |   if (!silentTipos) wrapEl.style.display = 'block';
16960 | });
16961 | }
16962 |
16963 | function openModalTipoOperacion(registro) {
16964 |   const backdrop = document.getElementById('modal-tipo-operacion-backdrop');
16965 |   const titulo = document.getElementById('modal-tipo-operacion-titulo');
16966 |   const idEl = document.getElementById('tipo-operacion-id');
16967 |   const form = document.getElementById('form-tipo-operacion');
16968 |   if (!backdrop || !titulo || !idEl || !form) return;
16969 |
16970 |   if (registro) {
16971 |     titulo.textContent = 'Editar tipo de operación';
16972 |     idEl.value = registro.id;
16973 |     document.getElementById('tipo-operacion-codigo').value = registro.codigo || '';
16974 |     document.getElementById('tipo-operacion-nombre').value = registro.nombre || '';
16975 |     document.getElementById('tipo-operacion-monedas-in').value = (registro.monedas_in || 'USD').toUpperCase();
16976 |     document.getElementById('tipo-operacion-monedas-out').value = (registro.monedas_out || 'USD').toUpperCase();
16977 |     document.getElementById('tipo-operacion-usa-intermediario').checked = registro.usa_intermediario === true;
16978 |     document.getElementById('tipo-operacion-activo').checked = registro.activo !== false;
16979 |     const im = (registro.icono_modos || 'auto').trim().toLowerCase();
16980 |     const modosVal = im === 'cheque' || im === 'custom' ? im : 'auto';
16981 |     document.querySelectorAll('input[name="tipo-operacion-icono-modos"]').forEach((inp) => { inp.checked = inp.value === modosVal; });
16982 |     const urlI = document.getElementById('tipo-operacion-icono-url');
16983 |     if (urlI) urlI.value = registro.icono_url_publica || '';
16984 |     updateTipoOperacionModalIconoCustomWrap();
16985 |     syncTipoOperacionModalIconoPreview();
16986 |   } else {
16987 |     titulo.textContent = 'Nuevo tipo de operación';
16988 |     idEl.value = '';
16989 |     form.reset();
16990 |     document.getElementById('tipo-operacion-monedas-in').value = 'USD';
16991 |     document.getElementById('tipo-operacion-monedas-out').value = 'USD';
16992 |     document.getElementById('tipo-operacion-usa-intermediario').checked = false;
16993 |     document.getElementById('tipo-operacion-activo').checked = true;
16994 |     document.querySelectorAll('input[name="tipo-operacion-icono-modos"]').forEach((inp) => { inp.checked = inp.value === 'auto'; });
16995 |     const urlN = document.getElementById('tipo-operacion-icono-url');
16996 |     if (urlN) urlN.value = '';
16997 |     updateTipoOperacionModalIconoCustomWrap();
16998 |     syncTipoOperacionModalIconoPreview();
16999 |   }
17000 |   backdrop.classList.add('activo');
17001 | }
17002 |
17003 | function closeModalTipoOperacion() {
17004 |   const backdrop = document.getElementById('modal-tipo-operacion-backdrop');
17005 |   if (backdrop) backdrop.classList.remove('activo');
17006 | }

```

```

17007 |
17008 | function saveTipoOperacion() {
17009 |     const idEl = document.getElementById('tipo-operacion-id');
17010 |     const id = idEl?.value?.trim() || '';
17011 |     const codigo = document.getElementById('tipo-operacion-codigo').value.trim();
17012 |     const nombre = document.getElementById('tipo-operacion-nombre').value.trim();
17013 |     if (!codigo || !nombre) {
17014 |         showToast('Código y nombre son obligatorios.', 'error');
17015 |         return;
17016 |     }
17017 |     const monedaIn = (document.getElementById('tipo-operacion-moneda-in').value || 'USD').toUpperCase();
17018 |     const monedaOut = (document.getElementById('tipo-operacion-moneda-out').value || 'USD').toUpperCase();
17019 |     const monedasPermitidas = ['USD', 'EUR', 'ARS', 'CHEQUE'];
17020 |     if (!monedasPermitidas.includes(monedaIn) || !monedasPermitidas.includes(monedaOut)) {
17021 |         showToast('Moneda IN y OUT deben ser USD, EUR, ARS o Cheque.', 'error');
17022 |         return;
17023 |     }
17024 |     if (monedaIn === 'CHEQUE' && monedaOut !== 'ARS' || monedaOut === 'CHEQUE' && monedaIn !== 'ARS') {
17025 |         showToast('Cheque solo puede combinarse con ARS (el otro lado debe ser ARS).', 'error');
17026 |         return;
17027 |     }
17028 |     const radIcono = document.querySelector('input[name="tipo-operacion-icono-modo"]:checked');
17029 |     const modoRaw = radIcono ? radIcono.value : 'auto';
17030 |     const iconoModo = modoRaw === 'cheque' || modoRaw === 'custom' ? modoRaw : 'auto';
17031 |     const urlIcono = (document.getElementById('tipo-operacion-icono-url') && document.getElementById('tipo-operacion-icono-url').value || '').trim();
17032 |     if (iconoModo === 'custom' && !isHttpsUrlSegura(urlIcono)) {
17033 |         showToast('En modo personalizado indicá una URL https válida o subí una imagen al Storage.', 'error');
17034 |         return;
17035 |     }
17036 |     const payload = {
17037 |         codigo,
17038 |         nombre,
17039 |         moneda_in: monedaIn,
17040 |         moneda_out: monedaOut,
17041 |         usa_intermediario: document.getElementById('tipo-operacion-usa-intermediario').checked,
17042 |         activo: document.getElementById('tipo-operacion-activo').checked,
17043 |         icono_modo: iconoModo,
17044 |         icono_url_publica: iconoModo === 'custom' ? urlIcono : null,
17045 |     };
17046 |     let dupQ = client.from('tipos_operacion').select('id').eq('codigo', codigo).eq('usa_intermediario', payload.usa_intermediario).limit(1);
17047 |     if (id) dupQ = dupQ.neq('id', id);
17048 |     dupQ.then((rDup) => {
17049 |         if (rDup.error) {
17050 |             showToast('Error: ' + (rDup.error.message || 'No se pudo validar. '), 'error');
17051 |             return;
17052 |         }
17053 |         if (rDup.data && rDup.data.length) {
17054 |             showToast('Ya existe un tipo con el mismo código y la misma opción Intermediario. Usá otro código o cambiá el toggle Intermediario.', 'error');
17055 |             return;
17056 |         }
17057 |         const ejecutarGuardado = (payloadFinal) => {
17058 |             const prom = id
17059 |                 ? client.from('tipos_operacion').update(payloadFinal).eq('id', id)
17060 |                 : client.from('tipos_operacion').insert(payloadFinal);
17061 |             return prom.then((res) => {
17062 |                 if (res.error) {
17063 |                     showToast('Error: ' + (res.error.message || 'No se pudo guardar. '), 'error');
17064 |                     return;
17065 |                 }
17066 |                 closeModalTipoOperacion();
17067 |                 loadTiposOperacion();
17068 |                 showToast(id ? 'Tipo de operación actualizado.' : 'Tipo de operación creado.', 'success');
17069 |             });
17070 |         };
17071 |         if (id) {
17072 |             ejecutarGuardado(payload);
17073 |             return;
17074 |         }
17075 |         client.from('tipos_operacion').select('orden_visual').order('orden_visual', { ascending: false }).limit(1).maybeSingle().then((rMax) => {
17076 |             if (rMax.error && supabaseErrorFaltaColumnaOrdenVisualTiposOperacion(rMax.error)) {
17077 |                 ejecutarGuardado(payload);
17078 |                 return;
17079 |             }
17080 |             if (rMax.error) {
17081 |                 showToast('Error: ' + (rMax.error.message || 'No se pudo obtener el orden. Ejecutá en Supabase sql/migracion_tipos_operacion_orden_visual.sql'), 'error');
17082 |                 return;
17083 |             }
17084 |             const max = Number(rMax.data?.orden_visual);
17085 |             payload.orden_visual = (Number.isFinite(max) ? max : 0) + 10;
17086 |             ejecutarGuardado(payload);
17087 |         });
17088 |     });
17089 | }
17090 |
17091 | function setupModalTipoOperacion() {
17092 |     const backdrop = document.getElementById('modal-tipo-operacion-backdrop');
17093 |     const btnClose = document.getElementById('modal-tipo-operacion-close');
17094 |     const btnCancel = document.getElementById('modal-tipo-operacion-cancelar');
17095 |     const form = document.getElementById('form-tipo-operacion');
17096 |     const btnNuevo = document.getElementById('btn-nuevo-tipo-operacion');
17097 |
17098 |     if (btnClose) btnClose.addEventListener('click', closeModalTipoOperacion);
17099 |     if (btnCancel) btnCancel.addEventListener('click', closeModalTipoOperacion);
17100 |     if (backdrop) setupBackdropCloseOnlyOnRealClick(backdrop, closeModalTipoOperacion);
17101 |     if (form) form.addEventListener('submit', (e) => { e.preventDefault(); saveTipoOperacion(); });
17102 |     if (btnNuevo) btnNuevo.addEventListener('click', () => openModalTipoOperacion(null));
17103 |

```

```

17104 | document.querySelectorAll('input[name="tipo-operacion-icone-modo"]').forEach((r) => {
17105 |     r.addEventListener('change', () => {
17106 |         updateTipoOperacionModalIconoCustomWrap();
17107 |         syncTipoOperacionModalIconoPreview();
17108 |     });
17109 | });
17110 | const urlIconoEl = document.getElementById('tipo-operacion-icone-url');
17111 | if (urlIconoEl) urlIconoEl.addEventListener('input', syncTipoOperacionModalIconoPreview);
17112 | ['tipo-operacion-codigo', 'tipo-operacion-nombre'].forEach((id) => {
17113 |     const el = document.getElementById(id);
17114 |     if (el) el.addEventListener('input', syncTipoOperacionModalIconoPreview);
17115 | });
17116 | const chkUsaInt = document.getElementById('tipo-operacion-usa-intermediario');
17117 | if (chkUsaInt) chkUsaInt.addEventListener('change', syncTipoOperacionModalIconoPreview);
17118 | const fileIconoEl = document.getElementById('tipo-operacion-icone-file');
17119 | const btnSubirIcono = document.getElementById('tipo-operacion-icone-subir');
17120 | if (btnSubirIcono && fileIconoEl) {
17121 |     btnSubirIcono.addEventListener('click', () => fileIconoEl.click());
17122 |     fileIconoEl.addEventListener('change', () => {
17123 |         const f = fileIconoEl.files && fileIconoEl.files[0];
17124 |         fileIconoEl.value = '';
17125 |         if (!f) return;
17126 |         subirIconoTipoOperacionStorage(f).then((url) => {
17127 |             if (!url) return;
17128 |             document.querySelectorAll('input[name="tipo-operacion-icone-modo"]').forEach((inp) => { inp.checked = inp.value === 'custom'; });
17129 |             updateTipoOperacionModalIconoCustomWrap();
17130 |             const u = document.getElementById('tipo-operacion-icone-url');
17131 |             if (u) u.value = url;
17132 |             syncTipoOperacionModalIconoPreview();
17133 |             showToast('Imagen subida; URL cargada en el campo.', 'success');
17134 |         });
17135 |     });
17136 | }
17137 | }
17138 |
17139 | // ===== Reglas de negocio CC (menú crítico, permiso abm_reglas_negocio) =====
17140 | let reglasNegocioCacheList = [];
17141 | let tiposOperacionReglasCache = [];
17142 |
17143 | function normalizarCodigoReglaNegocio(s) {
17144 |     if (s == null || typeof s !== 'string') return '';
17145 |     return s.replace(/s*~s*/g, '-').replace(/s+/g, '').trim().toUpperCase();
17146 | }
17147 |
17148 | const REG_NEG_MONTO_ORIGEN_OK = new Set([
17149 |     'mr', 'me', 'monto_transaccion', 'me_prorrrateado', 'mr_prorrrateado',
17150 |     'mr_menos_me', 'monto_efectivo_intermediario', 'comision_intermediario',
17151 | ]);
17152 |
17153 | function reglasNegocioClaveUnicaRow(r) {
17154 |     return [
17155 |         String(r.tipo_operacion_codigo || '').toUpperCase(),
17156 |         r.usa_intermediario === true ? '1' : '0',
17157 |         String(r.entidad_cc || 'cliente').toLowerCase(),
17158 |         String(r.pagador || '').toLowerCase(),
17159 |         String(r.cobrador || '').toLowerCase(),
17160 |         String(r.tipo_transaccion || '').toLowerCase(),
17161 |         coercePgBooleanStrict(r.es_comision) ? '1' : '0',
17162 |         String(r.estado_transaccion || '').toLowerCase(),
17163 |         coercePgBooleanStrict(r.contrapartida_ejecutada) ? '1' : '0',
17164 |         String(Number(r.linea) || 0),
17165 |     ].join('\t');
17166 | }
17167 |
17168 | function reglasNegocioBuscarDuplicado(payload, excluirId) {
17169 |     const k = reglasNegocioClaveUnicaRow(payload);
17170 |     return reglasNegocioCacheList.find((r) => {
17171 |         if (excluirId && String(r.id) === String(excluirId)) return false;
17172 |         return reglasNegocioClaveUnicaRow(r) === k;
17173 |     });
17174 | }
17175 |
17176 | function validarReglaNegocioNegocio(payload, tiposRows) {
17177 |     const errores = [];
17178 |     const advertencias = [];
17179 |     const cod = normalizarCodigoReglaNegocio(payload.tipo_operacion_codigo);
17180 |     if (!cod) errores.push('Código de tipo de operación obligatorio.');
```

```

17201 |     ? String(payload.condicion_estado_comision).trim()
17202 |     : '';
17203 | if (conc && ![ 'par_cliente', 'par_pandy_int' ].includes(conc)) {
17204 |     errores.push('condicion_estado_comision debe ser vacio, par_cliente o par_pandy_int.');
```

```

17205 | }
17206 | if (coercePgBooleanStrict(payload.es_comision) && !conc) {
17207 |     advertencias.push('Es comisión sin condicion_estado_comision: el motor puede no aplicar la fila como esperás (revisá docs).');
```

```

17208 | }
17209 | if (!coercePgBooleanStrict(payload.es_comision) && conc) {
17210 |     advertencias.push('condicion_estado_comision suele usarse solo con es_comision = true.');
```

```

17211 | }
17212 | if (mo === 'monto_efectivo_intermediario' && String(payload.entidad_cc).toLowerCase() !== 'intermediario') {
17213 |     advertencias.push('monto_efectivo_intermediario suele ir con entidad_cc = intermediario (p. ej. CHEQUE-ARS).');
```

```

17214 | }
17215 | if (mo === 'comision_intermediario' && !payload.usa_intermediario) {
17216 |     advertencias.push('comision_intermediario suele usarse con usa_intermediario = true.');
```

```

17217 | }
17218 | const tipoCat = (tiposRows || []).find(
17219 |     (t) => String(t.codigo || '').toUpperCase() === cod && !!t.usa_intermediario === !!payload.usa_intermediario
17220 | );
17221 | if (!tipoCat) {
17222 |     advertencias.push('No hay fila en Tipos de operación para (código + intermediario). Creá el tipo antes de usar estas reglas en órdenes.');
```

```

17223 | } else {
17224 |     const min = (tipoCat.moneda_in || '').toString().toUpperCase().replace('CHEQUE', 'ARS');
17225 |     const mout = (tipoCat.moneda_out || '').toString().toUpperCase().replace('CHEQUE', 'ARS');
```

```

17226 |     const setMon = new Set([min, mout].filter(x => x && x !== 'CHEQUE'));
17227 |     if (mon && setMon.size && !setMon.has(mon) && mon !== 'ARS') {
17228 |         advertencias.push('La moneda de la regla (' + mon + ') no coincide con IN/OUT del tipo en catálogo (' + [...setMon].join(', ') + '). Revisá si es intencional.');
```

```

17229 |     }
17230 | }
17231 | return { errores, advertencias, codigoNorm: cod };
17232 | }
17233 |
17234 | function reglasNegocioRowToForm(r) {
17235 |     return {
17236 |         id: r.id,
17237 |         tipo_operacion_codigo: r.tipo_operacion_codigo,
17238 |         usa_intermediario: r.usa_intermediario === true,
17239 |         entidad_cc: r.entidad_cc || 'cliente',
17240 |         pagador: r.pagador,
17241 |         cobrador: r.cobrador,
17242 |         tipo_transaccion: r.tipo_transaccion,
17243 |         es_comision: coercePgBooleanStrict(r.es_comision),
17244 |         estado_transaccion: r.estado_transaccion,
17245 |         contrapartida_ejecutada: coercePgBooleanStrict(r.contrapartida_ejecutada),
17246 |         linea: Number(r.linea) || 0,
17247 |         moneda: (r.moneda || 'USD').toUpperCase(),
17248 |         signo: Number(r.signo) === -1 ? -1 : 1,
17249 |         monto_origen: r.monto_origen,
17250 |         incluir_en_detalle: r.incluir_en_detalle !== false,
17251 |         concepto_leyenda: r.concepto_leyenda || '',
17252 |         condicion_estado_comision: r.condicion_estado_comision || '',
17253 |     };
17254 | }
17255 |
17256 | function reglasNegocioLeerFormulario() {
17257 |     const id = (document.getElementById('regla-negocio-id') && document.getElementById('regla-negocio-id').value || '').trim();
17258 |     return {
17259 |         id: id || null,
17260 |         tipo_operacion_codigo: (document.getElementById('regla-negocio-codigo') && document.getElementById('regla-negocio-codigo').value) || '',
17261 |         usa_intermediario: !!document.getElementById('regla-negocio-usa-intermediario') && document.getElementById('regla-negocio-usa-intermediario').checked,
17262 |         entidad_cc: (document.getElementById('regla-negocio-entidad-cc') && document.getElementById('regla-negocio-entidad-cc').value) || 'cliente',
17263 |         pagador: (document.getElementById('regla-negocio-pagador') && document.getElementById('regla-negocio-pagador').value) || '',
17264 |         cobrador: (document.getElementById('regla-negocio-cobrador') && document.getElementById('regla-negocio-cobrador').value) || '',
17265 |         tipo_transaccion: (document.getElementById('regla-negocio-tipo-transaccion') && document.getElementById('regla-negocio-tipo-transaccion').value) || '',
17266 |         es_comision: !!document.getElementById('regla-negocio-es-comision') && document.getElementById('regla-negocio-es-comision').checked,
17267 |         estado_transaccion: (document.getElementById('regla-negocio-estado-transaccion') && document.getElementById('regla-negocio-estado-transaccion').value) || '',
17268 |         contrapartida_ejecutada: !!document.getElementById('regla-negocio-contrapartida') && document.getElementById('regla-negocio-contrapartida').checked,
17269 |         linea: parseInt(document.getElementById('regla-negocio-linea') && document.getElementById('regla-negocio-linea').value, 10),
17270 |         moneda: (document.getElementById('regla-negocio-moneda') && document.getElementById('regla-negocio-moneda').value) || 'USD',
17271 |         signo: parseInt(document.getElementById('regla-negocio-signo') && document.getElementById('regla-negocio-signo').value, 10),
17272 |         monto_origen: (document.getElementById('regla-negocio-monto-origen') && document.getElementById('regla-negocio-monto-origen').value) || '',
17273 |         incluir_en_detalle: !!document.getElementById('regla-negocio-incluir-detalle') && document.getElementById('regla-negocio-incluir-detalle').checked,
17274 |         concepto_leyenda: (document.getElementById('regla-negocio-concepto') && document.getElementById('regla-negocio-concepto').value.trim()) || '',
17275 |         condicion_estado_comision: (document.getElementById('regla-negocio-condicion-comision') && document.getElementById('regla-negocio-condicion-comision').value) || '',
17276 |     };
17277 | }
17278 |
17279 | function reglasNegocioRellenarFormulario(r) {
17280 |     const d = reglasNegocioRowToForm(r);
17281 |     const idEl = document.getElementById('regla-negocio-id');
17282 |     if (idEl) idEl.value = d.id || '';
17283 |     const set = (id, val) => {
17284 |         const el = document.getElementById(id);
17285 |         if (el) el.value = val !== null ? val : '';
17286 |     };
17287 |     set('regla-negocio-codigo', d.tipo_operacion_codigo);
17288 |     const usa = document.getElementById('regla-negocio-usa-intermediario');
17289 |     if (usa) usa.checked = d.usa_intermediario;
17290 |     set('regla-negocio-entidad-cc', d.entidad_cc);
17291 |     set('regla-negocio-pagador', d.pagador);
17292 |     set('regla-negocio-cobrador', d.cobrador);
17293 |     set('regla-negocio-tipo-transaccion', d.tipo_transaccion);
17294 |     const esC = document.getElementById('regla-negocio-es-comision');
17295 |     if (esC) esC.checked = d.es_comision;
17296 |     set('regla-negocio-estado-transaccion', d.estado_transaccion);
17297 |     const ct = document.getElementById('regla-negocio-contrapartida');
```

```

17298| if (ct) ct.checked = d.contrapartida_ejecutada;
17299| set('regla-negocio-linea', String(d.linea));
17300| set('regla-negocio-monedas', d.monedas);
17301| set('regla-negocio-signo', String(d.signo));
17302| set('regla-negocio-monto_origen', d.monto_origen);
17303| const inc = document.getElementById('regla-negocio-incluir-detalle');
17304| if (inc) inc.checked = d.incluir_en_detalle;
17305| set('regla-negocio-concepto', d.concepto_leyenda);
17306| set('regla-negocio-condicion-comision', d.condicion_estado_comision || '');
17307| const codEl = document.getElementById('regla-negocio-codigo');
17308| const usaEl = document.getElementById('regla-negocio-usa-intermediario');
17309| const isEdit = !!d.id;
17310| if (codEl) {
17311|   codEl.readOnly = isEdit;
17312|   codEl.title = isEdit ? 'No se puede cambiar el tipo en una fila existente (evita duplicados). Creá una fila nueva o replicá matriz.' : '';
17313| }
17314| if (usaEl) {
17315|   usaEl.disabled = isEdit;
17316|   usaEl.title = isEdit ? 'Creá una fila nueva para otro valor de intermediario.' : '';
17317| }
17318| }
17319|
17320| function cerrarModalReglaNegocio() {
17321|   const b = document.getElementById('modal-regla-negocio-backdrop');
17322|   if (b) b.classList.remove('activo');
17323| }
17324|
17325| function abrirModalReglaNegocio(row, defaults) {
17326|   const backdrop = document.getElementById('modal-regla-negocio-backdrop');
17327|   const titulo = document.getElementById('modal-regla-negocio-titulo');
17328|   if (!backdrop || !titulo) return;
17329|   if (row) {
17330|     titulo.textContent = 'Editar regla de negocio';
17331|     reglasNegocioRellenarFormulario(row);
17332|   } else {
17333|     titulo.textContent = 'Nueva regla de negocio';
17334|     const fCod = document.getElementById('reglas-negocio-filtro-codigo');
17335|     const fUsa = document.getElementById('reglas-negocio-filtro-usa-int');
17336|     const defCod = (defaults && defaults.codigo) || (fCod && fCod.value) || '';
17337|     const defUsa = defaults && typeof defaults.usaIntermediario === 'boolean'
17338|       ? defaults.usaIntermediario
17339|       : (fUsa && fUsa.value === 'true' ? true : fUsa && fUsa.value === 'false' ? false : false);
17340|     reglasNegocioRellenarFormulario({
17341|       tipo_operacion_codigo: defCod,
17342|       usa_intermediario: defUsa,
17343|       entidad_cc: 'cliente',
17344|       pagador: 'cliente',
17345|       cobrador: 'pandy',
17346|       tipo_transaccion: 'ingreso',
17347|       es_comision: false,
17348|       estado_transaccion: 'ejecutada',
17349|       contrapartida_ejecutada: false,
17350|       linea: 0,
17351|       moneda: 'USD',
17352|       signo: 1,
17353|       monto_origen: 'monto_transaccion',
17354|       incluir_en_detalle: true,
17355|       concepto_leyenda: 'cobro_realizado',
17356|       condicion_estado_comision: '',
17357|     });
17358|     const idEl = document.getElementById('regla-negocio-id');
17359|     if (idEl) idEl.value = '';
17360|     const codEl = document.getElementById('regla-negocio-codigo');
17361|     const usaEl = document.getElementById('regla-negocio-usa-intermediario');
17362|     if (codEl) codEl.readOnly = false;
17363|     if (usaEl) usaEl.disabled = false;
17364|   }
17365|   backdrop.classList.add('activo');
17366| }
17367|
17368| function reglasNegocioPayloadParaDb(form) {
17369|   return {
17370|     tipo_operacion_codigo: normalizarCodigoReglaNegocio(form.tipo_operacion_codigo),
17371|     usa_intermediario: !!form.usa_intermediario,
17372|     entidad_cc: String(form.entidad_cc || 'cliente').toLowerCase(),
17373|     pagador: String(form.pagador || '').toLowerCase(),
17374|     cobrador: String(form.cobrador || '').toLowerCase(),
17375|     tipo_transaccion: String(form.tipo_transaccion || '').toLowerCase(),
17376|     es_comision: !!form.es_comision,
17377|     estado_transaccion: String(form.estado_transaccion || '').toLowerCase(),
17378|     contrapartida_ejecutada: !!form.contrapartida_ejecutada,
17379|     linea: Number(form.linea) || 0,
17380|     moneda: String(form.moneda || 'USD').toUpperCase(),
17381|     signo: Number(form.signo) === -1 ? -1 : 1,
17382|     monto_origen: String(form.monto_origen || '').toLowerCase(),
17383|     incluir_en_detalle: !!form.incluir_en_detalle,
17384|     concepto_leyenda: String(form.concepto_leyenda || '').trim(),
17385|     condicion_estado_comision: (form.condicion_estado_comision && String(form.condicion_estado_comision).trim())
17386|       ? String(form.condicion_estado_comision).trim()
17387|       : null,
17388|   };
17389| }
17390|
17391| function ejecutarGuardarReglaNegocio(form) {
17392|   const payload = reglasNegocioPayloadParaDb(form);
17393|   const id = form.id;
17394|   const dup = reglasNegocioBuscarDuplicado(payload, id);

```



```

17395|   if (dup) {
17396|       showToast('Ya existe otra fila con la misma clave (tipo, int., entidad, pagador/cobrador, comisión, estado, contrapartida, línea).', 'error');
17397|       return;
17398|   }
17399|   const prom = id
17400|     ? client.from('reglas_de_negocio').update(payload).eq('id', id)
17401|     : client.from('reglas_de_negocio').insert(payload);
17402|   prom.then((res) => {
17403|     if (res.error) {
17404|       showToast('Error al guardar: ' + (res.error.message || '') + (res.error.code === '23505' ? ' (clave duplicada)' : ''), 'error');
17405|       return;
17406|     }
17407|     cerrarModalReglaNegocio();
17408|     showToast(id ? 'Regla actualizada.' : 'Regla creada.', 'success');
17409|     loadReglasNegocioVista();
17410|   });
17411| }
17412|
17413| function intentarGuardarReglaNegocioFormulario() {
17414|   const form = reglasNegocioLeerFormulario();
17415|   if (!form.concepto_leyenda) {
17416|     showToast('Concepto leyenda obligatorio.', 'error');
17417|     return;
17418|   }
17419|   const payload = reglasNegocioPayloadParaDb(form);
17420|   const v = validarReglaNegocioNegocio(payload, tiposOperacionReglasCache);
17421|   if (v.errores.length) {
17422|     showToast(v.errores[0], 'error');
17423|     return;
17424|   }
17425|   const run = () => ejecutarGuardarReglaNegocio(form);
17426|   if (v.advertencias.length) {
17427|     showConfirm(
17428|       v.advertencias.join('\n') + '\n\n¿Guardar de todas formas?',
17429|       'Sí, guardar',
17430|       run,
17431|       () => {}
17432|     );
17433|     return;
17434|   }
17435|   run();
17436| }
17437|
17438| function eliminarReglaNegocio(id) {
17439|   if (!id) return;
17440|   showConfirm(
17441|     'Eliminar esta fila de reglas? Las órdenes existentes seguirán en base; al sincronizar CC el motor puede comportarse distinto o usar fallbacks.',
17442|     'Sí, eliminar',
17443|     () => {
17444|       client.from('reglas_de_negocio').delete().eq('id', id).then((res) => {
17445|         if (res.error) showToast('Error: ' + (res.error.message || ''), 'error');
17446|         else {
17447|           showToast('Fila eliminada.', 'success');
17448|           loadReglasNegocioVista();
17449|         }
17450|       });
17451|     },
17452|     () => {}
17453|   );
17454| }
17455|
17456| function reglasNegocioFiltrarLista() {
17457|   const selCod = document.getElementById('reglas-negocio-filtro-codigo');
17458|   const selUsa = document.getElementById('reglas-negocio-filtro-usa-int');
17459|   const cod = selCod && selCod.value ? String(selCod.value).toUpperCase() : '';
17460|   const usaVal = selUsa ? selUsa.value : 'all';
17461|   return reglasNegocioCacheList.filter((r) => {
17462|     if (cod && String(r.tipo_operacion_codigo || '').toUpperCase() !== cod) return false;
17463|     if (usaVal === 'true' && r.usa_intermediario !== true) return false;
17464|     if (usaVal === 'false' && r.usa_intermediario === true) return false;
17465|     return true;
17466|   });
17467| }
17468|
17469| function renderReglasNegocioTabla() {
17470|   const tbody = document.getElementById('reglas-negocio-tbody');
17471|   if (!tbody) return;
17472|   const list = reglasNegocioFiltrarLista().slice().sort((a, b) => {
17473|     const ca = String(a.tipo_operacion_codigo || '').localeCompare(String(b.tipo_operacion_codigo || ''));
17474|     if (ca !== 0) return ca;
17475|     if (!!a.usa_intermediario !== !!b.usa_intermediario) return a.usa_intermediario ? 1 : -1;
17476|     return reglasNegocioClaveUnicaRow(a).localeCompare(reglasNegocioClaveUnicaRow(b));
17477|   });
17478|   const canAbm = userPermissions.includes('abm_reglas_negocio');
17479|   tbody.innerHTML = list.map((r) => {
17480|     const intLabel = r.usa_intermediario ? 'Sí' : 'No';
17481|     const com = coercePgBooleanStrict(r.es_comision) ? 'Sí' : 'No';
17482|     const det = r.incluir_en_detalle !== false ? 'Sí' : 'No';
17483|     const cont = coercePgBooleanStrict(r.contrapartida_ejecutada) ? 'Sí' : 'No';
17484|     const btnEdit = canAbm
17485|       ? `<button type="button" class="btn-editar btn-regla-negocio-editar" data-id="${escapeHtml(String(r.id))}"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" width="18" height="18"><path d="M11 4H4a2 2 0 0-2 2v14a2 2 0 02 2h14a2 2 0 02-2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013 3L12 15l-4 1 1-4 9.5-9.5z"/></svg></span>Editar</button>`
17486|       : '';
17487|     const btnDel = canAbm
17488|       ? `<button type="button" class="btn-secondary btn-regla-negocio-eliminar" data-id="${escapeHtml(String(r.id))}" title="Eliminar fila"><span class="btn-icon"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" width="18" height="18"><polyline points="3 6 5 6 21 6"/><path d="M19 6v14a2 2 0 01-2 2H7a2 2 0 01-2-2V6m3 0V4a2`

```

```

2 0 012-2h4a2 2 0 012 2v2"/></svg></span></button>`
17489 : `;
17490 return `<tr data-id="${escapeHtml(String(r.id))}">
17491 <td><code>${escapeHtml(String(r.tipo_operacion_codigo || ''))}</code></td>
17492 <td>${escapeHtml(intLabel)}</td>
17493 <td>${escapeHtml(String(r.entidad_cc || ''))}</td>
17494 <td>${escapeHtml(String(r.pagador || ''))}</td>
17495 <td>${escapeHtml(String(r.cobrador || ''))}</td>
17496 <td>${escapeHtml(String(r.tipo_transaccion || ''))}</td>
17497 <td>${escapeHtml(com)}</td>
17498 <td>${escapeHtml(String(r.estado_transaccion || ''))}</td>
17499 <td>${escapeHtml(cont)}</td>
17500 <td>${escapeHtml(String(r.linea != null ? r.linea : ''))}</td>
17501 <td>${escapeHtml(String(r.moneda || ''))}</td>
17502 <td>${escapeHtml(String(r.signo != null ? r.signo : ''))}</td>
17503 <td><code style="font-size:0.75rem;">${escapeHtml(String(r.monto_origen || ''))}</code></td>
17504 <td>${escapeHtml(String(r.concepto_leyenda || ''))}</td>
17505 <td>${escapeHtml(det)}</td>
17506 <td><code style="font-size:0.75rem;">${escapeHtml(String(r.condicion_estado_comision || '-'))}</code></td>
17507 <td style="white-space: nowrap;">${btnEdit}${btnDel}</td>
17508 </tr>`;
17509 }).join('');
17510 tbody.querySelectorAll('.btn-regla-negocio-editar').forEach((btn) => {
17511 btn.addEventListener('click', () => {
17512 const rid = btn.getAttribute('data-id');
17513 const row = reglasNegocioCacheList.find((x) => String(x.id) === String(rid));
17514 if (row) abrirModalReglaNegocio(row);
17515 });
17516 });
17517 tbody.querySelectorAll('.btn-regla-negocio-eliminar').forEach((btn) => {
17518 btn.addEventListener('click', () => eliminarReglaNegocio(btn.getAttribute('data-id')));
17519 });
17520 }
17521
17522 function reglasNegocioRellenarFiltroCodigos() {
17523 const sel = document.getElementById('reglas-negocio-filtro-codigo');
17524 if (!sel) return;
17525 const prev = sel.value;
17526 const set = new Set();
17527 reglasNegocioCacheList.forEach((r) => {
17528 if (r.tipo_operacion_codigo) set.add(String(r.tipo_operacion_codigo).toUpperCase());
17529 });
17530 const sorted = [...set].sort((a, b) => a.localeCompare(b));
17531 sel.innerHTML = '<option value="">Todos</option>' + sorted.map((c) => `<option value="${escapeHtml(c)}">${escapeHtml(c)}</option>`).join('');
17532 if (prev && sorted.includes(prev)) sel.value = prev;
17533 }
17534
17535 function reglasNegocioRellenarSelectOrigenReplica() {
17536 const sel = document.getElementById('replicar-reglas-origen');
17537 if (!sel) return;
17538 const pairs = new Map();
17539 reglasNegocioCacheList.forEach((r) => {
17540 const cod = String(r.tipo_operacion_codigo || '').toUpperCase();
17541 const k = cod + '\t' + (r.usa_intermediario === true ? '1' : '0');
17542 if (!pairs.has(k)) pairs.set(k, { codigo: cod, usa: r.usa_intermediario === true });
17543 });
17544 const list = [...pairs.values()].sort((a, b) => {
17545 const c = a.codigo.localeCompare(b.codigo);
17546 return c !== 0 ? c : (a.usa === b.usa ? 0 : a.usa ? 1 : -1);
17547 });
17548 sel.innerHTML = list.map((p) => {
17549 const val = p.codigo + '|' + (p.usa ? '1' : '0');
17550 const lab = p.codigo + (p.usa ? ' con intermediario' : ' sin intermediario');
17551 return `<option value="${escapeHtml(val)}">${escapeHtml(lab)}</option>`;
17552 }).join('');
17553 }
17554
17555 function cerrarModalReplicarReglas() {
17556 const b = document.getElementById('modal-replicar-reglas-backdrop');
17557 if (b) b.classList.remove('activo');
17558 }
17559
17560 function ejecutarReplicarReglasNegocio() {
17561 const sel = document.getElementById('replicar-reglas-origen');
17562 const inpDest = document.getElementById('replicar-reglas-destino-codigo');
17563 const chkDest = document.getElementById('replicar-reglas-destino-usa-int');
17564 if (!sel || !inpDest || !chkDest) return;
17565 const raw = sel.value || '';
17566 const parts = raw.split('|');
17567 const srcCod = (parts[0] || '').trim().toUpperCase();
17568 const srcUsa = parts[1] === '1';
17569 const destCod = normalizarCodigoReglaNegocio(inpDest.value);
17570 const destUsa = !chkDest.checked;
17571 if (!srcCod) {
17572 showToast('Elegí un origen.', 'error');
17573 return;
17574 }
17575 if (!destCod) {
17576 showToast('Indicá el código destino (ej. EUR-USD).', 'error');
17577 return;
17578 }
17579 if (destCod === srcCod && destUsa === srcUsa) {
17580 showToast('Origen y destino no pueden ser idénticos.', 'error');
17581 return;
17582 }
17583 const srcRows = reglasNegocioCacheList.filter(
17584 (r) => String(r.tipo_operacion_codigo || '').toUpperCase() === srcCod && !!r.usa_intermediario === srcUsa

```

```

17585 | });
17586 | if (!srcRows.length) {
17587 |   showToast('No hay filas en caché para el origen elegido.', 'error');
17588 |   return;
17589 | }
17590 | const yaDest = reglasNegocioCacheList.some(
17591 |   (r) => String(r.tipo_operacion_codigo || '').toUpperCase() === destCod && !!r.usa_intermediario === destUsa
17592 | );
17593 | if (yaDest) {
17594 |   showToast('El destino ya tiene reglas. Eliminé o edité manualmente antes de replicar (evitamos sobrescribir sin control).', 'error');
17595 |   return;
17596 | }
17597 | const tipoOk = tiposOperacionReglasCache.some(
17598 |   (t) => String(t.codigo || '').toUpperCase() === destCod && !!t.usa_intermediario === destUsa
17599 | );
17600 | if (!tipoOk) {
17601 |   showConfirm(
17602 |     'No hay un tipo de operación en el catálogo para (código + intermediario) destino. Creá el tipo primero en "Tipos de operación".\n\n¿Ir igual con la réplica de reglas?',
17603 |     'Sí, replicar igual',
17604 |     () => reglasNegocioInsertarReplicaBatch(srcRows, destCod, destUsa),
17605 |     () => {}
17606 |   );
17607 |   return;
17608 | }
17609 | showConfirm(
17610 |   'Se insertarán ' + srcRows.length + ' filas para ' + destCod + (destUsa ? ' (con intermediario).' : ' (sin intermediario).') + ' Revisá monedas/signos después.',
17611 |   'Replicar',
17612 |   () => reglasNegocioInsertarReplicaBatch(srcRows, destCod, destUsa),
17613 |   () => {}
17614 | );
17615 | }
17616 | }
17617 | function reglasNegocioInsertarReplicaBatch(srcRows, destCod, destUsa) {
17618 |   const batch = srcRows.map((r) => ({
17619 |     tipo_operacion_codigo: destCod,
17620 |     usa_intermediario: destUsa,
17621 |     entidad_cc: r.entidad_cc || 'cliente',
17622 |     pagador: r.pagador,
17623 |     cobrador: r.cobrador,
17624 |     tipo_transaccion: r.tipo_transaccion,
17625 |     es_comision: r.es_comision,
17626 |     estado_transaccion: r.estado_transaccion,
17627 |     contrapartida_ejecutada: r.contrapartida_ejecutada,
17628 |     linea: r.linea,
17629 |     moneda: r.moneda,
17630 |     signo: r.signo,
17631 |     monto_origen: r.monto_origen,
17632 |     incluir_en_detalle: r.incluir_en_detalle !== false,
17633 |     concepto_leyenda: r.concepto_leyenda,
17634 |     condicion_estado_comision: r.condicion_estado_comision || null,
17635 |   }));
17636 |   const chunk = 80;
17637 |   function send(i) {
17638 |     if (i >= batch.length) {
17639 |       cerrarModalReplicarReglas();
17640 |       showToast('Replicación completada (' + batch.length + ' filas).', 'success');
17641 |       loadReglasNegocioVista();
17642 |       return;
17643 |     }
17644 |     const slice = batch.slice(i, i + chunk);
17645 |     client.from('reglas_de_negocio').insert(slice).then((res) => {
17646 |       if (res.error) {
17647 |         showToast('Error al replicar: ' + (res.error.message || ''), 'error');
17648 |         return;
17649 |       }
17650 |       send(i + chunk);
17651 |     });
17652 |   }
17653 |   send(0);
17654 | }
17655 | }
17656 | function loadReglasNegocioVista() {
17657 |   const loadingEl = document.getElementById('reglas-negocio-loading');
17658 |   const wrapEl = document.getElementById('reglas-negocio-tabla-wrap');
17659 |   const tbody = document.getElementById('reglas-negocio-tbody');
17660 |   const btnNueva = document.getElementById('reglas-negocio-btn-nueva');
17661 |   const btnRep = document.getElementById('reglas-negocio-btn-replicar');
17662 |   if (!loadingEl || !wrapEl || !tbody) return;
17663 |   const canAbm = userPermissions.includes('abm_reglas_negocio');
17664 |   if (btnNueva) btnNueva.style.display = canAbm ? '' : 'none';
17665 |   if (btnRep) btnRep.style.display = canAbm ? '' : 'none';
17666 |   if (!canAbm) {
17667 |     loadingEl.style.display = 'none';
17668 |     wrapEl.style.display = 'block';
17669 |     tbody.innerHTML = '<tr><td colspan="17">No tenés permiso <code>abm_reglas_negocio</code>. Un administrador puede otorgarlo en Seguridad → permisos por rol.</td></tr>';
17670 |     return;
17671 |   }
17672 |   const silentReglas = isPandiBackgroundRefresh();
17673 |   if (!silentReglas) {
17674 |     loadingEl.style.display = 'block';
17675 |     wrapEl.style.display = 'none';
17676 |     tbody.innerHTML = '';
17677 |   }
17678 |   return client.from('reglas_de_negocio').select('*').order('tipo_operacion_codigo').order('usa_intermediario').then((rR) => {
17679 |     if (rR.error) {
17680 |       loadingEl.style.display = 'none';
17681 |       const msg = rR.error.message || '';

```

```

17682 |     showToast('Error: ' + msg, 'error');
17683 |     if (!silentReglas) {
17684 |         wrapEl.style.display = 'block';
17685 |         tbody.innerHTML = '<tr><td colspan="17">Error: ' + escapeHtml(msg) + '</td></tr>';
17686 |     }
17687 |     return Promise.resolve();
17688 | }
17689 | reglasNegocioCacheList = rR.data || [];
17690 | return tiposOperacionFetchConFallbackOrdenVisual(
17691 |     () => client.from('tipos_operacion').select('codigo, usa_intermediario, moneda_in, moneda_out, orden_visual').order('orden_visual', { ascending: true }).order('codigo').
order('usa_intermediario').order('id'),
17692 |     () => client.from('tipos_operacion').select('codigo, usa_intermediario, moneda_in, moneda_out').order('codigo').order('usa_intermediario').order('id'),
17693 | ).then((rT) => {
17694 |     loadingEl.style.display = 'none';
17695 |     wrapEl.style.display = 'block';
17696 |     if (rT.error) {
17697 |         const msg = rT.error.message || '';
17698 |         showToast('Error: ' + msg, 'error');
17699 |         if (!silentReglas) tbody.innerHTML = '<tr><td colspan="17">Error: ' + escapeHtml(msg) + '</td></tr>';
17700 |         return;
17701 |     }
17702 |     tiposOperacionReglasCache = rT.data || [];
17703 |     reglasNegocioRellenarFiltroCodigos();
17704 |     renderReglasNegocioTabla();
17705 | });
17706 | }).catch(() => {
17707 |     loadingEl.style.display = 'none';
17708 |     if (!silentReglas) wrapEl.style.display = 'block';
17709 | });
17710 | }
17711 |
17712 | function setupModalReglasNegocio() {
17713 |     const back = document.getElementById('modal-regla-negocio-backdrop');
17714 |     const backRep = document.getElementById('modal-replicar-reglas-backdrop');
17715 |     const form = document.getElementById('form-regla-negocio');
17716 |     if (back) {
17717 |         const c = document.getElementById('modal-regla-negocio-close');
17718 |         const ca = document.getElementById('modal-regla-negocio-cancelar');
17719 |         if (c) c.addEventListener('click', cerrarModalReglaNegocio);
17720 |         if (ca) ca.addEventListener('click', cerrarModalReglaNegocio);
17721 |         setupBackdropCloseOnlyOnRealClick(back, cerrarModalReglaNegocio);
17722 |     }
17723 |     if (form) form.addEventListener('submit', (e) => { e.preventDefault(); intentarGuardarReglaNegocioFormulario(); });
17724 |     const btnRef = document.getElementById('reglas-negocio-btn-refrescar');
17725 |     if (btnRef) btnRef.addEventListener('click', () => loadReglasNegocioVista());
17726 |     const btnNueva = document.getElementById('reglas-negocio-btn-nueva');
17727 |     if (btnNueva) btnNueva.addEventListener('click', () => abrirModalReglaNegocio(null));
17728 |     const selCod = document.getElementById('reglas-negocio-filtro-codigo');
17729 |     const selUsa = document.getElementById('reglas-negocio-filtro-usa-int');
17730 |     if (selCod) selCod.addEventListener('change', () => renderReglasNegocioTabla());
17731 |     if (selUsa) selUsa.addEventListener('change', () => renderReglasNegocioTabla());
17732 |     const btnRep = document.getElementById('reglas-negocio-btn-replicar');
17733 |     if (btnRep) {
17734 |         btnRep.addEventListener('click', () => {
17735 |             reglasNegocioRellenarSelectOrigenReplica();
17736 |             const inp = document.getElementById('replicar-reglas-destino-codigo');
17737 |             if (inp) inp.value = '';
17738 |             const chk = document.getElementById('replicar-reglas-destino-usa-int');
17739 |             if (chk) chk.checked = false;
17740 |             if (backRep) backRep.classList.add('activo');
17741 |         });
17742 |     }
17743 |     if (backRep) {
17744 |         const cr = document.getElementById('modal-replicar-reglas-close');
17745 |         const cc = document.getElementById('modal-replicar-reglas-cancelar');
17746 |         const bex = document.getElementById('replicar-reglas-btn-ejecutar');
17747 |         if (cr) cr.addEventListener('click', cerrarModalReplicarReglas);
17748 |         if (cc) cc.addEventListener('click', cerrarModalReplicarReglas);
17749 |         setupBackdropCloseOnlyOnRealClick(backRep, cerrarModalReplicarReglas);
17750 |         if (bex) bex.addEventListener('click', () => ejecutarReplicarReglasNegocio());
17751 |     }
17752 | }
17753 |
17754 |
17755 | /** Configuración de vistas: [menuId, vistaId, título, permiso de vista]. Orden del menú. */
17756 | const VIEWS_CONFIG = [
17757 |     ['menu-inicio', 'vista-inicio', 'Panel de Control', 'ver_inicio'],
17758 |     ['menu-ordenes', 'vista-ordenes', 'Órdenes', 'ver_ordenes'],
17759 |     ['menu-cajas', 'vista-cajas', 'Cajas', 'ver_cajas'],
17760 |     ['menu-clientes', 'vista-clientes', 'Clientes', 'ver_clientes'],
17761 |     ['menu-intermediarios', 'vista-intermediarios', 'Intermediarios', 'ver_intermediarios'],
17762 |     ['menu-tipos-operacion', 'vista-tipos-operacion', 'Tipos de operación', 'abm_tipos_operacion'],
17763 |     ['menu-cuenta-corriente', 'vista-cuenta-corriente', 'Cuenta corriente', 'ver_cuenta_corriente'],
17764 |     ['menu-reglas-negocio', 'vista-reglas-negocio', 'Reglas de negocio (CC)', 'abm_reglas_negocio'],
17765 |     ['menu-configuracion-empresa', 'vista-configuracion-empresa', 'Empresa / marca', 'abm_configuracion_empresa'],
17766 |     ['menu-seguridad', 'vista-seguridad', 'Seguridad', 'ver_seguridad'],
17767 | ];
17768 |
17769 | function hasAnyViewPermission() {
17770 |     return VIEWS_CONFIG.some((r) => userPermissions.includes(r[3]));
17771 | }
17772 |
17773 | function canViewVista(vistaId) {
17774 |     if (!hasAnyViewPermission()) return true; // Sin migración de vistas: ver todo
17775 |     const row = VIEWS_CONFIG.find((r) => r[1] === vistaId);
17776 |     if (!row) return true;
17777 |     return userPermissions.includes(row[3]);

```

```

17778 }
17779
17780 function getFirstAllowedView() {
17781   const row = VIEWS_CONFIG.find((r) => userPermissions.includes(r[3]));
17782   return row ? [row[1], row[2]] : ['vista-inicio', 'Panel de Control'];
17783 }
17784
17785 function applyVistasMenuVisibility() {
17786   const useVistasPermisos = hasAnyViewPermission();
17787   VIEWS_CONFIG.forEach(([, menuId, , perm]) => {
17788     const menuEl = document.getElementById(menuId);
17789     if (!menuEl) return;
17790     menuEl.style.display = !useVistasPermisos || userPermissions.includes(perm) ? '' : 'none';
17791   });
17792 }
17793
17794 function setupVistasMenu() {
17795   VIEWS_CONFIG.forEach(([, menuId, vistaId, title]) => {
17796     const menuEl = document.getElementById(menuId);
17797     if (!menuEl) return;
17798     menuEl.addEventListener('click', (e) => {
17799       e.preventDefault();
17800       showView(vistaId, title);
17801     });
17802   });
17803 }
17804
17805 function updateSessionActivity() {
17806   const now = Date.now();
17807   if (now - lastActivityUpdate < SESSION_ACTIVITY_THROTTLE_MS) return;
17808   lastActivityUpdate = now;
17809   lastActivityTime = now;
17810 }
17811
17812 /** Indica si la vista actual tiene alguna sección expandible abierta (detalle de orden, menú/rol en Seguridad, etc.). Si es true, no conviene refrescar porque se colapsaría.
17813 */
17814 function hasExpandedSectionInCurrentView() {
17815   if (currentVistaId === 'vista-ordenes' && transaccionesOrdenIdActual) return true;
17816   const vistaSeguridad = document.getElementById('vista-seguridad');
17817   if (vistaSeguridad && vistaSeguridad.style.display === 'block') {
17818     if (vistaSeguridad.querySelector('.seguridad-permisos-menu-colapsable:not(.collapsed)')) return true;
17819     if (vistaSeguridad.querySelector('.seguridad-permisos-rol.expanded')) return true;
17820   }
17821   return false;
17822 }
17823
17824 /** Refresco suave de la vista actual cada REFRESH_DATA_INTERVAL_MS. No recarga la página; solo vuelve a pedir los datos de la vista. No se ejecuta si hay un modal abierto ni
17825 si hay una sección expandida (para no colapsar). Durante este ciclo `isPandiBackgroundRefresh()` es true: sin spinners ni tablas vacías. */
17826 function refreshCurrentViewData() {
17827   if (typeof pandiModoReducidoOffline !== 'undefined' && pandiModoReducidoOffline) return;
17828   if (document.querySelector('.modal-backdrop.activo')) return;
17829   if (hasExpandedSectionInCurrentView()) return;
17830   const loaders = {
17831     'vista-inicio': loadInicio,
17832     'vista-ordenes': loadOrdenes,
17833     'vista-cajas': loadCajas,
17834     'vista-clientes': loadClientes,
17835     'vista-intermediarios': loadIntermediarios,
17836     'vista-tipos-operacion': loadTiposOperacion,
17837     'vista-cuenta-corriente': loadCuentaCorriente,
17838     'vista-reglas-negocio': loadReglasNegocioVista,
17839     'vista-configuracion-empresa': loadConfiguracionEmpresa,
17840     'vista-seguridad': loadSeguridad,
17841   };
17842   const fn = loaders[currentVistaId];
17843   if (typeof fn !== 'function') return;
17844   pandiBackgroundRefreshActive = true;
17845   try {
17846     const ret = fn();
17847     Promise.resolve(ret)
17848       .catch(() => {})
17849       .finally(() => {
17850         pandiBackgroundRefreshActive = false;
17851       });
17852   } catch (e) {
17853     pandiBackgroundRefreshActive = false;
17854   }
17855 }
17856
17857 function startSessionTimeoutCheck() {
17858   if (sessionCheckIntervalId) clearInterval(sessionCheckIntervalId);
17859   lastActivityTime = Date.now();
17860   lastActivityUpdate = lastActivityTime;
17861   const events = ['mousedown', 'mousemove', 'keydown', 'scroll', 'touchstart', 'click'];
17862   events.forEach((ev) => document.addEventListener(ev, updateSessionActivity));
17863   sessionCheckIntervalId = setInterval(() => {
17864     if (sessionTimeoutMinutes <= 0) return;
17865     const inactiveMin = (Date.now() - lastActivityTime) / 60000;
17866     if (inactiveMin >= sessionTimeoutMinutes) {
17867       clearInterval(sessionCheckIntervalId);
17868       sessionCheckIntervalId = null;
17869       if (refreshDataIntervalId) clearInterval(refreshDataIntervalId);
17870       refreshDataIntervalId = null;
17871       events.forEach((ev) => document.removeEventListener(ev, updateSessionActivity));
17872       client.auth.signOut().then(() => {
17873         showLogin();
17874         showToast('Sesión cerrada por inactividad.', 'info');
17875       });
17876     }
17877   }, sessionCheckIntervalId);
17878 }

```

```

17873|     });
17874|   }
17875| }, 60000);
17876| }
17877|
17878| function finalizeSessionUiSetup() {
17879|   if (pandiSessionUiBootstrapped) return;
17880|   pandiSessionUiBootstrapped = true;
17881|   startSessionTimeoutCheck();
17882|   showAppContent();
17883|   // Recalcular CC y caja desde orden + transacciones (modelo autónomo); en segundo plano para no bloquear la UI.
17884|   sincronizarCcYCajaParaTodasLasOrdenesConInstrumentacion().catch(() => {});
17885|   const userEmailEl = document.getElementById('user-email');
17886|   if (userEmailEl) userEmailEl.textContent = currentUserEmail;
17887|
17888|   document.getElementById('btn-cerrar-sesion').addEventListener('click', () => {
17889|     if (sessionCheckIntervalId) clearInterval(sessionCheckIntervalId);
17890|     sessionCheckIntervalId = null;
17891|     if (refreshDataIntervalId) clearInterval(refreshDataIntervalId);
17892|     refreshDataIntervalId = null;
17893|     ['mousedown', 'mousemove', 'keydown', 'scroll', 'touchstart', 'click'].forEach((ev) => document.removeEventListener(ev, updateSessionActivity));
17894|     client.auth.signOut().then(() => showLogin());
17895|   });
17896|
17897|   const btnRefresh = document.getElementById('btn-refresh');
17898|   if (btnRefresh) btnRefresh.addEventListener('click', () => refreshPermisosYVista());
17899|
17900|   const sidebar = document.getElementById('sidebar');
17901|   const toggle = document.getElementById('sidebar-toggle');
17902|   if (localStorage.getItem(SIDEBAR_KEY) === '1') sidebar.classList.add('expanded');
17903|   function updateSidebarToggleLabel() {
17904|     if (!toggle) return;
17905|     const expanded = sidebar.classList.contains('expanded');
17906|     toggle.setAttribute('aria-label', expanded ? 'Contraer menú' : 'Expandir menú');
17907|     toggle.setAttribute('title', expanded ? 'Contraer menú' : 'Expandir menú');
17908|   }
17909|   updateSidebarToggleLabel();
17910|   pandiSyncSidebarBackdrop();
17911|   const sidebarBackdrop = document.getElementById('sidebar-backdrop');
17912|   if (sidebarBackdrop) {
17913|     sidebarBackdrop.addEventListener('click', () => {
17914|       if (!pandiIsMobileNavLayout() || !sidebar.classList.contains('expanded')) return;
17915|       sidebar.classList.remove('expanded');
17916|       localStorage.setItem(SIDEBAR_KEY, '0');
17917|       updateSidebarToggleLabel();
17918|       pandiSyncSidebarBackdrop();
17919|     });
17920|   }
17921|   if (toggle) {
17922|     toggle.addEventListener('click', () => {
17923|       sidebar.classList.toggle('expanded');
17924|       localStorage.setItem(SIDEBAR_KEY, sidebar.classList.contains('expanded') ? '1' : '0');
17925|       updateSidebarToggleLabel();
17926|       pandiSyncSidebarBackdrop();
17927|     });
17928|   }
17929|   let pandiResizeSidebarTid;
17930|   window.addEventListener('resize', () => {
17931|     clearTimeout(pandiResizeSidebarTid);
17932|     pandiResizeSidebarTid = setTimeout(() => pandiSyncSidebarBackdrop(), 150);
17933|   });
17934|   document.addEventListener('keydown', (e) => {
17935|     if (e.key !== 'Escape') return;
17936|     if (document.querySelectorAll('.modal-backdrop.activo').length > 0) return;
17937|     if (!pandiIsMobileNavLayout() || !sidebar.classList.contains('expanded')) return;
17938|     sidebar.classList.remove('expanded');
17939|     try {
17940|       localStorage.setItem(SIDEBAR_KEY, '0');
17941|     } catch (err) {
17942|       /* ignore */
17943|     }
17944|     updateSidebarToggleLabel();
17945|     pandiSyncSidebarBackdrop();
17946|   });
17947|
17948|   setupVistasMenu();
17949|   applyVistasMenuVisibility();
17950|   updateCcBotonesMovimientoManual();
17951|   setupPanelControl();
17952|   setupModalCliente();
17953|   setupModalIntermediario();
17954|   setupModalTipoOperacion();
17955|   setupConfigEmpresaMarcaControles();
17956|   setupModalReglasNegocio();
17957|   setupModalOrden();
17958|   setupModalOrdenOffline();
17959|   if (typeof window !== 'undefined') {
17960|     window.__closeModalOrden = function () { solicitarCierreModalOrden({ modo: 'listo', refrescarOrdenes: true }); };
17961|     window.__closeModalOrdenSolo = function () { solicitarCierreModalOrden({ modo: 'salir', refrescarOrdenes: true }); };
17962|   }
17963|   setupModalChatOrden();
17964|   setupModalTransacciones();
17965|   setupModalTransaccion();
17966|   setupModalMovimientoCaja();
17967|   setupModalTipoMovimientoCaja();
17968|   setupCajasToggle();
17969|   setupCajasMovFiltrosYTabs();

```

```

17970| setupCuentaCorriente();
17971| setupModalMovimientoCc();
17972| setupModalCcMovimientoManual();
17973| setupModalCcManualEditar();
17974| setupDelegacionAccionesCcManual();
17975| setupModalesDraggable();
17976| setupHelpPopovers();
17977| const [defaultVistaId, defaultTitle] = getFirstAllowedView();
17978| showView(defaultVistaId, defaultTitle);
17979| pandiUpdateOfflineToolBarButtons();
17980| if (refreshDataIntervalId) clearInterval(refreshDataIntervalId);
17981| refreshDataIntervalId = setInterval(refreshCurrentViewData, REFRESH_DATA_INTERVAL_MS);
17982| setTimeout(() => pandiRefreshOfflineCatalogosCache(), 400);
17983| }
17984|
17985| function onSessionReady(session) {
17986|   currentUserEmail = session.user.email || '';
17987|   currentUserId = session.user.id;
17988|   lastActivityTime = Date.now();
17989|   const loginErr = document.getElementById('login-error');
17990|   ensureProfile(session)
17991|     .then(() =>
17992|       Promise.all([client.rpc('get_my_permissions'), fetchAppEmpresaIntoState()])
17993|     )
17994|     .then(([res, _empRes]) => {
17995|       if (res.error) {
17996|         if (loginErr) loginErr.textContent = res.error.message || 'Error al cargar permisos.';
17997|         return Promise.reject(res.error);
17998|       }
17999|       userPermissions = res.data || [];
18000|       pandiCachePermissionsLocal(userPermissions);
18001|       aplicarMarcaEnTodaLaUI();
18002|       return Promise.all([
18003|         client.from('app_config').select('value').eq('key', 'session_timeout_minutes').maybeSingle(),
18004|         client.from('app_config').select('value').eq('key', 'usd_usd_comision_fija_config').maybeSingle(),
18005|       ]);
18006|     })
18007|     .then(([configRes, cfgFijaRes]) => {
18008|       if (configRes && !configRes.error && configRes.data && configRes.data.value) {
18009|         const n = parseInt(configRes.data.value, 10);
18010|         if (n > 0 && n <= 1440) sessionTimeoutMinutes = n;
18011|       }
18012|       if (cfgFijaRes && !cfgFijaRes.error && cfgFijaRes.data && cfgFijaRes.data.value) {
18013|         usdUsdComisionFijaConfig = parseUsdUsdComisionFijaConfigJson(cfgFijaRes.data.value);
18014|       }
18015|       aplicarValoresRadiosComisionFijaUsdWizard();
18016|       finalizeSessionUiSetup();
18017|     })
18018|     .catch((err) => {
18019|       if (session && session.user && pandiEsFalloConectividadBootstrap(err)) {
18020|         userPermissions = pandiLoadCachedPermissionsLocal();
18021|         aplicarMarcaEnTodaLaUI();
18022|         finalizeSessionUiSetup();
18023|         pandiApplyOfflineReducedModeUi();
18024|         return;
18025|       }
18026|       const msg =
18027|         err && typeof err.message === 'string'
18028|           ? err.message
18029|           : err && err.error && typeof err.error.message === 'string'
18030|             ? err.error.message
18031|             : 'Error al iniciar sesión.';
18032|       if (loginErr) loginErr.textContent = msg;
18033|       showLoginScreenDom();
18034|     });
18035| }
18036|
18037| /** Ayudas (help): al hacer clic en .help-icon-btn se abre un modal con el contenido del .help-popover asociado. */
18038| function setupHelpPopovers() {
18039|   const backdrop = document.getElementById('modal-help-backdrop');
18040|   const btnCerrar = document.getElementById('modal-help-cerrar');
18041|   const tituloEl = document.getElementById('modal-help-titulo');
18042|   const contenidoEl = document.getElementById('modal-help-contenido');
18043|   if (!backdrop || !btnCerrar || !contenidoEl || !tituloEl) return;
18044|
18045|   function closeHelpModal() {
18046|     backdrop.classList.remove('activo');
18047|     backdrop.setAttribute('aria-hidden', 'true');
18048|   }
18049|
18050|   function openHelpModal(html, title) {
18051|     tituloEl.textContent = title || 'Ayuda';
18052|     contenidoEl.innerHTML = html || '';
18053|     backdrop.classList.add('activo');
18054|     backdrop.setAttribute('aria-hidden', 'false');
18055|   }
18056|
18057|   btnCerrar.addEventListener('click', closeHelpModal);
18058|   setupBackdropCloseOnlyOnRealClick(backdrop, closeHelpModal);
18059|
18060|   document.addEventListener('click', (e) => {
18061|     const btn = e.target.closest('.help-icon-btn');
18062|     if (!btn) return;
18063|     e.preventDefault();
18064|     const wrap = btn.closest('.help-inline');
18065|     const popover = wrap?.querySelector('.help-popover');
18066|     const html = popover ? popover.innerHTML : '';

```

```
18067 |     openHelpModal(html, 'Ayuda');
18068 |   });
18069 |
18070 |   document.addEventListener('keydown', (e) => {
18071 |     if (e.key !== 'Escape') return;
18072 |     if (!backdrop.classList.contains('activo')) return;
18073 |     closeHelpModal();
18074 |   });
18075 | }
18076 |
18077 | setupSupabaseConnectivityMonitoring();
18078 |
18079 | // Inicio: getSession puede resolver *después* de un login rápido (autocompletar + Enter); sin guarda, el callback tardío llamaba showLogin() otra vez y duplicaba listeners.
18080 | client.auth.getSession().then(({ data: { session } }) => {
18081 |   if (authBootstrapFromGetSessionDone) return;
18082 |   authBootstrapFromGetSessionDone = true;
18083 |   if (!session) {
18084 |     showLogin();
18085 |     setupLoginAndRegister();
18086 |     return;
18087 |   }
18088 |   onSessionReady(session);
18089 | }).catch((e) => {
18090 |   console.warn('[Pandi] getSession:', e && e.message ? e.message : e);
18091 |   if (authBootstrapFromGetSessionDone) return;
18092 |   authBootstrapFromGetSessionDone = true;
18093 |   pandiSupabaseConnectivityIssue = 'unreachable';
18094 |   if (pandiUnreachableSinceMs == null) pandiUnreachableSinceMs = Date.now();
18095 |   updateSupabaseConnectivityBanner();
18096 |   showLogin();
18097 |   setupLoginAndRegister();
18098 | });
18099 |
18100 | client.auth.onAuthStateChange((event, session) => {
18101 |   if (event === 'SIGNED_OUT' && !session) showLogin();
18102 | });
```